The Parliament of Machines

The Desk Volume

An Audio Course in Distributed Systems — formal statements, proofs, protocols, traces, and problems

Narrated by Jeeves · Written for and with P. Milovanov, Esq.

Edition v1.6.5

How to Use This Volume

Each chapter of this volume, save the last, is the desk companion to one episode — and each is written to stand on its own. A chapter walks its episode's ground in the same order, one narrated pass — the same story, told for the page, with less chat and more proof — and the formal apparatus is set where the story reaches it: the model box before the first claim, the definitions and theorems at their story positions, the proofs immediately after their statements, the protocol boxes and trace exhibits at the dramas they record. The course's spoken names — the Post, the doppelgänger, the crowded club, the aunts — are standing vocabulary here, introduced beside their formal twins. The Whiteboard and the debt register close each narrated body, and then come the problems in three tiers. The closing chapter is apparatus — the registers map, the index of protocol boxes, and the pronunciation register — assembled once the season was complete. Tier I is calisthenics; Tier II is the real thing, including each chapter's *sabotage* problem, in which a plausibly broken protocol is supplied and your task is to exhibit the fatal execution; Tier III is starred and optional. Hints are segregated and spoiler-safe. Solutions are complete for Tier I, complete for the Tier II problems that finish arguments the chapters sketch, and withheld (hints only) for Tier III. Nothing here appears unmotivated by the audio, and nothing in the audio pretends to a proof that is not here.

Contents

Chapter 1

The Trouble with Other Computers

Companion to Episode One. The terrain, on paper: what distribution forfeits and why we distribute anyway; partial failure and the four silences; the Post's charter, formalized as fair-loss links with the layered construction upward; the models under which every claim of the season is made; the naive replicated counter and its three disasters, staged without a single machine failing and drawn here as trace exhibits; and the Two Generals theorem, written out as a clean induction. The proof technique it christens — the doppelgänger argument — will fell every major result to come.

1.1 Parts One and Two — The Scattering, and the End of Honest Dying

Why distribute at all? Three honest reasons and a candid fourth: *scale* (the workload genuinely exceeds any one machine); *geography* (light is slow, and a Toronto–Sydney round trip costs on the order of a sixth of a second before any machine does any work); *fault tolerance* (one machine is one fate — one power supply, one kernel, one fire); and *the organization already did* (the architecture ships the org chart, and the network does not care why it was invited in). What is given up is the paradise the rest of the field attempts to buy back at retail prices. A single machine enjoys three mercies so ordinary they are invisible: *shared memory* — one authoritative state, a fact of the matter about the value of x; *shared fate* — failure is total and honest, the whole stage aware of the death; and *one clock* — a single timeline on which every event has its place. The move from one machine to two forfeits all three at once, categorically: there is no such thing as slightly distributed.

The defining pathology follows from the second forfeit. In a fleet, a machine stops mid-operation — holding locks, halfway through an update — and *nobody is notified*; the survivors execute a plan that assumed a colleague who no longer exists. And the observer's predicament is worse than mere ignorance of the crash. Call it *the four silences*: a request has gone unanswered, and four worlds are consistent with the evidence — the server is dead; the server is slow (in the bath, to be embarrassed later); the request was lost; or the reply was lost *after the work was done*. All four present the identical face. The retry is the cure in the third world and a catastrophe in the fourth, and the sender cannot tell which world it inhabits (Problem 1.5 drills the enumeration and its consequences). A timeout, accordingly, is not a parameter but a *verdict* — a declaration of death without a body, sometimes wrong, and wrong expensively; the machinery for surviving the wrongly condemned (leases, fencing, epochs) arrives in Chapter 5.

Name the malice behind all this, for naming it is half the method. The fourth silence was no accident of luck; it is the move a *worst-case* world would choose, and the discipline of this field is to assume that world always obtains. So we personify it: everything harmful in this course — the letter that vanishes, the process frozen mid-stride, the colleague who lies — we lay at the door of a single presiding antagonist, the *adversary*:

Murphy's law furnished with a will and a timetable, who arranges the worst permitted event at the worst possible moment, who has read the protocol before choosing his move, and who is never so obliging as to fail in the manner one prepared for. Two disciplines follow. Correctness means correctness against his *every* move, not his likely one — a protocol that holds only when he is kind to us is not a protocol but a wish. And his single mercy, the more important clause of the treaty: the adversary is *leashed*, permitted only what the model allows, and the model is ours to write and his to obey — stating that leash exactly, before any claim, is the habit Part Four installs, for a theorem quoted without its model is a verdict delivered without a jurisdiction. He appears first and most often as the Post; as the scheduler of Chapter 3, who may freeze a healthy process for any finite age; and, at his most dangerous, as the Byzantine colleague of Chapter 5. One adversary, many coats.

Crashed and slow, then, are not hard to distinguish but *indistinguishable* — the first, unnamed appearance of the season's sharpest instrument, which receives its formal home at once.

Definition 1.1 (processes, messages, executions)**.** A *process* is a deterministic state machine with a local state, which takes steps: sending messages, delivering (receiving) messages, and local computation. Processes communicate only by messages over *links*. An *execution* is a sequence of steps of all processes together with the adversary's delivery decisions; a process's *view* in an execution is the sequence of its own inputs and delivered messages. Two executions are *indistinguishable to process p* if *p*'s view is identical in both; a deterministic *p* behaves identically in indistinguishable executions. This one-sentence observation is the doppelgänger argument, and it is the sharpest tool in the drawer.

1.2 Part Three — The Post

The adversary's first and favourite disguise is the network, engaged for the season under the name of *the Post* — not a wire but a postal service, a layered bureaucracy of queues and couriers subject to weather nobody can see. Its letters are messages *m*; its charter is the informal reading of the fair-loss link (Definition 1.4 below). Five clauses. The Post may *delay* any letter arbitrarily (clause one); may *lose* any letter silently, with no return-to-sender (clause two); may *duplicate* any letter (clause three); may *reorder* letters (clause four); and — the single mercy, whose revocation is Chapter 5's business — does not *forge* (clause five, formally the no-creation property FL3/PL3): every letter that arrives was genuinely written by the party whose name is on it. Clause three is redundant in a way that should worry the designer: a retrying sender atop a merely lossy network manufactures duplication and reordering endogenously, so the charter describes the system you build, not only the wire you bought (Problem 1.6 makes this precise). Nor is TCP a counterexample: it is a treaty negotiated with the Post clause by clause — sequence numbers against reordering, retransmission against loss, deduplication against the retransmission's own duplicates — holding within one connection, while it lives; at every connection break the oldest question in the field returns unanswered: *did my last letter arrive?*

The case for the charter is documentary, not temperamental. The *fallacies of distributed computing* (Deutsch and colleagues, Sun Microsystems, mid-nineties; the last clause added later by Gosling) name the eight false beliefs held by essentially every newcomer to networked systems, every one of them false, every one of them expensive:

1. *The network is reliable* — the founding fallacy, of which this whole chapter is the refutation.
2. *Latency is zero* — a floor is imposed by the speed of light and is not subject to procurement; every synchronous call across a network is a bet about latency, placed whether or not you noticed placing it.

3. *Bandwidth is infinite* — it is finite, shared, and most finite precisely when everyone wants it: during the incident.
4. *The network is secure* — chiefly a matter for other courses, though Chapter 5 lives on its border.
5. *Topology does not change* — machines are replaced, addresses reassigned, routes reconverge: routinely, automatically, and with particular enthusiasm in the middle of your request.
6. *There is one administrator* — there are dozens, in different departments and different companies, not attending the same change reviews; somewhere between you and your user, a device is being upgraded by someone who has never heard of you.
7. *Transport cost is zero* — serialization is compute, bytes are money, and the interface is a resource with a queue like any other.
8. *The network is homogeneous* — the fleet contains every vintage of hardware ever purchased and every version of every protocol ever deployed, simultaneously, forever.

Bailis and Kingsbury's "The Network is Reliable" (2014) supplies the evidence behind fallacy the first: published, attributable partitions at every scale, for every reason — switch firmware with novel opinions, redundancy correlated in the wiring closet, maintenance that partitioned the systems it maintained, and sharks, actual sharks, developing a taste for undersea fibre. The finding that matters most: *partitions happen* — the network sometimes splits into provinces, each internally healthy, each unable to distinguish "the other province is dead" from "the road between us is out." The charter is not pessimism; it is actuarial science. Both texts are assigned in the reading.

1.3 Part Four — The Rules of Engagement

The single most important methodological habit of the course: in this field, *every claim is conditional on a model, and a claim quoted without its model is not a fact but a noise*. The theorems do not say "consensus is impossible"; they say *under such-and-such assumptions* it is impossible — change the assumptions and the theorem falls silent, and something else becomes provable instead. A model is a point on three axes — timing, process failure, links — and the three are fixed now, formally, for the season.

Definition 1.2 (timing models)**.** A *timing model* fixes what a protocol may assume about the passage of time — the relative speed of processes and the delay of messages. Three are standard, in decreasing order of generosity to the protocol:

- **Synchronous:** known finite bounds hold *always*: every correct process takes a step within a known time, and every sent message is delivered within a known delay. Equivalently, computation proceeds in numbered rounds (the round model of the model box below). Here a timeout is a verdict that never errs.
- **Asynchronous:** *no* bound on process speed or message delay is assumed; the adversary controls both, constrained only by the link's own liveness (fair loss delivers eventually, but "eventually" may be arbitrarily late). A slow process and a dead one are then indistinguishable (Def. 1.1) — the model in which the sharpest impossibilities bite.
- **Partially synchronous:** (Dwork, Lynch, and Stockmeyer, 1988) the bounds exist but are unknown, or hold only after some unknown *global stabilisation time* (GST): before GST the system behaves asynchronously, after it, synchronously. This is the honest model of a real network — usually timely, occasionally not — and the one in which consensus becomes purchasable (Chapters 4–5).

A result is only as strong as the timing model it is proved in: an impossibility in the synchronous model is the most damning, for it survives every relaxation; a possibility in the synchronous model is the most suspect, for it may not survive asynchrony. *Course default: asynchronous.*

Definition 1.3 (process-failure models)**.** A *failure model* fixes how a faulty process may deviate from its protocol. A process that never deviates in a given execution is *correct*; the rest are *faulty*. Four are standard, in increasing order of malice:

- **Crash-stop:** a process follows its protocol exactly until, at some moment, it halts permanently and takes no further step. Nothing false is ever emitted; the only sin is silence.
- **Crash-recovery:** a crashed process may restart, losing its volatile state but retaining whatever it committed to stable storage (disk) beforehand. Amnesia rather than death — the survivor returns with holes in its memory (Paxos's persistence clause, Chapter 4; Problem 1.8).
- **Omission:** a process otherwise follows its protocol but may drop messages it should have sent or received — a fault of its own ports rather than of the link, and the least-used of the four, blurring as it does into link loss.
- **Byzantine:** (Lamport, Shostak, and Pease, 1982) a faulty process may deviate *arbitrarily*: send anything, to anyone, at any time, including well-crafted lies and collusion. The catch-all for bugs, corruption, compromise, and malice — and the model whose fault bounds are dearest to meet (Chapter 5).

Each model subsumes the milder ones: a protocol tolerating Byzantine faults tolerates crashes, but not the reverse. *Course default: crash-stop.*

The third axis is the Post's own charter, formalized — and then *built upon*, because the word "constructs" is the load-bearing one: reliability is never discovered, only manufactured, and always out of unreliable parts.

Definition 1.4 (fair-loss links)**.** A *fair-loss link* from p to q offers primitives $\mathsf{send}(m)$ and $\mathsf{deliver}(m)$ with:

FL1 *(fair loss)* if p sends m to q infinitely often and q is correct, then q delivers m infinitely often;

FL2 *(finite duplication)* if p sends m to q finitely often, then q delivers m at most finitely often;

FL3 *(no creation)* if q delivers m with sender p, then m was previously sent to q by p.

Fair loss is the formal name for *negligent but not criminal*: any single letter may vanish, but persistent resending defeats the Post eventually, and the Post invents nothing.

Definition 1.5 (stubborn links)**.** A *stubborn link* offers the same primitives with: **SL1** *(stubborn delivery)* if a correct p sends m once to a correct q, then q delivers m infinitely often; **SL2** *(no creation)* as FL3.

Definition 1.6 (perfect links)**.** A *perfect (reliable) link* offers the same primitives with: **PL1** *(reliable delivery)* if a correct p sends m to a correct q, then q eventually delivers m; **PL2** *(no duplication)* no message is delivered more than once; **PL3** *(no creation)* as FL3.

The ladder is climbed — stubbornness purchased by unbounded retransmission, perfection by deduplication atop it — in the coda after this chapter's main event (Lemma 1.2 and Protocol Box 2), and the top rung is yours: Problem 1.7.

The chapter's models, fixed in the box below before any formal claim is made — a habit this volume enforces without exception.

Model of this chapter

For the impossibility (Theorem 1.1). Two processes A (Alexandra) and B (Basil). *Deliberately generous timing*: the synchronous round model — computation proceeds in numbered rounds; in each round every process sends messages as a deterministic function of its state, then receives whatever the adversary delivers from that round, then updates state. Processes do not fail. The adversary may suppress (lose) any subset of messages. A holds an input $v \in \{\mathsf{attack}, \mathsf{abstain}\}$. The point of the generosity: the theorem is about *loss alone*; if coordination fails here, it fails a fortiori in the asynchronous model, where the adversary also controls delay.

Course default hereafter (unless a chapter declares otherwise): asynchronous timing, crash-stop processes, fair-loss links (Def. 1.4).

1.4 Part Five — The Counter That Could Not Count

The season's protocols are performed by a standing repertory company: replicas *Bingo*, *Tuppy*, and *Gussie* — processes p_1, p_2, p_3 at the desk — with Barmy and Oofy on the bench for scenes requiring five, and *Bertie*, an external client c, forever mildly inconvenienced and presently in the market for an umbrella. The casting notes are permanent equipment — Gussie is slow, Tuppy literal, Bingo ordinary and usually in the chair — because consistency of character is consistency of memory: three numbered replicas are hard to hold across a five-message exchange, but Bingo and Tuppy hold themselves without effort; the whimsy is load-bearing. The cast's written counterpart is the trace diagram, whose convention is fixed now, for the entire season; every counterexample drama in this volume gets its picture in this notation.

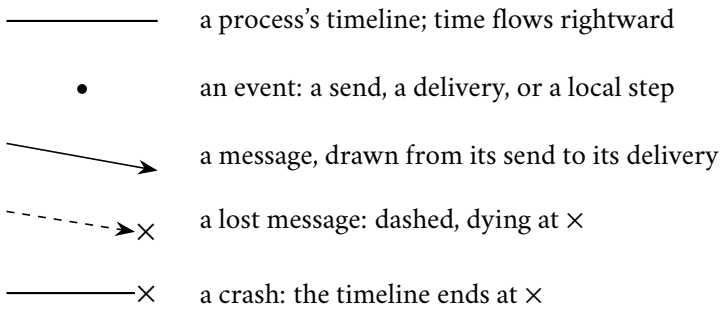


Exhibit 1.1 (conventions). A duplicated delivery is drawn as two message arrows leaving a single send event.

The scene: the three gentlemen operate the inventory of a small but ambitious online purveyor of umbrellas, sharing the simplest state imaginable — a counter of orders placed today, one copy each, because Part One decided the shop must survive the loss of a machine. The protocol, designed by someone who has not yet taken this course:

Protocol — Box 1. The naive replicated counter (the inaugural bug)

At each replica p_i, for a counter replicated on replicas $p_1, \ldots, p_n$:

1: **state:** $c \leftarrow 0$

2: **upon** client request $\langle \mathsf{INC} \rangle$:

```
3:      c ← c + 1
4:      for each replica q ≠ p_i: send(⟨INC⟩, q)

5:  upon deliver(⟨INC⟩):
6:      c ← c + 1
```

No identifiers, no acknowledgments, no retransmission, no ordering. Every one of those absences is a separate disaster: Exhibits 1.2 and 1.3 stage two of them, Problem 1.4 drills a third, and the third performance (the racing delivery notes) showed that for non-commuting state the protocol shape itself is unsalvageable without an order — the subject of Chapters 2 and 4.

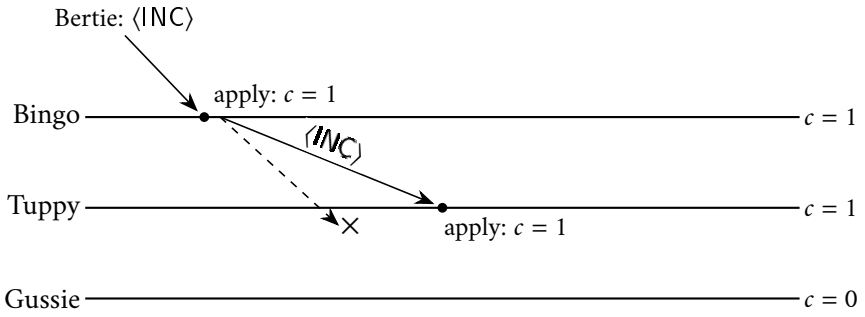


Exhibit 1.2 (the lost letter; performance the first). Bertie's order lands at Bingo; the propagation to Tuppy arrives; the propagation to Gussie dies in the Post. Divergence, and *silent* divergence: every machine is locally content, and the ensemble is wrong.

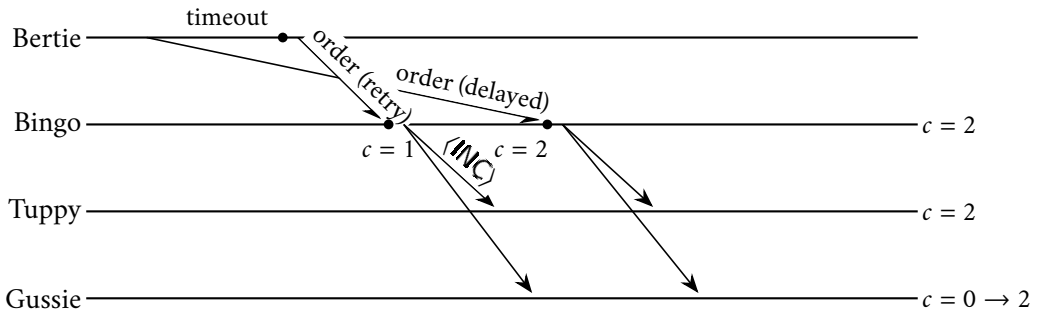


Exhibit 1.3 (the double charge; performance the second). The original order was not lost but *delayed*; the prudent retry and the ambling original both arrive, and the system converges — unanimously — on a world in which Bertie bought two umbrellas. Nobody misbehaved. Deduplication by identifier is the cure, and Problem 1.8 is the cautionary tale about administering it frugally.

Performance the third needed no exhibit, only a change of state: Bertie's two delivery notes — "leave with the porter," then, on reflection, "ring the top bell" — reached Gussie in the other order, clause four exercised, and his copy settled on the porter while his colleagues' settled on the bell. Every letter delivered, exactly once; the copies still disagree, because for *register* state, order is the whole game — and, as Chapter 2 will make precise, the question of who wrote last does not currently even mean anything. The counter, by contrast, forgave the same shuffling entirely: addition commutes, and an add-one and a subtract-one

crossing in the Post land on the same total either way. File the observation in a drawer marked *commutativity forgives disorder* — formally, the order-insensitivity of commuting operations; it returns in Chapters 7 and 12 wearing formal dress.

The audit of the whole scene is the point: divergence by loss, double charge by duplication-via-retry, divergence by reordering — and *not one machine failed*. Everything staged was weather, the Post's ordinary charter exercised on a protocol too innocent to have read it. And from the wreckage, the deepest question: Bingo says one, Gussie says nought — which of them is *right*? Right according to what? There is no master copy; the copies are all there is. If an authoritative answer is wanted, an authority must be appointed — in a way that survives the appointee's death, the Post's interference with the election, and the awkward possibility of two gentlemen each believing himself appointed. That problem is *consensus*, Chapters 3–5's story: first the proof that in the default model it cannot be done at all, then the lawyerly, partially-synchronous ways it is done every second of every day.

1.5 Part Six — Two Generals on Two Hills

The season's cornerstone (Akkoyunlu–Ekanadham–Huber 1975; christened and connected to transaction commit by Gray 1978). Two divisions on two hills; the enemy in the valley between, too strong for either alone, too weak for both together; General Alexandra holds the plan — *attack at dawn* — and the only instrument of coordination is a messenger who may be captured, silently, on any crossing. The intuition first: each acknowledgment does not discharge the doubt but *relocates* it — the uncertainty is a hot coal passed across the valley, and whoever sent the most recent rider is holding it when dawn comes. Intuition is where most tellings stop; this course does not. The problem, formally:

Definition 1.7 (the Coordinated Attack problem)**.** Two processes; A holds an input $v \in \{\mathsf{attack}, \mathsf{abstain}\}$; each process must output exactly one decision in $\{\mathsf{attack}, \mathsf{abstain}\}$. A protocol *solves Coordinated Attack* if in *every* execution permitted by the model:

- *(Termination)* both processes decide;
- *(Agreement)* both processes decide the same value;
- *(Validity)* if $v = \mathsf{abstain}$, both decide $\mathsf{abstain}$; and if $v = \mathsf{attack}$ and no message is lost, both decide attack.

Validity is the usefulness clause: without it, the protocol "always abstain" would satisfy the rest by universal cowardice.

Theorem 1.1 (Two Generals; coordinated attack is unachievable)**.** *In the model of this chapter's model box — synchronous rounds, reliable processes, links that may lose any subset of messages — no deterministic protocol solves Coordinated Attack.*

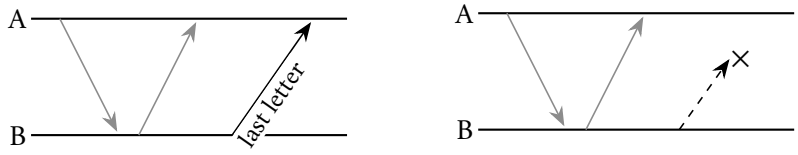
Proof. Suppose, for contradiction, that a deterministic protocol P solves Coordinated Attack in the stated model. Since processes are deterministic, an execution of P is completely determined by the input v and by the adversary's choice of which messages to suppress. Let α be the execution with $v = \mathsf{attack}$ in which the adversary loses nothing. By Termination and Validity, in α both processes decide attack, and they do so after finitely many rounds; let $m_1, m_2, \ldots, m_k$ be the messages delivered in α, listed in order of the round of their delivery, ties within a round broken by any fixed rule, so that m_k is a latest delivery.

For $0 \le i \le k$, let α_i denote the execution with $v = \mathsf{attack}$ in which the adversary delivers exactly $m_1, \ldots, m_{k-i}$ and suppresses everything else — including any message a process might newly send in consequence of the suppression. Thus $\alpha_0 = \alpha$. *Claim: in every α_i, both processes decide attack.* Induction on i; the

base case is α. For the step, let $m := m_{k-i}$ be the last message delivered in α_i, sent by s to r in round t; α_{i+1} additionally suppresses m. Compare the two executions from s's chair. In the round model, what s sends in each round is a function of s's state at that round's start, which is a function of the deliveries s has received in earlier rounds. Deliveries *to* s are identical in α_i and α_{i+1}: in rounds up to t they are the same messages among $m_1, \ldots, m_{k-i-1}$, and after round t neither execution delivers anything at all. So s's view is identical in both executions — the two are doppelgängers to s — and deterministic s therefore decides in α_{i+1} exactly what it decided in α_i: by the induction hypothesis, attack. And since P is assumed correct *in every execution*, Agreement holds in α_{i+1} in particular, so r decides attack as well. The claim follows, and taking $i = k$: in α_k, the execution with input attack in which *no message whatever is delivered*, both processes decide attack.

Now let β be the execution with $v =$ abstain in which the adversary likewise suppresses every message. Process B holds no input, and in both α_k and β it delivers nothing; its view is identical in the two — one final doppelgänger — so B decides in β what it decides in α_k, namely attack. But Validity requires that in β both processes decide abstain. This is the contradiction, and no such protocol P exists. □

Remark 1.8 (the same knife, two grips)*.* The induction has a companion telling, as a minimal counterexample — the *thriftiest counsel of war*: take a correct protocol of minimal best-case message count, examine the last letter of its best-case execution, and observe it is either decoration (strike it — contradicting thrift) or pivotal (capture it — the sender, unable to distinguish the worlds, attacks alone). That telling and the induction above are the same argument holding the knife by different ends: each peeling step of the induction is precisely the "decoration or pivotal" dilemma, resolved. Exhibit 1.4 draws the two worlds of a single peeling step. Note also the bookkeeping of the proof: Termination bought finiteness, Agreement propagated the decision along the chain, and Validity anchored both ends. Each hypothesis is spent exactly where it is needed — a habit of audit worth acquiring, since it tells you which weakening of the problem might escape the theorem, and Problems 1.9 and 1.12 explore exactly those escapes.



World I: delivered **World II**: captured

Exhibit 1.4 (the doppelgänger pair). One peeling step of Theorem 1.1: the grey arrows are the protocol's earlier traffic, identical in both worlds; the sender of the last letter, B, has an identical timeline in World I and World II and therefore behaves identically in both. If A's decision differs with the letter's fate, one general attacks alone; if it does not, the letter was decoration. Either way, the protocol dies.

The proof did not analyze strategies, and could not have — there are infinitely many. It did one thing, twice: it found two executions that some participant cannot tell apart, in which correctness demands that participant behave differently. That move is the master key of the field, and the course honours it with a name: *the doppelgänger argument* (Def. 1.1). It was met once already, unnamed, in the crashed-versus-slow diagnosis; it returns for FLP and CAP (Chapter 3), the Byzantine bound (Chapter 5), and the blocking of two-phase commit (Chapter 8).

Remark 1.9 (what is and is not forbidden)*.* The theorem forbids *certainty*, not usefulness. It does not forbid protocols that agree with overwhelming probability (Problem 1.9); it does not apply in gentler models, e.g. with a bound on total losses (Problem 1.12); and it says nothing against protocols that replace "both act at the deadline" with "both eventually act" — a change of specification, not a loophole. What it forbids exactly: guaranteed common action, over unreliable delivery, by a deadline. Two-phase commit will inherit this in Chapter 8, paying in the currency of blocking.

Remark 1.10 (common knowledge). The deep reading of Theorem 1.1 is epistemic: attacking safely requires not just that both generals know the plan, but that each knows the other knows, and so on without end — *common knowledge* in the sense of Halpern and Moses (1990), who proved that no finite exchange over unreliable channels can create it. Problem 1.11 walks the ambitious reader through the connection; the full treatment is beyond this chapter, and I say so plainly rather than gesture otherwise.

1.6 Coda — The Ladder of Links, Built

The theorem forbids guaranteed *coordination*; it says nothing against guaranteed *delivery*, and the contrast deserves to be felt before the season's thesis is stated: under the very charter that kills the generals' certainty, reliable point-to-point delivery is manufactured routinely, out of unreliable parts. The first rung is brute persistence as an algorithm.

Lemma 1.2 (stubbornness is purchasable)**.** *Stubborn links (Def. 1.5) can be implemented over fair-loss links (Def. 1.4) by unbounded retransmission (Protocol Box 2).*

Proof. Consider Protocol Box 2: on sb-send(m, q) the sender adds (m, q) to a set *sent* and thereafter reissues fl-send(m, q) for every element of *sent* forever, at any positive frequency. **SL1**: if the sender is correct, (m, q) remains in *sent* forever, so m is fl-sent to q infinitely often; if q is correct, FL1 yields that q fl-delivers m infinitely often, and each fl-delivery triggers an sb-delivery. **SL2**: an sb-delivery occurs only on an fl-delivery, which by FL3 was preceded by an fl-send of m by its named sender, which occurs only for messages actually sb-sent. □

Protocol — Box 2. Stubborn point-to-point link, over fair-loss

```
1: state: sent ← ∅

2: upon sb-send(m, q):
3:     sent ← sent ∪ {(m, q)};   fl-send(m, q)

4: every Δ time units:
5:     for each (m, q) ∈ sent: fl-send(m, q)

6: upon fl-deliver(m, p):
7:     sb-deliver(m, p)
```

Correct by Lemma 1.2; checked against the treatment in Cachin–Guerraoui–Rodrigues. Brute persistence as an algorithm. The *sent* set never shrinks and the repetition never stops; both costs are the price of the guarantee, and both are what Problem 1.7 teaches you to negotiate down.

The step from stubborn to *perfect* — suppressing the infinite repetition without losing the delivery — is deliberately left to you: it is Problem 1.7, it completes the layered construction, and its full solution is given.

1.7 Part Seven — Impossible in Principle, Routine in Practice

The season's axioms, binding hereafter. Fact the first: machines fail — independently, partially, and without notifying anyone. Fact the second: messages are delayed, lost, duplicated, and reordered — and the sender cannot tell which. Fact the third: there is no shared clock — and, as Chapter 2 makes precise, no shared

now for a clock to report. Every impossibility in the volume is a consequence of the three; every system sir operates is a negotiated settlement with them.

And the thesis those axioms serve: across the season the theory will prove things impossible, and the industry will do those very things before lunch, daily — the first instance being coordinated action over unreliable messages, impossible by Theorem 1.1 and performed by every online purchase. The reconciliation is always found in one of three places. *The model differs*: the impossibility was proved for a harsher world than the system inhabits. *The guarantee differs*: the theorem forbids certainty; the system delivers overwhelming probability, and the marketing department rounded up. *The price was paid somewhere you were not looking*: availability during partitions, latency on every write, a lease, a human on call — the theorem named a cost, the system paid it, and the invoice was filed where visitors do not go. Model, guarantee, price: finding which is *the* senior-engineer skill, and the season sharpens it into a formal triage.

1.8 The Whiteboard

For the design review.

1. Fix the model before the architecture: say aloud what the network may do, which failures you insure against, what timing you rent.
2. The catechism, for every arrow in the diagram: what happens when this message is *lost*? *duplicated*? *delayed* — and arrives anyway, after you acted on its absence? "That cannot happen" is a fallacy with a diagram.
3. Silence is not evidence. A timeout is a purchased guess; ask what happens when the condemned walks back in.
4. Never say "reliable" without saying *against what*.

The register. Debts issued: (1) the Post will learn to forge — payable Chapter 5, with a famous theorem attached. (2) Crashed versus slow is never resolved, only managed — instalments in Chapters 3, 5, and 11. (3) There is no shared clock — payable immediately: Chapter 2.

1.9 Problems

Tier I — Calisthenics

Problem 1.1 (fallacy audit)**.** For each vignette, name every fallacy of distributed computing it embodies, and state the first incident you would expect. *(a)* A mobile note-taking application saves each edit locally and syncs it "instantly" to the server; concurrent edits are resolved by "last write wins, by timestamp." *(b)* Two services in one datacentre call each other with thirty-second timeouts and unlimited retries on timeout; requests carry no identifiers; the runbook states that network issues are transient and retry storms are impossible. *(c)* A two-datacentre deployment replicates over a "reliable dedicated fibre link"; a cron job promotes the secondary when three consecutive pings to the primary fail.

Problem 1.2 (model classification)**.** Classify each system by timing model (Def. 1.2), process-failure model (Def. 1.3), and link assumptions, as in the model box. *(a)* An avionics control bus: message delivery guaranteed within a known bound; components fail only by halting. *(b)* A planet-scale web service: virtual machines pause and migrate without notice; processes restart with their disks intact; traffic crosses the

public internet. *(c)* A single-rack cluster with an engineered lossless fabric whose interface cards nonetheless drop frames silently under overload. *(d)* A settlement network operated jointly by mutually distrustful banks over leased lines with occasional long outages.

Problem 1.3 (link-property check)**.** For each channel, state which of FL1–FL3 and PL1–PL3 hold, with a one-line justification or counterexample. *(a)* Delivers every message exactly once, but arbitrarily late. *(b)* Loses each transmission independently with probability one half; never duplicates; never corrupts. *(c)* Delivers the first transmission of each distinct message and silently discards all retransmissions of it.

Problem 1.4 (trace reading)**.** Replicas Bingo, Tuppy, Gussie run Protocol Box 1; all counters start at nought. Events, in real-time order: (1) a client increment lands at Bingo; (2) Bingo's propagation to Tuppy is delivered; (3) Bingo's propagation to Gussie is delivered *twice* (the Post duplicated it); (4) a client increment lands at Gussie; (5) Gussie's propagation to Bingo is delivered; (6) Gussie's propagation to Tuppy is lost. Give the final counter at each replica, draw the trace in the Exhibit 1.1 convention, and mark the earliest moment at which the ensemble is already doomed to disagree forever.

Problem 1.5 (the four silences)**.** A client's request to a server has produced no reply within the timeout. *(a)* Enumerate the four worlds consistent with this evidence. *(b)* For each world, state what later observation, if any, would reveal it after the fact. *(c)* Exhibit two of the worlds as doppelgängers: describe the client's view and argue no protocol lets the client distinguish them *at the moment the timeout fires*. *(d)* In which worlds is an immediate retry harmless, and in which is it a double execution? What single property of the *operation* makes the distinction irrelevant?

Problem 1.6 (the charter is self-inflicted)**.** Assume a Post that only *loses and delays* — it never duplicates and never reorders distinct sends. Senders, however, retransmit on timeout. Construct executions showing that the application layer above nonetheless observes *(a)* duplicate deliveries of one logical message and *(b)* deliveries out of the order in which the logical messages were first sent. Conclude, in one sentence, why clauses three and four of the charter must be assumed even on networks that formally lack them.

Tier II — The Real Thing

Problem 1.7 (perfect links, built by hand)**.** Construct perfect links (Def. 1.6) over fair-loss links (Def. 1.4). Give the protocol for sender and receiver, using sequence numbers, a delivered-set, and acknowledgments; then *(a)* prove PL1, PL2, PL3; *(b)* identify precisely what the acknowledgments buy — correctness, or economy — by describing the protocol's behaviour with them deleted; *(c)* identify every piece of state that grows without bound, and say what assumption would be needed to bound it. *(Full solution provided.)*

Problem 1.8 (sabotage: the frugal clerk)**.** A designer, having read Problem 1.7, adds deduplication to the replicated counter. Client operations carry per-client sequence numbers, and the client retransmits an operation until acknowledged. Each replica keeps: the counter c, written synchronously to disk on every apply; and a table $high[s]$ — the highest sequence number applied per client s — kept in memory, "because it is rebuilt by traffic anyway." On $\langle \mathsf{INC}, s, n \rangle$: if $n > high[s]$, apply $c \leftarrow c + 1$ to disk and set $high[s] \leftarrow n$; in all cases acknowledge. Replicas may crash and recover (crash-recovery model); recovery reloads c from disk and starts $high$ empty. *(a)* Exhibit an execution in which one logical increment is applied twice. *(b)* State the invariant the design intends between $high$ and the disk state, and where it breaks. *(c)* Give the minimal repair and argue its correctness, including why the two persistent writes must be atomic with respect to each other. *(Certified fatal per the standing rules of this course: the fatal execution is written out in the solutions.)*

Problem 1.9 (riders and risk)**.** Alexandra intends the attack and may send messengers, each captured independently with probability p; captures are silent. *(a)* Protocol: she dispatches n riders with the order and

attacks at dawn unconditionally; Basil attacks if and only if at least one rider arrives. Show the probability of a solitary attack is exactly p^n, and compute the least n for which it falls below one in a million when $p = \frac{1}{2}$. *(b)* Modify: Alexandra attacks only if she receives at least one acknowledging rider from Basil. Enumerate the disagreement modes now possible and show the anxiety has moved, not died. *(c)* Prove that *every* protocol in this setting either never attacks or has disagreement probability strictly greater than zero. *(Hint provided.)*

Problem 1.10 (idempotency and its garbage)**.** Client operations carry globally unique identifiers; each replica keeps the set D of identifiers already applied and ignores repeats. *(a)* Prove this gives the counter effectively-once application per identifier, under fair-loss links and arbitrary client retries, replicas not crashing. *(b)* D grows forever. Propose a garbage-collection rule, then show that for *any* rule that ever discards an identifier the Post might still redeliver, the double-apply returns — and conclude that bounded deduplication memory is possible only by purchasing a bound on message lifetime, i.e. by strengthening the model. *(This is the deduplication-window lemma in embryo; it returns, industrialized, in Chapter 10.) (Hint provided.)*

Tier III — For the Ambitious (starred)

Problem 1.11 (⋆ common knowledge)**.** Following Halpern and Moses: for a fact φ, write $E\varphi$ for "both generals know φ," and define common knowledge $C\varphi$ as the infinite conjunction $\varphi \wedge E\varphi \wedge EE\varphi \wedge \cdots$. *(a)* Argue that any protocol solving Coordinated Attack yields, at the moment of attack, C("the attack is on"). *(b)* Show each additional level of the conjunction requires at least one further successfully delivered message, and conclude no finite execution over lossy links attains $C\varphi$ for any φ not common knowledge at the outset. *(c)* Reconcile with practice: what weakening of C do real systems settle for? *(Hints only.)*

Problem 1.12 (⋆ the budgeted Post)**.** Same round model as Theorem 1.1, but the adversary's total budget is at most f suppressed messages across the whole execution, and f is known. *(a)* Give a protocol solving Coordinated Attack with f+1 messages, and prove it. *(b)* Prove no protocol using at most f messages in its worst case can succeed. *(c)* Now let the budget be f *per round*, rounds unbounded. Show the impossibility of Theorem 1.1 returns. Moral: it is not the quantity of loss that kills, but its inexhaustibility. *(Hints only.)*

1.10 Hints

Hint (for Problem 1.9)**.** (a) Disagreement is exactly "all n captured." (b) Trace, for each pattern of captures, who stands alone at dawn. (c) Every execution in the peeling chain of Theorem 1.1's proof, and the final β, occurs with probability at least $p^{(\text{messages sent})} > 0$ once you fix the protocol; a deterministic protocol correct with probability one would have to behave correctly in each of them, and the proof of Theorem 1.1 shows it cannot.

Hint (for Problem 1.10)**.** (a) Set membership is exactly PL2's argument. (b) Fix the rule; the adversary reads it. Find the identifier it discards earliest, and have the Post redeliver just after. The contrapositive is the lemma: no double-apply forever ⇒ some identifier is remembered as long as the Post can redeliver it ⇒ memory is bounded only if redelivery lifetime is.

Hint (for Problem 1.11)**.** (a) If at the moment of attack some level E^k failed, exhibit the doppelgänger in which a general attacks alone. (b) Induction on k, with Theorem 1.1's peeling as the engine: the last delivered message is the only possible carrier of the topmost E. (c) Consider "eventually both know," and timeouts as purchased belief.

Hint (for Problem 1.12)**.** (a) Pigeonhole: f+1 riders, at most f captures. (b) The adversary can afford to suppress everything; then replay the α_k-versus-β doppelgänger. (c) In the peeling argument, check the adversary's budget: each α_i asks it to suppress only messages it can afford round by round.

1.11 Solutions

Tier I in full (tersely); Tier II in full where the problem completes an argument begun above (1.7, 1.8); hints only for Tier III, by standing policy.

Solution (1.1)**.** *(a)* Fallacies 1 and 2 (reliable, zero-latency); the timestamp clause also leans on a shared clock that does not exist — Chapter 2's subject; first incident: silently lost edits under concurrent writes. *(b)* Fallacies 1 and 3; no identifiers means retries are duplicates (Exhibit 1.3); "storms impossible" ignores that timeouts correlate under load — the metastability of Chapter 11; first incident: a double-applied operation during a latency spike. *(c)* Fallacies 1 and 5, plus the timeout-as-verdict error: three lost pings do not distinguish a dead primary from a partitioned one; first incident: two primaries — the split brain of Chapter 5.

Solution (1.2)**.** *(a)* Synchronous; crash-stop; reliable bounded-delay links. *(b)* Asynchronous; crash-recovery (disks survive); fair-loss. *(c)* Effectively synchronous timing with *omission* link faults — "lossless" fabrics lose at the edges. *(d)* Partially synchronous; Byzantine (mutual distrust); fair-loss with long partitions.

Solution (1.3)**.** *(a)* PL1–PL3 all hold (perfection does not promise punctuality); hence FL1–FL3 hold as well. *(b)* FL1 holds (infinitely many independent trials with success probability one half deliver infinitely often, with probability one — note the "almost surely," a nicety the definitions absorb); FL2, FL3 hold; PL1 fails (any single send may be lost). *(c)* FL3, FL2 hold; *FL1 fails*: send m infinitely often and it is delivered exactly once. Moral: this channel is not a fair-loss link at all — it is a deduplicating layer masquerading as one, and layers must be named for what they guarantee, not where they sit.

Solution (1.4)**.** Bingo: 1 (own) + 1 (Gussie's) = 2. Tuppy: 1 (Bingo's) + 0 (Gussie's lost) = 1. Gussie: 2 (Bingo's, duplicated) + 1 (own) = 3. The ensemble is doomed no later than event (3): with the duplicate applied and no deduplication or retransmission anywhere in Protocol Box 1, no future execution of the protocol reconciles the copies.

Solution (1.5)**.** *(a)* Server dead; server slow; request lost; reply lost after execution. *(b)* Dead: an operator finds the body. Slow: the reply arrives late. Request lost: server-side audit shows no arrival. Reply lost: the side effect exists without a reply — discoverable only by inspecting effects. *(c)* Worlds two and four at timeout: the client's view is "sent, then silence" in both; any client behaviour is a function of that view, hence identical — doppelgängers by Definition 1.1. *(d)* Harmless in world three; double-executing in world four; *idempotence* of the operation dissolves the distinction, which is why Problem 1.10 and Chapter 10 make it a design axiom rather than a hope.

Solution (1.6)**.** *(a)* Send m; the Post delays it past the timeout; retransmit; both copies arrive: two deliveries of one logical message. *(b)* Send m_1; it is delayed. Timeout; while the retransmission of m_1 is pending, send m_2, which arrives promptly; then m_1 (either copy) arrives: delivery order m_2, m_1 inverts send order. Conclusion: any layer that retries above a merely lossy network *manufactures* duplication and reordering; the charter describes the system you build, not only the wire you bought.

Solution (1.7)**.** Layer it: stubbornness first, then deduplication, then economy.

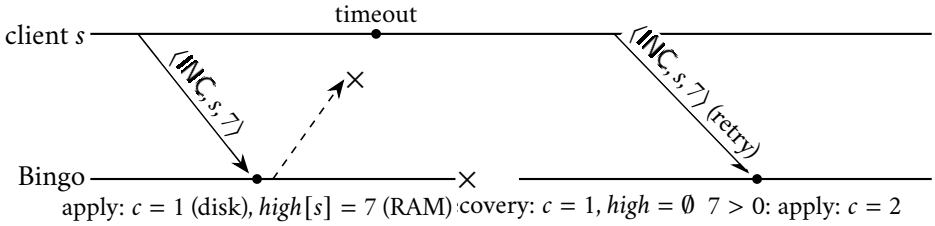
Protocol — Box 3. Perfect link over fair-loss (solution to Problem 1.7)

3: fl-send($\langle$ACK, $k\rangle$, p)

4: **if** $k \notin seen[p]$: $seen[p] \leftarrow seen[p] \cup \{k\}$; pl-deliver($m$, p)

(a) **PL1.** Suppose correct p pl-sends m to correct q, assigned index k. Either an $\langle$ACK, $k\rangle$ is eventually fl-delivered at p, which by FL3 was sent by q, which q does only upon fl-delivering $\langle$DATA, $m, k\rangle$ — so q delivered the data; on first such delivery, $k \notin seen$ and q pl-delivers m. Or no ack is ever fl-delivered: then (m, k) stays in *pend* forever, the data is fl-sent infinitely often, FL1 gives infinitely many fl-deliveries at q, and the first triggers the pl-delivery. **PL2.** q pl-delivers index k from p only when $k \notin seen[p]$, and inserts k in the same step; so at most once per index, and each pl-send uses a fresh index. (Two pl-sends of equal payload m are distinct logical messages with distinct indices — the abstraction deliberately distinguishes them.) **PL3.** A pl-delivery requires an fl-delivery of $\langle$DATA, $m, k\rangle$, which by FL3 was fl-sent by p, which the protocol does only for pl-sent messages. *(b)* Delete the acks: PL1–PL3 still hold by the second branch of the PL1 argument — the acknowledgments buy *economy only* (retransmission ceases; *pend* shrinks), never the guarantee. Reliability came from stubbornness plus deduplication. *(c)* $seen[p]$ grows forever (mitigable to a single counter if indices are delivered in order — but ordering is not among PL1–PL3; that is a stronger link, and Chapter 10's opening subject); *pend* is bounded only if acks eventually arrive, i.e. only between correct parties. Bounding *seen* outright is exactly Problem 1.10(b): a model purchase.

Solution (1.8)**.** *(a)* The fatal execution, in the convention of Exhibit 1.1:



The acknowledgment dies in the Post; Bingo crashes after the synchronous apply; recovery restores $c = 1$ but an empty table; the retry's sequence number seven exceeds the reborn $high[s] = 0$ and is applied again. One umbrella, two charges. *(b)* Intended invariant: $n \leq high[s]$ *if and only if operation* (s, n)*'s effect is in the durable state.* The crash breaks the only-if direction: the effect is durable, the record of it was not. The two facts lived at different durabilities, and the crash chose between them. *(c)* Repair: make the record as durable as the effect, *atomically* — one write (or one write-ahead log record) containing both the increment and (s, n); recovery rebuilds *high* from the log. Atomicity is not pedantry: persist the record before the effect and a crash between them loses the increment (the retry, now filtered, never reapplies it — a permanently undercounted umbrella); effect before record is part (a). The pair must survive or perish together — which is, in miniature, the transaction, and Chapter 8's opening bar.

Solution (1.9, 1.10, 1.11, 1.12)**.** By standing policy: hints only (the Hints section above). For 1.9(a), the arithmetic checkpoint: $n = 20$, since $(\frac{1}{2})^{20} = \frac{1}{1{,}048{,}576} < 10^{-6}$ and $(\frac{1}{2})^{19} > 10^{-6}$.

1.12 The Reading

The fallacies of distributed computing (Deutsch and colleagues, Sun Microsystems, mid-1990s). Five minutes; a list with scars. Read it as an audit against your last three incidents, and score honestly. Items one, five, and six are this chapter's material; item two was priced, with Toronto and Sydney, in Part One. It has not aged, because physics has not.

Bailis and Kingsbury, "The Network is Reliable," ACM Queue, 2014. The documentary record behind the Post's charter: a catalogue of published, attributable network partitions across cloud providers, enterprises, and measurement studies. Skim the incident inventory; savour the running observation that redundancy so often fails *correlatedly* — redundant on the diagram, coupled in the failure domain. Read it as an actuary reads flood data; it is why the model box is not pessimism.

Standing companion: Kleppmann, *Designing Data-Intensive Applications*, chapters 5–9. Assigned once, for the whole season, for parallel reading at leisure. Chapter 8 ("The Trouble with Distributed Systems") is this chapter's terrain in prose and will make an excellent second telling; chapters 5 and 9 will meet us again in Chapters 6 and 7 of this course.

Chapter 2

Time, Clocks, and the Ordering of Events

Companion to Episode Two. The season's third debt — there is no shared clock — paid in full, in the only honest currency: not a synchronization gadget but a replacement theory of order. The sins of physical clocks, priced from the operator's chair; the patent clerk's finding translated to machines — even perfect clocks would order only the causally meaningless; the two planks that survive distribution, and happens-before built upon them, with concurrency as its honest residue. Then the instruments: the Lamport clock, its clock condition proven whole; the vector clock, its two-way characterization proven entire; and the Chandy–Lamport photograph of a system that never holds still, correct by a shuffling argument that makes a present-that-never-was legitimate.

2.1 Part One — The Sins of Physical Clocks

In 1905, in Bern, a young examiner at the Swiss federal patent office — a technical expert, third class, whose applications for university posts had not prospered — published a paper on the electrodynamics of moving bodies, among whose consequences was one that unsettled physics and, in due course, everybody else: *simultaneity is not a fact about the world*. Whether two distant events happen "at the same moment" depends on the observer; two lightning strikes simultaneous for the passenger on the platform are successive for the passenger on the train, and neither is wrong. There is no master clock in the sky, no universal now; influence takes time to travel, and "at the same time, far apart" is a phrase from before we knew better. Seventy-three years later Lamport delivered the same news to computing, in "Time, Clocks, and the Ordering of Events in a Distributed System" (1978), since become one of the most cited papers the field possesses — and he has been candid about the lineage: the idea came directly from special relativity, with the message playing the role of the light signal, the fastest thing by which one place can influence another. Take that seriously and the question "which of these two events on two machines happened first" dissolves exactly as simultaneity dissolved in Bern. This chapter pays the third debt of Chapter 1's register — *there is no shared clock* — and pays it in full: not merely the observation that the machines' watches disagree, which is a complaint about engineering, but the finding that even with perfect watches, the question the watches were meant to answer does not, across machines, entirely exist.

The working engineer's instinct is that this is all a synchronization problem — tighten the discipline, buy better hardware, restore the timeline. The instinct is wrong twice over, and the first wrongness deserves an honest price before the second, deeper one is reached. Inside nearly every machine sits a sliver of quartz whose vibration is the machine's heartbeat, and it is not quite the frequency stamped on the tin: an ordinary crystal drifts by some tens of parts per million. Take twenty as a round figure: a day is 86,400 seconds, and twenty parts in a million of that is a little over a second and a half — call it a second and three quarters — *per day*. Left to itself, an ordinary server clock wanders by the better part of a minute a month. Nor is

the drift constant: it breathes with temperature, which is to say with load, which is to say the clocks run measurably differently during the incident than they did before it — a detail with a certain poetic cruelty.

So the clocks are disciplined. The instrument is the network time protocol — NTP — a hierarchy of authority descending from atomic clocks and satellite receivers down through strata of servers, and its central assumption is a fallacy from Chapter 1 wearing a lab coat: a client measures the round trip to a time server, assumes the outbound and return journeys took *equal* time — symmetric paths — and splits the difference to estimate its offset. On real networks, congested one way and quiet the other, routed differently in each direction, the assumption fails silently, and the failure becomes a bias no amount of polling repairs, because every poll makes the same assumption. Under good conditions NTP holds machines within a millisecond or a few of one another on a local network, and within some tens of milliseconds across the wide area; under bad ones — an overloaded server, a misconfigured upstream, an asymmetric route — it quietly holds them to nothing at all, while continuing to report success.

Then the discrete indignities. A daemon correcting a wrong clock chooses between *slewing* — running the clock slightly fast or slow until it converges, so that time never jumps — and *stepping* — yanking it to the correct value in one motion, so that time visibly lurches and occasionally, horribly, moves *backwards*; every piece of software that ever computed a duration by subtracting two wall-clock readings has an opinion about backwards time, and the opinion is usually an exception at three in the morning. Virtual machines add their own flourish: a paused guest wakes with its clock frozen in the past, then experiences the correction as a seizure. And there is the leap second, inflicted by international agreement: the Earth, being a large wet rock, does not keep atomic time, so the custodians of UTC periodically decree an extra second, and the clocks of the world are asked to experience a minute containing sixty-one seconds — a concept for which a remarkable amount of software was not prepared. On 30 June 2012 the scheduled leap second tripped a defect in the Linux kernel's timing machinery, and perfectly healthy machines across the internet sat down at midnight and spun; most memorably, as the trade press documented, an airline's check-in system went down with it, and queues formed at counters on the far side of the world because the planet's rotation had been adjusted by one second. The remedy that has since spread through the industry, published openly by Google, is the *leap smear*: rather than inflict the extra second at a stroke, dilute it homeopathically across the whole day, running every clock imperceptibly slow so that no clock is ever *suddenly* wrong — deliberately making every clock slightly and coordinatedly wrong so that none is surprised; civilized mendacity. The smear's fine print is this chapter's thesis in miniature: smearing is a local decision, so a fleet that smears and a fleet that does not — or two fleets smearing on different schedules — disagree with each other *by design* for the duration, and any cross-fleet timestamp comparison in that window inherits the disagreement. Among machines, even the corrections to time are relative.

One further sin you can prosecute in your own codebase this week, because it concerns not the clocks but which clock is consulted. Every machine offers two: the *time-of-day* clock — disciplined by NTP, naming the date, subject to slewing, stepping, smearing, and decree — and the *monotonic* clock, a humble stopwatch counting since boot, promising nothing about what o'clock it is and everything about never running backwards. The rule of hygiene falls straight out: durations, timeouts, and intervals are the monotonic clock's business; the time-of-day clock names moments for humans, and does nothing else. A duration computed by subtracting wall-clock readings is a subtraction that is occasionally negative, and "occasionally" chooses its moment — a respectable fraction of the industry's mystery timeouts trace to a duration computed across a clock step. Note the small, telling asymmetry of the bargain: the monotonic clock is trustworthy precisely because it has renounced the ambition of knowing what time it is. The season is full of that trade.

Collect the moral in two layers. The engineering layer: wall clocks across machines agree to within milliseconds on a good day and to within nothing in particular on a bad one, while a message between two

machines in the same building takes a fraction of a millisecond — the disagreement between the clocks is *larger than the events they would need to order*. Sorting cross-machine events by wall-clock timestamp is therefore not a slightly noisy measurement; at fine grain it is noise wearing the costume of measurement. Every engineer who has collated logs from several machines by timestamp has watched a reply apparently precede its request — an effect solemnly logged before its cause. The merged log was not revealing a miracle; it was committing perjury with digits. And the deeper layer, the chapter's real subject: suppose the engineering solved — every clock in the fleet agreed to the nanosecond. The timeline so obtained would *still* not answer the question actually cared about, which is never really "which happened first" but "*could this have influenced that*." Two events a nanosecond apart on opposite sides of a datacentre are ordered by the perfect clocks, and the order is causally meaningless: no message — no influence of any kind — could have crossed between them in that interval. The perfect timeline orders things whose order cannot matter, and it is silent about the only relation with consequences. Physics offered the patent clerk the same bargain, and he declined it. So shall we.

2.2 Part Two — Dismantling the Question

The question under dissection: event *a* happened on Bingo, event *b* on Gussie — which happened first? The question presumes a master timeline, a universal ledger in which every event in the fleet has its row, the rows have an order, and the order is a fact. Physics gave that picture up in 1905, and the reason translates into our vocabulary with almost embarrassing directness: in relativity the fastest carrier of influence is light, and events too far apart, too close in time, for light to travel from one to the other are causally sealed off from each other — and for exactly such pairs, observers in different states of motion genuinely disagree about which came first, with no fact of the matter to arbitrate. Order is real only where influence is possible; where influence is impossible, order is a bookkeeping convention.

Stage the disagreement with materials from the trade before translating the physics wholesale. Two services each keep a log. In Bingo's log, his cache refresh — at his own three minutes past eleven — precedes Gussie's write, which Bingo's monitoring scraped a moment later; in the collation run by the analytics pipeline, which trusts Gussie's timestamps, skewed those few milliseconds the other way, the write precedes the refresh. Two collations, two histories, both internally coherent, produced from the same events by nothing more sinister than a choice of whose clock to believe. Which is correct? If no message passed between the two events, the question has exactly the standing of the train passenger's quarrel with the platform: none. The logs are not lying, and they are not both right — the category of "the true order" simply does not extend to that pair of rows, and the sooner a postmortem admits this, the sooner it stops reconstructing fictions with a straight face.

Now translate. In a distributed system the fastest — indeed the only — carrier of influence between machines is the message; the wall clock carries none, merely decorating events with numbers whose cross-machine meaning Part One has priced. If no message, and no chain of messages, travels from the neighbourhood of event *a* to the neighbourhood of event *b*, then *a* cannot have influenced *b* — not "did not": *could not have* — and any answer to "which came first" is convention, not physics. The correspondence deserves to be held consciously rather than absorbed as mood, and Lamport drew it himself. The light signal becomes the message: the fastest carrier of influence — in our world, the only one. The light cone becomes the event's causal future: everything reachable from it by roads of letters. Spacelike separation becomes concurrency: merely elsewhere. The observer, with his private simultaneity, becomes a log collation, with its private ordering of other machines' events. And the one thing all observers agree upon in relativity — the order of events connected by a signal — becomes the one thing all collations agree upon here: the order of events connected by messages. The analogy is not decoration; it is load-bearing enough that a physicist and

a distributed-systems theorist can, with a small dictionary, read one another's impossibility results. Ours is merely the cheaper laboratory.

Two things survive, and they are the only bedrock there is. First: *within a single process, order is real* — Bingo is one machine executing one sequence of steps, and his diary is trustworthy about his own day. Second: *a message is received after it is sent* — whatever the Post does to a letter, it does not deliver into the past. That is the complete inventory; every other ordering claim across machines must be built from these two planks, or it is decoration. It seems a poverty. It is a foundation, and the rest of the chapter is the architecture — beginning with the substrate, restated with the season's formality, and the models under which every claim below is made.

Definition 2.1 (events and executions, recalled and sharpened)**.** Processes are as in Definition 1.1: deterministic machines whose steps are *events* of three kinds — local steps, sends, and receives. An *execution* α is a (finite or infinite) sequence of events such that (i) the subsequence at each process is consistent with that process's machine; (ii) every $\textsc{receive}(m)$ is preceded in α by its matching $\textsc{send}(m)$; and (iii) the channel discipline of the model box holds. Events of the reference trace (Exhibit 2.1) are named $b_1, b_2, \ldots$ per process in program order.

Model of this chapter

For orders and clocks (Defs. 2.2–2.8, Thms. 2.1–2.2). Asynchronous message passing among deterministic processes $p_1, \ldots, p_n$ (Def. 1.1); no process failures in this chapter; links promise only *no creation* — every delivered message was previously sent. Loss, duplication, and reordering are irrelevant to the definitions: happens-before is defined over the deliveries that occur. No wall clocks appear anywhere.

For the photograph (Def. 2.10, Thm. 2.4). A strictly better Post, purchased and declared: channels are *reliable* and *first-in-first-out* — every message sent on a channel is delivered, exactly once, in the order sent — and the postal map is strongly connected, so markers can reach everyone. Buildable over fair-loss links by Chapter 1's constructions (retransmit, deduplicate) plus per-channel sequence numbers; the FIFO clause is spent at exactly one step of Theorem 2.4 and nowhere else (Rem. 2.11).

Failures: deliberately absent. Detecting them is Chapter 3's business; photographing across them is Chapter 10's recovery business. In this chapter the company is immortal and merely unpunctual.

2.3 Part Three — Before, After, and Elsewhere

Now the order the two planks support. Lamport called it *happens-before* — read "a happens before b," written $a \to b$ — and it earns its three clauses stated precisely, because it is the load-bearing definition of the season and possibly of the field.

Definition 2.2 (happens-before; Lamport 1978)**.** The relation $\to$ on the events of an execution is the smallest relation such that

HB1 if a and b occur at the same process and a earlier, then $a \to b$;

HB2 if $a = \textsc{send}(m)$ and $b = \textsc{receive}(m)$, then $a \to b$;

HB3 if $a \to b$ and $b \to c$, then $a \to c$.

Equivalently: $a \to b$ iff there is a *causal path* $a = e_0, e_1, \ldots, e_k = b$ with $k \geq 1$, each consecutive pair related by HB1 or HB2. The relation is a strict partial order: every HB1/HB2 step moves strictly later in the execution sequence, so no cycles arise.

Nothing is before anything else unless these rules force it. Concrete, with the company: Bingo performs a local step — his morning entry — then writes to Tuppy; Tuppy receives it, makes an entry of his own, and writes onward to Gussie; Gussie receives that. Bingo's entry precedes his send by HB1; the send precedes Tuppy's receipt by HB2; receipt before entry before send, HB1 again; HB2 into Gussie's receipt; and HB3 chains the whole procession — Bingo's morning entry happens before Gussie's receipt, through two letters and three desks. Influence had a road, and the relation records the roads. Meanwhile Gussie, before his letter arrived, had paused to make himself a cup of tea — and the tea is the other half of the vocabulary: no clause relates it to Bingo's doings in either direction. The desk's drill-ground for all of this, fixed now:

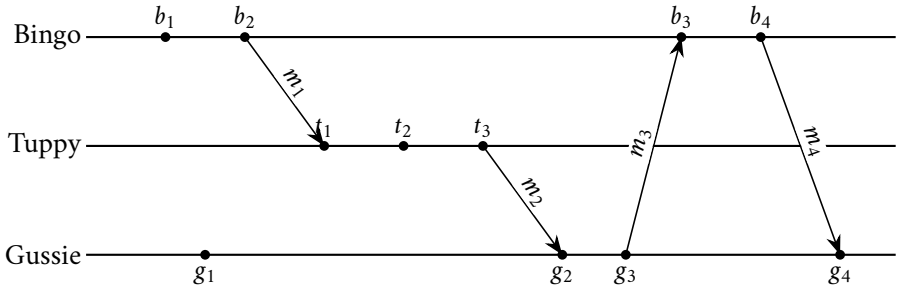


Exhibit 2.1 (the reference trace). Three processes, eleven events, four messages: $m_1 : b_2 \to t_1$; $m_2 : t_3 \to g_2$; $m_3 : g_3 \to b_3$; $m_4 : b_4 \to g_4$. Events b_1, g_1, t_2 are local steps (g_1 is, canonically, the tea). All of this chapter's calisthenics compute on this trace.

The season's phrase for the tea's standing, to be used exactly and often: *concurrent* — not before, not after, merely elsewhere.

Definition 2.3 (concurrency)**.** Distinct events a, b are *concurrent*, written $a \parallel b$, iff neither $a \to b$ nor $b \to a$. The relation is symmetric and *not* transitive: on Exhibit 2.1, $b_2 \parallel g_1$ and $g_1 \parallel t_2$, yet $b_2 \to t_2$.

The same trace, seen truly. Exhibit 2.1 draws the events along a horizontal axis that reads, irresistibly and wrongly, as time — the very timeline Part One abolished. Strip that axis and redraw the trace as what happens-before actually is, a directed acyclic graph, and the partial order shows its true shape.

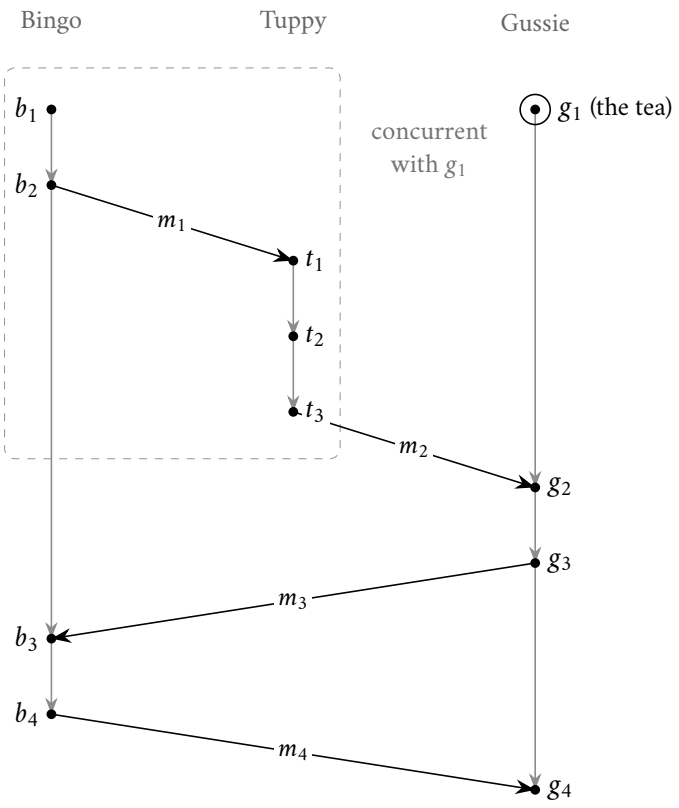


Exhibit 2.2 (the reference trace as a causal graph). The eleven events of Exhibit 2.1, redrawn as the directed acyclic graph of happens-before. The horizontal axis is gone: columns mark only *which desk*, and height is *causal depth*, so a downward road is the sole meaning of "before." Two edge-classes, and only two — the transitive reduction — corresponding to the two planks of Part Two: grey edges are program order (HB1, same desk); black arrows are messages (HB2, a letter), $m_1, \ldots, m_4$ as before. Then $a \to b$ iff a directed road runs from a down to b, and $a \parallel b$ iff none runs either way. The tea g_1 is the picture's lesson: no road reaches it and it reaches nothing until g_2, so it is concurrent with the whole upstream block $\{b_1, b_2, t_1, t_2, t_3\}$ (dashed) — and because that block holds both b_2 and t_2, which are themselves ordered ($b_2 \to t_1 \to t_2$), one sees at a glance why "$\parallel$" does not chain: $b_2 \parallel g_1$ and $g_1 \parallel t_2$, yet $b_2 \to t_2$. Not a timeline but a lattice of roads.

Attend to what concurrency means and does not mean, for the word is a standing trap. It does not mean "at the same time" — "the same time" has been abolished; that is the whole point. It does not mean the events were close together, or far apart, or anything about wall clocks at all. It means precisely that no road of influence connects the two events, in either direction: each is outside the other's reach. And note the epistemic flavour, which pays dividends all season: happens-before is *potential* causality, generously drawn — if $a \to b$, influence was possible, whether or not anything of consequence travelled the road; the relation can overstate influence, never understate it. Its negation, however, is ironclad: if two events are concurrent, no protocol, no cleverness, no forensic reconstruction can make one the cause of the other. "Concurrent" is not a confession of ignorance about the true order; it is the report that *there is no true order to be ignorant of*.

Drill it once on the trace, because fluency here is fluency everywhere later. The send b_2 and the tea g_1: concurrent, as established. Bingo's entry b_1 and Tuppy's entry t_2: ordered — the road runs through m_1. Tuppy's send t_3 and the tea g_1: no shared process, no connecting letter in either direction — concurrent, despite the fact that one of them would, on any wall clock consulted, have "happened first." The genealogical

image serves well and is kept: ordered events are ancestor and descendant; concurrent events are cousins — contemporaries of a sort, sharing ancestors perhaps, but neither descended from the other. "Ancestor or cousin" is, formally, comparable or incomparable under $\rightarrow$; the season will ask the question a hundred times before it ends, and the answer will move money. Problem 2.1 is the drill.

One more observation before the instruments. Happens-before is a *partial* order: some pairs are ordered, concurrent pairs simply are not, and the shape of the whole is not a timeline but a lattice of roads. Engineers meet this partiality and itch to totalize it — to force every pair into an order, because timelines are comfortable and databases have sort keys. There are exactly two honest ways, and the season prices both: *manufacture* an order arbitrarily but consistently, which costs meaning — the next two Parts build the machinery and post the warning signs; or *rent* an order from a distinguished authority through whom all events pass — formally, a total order imposed by sequencing through a distinguished process, which costs a throne, and thrones must be defended: Chapter 4 (contrast the manufactured order of Remark 2.7 below). What one may not do is read a total order off the wall clocks and call it causality. That is the perjury of Part One, now with a theorem behind the indictment.

2.4 Part Four — The Lamport Clock

Having demolished the wall clock's claim to order events, Lamport's paper does something with the confidence of a conjuror: it builds a clock anyway — a *logical* clock, made of nothing but a counter, that captures exactly as much of "before" as actually exists, and not one drop more. The target first, then the instrument.

Definition 2.4 (logical clocks; the clock condition)**.** A *logical clock* is an assignment C of values from a totally ordered set to events, computed by each process from local information only. C satisfies the *clock condition* if $a \rightarrow b \implies C(a) < C(b)$.

The mechanism is three rules that fit in a waistcoat pocket: tick before every event — the "tick" is the increment preceding every event, here and in Box 5 — carry the stamp on every letter; and on receipt, first raise the counter to the larger of its own value and the carried one, then tick.

Protocol — Box 4. The Lamport clock (Lamport 1978)

At each process:

1: **state:** $c \leftarrow 0$

2: **upon** any event e (local step, send, or receive):
3: **if** $e = \mathsf{deliver}(m)$: $c \leftarrow \max\big(c,\ \mathrm{stamp}(m)\big)$
4: $c \leftarrow c + 1$; $C(e) \leftarrow c$
5: **if** $e = \mathsf{send}(m, q)$: attach $\mathrm{stamp}(m) \leftarrow c$

Checked against Lamport (1978), implementation rules IR1–IR2, with unit increments. The tick precedes every event; a receive is stamped downstream of the merge. Theorem 2.1 is its warranty and Remark 2.6 its warning label.

Definition 2.5 (the Lamport clock)**.** The algorithm of Box 4; $C(e)$ denotes the counter value it assigns to event e.

Run it once with the company, small numbers. Bingo's morning entry: $C = 1$; his send: $C = 2$, the letter carrying the number two. Tuppy, idle at nought, receives it: raise to the larger of nought and two, tick

— the receipt is stamped 3; his entry, 4; his send to Gussie, 5. Gussie, whose solitary tea sits at 1, receives it: the larger of one and five is five, tick, 6. The merge rule is the whole trick: the receipt's stamp is forced above the send's, however far the recipient's own counter had lagged. The letter did not merely carry news; it carried *time*, and dragged the recipient's clock forward to meet it.

The instrument's warranty is the clock condition, and its proof fits in two sentences: causality is a road, the counter climbs at every paving stone, and a finite chain of strict increases ends strictly higher than it began. Written out:

Theorem 2.1 (the clock condition holds; Lamport 1978)**.** *The Lamport clock of Box 4 satisfies the clock condition: if $a \to b$ then $C(a) < C(b)$.*

Proof of Theorem 2.1. Let $a \to b$, witnessed by a causal path $a = e_0, e_1, \ldots, e_k = b$, $k \geq 1$. Consider one step. If e_i and e_{i+1} occur at the same process with e_i earlier (HB1), then between the two the counter never decreases and is ticked at least once — before e_{i+1} itself if not sooner — so $C(e_{i+1}) \geq C(e_i) + 1$. If $e_i = \textsc{send}(m)$ and $e_{i+1} = \textsc{receive}(m)$ (HB2), the recipient first raises its counter to at least $\text{stamp}(m) = C(e_i)$ and then ticks for the receive, so again $C(e_{i+1}) \geq C(e_i) + 1$. A finite chain of strict increases ends strictly higher than it began: $C(b) \geq C(a) + k > C(a)$. □

Now the trap, which catches working engineers by the dozen, and which is the precise point where this beautiful instrument, misread, becomes the wall-clock perjury all over again: the clock condition is a one-way street. The run just performed already contains the counterexample — the tea stamped 1, Bingo's send stamped 2, and the two events concurrent: the numbers put them in an order the world does not contain.

Remark 2.6 (the converse fails; what a comparison licenses)*.* Exhibit 2.3: $C(g_1) = 1 < 2 = C(b_2)$, yet $g_1 \parallel b_2$. The sole inference licensed by $C(a) < C(b)$ is $\neg(b \to a)$ — "b did not happen before a." Sorting by Lamport time produces a linear order *consistent with* causality (Rem. 2.7); reading that order *as* causality is the wall-clock perjury of Part One, recommitted with better tooling.

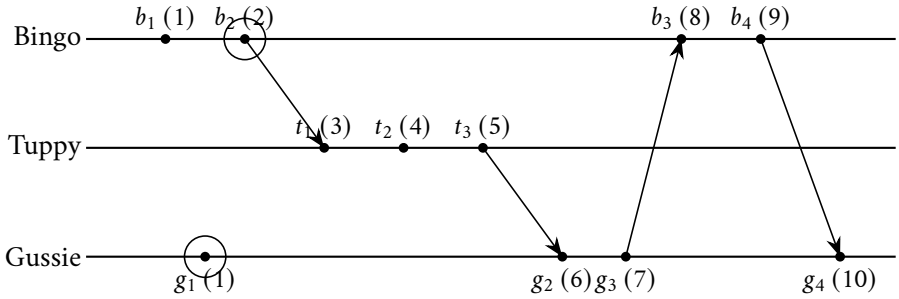


Exhibit 2.3 (Lamport stamps; the inversion). Each event carries its Box-4 stamp in parentheses. The circled pair is the warning label of Remark 2.6 made visible: $C(g_1) = 1 < 2 = C(b_2)$, yet $g_1 \parallel b_2$ — the numbers impose an order the world does not contain. The full computation is Solution 2.2.

An engineer who sorts events by Lamport time and reads the result as a causal history has manufactured exactly the fiction Part Two demolished, with better tooling; it has been done in postmortem documents by people who cite the paper. So what is the manufactured order *for*? Break the remaining ties — concurrent events with equal counters — by any fixed convention; alphabetical order of the gentlemen's names will do.

Remark 2.7 (the manufactured total order)*.* Fix any total order on process names and define $a \Rightarrow b$ iff $C(a) < C(b)$, or $C(a) = C(b)$ and a's process precedes b's by name. Then $\Rightarrow$ is a strict total order on events extending $\to$: two events at one process never share a stamp (the tick), cross-process ties break by name, and Theorem 2.1 ensures $\to$-related pairs already agree. Lamport's mutual-exclusion protocol runs on $\Rightarrow$; so does the replicated state machine (Chapter 5).

An order manufactured, not discovered — arbitrary where the world was silent, but consistent with every road that does exist, and computable by everyone from local information; the warning label stays glued on. And this is where Lamport's paper turns from diagnosis to architecture: it puts ⇒ straight to work on mutual exclusion — granting a shared resource to one claimant at a time, with no coordinator to do the granting. In one breath: every request for the resource is broadcast to the whole company, stamped, and queued; every gentleman sorts his copy of the queue by the manufactured order; and a claimant helps himself precisely when his own request stands at the head of his own copy and he has heard something later-stamped from everyone else — the hearing-from-everyone being how he knows no earlier-stamped request can still be loitering in the Post. No lock server, no chairman: the order itself is the arbiter, and because every process computes the *same* total order from purely local information, all of them agree who stands at the head without ever discussing it. Agreement achieved not by negotiation but by determinism applied to a shared order — and the paper then, almost as an aside, sketches the generalization: *if several replicas of a deterministic machine apply the same commands in the same total order, they remain perfect copies of one another, forever.* One paragraph, tossed off in passing in 1978, in the middle of a paper about clocks — a throwaway remark the industry would spend the next thirty years industrializing. Chapter 5 collects that inheritance in full.

And one landing on the estate you already operate, since the course's standing promise is that theory arrives somewhere you already work. You have been running Lamport clocks for years under assumed names: every *term* in Raft, every *ballot* in Paxos, every *epoch*, *generation*, and *fencing token* from Chapter 4 onward is a Lamport clock in livery — a counter that only rises, carried on messages, merged by taking the maximum, whose entire purpose is to let a recipient say "this instruction is stale — I have seen a higher number" without consulting any wall clock whatsoever. When ZooKeeper stamps its transactions, when a freshly elected leader raises the epoch and the deposed leader's letters begin to bounce, that is the merge rule drawing a salary. The paper is dated 1978; the payroll is current.

The modest summary: the Lamport clock gives a cheap, consistent, manufactured total order, plus a one-way guarantee about the real partial one. What it cannot do is *answer questions* about the real one — it cannot look at two stamps and tell "ordered" from "merely elsewhere." For that, one counter is simply not enough information. The repair is delightfully literal: carry more counters.

2.5 Part Five — The Vector Clock

Diagnose the limitation precisely and the repair falls straight out of the diagnosis. A single counter compresses an event's entire causal ancestry — everything it has heard of, through every chain of letters — into one number; compression loses information; two very different pasts can compress to comparable numbers, and then the numbers imply an order the pasts do not have. If the stamps are to *characterize* causality — to answer "before, after, or elsewhere" from the stamps alone — the stamp must summarize the past without losing it, and the minimal honest summary turns out to be one counter *per colleague*. This is the vector clock, and its history has a pleasing symmetry: it was introduced independently by Colin Fidge in Australia and Friedemann Mattern in Germany — the field discovering the same instrument twice, on opposite sides of the planet, presumably concurrently. (The citations read Fidge 1988 and Mattern 1989, the German account having reached print the following year.) Each process now keeps not one counter but a full roster — an entry for every member of the company — read in the season's fixed template, "Bingo's clock reads: Bingo three, Tuppy one, Gussie nought," and written $V = (3, 1, 0)$, coordinates fixed as (Bingo, Tuppy, Gussie) $= (p_1, p_2, p_3)$. An event's vector timestamp is a précis of its entire ancestry: for each colleague, how much of that colleague's diary the event could possibly have been influenced by. The rules mirror Box 4's — tick your own entry on any event; carry the whole roster on every letter; on receipt, merge entrywise, then tick

your own.

Protocol — Box 5. The vector clock (Fidge 1988; Mattern 1989)

At process p_i, for a company of n:

1: **state:** $V \leftarrow (0, \ldots, 0) \in \mathbb{N}^n$

2: **upon** any event e:
3: **if** $e = \mathsf{deliver}(m)$: **for each** j: $V[j] \leftarrow \max\big(V[j],\ \mathrm{stamp}(m)[j]\big)$
4: $V[i] \leftarrow V[i] + 1$; $V(e) \leftarrow V$
5: **if** $e = \mathsf{send}(m, q)$: attach $\mathrm{stamp}(m) \leftarrow V$

Checked against Fidge (1988) and Mattern (1989). Conventions in the literature differ on where the receive's own tick falls; ours ticks *after* the merge and stamps the receive with the result, exactly as the ledger lemma of Theorem 2.2 requires.

One roster is *below* another when every entry is less than or equal and at least one entry is strictly less; formally:

Definition 2.8 (the vector clock; the vector order)**.** The algorithm of Box 5 over n processes; $V(e) \in \mathbb{N}^n$ denotes the roster it assigns to event e. For rosters U, W: $U \le W$ iff $U[i] \le W[i]$ for every i; $U < W$ iff $U \le W$ and $U \neq W$; $U \parallel W$ (*incomparable*) iff neither $U \le W$ nor $W \le U$.

The same run, now in rosters. Bingo's morning entry: $(1, 0, 0)$; his send: $(2, 0, 0)$, the letter carrying that whole roster. Tuppy's receipt: merge, then tick — $(2, 1, 0)$; his entry, $(2, 2, 0)$; his send to Gussie, $(2, 3, 0)$. And Gussie, whose solitary tea gave him $(0, 0, 1)$, receives it: merge and tick, $(2, 3, 2)$. Read the final roster as the précis it is: this receipt sits downstream of two of Bingo's events and three of Tuppy's, and of one prior event of Gussie's own — the ancestry, itemized. Compare instead the tea, $(0, 0, 1)$, against Bingo's send, $(2, 0, 0)$: the tea is ahead on Gussie's entry, the send ahead on Bingo's; neither below the other; incomparable. The theorem that makes this the chapter's second instrument rather than a mere elaboration is stated with the respect an if-and-only-if deserves — both directions, this time: the stamps no longer merely respect causality; they *encode* it, completely.

Theorem 2.2 (the vector characterization; Fidge 1988; Mattern 1989)**.** *For distinct events a, b of an execution:*

$$a \to b \iff V(a) < V(b),$$

and consequently $a \parallel b \iff V(a) \parallel V(b)$.

The strategy first, then the argument entire. The forward direction is the Lamport argument run entrywise — roads lift rosters. The converse is where the extra information earns its keep, and its engine is the *ledger lemma*: the p-entry of an event's roster counts exactly the p-events in its causal past — a large number in your ledger is testimony, and testimony travels only by post.

Proof of Theorem 2.2. For an event e and a process p, let

$$K_p(e) \;=\; \{\, p\text{-events } f \;:\; f \to e \text{ or } f = e \,\}.$$

Each $K_p(e)$ is a *prefix* of p's timeline: if f' precedes f at p and $f \in K_p(e)$, then $f' \to f$ (HB1) and hence $f' \to e$ by transitivity (or $f' \to e$ directly when $f = e$), so $f' \in K_p(e)$.

Step 1 (the ledger lemma). For every event e and every process p:

$$V(e)[p] \;=\; |K_p(e)|.$$

"The p-entry of an event's roster counts exactly the p-events in its causal past, itself included when the event is p's own."

We induct along the execution order. Let e occur at process q, and let e^- be q's previous event (if any).

Case: e is a local step or a send. Then e receives nothing, so every causal path into e must make its final step by HB1 — through e^-. Hence $f \to e$ iff $f \to e^-$ or $f = e^-$. For $p \neq q$: $K_p(e) = K_p(e^-)$, and Box 5 leaves $V[p]$ unchanged. For $p = q$: $K_q(e) = K_q(e^-) \cup \{e\}$, one element larger, and Box 5 ticks $V[q]$ by one. (Base case, e the first event of q: $K_q(e) = \{e\}$ and the tick raises $V[q]$ from 0 to 1; $K_p(e) = \emptyset$ for $p \neq q$, matching the untouched zeros.)

Case: $e = \textsc{receive}(m)$, with $s = \textsc{send}(m)$. A causal path into e makes its final step either by HB1 (through e^-) or by HB2 (through s). Hence

$$K_p(e) \;=\; K_p(e^-) \;\cup\; K_p(s) \;\cup\; \big(\{e\} \text{ if } p = q\big).$$

Now the decisive observation: $K_p(e^-)$ and $K_p(s)$ are both prefixes of p's timeline, and *the union of two prefixes is the longer of them*, so $|K_p(e^-) \cup K_p(s)| = \max\big(|K_p(e^-)|, |K_p(s)|\big) = \max\big(V(e^-)[p], \text{stamp}(m)[p]\big)$ by the induction hypothesis — which is precisely the entrywise merge of Box 5; the subsequent tick accounts for $\{e\}$ when $p = q$. This proves the lemma, and it is "testimony travels only by post" made mechanical: entries change only at ticks and merges, and merges happen only at receipts.

Step 2 (the theorem). ($\Rightarrow$) Let $a \to b$, with a at process p and b at process q. For every process r, $K_r(a) \subseteq K_r(b)$: any f with $f \to a$ or $f = a$ satisfies $f \to b$ through $a \to b$. By the lemma, $V(a) \leq V(b)$ entrywise. Strictness at coordinate q: $b \in K_q(b)$, but $b \notin K_q(a)$ — for $b \to a$ together with $a \to b$ would close a cycle in a strict order, and $b = a$ is excluded — so $V(a)[q] < V(b)[q]$. Hence $V(a) < V(b)$.

($\Leftarrow$) Let $V(a) < V(b)$, and let a be the k-th event of its process p, so that $V(a)[p] = k$ by the lemma ($K_p(a)$ is exactly p's first k events). Then $V(b)[p] \geq k$, so $K_p(b)$, a prefix of p's timeline of size at least k, contains p's k-th event — namely a. Thus $a \to b$ or $a = b$; the events are distinct; so $a \to b$.

Finally, $a \parallel b \iff V(a) \parallel V(b)$ is the conjunction of the two directions, negated. □

Concurrency thereby acquires its final and best characterization: two events are concurrent exactly when each is ignorant of some part of the other's past — mutual, certified, provable ignorance. That is what "merely elsewhere" cashes out to.

Now the instrument doing the one job that will matter most in the weeks ahead: detecting concurrency *in anger*. Replay Chapter 1's third performance — Bertie's two delivery notes, "leave with the porter" and then, on reflection, "ring the top bell," racing through the Post and reaching Gussie in the wrong order — with rosters. Both notes enter at Bingo: the first is applied and forwarded stamped, say, $(4, 2, 1)$; the correction enters a moment later and leaves stamped $(5, 2, 1)$. The stamps are ordered — the correction demonstrably saw the original; it is a true successor, and every replica, in whatever order the Post delivers the pair, can apply the elder first or discard it outright: the stamp, not the arrival time, carries the truth, and Gussie's little tragedy becomes impossible. Now the darker variant: Bertie, impatient with Bingo's silence, sends the porter note to Bingo — stamped in due course $(4, 2, 1)$ — and the bell note directly to Gussie, who, having heard little from Bingo lately, stamps it $(2, 2, 3)$. Each roster is ahead of the other somewhere: incomparable; concurrent. And the system now knows something Chapter 1's system could not even express: these two instructions are not an update and its correction — they are *rivals*, written in mutual ignorance, and no protocol has any business silently pretending that one superseded the other. What one ought to *do* with certified rivals — keep both and ask Bertie; merge them under some lawful

rule — is a genuine question with a genuine menu, and it belongs to Chapters 6–7; the shopping cart obliged to hold two kettles at once is exactly this moment at industrial scale. The point here is smaller and sharper: the vector clock converts "the replicas mysteriously disagree" into "the replicas hold two provably concurrent writes" — from a haunting into a diagnosis. Diagnoses have treatments; hauntings merely have postmortems.

The costs, stated honestly, because this instrument is priced by the roster. Every letter now carries an entry for every member of the company: the freight grows with the fleet, and a fleet of thousands makes the stamps heavier than many payloads. Worse, the roster presumes you know who the company *is*: membership — who is in the vector — is itself a distributed agreement problem, which should by now give you a familiar prickle at the back of the neck; it is met properly when agreement is met properly. Engineering has answers — prune entries, gossip compactly, and a refinement called the *dotted version vector*, named here and deliberately left wrapped: it belongs to Chapter 6, where the shopping cart will need to hold two kettles at once and ask which one Bertie truly meant. One might hope instead to compress the roster — to shrink a company of a thousand down to a stamp of civilized size while still answering "ancestor or cousin" exactly. The hope is natural, and it is vain: Problem 2.11 walks the ambitious reader to the theorem that the vector's width is not an implementation's clumsiness but the problem's true dimension — characterizing causality in a company genuinely requires one coordinate per member. What the industry actually does, therefore, is not compress the truth but ration it: track fewer participants (versions per key rather than per node); prune the retired; or retreat to the one-number clock and accept its one-way honesty. Every deployed scheme is one of those three retreats, priced differently.

Side by side, then, the honest summary of the two instruments: the Lamport clock is one number — cheap, total after tiebreaking, manufactured, one-way. The vector clock is a roster — costly, partial, *truthful*, two-way. One gives an order to act on; the other gives the facts of the matter. Mature systems tend to carry both and confuse neither.

2.6 Part Six — A Glimpse of Disciplined Delivery

Before the set piece, a short corridor, glimpsed and deliberately not entered. The Post reorders, and with the vocabulary now owned one can say precisely which reorderings are rude: the Post may show a process an event before showing it that event's *causes*. Tuppy asks a question of the room; Gussie's answer, racing through a luckier route, reaches Bingo before the question does — and Bingo beholds a reply to nothing, an effect arriving in advance of its cause. Nothing in Chapter 1's charter forbids it. But with vector clocks Bingo can *detect* the rudeness: the answer's roster confesses its ancestry — it sits downstream of a Tuppy event Bingo has not yet seen — so Bingo simply sets the letter aside, unopened, until its prerequisites arrive. Hold the mail until its past has been delivered. Done systematically, this is *causal delivery*: a Post that never shows anyone an effect before its cause — still free to shuffle *concurrent* letters, because merely-elsewhere events have no order to violate, but forbidden to invert the roads.

That one idea — buffer until the ancestry is present — is a genuine consistency *contract*, purchasable at known cost, and it sits at a very particular altitude: stronger than the Post's anarchy, far cheaper than a total order, and exactly aligned with what the roads actually require. But contracts belong in a catalogue, alongside their prices and their fine print, and the catalogue is Chapter 7 — where causal consistency takes its place in a hierarchy, and where its practical children, the *session guarantees*, turn out to be things your systems already half-promise under folk names like read-your-own-writes. The corridor exists; it was the vector clock that unlocked it. Onward; the photograph awaits.

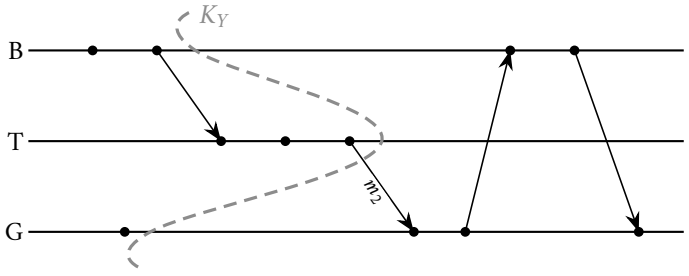
2.7 Part Seven — The Photograph

The set piece, and the problem before the miracle, because the problem is the chapter's own paradox wearing a business suit. One operates a system — say, a small bank — and would like to know its *global state*: every account balance, everywhere, at once. The reasons are not exotic; they are Tuesday. A backup — a checkpoint from which the world could be restored. An audit — does the money add up. A detector — is the system deadlocked, is a computation finished, has an invariant quietly broken. Every one of these questions has the same grammar — *what is the state of the whole system, now?* — and the whole chapter has been spent executing "now." There is no universal instant to photograph; machines cannot even agree what time it is, and if they could, the agreement would be causally meaningless. The question seems not merely hard but *ill-posed* — and yet the backup must be taken. Industry, as usual, cannot wait for the philosophy to resolve.

First, what goes wrong when one photographs naively. Bingo keeps Bertie's current account: £60. Tuppy keeps his savings: £40. One hundred pounds in the world — the invariant an audit should confirm. Bertie moves ten pounds from current to savings: Bingo debits ten — his ledger reads £50 — and posts the ten pounds to Tuppy in a letter. While that letter is in the Post, the auditor telephones. He calls Bingo first: fifty. He calls Tuppy a moment later — the letter has still not arrived: forty. Total: £90. *Ten pounds have vanished from the audit.* Run the interleaving the other way — Tuppy called just *after* the letter arrives (fifty), Bingo just *before* the debit (sixty) — and the audit totals £110: *ten pounds minted.* Note the precise shape of the second failure, because it is the shape to remember: the photograph included a letter's *arrival* without including its *departure* — an effect without its cause — and such a picture corresponds to no state the system was ever in, or could ever have been in. Money conjured by bad photography. (Problem 2.6 enumerates all four interleavings and names each one's sin.)

So say what a *good* photograph would even mean, since "the state at one instant" is off the table. Picture the space-time diagram of the whole bank — every gentleman's timeline, every letter an arrow between them — and draw a single frontier across it, separating each timeline into a past and a future. The frontier, formally, is a *cut*; "no effect without cause" is the cut's *consistency*. Arrows may cross the frontier forwards — a letter sent in the past, still undelivered: money in flight, and an honest photograph records it as such — but no arrow crosses backwards.

Definition 2.9 (cuts; consistent cuts)**.** A *cut* K of an execution is a set of events whose restriction to each process is a prefix of that process's events; its *frontier* is the last K-event at each process. K is *consistent* iff every receive in K has its matching send in K: no effect without its cause. (Check: this is exactly closure of K under $\rightarrow$ — HB1 closure is the prefix property, HB2 closure is the stated clause, and HB3 adds nothing new; Problem 2.9.)



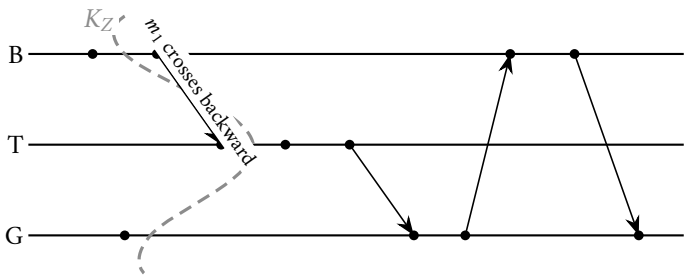


Exhibit 2.4 (two frontiers). Top: the cut K_Y (Bingo through b_2, Tuppy through t_3, Gussie through g_1) is consistent; m_2 crosses it *forward* — mail in flight, recorded as channel state, entirely lawful. Bottom: the cut K_Z (Bingo through b_1, Tuppy through t_1, Gussie through g_1) is inconsistent: $t_1 = \text{RECEIVE}(m_1)$ lies inside while $b_2 = \text{SEND}(m_1)$ lies outside — an effect without its cause. Problem 2.5 classifies these and two more.

Here, then, is the redefinition that rescues the ill-posed question, and it is pure Lamport in spirit: abandon "the state at an instant," which does not exist, and pursue "the state along a consistent cut," which does. The frontier need not be straight; it need not correspond to any single wall-clock moment; different processes may be photographed at wildly different local times. It need only respect the roads. What the snapshot must deliver — and the currency of reachability between global states, $S \rightsquigarrow S'$ (some legal execution leads from S to S'), in which its strongest promise is denominated — is fixed now:

Definition 2.10 (global state of a cut; the snapshot problem)**.** For a consistent cut K: the *global state* $S(K)$ comprises, for each process, its state after executing exactly its K-prefix, and, for each channel $c = (p, q)$, the *cross-frontier mail* $L_c(K) = \{\, m \text{ sent on } c \text{ in } K : \text{RECEIVE}(m) \notin K \,\}$. The *snapshot problem*: during a live execution, the processes must, by exchanging ordinary messages, record process states and channel states equal to $S(K)$ for *some* consistent cut K of that execution. The *strong form* adds: $S(K)$ is reachable from the global state at which the protocol began, and the global state at which it completed is reachable from $S(K)$. Theorem 2.4 delivers the strong form.

Very well — but *how*? The gentlemen cannot see the diagram; they live inside it. Each knows only his own ledger and his own mail. And they cannot stop: a bank that pauses the world to take its picture has solved the problem by not having a system. The photograph must be taken live, from within, by the participants, while the letters keep flying. Enter, in 1985, in the ACM's Transactions on Computer Systems, K. Mani Chandy and Leslie Lamport — the same Lamport, seven years on, now finishing the thought — with an algorithm of such economy that, once seen working, one suspects it of hiding something. It hides nothing. The entire mechanism is one special letter — the *marker*, the control message of Box 6, drawn densely dotted in the exhibits from here to the season's end (a convention added this chapter) — and two rules; the honest disclosure, per the standing terms of the course, is that the algorithm requires the better Post already purchased and declared in this chapter's model box — channels reliable and first-in-first-out — and the exact instant where the FIFO clause earns its entire keep is marked below. The rules, as correspondence. *The recording rule*: the initiator records his own ledger, at once, and *immediately* — before conducting one further scrap of business — posts a marker on every outgoing channel. *The marker rule*: any gentleman receiving his *first* marker does the same at once — records his ledger, notes the channel that marker arrived on as having empty in-flight state, and posts markers on all his outgoing channels; and for every other incoming channel he opens a *docket* — formally, that channel's recorded state — collecting every ordinary letter that arrives after his own recording but before that channel's marker, at which point the docket closes. The marker is a sweep — a line drawn down each channel, saying: everything ahead of me on this route belongs to the past of the photograph; everything behind me, to its future.

Protocol — Box 6. The Chandy–Lamport snapshot (1985)

Control message ⟨marker⟩ (tagged with an audit identifier if audits may overlap). At each process:

1: **state:** *recorded* ← **false**; **for each** incoming channel c: docket(c) ← ⊥ (unopened)

2: **procedure** RECORD():
3: record own application state; *recorded* ← **true**
4: **immediately** — before any further application send:
5: **for each** outgoing channel: send(⟨marker⟩)
6: **for each** incoming channel c with docket(c) = ⊥: docket(c) ← ⟨ ⟩ (open)

7: **upon** initiating the audit: RECORD()

8: **upon** deliver(⟨marker⟩) on channel c:
9: **if** ¬*recorded*: docket(c) ← ⟨ ⟩, closed (photographs empty); RECORD()
10: **else**: close docket(c) (its contents are c's recorded state)

11: **upon** deliver(m) on channel c, m an application message:
12: process m normally; **if** docket(c) is open: append m to docket(c)

13: locally complete when *recorded* and all dockets closed;
14: the fragments (state, dockets) travel to the collector by ordinary mail.

Checked against Chandy and Lamport (1985), §3 (their Marker-Sending and Marker-Receiving Rules). The word **immediately** on line 4 *is* the algorithm — it is what welds the marker's channel position to the sender's recording instant, and it is the entire content of the FIFO purchase (Rem. 2.11). Problem 2.10 is what happens when an implementer economizes on it.

Now the performance, with Bertie's ten pounds in flight, so the money may be watched being counted exactly once. The state of play: Bingo has debited — his ledger reads £50 — and the ten-pound letter is in the Post toward Tuppy, whose ledger reads £40. At this delicate moment *Tuppy* takes the chair: records £40; writes to Bingo, immediately: marker; and opens a docket on his incoming channel from Bingo. Two letters are now converging on the participants — the ten pounds, ambling toward Tuppy; the marker, hurrying toward Bingo. The ten pounds arrives first. Tuppy has already recorded, so the tenner does not touch his recorded forty: he credits his *live* ledger, which now reads fifty, and — the channel's marker not yet having arrived — into the docket it goes. Ten pounds, noted as in flight. Then Tuppy's marker reaches Bingo. It is Bingo's first marker: he records his ledger — fifty — notes the channel from Tuppy as empty, and posts a marker back to Tuppy. That marker travels the same route as the tenner did, and here first-in-first-out collects its salary: posted after the tenner, it *cannot overtake it*; the sweep arrives behind the money, never ahead of it. It reaches Tuppy; the docket closes, containing exactly one item. The audit assembles: Tuppy recorded, forty; Bingo recorded, fifty; in flight from Bingo to Tuppy, per the docket, ten. One hundred pounds — conserved to the penny, while the money was moving, while both ledgers were changing, without one instant of pause, without one glance at a wall clock.

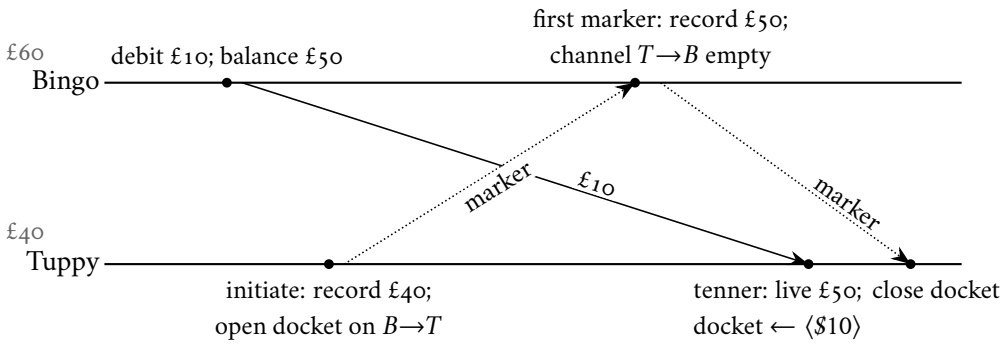


Exhibit 2.5 (the audited bank; Part Seven's walkthrough). Markers are drawn densely dotted — a season convention added this chapter. Tuppy initiates mid-transfer. On the channel Bingo → Tuppy, Bingo's marker is enqueued *behind* the tenner and, by FIFO, may not overtake it — the exact moment the model box's purchase is spent. The assembled photograph: $40 + $50 + $10 in flight = $100, conserved to the penny, without a pause and without a wall clock.

With the full company, the general protocol is the same sentence pronounced louder. Markers go out on *every* outgoing channel — had Gussie been holding Bertie's deposit box as well, Tuppy's initiation would have posted him a marker too; Gussie, on his first marker, records his own ledger, returns markers to both colleagues, and opens dockets on each remaining incoming channel, closing each as that channel's own marker arrives. Every channel is swept exactly once; every gentleman records exactly once; the audit is complete when every docket in the company has closed. The bill for the elegance reads straight off the mechanism: one marker per channel, so a fully connected company of n gentlemen pays on the order of n^2 markers per photograph — trifling for three, worth budgeting for three thousand, and one reason the industrial costume this algorithm eventually wears (Chapter 10) sends its markers down a pipeline rather than around a clique. The photograph scales exactly as well as the postal map allows.

Why it works — not for this run, but always — is the chapter's last labour, and it needs one piece of equipment first: the "innocent reshuffling" on which the coda below will lean, stated as a lemma. Adjacent concurrent events in an execution may be transposed, and nothing any process could testify to changes — the relative order of concurrent events is a coordinate of the description, not of the system. Problem 2.7 has you build it by hand; the proof is given in full.

Lemma 2.3 (commutation of concurrent neighbours)**.** *Let $\alpha = \langle \dots, e, f, \dots \rangle$ be an execution in which e immediately precedes f and $e \parallel f$. Then $\alpha' = \langle \dots, f, e, \dots \rangle$, the same sequence with the pair transposed, is an execution with the same events, the same message pairing, the same per-process local histories, and the same happens-before relation; and the global state at every position outside the transposed pair — in particular before e and after the pair — is identical in α and α'.*

Proof of Lemma 2.3. Write P and Q for the processes of e and f. First, $P \neq Q$: adjacent events of one process are HB1-ordered, contradicting $e \parallel f$. Second, f is not the receive of a message sent at e: that is HB2, again contradicting $e \parallel f$.

α' is an execution. (a) Per-process subsequences are unchanged — only the interleaving of P's and Q's steps moved — and a deterministic machine's behaviour depends only on its own prior events and the contents of messages it has received, all unchanged. (b) If $f = \textsc{receive}(m_f)$, the matching send precedes f in α and is not e; it therefore still precedes f in α'. (c) Channel discipline. All sends on a given channel are events of one process, and all receives of one channel occur at one process; since $P \neq Q$, the pair e, f cannot contain two sends, nor two receives, of the same channel, so the per-channel order of sends and the

per-channel order of receives are both unchanged, and the FIFO pairing (the k-th receive matches the k-th send) is untouched.

Invariants. Events, message pairing, and per-process local histories are identical by construction; hence HB1 and HB2, and with them $\rightarrow$, are identical. For states: each process's state at each of its own events depends only on its local history, unchanged; and at any position of the sequence outside the transposed pair, the set of executed events is the same in α and α', with identical per-process prefixes and identical channel contents (same sends minus same receives, in the same per-channel order). In particular the global states immediately before e and immediately after the pair coincide. Between the two transposed positions the executions differ — which is the point. □

With the lemma in hand, the correctness of the photograph, entire: termination, consistency of the cut, exactness of the dockets, and "a photograph of a possible present" — the reachability guarantee of part (iii).

Theorem 2.4 (the photograph; Chandy–Lamport 1985)**.** *Run Box 6, initiated by any single process, within an execution α over this chapter's model. Then:*

(o) ***Termination.*** *Every process records exactly once; every channel carries exactly one marker; every docket closes.*

(i) ***Consistency.*** *The set K of* pre-recording *events — events at each process before that process records — is a consistent cut.*

(ii) ***Accounting.*** *The recorded process states and dockets are exactly the global state $S(K)$; in particular, for every channel c,* $\mathrm{docket}(c) = L_c(K)$*, the cross-frontier mail.*

(iii) ***A possible present.*** *There is an execution α', obtained from α by finitely many transpositions of Lemma 2.3 — hence with identical local histories at every process — in which $S(K)$ is the actual global state at the moment the last pre-recording event completes. Consequently $S_{\text{init}} \rightsquigarrow S(K) \rightsquigarrow S_{\text{compl}}$, where S_{init} and S_{compl} are the global states at the protocol's initiation and completion.*

Proof of Theorem 2.4. (o) Termination. The initiator sends one marker on every outgoing channel; every process, on its first marker, records and does likewise; the "first marker" guard ensures no process records twice. Channels are reliable and the postal map strongly connected, so by induction on distance from the initiator every process eventually receives a marker and records; consequently every process sends exactly one marker per outgoing channel, every channel carries exactly one marker, and every docket eventually closes.

Setup. For each process q let $\mathrm{rec}(q)$ be the point at which q records. An application event at q is *pre-recording* if it occurs before $\mathrm{rec}(q)$, *post-recording* otherwise; K is the set of pre-recording events. (Markers and the recording acts are control events, superimposed on the application execution and not counted among its events.) By construction K restricts to a prefix at each process.

Key Lemma (where FIFO is spent). If $\textsc{send}(m)$ is post-recording, then $\textsc{receive}(m)$ is post-recording. *Proof.* Let m travel channel $c = (p, q)$. Since p sends m after recording, and Box 6 emits p's marker on c *immediately* upon recording — before any further application send — the marker entered c ahead of m; FIFO keeps it ahead; so the marker reaches q before m does, and its arrival forces q to record if it had not already. Hence q has recorded before m arrives; the lemma is proved.

(i) Consistency. K is a union of per-process prefixes. If $\textsc{receive}(m) \in K$ it is pre-recording; by the Key Lemma's contrapositive $\textsc{send}(m)$ is pre-recording, hence in K. By Definition 2.9, K is a consistent cut.

(ii) Accounting. Fix a channel $c = (p, q)$. The marker enters c at $\mathrm{rec}(p)$; by FIFO, the messages arriving at q on c *ahead of the marker* are exactly those sent *before* $\mathrm{rec}(p)$ — the pre-recording sends — and by

reliability every one of them does arrive ahead of the marker, hence before the docket closes. The docket collects arrivals after $\mathrm{rec}(q)$ and before c's marker; therefore

$$\mathrm{docket}(c) \;=\; \{\, m \text{ on } c : \textsc{send}(m) \in K,\ \textsc{receive}(m) \notin K \,\} \;=\; L_c(K).$$

The channel on which a process's *first* marker arrived is recorded empty, correctly: on that channel every pre-recording send arrived ahead of the marker, i.e., before $\mathrm{rec}(q)$ — such messages are received *in* K and are not cross-frontier mail. Finally, each process records its state at $\mathrm{rec}(q)$, which is its state after executing exactly its K-prefix. The recorded data is therefore precisely $S(K)$.

(iii) A possible present. Events before the first recording are all pre-recording; events after the last recording are all post-recording; the mixture is confined to the window between. While some post-recording event e immediately precedes a pre-recording event f, transpose them. This is licensed: the two are at different processes (each process's pre-recording events all precede its post-recording ones); adjacency admits $e \to f$ only by HB1 (same process — excluded) or HB2 (e a send received at f — excluded, since by the Key Lemma the receive of a post-recording send is post-recording, while f is pre-recording); and $f \to e$ is impossible since causal paths run forward in α. Hence $e \parallel f$ and Lemma 2.3 applies: the result is an execution with the same events, local histories, $\to$, and boundary states. Each transposition strictly reduces the number of ordered pairs (post-recording before pre-recording), which is finite; so the process terminates in an execution α' in which every pre-recording event precedes every post-recording one.

In α', consider the position just after the last pre-recording event. Every process has executed exactly its K-prefix, so its state is its recorded state; every channel holds exactly the messages sent in K and not received in K — that is, $L_c(K)$, the dockets. The global state at that position is therefore $S(K)$, occupied by the legal execution α'. Since α' agrees with α before the window and, statewise, from the window's end onward (Lemma 2.3 preserves states outside each transposed pair; induct over the transpositions), α' passes through S_{init}, then $S(K)$, then S_{compl}:

$$S_{\mathrm{init}} \rightsquigarrow S(K) \rightsquigarrow S_{\mathrm{compl}}.$$

And because local histories in α' are identical to those in α, no process — and no client interrogating any process — could distinguish the world in which the photograph "really happened." □

Remark 2.11 (the FIFO purchase, itemized)*.* The FIFO hypothesis is spent exactly once — in the Key Lemma of the proof (a post-recording send is never received pre-recording, because the sender's marker entered the channel first and cannot be overtaken) — and nowhere else. Problem 2.12 buys the same theorem in a different currency: a one-bit colour smeared onto every letter.

What is a photograph *for*? The classic answer — the one Chandy and Lamport built the paper around — is the detection of stable properties: conditions which, once true, stay true. Here the plurality of presents turns from liability into licence: the photograph depicts a state the system could genuinely have occupied, situated between the audit's beginning and its end, so a stable truth read off it held in a reachable state and stability carries it forward into the real present, while a stable falsehood had not yet occurred when the audit began.

Corollary 2.5 (stable properties; the licensed uses of a photograph)**.** *Call a predicate φ on global states* stable *if $\varphi(S)$ and $S \rightsquigarrow S'$ imply $\varphi(S')$. Then $\varphi(S(K))$ implies φ holds at completion and ever after; and $\neg\varphi(S(K))$ implies φ did not hold at initiation. Deadlock, termination of a computation, loss of a token, and permanent breakage of an invariant are stable. "The queue is exactly seven deep" is not, and about such properties the photograph testifies to nothing.*

Proof of Corollary 2.5. By Theorem 2.4(iii), $S_{\text{init}} \rightsquigarrow S(K) \rightsquigarrow S_{\text{compl}}$. If $\varphi(S(K))$, stability propagates φ along $S(K) \rightsquigarrow S_{\text{compl}}$ and every continuation beyond. If $\varphi(S_{\text{init}})$, stability gives $\varphi(S(K))$; contrapositively, $\neg\varphi(S(K))$ implies $\neg\varphi(S_{\text{init}})$. □

Two practical notes, so the instrument leaves the desk usable. First, the mechanics of collection: recording is entirely local — each gentleman ends the protocol holding his own ledger copy and his dockets — so the fragments must still be gathered, by ordinary post, to whoever wants the album; and the gathering may be as leisurely as one pleases, because recorded fragments do not age. Second, there need not be one photographer: several gentlemen may initiate audits independently — markers carry an audit's identifier, dockets are kept per audit, and the runs interleave without quarrel. And throughout all of it the letters keep flying; at no point did anyone ask the bank to close. The pause that could not be afforded turns out to be a pause nobody needed.

And the coda, the chapter's designated moment of wonder, now standing on the mathematics above. Ask the awkward question: the state the audit assembled — Tuppy forty, Bingo fifty, ten in the Post — *was the bank ever actually in that state?* Scan the true history: no. At no actual moment did that exact configuration obtain; the pieces were recorded at different times, mid-flow. The photograph depicts an instant that never happened — and it is nonetheless a *legitimate* instant, by part (iii): herd every pre-recording event before every post-recording one, one innocent transposition of Lemma 2.3 at a time — the consistency of the cut is exactly what makes the herding possible — and the recorded state is the state between the herds, reachable from the initiation, the true completion reachable from it, and no process, no client interrogating any process, able to testify against it. In a world with a master clock such a photograph would be a falsehood. But there is no master clock — that is the whole chapter — and the honest accounting is this: the system's history was never one sequence of instants; it was a lattice of roads, and *every* consistent cut through that lattice is as much a "now" as any other. The algorithm did not fake a present. It selected one — legitimately, from the plurality of presents the mathematics actually offers. The season permits itself one admiration per chapter, and this chapter's is spent here — a small piece of philosophy that compiles. One receipt filed before the close, so the ledger stays honest: this exact algorithm is not a museum piece; it runs, under an assumed name and industrial tailoring, inside stream-processing machinery sir personally operates, its markers travelling the pipelines at this hour. The unmasking is scheduled, with full ceremony, for Chapter 10.

2.8 The Whiteboard

For the design review.

1. Never order cross-node events by wall clock: a cross-machine timestamp comparison is an opinion wearing digits, and at message granularity not a well-informed one. If a design sorts, joins, or adjudicates by cross-machine timestamps — last-write-wins by clock, log-merge forensics, expiry decided by somebody else's watch — demand the uncertainty bound in writing, or call it folklore with a schema (Chapter 8 buys the bound, and the price will clarify why nobody gets it for free).
2. Two clocks, two jobs, never crossed: durations, timeouts, and intervals belong to the monotonic clock; naming moments for humans belongs to the time-of-day clock. Any duration computed by subtracting wall-clock readings is a latent incident with a date of its own choosing.
3. If concurrent writes can happen, choose the regime on purpose: carry causality explicitly (vectors, version stamps) so that rivalry is detectable and adjudicable, or rent a total order from a leader so that rivalry is impossible by construction (Chapter 4). Either is defensible; drifting between the two, the industry's default posture, is precisely where the lost updates live.

4. A snapshot is not a pause: any design that requires stopping the world for a consistent view is paying for a theorem nobody proved; markers deliver consistency in motion. Corollary for the estate you already operate: interrogate the checkpoint and backup tooling about what its "consistent" actually means — a consistent cut, or a superstition with a cron schedule.

The register. Paid, with a small ceremony — the season's first debt to move from the left column to the right: debt #3, *no shared clock* (issued Ep. 1), and note the currency — not a synchronization protocol, which would have been paying a falsehood with a gadget, but a replacement theory of order, under which the clock's absence stopped being a defect and became a fact with structure. Issued, four fresh entries, each due later in the season: physical clocks return, armed and expensive — a company purchasing better physics and tendering the apology in hardware (Ch. 8); the photograph returns in industrial costume, its markers in the pipelines (Ch. 10); a total order may be rented from a leader — the throne, and the defence of the throne (Ch. 4); and causal delivery, glimpsed through a doorway, becomes a purchasable contract with a price tag attached (Ch. 7).

2.9 Problems

All computations refer to the reference trace, Exhibit 2.1, unless a problem states otherwise.

Tier I — Calisthenics

Problem 2.1 (ordering drill)**.** Classify each pair as $\rightarrow$, $\leftarrow$, or $\|$, exhibiting the causal path where one exists: (b_1, t_2), (g_1, b_2), (t_2, g_3), (b_3, g_3), (t_1, g_1), (b_4, g_2).

Problem 2.2 (Lamport stamps and their inversions)**.** Compute $C(e)$ for all eleven events by Box 4. Then list every pair involving g_1 whose stamps are ordered while the events are concurrent, and every pair involving g_1 whose stamps tie. State the one inference that $C(a) < C(b)$ does license.

Problem 2.3 (vector clocks)**.** Compute $V(e)$ for all eleven events by Box 5, coordinates (Bingo, Tuppy, Gussie). Verify Theorem 2.2 on the pairs (b_2, g_2), (g_1, t_2), and (t_2, b_4).

Problem 2.4 (the merge drill)**.** Let $u = (3, 1, 4)$, $v = (2, 5, 0)$, $w = (3, 5, 4)$. (a) Compute the entrywise suprema $\sup(u, v)$, $\sup(u, w)$, $\sup(v, w)$. (b) Which pairs are comparable under the vector order? (c) Suppose a, b, c are distinct events with $V(c) = \sup(V(a), V(b))$. Show that $a \rightarrow c$ and $b \rightarrow c$. (You will want: the map $e \mapsto V(e)$ is injective on events — prove it from the ledger lemma.)

Problem 2.5 (frontiers, classified)**.** Four cuts of the reference trace, given by per-process prefixes: $X = (\leq b_2, \leq t_1, \leq g_1)$; $Y = (\leq b_2, \leq t_3, \leq g_1)$; $Z = (\leq b_1, \leq t_1, \leq g_1)$; $W = (\leq b_4, \leq t_1, \leq g_3)$. Which are consistent? For each consistent cut, list every channel's cross-frontier mail; for each inconsistent one, name the offending letter.

Problem 2.6 (the auditor's telephone)**.** The bank of Part Seven: Bingo holds £60, Tuppy £40; one transfer of £10 is executed (debit d at Bingo, letter m, credit r at Tuppy). A naive auditor telephones each gentleman once and sums the two spoken balances. (a) Enumerate the four cases (call to Bingo before or after d) $\times$ (call to Tuppy before or after r) and give each audit's total. (b) Treating the frontier "just after each call" as a cut, say for each bad total whether the sin is an *inconsistent cut* or a *consistent cut with an ignored channel*. (c) Conclude which sin the markers of Box 6 cure and which the dockets cure.

Tier II — The Real Thing

Problem 2.7 (the swap lemma, built by hand)**.** Prove Lemma 2.3 directly from Definitions 2.1–2.3: (a) show the two events occur at different processes and are not a matched send–receive pair; (b) show the transposed sequence is an execution (programs, message availability, channel discipline); (c) show events, pairing, local histories, and $\rightarrow$ coincide; (d) show the global state at every position outside the pair coincides. This is the engine inside Theorem 2.4(iii); the coda's "innocent reshuffling" is exactly this lemma, iterated.

Problem 2.8 (serialization freedom)**.** (a) Show that *every* linear extension β of $\rightarrow$ on the events of an execution α is itself an execution. (b) Show β is reachable from α by finitely many transpositions of Lemma 2.3. (c) Conclude that for $e \parallel f$ there are executions, indistinguishable to every process from α, in which e precedes f and in which f precedes e — the relative order of concurrent events is a coordinate of the description, not of the system.

Problem 2.9 (the lattice of frontiers)**.** (a) Prove the parenthetical of Definition 2.9: a cut is consistent iff it is closed under $\rightarrow$ (in particular, HB3 adds no new obligation). (b) Show consistent cuts are closed under union and intersection. (c) Show $\downarrow e = \{f : f \rightarrow e\} \cup \{e\}$ is the least consistent cut containing e, and compute $\downarrow g_2$ on the reference trace.

Problem 2.10 (sabotage: the thrifty auditor)**.** A well-meaning implementer economizes on Box 6: *"upon recording, a process first dispatches any application messages already queued in its outgoing tray, and only then emits its markers — the markers ride at the back of the tray, to save a round of envelopes."* All else is unchanged. (a) Exhibit an execution of the two-gentleman bank in which the audit reports £110 while the true total is £100 throughout. (b) Identify precisely which statement in the proof of Theorem 2.4 dies — and which, perhaps surprisingly, survives. (c) State the minimal repair, and point to the exact word of Box 6 that constitutes it.

Tier III — For the Ambitious (starred)

Problem 2.11 ($\star$ the incompressible roster)**.** Suppose some scheme assigns to every event of every n-process execution a stamp $W(e) \in \mathbb{R}^k$ such that, for all distinct a, b: $a \rightarrow b \iff W(a) < W(b)$ (entrywise $\leq$, strict somewhere). Show $k \geq n$. Guided route: (a) show such a W yields k linear extensions of $\rightarrow$ whose intersection is exactly $\rightarrow$ — so the *order dimension* of $\rightarrow$ is at most k; (b) consult Charron-Bost (1991) for an n-process computation whose causality order has dimension exactly n, and conclude. Moral: the vector's width is not an implementation's clumsiness but the problem's true dimension; the industry *rations* the roster (versions per key, retired members pruned) — it does not compress it.

Problem 2.12 ($\star$ photographs without first-in-first-out)**.** Channels are reliable but may reorder. Design a snapshot in the style of Lai and Yang (1987): every application message carries one bit — *white* if its sender had not yet recorded, *red* otherwise. (a) Specify the protocol: when does a process record, what do the dockets collect, and how does a docket know it may close? (b) Prove the pre-recording cut is consistent. (c) Say exactly which two services FIFO was providing in Box 6, and what purchases replace them here.

2.10 Hints

Hint (Problem 2.9)**.** For (a): a causal path's final step into a cut-member is HB1 or HB2, and each of those is already closed by the two stated clauses; induct backwards along the path. For (c): $\downarrow e$ is a prefix at each process by transitivity, and receive-closed because HB2 $\subseteq \rightarrow$.

Hint (Problem 2.10)**.** Follow the second tenner. Arrange for the marker to reach Bingo while Bertie's next order sits in the outgoing tray.

Hint (Problem 2.11)**.** For (a): tie-break every coordinate by one fixed linear extension of $\rightarrow$ itself; check that ordered pairs agree in all k linearizations and concurrent pairs disagree between some two. For (b): the crown poset is the shape to look for.

Hint (Problem 2.12)**.** The colour answers the only question the marker ever answered: was this letter posted before or after its sender's photograph? What the colour cannot do is announce that the *last* white letter has arrived — FIFO's second, quieter service. Sell that separately: a per-channel count, sent once, at recording time.

2.11 Solutions

Solution (to Problem 2.1)**.** $b_1 \rightarrow t_2$ (path b_1, b_2, m_1, t_1, t_2). $g_1 \parallel b_2$ (no path either way). $t_2 \rightarrow g_3$ (path t_2, t_3, m_2, g_2, g_3). $b_3 \leftarrow g_3$, i.e. $g_3 \rightarrow b_3$ (message m_3). $t_1 \parallel g_1$. $b_4 \leftarrow g_2$, i.e. $g_2 \rightarrow b_4$ (path g_2, g_3, m_3, b_3, b_4).

Solution (to Problem 2.2)**.** C: b_1=1, b_2=2, t_1=3, t_2=4, t_3=5, g_1=1, g_2=6, g_3=7, b_3=8, b_4=9, g_4=10. Ordered-but-concurrent pairs involving g_1: (g_1, b_2), (g_1, t_1), (g_1, t_2), (g_1, t_3) — stamps $1 < 2, 3, 4, 5$, all four pairs concurrent. Tie: (b_1, g_1), both stamped 1, concurrent. The sole licensed inference from $C(a) < C(b)$: $\neg(b \rightarrow a)$.

Solution (to Problem 2.3)**.** V: b_1=(1, 0, 0), b_2=(2, 0, 0), t_1=(2, 1, 0), t_2=(2, 2, 0), t_3=(2, 3, 0), g_1=(0, 0, 1), g_2=(2, 3, 2), g_3=(2, 3, 3), b_3=(3, 3, 3), b_4=(4, 3, 3), g_4=(4, 3, 4). Checks: $V(b_2) = (2, 0, 0) < (2, 3, 2) = V(g_2)$ and indeed $b_2 \rightarrow g_2$ via m_1, t_3, m_2. $V(g_1) = (0, 0, 1)$ and $V(t_2) = (2, 2, 0)$ are incomparable; indeed $g_1 \parallel t_2$. $V(t_2) = (2, 2, 0) < (4, 3, 3) = V(b_4)$; indeed $t_2 \rightarrow b_4$ via $t_3, m_2, g_2, g_3, m_3, b_3$.

Solution (to Problem 2.4)**.** (a) All three suprema equal $(3, 5, 4)$. (b) $u < w$ and $v < w$; $u \parallel v$ (first coordinate favours u, second favours v). (c) Injectivity: if $V(e) = V(f)$ with e at p, the ledger lemma gives $V(f)[p] = V(e)[p] = |K_p(e)| \geq$ the index of e, so $e \in K_p(f)$: $e \rightarrow f$ or $e = f$; symmetrically $f \rightarrow e$ or $f = e$; a two-way $\rightarrow$ closes a cycle, so $e = f$. Now $V(a) \leq \sup(V(a), V(b)) = V(c)$ and $a \neq c$, so by injectivity $V(a) < V(c)$, whence $a \rightarrow c$ by Theorem 2.2; likewise $b \rightarrow c$.

Solution (to Problem 2.5)**.** X: consistent; every channel empty (m_1's send and receive both inside; nothing else sent). Y: consistent; cross-frontier mail $\{m_2\}$ on Tuppy $\rightarrow$ Gussie, all other channels empty. Z: inconsistent; offending letter m_1 (t_1 inside, b_2 outside). W: inconsistent; offending letter m_2 (g_2 inside via $\leq g_3$, while t_3 lies outside $\leq t_1$). Note m_3 is blameless in W: send g_3 and receive b_3 are both inside.

Solution (to Problem 2.6)**.** (a) Before d / before r: 60 + 40 = 100. Before d / after r: 60 + 50 = 110. After d / before r: 50 + 40 = 90. After d / after r: 50 + 50 = 100. (b) The 110 case is an *inconsistent cut*: r inside, d outside — the credit without its debit; £10 minted. The 90 case is a *consistent* cut whose cross-frontier mail (m, in flight) the telephone audit simply ignores; £10 burned. (c) Markers enforce consistency — no minting; dockets record the crossing mail — no burning. The naive audit commits whichever sin the interleaving selects; Box 6 commits neither.

Solution (to Problem 2.7)**.** (a) Adjacent events of one process are HB1-ordered and a matched send–receive pair is HB2-ordered; either contradicts $e \parallel f$. (b) Per-process subsequences are unchanged, and a deterministic machine's next step depends only on its own history and the contents of delivered messages, both unchanged; if $f = \textsc{receive}(m_f)$, its send precedes f in α and is not e, hence still precedes f in α'; all sends of

one channel issue from one process and all receives of one channel land at one process, so with $P \neq Q$ the pair contains at most one send and at most one receive of any given channel — per-channel send order and receive order are unchanged, and the FIFO pairing (the k-th receive matches the k-th send) is untouched. (c) Events, pairing, and per-process orders are unchanged, hence HB1, HB2, and their transitive closure $\rightarrow$ are unchanged; local histories likewise. (d) At any position outside the pair, the executed multiset of events and every per-process prefix coincide, so process states coincide; channel contents are the same sends minus the same receives in the same per-channel order.

Solution (to Problem 2.8)**.** (a) β extends $\rightarrow$, which contains per-process order (program order preserved) and every send-to-receive pair (availability preserved); per-channel receive order equals the receiving process's program order, preserved; the pairing is unchanged, so channel discipline holds; by determinism each process, seeing an identical local history, performs identical events. (b) Induct on the number of pairs ordered oppositely by α and β. If nonzero, some *adjacent* pair e, f of α is ordered oppositely in β; then $\neg(e \rightarrow f)$ (else β would violate $\rightarrow$) and $\neg(f \rightarrow e)$ (else α would); so $e \parallel f$, and Lemma 2.3 transposes them, reducing the count by one while preserving local histories. (c) Incomparable elements of a partial order admit linear extensions with either relative order; apply (a) and (b) to each. Since local histories are identical throughout, no process — and no client of any process — can testify to the relative order of concurrent events. It is a coordinate of the description, not of the system.

Solution (to Problem 2.9)**.** (Sketch; the hint carries the weight.) (a) Closure under HB1 is the prefix property; under HB2, the stated clause; a causal path into a member enters by one of these steps, so induction along the path gives full $\rightarrow$-closure — HB3 adds nothing. (b) Both clauses are preserved by $\cup$ and $\cap$. (c) $\downarrow e$ is a prefix at each process (transitivity) and receive-closed (HB2 $\subseteq \rightarrow$), hence consistent; any consistent $K \ni e$ is $\rightarrow$-closed by (a), hence contains $\downarrow e$. On the trace: $\downarrow g_2 = \{b_1, b_2, t_1, t_2, t_3, g_1, g_2\}$.

Solution (to Problem 2.10)**.** (a) The minting run, Exhibit 2.6. Initially Bingo £60, Tuppy £40. (1) Bingo debits £10 (live £50) and posts the first tenner. (2) Tuppy initiates: records £40; sends his marker; opens the docket on Bingo $\rightarrow$ Tuppy. (3) The first tenner arrives: Tuppy's live balance £50; docket $\langle \$10 \rangle$. (4) Tuppy's marker reaches Bingo, who records £50 — but the thrifty rule first dispatches Bertie's queued order: Bingo debits £10 (live £40) and posts the *second* tenner, and only then his marker. By FIFO the second tenner travels *ahead* of the marker. (5) It lands at Tuppy in the still-open docket: $\langle \$10, \$10 \rangle$; live £60. (6) The marker arrives; the docket closes. The audit assembles $40 + 50 + 20 = \$110$. The live total was £100 at every instant. Ten pounds minted.

(b) What dies is the *accounting*, part (ii): Box 6's marker enters the channel at the instant of recording, which is what makes "ahead of the marker" synonymous with "sent pre-recording"; the thrifty variant enqueues the marker late, the synonymy breaks, and the docket over-collects — it admits the second tenner, a *post-recording* send, so $\text{docket}(c) \supsetneq L_c(K)$. The £10 that letter carries is still inside Bingo's recorded £50; the photograph counts it twice. What survives, perhaps surprisingly, is the Key Lemma and with it consistency, part (i): the second tenner is sent post-recording *and* received post-recording (Tuppy recorded long before), so the cut K itself is unharmed. The sabotage kills the bookkeeping, not the frontier — which is precisely why every individual record looks locally honest while the ledger lies.

(c) Restore the word **immediately** (Box 6, line 4): between recording and marker emission, no application sends whatsoever. The word is the algorithm.

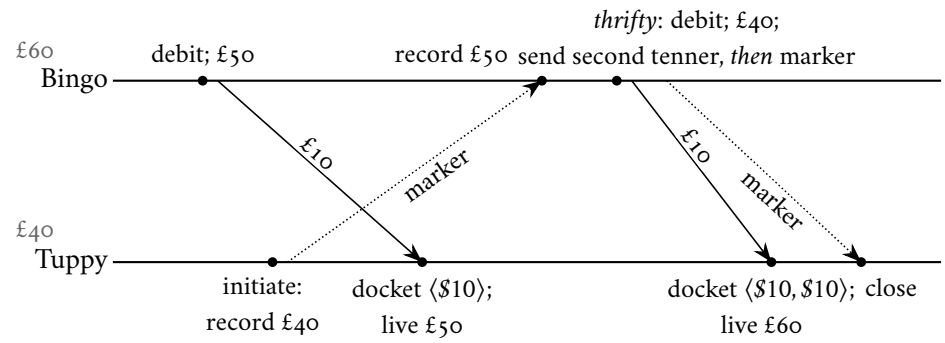


Exhibit 2.6 (the thrifty auditor's minting run; Solution 2.10). The second tenner — a post-recording send — rides ahead of the belated marker and lands in the open docket. Audit: $40 + 50 + 20 = \$110$ against a true £100. Fatality certified.

Solution (to Problem 2.11)**.** (Pointer.) (a) Fix one linear extension σ of $\rightarrow$ and, for each coordinate i, linearize events by $W(\cdot)[i]$ with ties broken by σ. If $a \rightarrow b$ then every coordinate has $W(a)[i] \leq W(b)[i]$ and σ agrees, so a precedes b in all k orders; if $a \parallel b$ then $W(a)$ and $W(b)$ disagree strictly in both directions, so some order puts a first and another b first. The intersection of the k linear orders is exactly $\rightarrow$: its order dimension is at most k. (b) Charron-Bost (1991) exhibits, for every n, an n-process computation whose causality order has dimension exactly n; hence $k \geq n$.

Solution (to Problem 2.12)**.** (Pointer; the hint is most of it.) Record before processing the first red letter: then any letter processed while white had a white sender — the pre-recording cut is receive-closed, hence consistent, and the Key Lemma has been repurchased with a bit instead of an ordering. Dockets collect white letters arriving after recording; each sender, upon recording, posts on every channel a count of the whites it sent there, and the docket closes when the count is met. FIFO's two services — separating pre from post, and announcing the last of the pre — are bought separately: a colour and an integer. Lai and Yang (1987).

2.12 The Reading

Lamport, "Time, Clocks, and the Ordering of Events in a Distributed System," Communications of the ACM, 1978. This chapter's text, and short. When the field established a prize for its most influential work, in the year 2000, this paper was the inaugural selection; the honour has since taken Dijkstra's name. Read it whole; the happens-before relation and the clock condition are its first half, the mutual-exclusion protocol its second — and when you reach the small paragraph about state machines, pause and smile: you are holding an acorn, and Chapter 5 is the oak.

Chandy and Lamport, "Distributed Snapshots: Determining Global States of Distributed Systems," ACM Transactions on Computer Systems, 1985. Brief, complete, and quietly perfect. As you read, mark the exact sentence where first-in-first-out is spent, and compare it with our Key Lemma; notice it is spent nowhere else. Their stable-property applications are Corollary 2.5 in the original tailoring.

The vector clock's two births: Fidge (1988); Mattern, "Virtual Time and Global States of Distributed Systems" (1989). Optional. The same instrument, discovered independently on opposite sides of the planet — presumably concurrently. Read either for the characterization theorem in its discoverer's own notation.

For the starred doors. Charron-Bost, "Concerning the size of logical clocks in distributed systems," Information Processing Letters, 1991 (Problem 2.11). Lai and Yang, "On distributed snapshots," Information Processing Letters, 1987 (Problem 2.12).

Standing companion: Kleppmann, chapter 8. His pages on unreliable clocks pair with Part One; his treatment of happens-before and concurrency detection, with Parts Three through Five. Assigned once for the season; here it earns its keep.

Chapter 3

What Mathematics Forbids

Companion to Episode Three. The volume's flagship, and the proof this volume was built to hold. The consensus problem, stated in three modest clauses; the adversary's formal commission — the scheduler, its powers and its leash; the valence apparatus, and the Fischer–Lynch–Paterson impossibility, written out whole, to the last inequality; the fine print, and the three gates — partial synchrony, randomization, failure detectors — through which every working system escapes; the CAP theorem, proved in Gilbert and Lynch's dress and disinfected of a decade of folklore; and the season's standing triage — impossible, expensive, merely inadvisable — installed as permanent equipment. Crashed-versus-slow, filed in Chapter 1 as the primordial nuisance, is promoted here to murder weapon.

3.1 Part One — A Modest Ask

In April of 1985 the Journal of the ACM published a paper by Michael Fischer, Nancy Lynch, and Michael Paterson under a title of a bluntness one rarely encounters in the literature: "Impossibility of Distributed Consensus with One Faulty Process." No hedging subtitle, no "towards." A death certificate, with the cause of death in the title. The paper runs to about nine printed pages; the argument that matters occupies scarcely three. It had circulated since 1983 — first presented at a symposium on database systems, which tells you who was bleeding — and by the time the journal printed it, it had done its work: a decade of protocol tinkering was over. Through the late seventies and early eighties, agreement protocols — schemes by which machines, some of which may fail, settle on a common decision — had been proposed, published, and deployed, and every one of them, under sufficiently malicious inspection, revealed a corner: some arrangement of delays and silences in which the protocol either hung forever or agreed on two different things at once. Each corner, patched, moved. None closed. Fischer, Lynch, and Paterson explained why: the corner is not in the protocol; the corner is in the problem. Every protocol that can ever be written — patched, garlanded with rounds and timeouts, designed by our betters or by machines yet unbuilt — has such a corner, and the proof needs no mathematics beyond careful bookkeeping and one idea of genuine genius. (The field's influential-paper prize, whose inaugural selection in 2000 was Lamport's paper on time, went in its second year to this one.)

Why this problem sits at the exact centre of the season's first half: every road walked so far funnels into it. Chapter 1 ended in the wreckage of the umbrella counter with the deepest question in the rubble — Bingo says one, Gussie says nought; which of them is *right*? — and found the question has no answer until an authority is appointed, by parts that fail, in a way that survives the appointee's death. Chapter 2 built order out of messages and then confessed that the better grade of order — the total order, the single timeline all replicas apply in lockstep — must be rented from a leader, leaving as a debt how a leaderless rabble elects

one; an election is an agreement about who won. Transactions, replication, configuration, membership: lift the floorboards of any of them and the same joist appears. One bit, agreed upon by machines that fail, is the load-bearing member of the entire edifice — and the bit is the atom of coordination: settle one bit reliably and you can settle a sequence of them; a sequence of settled bits is a log; a log driven through identical machines is, per the acorn paragraph of Lamport 1978, a replicated anything. Now the mathematics says the atom cannot be split — not in the world honestly assumed since Chapter 1. And mark the theorem's peculiar cruelty from the start: the proof does not use the Post's malice. No letter is lost, none forged, none duplicated. Every letter is delivered, and the theorem kills anyway — with delay alone, wielded with perfect judgment. Loss defeated two generals; mere unpunctuality defeats any number of them.

The problem itself, with no theatre, because its plainness is the point. Some number of gentlemen — our three — must jointly make a single yes-or-no decision; the question before the house is whether to adopt tomorrow's new price list. Each arrives with a private ballot — nought for the old prices, one for the new — formed from whatever local evidence he has; they may correspond through the Post as much as they please; and at some point each must write his decision in his ledger, in ink — once written, never revised. Success is three clauses. *Agreement*: no two gentlemen ink different decisions — not "rarely differ"; never, in any execution, under any weather; a shop whose branches disagree about the price list is Chapter 1's umbrella counter with stationery. *Validity*: the decision must be some gentleman's actual original opinion — in particular, a unanimous ballot prevails. The clause bars universal cowardice, kin to Coordinated Attack's validity clause: a protocol that ignores all ballots and always inks nought satisfies Agreement magnificently and decides nothing worth deciding. *Termination*: every gentleman who remains alive eventually writes. The meeting ends. This is the clause to watch — the way one watches the quietest man in the room. And one condition of the whole exercise: the three guarantees must be delivered even if *one* gentleman fails — crash-stop, in Chapter 1's vocabulary: he halts silently, at any moment of the adversary's choosing, and never returns. One chair, possibly empty, and nobody told whose or when or whether.

The possibly-empty chair is the entire difficulty of the field's central problem. Without it the protocol writes itself: every gentleman posts his ballot to every other, waits until all ballots are in hand, applies the same fixed rule, and inks the result — the Post may dawdle, but every letter arrives eventually, every gentleman eventually holds the same three ballots, and all three clauses follow trivially. With it, the committee is caught between waiting for a colleague who may never speak and proceeding without a colleague who may merely be clearing his throat: wait for all three ballots and termination dies with the emptied chair; act on a partial count and two gentlemen may act on two *different* partial counts, with agreement in the gravest peril. This is crashed-versus-slow, the primordial confusion of Chapter 1, promoted from a nuisance of operations to the exact hinge on which the possibility of agreement turns. A single bit is all that is asked; it is difficult to ask less, and that is precisely why the impossibility is a monument and not a curiosity — a proof that the humblest nontrivial thing cannot be done closes the conversation.

The problem, formally. The ballots are the inputs x_i; the ink is a write-once register.

Definition 3.1 (the consensus problem)**.** Each process p_i holds an input $x_i \in \{0, 1\}$ and has a write-once output register y_i, initially blank. A process *decides* v by writing $y_i \leftarrow v$; written ink is never revised. A protocol *solves consensus tolerating one fault* if in every admissible run (Def. 3.4):

- *(Agreement)* no two processes decide different values;
- *(Validity)* the decided value is some process's input; in particular a unanimous input is the only permissible decision;
- *(Termination)* every process that does not crash eventually decides.

FLP prove their theorem under a hypothesis weaker than Validity — *non-triviality*: both 0-deciding and 1-deciding runs exist — which makes the impossibility stronger, since any protocol satisfying Validity sat-

isfies non-triviality. We state Validity because the course does; every proof below uses it only through non-triviality except Lemma 3.2, where the difference is cosmetic (Problem 3.2).

Remark 3.2 (what Validity does not ask; coordination in the pure). Validity is the whole of the constraint on *which* value may be inked, and it is a strikingly thin one: it asks that the decision be *somebody's* input, never that it be the right input, the prudent input, or an input any process finds tolerable. The vernacular picture of "reaching agreement" — a negotiation among parties who hold private utilities and ratify a proposal only where it falls favourably against them — adds exactly such a constraint, and in the adding becomes a different and strictly harder problem, one this course does not treat. The omission is deliberate and load-bearing. Strip the value of all merit and what remains to be arranged is *coordination in the pure* — the bare securing of one common answer against crash and asynchrony — and it is coordination in the pure that Theorem 3.4 proves unattainable by guarantee. Restore the preferences and every result below only hardens: a protocol that cannot promise agreement on an arbitrary bit certainly cannot promise it on the acceptable ones. The bit is stripped naked so that its impossibility may be laid to coordination and to nothing else.

3.2 Part Two — Opposing Counsel

Every impossibility proof in this course is a game, and the opposing player is now introduced properly. Chapter 1 called it the Post's malice and promised it a formal commission; here it is. The adversary is the *scheduler* — picture not weather but counsel: opposing counsel of formidable diligence, who has read the protocol entire — every rule, every threshold, every contingency — before the game begins. There are no secrets from the scheduler, because a protocol is a published rulebook, and a guarantee must hold against every possible arrangement of delays, which is the same as holding against the cleverest one. Assuming the adversary reads the playbook is not pessimism; it is the definition of a guarantee.

The powers, and the leash. The scheduler controls timing, and timing alone: when several letters sit undelivered in the Post's bag, it chooses which lands next; when several gentlemen are ready to take a step, it chooses who moves and who stands idle, and for how long. It may hold any letter as long as it pleases, short of forever; it may starve any gentleman of turns as long as it pleases, short of forever; in the asynchronous model there is no bound it must respect and no clock it must answer to. Delay is its entire arsenal, and its aim with that one weapon is perfect. The leash is *fairness*, with two straps: every letter posted to a living gentleman is eventually delivered, and every living gentleman is eventually given his next turn, again and again, without end. The scheduler may bench Gussie for as long as it likes; it must keep letting him back into the game. It also holds one dark privilege, already announced: it may, at most once, retire a gentleman permanently — the crash — and it never files notice. But it may not lose a letter. The Post has been reformed, sir — punctuality itself, minus the punctuality. And observe what these powers amount to, because they amount to something intimately familiar: the scheduler can take any healthy gentleman and, for any finite stretch of its choosing, make him indistinguishable from a corpse — withhold his letters, deny his turns, let his colleagues' suspicions harden — and then, at the moment maximally inconvenient, produce him alive, bookwork intact, entirely innocent. Crashed-versus-slow is not merely a confusion the scheduler exploits; it is the scheduler's stock-in-trade, its entire craft, and in the proof it is promoted from operational nuisance to the murder weapon of a mathematical argument — the instalment Chapter 1's debt register marked due in this chapter, paid in full below.

The victory conditions are asymmetric, and the asymmetry is the civilization already present in the framing. Our side's play is fixed the moment the rulebook is published; every variation in how the game unfolds is the scheduler's choice and only the scheduler's. We must win every fair play, with up to one crash, all three clauses holding; counsel needs one fair play — a single one — in which some clause fails. When the industry says a system "works," it generally means no one has yet been paid to find the line of play; when

mathematics says a protocol works, it means counsel has been defeated in advance, for all lines, forever. The distance between those two sentences is the distance between testing and proof. And the fairness leash matters more than it looks: an adversary allowed to suppress letters forever is merely Chapter 1's Post, against which coordination died cheaply — two generals settled it. This theorem is played against a *leashed* adversary, and that is what makes it deep: Fischer, Lynch, and Paterson conceded to the network everything the engineer could dream of demanding — every letter delivered, every process scheduled, forever — and showed the concession does not help. When a theorem still stands after the other side has been granted everything it ever asked for, the trouble was never where you were looking.

The commission, notarized — both of the chapter's terrains fixed before any formal claim is made, as Chapter 1's discipline demands.

Model of this chapter

For the impossibility (Defs. 3.1–3.5, Theorem 3.4) — the FLP model, checked against the 1985 paper. Processes $p_1, \ldots, p_n$ ($n \geq 2$) are deterministic automata communicating by messages. The *message buffer* is a multiset of (addressee, message) pairs, with two operations: $\mathsf{send}(p, m)$ adds a pair; $\mathsf{receive}(p)$ removes and returns some message addressed to p, *or returns the special value* $\varnothing$ (null) — the choice is the adversary's. A *step* of p: one receive, then a state change and a finite set of sends, both determined by p's state and the value received. Timing: fully asynchronous — no bounds on delay or relative speed; the adversary orders all steps and deliveries. **The reformed Post:** within admissible runs (Def. 3.4) no message is lost, none duplicated, none forged; channels need not preserve order. Failures: crash-stop, *at most one* process in any run. This chapter's demonstrations use $n = 3$ (the company); the theorem holds for all $n \geq 2$.

For one bad afternoon (Theorem 3.5) — the Gilbert–Lynch setting. An asynchronous network of at least two nodes implementing a single read/write register (Def. 3.9); clients issue reads and writes to any node. A *partition* splits the nodes into two or more non-empty components; every message between components is lost while it lasts. Nodes do not fail; the network does. Consistency means linearizability (Def. 3.9, by promissory note — the full treatment is Chapter 7's); availability means every request received by any node eventually receives a response.

The vocabulary of play, exactly as the 1985 paper keeps it. A position of the game — everyone's books, plus the bag — is a *configuration*; the bag is the *message buffer* of the model box; a move, a turn, is an *event* applied to a configuration.

Definition 3.3 (configurations, events, schedules)**.** A *configuration* C comprises the local state of every process together with the message buffer. An *event* $e = (p, m)$ — the delivery to p of message m addressed to p, or of $m = \varnothing$ — is *applicable* to C iff $m = \varnothing$ or m lies in C's buffer addressed to p; applying it yields the configuration $e(C)$ in which p has taken the corresponding step. A *schedule* σ is a finite or infinite sequence of events, applied left to right; a finite σ applicable to C yields $\sigma(C)$, and every configuration so obtained is *reachable* from C. Note the standing fact (used silently throughout): an event applicable to C remains applicable at every configuration reachable from C by a schedule not containing it — undelivered mail stays in the bag.

A full unfolding of the game is a run; a fair play — the leash honoured — is an *admissible* one, and the dark privilege appears as the single permitted fault.

Definition 3.4 (runs; admissibility; deciding runs)**.** A *run* from C is a (finite or infinite) schedule applicable to C. A process is *faulty* in an infinite run if it takes only finitely many steps, *correct* otherwise. A run is

admissible if at most one process is faulty and every message addressed to a correct process is eventually delivered. A run is *deciding* if some process decides in it. Total correctness: every admissible run of the protocol is deciding — this is the form of Termination the adversary attacks.

3.3 Part Three — The Proof on the Fence

Now the set piece, in four movements: a vocabulary, an opening, a small geometric lemma, and the heart. The proof's single greatest idea is a way of seeing, and the seeing needs a word. Freeze any position of the game and ask, of the frozen position, a question about its futures: among all the ways the scheduler could legally continue, does some continuation end by inking nought? Does some end at one? If only nought-endings lie ahead, the position is committed to nought — 0-*valent*; if only one-endings, 1-*valent*; and if both futures remain alive — if the scheduler still holds the power to steer the game to either verdict — the position is *bivalent*: on the fence. Note well that valence is a fact about the position, not about anybody's knowledge of it: a chess position can be lost beyond rescue while both players still believe the game open — the loss is laid up in its futures, visible only to the tablebase. Valence is the tablebase view, and for the purposes of the proof counsel is *granted* the tablebase: opposing counsel, holding the rulebook, is permitted unlimited analysis. That is fair play in an impossibility argument, where even the strongest legal adversary must be beaten.

Definition 3.5 (valence)**.** For a configuration C reachable in a given protocol, let $V(C) \subseteq \{0, 1\}$ be the set of values decided in configurations reachable from C. C is *bivalent* if $V(C) = \{0, 1\}$, *v-valent* (committed to v) if $V(C) = \{v\}$. In a totally correct protocol $V(C) \neq \emptyset$ (some admissible continuation decides), so every configuration is exactly one of the three.

The second feature of the vocabulary is the bolt that fastens the whole proof — ink implies commitment, and therefore, said the other way round, a position on the fence is a position where no pen has yet touched any page.

Remark 3.6 (ink implies commitment; the fence has clean books)*.* In a protocol satisfying Agreement, any configuration in which some process has decided v is v-valent: a reachable decision of $1 - v$ would put two inks in one run. Contrapositively, *a bivalent configuration contains no decision anywhere* — the adversary that preserves bivalence forever thereby prevents every decision forever. Indecision is not a failure to reach the goal; indecision is the goal.

The adversary's campaign therefore has exactly two obligations: begin on the fence, and never leave it. Before either is discharged, one small tool is laid on the bench — Chapter 2's swap lemma in consensus dress. Two letters addressed to different desks may land in either order and the game does not notice: each gentleman's reaction is a deterministic function of his own books and his own letter, and neither observes the other's step in between. In Chapter 2 the transposition of merely-elsewhere events was a philosophical instrument, certifying photographs of presents that never quite happened; here the same lemma is a weapon — the *diamond*, after the shape of its picture (Exhibit 3.2 below).

Lemma 3.1 (commutation; disjoint schedules)**.** *Let σ_1, σ_2 be finite schedules applicable to C such that no process takes a step in both. Then σ_2 is applicable to $\sigma_1(C)$, σ_1 to $\sigma_2(C)$, and*

$$\sigma_2(\sigma_1(C)) = \sigma_1(\sigma_2(C)).$$

Proof of Lemma 3.1. Induct on $|\sigma_1| + |\sigma_2|$; the essential case is two single events $e_1 = (p_1, m_1)$, $e_2 = (p_2, m_2)$ with $p_1 \neq p_2$, both applicable to C. Applicability is preserved: e_1 neither consumes m_2 (messages are consumed only by their addressee, and m_2 is addressed to $p_2 \neq p_1$) nor changes p_2's state; so e_2 is applicable to

$e_1(C)$, and symmetrically. In $e_2(e_1(C))$ and $e_1(e_2(C))$: process p_1 has taken the same step from the same local state with the same delivered value in both, likewise p_2, and no other process moved; local states agree. The buffer in both equals C's buffer minus $\{m_1, m_2\}$ plus the sends of the two steps — and each step's sends are determined by its process's prior state and delivered value, identical in both orders. The two configurations are equal. For the induction step, splice one event at a time across the other schedule using the essential case. □

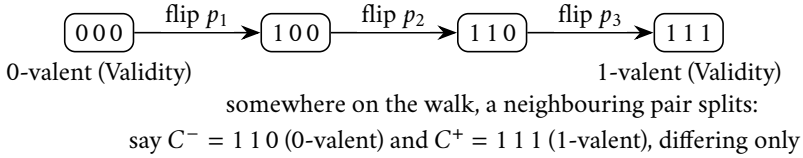
The opening, then. Does the game even begin on the fence? Not obviously: there are finitely many starting positions — one per assignment of ballots — and one might hope the protocol commits from the first whistle, its verdict a foregone function of the ballots, the correspondence a mere formality of counting. The first lemma kills the hope — skeleton: the paper-doll chain of adjacent initial configurations, plus one silenced juror — and the doppelgänger performs its first murder of the chapter doing so.

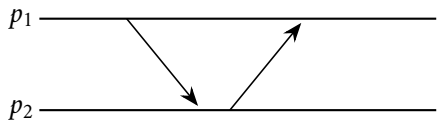
Lemma 3.2 (initial bivalence)**.** *Every protocol solving consensus tolerating one fault has a bivalent initial configuration.*

Proof of Lemma 3.2. Suppose instead that every initial configuration is univalent. By Validity, the all-zeros initial configuration is 0-valent (every reachable decision is 0) and the all-ones is 1-valent. March from all-zeros to all-ones flipping one process's input at a time — a path of n+1 initial configurations, each adjacent pair differing at a single process. The path begins at a 0-valent configuration and ends at a 1-valent one, and every configuration on it is univalent by supposition; hence some adjacent pair C^-, C^+ has C^- 0-valent and C^+ 1-valent, the two differing only in the input of a single process q.

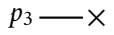
Consider any admissible deciding run from C^- in which q takes no steps — one exists, since a run in which q crashes at the outset and every message among the others is delivered is admissible, and total correctness makes it deciding. Let σ be a finite deciding prefix of its schedule; σ contains no event of q. Then σ is applicable to C^+ as well: the two initial configurations differ only in q's local input, no event of σ touches q, and by induction along σ every process other than q has identical state and identical deliveries in the two executions (the doppelgänger, mechanized). The deciding process decides the same value v in both. But $\sigma(C^-)$ is reachable from a 0-valent configuration, forcing $v = 0$, and $\sigma(C^+)$ from a 1-valent one, forcing $v = 1$. Contradiction; some initial configuration is bivalent. (Exhibit 3.1. Under bare non-triviality the endpoints are supplied differently: if all initial configurations were univalent and all of one valence, no run could ever decide the other value; so both valences occur among them, and the cube being connected, an adjacent split pair exists. The remainder is unchanged.) □

Savour the economy: the adversary has not yet used its scheduling powers at all — a row of paper dolls, one silenced juror, and the game is guaranteed to open with both verdicts alive. And note what the argument turned against us, because it will rhyme throughout the chapter: it was the *tolerance* of a crash — the constitutional requirement that the survivors carry on — that let the adversary show them two worlds through the hole where Gussie used to be. The dramas of this chapter happen in state space rather than on timelines, so its exhibits draw configurations as boxes and steps as arrows — the game-tree convention, joining the season's space-time convention (Exhibit 1.1).





the survivors' play σ:
identical from C^- and C^+,
hence identical ink — contradiction



p_3 crashed at the whistle; its input never spoke

Exhibit 3.1 (initial bivalence; the paper-doll chain and the silenced juror). Top: the walk from all-noughts to all-ones, one ballot flip per step; if every corner were committed, some adjacent pair splits. Bottom: crash the differing juror before its first step; the two worlds are doppelgängers to the survivors, who must nonetheless decide — and decide identically, contradicting the split. Lemma 3.2.

The heart of the matter — written out below to the last inequality, with the neighbour construction most retellings compress to a single sentence; Problem 3.6 rehearses that case analysis as a worked drill. First the campaign's remaining vocabulary. The withheld letter — any letter the leash is currently owed; say, one addressed to Tuppy — is the *pivot*, the event e; the single neighbouring step later found to separate a nought-committing landing from a one-committing one is the *switch*, the event e'. The claim that wins the war: from the fence, the scheduler can pay any owed letter — actually deliver the thing, fairness honoured — and land still on the fence. Feel what that does before reading it: the leash constrains *which* letters land; the fence is defended by choosing *when*. Fairness, it turns out, is no protection at all. Skeleton of the proof: assume every landing is committed; show both commitments occur among the landings; find neighbouring landings split by one step, the switch; if the switch sits at another desk, the diamond kills the split, and if at the same desk, bench that process, let the survivors play to a verdict — they must, for they cannot tell benched from dead — and commute both withheld letters across their play, delivering both futures to the *verdict position*, a configuration with ink dry on its books.

Lemma 3.3 (the pivot lemma; FLP's third)**.** *Let C be a bivalent configuration of a totally correct protocol and let $e = (p, m)$ be an event applicable to C. Let $\mathcal{C}$ be the set of configurations reachable from C by schedules not containing e, and let $\mathcal{D} = \{\, e(E) : E \in \mathcal{C} \,\}$ — the positions in which e has just landed. Then $\mathcal{D}$ contains a bivalent configuration.*

Proof of Lemma 3.3. Suppose instead that every configuration in $\mathcal{D}$ is univalent.

Step 1: $\mathcal{D}$ contains configurations of both valences. Since C is bivalent, for each $i \in \{0, 1\}$ some i-valent E_i is reachable from C. If E_i is reached by a schedule avoiding e, then $E_i \in \mathcal{C}$ and $F_i := e(E_i) \in \mathcal{D}$; F_i is reachable from the i-valent E_i, so $V(F_i) \subseteq \{i\}$, and being univalent by supposition, F_i is i-valent. Otherwise e occurs en route: write the schedule as $\sigma_1\, e\, \sigma_2$; then $F_i := e(\sigma_1(C)) \in \mathcal{D}$, and E_i is reachable from F_i, so $i \in V(F_i)$ and univalence makes F_i i-valent. Either way $\mathcal{D}$ has an i-valent member, for both i.

Step 2: neighbours. Call $E \in \mathcal{C}$ an *i-class* configuration if $e(E)$ is i-valent; by the supposition and Step 1, every member of $\mathcal{C}$ has a class and both classes are inhabited. C itself lies in some class; relabel values so that C is 0-class. Pick a 1-class $E^* \in \mathcal{C}$ and a schedule from C to E^* avoiding e; walking it, the class flips somewhere: there exist $C_0, C_1 \in \mathcal{C}$ and a single event $e' = (p', m')$ with

$$C_1 = e'(C_0), \qquad D_0 := e(C_0) \text{ is 0-valent}, \qquad D_1 := e(C_1) \text{ is 1-valent}.$$

(The walk begins 0-class and reaches 1-class; take the first flip. Problem 3.6 rehearses Steps 1–2 as a worked drill.)

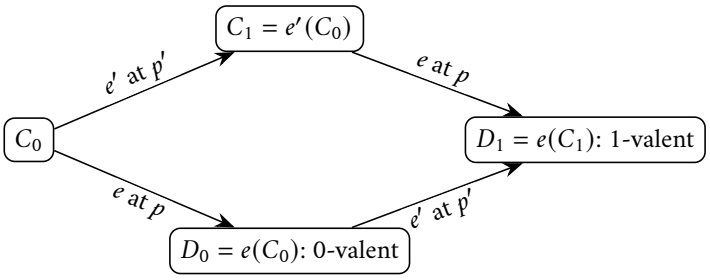
Step 3: the switch at another desk. Suppose $p' \neq p$. The single-event schedules e and e' involve distinct processes, so Lemma 3.1 gives $e'(D_0) = e'(e(C_0)) = e(e'(C_0)) = D_1$: the 1-valent D_1 is reachable from the 0-valent D_0, whence $1 \in V(D_0) = \{0\}$ — contradiction. (Exhibit 3.2.)

Step 4: the switch at the same desk. Suppose $p' = p$. Consider an admissible deciding run from C_0 in which p takes no steps (it exists: crash p at C_0, deliver everything else; total correctness decides it); let σ be a finite deciding prefix, so σ contains no event of p, and let $A = \sigma(C_0)$ — the verdict configuration; some process has decided in A, so by Remark 3.6, A is univalent. Both e and e' (and hence the two-event schedule $e'e$) involve only p, and σ involves only processes other than p; Lemma 3.1 therefore commutes them:

$$e(A) = e(\sigma(C_0)) = \sigma(e(C_0)) = \sigma(D_0), \qquad e(e'(A)) = \sigma(e(e'(C_0))) = \sigma(D_1).$$

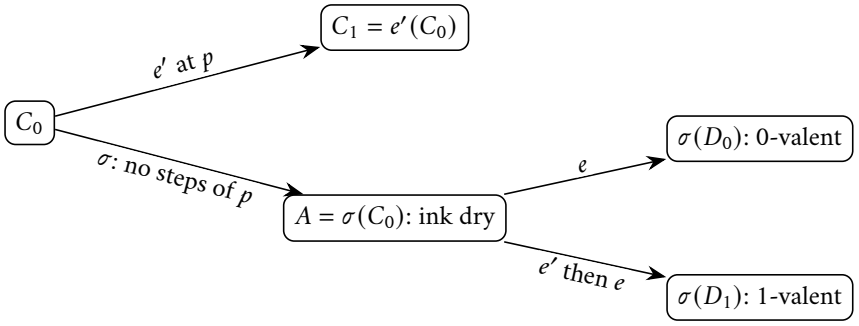
Now $\sigma(D_0)$ is reachable from the 0-valent D_0, so from A a 0-valent configuration is reachable (via e); and $\sigma(D_1)$ is reachable from the 1-valent D_1, so from A a 1-valent configuration is reachable (via e' then e). Hence $V(A) = \{0, 1\}$: the verdict configuration is bivalent — with ink dry on its books, contradicting Remark 3.6. (Exhibit 3.3.)

Both cases are absurd; the supposition falls, and $\mathcal{D}$ contains a bivalent configuration. □



$p' \neq p$: the square commutes (Lemma 3.1),
so the 1-valent D_1 is reachable from the 0-valent D_0 — absurd

Exhibit 3.2 (the diamond; the switch at another desk). Step 3 of Lemma 3.3: when the pivot e and the switch e' land at different desks, both routes around the square meet, and a committed position would have to re-open.



both futures alive at A: the verdict position is bivalent
with ink dry — contradicting Remark 3.6

Exhibit 3.3 (the switch at the same desk; the benched juror). Step 4 of Lemma 3.3: with p benched, the survivors play σ to a verdict at A — they cannot tell benched from crashed, and tolerance obliges them to finish. Both withheld letters commute across σ, delivering a 0-valent and a 1-valent future to a decided configuration.

Remark 3.7 (where each hypothesis is spent)*.* Audit the proof as Chapter 1 taught: *determinism* powers every doppelgänger and the diamond; *one-crash tolerance* manufactures the deciding run without p (Step 4) and without q (Lemma 3.2) — the fire escape the burglar uses; *asynchrony* lets the adversary withhold e and bench p for unboundedly long without breaching the leash — benched and crashed are doppelgängers to the survivors, which is Chapter 1's crashed-versus-slow, spent here as ammunition. Fairness is never breached: every withheld letter is eventually paid (Theorem 3.4).

The endgame is short, because the war is won and this is the occupation. The scheduler's full campaign assembles as the *rota* — the round-robin construction: open at a fenced position, which Lemma 3.2 guarantees; serve the gentlemen round and round forever; on each gentleman's turn, declare his oldest owed letter the pivot and land it by Lemma 3.3, the fence held. The audit of the leash, strap by strap, is inside the proof: the run is fair — it is fairness itself — and nobody ever crashes; the dark privilege stays sheathed from first move to the — but there is no last move.

Theorem 3.4 (Fischer–Lynch–Paterson, 1985)**.** *In the asynchronous model with crash-stop failures, no deterministic protocol solves consensus tolerating one fault: every protocol satisfying Agreement and Validity in all admissible runs has an admissible run — indeed one with no crash and no message loss — in which no process ever decides.*

Proof of Theorem 3.4. Suppose a protocol solves consensus tolerating one fault. By Lemma 3.2 fix a bivalent initial configuration $C_{(0)}$. Maintain the processes in a queue $p_1, \ldots, p_n$ (any initial order). Given bivalent $C_{(k)}$, stage k+1 is: let p be the process at the head of the queue, and let m be the oldest message addressed to p in $C_{(k)}$'s buffer — "oldest" by buffer entry order, ties broken arbitrarily — or $m = \varnothing$ if none. Set $e = (p, m)$: an applicable event. By Lemma 3.3 (with $C = C_{(k)}$) some bivalent $D \in \mathcal{D}$ exists: that is, there is a finite schedule σ_k avoiding e, followed by e itself, with $C_{(k+1)} := e(\sigma_k(C_{(k)}))$ bivalent. Apply it; move p to the tail of the queue.

The resulting infinite run is admissible with *zero* faults: every process heads the queue infinitely often and takes a step at each of its stages (and possibly within the σ_k's), so all processes are correct; and every message is eventually delivered — once posted, a message addressed to p has only finitely many predecessors in the buffer, each stage at p's head consumes its oldest pending message, and p heads the queue every n stages, so the message's seniority strictly improves until it is served. Yet every $C_{(k)}$ is bivalent — and no configuration anywhere in the run, the σ_k-interiors included, contains a decision: had some process decided at a configuration X of the run, X would be univalent (Remark 3.6), every configuration after X would satisfy $V \subseteq V(X)$, and the next $C_{(k)}$ — bivalent, and reachable from X — could not exist. So no process ever decides: Termination fails in an admissible run with no crash and no loss. No such protocol exists. □

Stand in the ruins a moment, as on the hill with the generals, for the proof has a property worth carrying out of the room. Ask what the adversary actually *did*. It lost nothing — every letter arrived. It forged nothing, duplicated nothing, crashed no one. Its entire output, over an infinite game, was a sequence of perfectly legal decisions about *when* — this letter now, that gentleman next. And yet it was the crash that did the killing: not a crash that happened — the crash that *might*. Twice, at the opening and in the heart's same-desk case, the argument reached for the same lever: the protocol is required to survive a silent chair, therefore the survivors must be prepared to finish without any given colleague, therefore the scheduler can bench that colleague and *borrow* the survivors' preparedness — march them down the very corridor built for the emergency, then produce the missing man alive and use the corridor's existence to spring the trap. The tolerance is the vulnerability; the fire escape is how the burglar gets in. Everything that proceeds on silence can be led by silence — hesitation wielded as a weapon, the threat sufficing where the deed is never done. I know of nothing else in computing quite like this argument: three pages that took the field's fondest ambition, granted it every concession it had ever asked of the network, and defeated it with the one thing

no message-passing system can buy at any price — an answer to the question *is he dead, or merely slow?* The confusion filed in Chapter 1 as the primordial nuisance turns out to be a conservation law. The corner the decade of tinkerers kept moving was this corner. It cannot close, because it is load-bearing.

3.4 Part Four — The Fine Print

An impossibility theorem misremembered is worse than none — Chapter 1's rule — and this theorem is misremembered for a living. What exactly was proven: in the asynchronous model, no *deterministic* protocol can *guarantee* agreement, validity, and *termination* for even a single bit, in every fair execution, while tolerating even one crash-stop failure. Each emphasized word is load-bearing, and each is a hinge on which Part Five's gates turn.

Termination is the casualty — the only casualty; nothing in the killing run touches Agreement or Validity, for it never lets anyone decide at all. The whiteboard phrasing, the single most practical sentence of the chapter: *safety is free; liveness is the hostage*. There exist protocols — sir operates several — that never, in any execution, under any weather, ink two different verdicts: safety unconditional, held through arbitrary asynchrony, through crashes, through the scheduler's worst. What no protocol can do is *also* promise the meeting always ends. The parliament that cannot be made to lie can still be made to filibuster. Chapter 4 will exhibit the division of the estate made explicit in a protocol's very structure: a safety argument invoking no clocks, no timeouts, no luck, and a liveness section that begins, in effect, "assuming the weather improves" — not an engineering apology but FLP, cited as fine print. *Deterministic*: the scheduler's campaign was a tablebase strategy — it steered toward the fence because, holding the rulebook, it could compute where the fence was; put a coin in any gentleman's pocket and the tablebase clouds over — not yet a solution, but a doorway, with Ben-Or standing in it (Part Five). *One faulty process*: the title's modesty, and its menace — no lying, no forging, no Byzantine aunts; one possible crash-stop, the gentlest failure in the taxonomy, suffices to poison the well. And mind the grammar of "possible": in the killing run itself nobody crashes; it is the protocol's *obligation* to survive a crash — the fire escape it is required to build — that the adversary turns. A protocol for machines guaranteed immortal escapes the theorem trivially; a protocol for the actual world, where the guarantee cannot be had, walks into it. Finally, the reformed Post — the detail the folklore always drops: no loss. Two Generals was an impossibility of *loss*: kill a letter and knowledge cannot become common. FLP is an impossibility of *delay*: deliver every letter, and mere freedom about when still forbids guaranteed agreement. Two different theorems about two different liberties of the Post, bracketing the field's misery from both ends — an engineer who remembers only "networks are unreliable" has learned half the lesson, for the reliable network with no latency bound is also a theorem-grade adversary.

The other side of the ledger — what was *not* proven — is a standing hazard to good engineering. The theorem does not say consensus is impossible in practice; it does not say the etcd cluster is broken, or that its designers evaded mathematics by marketing. Real networks, whose delays are noisy rather than malicious, wander off the razor's edge of the killing schedule almost immediately — disturb it at any of infinitely many points, one letter landing a moment early, one timeout misfiring in our favour, and the parliament falls off the fence and decides — which is why the clusters decide millions of times a day. What the theorem forbids is a certain sentence ever being true: "this system is *guaranteed* to decide, whatever the timing." A vendor page containing that sentence about an asynchronous deterministic system is selling a theorem violation, and one is entitled — Chapter 1's triage — to ask which word is being quietly redefined: the model, the guarantee, or the price. The scope, in its formal frame:

Remark 3.8 (scope; what is and is not forbidden). The casualty is Termination alone: the killing run of Theorem 3.4 never violates Agreement or Validity — it never lets anyone decide. Protocols exist whose safety is unconditional under arbitrary asynchrony (Chapter 4 exhibits one); what none can add is a termination

guarantee. Each italicized hypothesis is load-bearing and is a gate: *asynchronous* (under partial synchrony consensus is solvable with safety always and termination after stabilization — Dwork–Lynch–Stockmeyer 1988; Chapter 4); *deterministic* (Ben-Or 1983 terminates with probability one — Box 7, Problem 3.10); and the *interface* (an eventually-accurate suspicion oracle restores solvability with a correct majority — Chandra–Toueg 1996; the weakest such oracle is the eventual leader, Chandra–Hadzilacos–Toueg 1996, stated here with respect and proved nowhere in this course). The theorem does not predict routine failure: the killing schedule is a single tablebase thread, and real timing noise departs from it almost surely; its production shadow is livelock — the election storm — not deadlock.

3.5 Part Five — The Loopholes

A good impossibility theorem is not a wall but a survey: it tells you precisely where the fence runs, and therefore precisely where the gates must be. The emphasized words of Part Four were the survey markers — *asynchronous*, *deterministic*, *guarantee* — three words, three gates, and the entire practising industry files through them daily. Chapter after chapter from here forward is spent on the far side of one or another; the gate is stamped in the passport at each crossing.

Gate one: renegotiate *asynchronous* — partial synchrony. The fence was surveyed in the model where delay is boundless and the scheduler's aim eternal; recall instead Chapter 1's middle terrain, articulated by Cynthia Dwork, Nancy Lynch — the same Lynch, fencing from both sides, having proven the impossibility she now helps to escape — and Larry Stockmeyer: "Consensus in the Presence of Partial Synchrony," Journal of the ACM, 1988. Its two readings are known from Definition 1.2: bounds exist but are unknown, or hold only after some unknown stabilization time. Its consensus finding is the industry's charter: in partial synchrony, consensus is solvable — with safety holding *unconditionally*, through the wild weather, and termination guaranteed once the weather settles. That is the only shape the mathematics permits: conditional exactly where FLP says it must be, unconditional exactly where safety is free. Every timeout in every consensus deployment is this gate in miniature — a wager that the bound, whatever it is, has probably been reached; every adaptive timeout, doubling patiently, is a protocol groping for the unknown bound from below. The industry does not defeat FLP; it rents a better model by the millisecond and keeps the receipts. Chapter 4 lives entirely inside this gate.

Gate two: renegotiate *deterministic* — randomization. 1983, the same year FLP itself was first presented: Michael Ben-Or, in a conference paper of a handful of pages — this field's monuments are short; suspect the long ones — titled "Another Advantage of Free Choice." The advantage in question: a coin the scheduler cannot read. The protocol proceeds in rounds of votes; when a round's votes are decisive, processes adopt and eventually ink; when they are not, each flips a private coin for a fresh opinion and tries again. The scheduler still controls all timing — but the fence campaign required *steering*, steering requires forecasting reactions, and a coin has no forecast: the tablebase is blind one move ahead, forever. The price, stated with equal honesty: termination only *with probability one* — certainty of a new and weaker currency; the meeting ends almost surely, no fixed deadline can be promised, and with naive private coins the expected wait grows savage as the parliament grows — exponential rounds in the worst arrangements, repaired to civilized figures by coins shared among the company. The gate is real, it is beautiful, and the industry uses it less than the theory admires it, mostly because gate one is cheaper and the tooling won; but mark it as the answer to a precise question — it is the *deterministic* in FLP doing exact work. One box in this chapter, and deliberately from the far side of this gate: the chapter's theorems forbid; the box is the earliest famous escape.

Protocol — Box 7. Ben-Or's randomized consensus (1983), crash-fault form

n processes, at most t crashes, $2t < n$; each process p holds estimate $x \in \{0, 1\}$. All messages are broadcast to all (including the sender). Round $r = 1, 2, \ldots$ at each process:

1: **phase one:** broadcast $\langle \textsc{report}, r, x \rangle$
2: wait for $n - t$ messages $\langle \textsc{report}, r, * \rangle$
3: **if** more than $n/2$ of *all* n possible reports seen carry the same v:
4: broadcast $\langle \textsc{proposal}, r, v \rangle$
5: **else** broadcast $\langle \textsc{proposal}, r, ? \rangle$
6: **phase two:** wait for $n - t$ messages $\langle \textsc{proposal}, r, * \rangle$
7: **if** at least $t + 1$ of them carry the same value $v \neq ?$: decide v
8: **if** at least one of them carries some $v \neq ?$: $x \leftarrow v$
9: **else** $x \leftarrow$ fair coin flip
10: proceed to round $r + 1$ (deciders too: participation continues, which is what lets laggards catch up; practical variants stop after one further round)

Checked against Ben-Or (1983); the guard on line 3 is a majority of n, not of the $n - t$ received — this is what makes two different proposals in one round impossible (two majorities of n share a reporter: the crowded-club lemma, making its first appearance a chapter early). Safety is deterministic; only *waiting time* is random. Problem 3.10 walks safety, validity, and the almost-sure termination argument, and names honestly which adversary the elementary argument defeats.

Gate three: renegotiate the *interface* — failure detectors, the subtlest gate and, for the trade, the most clarifying. 1996, Journal of the ACM, Tushar Chandra and Sam Toueg: instead of changing the timing model wholesale, package the timing assumption behind an interface. Give each gentleman one additional instrument — a *suspicion service*, a little oracle at his elbow maintaining a list of colleagues it currently suspects of being dead. Formally: an unreliable failure detector, and the service may be wrong — that is the entire point. It may slander the living; it may, for a while, acquit the dead; it may change its mind hourly. Chandra and Toueg's question: how much *eventual* honesty must such an oracle deliver for consensus to become solvable? Their celebrated answer — one of a family, and the family's weakest useful member: it suffices that eventually, some one correct gentleman is never again suspected by any survivor — that, plus a majority of the parliament alive, and consensus is solvable. All the slander in the world, forgiven, so long as the service eventually settles on trusting somebody trustworthy. The companion result — Chandra with Vassos Hadzilacos and Toueg, same journal, same year — closed the question from below: the *weakest* oracle that suffices is exactly an eventual-leader service, one that eventually points every survivor at the same living gentleman. The theorem is stated here with respect and its proof declined — genuinely beyond this course; the honesty clause, exercised. But the abstraction must not be filed as theoretical: a suspicion service is a timeout with its epistemology showing. Every health check in the fleet, every phi-accrual dial, every lease expiry is an implementation of an unreliable failure detector; the theory's contribution is the specification — how little accuracy suffices, and what the protocol may lawfully do with slanderous evidence. Chapter 5 arms the detectors with leases and fences; Chapter 11 prices their false positives in retry storms.

Three gates, and one observation to bind them, worth a pin through the whole Part: all three purchase the same commodity by different contracts. Partial synchrony buys it with time — eventually the network calms and a leader's authority sticks. Randomization buys it with luck — eventually the coins align. Detectors buy it with delegated suspicion — eventually the oracle settles on a leader all can follow. In each case the purchased good is a *stretch of settled asymmetry*: some interval in which one distinguished gentleman is, and is believed to be, in charge for long enough to shepherd a decision through. That is the commodity

FLP proves cannot be manufactured from asynchronous determinism alone — and Chapter 4's Synod is precisely the art of acquiring it gracefully.

3.6 Part Six — One Bad Afternoon

A change of theorems now, though not of technique: from the profound impossibility to the famous one — from the theorem every engineer should know and few can state, to the theorem every engineer can state and few state correctly. It concerns not eternity but one bad afternoon — formally, an execution containing a partition. The history, dates exact: in July 2000, at the ACM's Symposium on Principles of Distributed Computing — the field's own parliament — Eric Brewer of Berkeley delivered an invited keynote proposing a rule of thumb from a decade of building web systems: of three desirable properties — consistency, availability, and tolerance of network partitions — a system may have at most two. A conjecture, offered as engineering folk-wisdom in the honourable sense. In 2002 Seth Gilbert and Nancy Lynch — the same Nancy Lynch a third time in this chapter; she fences on every piste in this field — published a short formalization and proof in SIGACT News, and the conjecture became the CAP theorem: Brewer's christening, Gilbert and Lynch's mathematics.

A theorem is its definitions, and every abuse of this one begins with a blurred letter. *C*, consistency, means something quite specific: linearizability — not "consistent" as the transactions department uses it, not "the data satisfies my invariants," not eventual anything. The replicated service behaves, observably, as if there were a single copy of the data, applying operations one at a time, respecting real time — every read returns the latest completed write, full stop. The strictest promise on the shelf, wearing the field's most abused word; defined here by promissory note, and read its full legal rights in Chapter 7. *A*, availability, is *total*: every request that arrives at any non-failed node eventually receives a meaningful response — attend to the quantifiers. If one lonely node on the wrong side of a partition receives a read and cannot answer it, availability in the theorem's exacting sense is forfeit. The marketing department's A is a fraction with some nines in it; the two share a letter and nothing else. *P*, partition tolerance, means the guarantee in question survives the Post severing itself — provinces isolated, every letter between them lost while the outage lasts. And here mark the asymmetry that undoes most recitations: C and A are goods you promise; P is weather. Chapter 1 entered the documentary record — the Bailis–Kingsbury catalogue, the correlated redundancy, the sharks; partitions are not a menu item you decline but a thing the world does to you, and "choosing" partition intolerance is choosing to have no promise for one of the afternoons that will certainly come.

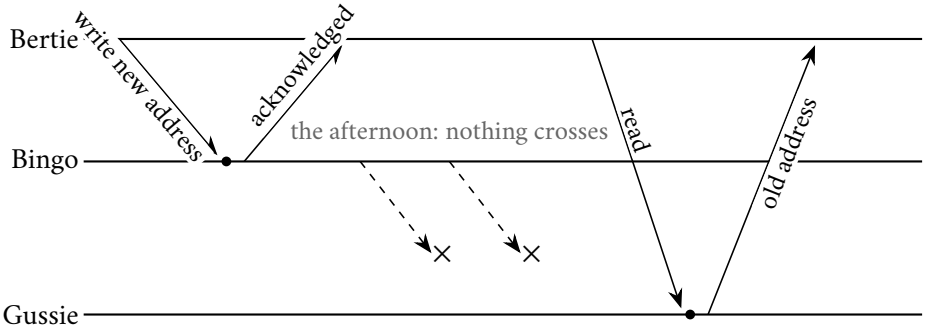
Definition 3.9 (the register service; linearizability; availability)**.** A *read/write register service* accepts `write`(v) and `read`() requests at any node, each eventually returning a response (an acknowledgment; a value). An execution of the service is *linearizable* (atomic) if there is a total order on the completed (and a subset of the pending) operations, consistent with real time — each operation ordered somewhere between its invocation and its response — under which every read returns the value of the latest preceding write. (This suffices for now; Chapter 7 gives the definition its full formal apparatus and its locality theorem.) The service is *available* if every request received by any node eventually receives a response.

The theorem, and the proof, with the company's own cast — Bertie's write completing on Bingo's side of the silence, his read answered on Gussie's. The skeleton is an old friend, the doppelgänger's fourth appearance in this chapter: availability forces both a completed write on one side and an answered read on the other; no information crossed; indistinguishability forces the stale answer. Choose your betrayal — the course's name for the consistency-or-availability dichotomy under partition.

Theorem 3.5 (Brewer's conjecture, proved; Gilbert–Lynch 2002)**.** *In the asynchronous network model, no implementation of a read/write register service guarantees both linearizability and availability in all executions, including those in which the network partitions.*

Proof of Theorem 3.5. Suppose an implementation guarantees both properties. Partition the nodes into non-empty components G_1 and G_2; from time t_0 every message between them is lost. Let the register's value at t_0 be v_0. A client submits $\mathsf{write}(v_1)$, $v_1 \neq v_0$, to a node in G_1 at t_0. By availability the write completes with an acknowledgment at some finite time $t_1 > t_0$; call this execution α_1. After t_1, a client submits $\mathsf{read}()$ to a node in G_2; by availability it returns a value at some finite $t_2 > t_1$.

Compare with the execution α_0 that is identical within G_2 — same initial states, same intra-G_2 traffic, same timing — but in which the write was never issued. Since no message crosses the partition in either execution, every node of G_2 has the same view in α_1 as in α_0 (induction on G_2's events: initial states equal, deliveries equal); the read therefore returns the same value in both. In α_0, linearizability forces the read to return v_0 (no other write exists). Hence in α_1 the read returns v_0 as well — but in α_1 the write of v_1 completed at $t_1 < t_2$, so every linearization must order it before the read, which must therefore return $v_1 \neq v_0$. Linearizability fails in α_1; the two guarantees cannot coexist. (Exhibit 3.4. Gilbert and Lynch run the same knife in a partially synchronous model, where it cuts equally; see the Reading.) □



to Gussie's province this world is the writeless world;
identical observations compel the identical (stale) answer

Exhibit 3.4 (one bad afternoon; Theorem 3.5). The write completes on Bingo's side (availability); the read answers on Gussie's side (availability); no letter crosses the severed Post, so the read returns the old value and linearizability dies — or Gussie stays silent and availability dies. Choose your betrayal.

The mathematics is honestly this small — it fit on a keynote slide in 2000 — which makes the subsequent decade of abuse the more instructive; the folklore is prosecuted exhibit by exhibit. Abuse the first: "choose two of three," recited as if C, A, and P were three dishes and every architect picks a pair — menu fraud, twice over. P is weather: it is not chosen, it arrives; a "CA system" is a system with no stated behaviour for an afternoon that is certainly coming, which is not a category but a confession — an unstated model, in the sense of Chapter 1's discipline. And in fair weather the theorem is silent entirely: nothing in Gilbert and Lynch prevents a well-built system from being consistent *and* available on every ordinary day, and the good ones are. The honest sentence is smaller and sharper: *when* a partition strikes, and only then, each affected operation must betray either consistency or availability. Its citation as a universal design trilemma is heraldry, not law. Abuse the second: the labels. An industry of taxonomists arose, stamping each database CP or AP, as though the theorem issued licence plates; but the letters are extremes — C is full linearizability, A is every-node-answers — and real systems are neither pole: configurable per operation, quorum-tunable, differently behaved for reads and writes; a system may serve linearizable writes and stale-tolerant reads in

the same breath, and most do. The label compresses a contract worth reading into a syllable worth nothing — Kleppmann's 2015 plea, assigned in the Reading: stop calling databases CP or AP. When a vendor page says "CP," the diligent question is Chapter 1's: which failures, which guarantee, at which price — show me the afternoon in the documentation. Abuse the third, subtler, and the one whose correction earns its keep in design reviews: treating the partition dichotomy as binary when it is a dial. In 1999, before the conjecture was even conjectured publicly, Armando Fox and the same Brewer published "Harvest, Yield, and Scalable Tolerant Systems," the grown-up vocabulary for degradation: *yield*, the fraction of requests answered — the answer rate; *harvest*, the fraction of the relevant data each answer reflects — the answer's completeness. The partitioned search service answering every query from the reachable half of its index has kept its yield and halved its harvest — every client served, every answer honest about being partial; the banking service refusing writes on the minority side has spent yield to keep harvest whole. Between "consistent" and "available" runs a whole negotiated frontier — bounds on staleness, explicit partial-results flags, locally accepted writes with declared reconciliation debts — and systems that degrade *well* are precisely systems whose designers chose a point on the frontier deliberately, in peacetime, and wrote it down. CAP names the frontier's endpoints; harvest and yield survey the middle, where the actual engineering lives (Problem 3.7 makes both formal). And abuse the fourth — the omission, the one that costs the most money: partition-fixation, the inference that a sturdy network makes strong consistency free. The corrective is *PACELC* (Abadi, IEEE Computer, 2012): if Partition, then the trade is Availability versus Consistency — CAP, conceded; Else — in ordinary weather, which is most weather — the trade is Latency versus Consistency. For consider what one-copy behaviour costs on a perfectly healthy network: coordination. The write that must be seen by every subsequent read everywhere must reach agreement machinery — a quorum's round trip, a leader's confirmation — before acknowledgment; the toll is collected on *every* write, on the best afternoon of the year, in the currency users actually feel. That sentence explains at a stroke the entire respectable half of the eventual-consistency industry: geo-replicated systems relax consistency not mainly because partitions terrify them but because the speed of light bills them daily — Chapter 1 priced Toronto to Sydney at a sixth of a second, and no theorem is required to find that expensive, merely a profit-and-loss statement. Chapter 8 pays the bill in the other direction: consistency across continents purchased with better physics, seven milliseconds of deliberate waiting at a time. CAP is the afternoon the network betrays you; PACELC is every day, including that one. The formal middle ground:

Remark 3.10 (the partially synchronous variant; the middle ground). Gilbert and Lynch prove further that the impossibility survives partial synchrony — clocks and bounded delays do not rescue both properties through a partition — and that weakened contracts become achievable there (their *t-connected consistency*: stale reads permitted during partitions, reconciliation obligatory after). Between the poles runs a negotiated frontier: bounded-staleness answers, explicit partial results, locally accepted writes with declared reconciliation debts. Harvest and yield (Fox–Brewer 1999; Problem 3.7) are its surveying instruments, and PACELC (Abadi 2012) its peacetime ledger: if Partition, Availability versus Consistency; Else, Latency versus Consistency — the coordination toll is collected on every write on the best afternoon of the year.

The pattern, one more time, because by Chapter 8 the footsteps should be audible in the first paragraph: the doppelgänger felled consensus in the grand theorem and felled the two-out-of-three dream in the small one, with the identical thrust — two worlds, one observer, no telling them apart. The master key, as advertised; it returns wearing a notary's collar to convict two-phase commit.

3.7 Part Seven — The Triage

Two theorems in this chapter, of very different weights, both routinely miscited — the correct moment, then, to install the course's standing triage, promised in Chapter 1. When any claim of the form "that cannot

be done" crosses the desk — in a design review, a vendor negotiation, a postmortem, an architecture council — it goes into one of three bins, and the sorting is the most consequential single habit this course will leave behind.

Bin one: *impossible*. A theorem applies — this model, this guarantee, this exact ask — and the answer is no, permanently, for everyone, including the vendor whose deck says otherwise. Guaranteed-terminating deterministic consensus in pure asynchrony with a possible crash: bin one, by Theorem 3.4. Linearizability with every-node availability through a partition: bin one, by Theorem 3.5. Certainty of common action over a channel that may lose the last letter: bin one, by Theorem 1.1. The correct response to bin one is never effort; it is renegotiation — change the model (buy synchrony, buy detectors), change the guarantee (probability one, eventual, bounded-stale), or change the ask. The bin's discipline cuts both ways, mind: to *place* a claim here one must produce the theorem and match its model to the world, clause by clause — Chapter 1's rule that a theorem quoted without its model is a noise. Most things called impossible in industry are not in this bin.

Bin two: *expensive*. Possible, proven so, shipping in production — and carrying a price tag that arrives on schedule, forever. Consensus under partial synchrony: possible, at the price of timeout tuning, election storms in bad weather, and a liveness clause that reads "when the network settles." Linearizability across a continent: possible, at a round trip — or at seven milliseconds of purchased physics — per write, every write, peacetime included; PACELC's E-clause is this bin's ledger. Exactly-once *effect*: possible (Chapter 10), at the price of idempotence engineering throughout. The correct response is neither refusal nor heroism; it is an invoice: name the recurring cost, decide who pays it, and write both down. Most of the season from Chapter 4 onward is a walking tour of this bin — the honest accounting of how the impossible's neighbours are made routine, at what price, in which model.

Bin three: *merely inadvisable*. Possible, affordable — and unwise, for this workload, this year. Consensus on the critical path of every read when a lease would do. Strong consistency for a cache whose staleness nobody can observe. A cross-region transaction where a schema change would dissolve the need — a proverb in due course. This is the bin of judgment, and it has a diagnostic all its own: bins one and two are properties of mathematics and markets; bin three is a property of the *requirements*, and it is the only bin whose contents change when the business does. The senior failure mode is misfiling between two and three — paying bin-two invoices out of habit for guarantees the product stopped needing, or declaring bin-three inconveniences "impossible" to end a meeting. Impossible, expensive, merely inadvisable: which bin, on whose theorem or whose invoice or whose judgment — and the conversation that follows is shorter and considerably more honest than the one it replaces.

3.8 The Whiteboard

For the design review.

1. State the model before citing the impossibility — either impossibility, any impossibility. FLP names its terms: asynchronous, deterministic, one crash tolerated, guaranteed termination. CAP names its terms: a partition in progress, total availability, linearizability. Quote the theorem with its terms attached or not at all; a three-letter incantation without its model is heraldry, and design reviews are not the College of Arms.

2. Safety is free; liveness is the hostage. Interrogate any consensus-bearing design at exactly the joint the theorem names: what does it do *while it cannot decide*? What do clients see during the election storm, and what is the story when the storm outlives the deadline? A design with a crisp answer has read its FLP; a design that answers "that won't happen" has not, and the scheduler is patient.

3. CAP is a theorem about one bad afternoon; PACELC is about every day. Budget both, separately, in writing: the partition posture — which operations betray consistency and which betray availability when the Post severs, chosen in peacetime, tested on schedule — and the peacetime toll — what every write pays for the selected consistency level, in milliseconds, on the invoice the users can feel. A review that discusses only the afternoon has priced the flood and forgotten the rent; between the poles, degrade deliberately, with the dials that have names: harvest and yield.

4. Triage every impossibility claim into its bin — impossible / expensive / merely inadvisable — and demand the bin's paperwork: a theorem with matching model for the first; a recurring invoice with a named payer for the second; a requirements argument, dated this fiscal year, for the third.

The register. Instalment paid: debt (2) of Chapter 1, crashed-versus-slow — managed, never cured — received its formal apparatus (the scheduler) and its promotion to murder weapon; the debt is not retired — it cannot be retired; that is rather the theorem — but it is henceforth managed by mathematics instead of anecdote, with instalments remaining in Chapter 5 (leases, fences) and Chapter 11 (pricing false suspicion). Debts issued: (1) partial synchrony, surveyed as gate one — cashed in Chapter 4, when the Synod moves in and furnishes it; (2) the suspicion services of gate three — machinery in Chapter 5, price in Chapter 11; (3) CAP's C, linearizability, defined by gesture and promissory note — read its full legal rights in Chapter 7.

3.9 Problems

Tier I — Calisthenics

Problem 3.1 (valence drill)**.** A toy protocol's reachable configurations form the finite game tree below: arrows are the adversary's available moves; leaves are deciding configurations labelled with their decision.

Node	Successors (or decision)
A	B, C
B	D, E
C	E, F
D	decides 0
E	G, H
F	decides 1
G	decides 0
H	decides 0

(a) Classify every node as 0-valent, 1-valent, or bivalent. (b) List every *critical* move: an arrow from a bivalent node to a univalent one. (c) At which nodes could a process have already decided, consistently with Remark 3.6? (d) If the adversary plays to avoid decisions forever, where does it get stuck, and why does no such trap exist for Theorem 3.4's adversary?

Problem 3.2 (the fence walk)**.** Three processes, binary inputs: eight initial configurations. A colleague claims a correct one-fault-tolerant protocol may commit at the whistle, with the valence of each initial configuration equal to the *majority* of the three inputs. (a) Exhibit the walk of Lemma 3.2 for this claim and point to a splitting pair. (b) Describe the crash execution that refutes the claim at that pair, naming who is crashed and what the survivors see. (c) Show the same refutation works for *any* committed-at-the-whistle claim, not just majority. (d) Where exactly did (b) use one-fault tolerance? (e) Rewrite the endpoints of the walk under bare non-triviality (Def. 3.1), without Validity.

Problem 3.3 (the leash, audited)**.** Three processes; classify each infinite run as admissible or not, and if not, name the violated clause (Def. 3.4): (a) the rota of Theorem 3.4's proof; (b) a run in which one message to a correct process is withheld forever while its addressee takes infinitely many null steps; (c) a run in which p_3 takes no steps after some point and p_1 crashes; (d) a run in which every message is delivered but p_2, alive, is given only finitely many steps; (e) a run in which p_3 crashes and every message addressed to p_1 and p_2 is eventually delivered.

Problem 3.4 (one afternoon, catalogued)**.** Five nodes; a partition isolates $\{D, E\}$ from $\{A, B, C\}$ for one hour. During the hour: a write of v_2 (over v_1) completes at A; a read at B returns v_2; a read at D returns v_1; a write at E is refused with an error. (a) Which responses, if any, violate linearizability? (b) Which behaviours forfeit availability in Gilbert–Lynch's total sense? (c) Classify the system's evident posture on each side of the partition. (d) Compute the hour's yield if the four operations above were the only requests, and the harvest of the read at D if the register's ideal answer reflects all completed writes.

Problem 3.5 (triage drill)**.** File each claim in its bin — impossible / expensive / merely inadvisable — citing the theorem (with model match), the recurring invoice, or the requirements argument: (a) "our asynchronous replicated store guarantees failover within one second, always"; (b) "clients in every region read their nearest replica, linearizably, throughout partitions"; (c) "all writes are strictly serializable across three continents"; (d) "every cache read goes through consensus for freshness"; (e) "our queue delivers each message exactly once, guaranteed"; (f) "our queue's consumers process each message exactly once, effectively."

Tier II — The Real Thing

Problem 3.6 (the compressed case, completed)**.** Prove Steps 1 and 2 of Lemma 3.3 in full, from Definitions 3.3–3.5: (a) $\mathcal{D}$ contains members of both valences (mind both sub-cases: e avoided, e en route); (b) the class walk yields neighbours $C_1 = e'(C_0)$ with $D_i = e(C_i)$ i-valent; justify the relabeling that lets the walk start in class 0. (c) State precisely where (a) uses the standing fact that e remains applicable throughout C. *(Full solution.)*

Problem 3.7 (harvest and yield, formalized)**.** Following Fox and Brewer: for an execution window, *yield* is the fraction of issued requests that receive responses; for a query whose ideal answer aggregates a data set S, the *harvest* of a response is the fraction of S it reflects. (a) Formalize both for a service sharded over n nodes, data uniform, a partition leaving k of n shards reachable from the serving side. (b) Show a service that always answers achieves yield 1 with harvest k/n on aggregate queries, and that any deterministic always-answering service admits a query whose harvest is at most k/n. (c) Show that enforcing a harvest floor $h > k/n$ costs yield: the requests that cannot be served at floor h during the partition are exactly the aggregate queries, and compute the resulting yield for a workload that is a fraction q aggregates. (d) Conclude with the design sentence: harvest and yield are the two currencies in which the partitioned afternoon may be paid, and the exchange rate is the workload mix.

Problem 3.8 (agreement among the dead)**.** *Uniform agreement* strengthens Agreement: no two processes — crashed later or not — ever decide differently. (a) Exhibit a protocol and an admissible execution in which Agreement (among the correct) holds but uniform agreement fails: a process decides, externalizes nothing, and crashes; the survivors decide otherwise. (b) Explain in one paragraph why the distinction has a body count in practice: what may a process do with its decision *before* crashing? (c) Show Theorem 3.4 applies a fortiori to uniform consensus. (d) In the *synchronous* crash model both variants are solvable; look up (or conjecture) whether they cost the same in rounds, and record the moral: uniformity is a real good with a real price. *(Hint provided; solution sketches (a)–(c).)*

Problem 3.9 (sabotage: the impatient tribunal)**.** A well-meaning colleague, having read Part Five, designs "consensus" for three processes with a rotating chairman and timeouts standing in for a failure detector. Round r has chairman $p_{(r \bmod 3)+1}$. Rules, at each process, current estimate x:

- *Chairman of round r:* broadcast $\langle \textsc{propose}, r, x \rangle$; on receiving $\langle \textsc{ack}, r \rangle$ from at least one other process (a majority of three with himself), decide x and broadcast $\langle \textsc{decide}, x \rangle$.
- *Any process, on $\langle \textsc{propose}, r, v \rangle$ in its current round r:* set $x \leftarrow v$; reply $\langle \textsc{ack}, r \rangle$ to the chairman.
- *Any process, on $\langle \textsc{decide}, v \rangle$:* decide v.
- *Timeouts:* a non-chairman waiting in round r for a proposal, or a process that has acknowledged but seen no decision, advances on timeout to round $r + 1$, keeping its estimate. The chairman likewise advances if his acknowledgment is slow.
- Stale-round messages (r less than the recipient's current round) are ignored.

Inputs: $x_1 = 1$, $x_2 = 0$, $x_3 = 0$. (a) Exhibit an admissible execution in which two correct processes ink different values. (b) Name the missing discipline in one sentence, and say which protocol of Chapter 4 supplies it. (c) Explain why no re-tuning of the timeouts repairs the protocol, citing the theorem that says so. *(Certified fatal per the standing rules: the split-brain execution is written out in the solutions, ink and all.)*

Tier III — For the Ambitious (starred)

Problem 3.10 (★ Ben-Or, guided)**.** Throughout, Box 7 runs with $2t < n$. (a) *(one proposal per round)* show that if $\langle \textsc{proposal}, r, v \rangle$ and $\langle \textsc{proposal}, r, w \rangle$ are both sent with $v, w \neq ?$, then $v = w$ — two majorities of n share a reporter. (b) *(decision cascades)* show that if some process decides v in round r, every process completing round r adopts v, and every process completing round r+1 decides v — pigeonhole t+1 against $n-t$. (c) *(safety, validity)* conclude Agreement; show unanimous inputs decide in round one. (d) *(termination)* prove: if at the start of a round all correct estimates agree, all correct processes decide within two rounds; then show that against an adversary that fixes its schedule *before* the coins are flipped, each round ends with agreeing estimates with probability at least 2^{-n}, so termination has probability one and expected round count at most $2^n \cdot 2$. (e) State honestly why (d)'s adversary is weaker than the scheduler of this chapter (it may not adapt to coin outcomes), and record the pointer that the theorem holds against the adaptive adversary as well, with a more careful argument (Aguilera and Toueg, 1998), and that shared coins repair the exponential expectation (the door stays ajar for the cryptographically inclined). *(Hints only.)*

Problem 3.11 (★ the impossibility propagates)**.** *Atomic broadcast*: all correct processes deliver the same messages in the same order; every message broadcast by a correct process is eventually delivered. (a) Reduce consensus to atomic broadcast in the asynchronous crash model: each process broadcasts its input and decides the first value delivered; verify Agreement, Validity, Termination. (b) Conclude from Theorem 3.4 that no deterministic atomic broadcast protocol tolerates one crash in the asynchronous model. (c) Extend to state-machine replication: any deterministic fault-tolerant replicated log in the asynchronous model must, somewhere, be spending one of the three gates' currencies — find the sentence in your favourite system's documentation where the purchase is disclosed, or the absence where it is not. *(Hints only.)*

3.10 Hints

Hint (Problem 3.6)**.** For (a), the two sub-cases differ in where e sits relative to the schedule reaching E_i; in the en-route case, cut the schedule at e and let the prefix define the member of $\mathcal{D}$. For (b), the walk is along a schedule inside C; the class function is total on C because every member of $\mathcal{D}$ is univalent by supposition.

Hint (Problem 3.8)**.** For (a): a chairman who decides his own value on receipt of one acknowledgment, then crashes before his decide-message survives, while a fallback path decides the other value, is enough — the tribunal of Problem 3.9 can be arranged to do it. For (c): a uniform-consensus protocol is in particular a consensus protocol.

Hint (Problem 3.9)**.** Let the round-one chairman's proposal reach p_3 promptly but p_2 slowly; let acknowledgments and decide-messages dawdle. Two chairmen, two majorities of two, no common member — the crowded club needs majorities of the *same* parliament, and acknowledgments here carry no promise about the future. Fatal execution: Solution and Exhibit 3.5.

Hint (Problem 3.10)**.** (a) Each process reports once per round. (b) $(t+1)+(n-t) > n$. (d) Condition on the round's fixed message schedule; whatever subsets are waited for, either some process sees a proposal (then all coin flips landing on that proposal's value suffice) or none does (then all coins landing equal suffice); both events have probability at least 2^{-n}.

Hint (Problem 3.11)**.** (a) Total order makes "first delivered" identical everywhere; Validity holds because only inputs are broadcast; Termination because a correct process's broadcast is delivered. (b) Contrapositive.

3.11 Solutions

Tier I in full (tersely); Tier II in full for Problems 3.6 and 3.9 (the first works the pivot lemma's neighbour construction, the second is the sabotage certification), solution sketch for Problems 3.7 and 3.8; hints only for Tier III, by standing policy.

Solution (to Problem 3.1)**.** (a) D, G, H: 0-valent (deciding); F: 1-valent; E: 0-valent (G, H only); B: 0-valent (D, E); C: bivalent (E reaches 0, F reaches 1); A: bivalent (via B reaches 0, via C, F reaches 1). (b) Critical moves: $A \to B$ (0-committing), $C \to E$ (0-committing), $C \to F$ (1-committing). (c) Only at univalent nodes; and only with the decided value matching the valence — so possibly at D, E, F, G, H and B (value 0); never at A or C. (d) At C every move commits; the fence cannot be held. No such trap exists in Theorem 3.4 because Lemma 3.3 guarantees a bivalent successor through *every* owed delivery — infinite trees with the pivot property have no forced-commitment nodes on the adversary's chosen path.

Solution (to Problem 3.2)**.** (a) Walk $000 \to 100 \to 110 \to 111$: majority-valences $0, 0, 1, 1$; the split pair is $(100, 110)$, differing at p_2. (b) Crash p_2 at the whistle in both worlds. Survivors p_1, p_3 see input multiset $\{1, 0\}$ in both, receive identical deliveries, and by determinism ink identically — contradicting 0-valence of 100 and 1-valence of 110 simultaneously. (c) Any total valence-assignment on the cube agreeing with Validity at the two unanimous corners changes value along some edge of any corner-to-corner path; the argument of (b) applies at that edge verbatim — nothing about majority was used. (d) In the survivors' obligation to decide at all: without one-fault tolerance they may lawfully wait for p_2 forever, and the contradiction evaporates. (e) If all initial configurations were univalent with a single common valence v, no run decides $1 - v$ anywhere, contradicting non-triviality; so both valences occur among the corners, and any path between differently-valent corners supplies the split pair.

Solution (to Problem 3.3)**.** (a) Admissible; zero faulty (shown in the proof of Theorem 3.4). (b) Inadmissible: delivery clause violated (the addressee is correct — infinitely many steps — but the message never arrives). (c) Inadmissible: two faulty processes. (d) Inadmissible: p_2 is faulty by definition (finitely many steps), which is permitted — but then the run must deliver all messages to the *correct* processes; if it does, it is admissible with faulty set $\{p_2\}$ — so: admissible iff no second process is starved and all mail to p_1, p_3 arrives; as

posed ("every message is delivered") it is admissible, with p_2 counted as the one fault. The lesson: "alive but starved forever" *is* the model's crash — benched-forever and crashed are the same run, which is exactly the doppelgänger the proofs spend. (e) Admissible: one faulty, mail to the correct delivered.

Solution (to Problem 3.4)**.** (a) The read at D violates linearizability: it follows (in real time) the completed write of v_2 yet returns v_1. The read at B is consistent; the refusal at E is not a linearizability violation (no response, no misordering). (b) The refusal at E forfeits total availability (E is non-failed and answered with an error, not a meaningful response); everything else answered. (c) Majority side: consistent and available (it can be both — it holds the quorum); minority side: D chose availability over consistency, E chose consistency (refusal) over availability — one system, two postures, which is precisely why CP/AP licence plates fail. (d) Yield 3/4; harvest of D's read: it reflects the state as of the partition's onset — all writes except v_2, hence (with two completed writes in history, v_1 then v_2) 1/2 of the relevant write-set; as a fraction of the ideal single-value answer, simply: stale by one write.

Solution (to Problem 3.5)**.** (a) Impossible as guaranteed: detection of the dead coordinator in bounded time in an asynchronous model contradicts Theorem 3.4's terrain (crashed-versus-slow); re-file as expensive under partial synchrony with the timeout named. (b) Impossible: Theorem 3.5 (linearizability, total availability, partition — all three claimed). (c) Expensive: a round trip or purchased clock uncertainty per commit (Chapter 8's invoice); possible and shipping. (d) Merely inadvisable (usually): a lease (Chapter 5) delivers the freshness at a fraction of the cost; the requirements argument decides. (e) Impossible: Two Generals (Theorem 1.1) — the last acknowledgment problem; "guaranteed exactly-once delivery" over a lossy channel is bin one. (f) Expensive: idempotence or deduplication engineering throughout (Chapters 1 and 10) — possible, priced, routine.

Solution (to Problem 3.6)**.** (a) Let $i \in \{0, 1\}$. C bivalent gives an i-valent E_i reachable from C by some schedule τ. *Case $e \notin \tau$:* then $E_i \in \mathcal{C}$ by definition, and since e is applicable to C and no event of τ delivers m (only e does), e remains applicable at E_i — this is where the standing fact of Definition 3.3 is spent, answering (c); so $F_i := e(E_i) \in \mathcal{D}$. F_i is reachable from E_i, so $V(F_i) \subseteq V(E_i) = \{i\}$; by the supposition F_i is univalent, hence i-valent. *Case $e \in \tau$:* write $\tau = \sigma_1 \, e \, \sigma_2$ with $e \notin \sigma_1$. Then $\sigma_1(C) \in \mathcal{C}$ and $F_i := e(\sigma_1(C)) \in \mathcal{D}$; moreover $E_i = \sigma_2(F_i)$ is reachable from F_i, so $i \in V(F_i)$, and univalence forces F_i i-valent. Both valences therefore occur in $\mathcal{D}$.

(b) Define, for $E \in \mathcal{C}$, class(E) = the valence of $e(E)$ — total by the supposition that all of $\mathcal{D}$ is univalent. By (a) both classes are inhabited: choose witnesses $W_0, W_1 \in \mathcal{C}$ with class$(W_j) = j$. $C \in \mathcal{C}$ has some class; set $b = \text{class}(C)$ and relabel the values $0 \leftrightarrow 1$ if necessary so that $b = 0$ — the relabeling is harmless because every definition in play is symmetric in the two values, and (a) survives relabeling since it produced witnesses of both classes. Take the schedule (avoiding e) from C to W_1; its configurations all lie in $\mathcal{C}$; the class starts at 0 and ends at 1, so consecutive configurations $C_0, C_1 = e'(C_0)$ exist with classes 0, 1: that is, $D_0 = e(C_0)$ is 0-valent and $D_1 = e(C_1)$ is 1-valent.

Solution (to Problem 3.7)**.** (Sketch.) (a) Yield = |responded|/|issued| over the window; harvest of a response to query Q with ideal input S_Q: $|S_Q \cap \text{reachable}|/|S_Q|$. (b) Always-answering gives yield 1; an aggregate over all shards has $|S_Q \cap \text{reachable}| = k/n \cdot |S_Q|$ under uniformity; for the "at most" clause, the adversary partitions away the shards the service cannot reconstruct, and determinism bars invention. (c) Aggregates cannot meet the floor, point reads can; yield = $1 - q$ (aggregates refused), harvest = 1 on everything served. (d) As stated: the workload mix q is the exchange rate between the two currencies.

Solution (to Problem 3.8)**.** (Sketch.) (a) Run Problem 3.9's tribunal with the schedule of its solution, but crash p_1 immediately after his decide step and before his $\langle \textsc{decide}, 1 \rangle$ letters leave (or lose nothing: let them

be delivered after the others decided — the violation among *correct* processes then needs p_1 dead, so crash him; the surviving pair agree on 0). Agreement-among-correct holds; the corpse holds contrary ink. (b) The body count: a process may act on its decision before crashing — release a lock, ship an order, acknowledge a client; uniform agreement is the difference between "the dead disagreed harmlessly" and "the dead charged the card." (c) Uniform agreement implies Agreement, so a uniform-consensus protocol solves consensus, and Theorem 3.4 forbids it a fortiori in the asynchronous model. (d) Pointer: in the synchronous crash model both are solvable; early-deciding bounds differ (uniformity costs a round in well-studied settings — Charron-Bost and Schiper's line of work); moral as stated.

Solution (to Problem 3.9)**.** (a) The split-brain execution, drawn as Exhibit 3.5. Inputs $x_1 = 1$, $x_2 = 0$, $x_3 = 0$; round one chairman p_1.

(1) p_1 broadcasts $\langle \textsc{propose}, 1, 1 \rangle$. The Post delivers it to p_3 promptly; the copy to p_2 dawdles.

(2) p_3 (in round one) sets $x_3 \leftarrow 1$ and replies $\langle \textsc{ack}, 1 \rangle$; the acknowledgment travels slowly but surely.

(3) p_2, hearing no proposal, times out and advances to round two. The stale proposal later reaches p_2 and is ignored (stale rule). p_3, having acknowledged and seen no decision, times out in turn and advances to round two, *keeping* $x_3 = 1$? — no: attend to the rule; he keeps his *current* estimate, which the round-one proposal already overwrote to 1. To arrange the split we need p_3's acknowledgment to have committed him to nothing — and it has not: no rule binds an acknowledger's future. Continue.

(4) Round two, chairman p_2, estimate 0: broadcasts $\langle \textsc{propose}, 2, 0 \rangle$. It reaches p_3 (now in round two): p_3 sets $x_3 \leftarrow 0$ and acknowledges. p_2 receives the acknowledgment: majority of two (with himself); p_2 **decides** 0 and broadcasts $\langle \textsc{decide}, 0 \rangle$; p_3, receiving it, decides 0 as well.

(5) Only now does the Post deliver p_3's *round-one* acknowledgment to p_1 — still lawfully in round one, his timeout generous. Majority of two (with himself): p_1 **decides** 1 and broadcasts $\langle \textsc{decide}, 1 \rangle$, which arrives at two gentlemen who have already inked 0.

Two correct processes, two inks. Every delay was finite; every message was delivered; the execution is admissible with zero crashes. Fatality certified.

(b) The missing discipline: an acknowledgment binds its maker to nothing about later rounds — the protocol lacks the promise-and-adopt machinery by which a new chairman must first learn what an old majority may already have committed. Chapter 4's phase one (Paxos's prepare/promise, with adopt-the-highest) supplies exactly this. (c) No timeout tuning helps: the executions above violate no timing bound the protocol could enforce, because in the asynchronous model the adversary may realize any finite pattern of delays; and a protocol of this rotating-suspicion shape that *always* terminated would contradict Theorem 3.4. Timeouts may make the split rare; only the missing discipline makes it impossible — safety must never rest on the clock.

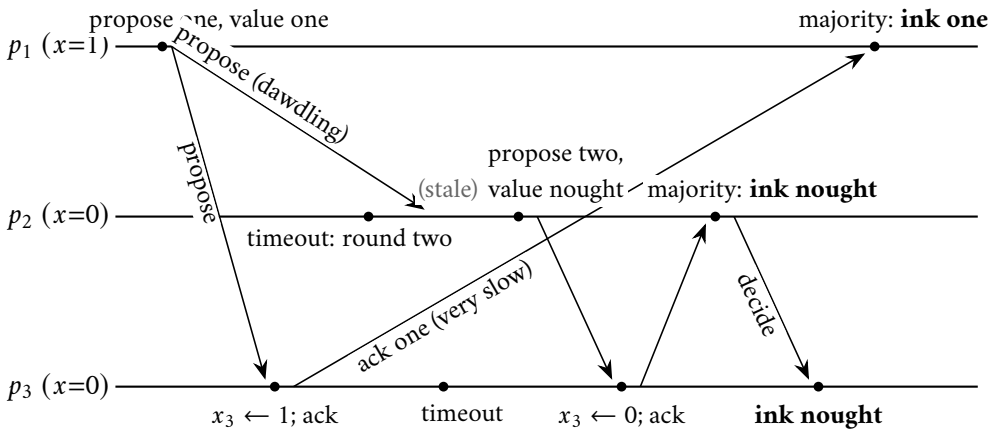


Exhibit 3.5 (the impatient tribunal's split brain; Solution 3.9). The round-one acknowledgment, slow but lawful, arrives after the parliament has moved on and decided nought elsewhere; the deposed chairman, none the wiser, completes his old majority and inks one. Nobody crashed; every letter arrived. Fatality certified.

Solution (to Problems 3.10 and 3.11)**.** By standing policy: hints only (the Hints section above). For Problem 3.10(a), the arithmetic checkpoint: two sets of reporters each of size exceeding $n/2$ intersect; the shared reporter sent one report.

3.12 The Reading

Fischer, Lynch, and Paterson, "Impossibility of Distributed Consensus with One Faulty Process," Journal of the ACM, April 1985. This chapter's text, and the volume's namesake proof. Nine pages; the fatal argument is three. First presented in 1983, at a symposium on database systems — which tells you who was bleeding. Read the model section slowly (their message buffer is this chapter's bag, their Lemma 1 our diamond), then read Lemma 3 twice: it is the pivot lemma, its neighbour construction rehearsed here as Problem 3.6. Honoured with the field's influential-paper prize in 2001, its second year. When you finish, look up and recall that this fenced off a corner of the possible for every machine that will ever be built.

Gilbert and Lynch, "Brewer's Conjecture and the Feasibility of Consistent, Available, Partition-Tolerant Web Services," ACM SIGACT News, June 2002. Equally short. Read for the definitions above all: what *available* totally means, what *atomic* precisely is; then their second theorem — the partially synchronous variant the keynotes never mention — and the t-connected consistency of their final section, which is the negotiated frontier of Remark 3.10 in their own tailoring. As disinfectant afterwards: Kleppmann, "A Critique of the CAP Theorem" (2015) — the plea about the licence plates.

Abadi, "Consistency Tradeoffs in Modern Distributed Database System Design," IEEE Computer, February 2012. The PACELC column. An evening's read, one acronym, and a permanent upgrade to the design-review vocabulary: the peacetime latency toll, named and billed. Read alongside the Whiteboard's third item.

For the starred doors. Ben-Or, "Another Advantage of Free Choice: Completely Asynchronous Agreement Protocols," PODC 1983 (Problem 3.10) — a handful of pages; this field's monuments are short. Aguilera and Toueg, "Failure Detection and Randomization: A Hybrid Approach to Solve Consensus," SIAM

Journal on Computing, 1998, for the adaptive-adversary analysis. Chandra and Toueg, "Unreliable Failure Detectors for Reliable Distributed Systems," and Chandra, Hadzilacos, and Toueg, "The Weakest Failure Detector for Solving Consensus," both Journal of the ACM, 1996 — the third gate, surveyed by its builders.

Standing companion: Kleppmann, chapter 9. His treatment of linearizability and the CAP critique pairs with Part Six; his pages on total order broadcast anticipate Problem 3.11 and Chapter 5. Assigned once for the season; in this chapter it earns its keep twice.

Chapter 4

The Part-Time Parliament

Companion to Episode Four. The island manuscript, its unamused reviewers, and its eight years in the drawer; the single-decree problem sized — offices, majorities, and the crowded club, admitted at last with one line of proof and four chapters of consequences; the Synod of Paxos derived rather than recited — three naive parliaments dead on stage, two repairs, every rule a scar — then boxed rule by rule against "Paxos Made Simple" and proved safe through the invariant, its induction written out in full; the fine print — liveness sold separately, persistence load-bearing, two sabotages certified fatal; Raft, with leader completeness proved after Ongaro's thesis and the counting chairman staged to his ruin; Viewstamped Replication restored to its rightful seniority; and Herlihy's universality theorem, by which this one small agreement is the atom of everything the season builds hereafter.

4.1 Part One — One Law at a Time

In 1990 Leslie Lamport submitted to the ACM's *Transactions on Computer Systems* a paper describing the parliament of a vanished Greek island called Paxos, whose part-time legislators — merchants and mariners, forever wandering out of the chamber and back — kept the island's law consistent over unreliable messengers: one law per numbered slot, the same law in every ledger. The correctness argument wore a fictional archaeology, complete with Paxon dignitaries whose phonetic Greek encoded the names of contemporary computer scientists. The reviewers found the result interesting and the presentation insufferable; the author's position was that the jokes were load-bearing; neither side yielded, and the solution to fault-tolerant consensus — the working escape through the very gate Chapter 3 surveyed — went into a drawer for eight years, while outside it the industry shipped lock managers, replication schemes, and failover scripts that reinvented fragments of its contents, usually wrongly, at considerable expense. It was published at last in 1998, jokes intact, by which time the field needed it desperately enough to forgive it almost anything. This course borrowed the paper's spirit for its very title — a parliament of machines, part-time, unreliable, and lawful nonetheless — and this chapter honours the paper's actual lesson by *deriving* the protocol rather than reciting it: recitation is how Paxos acquired its reputation, a liturgy of prepares and promises that seem arbitrary until one has personally needed each of them. Every rule below is a scar, and every scar has a story.

Size the problem honestly before solving it: half of Paxos's reputation for difficulty is the difficulty of three problems being solved at once by people who thought they were solving one. This chapter treats the *single-decree* problem — one law, decided once — and nothing else; the log of many laws is Chapter 5's business, and elections enter only as liveness machinery, exactly where Theorem 3.4 says they must. The scene: the umbrella shop must fix a single by-law — whether Thursday deliveries shall be free. Motions may originate anywhere: Bingo moves one version, Tuppy another; Bertie merely wants to know what was

decided, preferably before Thursday. The gentlemen who keep ledgers and cast votes are Bingo, Tuppy, and Gussie; the Post delays and reorders without bound but neither forges nor permanently loses; and the legislators are part-time in the paper's exact spirit — they crash and *recover*, wandering back into the chamber with whatever the written ledger preserved. Which imposes the rule the course files under *amnesia forbidden*: anything a gentleman must stand behind later, he writes in the ledger before answering, because a promise remembered only in one's head does not survive a nap, and a parliament of nappers will cheerfully vote both sides of the same question (Problem 4.7 is the certified disaster).

Model of this chapter

For the Synod and for Raft (this whole chapter). Asynchronous message passing; the Post may delay and reorder without bound but neither forges nor (for the liveness discussions) permanently loses — retransmission is assumed beneath the protocol, per Chapter 1's constructions. Failures: *crash-recovery*: a process may stop at any point and later resume with exactly its *stable storage* (its ledger) intact and its volatile state gone. Every protocol variable marked persistent in Boxes 8 and 9 is written to stable storage *before* any message reporting it is sent. Quorums are majorities of a *fixed* membership (reconfiguration is debt #14, deferred to Chapter 5); only pairwise quorum intersection is used by any proof (Rem. 4.3).

Safety versus liveness, divided per Theorem 3.4. All safety claims below hold in the bare model above — any schedule, any finite number of crashes and recoveries, duelling proposers included. No termination claim is made or provable there; liveness discussions (Rem. 4.8) assume partial synchrony after stabilization plus a single distinguished proposer/leader, i.e. exactly the purchases Chapter 3's gates license.

A word on offices before legislating, because the paper's division of labour misleads newcomers into imagining three kinds of machine. There are three *roles*, not three species: a *proposer* is whoever currently has ambitions for the by-law — Bingo and Tuppy both, which is precisely the trouble; an *acceptor* is a keeper of the ledger and a caster of votes — the parliament proper, and the only office whose conduct decides safety; a *learner* is anyone who wishes to discover what was chosen without necessarily having voted — Bertie's whole existence, this chapter. One machine ordinarily holds all three commissions at once, the way a gentleman may be simultaneously a voter, a candidate, and a reader of newspapers; keep the offices separate all the same, for every rule in this chapter belongs to exactly one of them. The specification carries Chapter 3's three clauses shifted one notch toward practice: a value is *chosen* when the parliament has irrevocably settled on it — rendered formally as *some quorum all accepted it at one ballot* — and termination, with the last chapter's scars showing, is deliberately not promised: safety must hold unconditionally, in any weather, while liveness is purchased separately in Part Four. That division is not a compromise settled for; it is the correct engineering response to Fischer–Lynch–Paterson: since no protocol can promise both under asynchrony, build one whose promises never include the impossible part.

Definition 4.1 (single-decree consensus; roles; chosen)**.** Fixed acceptors $a_1, \ldots, a_n$ (the parliament); any number of proposers; learners at leisure. Proposers hold input values (motions). A value v is *chosen* iff there exist a ballot b and a quorum Q of acceptors such that every member of Q has accepted the proposal (b, v). Required, in every execution of the model box: *(Agreement)* at most one value is chosen, ever; *(Validity)* any chosen value is some proposer's motion. Termination is deliberately not required (Theorem 3.4); liveness is discussed, not promised, in Remark 4.8.

Remark 4.2 ("chosen" is a silent, stable fact)*.* Read the definition for what it does *not* contain: no announcement, no instant of proclamation, no participant who must know. "Chosen" is a predicate on the acceptors'

collective ledgers, true exactly when some quorum's acceptances of one (b, v) all exist, and it may hold while every gentleman in the house believes the matter open — the witness of Lemma 4.1 need not know what he carries. Three consequences, each load-bearing. *First*, being chosen and anyone *knowing* it are distinct events: knowledge is not delivered but *collated*, and a learner arriving cold must take the house's deposition — reading the parliament is itself a ballot — rather than await a herald. *Second*, there is no privileged instant at which the fact is born: asynchrony admits no global "now" (Chapter 2's ghost: there is no *the* current state, only states along cuts), so "the moment of choosing" is not well-defined across the house. *Third* — and this is what rescues the predicate from that relativity — "chosen" is *stable*: once true along any consistent cut it is true along every later one, which is the entire content of *irrevocability* (the Agreement clause above). A stable predicate is exactly the kind on which all consistent observers eventually agree, even without agreeing on *when*; the naive "a quorum *currently* agrees" of the coming false start is unstable, and so has no observer-independent truth at all — two learners, two audits, two verdicts. The protocol's whole cleverness is spent engineering a fact whose *becoming-true* no one witnesses, and then forbidding that fact ever to be un-made.

Why majorities? Unanimity tolerates no absentees: one gentleman in the bath and the house is struck dumb, which surrenders the entire fault-tolerance ambition that brought us here — we distribute *because* machines fail. A fixed pair of appointees concentrates the fragility: lose one and no quorum can ever form again. The majority spreads the risk evenly and prices it honestly: a house of three proceeds despite any one absentee, a house of five despite any two — in general the house survives the silence of just under half its members, the exact exchange rate between seats purchased and failures survived, first line of a cost table the season will extend. The deeper truth is that nothing in this chapter's proofs uses "more than half" as such; they use only the crowded-club property — every quorum overlaps every other — of which majorities are merely the cheapest symmetric purchase. The lemma the course has been promising since the plan was drawn, on stage at last: two clubs each containing more than half the members cannot be disjoint, for the room is not large enough.

Lemma 4.1 (the crowded club)**.** *Any two majorities of the same finite set intersect: if* $|Q_1|, |Q_2| > n/2$ *then* $Q_1 \cap Q_2 \neq \emptyset$*.*

Proof of Lemma 4.1. If $Q_1 \cap Q_2 = \emptyset$ then $n \geq |Q_1 \cup Q_2| = |Q_1| + |Q_2| > n/2 + n/2 = n$. □

Feel what it buys: a majority is a quorum with *memory*. Whatever one majority has done, any later majority *contains a witness to it* — the shared member the lemma supplies is this volume's *witness*, and he need not know what he carries. No gentleman ever consults the whole house, which may never be reachable; he consults a majority, and the intersection guarantees that majorities cannot contradict one another in ignorance. Every protocol in this chapter, and most of the season, is an exercise in arranging for the right witness to be in the right club at the right time.

Remark 4.3 (what the proofs actually use)*.* Every argument in this chapter uses quorums only through Lemma 4.1 — pairwise intersection — never through "more than half" as such. Any *quorum system* (a family of subsets, every two of which intersect) supports the Synod verbatim; majorities are the cheapest symmetric choice. Sharper still, the two phases can use *different* families, and only cross-phase intersection is required: that refinement is Howard, Malkhi, and Spiegelman's flexible Paxos, and it is starred as Problem 4.10.

4.2 Part Two — The Derivation, Act One: Three Ways to Die

Attempt the first: first come, first served. Every gentleman accepts the first motion to reach him, and never another; a motion supported by a majority is chosen. It even has the crowded club on its side: two motions cannot both collect majorities, because the shared member would have had to accept both — safety,

proven in one breath. Run it. Bingo moves free Thursday delivery; Tuppy, concurrently — merely elsewhere, as Chapter 2 taught us to say — moves a sixpence surcharge; Gussie, in a spirit of independent inquiry, moves that the shop stock tea. The Post shuffles honestly, the letters land, and the votes fall one–one–one, each gentleman having accepted his own motion first. Now every seat is spent: no future letter can move anyone; the parliament is not deadlocked by crash or lost mail — every machine healthy, every letter delivered — it has voted itself into a wall, permanently, by the ordinary operation of its own rule. The lesson: acceptors must be able to change their minds, or split votes end the republic.

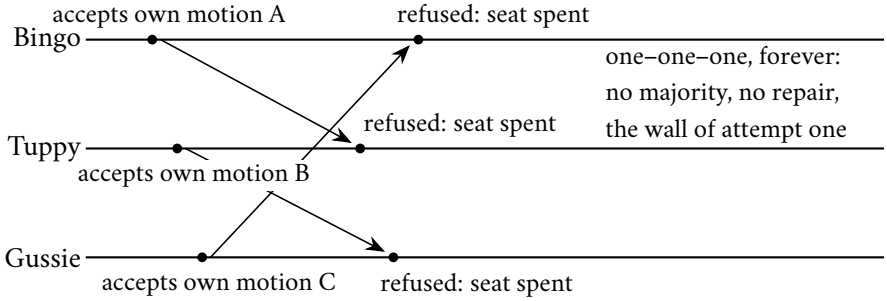


Exhibit 4.1 (the split-vote wall). Accept-once majority voting: three motions land first at their own desks; every later letter is refused; no majority ever forms and none ever can. Safe the way a sealed room is burglar-proof.

Attempt the second: latest word wins. Let a gentleman accept every motion that arrives, each superseding the last, and let "chosen" mean a majority *currently* agrees. Bingo's free-delivery letters land at Bingo and Gussie; Bertie, auditing the house at that moment, finds two ledgers of three in agreement — a majority — notes the by-law settled, and instructs his tailor accordingly. But Tuppy's surcharge letters were merely dawdling, clause one of the charter, and presently they land at Gussie and at Bingo, superseding; Bertie's aunt, auditing an hour later with identical diligence, finds a majority for the surcharge. Two learners, two audits, two verdicts, each impeccable at its instant — and no future audit is safe either, for there is no moment after which the mails are known to be empty. "Currently agrees" is a fiction in a world of letters — Chapter 2's ghost: there is no *the* current state, only states along cuts. Choice must be *irrevocable* — that is the entire content of the word — and perpetual mind-changing is irrevocability's opposite. Wanted: something between one acceptance forever and acceptance without commitment — mind-changing that is *ordered*, where the house can tell an old thought from a new one and never lets the old displace the new.

Attempt the second and a half: appoint a chairman. The instinct every operator has next: why suffer rival motions at all? Let the house agree that Bingo, and only Bingo, proposes. While Bingo stands healthy and reachable this works magnificently — a fact exploited before the chapter is out — but test it against the terrain: the chair goes silent, and crashed-versus-slow is exactly what Chapter 3 proved the house cannot know. Wait forever, and one nap halts the republic; appoint a successor on timeout, and the timeout is a verdict without a body — Bingo may be alive, his motions still crawling through the Post, and now *two* gentlemen believe themselves chairman, each proposing with full authority: the rival-motions problem returned wearing a sash. Leadership does not solve agreement; leadership *presupposes* it — agreement on who leads, and on what the old regime did before it fell, neither of which the sash provides. To earn a chairman, the house first needs machinery that stays safe *while the chairmanship is confused* — indifferent to how many gentlemen currently believe themselves in charge. The thought returns in Part Four with the price tag attached.

Attempt the third: ballots. Here enters the object around which everything turns. A gentleman wishing to move a motion first takes a *ballot number* — formally, a ballot $b \in \mathcal{B}$, totally ordered and uniquely

owned — and the rule of the house becomes: higher ballots supersede lower ones, never the reverse; a motion is chosen when a majority accepts it *at the same ballot number*. Old thoughts now lose to new thoughts by rule, so the wall of attempt one falls: whoever tries again at a higher ballot gathers the house afresh. And observe what a ballot number is — a Lamport clock in parade uniform: a counter that only rises, carried on letters, whose entire purpose is to let a recipient pronounce the word *stale*. The payroll of Chapter 2 remains current.

Definition 4.4 (ballots)**.** $\mathcal{B}$ is an unbounded totally ordered set of *ballot numbers*, partitioned among proposers so no two proposers ever use the same ballot (round-robin residues suffice: with two proposers, odds and evens). Each ballot has at most one proposal (b, v), issued by b's owner.

Run it, and watch it die the instructive death. Bingo takes ballot one, moves free delivery, and the Post is kind: Bingo, Tuppy, and Gussie all accept — a majority, indeed the whole house, at one ballot. *Chosen*, irrevocably, by our own definition; Bertie is informed; perhaps the Thursday circulars are printed. Meanwhile Tuppy — slow to hear the outcome, his confirmation letters dawdling, his colleagues to all appearances never having voted — grows impatient in precisely the way the last chapter licensed him to, takes ballot two, and moves his surcharge. The letters land, and every gentleman, obeying the one rule of the house, accepts ballot two. Any learner arriving now sees the surcharge chosen. The free delivery was chosen — and then it was *unchosen*; the circulars are wrong; agreement is dead by overwrite. And nobody misbehaved: Tuppy followed the protocol to the letter. The protocol simply permits a gentleman with a fresh ballot to propose whatever he pleases, in perfect ignorance of a decision already taken — ballots, which order thoughts, do nothing to inform them.

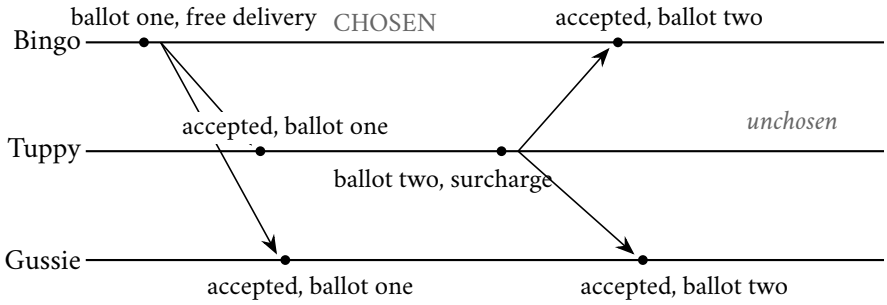


Exhibit 4.2 (the stale-ballot overwrite; ballots without promises). Free delivery is chosen at ballot one by the full house; Tuppy, ignorant, runs ballot two with a different motion, and "higher supersedes" obediently destroys the choice. A new ballot must not be free: phase one exists to make ballot two *learn*.

The true lesson, in capitals: **a new ballot must not be free.** The right to propose at ballot two must come with an obligation — to *learn*, first, what the house may already have done, and to be bound by it. Choice must be irrevocable not merely in the ledgers but *against all future ballots*, which sets the repair problem precisely: arrange matters so that any value chosen at any ballot is the only value any higher ballot can possibly propose. Futures may ratify the past; they may never contradict it.

4.3 Part Three — The Derivation, Act Two: The Synod Stands

The repair comes in two rules. **Repair the first: the promise.** If a new ballot must learn the past before acting, the past must hold still while being learned — Chapter 2's photograph requirement in new dress — so a ballot's life splits into two phases. Phase one, the *prepare*: the proposer of ballot b writes to the house —

formally, the phase-1a message ⟨PREPARE, *b*⟩ of Box 8 — not yet moving any motion, merely announcing the ballot. A gentleman who has not already dealt with a higher ballot replies with two things. First, the *promise* — *I shall entertain nothing below b from this moment* — written in his ledger before the reply leaves the desk: *promised* ← *b*, persisted before the phase-1b answer, amnesia being forbidden. Second, the *confession*: the highest-numbered proposal he has ever *accepted*, ballot and value both — the reply's pair (*ab*, *av*) — so the proposer learns the past from the very gentlemen who constitute it. Promises from a majority close phase one. The promise freezes the ground under the proposer's feet: the majority that answered can no longer quietly accept some straggling lower-ballot motion between his reading of the past and his writing of the future — the confession in hand cannot be aged into a lie by mail still in flight. The prepare phase is asking the room's permission while taking the room's deposition, and the promise is what makes the deposition durable.

Repair the second: adopt the highest. Phase one closes; the proposer holds a majority's promises and confessions; what may he move? The rule — Box 8's phase-2a value rule, the least obvious in the protocol, the one separating everyone who has memorized Paxos from everyone who has understood it: among the confessions in hand, find the one bearing the highest ballot number, and propose *its* value; only if every confession in the majority is empty may the proposer move his own motion. Why the highest? Put the alternatives on trial with the debris run, five gentlemen for elbow room. Ballot one: Bingo moves the surcharge; his accept letters straggle; exactly one — to Oofy — arrives before events overtake it. One acceptance of five: nothing chosen, the mere *fossil* of an ambition — *debris*: formally, a minority acceptance of an abandoned ballot. Ballot two: Tuppy, having prepared against a majority that confessed nothing (Oofy not among them, such is the Post), moves free delivery; his accepts land at Tuppy, Gussie, and Barmy — three of five at one ballot: *chosen*, whether or not anyone yet knows it. Ballot three: Bingo returns and prepares against Bingo, Oofy, and Gussie; the confessions in his hand: nothing; ballot one, the surcharge; ballot two, free delivery. Now the wrong rules on the bench. "Adopt any confessed value": Bingo may lawfully pick the surcharge, and a chosen law is overwritten at ballot three — dead on arrival. "Adopt the plurality": one vote surcharge, one vote free delivery, a tie resolved by whim — dead the same way, merely ashamed of it. Both die because both treat confessions as opinions to be polled, when they are *strata to be read*: the fossil at ballot one is older than the law at ballot two, and age is recorded in ballot numbers, not headcounts. Only the highest-numbered confession reads the strata true, and the induction written out as Lemma 4.3 below proves the rule not merely sufficient but *forced*: any rule that could ever prefer a lower-numbered confession admits a counterexample built exactly like the disaster (Problem 4.4 weighs popularity against strata). The scar closes.

Remark 4.5 (deference to the past is safety, not freshness)*.* The adoption rule is easily misread as a preference for recent proposals over stale ones, as though a value surviving from an abandoned ballot were spoiled goods to be discarded in favour of something fresher. It is the reverse. A proposer adopts the highest confession not because newer values are better — consensus ranks values not at all (Rem. 3.2); the sole constraint on a decision is that some process proposed it — but because a value a quorum accepted at an earlier ballot may already be *chosen*, and Agreement forbids any later ballot contradicting a choice already made. The deference therefore runs *backwards* in time: the proposer binds himself to the past precisely so as never to overturn a verdict he cannot see, and he does so even when — as in the clean run below, where the confessed value was never chosen — the past holds mere debris, for from inside the room a fossil and a law are indistinguishable, and only the cautious reading is safe against both. The ballot numbers are thus not a freshness ranking but a memory: strata to be read, not opinions to be polled, and Lemma 4.3 shows the highest-first reading to be the unique rule under which the future can only ratify. "Stale" is the wrong lens entirely — the rule does not shun old values, it *honours* the one old value that might already be law.

And with it, the Synod of Paxos stands complete. The acceptor's entire estate is two persistent entries:

Definition 4.6 (acceptor state)**.** Each acceptor keeps, on stable storage: *promised* $\in \mathcal{B} \cup \{\bot\}$ — the highest ballot whose PREPARE it has answered; and *accepted* $\in (\mathcal{B} \times V) \cup \{\bot\}$ — the highest-ballot proposal it has accepted, with its value. Box 8 is the only writer of either.

Said once, end to end, so one may see how little of it there suddenly is. Phase one: the proposer of ballot b writes — prepare, ballot b; each gentleman, unless already promised to a higher ballot, writes the promise in his ledger and replies, enclosing his confession. Phase two: holding a majority's promises, the proposer moves — accept, ballot b, and the value the confessions force upon him, his own only if the past is silent; each gentleman, unless promised meanwhile to a higher ballot, writes the acceptance in his ledger — *accepted* $\leftarrow (b, v)$, persisted before the phase-2b reply — and acknowledges. When a majority has accepted at one ballot, the by-law is chosen — irrevocably, this time in truth, because every future ballot is a hostage of the past. Two phases, two rules of refusal, one rule of adoption, a ledger, and a crowded room; everything else ever written about Paxos is performance notes.

Protocol — Box 8. The Synod: single-decree Paxos (checked against "Paxos Made Simple")

Persistent acceptor state per Def. 4.6; all writes to it complete before the reply they enable is sent.

Acceptor:

1: **upon** ⟨PREPARE, b⟩ from proposer P:
2: **if** *promised* $= \bot$ **or** $b >$ *promised*:
3: *promised* $\leftarrow b$ (persist)
4: reply ⟨PROMISE, b, *accepted*⟩ (the confession: (ab, av) or $\bot$)
5: **else** ignore (or reply NACK *promised*, an economy)

6: **upon** ⟨ACCEPT, b, v⟩:
7: **if** *promised* $= \bot$ **or** $b \geq$ *promised*:
8: *promised* $\leftarrow b$; *accepted* $\leftarrow (b, v)$ (persist both)
9: reply ⟨ACCEPTED, b⟩ (to proposer and/or learners)
10: **else** ignore

Proposer (owner of ballot b, motion m):

1: send ⟨PREPARE, b⟩ to the acceptors
2: collect ⟨PROMISE, $b, \cdot$⟩ replies
3: **when** a quorum Q' has replied:
4: $C \leftarrow$ the non-$\bot$ confessions in the replies
5: **if** $C \neq \emptyset$: $v \leftarrow av$ of the highest-ab member of C
6: **else** $v \leftarrow m$
7: send ⟨ACCEPT, b, v⟩ to the acceptors (phase two, once)
8: a value is *chosen* when some quorum has replied ⟨ACCEPTED, b⟩ for one ballot b

Checked against Lamport, "Paxos Made Simple" (2001): phases 1a/1b/ 2a/2b with the same guards ($b >$ *promised* for prepare, $b \geq$ *promised* for accept — an acceptor may accept the very ballot it just promised), the same confession, the same value rule, and the paper's explicit persistence requirement. Learning schemes (§2.3 there) per Remark 4.10.

Remark 4.7 (the four messages, and the three gates)*.* Box 8 in words, for the reader who wants the wiring without the pseudocode. *Four message types* cross the room, and no others: PREPARE(b), a proposer announcing ballot b without yet naming a value; PROMISE($b, \cdot$), an acceptor's reply carrying two distinct things —

the promise proper (a vow about the future: *I will entertain nothing below b*) and, riding in the same envelope, the *confession* (a disclosure about the past: the acceptor's highest accepted (ab, av), or $\perp$); ACCEPT(b, v), the proposer moving the adopted value; and ACCEPTED(b), an acceptor's assent, posted to proposer and learners alike. The two names are not two words for one thing: a vow about the future is not a report of the past, and they vary independently — an acceptor may promise while confessing $\perp$. (The *confession* is our name for what Lamport's *Part-Time Parliament* calls the LASTVOTE message; "promise" for the phase-1b reply is standard usage.) *Three gates* decide everything. The proposer's: he advances from phase one to phase two only while holding PROMISES from a whole quorum — short of a quorum he does not proceed, but abandons b and retries at a higher ballot. Each acceptor's two, differing by a hair that matters: he grants a PROMISE iff the offered ballot stands *strictly above* his last ($b > promised$), and he ACCEPTS iff it stands *no lower* ($b \geq promised$) — so an acceptor may accept the very ballot it has just promised, and neither gate ever opens downward. A value is chosen the instant a quorum has passed the third gate for one ballot; nothing announces it (Rem. 4.2).

The safety argument threaded through the derivation is now written against the boxed lines. First the bookkeeping: one ballot, one proposal.

Lemma 4.2 (one value per ballot)**.** *For each ballot b, at most one proposal (b, v) is ever issued, and every acceptance at b is of that proposal.*

Proof of Lemma 4.2. Ballot b belongs to a single proposer (Def. 4.4), which issues at most one phase-2a message per ballot it owns (Box 8, proposer line 8: phase two is entered at most once per prepared ballot, with one value fixed there). Acceptors accept at b only the proposal (b, v) delivered to them (Box 8, acceptor lines 8–10), by no-forgery the one issued. □

Then the invariant — futures may only ratify — with its skeleton in words before the formalities: the phase-one quorum of the higher ballot meets the choosing quorum of the lower; the shared witness accepted *before* promising (his promise would otherwise have barred that acceptance), so his confession is numbered at least b; by induction, every confession numbered $\geq b$ carries the chosen value; the highest confession is among them; adopt-the-highest does the rest. Step 1 — the promise-ordering subtlety — is the hinge of the whole argument; Problem 4.6 completes it as a standalone lemma.

Lemma 4.3 (the invariant: futures may only ratify)**.** *In any execution of Box 8: if the value v is chosen at ballot b, then every proposal issued at any ballot $b' > b$ has value v.*

Proof of Lemma 4.3. Fix the execution and let v be chosen at ballot b via the quorum Q: every $a \in Q$ accepted (b, v). We prove by strong induction on $b' > b$: *every proposal issued at b' has value v.*

So let (b', w) be issued, and assume the claim for all ballots in (b, b'). The proposer of b' entered phase two holding PROMISE replies for b' from a quorum Q' (Box 8, proposer lines 4–7). By Lemma 4.1 pick a witness $a \in Q \cap Q'$.

Step 1 (the ordering subtlety). At the moment a answered $\langle$PREPARE, $b'\rangle$, it had already accepted (b, v). For suppose not: then a's acceptance of (b, v) came after its promise. But answering the prepare set $promised \geq b'$ (Box 8, acceptor line 4), the variable is persistent and non-decreasing (its only assignment raises it, and recovery restores it), and the acceptance guard (acceptor line 8) requires $b \geq promised \geq b' > b$ — absurd. So the acceptance preceded the promise. (This step is why the promise is persisted *before* the reply: a crash between reply and write re-opens exactly this door; Problem 4.7.)

Step 2 (the confession's floor). Therefore a's reply to b' confessed its then-highest accepted proposal (ab, av) with $ab \geq b$. Its ceiling: $ab < b'$, since a confessed acceptance is an acceptance, and a could not accept any ballot $\geq b'$ before promising b' (the acceptance guard again, applied at the earlier time). So some

confession in the proposer's hands is numbered in $[b, b')$; in particular the *highest* confession (ab^*, av^*) across Q' satisfies $b \le ab^* < b'$.

Step 3 (the strata read true). Every confession numbered in $[b, b')$ carries value v: a confession (ab, av) is an acceptance of the proposal issued at ab (Lemma 4.2); if $ab = b$ that proposal is (b, v); if $b < ab < b'$ the induction hypothesis gives its value as v. Hence $av^* = v$.

Step 4 (adoption). Box 8's value rule (proposer line 6) sets $w = av^*$ whenever any confession is non-empty — and Step 2 produced one. So $w = v$, completing the induction. □

A clean run at full correspondence, for tracing pleasure — the reference trace of the Tier-I calisthenics. The house of three; Gussie's ledger holds a fossil from some abandoned early skirmish: accepted, ballot two, motion S; nothing was ever chosen. Bingo opens ballot five: Tuppy promises with nothing to confess; Gussie promises and confesses (2, S); Bingo's own ledger answers likewise. Phase one closes; the adoption rule reads the room, and Bingo's own ambitions go in the drawer — mark it: *nothing was ever chosen*, the fossil was one acceptance in an abandoned ballot, and the rule binds him to it anyway. The rule cannot see whether a confessed value was chosen — that is the whole epistemic predicament — so it treats any confessed value as *possibly* chosen and defers to the highest: cautious to the point of superstition, and correct precisely because of it. Cheap deference is the premium on an insurance policy against a catastrophe no one can rule out from inside the room. Accept, ballot five, S; Tuppy and Gussie write it in their ledgers; a majority has accepted at one ballot, S is *chosen*, and Bertie may at last be told something that will never afterwards be untrue.

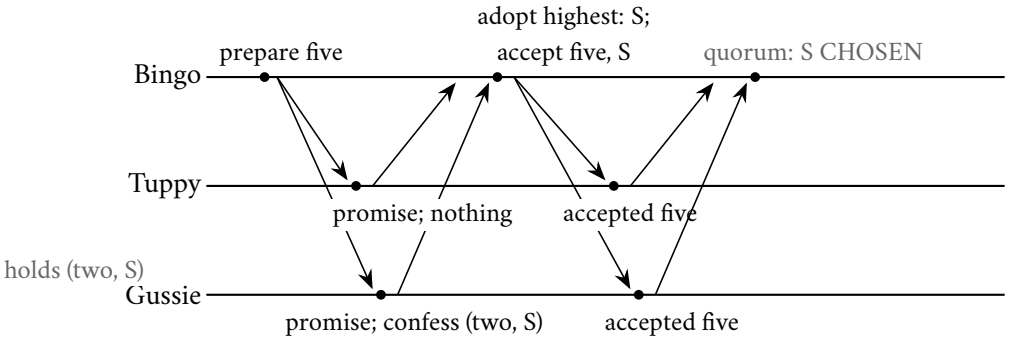


Exhibit 4.3 (a clean Synod run; the reference trace). Gussie's ledger holds a fossil — accepted, ballot two, motion S, never chosen. Bingo's ballot five must adopt it anyway: the rule cannot see whether a confession was chosen, so it defers to the highest. Cheap deference, certain safety. Tier-I calisthenics compute on this trace.

The shape of the finished machine deserves a paragraph of admiration before the fine print. Each naive death retaught the same word: the wall of attempt one, the fiction of attempt two, the overwrite of attempt three — each was the past failing to *bind* the future. The finished machine is nothing but that binding, built twice: promises bind the past against revision from below; adoption binds the future against invention from above. Once chosen, always chosen — not because any gentleman remembers the choosing; no gentleman need ever *know* a choice occurred. The choice is held by the *structure* — lodged in the intersection of clubs, carried forward by witnesses who do not know they are witnesses, adopted by proposers who may believe they are merely being cautious. It is the first machine built in this course that is wiser than every one of its parts, sir, and it will not be the last.

4.4 Part Four — The Fine Print

Clause one: what exactly is guaranteed, and against what. The argument above invoked no clocks, no timeouts, no probabilities, and no luck — read it back and check: pure bookkeeping about ballots and majorities, valid under any schedule the adversary cares to inflict, any delays, any interleaving of duelling ballots, any pattern of crashes and recoveries the ledgers survive. At most one value is ever chosen, in any weather — unconditional safety, delivered in the asynchronous model where Chapter 3's theorem lives. The theorem is not contradicted; attend to what was quietly never promised: that any ballot ever *finishes*. The audit is two lines: two chosen values collide either within one ballot (impossible: one proposer, one proposal per ballot) or across ballots (impossible: the invariant).

Theorem 4.4 (Synod safety)**.** *In every execution of Box 8 under the model box — any schedule, any crashes and recoveries — at most one value is chosen, and any chosen value is some proposer's motion.*

Proof of Theorem 4.4. Agreement. Suppose v_1 is chosen at b_1 and v_2 at b_2, with $b_1 \le b_2$. If $b_1 = b_2$: both are acceptances at one ballot, so $v_1 = v_2$ by Lemma 4.2. If $b_1 < b_2$: being chosen at b_2 requires acceptances of the proposal issued at b_2 (Lemma 4.2), whose value is v_1 by Lemma 4.3; so $v_2 = v_1$.

Validity. By induction on ballots: the value proposed at any ballot is either the proposer's own motion or, by the adoption rule, the value of some confessed — hence earlier-proposed — proposal; the chain of adoptions descends strictly in ballot number and terminates at a motion. □

Clause two: liveness, purchased separately — and the purchase is compulsory. Stage the failure with the two gentlemen it belongs to. Bingo prepares ballot three; the house promises. Before his accept letters land, Tuppy — hearing nothing and growing civil-servant anxious — prepares ballot four; the house, obedient to its promises, is closed below four, and Bingo's accepts are politely refused. Bingo, discovering himself superseded, prepares five; Tuppy's accepts die against five; Tuppy prepares six — the rhythm is audible. Two flawlessly polite gentlemen, each forever asking the room's permission and each thereby cancelling the other's: an infinite minuet of prepares in which no accept ever lands on open ground — the *duelling proposers*, formally the non-terminating prepare-interference execution, Chapter 3's forever-on-the-fence no longer an adversary construction but an operational hazard with names attached. When monitoring some night shows a coordination cluster's epoch counter ratcheting upward at a steady, fruitless clip, this minuet is playing, and a 1985 theorem is calling the tune.

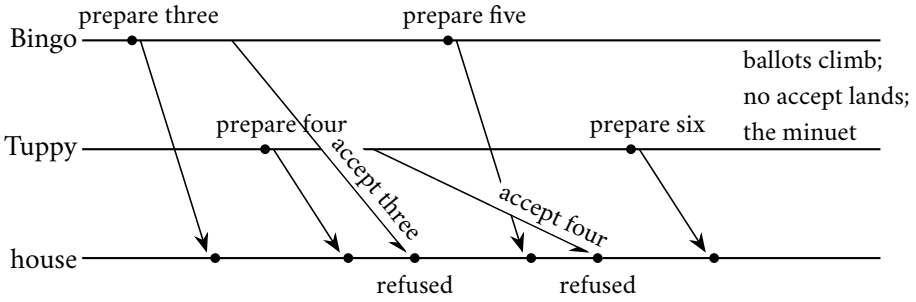


Exhibit 4.4 (the duelling proposers). Each prepare voids the rival's pending phase two; the house (drawn as one line for the quorum's behaviour) refuses each accept against its newer promise. An admissible non-terminating execution — Theorem 3.4 walking the production floor. The cure is a chair, not a rule: Remark 4.8.

The escape is exactly the gate Chapter 3 licenses — the casting note "today, Bingo has the chair" is, formally, the *distinguished proposer*, and the remark below prices the purchase. Its closing sentences are the single most quoted moral of the chapter: the leader is a liveness optimization; safety lives in the quorums.

Remark 4.8 (liveness, purchased separately). No execution of Box 8 violates safety, but Exhibit 4.4's duel — two proposers alternately preparing ever-higher ballots, each voiding the other's phase two — is an admissible non-terminating execution, as Theorem 3.4 requires some to be. The standard purchase: a *distinguished proposer* (elected by timeout, held by heartbeat — an implementation of the eventual-leader oracle Ω, Chapter 3's third gate, here cashing its weakest-detector cheque), plus partial synchrony so the distinction eventually sticks. Then phase one succeeds once and phase two repeats — the multi-decree amortization (multi-Paxos) taken up in Chapter 5. Two simultaneous believers in the chair cost throughput, never safety: stale ballots die against promises.

Clause three: the ledgers themselves. Every load-bearing word above has been *written* — the promise recorded before the reply, the acceptance before the acknowledgment — and this is the failure model's bill, not implementation fussiness. A gentleman who promises ballot four, naps, and wakes with an empty head is a gentleman who will cheerfully accept ballot three — the very acceptance his promise forbade — and with two such nappers the parliament manufactures two chosen values in an afternoon. "Because it is rebuilt by traffic anyway" is a sentence heard in production reviews, spoken by persons of otherwise sound judgment; Problem 4.7 stages it precisely, and the disaster is certified fatal in the solutions, double choice written out to the last letter. Amnesia is not a degraded mode of the protocol; it is a different protocol, and a wrong one.

Remark 4.9 (persistence is load-bearing). Theorem 4.4's proof reads the acceptor's *promised* and *accepted* across crashes; if either reverts on recovery, Lemma 4.3's ordering argument dies and two values can be chosen in one afternoon — Problem 4.7 manufactures exactly that, and Exhibit 4.8 certifies it. The write precedes the reply; there is no lighter correct discipline.

Two footnotes the folklore habitually loses. First, the learners: a value being chosen and anyone *knowing* it are different events — in the debris run, free delivery was law while every gentleman believed the matter still open. Choice lives in the ledgers collectively; knowledge requires collation.

Remark 4.10 (learning; a read is a ballot). Choice and knowledge of choice are separate events. Acceptors report acceptances to learners (or to the chair, who announces); a learner holding a quorum's acceptances at one ballot holds a certificate. A cold-start learner must not trust a bare poll of ledgers — it may observe a mid-ballot mixture (the Chapter-2 auditors' sin in new dress); the clean method is to run a ballot: prepare, adopt the highest, propose it, and read what the house confirms. Mark precisely what that ballot certifies, for it is a common slip: a phase-one sweep returns only the *candidate* the adoption rule would force — the highest confession, which may be mere debris never chosen (the clean run's lone fossil) — so a confession proves *candidacy*, not choice. Only a quorum's ACCEPTED at one ballot certifies that a value is chosen; learning is therefore a *completed* ballot, not a solicitation of promises. The cost of reads through consensus, and what leases may lawfully shortcut, is Chapter 5's business.

Second, the invoice. Count the letters in the clean run: one round trip of prepares and promises, one of accepts and acceptances — two round trips of the Post per decision, each touching a majority, plus a ledger write on every gentleman at each step. That is the sticker price of one by-law, and for a parliament deciding by-laws all day — which is what a replicated database is — it is plainly extortionate. The remedy is the chairman now lawfully earned: a *stable* leader runs phase one *once*, for all future decisions at a stroke — one standing prepare, a majority's standing promises — and thereafter each decision costs a single accept round; phase one amortized away, the common case one round trip. That arrangement — the Synod with a standing chairman and a numbered docket of decisions — is multi-Paxos, it is what every production deployment actually runs, and it is the door at which Chapter 5 knocks.

4.5 Part Five — Raft, or Understandability as an Engineering Requirement

Twenty-odd years pass. Paxos wins in production — inside lock services and databases, usually via the multi-decree extension — and simultaneously acquires a folklore: that it is fiendish, that nobody implements it correctly from the papers, that every real deployment is a nest of undocumented amendments. Some of this was costume, as established; some was real: the single-decree Synod is small, but composing it into a practical replicated log leaves many load-bearing decisions as exercises for the implementer, and implementers' exercises become outages. In 2014 Diego Ongaro and John Ousterhout of Stanford published, at the USENIX technical conference, a protocol called Raft, built on a premise the field found mildly scandalous: *understandability is an engineering requirement* — not a courtesy, a requirement, on the grounds that operators debug at three in the morning what they can hold in their heads, and safety on paper is not safety in a fleet run by the confused. They measured it: a formal user study — two lecture videos of equal polish, students quizzed on both protocols, order counterbalanced like a proper trial — with Raft's comprehension scores materially higher and the quizzes published so the skeptical could re-run the experiment. The method is the transferable part: decompose the problem into nearly independent concerns — leader election, log replication, safety, membership — so each fits in a head separately; prefer one strong leader, not because symmetry is impossible but because asymmetry is narratable; shrink the state space wherever a rule can be simplified at modest cost. The stance in one line: if operators cannot re-derive your invariants at three in the morning, your invariants are decorative.

The furniture is this chapter's machinery in working clothes, and it is given here at full inventory, since every rule in Box 9 is a comparison over exactly these parts. Time divides into *terms* — a term is a ballot wearing a calendar, the Lamport clock drawing yet another salary — and "up to date," the phrase on which committed law will hang, is the log-ending order $\preceq$ fixed now.

Definition 4.11 (Raft vocabulary)**.** Time divides into *terms* $1, 2, \ldots$; each term holds at most one leader. At every moment each server occupies exactly one of three *offices*: *follower* (passive — answers letters, never initiates), *candidate* (a follower whose election timeout expired, soliciting votes), or *leader* (a candidate who won; all client business flows through it). All begin as followers. Each server persists *currentTerm*, *votedFor* (at most one vote per term), and a *log* of entries; the entry at index i carries the term in which a leader created it. Volatile, rebuilt after a crash: *commitIndex*, the highest index the server knows committed. An entry is *committed* when it is durable on a quorum *and* the commit rule of Box 9 licenses the announcement (the distinction is Problem 4.8). For logs L, L' with final entries of terms t, t' and lengths ℓ, ℓ': $L \preceq L'$ (L' is *at least as up to date*) iff $t < t'$, or ($t = t'$ and $\ell \leq \ell'$). A voter grants its vote only to candidates whose log is at least as up to date as its own.

One rule governs every letter before any handler reads its contents, and it is the hinge on which the offices turn. Every Raft message, in both directions — solicitation and reply, replication and acknowledgement — carries its sender's *currentTerm*, and the recipient compares that term against its own first. Higher: the recipient adopts the letter's term on the spot, clears its vote — a fresh term is a fresh slate — and becomes a follower, whatever it was; a candidate abandons the candidacy, a leader the gavel, and neither is consulted in the matter. Lower: the letter is refused unread, the reply carrying the recipient's own term so the sender may discover itself behind and stand down. This one comparison, run on every message, is how Raft retires a stale chairman without ever establishing that he died: the deposed leader deposes himself the moment anybody addresses him in the language of a later reign. It stands first in both handlers of Box 9, and it is half of what keeps Theorem 4.7's witness armed.

The essentials, boxed against the paper's own summary figure:

Protocol — Box 9. Raft essentials (checked against the 2014 paper's Figure 2)

Persistent per server, written before any reply leaves the desk: *currentTerm*, *votedFor*, *log*. Volatile, rebuilt after a crash: *commitIndex*; on a leader, per follower: *nextIndex* (the first slot to try) and *matchIndex* (the highest slot known replicated there).

1: **offices:** every server is exactly one of follower, candidate, leader; all begin as followers
2: **the term rule, on every message in either direction, before its contents are read:** a term above *currentTerm* is adopted at once — persist it, clear *votedFor*, become a follower, whatever the office held; a term below *currentTerm* is refused, the reply carrying *currentTerm* so the sender stands down
3: **election:** a follower hearing no AppendEntries for its randomized timeout increments *currentTerm*, votes for itself (persist), becomes a candidate, and sends every peer ⟨RequestVote, *currentTerm*, *lastLogIndex*, *lastLogTerm*⟩ — the ending of its own log
4: **upon** ⟨RequestVote, *t*, candidate log ending⟩, the term rule having run:
5: grant iff *votedFor* ∈ {⊥, candidate} **and** own log ⪯ candidate log (Def. 4.11); persist the vote before the reply leaves
6: **an election ends in one of three ways:** a quorum of grants (the candidate's own vote counted) makes it leader of the term; an AppendEntries bearing an equal or later term makes it a follower again; or its timeout expires once more — a split vote — and it stands afresh in the next term
7: **leader:** on a client motion, append to own log at the next index, stamped *currentTerm*; send each follower, from that follower's *nextIndex*: ⟨AppendEntries, *currentTerm*, *prevIndex*, *prevTerm*, *entries*, *commitIndex*⟩; empty *entries* at a steady cadence is the *heartbeat*, carrying the other fields all the same
8: **upon** ⟨AppendEntries, *t*, *prevIndex*, *prevTerm*, *entries*, *leaderCommit*⟩, the term rule having run:
9: **consistency check:** refuse unless own log holds an entry of term *prevTerm* at index *prevIndex*
10: **else:** delete any suffix conflicting with *entries*; append what is new (persist); set *commitIndex* ← min(*leaderCommit*, index of last new entry); acknowledge
11: **repair:** on a refusal the leader decrements that follower's *nextIndex* and retries, walking backward until the ledgers meet, then shipping its own log forward; **leader append-only:** it never deletes or overwrites an entry of its own log
12: **commit rule:** the leader advances *commitIndex* to *i* only when (a) *matchIndex* ≥ *i* at a quorum, its own log counted, **and** (b) the entry at *i* carries *the leader's own current term*; earlier-term entries commit only transitively, beneath such an *i*
13: **apply:** every server applies committed entries to its state machine in index order as its *commitIndex* advances; the leader answers the client upon applying the motion

Checked against Ongaro and Ousterhout (2014), Figure 2 and §§5.1–5.4: the offices and the term rule, the vote guards, the consistency check, conflict-suffix deletion, the commit-index advance by *leaderCommit*, the up-to-date restriction, and the §5.4.2 commitment restriction — clause (b) of the commit rule — whose necessity is Problem 4.8.

Each term opens with an election, and the walk deserves its full deposition. A follower keeps a clock — the *election timeout*, his allowance of patience — and a gentleman who has heard nothing from the chair for his patience's length concludes the throne vacant and stands for it: he increments *currentTerm*, votes for himself and persists the vote, becomes a candidate, and writes every peer a RequestVote carrying the new term and the *ending* of his own log. The timeout is the purchased verdict it always is, and Raft owns

the purchase openly. The voter's decision is three questions, asked in a fixed order. First, the term — the term rule above: below his own, refuse and correct the sender; above it, adopt it and clear the vote, a fresh term being a fresh slate. Second, the vote: already spent this term on another, refuse. Third, the ending: the candidate's log must be at least as up to date as the voter's own ($\preceq$ of Def. 4.11, term before length), and this refusal is the veto on which committed law will shortly hang. Three questions survived, the vote is written in the ledger *first*, then granted — amnesia forbidden, precisely as with promises and for precisely the same reason: a gentleman who forgets his vote across a crash and votes twice in one term has manufactured two leaders with his own hand.

An election ends in exactly one of three ways. A quorum of grants, the candidate's own vote counted, makes him leader of the term, and he announces himself by writing to everybody at once, which stops the others standing. Or a letter arrives mid-candidacy from a leader of his own term or later — somebody got there first — and the term rule returns him to the ranks without a fight. Or nobody wins — a *split vote* — and nothing is broken and nothing is decided: patience expires again and the house stands afresh in the next term. Raft would sooner waste a term than risk two leaders in one. A sitting leader holds the house by *heartbeat* — empty replication letters at a steady cadence, each saying merely *the chair is occupied* — and the empty letter does two load-bearing jobs: it suppresses elections, a follower who hears from the chair having no cause to stand, and it carries the leader's term, which by the term rule quietly deposes any stale claimant in earshot. Duelling candidates are dissolved by the humblest trick in the paper, *randomized* election timeouts — each gentleman draws his allowance of patience from a range, so no two lose patience together, and someone almost surely moves first and collects his quorum before a rival stirs. The pinch of randomness from Chapter 3's second gate, deployed here against ordinary bad luck: the theorem still holds, the storm is still possible, the dice make it embarrassingly improbable per round. And the crowded club yields the first of the paper's five warranty clauses in the time it takes to say it:

Lemma 4.5 (Raft election safety)**.** *At most one server becomes leader in any given term.*

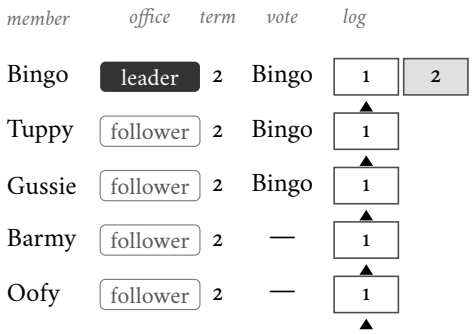
Proof of Lemma 4.5. A leader of term t collected GRANTED votes for t from a quorum. Each server persists *votedFor* for term t before granting (Box 9), and grants at most once per term. Two leaders of term t would give two quorums whose shared member (Lemma 4.1) granted twice. Contradiction. □

The warranty's remaining clauses — leader append-only (a leader never deletes or overwrites its own log), log matching, leader completeness, and, resting on the others, state-machine safety — belong to Raft's real subject: the *log*, the numbered ledger of laws that Chapter 5 makes the star. Raft is log-native, and the replication letter deserves its full manifest, because one of its four items does the work of the protocol. An AppendEntries carries: the leader's term; the entries being sent, if any; the stamp — index and term — of the slot *immediately preceding* them, the subject of the *consistency check*; and the leader's *commitIndex*. The follower's rule: before writing anything, look up the named predecessor slot in his own log. An entry there of that index and term means leader and follower agree at least that far back — accept, write, acknowledge. Anything else — the slot empty, or bearing a different term — means refuse, without arguing and without proposing amendments.

The repair that follows a refusal is prettier in operation than in description. Suppose the leader, newly seated in term four, believes Tuppy's log matches his own through slot seven and writes accordingly: *here is slot eight; the slot before it, slot seven, bears term four*. Tuppy consults his own slot seven, finds term two sitting there — the residue of an earlier reign — and refuses. The leader decrements Tuppy's *nextIndex* and tries again: *here are slots seven and eight; the slot before them, slot six, bears term four*. The check passes; Tuppy deletes the conflicting entry at slot seven and writes the leader's slots seven and eight in its place. The leader walks backward until the two logs meet, then ships his own version forward over whatever the follower held; followers never negotiate. A slot replicated on a majority becomes *committed* — chosen, in

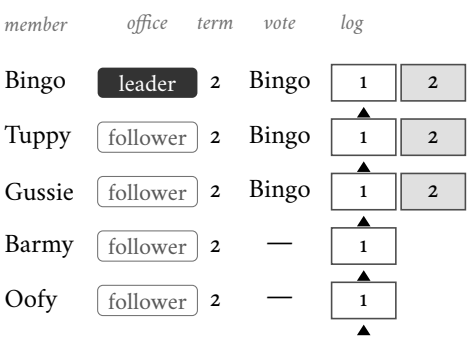
this chapter's vocabulary — under the commit rule whose necessity Barmy is shortly to demonstrate, and the announcement has its own plumbing: the leader, counting a quorum of acknowledgements for an entry of his own term, advances *commitIndex*, applies the motion, and answers the client; the followers learn from the *leaderCommit* field of the very next letter, an empty heartbeat sufficing. The leader knows a thing is committed one letter before the house does — a permanent asymmetry, and not a defect. Exhibit 4.5 photographs the whole happy case, desk by desk, one panel per beat.

beat one: the leader appends and posts



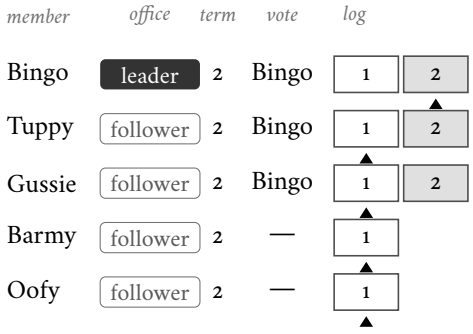
Bertie's motion reaches the chair alone. Bingo appends (2 : 2) and posts append-entries to all four: term 2; the entry; predecessor stamp slot 1, term 1; commit index 1.

beat two: the check passes at two desks



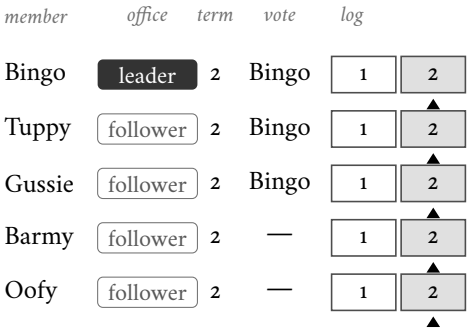
Tuppy and Gussie find term 1 at slot one: the check passes — write, acknowledge. The letters to Barmy and Oofy dawdle in the Post.

beat three: a quorum at the sitting term



Two acknowledgements plus his own desk: a quorum at the sitting term. Bingo advances his commit index, applies, and answers Bertie; the house does not know yet.

beat four: the commit index travels



The next heartbeat carries commit index 2; the dawdling letters land at last. Five identical logs, every triangle at slot two.

Exhibit 4.5 (the happy case, desk by desk). Filmstrip conventions, holding through Exhibit 4.7: each panel narrates, beneath its grid, the moves of one beat, and the grid photographs the whole house *after* those moves have completed — the state the beat leaves behind. Per member: the office (dashed: paused or crashed), the current term, the vote cast in that term (— while unspent), and the log, one cell per slot, the numeral being the term stamped in the entry. One shaded motion appears here: the light cells are the term-two motion, the only law in play in the happy case. A darker shade is reserved for the rival term-three motion, which enters with the figure eight of Exhibit 4.6 — Problem 4.8's staging reuses this opening scene. The filled triangle beneath a cell is that member's commit index; a hollow triangle, a commit index advanced by the sabotaged counting rule. Here every letter lands, and one round trip commits.

The central discipline underneath is log matching, maintained by the consistency check at every single write: a follower who disagrees about the predecessor refuses, the leader steps back until the ledgers meet, then overwrites forward — an induction that never starts wrong.

Lemma 4.6 (log matching; Ongaro–Ousterhout)**.** *If the logs of two servers contain entries with the same index and the same term, then the entries are identical and so are the two logs at every lower index.*

Proof of Lemma 4.6. Both claims by induction on the appending history. A leader creates at most one entry per (index, term) pair — it assigns consecutive indices within its term and never overwrites its own log (leader append-only, Box 9) — so equal (index, term) implies the same created entry. For the prefix claim: a follower installs the entry at index i with term t only through an AppendEntries message whose consistency check it passed — the message carries the (index, term) of the predecessor entry, and the follower refuses unless its own log matches at index $i-1$ (Box 9). Inductively the leader's and follower's logs agree through $i-1$ at installation time, and any later divergence at or below i would require an overwrite, which followers perform only by truncating a conflicting suffix and re-installing under the same check. □

Elegant, effective — and the source of a trap, because followers' unreplicated tails can be overwritten, and an election can crown a gentleman whose ledger lacks another's recent scribbles. To keep committed law safe, Raft adds the voting restriction — the up-to-date veto of Def. 4.11 — and the safety argument is the Synod's wearing electoral dress. A committed slot lives on a majority: commitment requires no less. A winning candidate persuaded a majority: that is what winning means. The clubs intersect; some voter holds the committed slot; and that voter's vote was conditional — he compared endings and granted only to a ledger at least as up to date as his own, which a ledger missing his committed slot cannot be. The witness's veto: the crowded club supplies the witness, the up-to-date rule arms him, and no gentleman reaches the gavel without carrying every law the house has passed. Where the Synod's proposer *reads* the past out of a majority and adopts it, Raft's electorate refuses any future that has not already done its reading. The claim deserves a full induction over the terms between commit and election, structured after Ongaro's thesis:

Theorem 4.7 (leader completeness)**.** *If a log entry is committed (per Box 9's rule) in term t, then that entry is present in the log of the leader of every term $t' > t$.*

Proof of Theorem 4.7. Structured after Ongaro's thesis. Let the entry e (index i, term t) be committed in term t: the leader of term t counted e durable on a quorum Q *within term t* (Box 9's rule). Induct on $t' > t$, assuming the leaders of all terms in (t, t') — those that existed — held e.

The leader L' of t' won votes from a quorum Q'; pick a witness $a \in Q \cap Q'$ (Lemma 4.1). The witness stored e before voting: it appended e in term t, and between t and its vote in t', no entry it held at index i was ever displaced — displacement requires an AppendEntries from some leader of an intermediate term, every such leader held e at index i by the induction hypothesis, and log matching (Lemma 4.6) makes their transmissions at index i be e itself.

Now the veto arithmetic. When a voted for L', the up-to-date comparison (Def. 4.11) held: a's log $\preceq$ L''s log. Suppose toward contradiction L' lacks e. Case (i): L''s final term equals t. Then L''s final entry was created by the leader of term t (Lemma 4.6's first claim), i.e. L''s log is a subsequence of that leader's log truncated at some length; lacking e means L''s log is shorter than $i \leq$ the length of a's log, whose final term is at least t — if a's final term is exactly t, the length comparison fails ($\ell_a \geq i > \ell_{L'}$), and if it exceeds t, the term comparison already fails; either way a's log $\not\preceq L''$s. Case (ii): L''s final term is less than t: then since a's final term is at least t, again a's log $\not\preceq L''$s. Case (iii): L''s final term t'' lies in (t, t'): the entry ending L''s log was created by the leader of t'', which by the induction hypothesis held e; by log matching, any log containing an entry created in t'' at an index $\geq i$ agrees with that leader's log through that index and so contains e — if L''s final index is $\geq i$, L' holds e after all; if it is $< i$... it cannot be, for the leader of t'' appends after its own copy of e, so every entry it creates sits at an index $> i$. All cases contradict; L' holds e. □

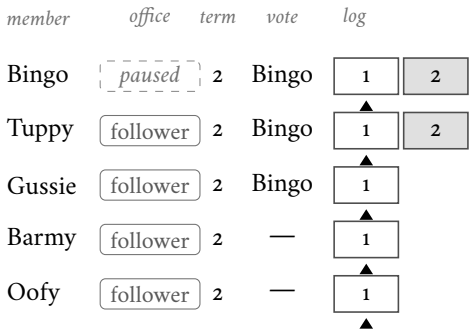
And the clause the tenant actually buys:

Corollary 4.8 (state-machine safety)**.** *If any server has applied the entry at index i to its state machine, no server ever applies a different entry at index i.*

Proof of Corollary 4.8. A server applies index i only once i is committed within its leader's announcement chain; by Theorem 4.7 every later leader's log contains the committed entry at i, and by Lemma 4.6 any server's log that ever carries a committed prefix carries the identical entries there; two different applications at i would exhibit two committed entries at i, i.e. two leaders of some terms both completing announcements for conflicting entries, contradicting Theorem 4.7 applied to the earlier commitment. □

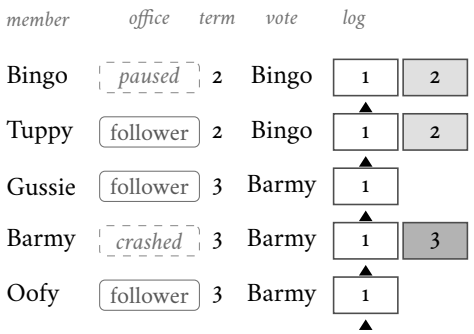
One trap remains, subtle enough that the paper devotes its most famous figure to it — the figure eight — staged here with the full company of five; the ledger-by-ledger reconstruction is Problem 4.8, certified fatal in the solutions and drawn frame by frame below (Exhibits 4.6 and 4.7). The rule it justifies: **a leader may count replicas toward commitment only for slots of his own term** — never for an inherited slot, however widely replicated. The staging, in brief. Term two: chairman Bingo writes a motion into his ledger's second slot and replicates it to Tuppy alone — Gussie's copy, Gussie being Gussie, dawdles in the Post — before going quiet, a pause of the garbage-collecting kind. Term three: Barmy, on his debut in this series, stands and gathers votes from Gussie and Oofy, whose short ledgers see nothing wrong with his (Tuppy, holding the term-two slot, would refuse — but two votes and his own make the majority); Chairman Barmy writes his own pet motion into *his* second slot, stamped term three, and, in the finest tradition of his casting, crashes on debut, having replicated it to no one. Term four: Bingo wakes, none the wiser, stands, and wins — Tuppy, Gussie, and his own vote — then resumes his term-two business and replicates the old second slot to Gussie at last: three ledgers of five now hold it, durable on a majority, and the naive rule pronounces it committed; Bertie is informed; the by-law is applied to somebody's Thursday. Term five: Bingo pauses again, Barmy recovers and stands, and the sting is delivered by the very rule installed to protect committed law: Barmy's ledger ends with a term-three slot, his voters' with term-two slots; later term wins the comparison, and Gussie and Oofy have no lawful grounds for refusal. Chairman Barmy replicates his term-three slot to the whole house; log matching does the rest, and the motion a chairman counted to a majority and pronounced law is expunged from every ledger on the island — replaced by the pet project of a gentleman whose entire tenure consists of one scribble and two crashes. The counting was the crime: replication on a majority made the slot *durable*, but elections protect committed law by comparing *terms*, and an inherited slot's term is old news — durability is a fact about copies; commitment is a claim about elections yet to come, and only the sitting term's replication makes the claim good, because only the sitting term is what future voters will compare. Under the lawful rule the trap fails to spring: Bingo in term four first commits a fresh term-four motion — a no-operation will do; dignity is not required — and log matching drags the inheritance into safety underneath; Barmy's second candidacy then dies at Gussie's veto, exactly where Theorem 4.7's witness stands. The deepest sentence in the paper, and the opposition of *durable versus committed* — on a quorum, versus on a quorum in the leader's own term: *replication makes a slot durable; only the sitting term's replication makes it committed.*

term two: the old motion strands



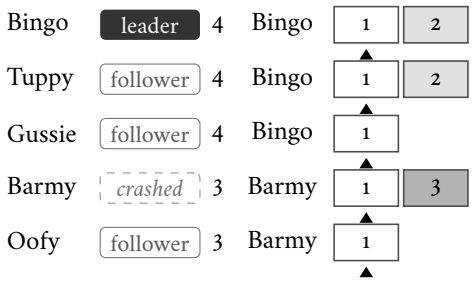
Bingo, leading term 2, appends (2 : 2) and replicates: Tuppy's copy lands; Gussie's dawdles in the Post; Barmy and Oofy see nothing. Then the chair goes quiet.

term three: elected, one scribble, crashed



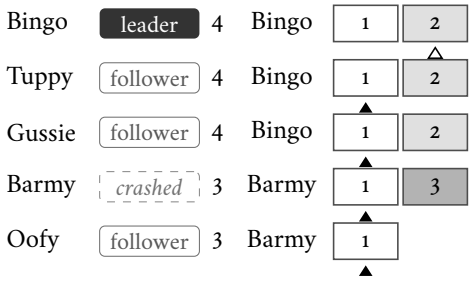
Barmy stands in term 3, soliciting Gussie and Oofy: their endings (1 : 1) match his own — granted. Tuppy, who would refuse, is not asked. Barmy appends (2 : 3) and crashes.

term four: Bingo restored



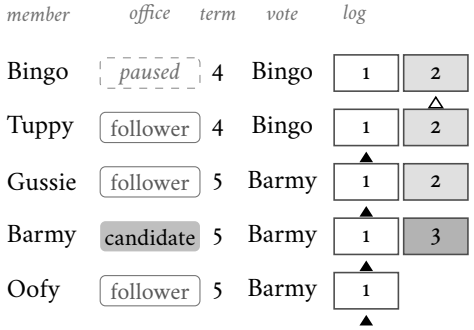
Bingo wakes, none the wiser, and stands in term 4. Tuppy: equal ending, equal length — granted. Gussie: (1 : 1) is behind — granted. Three of five; Bingo resumes his term-2 business.

term four: the fatal count



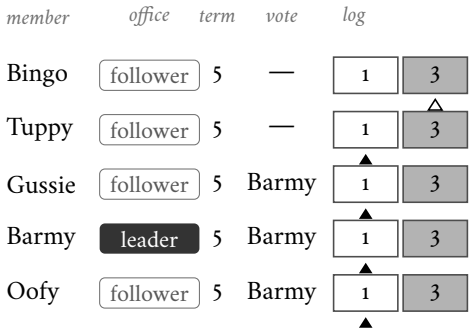
Bingo replicates (2 : 2) to Gussie: the term-2 entry now sits on three desks of five. The sabotaged rule counts across terms and pronounces it committed; Bertie is told.

term five: the compelled votes



Bingo pauses again; Barmy recovers and stands in term 5. His ending (2 : 3) beats (2 : 2) and (1 : 1) — later term wins, whatever the length — so Gussie and Oofy must grant. Two votes and his own.

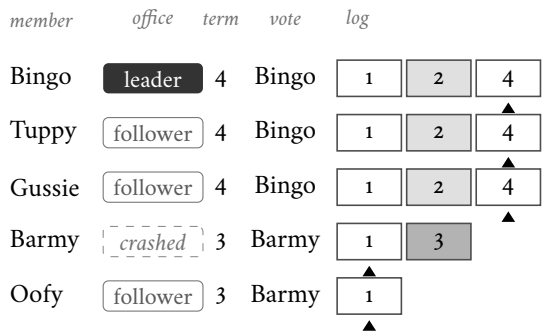
term five: the overwrite



Barmy replicates (2 : 3); the check passes at slot one; conflicting suffixes are deleted. The "committed" entry survives on no desk — and Bingo has applied a law his log no longer holds.

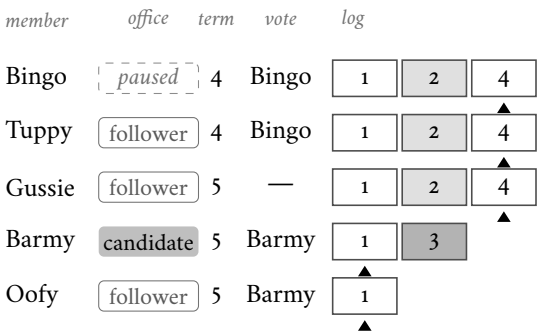
Exhibit 4.6 (the counting chairman's figure eight; the staging of Problem 4.8). Six panels, terms two through five, conventions as in Exhibit 4.5; each grid, as ever, shows the state *after* the moves narrated beneath it. The counting was the crime: replication on a quorum made the term-two slot *durable*, but elections protect committed law by comparing *terms*, and an inherited slot's term is old news. Bingo's hollow triangle in the final panel is the corpse: a state machine has applied a law its own log no longer contains.

term four, lawful: a fresh no-op first



Under the true rule, Bingo first replicates a fresh no-op (3 : 4) to Tuppy and Gussie — a quorum of his own term. Slot three commits by its own count; slot two beneath it, by log matching.

term five, lawful: the veto falls



Barmy solicits again: Gussie's ending (3 : 4) beats his (2 : 3) — refused; his vote in term 5 stays unspent. Only Oofy grants: two of five, no quorum. The by-law stands forever.

Exhibit 4.7 (the lawful replay; the trap fails to spring). The same schedule from Bingo's term-four election, under the true commit rule; the fresh entry's own-term count commits both slots, and Barmy's second candidacy dies at Gussie's veto — exactly where Theorem 4.7's witness stands (Lemma 4.6 dragging the inheritance into safety beneath the no-op).

4.6 Part Six — The Family Restored to Order

Set the two protocols side by side and the differences are smaller than the war of pamphlets suggests: terms are ballots; votes are promises; the up-to-date veto is the confession-and-adoption rule turned inside out. Howard and Mortier, in a tidy comparison published in 2020, put it near enough thus: strip the presentation and the two are one algorithm wearing two management philosophies — Paxos trusts any proposer to learn the past on demand; Raft insists the past be pre-installed in the leader before the gavel (Problem 4.11 builds the family dictionary). Multi-Paxos in sensible tweeds.

And one paragraph of justice, overdue by decades. In 1988 — before the island manuscript began circulating, and a decade before it saw print — Brian Oki and Barbara Liskov published, at the ACM's symposium on the principles of distributed computing, a protocol called *Viewstamped Replication*: a primary per numbered *view* — the term-analogue, its seniority restored here — view changes on suspicion of failure, a majority carrying the state forward across the change, the new primary obliged to adopt the survivors' latest state before proceeding. Views are terms are ballots; the view change is the election is phase one; the adoption obligation is the rule proved forced above. The same machine, discovered independently and published *first*, in the open literature, while the rival manuscript rested in its drawer being witty; this volume enters Viewstamped Replication into the record at its rightful seniority. Why the island conquered anyway: partly the drawer's own legend — nothing advertises a paper like eight years of samizdat circulation among people who insist it is unreadable — and mostly a decade of timing: when the great industrial deployments were built, the island paper had just surfaced, its author was writing exasperated simplifications, and the war stories that followed (a celebrated Google account is titled, with feeling, "Paxos Made Live") cemented the vocabulary: ballots, proposers, acceptors. Vocabulary is destiny in this trade. The census, as the course will keep it: one theorem-shaped idea, four decades of independent tailoring — Viewstamped Replication first in print; Paxos first in folklore; Zab, born industrial in ZooKeeper livery, first in the production fleet (Chapter 5); Raft first in the textbooks. The family reunion takes four sentences; perform it gently, for people grow attached to their vocabularies, having paid tuition in outages for each.

4.7 Part Seven — The Universal Machine

The last movement widens the lens on why the problem was worth two acts of derivation. In 1991 Maurice Herlihy published "Wait-Free Synchronization," whose centerpiece ranks every shared object by a single number — the *consensus number*, the largest house the object can seat. The rankings startle on first reading: read/write registers — the substrate a lifetime of single-machine programming teaches one to trust — rank one: two gentlemen sharing any quantity of registers cannot solve consensus between just the two of them, and the proof is an old friend in new rooms — bivalence, with writes at distinct registers as merely-elsewhere events that commute, a read learning only what the schedule permits, and the fence held in shared memory precisely as it was held in the Post. Chapter 3's machinery was never about message passing; it was about asynchrony and one possible absentee, and it follows the pair wherever they go. Higher up: test-and-set, the humble queue, the stack — rank two, able to settle a pair, helpless before three. And at infinity: compare-and-swap, wearing silicon in every processor, and consensus itself, which settles any house whatsoever. The theorem that earns the movement its title — *consensus is universal*: own the bit, and you own the world.

Remark 4.12 (Herlihy's hierarchy; universality). Assign each object type the largest n for which it solves n-process wait-free consensus: read/write registers rank 1 (the bivalence machinery of Chapter 3 transplants to shared memory: writes to distinct registers commute, a read learns only what the schedule permits, and the fence holds); test-and-set, queues, stacks rank 2; compare-and-swap and consensus itself rank ∞. The *universality theorem*: from consensus, a fault-tolerant linearizable implementation of any sequential object — agree, repeatedly, on the next operation; feed the agreed log through deterministic replicas. Statement with pointer only (Herlihy 1991); the construction in miniature is Chapter 5's opening subject, and "linearizable" is debt #13, payable in Chapter 7.

The architecture funnels here, and Chapter 5 is already written by it: one value must become a log; the log must become a service; and the services — Chubby, ZooKeeper, the captive parliaments of industry — must learn to live with tenants. Consensus in captivity; bring a lease.

4.8 The Whiteboard

For the design review you will sooner or later chair.

1. Never hand-roll consensus. The derivation above is the *short* version, and every corner it turned is a production incident in some company's private history. Rent the machinery: a maintained Raft library, a coordination service, a database that has done its Jepsen penance. The parliament is infrastructure, not a weekend project.

2. But recognize the skeleton anywhere: a total order of attempts (ballots, terms, views, epochs); two phases whose quorums overlap — a read of the past that freezes it, a write of the future bound by it; and adopt-the-highest in some costume. "Version numbers plus a majority write" with no read-and-adopt phase is the stale-ballot overwrite in embryo. Practise on machinery already in the fleet: ZooKeeper stamps every transaction with an identifier whose upper half is an epoch — a ballot number, so a deposed leader's letters die on arrival; etcd campaigns in terms; the fencing token of Chapter 5 is the skeleton's smallest bone.

3. The leader is a liveness optimization; safety lives in the quorums. Interrogate any leader-based design at exactly that joint: what breaks when two leaders exist at once? The correct answer is "throughput, briefly"; any answer involving the word "corruption" means the ledgers are trusting the throne, and thrones wobble.

4. Persistence is part of the protocol, not an implementation detail. What is fsynced before what is acknowledged is a safety question, not a tuning question; treat a hand-wave there exactly as you would treat one about the quorum size.

The register. Paid: #7 — a total order may be rented from a leader, issued at Chapter 2's mutual-exclusion protocol and paid in full: the throne stands, rented from partial synchrony, defended by ballots, priced as an optimization rather than a foundation; and #9 — partial synchrony, cashed by the Synod the moment the distinguished proposer took the chair. Issued: #12 — one value must become a log: multi-decree composition, its lock services and leases (Chapter 5). #13 — "behaves as a single machine" is owed its proper name: linearizability, defined with teeth (Chapter 7). #14 — the parliament's own membership was quietly assumed fixed all along; reconfiguration deferred, with a respectful shudder, to Chapter 5.

4.9 Problems

Tier I — Calisthenics

Problem 4.1 (club arithmetic)**.** (a) For parliaments of three, five, and seven: the quorum size, the number of tolerated absentees, and the number of distinct quorums. (b) Show that a parliament of four tolerates no more absentees than a parliament of three; conclude the standing preference for odd houses. (c) Exhibit a pairwise-intersecting quorum family on nine acceptors in which every quorum has size four (arrange the house in a three-by- three grid), and verify Lemma 4.1's *conclusion* for it directly. (d) Which chapter proofs would survive replacing majorities by your grid family? (Answer: all of them — Rem. 4.3.)

Problem 4.2 (reading the reference trace)**.** For Exhibit 4.3, tabulate after every event: each acceptor's (*promised*, *accepted*), and what is chosen. (a) At which exact event does S become chosen? (b) Construct the latest-possible arrival of a stray $\langle \textsc{accept}, 2, S\rangle$ retransmission and show it changes nothing. (c) Suppose instead a stray $\langle \textsc{accept}, 4, F\rangle$ from an abandoned ballot four arrives at Tuppy *before* Bingo's prepare: recompute the run and say what Bingo must now propose.

Problem 4.3 (the refusal drill)**.** An acceptor's ledger reads *promised* = 6, *accepted* = (4, B). For each arrival, give the reply and the new ledger: (a) $\langle \textsc{prepare}, 5\rangle$; (b) $\langle \textsc{prepare}, 6\rangle$; (c) $\langle \textsc{prepare}, 9\rangle$; (d) $\langle \textsc{accept}, 6, \mathrm{C}\rangle$; (e) $\langle \textsc{accept}, 5, \mathrm{C}\rangle$; (f) a crash and recovery, then $\langle \textsc{accept}, 6, \mathrm{C}\rangle$ again. Justify (d) against Box 8's guard ($\geq$, not $>$) and say why the asymmetry with (a) is correct.

Problem 4.4 (the adoption drill)**.** A proposer of ballot nine holds promises from three of five acceptors with confessions $\{\bot,\ (3, \mathrm{A}),\ (7, \mathrm{B})\}$. (a) What must it propose? (b) Same, with confessions $\{(2, \mathrm{B}),\ (2, \mathrm{B}),\ (5, \mathrm{A})\}$ — popularity versus strata. (c) Same, with $\{\bot, \bot, \bot\}$. (d) In (b), could B have been chosen at ballot two consistently with the rest of the execution? Could A have been chosen at five? What does the proposer *know*, and what must it merely insure against?

Problem 4.5 (up-to-date drill)**.** Five Raft logs, given as sequences of entry terms: $L_1 = (1, 2, 2)$; $L_2 = (1, 2)$; $L_3 = (1, 2, 2, 2)$; $L_4 = (1, 3)$; $L_5 = (1)$. (a) Order them by $\preceq$ (Def. 4.11), noting incomparabilities if any. (b) Which servers would vote for a candidate with log L_4? With L_3? (c) A candidate with L_4 and a candidate with L_3 stand in the same term: who can win, and what does your answer illustrate about "latest term" versus "most entries"?

Tier II — The Real Thing

Problem 4.6 (the ordering subtlety, completed)**.** Lemma 4.3, Step 1, claims the witness accepted (b, v) before promising b'. (a) Rewrite that step as a standalone lemma about Box 8: *an acceptor's confession to a*

prepare it answers is never lower than any acceptance it made before answering, and prove it from the monotonicity of *promised* and the acceptance guard. (b) Exhibit the counterexample execution when the promise is volatile (crash between reply and persist) — this is the bridge to Problem 4.7. (c) Point to the exact line of Box 8 that your proof in (a) leans on, and the exact clause of the model box. *(Full solution: completes the proof of Lemma 4.3.)*

Problem 4.7 (sabotage: the forgetful acceptor)**.** A well-meaning implementer keeps *accepted* on disk but *promised* in memory, "because it is rebuilt by traffic anyway." Three acceptors; two proposers with motions F and S. (a) Exhibit an execution in which both F and S are chosen. (b) State the invariant of Lemma 4.3 that dies, and mark the exact step of its proof that the amnesia invalidates. (c) Give the minimal repair and argue it restores Step 1 of Lemma 4.3. (d) Why does persisting *accepted* alone — the half the implementer kept — not help? *(Certified fatal per the standing rules: the double-choice execution is written out in the solutions and drawn as Exhibit 4.8.)*

Problem 4.8 (sabotage: the counting chairman)**.** Modify Box 9's commit rule to drop clause (b): a leader marks any index committed once a quorum stores it, whatever the entry's term. Five servers; the staging of Part Five. (a) Reconstruct the disaster ledger by ledger — every term, every vote (with the up-to-date check verified at each grant), every append, the fatal count, and the final overwrite of a "committed" entry. (b) Identify which of the five Raft properties survives and which dies, and reconcile with Lemma 4.5 and Theorem 4.7 (whose proof step fails without clause (b)?). (c) Show that under the true rule the same schedule is harmless: where exactly does Barmy's second candidacy now die? (d) Explain in two sentences why "wait for the entry to reach all five" is not an acceptable substitute for clause (b). *(Certified fatal: the reconstruction is written out in the solutions; Part Five's filmstrips, Exhibits 4.6 and 4.7, draw it frame by frame.)*

Problem 4.9 (the veto arithmetic, completed)**.** Theorem 4.7's proof compresses the case analysis of the up-to-date comparison. Prove the standalone claim it uses: *if voter a holds a committed entry e of term t at index i, and candidate c's log lacks e, and every leader of terms in (t, t_c) held e (where t_c is c's final log term), then $a \not\preceq c$* — splitting into $t_c < t$, $t_c = t$, and $t < t_c$ cases as in the text, and justifying the "every entry it creates sits at an index above i" claim from leader append-only. *(Solution sketch provided.)*

Tier III — For the Ambitious (starred)

Problem 4.10 (⋆ flexible quorums)**.** Let phase one use quorums from family $\mathcal{Q}_1$ and phase two from $\mathcal{Q}_2$. (a) Re-run the proof of Lemma 4.3 and mark every use of intersection; show the only requirement is $Q_1 \cap Q_2 \neq \emptyset$ for all $Q_1 \in \mathcal{Q}_1, Q_2 \in \mathcal{Q}_2$ — two phase-two quorums need never meet. (b) On six acceptors, design $\mathcal{Q}_2$ of size two and the matching minimal $\mathcal{Q}_1$; compute the fault tolerance of each phase and say what operational bet the asymmetry encodes (hint: how often does each phase run under a stable chair?). (c) Reconcile with Raft: which of its quorums could be shrunk by the same argument, and why did its authors decline? (Howard, Malkhi, Spiegelman, 2016.) *(Hints only.)*

Problem 4.11 (⋆ the family dictionary)**.** Construct the translation table Paxos ↔ Raft ↔ Viewstamped Replication: ballots, terms, views; promise, vote, START-VIEW-CHANGE; adopt-the-highest, the up-to-date veto, DO-VIEW-CHANGE state transfer; chosen, committed. (a) Where is the genuine structural difference between Paxos and Raft (which office does the learning: the proposer per decision, or the electorate per reign)? (b) Show each system's "deposed leader" letter dies by the same mechanism, and name that mechanism in one word. (c) After Howard and Mortier (2020): state one concrete engineering consequence of Raft's choice (hint: log divergence bounds, or leader hand-off cost). *(Hints only.)*

4.10 Hints

Hint (Problem 4.1)**.** (c) A row alone is too small, and two rows never meet; the cure is a seat every quorum is obliged to include. (Rows as one family and columns as another — a row meets a column always — is a different, *cross-intersecting* economy: peek ahead to Problem 4.10.)

Hint (Problem 4.6)**.** (a) Two timestamps: the acceptance write and the promise write to *promised*; both raise the same persistent variable; run the guard at the earlier one. (b) The crash erases the ordering because one of the two writes never happened.

Hint (Problem 4.7)**.** Let the first proposer complete phase one against a quorum containing the future amnesiac; deliver its accept letters late. The amnesiac reboots between promise and accept.

Hint (Problem 4.8)**.** Follow Part Five's staging exactly: terms two through five, chairs Bingo, Barmy, Bingo, Barmy. The fatal count happens in term four over a term-two entry. For (c): Gussie's final log term after the term-four append of a fresh entry.

Hint (Problem 4.10)**.** Intersection is used exactly once, in Step 2 of Lemma 4.3: the choosing quorum (phase two) against the preparing quorum (phase one). Nowhere do two choosing quorums meet — two values chosen at two ballots are excluded by the invariant, not by intersection of their quorums.

Hint (Problem 4.11)**.** (b) One word: staleness — a number that only rises, checked at receipt. (a) Ask *who* runs adopt-the-highest, and how often.

4.11 Solutions

Tier I in full (tersely); Tier II in full for Problems 4.6, 4.7, and 4.8 (the first completes Lemma 4.3's proof; the other two are the sabotage certifications), sketch for Problem 4.9; hints only for Tier III, by standing policy.

Solution (to Problem 4.1)**.** (a) Three: quorums of two; one absentee; three quorums. Five: three; two; ten. Seven: four; three; thirty-five. (b) Four needs quorums of three and tolerates one — the fourth seat buys correlation risk and no tolerance; hence odd houses. (c) Fix the centre seat of the grid. The family: each row augmented by the centre seat (the middle row, which holds it already, takes any fourth member instead). Three quorums of size four, and any two share the centre seat — Lemma 4.1's conclusion verified by construction, no counting required. (Rows and columns *without* augmentation form two families of three that always meet each other though not themselves — the cross-phase economy of Problem 4.10.) (d) All: every proof invokes Lemma 4.1 only for the intersection *conclusion* (Rem. 4.3); a family that delivers the conclusion delivers the proofs.

Solution (to Problem 4.2)**.** Ledger table (P = promised, A = accepted): initially Gussie $(2, (2, \mathrm{S}))$, others $(\bot, \bot)$. After prepares of five: Tuppy $(5, \bot)$; Gussie $(5, (2, \mathrm{S}))$; Bingo (self) $(5, \bot)$. After accepts of five: each in turn $(5, (5, \mathrm{S}))$. (a) S is chosen at the moment the *second* acceptor persists $(5, \mathrm{S})$ — a quorum of the three — regardless of when replies arrive. (b) A stray $\langle \textsc{accept}, 2, \mathrm{S} \rangle$ is refused everywhere: $2 < \mathit{promised} = 5$; ledgers unchanged. (c) Tuppy's ledger becomes $(4, (4, \mathrm{F}))$ before the prepare; his promise to five confesses $(4, \mathrm{F})$; the highest confession across the quorum is now $(4, \mathrm{F})$ over Gussie's $(2, \mathrm{S})$, and Bingo must propose F. Nothing was chosen in either history; the rule simply insures against what it cannot see.

Solution (to Problem 4.3)**.** (a) Ignore (or nack six): $5 \not> 6$. (b) Ignore: $6 \not> 6$ — prepare needs strict excess. (c) Promise nine, confess $(4, \mathrm{B})$; ledger $(9, (4, \mathrm{B}))$. (d) Accept: $6 \geq 6$; ledger $(6, (6, \mathrm{C}))$; the $\geq$ lets a ballot use the very promise it obtained — without it no ballot could ever complete. (e) Ignore: $5 < 6$. (f) Recovery

restores $(6, (4, \text{B}))$ from disk; the accept at six is honoured as in (d). The asymmetry (strict for prepare, non-strict for accept) is correct because a second prepare at the same ballot could hand the ballot's owner two different confession sets — one ballot, one reading of the room.

Solution (to Problem 4.4)**.** (a) B — highest confession $(7, \text{B})$. (b) A — the stratum at five outranks two copies of the stratum at two; popularity is not a credential. (c) Its own motion; the past is silent. (d) In (b): B at two *could* have been chosen (the two confessions might be two members of a choosing quorum) — and A at five could *not* have been chosen consistently if B was, by Lemma 4.3; but the proposer cannot distinguish "B chosen at two, A a later fossil that should be impossible" from "nothing chosen anywhere." Precisely because the first reading would make A's existence contradict the invariant, the consistent readings are: nothing chosen, or A's stratum descends from a lawful adoption chain — in either case the highest stratum is the only safe proposal. The proposer knows nothing; the rule insures against everything.

Solution (to Problem 4.5)**.** (a) $L_5 \preceq L_2 \preceq L_1 \preceq L_3 \preceq L_4$: the term-three ending of L_4 outranks every term-two ending regardless of length; among term-two endings, length orders. No incomparabilities — $\preceq$ is total on log endings. (b) For L_4: all five (every log $\preceq L_4$). For L_3: holders of L_5, L_2, L_1, L_3 grant; the holder of L_4 refuses. (c) Both can assemble a quorum of grants (L_3 from the four shorter-or- equal logs, L_4 from anyone), but whoever moves first with a quorum wins the term (Lemma 4.5); if both gather votes, the voters' one-vote rule splits the house and the term may elect nobody — randomized timeouts exist for exactly this. The illustration: L_4, *shorter* but ending later, outranks L_3 — "latest term" dominates "most entries," because terms, not lengths, encode electoral legitimacy.

Solution (to Problem 4.6)**.** (a) *Lemma.* If acceptor a accepts (b, v) at time s, and at time $s' > s$ answers $\langle \textsc{prepare}, b' \rangle$, then its confession at s' reports a ballot $\geq b$. *Proof.* At s, the guard of Box 8 (acceptor line 7) gave $b \geq \mathit{promised}(s)$, and line 8 set *accepted* to (b, v); *accepted*'s ballot coordinate never decreases (its only assignments are acceptances, each with ballot $\geq$ the standing promise $\geq$ every earlier acceptance's ballot — the variable *promised* is non-decreasing since both assignments, lines 3 and 8, only raise it, and recovery restores the persisted value). Hence at s' the stored acceptance has ballot $\geq b$, and the confession reports it. Conversely — the direction Step 1 uses — if a is known to have accepted (b, v) at *some* time and to have answered $b' > b$ at some time, the answer cannot precede the acceptance: after answering, *promised* $\geq b'$ forever (monotone, persistent), and the acceptance guard $b \geq \mathit{promised}$ would then fail for $b < b'$. (b) Volatile promise: a answers b' (promise in memory only), crashes, recovers with *promised* $= \bot$, then accepts the old (b, v) — acceptance after promise, confession to b' already sent as $\bot$. Step 1's "absurd" branch is now a real execution, and Problem 4.7 rides it to a double choice. (c) Box 8 acceptor lines 3–4 (persist, *then* reply); model box clause: recovery restores stable storage exactly.

Solution (to Problem 4.7)**.** (a) The double-choice execution — six moves, one reboot — drawn as Exhibit 4.8. Acceptors Bingo, Tuppy, Gussie; proposer one owns odd ballots (motion F), proposer two even (motion S). (1) $\langle \textsc{prepare}, 1 \rangle$ reaches Bingo and Tuppy: both promise (in memory), confessions $\bot$; proposer one lawfully issues $\langle \textsc{accept}, 1, \text{F} \rangle$; the Post dawdles with both copies. (2) $\langle \textsc{prepare}, 2 \rangle$ reaches Tuppy and Gussie: Tuppy's guard $2 > 1$ passes — promise, confession $\bot$ (honest: he has accepted nothing); Gussie likewise; proposer two, the past silent, lawfully issues $\langle \textsc{accept}, 2, \text{S} \rangle$. (3) The accept of ballot two lands at Gussie: guard $2 \geq 2$; persists $(2, \text{S})$. The copy addressed to Tuppy dawdles. (4) **Tuppy reboots.** His *accepted* $= \bot$ survives on disk; both promises he ever made evaporate with his memory. (5) The dawdling accept of ballot one lands: at Tuppy, guard $1 \geq \bot$ passes — he persists $(1, \text{F})$; at Bingo, guard $1 \geq 1$ passes — he persists $(1, \text{F})$. **F is chosen** at ballot one, quorum {Bingo, Tuppy}. (6) The dawdling accept of ballot two lands at Tuppy: guard $2 \geq 1$ passes — he persists $(2, \text{S})$. **S is chosen** at ballot two, quorum {Tuppy, Gussie}. Two values, two lawful quorums, every guard honoured by the amnesiac protocol; without the reboot, step (5)'s guard at Tuppy reads $1 \geq 2$ and the run is harmless.

(b) Lemma 4.3 dies at Step 1 — the ordering subtlety. The invariant's proof shows a witness in both quorums must have accepted ballot one *before* promising ballot two, so that ballot two's confessions report the acceptance. Tuppy is that witness, and amnesia reverses him: his acceptance of ballot one *follows* his (evaporated) promise to ballot two, and ballot two's confession set lied by omission. The invariant's witness was disarmed, exactly as Problem 4.6(b) predicts. (c) Persist *promised* before every reply (Box 8, line 3 as written); recovery then restores the promise to two, step (5)'s guard reads $1 \geq 2$, the stale accept dies, F is never chosen, and Step 1 is a theorem again. (d) Persisting *accepted* alone keeps the confessions honest about the past but not the standing bar against the past's *return*: the fatal guard in step (5) consulted *promised*, not *accepted*. Both halves of the ledger are load-bearing.

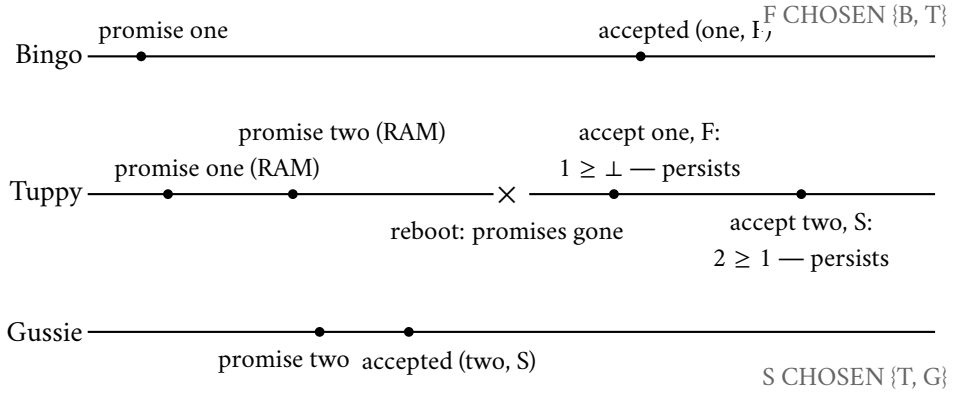


Exhibit 4.8 (the forgetful acceptor's double choice; Solution 4.7). Tuppy's promises live in memory and die in one reboot; his ledger of acceptances is honest throughout, and the parliament chooses two by-laws anyway. Both halves of the ledger are load-bearing. Fatality certified.

Solution (to Problem 4.8)**.** (a) The reconstruction, drawn frame by frame as Exhibit 4.6; servers Bingo, Tuppy, Gussie, Barmy, Oofy; log slots shown as (index: term). *Term two*: Bingo leads (votes: Bingo, Tuppy, Gussie, say — all logs empty, every $\preceq$ check passes). Appends (2: two) after an initial (1: one) common entry; replicates slot two to Tuppy only. *Term three*: heartbeats lapse; Barmy stands; voters Gussie and Oofy hold logs ending (1: one), Barmy likewise — $\preceq$ passes; Barmy leads, appends (2: three) locally, crashes before any replication (his debut, honoured). *Term four*: Bingo stands; voters Tuppy (ends (2: two) — equal to Bingo's ending, equal length: passes), Gussie (ends (1: one): passes); Bingo leads, resumes replicating (2: two) to Gussie — slot two now on {Bingo, Tuppy, Gussie}, a quorum. **The sabotaged rule marks index two committed.** Bingo pauses. *Term five*: Barmy recovers, stands; his log ends (2: three); voters Gussie and Oofy end (2: two) and (1: one): two $<$ three, both must grant. Barmy leads and replicates (2: three) everywhere; the consistency check at slot one passes, the conflicting (2: two) suffixes are deleted — the "committed" entry is expunged from every log. (b) Election safety survives (every term had one leader — Lemma 4.5 never depended on logs). Leader completeness dies: term five's leader lacks a "committed" entry — precisely because the committed-in-term-t premise of Theorem 4.7 was counterfeit: the count in term four was of a term-*two* entry, and the proof's witness argument needs the voter's veto to see the commitment's term, which it cannot when the count crosses terms. (c) Under the true rule, Bingo in term four first appends and replicates (3: four) to {Bingo, Tuppy, Gussie}; that count is of his own term: index three commits, and index two commits beneath it by Lemma 4.6. Barmy's term-five candidacy then meets Gussie ending (3: four) $>$ (2: three): refused; with only Oofy and himself, no quorum — the candidacy dies at Gussie's veto, exactly where Theorem 4.7's witness stands (Exhibit 4.7). (d) Waiting for all five confuses durability with commitment in the other direction and surrenders availability: one slow or dead server

(Gussie on an ordinary day) blocks all progress — and even then the entry's term still loses the up-to-date comparison to any later-term log, so the window merely narrows. Terms, not counts, are what elections read.

Solution (to Problem 4.9)**.** (Sketch.) a's log ends at term $\geq t$ (it holds e; anything it appended later has term $\geq t$ by term monotonicity along one server's history). Case $t_c < t$: term comparison fails immediately. Case $t_c = t$: c's entries of term t were created by t's unique leader (Lemma 4.5); by Lemma 4.6, c's log is a prefix-consistent portion of that leader's log; lacking e at index i forces c's length $< i$; a's length $\geq i$; with equal final terms, length decides against c. Case $t < t_c$: t_c's leader held e (induction hypothesis of Theorem 4.7); it appends only above its own log (leader append-only), so every term-t_c entry sits at index $> i$; c holding a term-t_c entry at its end and passing the consistency chain beneath it (Lemma 4.6) therefore holds everything of that leader's log through some index $> i$ — including e: contradiction with "lacks e," so this case cannot occur at all (rather than failing the comparison, it fails the premise).

Solution (to Problems 4.10 and 4.11)**.** By standing policy: hints only (the Hints section above). For Problem 4.10(b), the arithmetic checkpoint: with $|Q_2| = 2$ on six acceptors, every Q_1 must meet every pair — Q_1 must have at least five members.

4.12 The Reading

Lamport, "Paxos Made Simple," ACM SIGACT News, 2001. This chapter's text. The abstract is one sentence; quote it with relish. Read §2.2 against Lemma 4.3 — his P2c is our invariant, derived there in the same spirit of forced moves — and §2.3 against Remark 4.10. The persistence requirement is stated in one quiet sentence; you now know its full price (Exhibit 4.8).

Ongaro and Ousterhout, "In Search of an Understandable Consensus Algorithm," USENIX ATC, 2014. Figure 2 is Box 9's source; §5.4.1 is the up-to-date veto; §5.4.2 is Problem 4.8 in the authors' own staging; the user study is §9's dare. The extended version and Ongaro's thesis (Stanford, 2014) carry the full safety argument our Theorem 4.7 follows.

Lamport, "The Part-Time Parliament," ACM Transactions on Computer Systems, 1998. Unassigned; unlocked. Read it after the others, for pleasure — the way one reads a founding document once the constitutional law is understood — and notice the jokes were, in fact, load-bearing.

Oki and Liskov, "Viewstamped Replication," PODC 1988. For the pleasure of watching the future arrive early: views, view changes, the adoption obligation — the same machine, first in print.

For the starred doors. Howard, Malkhi, and Spiegelman, "Flexible Paxos: Quorum Intersection Revisited," OPODIS 2016 (Problem 4.10). Howard and Mortier, "Paxos vs. Raft: Have We Reached Consensus on Distributed Consensus?," PaPoC 2020 (Problem 4.11). Herlihy, "Wait-Free Synchronization," ACM TOPLAS 1991 (Rem. 4.12). And Chandra, Griesemer, and Redstone, "Paxos Made Live" (PODC 2007), for what the island cost to industrialize — assigned properly in Chapter 5.

Standing companion: Kleppmann, chapter 9. His treatment of total order broadcast is Chapter 5's bridge; his pages on distributed transactions wait for Chapter 8. His consensus section pairs with the derivation above as a second telling in calmer prose.

Chapter 5

Consensus in Captivity

Companion to Episode Five. Consensus put to work: one value becomes a log and the log becomes the replicated state machine, the identical-replicas theorem proved in a page; the parliament amends its own membership without dissolving its own mathematics; the two landlords, Chubby and ZooKeeper, and the economics of renting coordination; the resurrection problem and its two exorcists — the lease, proved safe from drift-bounded clocks alone, and the fencing token, by which the store rather than the lock has the last word; the Post taught to forge at last — the three-node Byzantine impossibility, written out here with its worlds side by side; the $3f+1$ bound and the arithmetic of perjured witnesses; PBFT boxed; Nakamoto's lottery given its one respectful movement; and the senior discipline of when not to convene the parliament at all.

5.1 Part One — One Value Becomes a Log

On the second of November, 2020, a portion of Cloudflare's control machinery — the systems by which its operators configure and observe the global network — went dark for several hours, and the postmortem published within the week bears a title one does not expect outside the theoretical literature: "A Byzantine Failure in the Real World." The anatomy, in this course's vocabulary. At the heart of the affected system sat a small parliament — an etcd cluster, three members, running Raft, precisely Chapter 4's machinery, industrious and correct — and in the network around it a switch began to fail: not honestly, not totally, but *partially*, and asymmetrically. Member one could hear member two; member two could not reliably answer; the third saw another weather entirely. The parliament was built to survive members *dying*; nobody had promised it members that were *half-audible* — a house in which every gentleman holds a different, internally consistent, mutually contradictory account of who can hear whom. The cluster did not split its brain — the crowded club held, as the mathematics said it must — but it thrashed: elections that almost completed, leaders deposed by phantom silences, terms climbing, work undone. Safety, the theorem-grade guarantee, survived without a scratch; everything *around* the guarantee had a very bad day, because the world had wandered outside the model the guarantees were purchased in.

Two footnotes to the incident, both load-bearing. First, the parliament at its centre deserves its landing: etcd — spoken *et-see-dee*, a pronunciation the course's lexicon defends with its life — is the coordination service keeping, among much else, every Kubernetes control plane honest; whoever has operated Kubernetes has operated a rented Raft parliament, whether or not the tenancy agreement was ever shown to him. It is the direct descendant of this chapter's two landlords: the Chubby pattern rebuilt in the open, with Raft where Chubby keeps Paxos. Second, the postmortem's authors reached, correctly, for the oldest word in this corner of the field — *Byzantine*, forty years old, entering the course through the front door in Part Five — and yet the incident is *not*, in the strict sense, a Byzantine story: no component lied with intent; a switch

merely failed in a way that made honest machines *look* inconsistent to one another. The gap between the parliament in the proof and the parliament in the fleet is this chapter's whole subject. Chapter 4 built consensus in the abstract; here it goes into captivity: put to work, rented out, operated, resurrected wrongly, and finally confronted with correspondents of genuinely low character.

To work first. The Synod of Chapter 4 decides one by-law at a price of two round trips of the Post, and then stands complete, its single page of ledger filled, forever; a shop needs a new by-law every few milliseconds. The composition is the obvious one: number the decisions. Slot one, slot two, slot three — the *docket*, formally the log positions, slot k decided by its own instance k of the Synod, independent in principle, the whole sequence constituting the *log*: the ordered record of everything the parliament has ever resolved. Done naively this is extortionate — two round trips and a quorum's disk writes per slot, every slot fighting its own phase-one battle — but the remedy was earned in Chapter 4's fine print: the chairman. Let the house elect a distinguished proposer — today, as ever in the happy case, Bingo has the chair — and let him run phase one *once*, a single ballot number covering every slot at a stroke; the house promises once, confesses per slot whatever fragments it holds, and thereafter, for as long as the chairmanship endures, every new decision costs a single accept round. That arrangement — the Synod per slot, phase one amortized under a stable chair — is *multi-Paxos*: the thing actually deployed wherever the word Paxos is spoken with a straight face. Raft, as Chapter 4 observed, is structurally the same animal in different tailoring — the chairmanship made load-bearing and the log made primary.

Watch the machine run once at full correspondence, fair weather first. Bertie submits two items of business in quick succession — debit account seven by ten pounds; credit account nine by ten pounds. Bingo assigns slots from his docket — the debit takes slot forty-one, the credit slot forty-two — and writes to the house: accept, ballot seven, each slot, each motion. Tuppy's ledger and acknowledgments give each slot its majority — the chair does not wait for unanimity, only for clubs; Gussie's replies arrive presently — and both motions are chosen; the state machines apply forty-one then forty-two, in docket order, at every member as the news reaches it; one round trip per decision, the two decisions travelling in the same envelopes as often as not. Now the foul weather, where the composition earns its subtleties. Bingo goes quiet mid-stream: slot forty-one reached a majority, slot forty-two's accept reached only Gussie before the silence fell. Tuppy suspects in due course and prepares ballot eight for all slots; the house confesses per slot. For forty-one: accepted at seven, the debit — adopt and re-ratify, Chapter 4's invariant working slotwise. For forty-two: Gussie confesses the credit at seven — adopt the highest; the credit survives its author's silence. Had the weather been one letter crueller — forty-two's accept reaching *nobody* — the confessions come back empty and the docket has a hole; and the hole cannot stand, because the state machines apply the log *in order* — entry k only after entries $1..k{-}1$ — and an unfilled slot dams the river: every decision downstream, however thoroughly chosen, sits inapplicable until the hole resolves. So the new chair fills it forthwith — with the lost motion if any fragment survives in some confession, otherwise with a formal *no-operation*: the identity command, filling an unresolvable slot, moving nothing, letting the river flow. Undignified, perfectly lawful, and the direct ancestor of a rule met in Raft dress: the fresh leader who must commit an entry of his own term before pronouncing on inherited ones is performing exactly this rite — assert the new reign across the docket's tail, then let the past flow through.

Protocol — Box 10. Multi-Paxos essentials (checked against "Paxos Made Simple" §3 and "Paxos Made Live")

1: **leadership:** the chair holds ballot b prepared *for all slots*: one $\langle \textsc{prepare}, b \rangle$; promises carry, per slot, any accepted fragments
2: **steady state:** client command c arrives; chair assigns the next free slot k; sends $\langle \textsc{accept}, b, k, c \rangle$;

chosen at a quorum of ⟨ACCEPTED, b, k⟩

3: **apply rule:** each replica applies slot k only after slots $1..k-1$ — a hole dams the river

4: **succession:** new chair prepares $b' > b$ for all slots; per slot adopts the highest confessed fragment; unresolved slots below the frontier are proposed as NO-OP

5: **batching and pipelining** amortize the round trip; reads are served through the log, or under the leader lease of Box 11 (debt #16 fine print: Chapter 7)

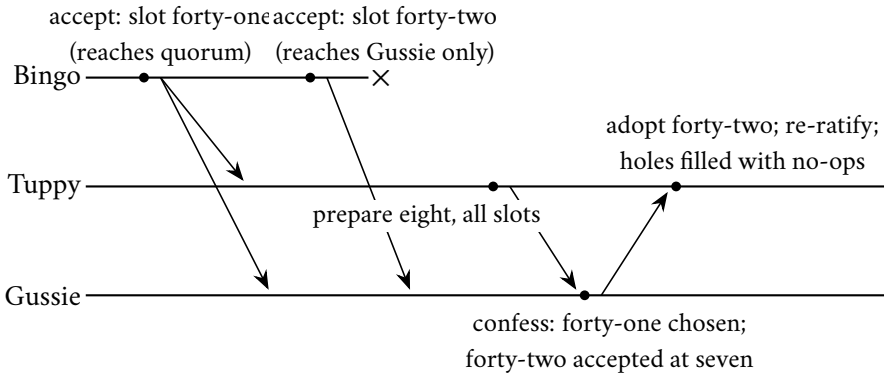


Exhibit 5.1 (multi-Paxos succession; the docket with a hole). Bingo's chairmanship ends mid-stream; Tuppy prepares a higher ballot for all slots at a stroke, adopts the surviving fragment of slot forty-two from Gussie's confession, and closes any void slots with no-operations so the state machines' river can flow. Raft's commit-own-term rite is this rite in other dress.

The chapter's models, fixed now, before the first formal claim — the second paragraph a promissory note for Part Five, where the charter's mercy clause is revoked.

Model of this chapter

For the log, the landlords, and the exorcists (Defs. 5.1–5.4, Thms. 5.1–5.3, Lemma 5.2). As Chapter 4: asynchronous; crash-recovery with stable storage; quorums are majorities of the current configuration, membership changed only through the log. For the lease theorem *only*, a physical-clock purchase, small and declared: every clock's *rate* lies within a factor $[1-\rho,\ 1+\rho]$ of true time, $0 \le \rho < 1$; nothing whatever is assumed about offsets — no clock knows the hour.

For the Byzantine impossibility (Def. 5.5, Thms. 5.4–5.5). Deliberately generous timing, as in Chapter 1's impossibility: synchronous rounds, no message ever lost — the theorem is about *forgery alone*, and a fortiori kills the asynchronous case. Failures: up to f processes Byzantine (arbitrary state machines, full collusion, adversary-chosen). Links are *oral*: delivery is reliable and the immediate sender of each message is known to its receiver, but a relayed content is entirely the relayer's choice — hearsay cannot be audited. (The *signed-messages* variant — unforgeable, verifiable relays — changes the theorem: Problem 5.11.)

And now the christening the course has owed since Chapter 2: why the *log* — not the consensus, the log — is the character the industry built its temples around. Because of the acorn, grown to full oak: *the replicated state machine*. Take any deterministic machine — a key-value store, a lock table, a bank ledger, anything defined one operation at a time. Run one copy on each gentleman; feed every copy the same operations in the same order — which is precisely what the log provides — and the copies cannot help but remain

identical, forever, having at every step the same state, the same input, and the same rulebook. Lamport tossed the observation off in a paragraph in 1978; Schneider's 1990 tutorial in ACM Computing Surveys made it doctrine, and the doctrine is the industry's entire architecture: *fault tolerance is not a property added to a service; it is a property inherited by driving the service from a replicated log.* The service does not replicate; the log replicates, and the service merely reads it. Agreement is needed once, at the narrow waist — slot by slot, on the order of operations — and everything above the waist gets consistency free, by determinism: Chapter 4's universality theorem wearing overalls — agree on the sequence, and any machine whatsoever rides along. Chapter 10 gives the log its own feature; Chapter 12 will find it feeding gradients to accelerators. Here it is born properly and named.

Definition 5.1 (replicated state machine)**.** A *state machine* is a deterministic transition function $\delta(s, c) = (s', o)$: state and command in, state and output out. A *replicated state machine* over a log: each replica holds a copy of δ, an applied-prefix length, and applies committed log entries strictly in slot order, each exactly once. Commands must be pure in (s, c): any environmental input (time, randomness) is fixed at proposal and carried inside c.

The constitutional theorem is proved by the shortest useful induction in the volume: induction on the log prefix, determinism carrying equality one entry forward.

Theorem 5.1 (identical replicas)**.** *If two replicas of the same state machine have applied prefixes of the same committed log, then after applying prefixes of equal length their states and emitted outputs are identical; in particular, replicas that have applied unequal lengths differ only by not-yet-applied suffixes, never by divergence.*

Proof of Theorem 5.1. Induction on prefix length k. Base: equal initial states. Step: both replicas hold state s_{k-1} (induction hypothesis) and apply the same entry c_k — the same log, the same slot, and committed entries are unique per slot (Chapter 4's safety theorems for whichever family runs the log). Determinism of δ and purity of c_k give the same (s_k, o_k) at both. Exactly-once, in-order application ensures k increments without gaps or repeats; divergence would require a first differing slot, contradicting the step. □

The trap in the constitutional theory, flagged because production falls into it weekly: the theorem's engine is *determinism*, and determinism is a stricter diet than programmers assume. Wall-clock timestamps consulted mid-apply, random draws, iteration over collections whose order the language declines to promise, reads of the local machine's environment — each is a hidden second input, different at every replica, and each is a slow leak: copies that agree on every slot and drift apart anyway, the divergence surfacing weeks later as an inexplicable audit failure, because the consensus machinery was never at fault. The discipline is Def. 5.1's final clause: anything nondeterministic is decided *once*, by the leader, at proposal, and written into the log entry itself — the timestamp chosen at proposal time, the random draw made before sequencing — so that what the replicas apply is pure clockwork; Problem 5.5 makes it a drill. And one clause banked before the parliament learns new tricks: with all client traffic sequenced by the chair, ratified by the house, and applied by the copies, the ensemble now behaves, from outside, *as if it were a single machine* — one that survives the death of any minority of its parts. That phrase does enormous, undefined work; Chapter 7 pays the debt in full, with a definition sharp enough to test systems against.

5.2 Part Two — Amending the Parliament

A parliament that cannot change its own membership carries a countdown clock: machines age, racks retire, capacity moves. And membership is not configuration in the operational sense — a line in a file — it is *the definition of the quorum*, the very denominator of the crowded club; change it carelessly and one does not

misconfigure the system but dissolves the mathematics. The naive disaster, staged in one breath: the house of five — Bingo, Tuppy, Gussie, Barmy, Oofy — is to become a house of seven; the new roster is announced by ordinary letter; and for one terrible interval some gentlemen believe the house is five while others believe it is seven. A majority of the old five is three; a majority of the new seven is four — and the four may be assembled entirely from gentlemen who have heard, the two newcomers and two of the old guard, while the three are drawn from those who have not. Arrange the letters maliciously — the Post will oblige — and two disjoint clubs each lawfully consider themselves a quorum: two chosen values, two leaders, the full split brain, achieved without a single machine failing, purely by disagreement about *what the parliament is*. The membership is itself a piece of replicated state and must be changed with the same ceremony as any other — through the log — with the added subtlety that the state in question defines the machinery doing the changing: the parliament must vote on the shape of the parliament, using quorums, while the meaning of "quorum" is the thing being voted.

Two disciplined solutions, both in production. The first is *joint consensus*, Raft's original proposal and multi-Paxos folklore: transition through an interregnum in which decisions require majorities of *both* houses — the old five and the new seven, simultaneously. No decision passes without a club from each roster, the two eras cannot contradict one another, and once the joint arrangement is itself committed, a second log entry retires the old house. Correct, general, and fiddly — two overlapping quorum disciplines live in the code at once. The second is *one-server-at-a-time*: permit only single additions or removals, and rest on a pleasing little lemma, small enough to fit in a breath, which polices the eras exactly as the crowded club has policed everything since Chapter 4.

Lemma 5.2 (consecutive houses police each other)**.** *Any majority of a set of n members and any majority of a set of $n+1$ members that extends it (or shrinks it by one) intersect.*

Proof of Lemma 5.2. Let $|Q| > n/2$ and $|Q'| > (n{+}1)/2$ with rosters differing by one member; the combined roster has at most $n+1$ members. Integrality sharpens the majorities: $|Q| \geq \lfloor n/2 \rfloor + 1$ and $|Q'| \geq \lfloor (n{+}1)/2 \rfloor + 1$, and since $\lfloor n/2 \rfloor + \lfloor (n{+}1)/2 \rfloor = n$ for either parity, $|Q| + |Q'| \geq n + 2 > n + 1 \geq |Q \cup Q'|$; the excess is the intersection. □

The shared member means consecutive configurations cannot contradict each other in ignorance, so no interregnum machinery is needed at all — provided, and only provided, the changes truly come one at a time, each committed through the log before the next begins. This simpler scheme is what most modern deployments actually run, with one honest asterisk, flagged per the standing rules of this course: its original published form contained a subtle interaction between a half-completed membership change and a concurrent leadership change, found after publication and corrected by the scheme's own author — The Reading gives the trail. The moral is not that the scheme is unsound; patched, it is sound and standard. The moral is that *this corner of the protocol is where the dragons winter*. Membership interacts with leadership interacts with the log's tail; the new member must be caught up before he counts, lest quorums include gentlemen who know nothing; the removed member must somehow be told to stop campaigning, lest he haunt the elections he no longer belongs to — every clause a place where review headroom goes. When a design document proposes a bespoke reconfiguration scheme, that is the page to read twice.

5.3 Part Three — The Landlords

Almost nobody who needs consensus runs their own parliament, and this is not laziness but Chapter 4's whiteboard correctly applied: consensus is rented, from a small number of intensively engineered coordination services, two of which are historical monuments worth knowing properly. The first is Chubby, from

Google, described by Burrows at OSDI 2006 ("The Chubby Lock Service for Loosely-Coupled Distributed Systems") — assigned in The Reading as literature as much as engineering, one of the field's great candid documents. Chubby is, on paper, a lock service: a small filesystem-like namespace of tiny files, on which clients take locks and store small metadata, backed by five replicas running Paxos. The paper's confessions are the treasure. Burrows reports, with the weariness of a man who has read the tickets, what the service became in its tenants' hands — a name service, a rendezvous point, a home for configuration that absolutely must not be lost, an election mechanism for client fleets: the whole pattern this course calls the *coordination kernel* — locks, leases, elections, membership, and small hot metadata, rented from one intensively guarded parliament rather than rebuilt badly in every service that needs them. He reports the outages, in a table, with causes — most tracing not to Paxos but to everything around it: operator error, misconfiguration, client abuse, the janitorial reality this chapter is named for. And he reports the design consequence Google drew, since become doctrine: concentrate the hard agreement problem in one small, obsessively engineered service, and let ten thousand tenants inherit its guarantees through leases and locks rather than running ten thousand parliaments. The economics of consensus: the machinery is expensive to operate correctly, so operate it once, well, and sell coordination retail.

The mechanics of the tenancy are Part Four's lease already in action, wholesale. A Chubby client does not take out one lease per lock; it maintains a single *session* with the landlord — one master lease, the one client–landlord liveness contract under which every lock, every cached file, every open handle shelters, kept alive by periodic exchanges. Caching is why the arrangement scales: clients cache the tiny files aggressively and consistently, because the landlord tracks who caches what and, before committing any change, first posts invalidation notices and waits for the tenants to acknowledge — the change blocks until the caches are provably clean. Master lease, tracked caches, blocking invalidations: three decisions that turn a five-machine parliament into a service ten thousand clients can lean on without flattening it. And one operational tooth, worth remembering at three in the morning: when a session's lease enters jeopardy and recovers, or fails outright, the client library tells the application, in effect, *everything you cached is now suspect and every lock you held may be gone* — and applications that ignore that callback are Part Four's resurrection problem, volunteering.

The second landlord is ZooKeeper, published by the Yahoo team — Hunt, Konar, Junqueira, and Reed — at the USENIX technical conference in 2010, and subsequently the open-source world's coordination kernel of record: the metadata heart of Kafka, Hadoop, and a decade of operated systems. The tenancy pattern is Chubby's; the furniture *is* the product. The namespace is a little tree of znodes — tiny files, kilobytes at most, for this is a parliament, not a warehouse — and three modest mechanisms compose into everything else. An *ephemeral* node lives exactly as long as the session that created it: the machine dies, or pauses past its heartbeats, and the node evaporates — presence itself, as data. A *sequence* node is created with a number appended — one that only rises, courtesy of the log underneath. A *watch* is a subscription: tell me when this node changes, once, cheaply. Assembled, they make the smallest election protocol in production service. Every candidate creates an ephemeral sequence node in the election directory: Bingo's request comes back stamped candidate seven; Tuppy's, candidate eight; Gussie's, candidate nine. The rule: the lowest number reigns. Bingo, seeing no number beneath his own, takes the chair. And the others do not — this is the touch worth admiring — crowd around watching the throne; each watches only the candidate immediately beneath himself: Gussie watches Tuppy; Tuppy watches Bingo. When Bingo's session dies, his node evaporates; the landlord notifies exactly one tenant — Tuppy — who checks, finds nothing beneath him, and reigns. No thundering herd at the moment of succession, no polling, and the whole edifice — presence, order, notification — is three primitives deep, every one a rental of the log beneath. That is what a coordination kernel is: not a feature list but a small set of correct atoms from which tenants compose their own protocols, mostly without ruin.

The protocol underneath, Zab, is the family's fourth member, and its distinction from Raft is instructive rather than cosmetic. Its stamp first: every proposal carries a *zxid* — the pair (epoch, counter), compared lexicographically, epoch first — and a follower discards anything from a stale epoch on sight. Ballot, term, view, zxid: the stamp has now been seen four times in four dialects, and the Lamport clock's salary needs no further announcing; consider it tenured. The distinction lies in what the protocol promises about *order*. Zab is engineered as a *primary-order atomic broadcast*: its contract is that everything a given primary proposed is delivered in exactly the order that primary proposed it, and that a new epoch begins only after the new primary has provably absorbed every delivery the old epochs committed — recovery first, then broadcast, as strictly separated phases. Raft achieves the matching end state with its interleaved machinery — the up-to-date veto and log matching, policing continuously. The engineering flavour differs for a reason: Zab was built to sit under a primary-backup database (ZooKeeper's own), where the primary's proposal order *is* the database's semantics and must be preserved exactly through failover; Raft was built to be one teachable algorithm. Problem 5.9 sets the two recovery disciplines side by side and marks where each spends its guarantees. The family census thus completes: Viewstamped Replication first in print, Paxos first in folklore, Zab first in your production fleet, Raft first in the textbooks — one idea, four working dialects. And one sentence of connective tissue, planted deliberately: everything the landlords rent — every lock, every election, every ephemeral node — is ultimately a *claim about time*: who holds what, until when, according to whom. Which delivers us directly into the crypt.

5.4 Part Four — The Resurrection Problem

The most instructive horror story in production distributed systems, told first as the composite it usually is — assembled from familiar parts, per the postmortem clause — and then dismantled. Bertie's firm runs a payments pipeline; to prevent double-processing, a worker must hold the lock — rented, naturally, from the coordination service — before touching an account. Bingo, a worker, acquires the lock for account seven and begins his business. And then Bingo *pauses*. Not crashes — pauses: a garbage collection of heroic length, a virtual-machine migration, a laptop lid of the metaphorical kind; the reasons are legion and all of them are Chapter 1's bath. The coordination service, hearing no heartbeat, does exactly what it is designed to do: declares Bingo's session dead, releases the lock, and grants it to Tuppy, who begins processing account seven. And then Bingo *wakes up*. His pause, from his own side, was nothing at all — a skipped beat, invisible; no exception fired, no flag was raised; his memory says, with perfect internal consistency, *I hold the lock for account seven*, and he resumes mid-instruction, writing to the very account Tuppy is halfway through. Two writers, one account, and every component behaved as designed: the service timed out a silent client, as it must; Bingo trusted his own memory, as programs do. The corpse walked back in — Chapter 1's exact phrase, three chapters early — and the wrongly condemned, restored to life, carries all his old authority and none of the news.

No tuning fixes this. Lengthen the session timeout and every legitimate failover in the fleet slows; shorten it and resurrections multiply; and *no* finite timeout eliminates them, because the pause is unbounded — that is what asynchrony means, and Chapter 3 proved the confusion underneath, crashed-versus-slow, a conservation law, not an engineering shortfall. Nor does it present honestly in production; the presentation is designed by malice's cousin, coincidence, to mislead. The double-write is rare, the pause that caused it left no error anywhere, and the corruption surfaces hours or days later, in reconciliation, wearing the costume of an application bug: the investigating engineer finds the lock service blameless — it *was* blameless — finds both workers' logs internally coherent, and files the ticket under mystery. The tell, when one learns to look: a session-expiry event on one machine, bracketing a gap in its own logs, at the exact timestamp the service granted the lock elsewhere. Every seasoned operator has one of these in the drawer; the drawer now gets a

label — and the debt register's promised instalments on the managed confusion fall due here, paid in two instruments.

Instrument the first: the *lease*. A lock, as naively conceived, is a permission without an expiry — and a permission without an expiry, granted over the Post to mortals who may pause indefinitely, is exactly the corpse's charter. So the landlords do not grant locks; they grant leases: permission *for a bounded time*, renewable by heartbeat — and, the delicate part, the bound is enforceable *without synchronized clocks*. Chapter 2 forswore pretending the gentlemen's clocks agree, and the lease argument never compares two clocks: it compares two *durations*, and assumes only that no clock runs at a wildly wrong *rate* — the drift bound ρ of the model box, a far humbler purchase than agreement on the time of day. The grantor starts his stopwatch at the grant and will not re-issue until a full period T has elapsed by his own watch; the holder starts his stopwatch at the *request* — earlier than any grant could have occurred, a deliberately pessimistic anchor — and abandons the lease when a discounted horizon has elapsed by his: the *discount* — formally, the horizon $T(1-\rho)/(1+\rho)$ measured from the request. Run the arithmetic once with a ten-second period and a drift of one part in a hundred — both watches possibly wrong about the hour by minutes, neither wrong about a duration by more than the bargain, the two true-time windows kissing and never crossing at the price of a sliver of tenancy sacrificed to drift; Problem 5.2 re-runs it with pen and paper. Theorem 5.3 proves it in general: the holder's true-time window is contained strictly inside the grantor's true-time reissue guard — pessimistic anchor plus drift discount; rate error only, never offset.

Definition 5.2 (lease)**.** A *lease* with period T is granted by a grantor (itself replicated; the grant is a log entry) to at most one holder at a time. Protocol as Box 11: the grantor measures T from the grant by its own clock and will not re-grant earlier; the holder measures a discounted horizon from its *request* by its own clock and renounces all authority when the horizon passes.

Theorem 5.3 (lease safety under bounded drift)**.** *Under the drift bound ρ, with holder horizon $H = T \cdot (1-\rho)/(1+\rho)$ measured from the request: at no instant of true time do a holder still trusting its lease and a grantor having re-granted (or a successor holder) coexist. Mutual exclusion in calendar time from rate bounds alone; no clock offsets are assumed or used.*

Proof of Theorem 5.3. Let the holder post its request at true time t_r, the grantor record the grant at true time $t_g \geq t_r$ (the request precedes the grant: the grant is the grantor's reaction to it), and let ρ bound both drift rates.

Holder's horizon, in true time. The holder counts H on its own clock starting at t_r. A clock running fast by the full factor $1+\rho$ accumulates H on its dial after only $H/(1+\rho)$ of true time — but that ends the tenancy early, which is safe. The dangerous direction is a slow clock: dial time H is reached after at most $H/(1-\rho)$ of true time. So the holder trusts its lease during at most

$$\big[\, t_r,\ \ t_r + H/(1-\rho) \,\big] \;=\; \big[\, t_r,\ \ t_r + T\,(1-\rho)/\big((1+\rho)(1-\rho)\big) \,\big] \;=\; \big[\, t_r,\ \ t_r + T/(1+\rho) \,\big].$$

Grantor's guard, in true time. The grantor counts a full T on its own clock from t_g before re-granting. A fast grantor clock is the dangerous direction: dial time T arrives after at least $T/(1+\rho)$ of true time. So no re-grant occurs before $t_g + T/(1+\rho)$.

No overlap. The holder's trust expires by $t_r + T/(1+\rho) \leq t_g + T/(1+\rho)$, the earliest possible re-grant instant — with strict slack whenever the request-to-grant delay $t_g - t_r$ is positive, which it is by any positive Post delay. A successor's authority begins no earlier than the re-grant. Hence no true-time instant hosts both an old trusting holder and a new grant. (The pessimistic anchor at t_r is what absorbs the unknowable request-to-grant delay; Problem 5.7 shows anchoring at grant-receipt is fatal.) □

It is the first real guarantee this course has bought from physical clocks, and the invoice is worth reading twice: mutual exclusion in calendar time, purchased with a bound on *rate* error, not *offset* error. Chapter 8 goes shopping in the same market with a vastly larger budget. But the lease alone has not exorcised the corpse — and this is the step the industry forgets.

Remark 5.3 (what the lease does not fix). Theorem 5.3 guards the *schedule*; it cannot make a paused process consult its watch. The holder's renunciation is a check in the holder's code, and the holder may pause between check and effect. The residual window is closed only at the resource: Def. 5.4 and Problem 5.6. Enforcement belongs at the point of effect.

Instrument the second: the *fencing token*, argued with particular clarity by Kleppmann in the great distributed-locking debate of 2016 — the rare industry quarrel conducted at theorem grade. A popular recipe called Redlock proposed taking locks by racing timestamped grants across several independent stores, its safety resting, on inspection, on bounded pauses and well-behaved clocks — assumptions now recognizable as purchases, not defaults, and purchases the recipe had not priced. Kleppmann's essay "How to Do Distributed Locking" made the case; the counter-essay from the recipe's author made it a debate (both sides preserved in The Reading); and the durable residue is the insight before us: the last line of defence cannot live in the lock holder, because the lock holder is the party whose sanity is in question. It must live in the *resource*. Every lease the landlord grants carries a number of the smiling kind — one that only rises: formally a monotone grant counter z, incremented at every grant, the smallest bone of the skeleton. Every write the holder issues is stamped with its token; the store keeps, per protected object, a high-water mark hw — the highest token it has ever seen — and refuses any stamp below it. Bingo's posthumous write arrives stamped thirty-three against a mark of thirty-four and dies at the door with a form-letter rejection: the corpse may walk; the bank teller checks the date on his letters. And note the architecture of the fix, the deepest thing in this chapter this side of the Byzantines: the lock service *alone* cannot deliver the guarantee — safety migrated *into the storage interface*, one compare on one integer, and the system is safe even against clients arbitrarily late, arbitrarily confused, and arbitrarily convinced of their own authority. The store, not the lock, has the last word; the enforcement point must be the point of effect — a general law struggling to be named, and the next two chapters will keep meeting it. Ballot number, term, epoch, fencing token: one skeleton, its smallest bone now in production clothing.

Definition 5.4 (fencing)**.** Each grant carries a *token* z, strictly increasing across grants (the grantor's log supplies monotonicity). Every operation a holder issues to a protected resource is stamped with its token. The resource keeps, durably, a high-water mark hw per protected object and executes an operation iff $z \geq hw$, setting $hw \leftarrow z$; lower stamps are refused.

Protocol — Box 11. Lease grant and hold, under drift ρ

Grantor state is replicated (grants are log entries); tokens rise monotonically (Def. 5.4).

1: **holder:** record local time $r \leftarrow$ now(); send $\langle$ACQUIRE$\rangle$
2: **grantor, on** $\langle$ACQUIRE$\rangle$ with no live lease: $z \leftarrow z + 1$; log the grant; record local time g; reply $\langle$GRANTED, $z, T\rangle$
3: **holder, on** $\langle$GRANTED, $z, T\rangle$: trust the lease while $\text{now}() - r \; < \; T \cdot (1-\rho)/(1+\rho)$; stamp every resource operation with z
4: **grantor:** treat the lease as live while its local clock shows less than T since g; renewals re-run the exchange (same token) before either horizon lapses
5: **resource, on** operation stamped z: execute iff $z \geq hw$; then $hw \leftarrow z$ (durable); else refuse

Lines 1–4 are Theorem 5.3; line 5 is Def. 5.4 and closes the residual window of Rem. 5.3. After Gray

and Cheriton (1989) for the lease; the fencing discipline per the 2016 locking debate (The Reading).

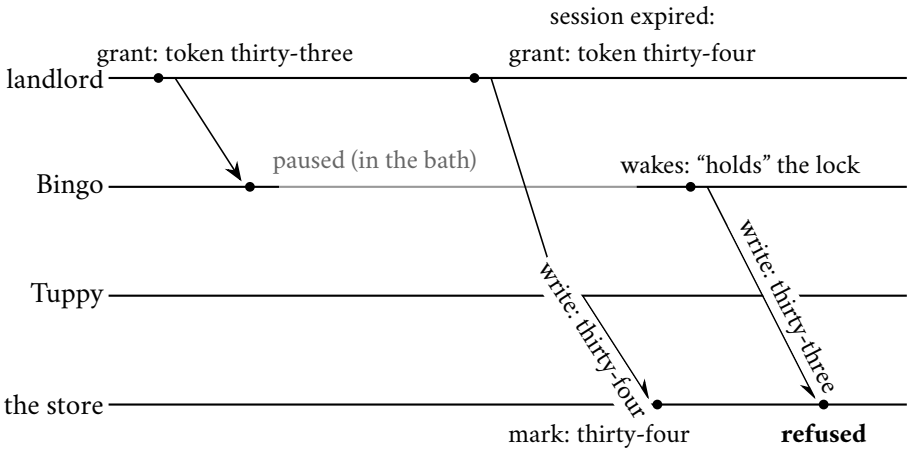


Exhibit 5.2 (the resurrection, fenced). Bingo's pause outlives his session; Tuppy is lawfully granted the successor lease with a higher token; Bingo's posthumous write dies at the store's high-water mark. The lock service never had the last word — the resource did.

5.5 Part Five — The Post Learns to Forge

The last model change of the chapter, four chapters in the promising. Every failure this course has yet permitted has been a failure of *silence* — machines that stop, letters that vanish, pauses that deceive by absence. Clause five of the Post's charter, the single mercy, has stood since Chapter 1; it is now revoked. Letters may lie about their authorship; machines may say different things to different colleagues; and a member of the parliament may be, at long last, *suborned by Aunt Agatha*: formally, *Byzantine-faulty*, arbitrary and possibly colluding behaviour, directed by an intelligence of unlimited malice and excellent penmanship toward the maximum embarrassment of the house; a member so corrupted in a staged run is *the nephew*. A corrupted machine may do anything — lie, forge, collude, selectively participate, imitate correctness for years and defect at the worst instant. We model malice not because every fault is malicious — return to the chapter's opening: a half-dead switch has no intentions whatever — but because malice is the *closure* of all misbehaviour: a guarantee that survives Aunt Agatha survives every lesser confusion, every bit-flip, every half-audible switch, as a special case. One insures against the worst tenant, and the merely careless are covered by the same policy.

The history, dates exact, because this literature founded a subfield. In 1980, in the Journal of the ACM, Pease, Shostak, and Lamport published "Reaching Agreement in the Presence of Faults": the mathematics of agreement among processors some of which may be arbitrarily faulty, containing the bound that closes this part. Two years later the same trio, reordered as Lamport, Shostak, and Pease, published "The Byzantine Generals Problem" in TOPLAS, which contributed, beyond refinements, the *framing*: divisions of an army ringing a city, some generals loyal, some traitorous, needing to agree on attack or retreat by messenger. Lamport has been candid that the story was chosen deliberately — after the Albanian generals were retired for diplomatic reasons — precisely so the result would be remembered. It was; it is perhaps the field's most successful act of marketing, and this author's views on whether presentation is load-bearing are on the record.

How the problem stands to the consensus of Chapters 3 and 4, so the furniture is arranged before the play. There, every gentleman arrived with his own opinion and the house settled on one of them; here, one designated voice — the commander — holds the input, and the task is to broadcast it faithfully through mud: every loyal participant must end holding the same account of what the commander said, even if the commander himself is the liar. The two problems are cousins with an exchange rate: run one faithful broadcast per member, so that every loyal gentleman holds the same vector of everyone's opinions — the literature's *interactive consistency* — and let each apply the same rule to the same vector; private opinions become common knowledge become agreement. Impossibility for the broadcast therefore poisons consensus, and a bound proven for three generals is a bound on the whole Byzantine estate. The success clauses are two: IC1, all loyal lieutenants agree — the traitors' final opinions being nobody's concern; IC2, a loyal commander is obeyed — the usefulness clause, without which the lieutenants could agree on retreat unconditionally and call it a day: universal cowardice in epaulettes. And the messages are *oral*: delivery unforgeable, relay forgeable — a report's content is its reporter's choice, and hearsay cannot be audited.

Definition 5.5 (interactive consistency; the generals)**.** A designated *commander* holds an order v — `attack` or `abstain` — and lieutenants exchange messages per the (oral) model box; every loyal participant must output an order. Required: **IC1** all loyal lieutenants output the same order; **IC2** if the commander is loyal, that common output is the commander's order. (With `abstain` read as "retreat," this is Lamport–Shostak–Pease 1982, oral-messages form.)

The celebrated first question: with three participants, one of whom may be a traitor, can it be done? Three gentlemen, one liar, majority intact — every intuition from the crash-stop world says comfortably yes. The answer is no, and the proof — the doppelgänger returned exactly as promised, this time carrying forged letters — traps a loyal lieutenant between two worlds, twice over: Tuppy's pair compels attack via IC2, Gussie's mirrored pair compels retreat via IC2, and the world in which both lieutenants are loyal violates IC1. The bookkeeping needs only three worlds, the third built as the second's mirror.

Theorem 5.4 (three cannot tolerate one; LSP 1982)**.** *With three participants of which one may be Byzantine, no protocol — of any length, in the oral model — satisfies IC1 and IC2.*

Proof of Theorem 5.4. Suppose a protocol P satisfies IC1 and IC2 with participants B (commander), T, G, tolerating one Byzantine member. The adversary constructs three executions; Exhibit 5.3 draws the two comparisons.

World 1 (B Byzantine; T, G loyal). B runs, toward T, the exact program a loyal commander with order `attack` would run; toward G, the exact program a loyal commander with order `abstain` would run. (A Byzantine member may run any program; in particular two loyal programs spliced.) T and G follow P honestly, including all lieutenant-to-lieutenant exchange.

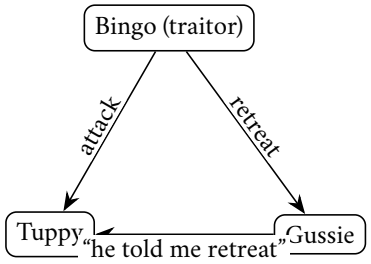
World 2 (G Byzantine; B, T loyal, order `attack`). B behaves loyally with `attack` toward both. G runs, toward T, the program it would run in World 1 — i.e. the honest lieutenant's program as executed when the commander's letters to G say `abstain`. (Feasible for a Byzantine G: the program is fixed, and the adversary feeds it the fabricated commander input.)

T's views coincide in Worlds 1 and 2. By induction on rounds: T's inputs are messages from B and from G. From B: in both worlds, the loyal-`attack` program's messages to T (that is World 2's definition; in World 1, B's splice sends exactly these). From G: in both worlds, the messages of the honest-lieutenant program running against a loyal-`abstain` commander (in World 1 because that is what G honestly experiences; in World 2 by the adversary's construction) — and inductively T's own messages to G are identical, so the simulated G's inputs, hence outputs, are identical. The views are equal round by round.

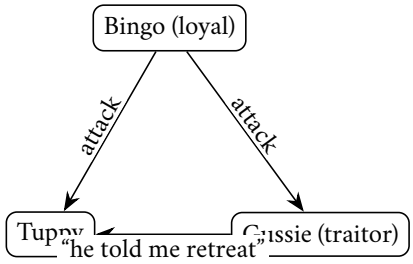
By IC2 in World 2 (B loyal, order `attack`), loyal T outputs `attack`. By indistinguishability, *T outputs `attack` in World 1.*

World 3 (T Byzantine; B, G loyal, order abstain*).* Mirror of World 2: *B* loyal with abstain toward both; *T* runs toward *G* the honest program as if fed attack from the commander. By the mirrored induction, *G*'s views in Worlds 1 and 3 coincide (from *B*: loyal-abstain messages in both; from *T*: honest-program-fed-attack messages in both). By IC2 in World 3, loyal *G* outputs abstain; hence *G outputs abstain in World 1.*

In World 1 both lieutenants are loyal, so IC1 requires equal outputs; but *T* outputs attack and *G* outputs abstain. Contradiction. No such *P* exists — and note the proof nowhere bounded the number of rounds: longer transcripts extend the inductions verbatim, giving the forger more pages to copy. □



World one
(both lieutenants loyal)



World two
(IC2 compels Tuppy: attack)

Exhibit 5.3 (the forged-letter doppelgängers). Tuppy's complete evidence — the commander's letter and the colleague's report — is identical in the two worlds, so his verdict is too; IC2 in world two forces *attack*, exporting *attack* into world one. The mirrored pair (Worlds one and three, drawn by transposing the roles) forces Gussie to *retreat* in world one, and IC1 dies there. Full inductions: Theorem 5.4.

The engine, noted for the record: a lieutenant *cannot audit hearsay* — a report about a letter is worth only its reporter's character, and with every voice worth the same, one liar in a triangle can always place each honest member in a world where the liar is someone else. The pigeons of indistinguishability always come home. The general bound follows the same blood type: any smaller house than $3f+1$ could be partitioned so that three "super-generals," each played by a bloc, re-enact the three-way impossibility — simulate and generalize, structured and starred as Problem 5.10.

Theorem 5.5 (the Byzantine bound; PSL 1980)**.** *Interactive consistency with f Byzantine participants is solvable in the oral model iff the number of participants is at least $3f + 1$. (Achievability: the oral-messages algorithm of LSP, $f+1$ rounds — pointer only. Necessity: reduction to Theorem 5.4; structured as Problem 5.10, starred.)*

What became of the old arithmetic is more instructive than the new number. In the crash world, tolerating f failures cost $2f+1$ seats, and the entire safety argument rested on one property of the witness in the quorum overlap: that he *exists* — an honest-but-silent world has no false testimony, so whatever the witness's ledger says is true, and the only question was whether a witness would be present to say it. Rerun Chapter 4's Synod safety argument with a liar in a house of three and the shared member may be the nephew, whose confession to the new ballot is whatever Aunt Agatha pleases: the witness stands in the box and perjures himself — the *perjured witness*: formally, a Byzantine member of a quorum intersection — and every line of the proof holds except the one that assumed testimony is true.

Remark 5.6 (why crash arithmetic dies: the perjured witness)*.* Chapter 4's safety proofs used quorum intersection through one property: a witness in the overlap *exists*, and his ledger tells the truth. A Byzantine witness perjures: in a three-member Synod with one nephew, the choosing and preparing quorums may intersect in the nephew alone, whose confession is Aunt Agatha's choice, and Lemma 4.3's Step 2 collapses. The repair prices the overlap: with $n = 3f+1$ and quorums of $2f+1$, two quorums share at least $f+1$ members

— an honest *majority of the overlap* survives the deduction of f liars, and protocols demand $f+1$ matching voices before believing anything. Existence was enough against silence; against perjury one needs an honest outvoting. Same club, steeper cover charge.

Spoken as the design review will demand it: five seats survive two crashes but only one liar; seven seats, three crashes but two liars. When a vendor's five-node cluster claims to "tolerate two failures," the diligent question is Chapter 1's — *failures of which kind?* — with this arithmetic as the audit (Problem 5.3). And the chapter's wonder budget is spent here, briefly, because the object earns it on economy alone. The proof of Theorem 5.4 used no probability, no machinery beyond the construction of two afternoons of correspondence and the observation that a loyal gentleman cannot tell them apart — the Two Generals' knife, the fence's knife, honed now against forgery: the third great result of the course to fall to the identical thrust. There is something close to aesthetic law in it: every fundamental limit this field possesses is, at bottom, a statement that someone, somewhere, cannot tell two worlds apart; and every protocol that works is a machine for manufacturing distinguishability — stamps, terms, tokens, certificates — faster than the adversary can erase it. No cleaner organizing principle is known to this desk in any engineering discipline, and the season is barely half through.

Can the bound be beaten? Only by changing the model — Chapter 3's triage, applied on schedule. Give the generals unforgeable signatures — *sealed letters*: signed messages, relays verifiable, composition impossible — and hearsay becomes auditable: a lieutenant relaying the commander's letter relays the *sealed* original, and forgery of the seal is off the menu. The three-node wall then dissolves, because the engine of the proof was precisely that reports could not be audited: Gussie's lie in the second world required him to *compose* a letter in Bingo's name, and against seals, composition fails — Tuppy demands the sealed original, Gussie has none, and the two worlds come apart in Tuppy's hands. Signatures do not abolish Byzantine trouble — a sealed liar can still stay silent, or seal contradictory letters to different colleagues and be caught only when the seals are compared — but they re-price it: with signatures, agreement survives any number of traitors for some problems, and the three-node wall in particular falls. Problem 5.11 stars that door with the honest label on the price tag: the seals themselves now carry the load, and cryptography enters the trusted base.

With or without seals, the practical monument is the protocol Castro and Liskov — the same Liskov; the field is a small parish — published in 1999 as "Practical Byzantine Fault Tolerance": the Synod's spirit rebuilt for the three-f-plus-one world. Its happy case, played out as correspondence in the smallest interesting house: four members — Bingo in the chair, Tuppy, Gussie, and one nephew of Aunt Agatha's, name withheld by the court — with f equal to one and working clubs of three. The chair assigns Bertie's request a slot and announces it (the *pre-prepare*); every member who receives the announcement broadcasts having seen it (the *prepare*), collecting matching statements from three distinct members into a certificate — three statements in a house of four leave room for only one absentee or liar, so no two honest gentlemen can hold certificates for different requests at the same slot: the nephew cannot tell Tuppy one story and Gussie another and have both notarized. The ceremony repeats one level up (the *commit*), which is what makes the decision durable across view changes; then all members answer Bertie directly, who believes at two matching replies, since two answers cannot both be nephews. When the chair itself falls under suspicion, a view change — an election, with epochs; the skeleton is eternal — installs a successor who must *prove*, with certificates, that he carries forward everything the old view committed. Three rounds in fair weather, correspondence quadratic in the house — everyone writing to everyone, the price of trusting no single voice — and a decade of scepticism about practicality that the paper's very title was chosen to answer.

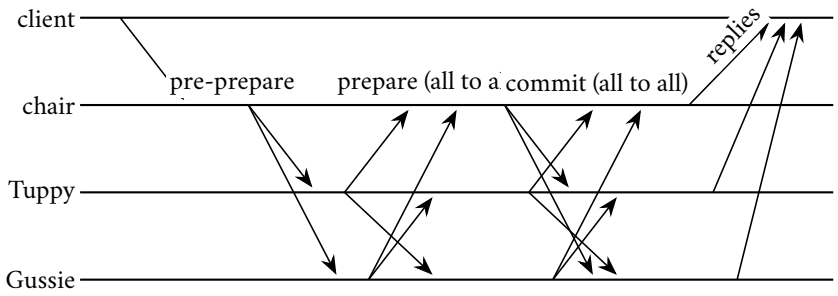
Remark 5.7 (PBFT, and what the certificates buy). Box 12 gives the normal case of Castro–Liskov (1999) for $n = 3f+1$. A *prepare certificate* ($2f+1$ matching prepares) pins (view, slot, request): two conflicting certificates would share $f+1$ members, hence an honest member who prepared both — and honest members prepare

one request per (view, slot). The *commit certificate* makes the binding survive view changes, whose successor must present certificates for everything committed. Replies are believed at $f+1$ matching answers. Full proof declined with a pointer (the paper, and the thesis behind it); the message pattern and the two-certificate architecture are the transferable content.

Protocol — Box 12. PBFT, normal case (checked against Castro–Liskov 1999)

$n = 3f+1$; view v has a fixed primary; certificates are $2f+1$ matching messages; all messages authenticated pairwise.

1: client sends request m to the primary of view v
2: **pre-prepare:** primary assigns slot k; sends $\langle \text{PRE-PREPARE}, v, k, m \rangle$ to all
3: **prepare:** each replica accepting the pre-prepare (fresh (v, k), valid m) broadcasts $\langle \text{PREPARE}, v, k, m \rangle$; a replica is *prepared* on collecting a certificate: the pre-prepare plus $2f$ matching prepares
4: **commit:** prepared replicas broadcast $\langle \text{COMMIT}, v, k, m \rangle$; on a commit certificate ($2f+1$ matching), execute m in slot order
5: **reply:** every replica answers the client directly; the client accepts a result vouched by $f+1$ matching replies
6: **view change** (outline): on primary suspicion, replicas broadcast their prepare certificates; the new primary of view $v+1$ must carry forward every certified slot — the adopt-the-highest rite, notarized



the nephew's line is whichever he pleases; certificates of three
out of four make his stories collide — the client believes two matching replies

Exhibit 5.4 (PBFT's normal case, four members, f equal to one). Pre-prepare fans out; prepare and commit are all-to-all; every certificate needs three of four, any two certificates share two members, at most one of whom lies. Box 12.

One deflationary note, in the opening's spirit. Between the crash world and Aunt Agatha lies most of production reality: switches that half-fail, disks that flip bits, processes that emit one garbled message and then behave for a year. The industry's honest position is that full Byzantine machinery is bought where the threat model earns it — among mutually distrustful institutions, at blockchain frontiers, in flight computers — while inside a single company's fleet one usually buys checksums, sanity checks, and crash-tolerant consensus, and accepts, eyes open, that the Cloudflare afternoon is the residual risk. That is not negligence; it is the triage — expensive versus merely inadvisable — executed with a signature on the risk register. What is negligence is the middle posture: believing one's crash-tolerant parliament is protected against corruption because it has "consensus" in the name. Raft agrees on what the leader says, faithfully, even when what the leader says is corrupted nonsense. Agreement is not truth; it is only, ever, agreement.

5.6 Part Six — Consensus by Lottery

One movement, by standing ruling, the eyebrow at the altitude the ruling specified. In 2008 a pseudonymous author published nine pages describing a currency, and in passing an agreement mechanism of genuine theoretical interest: *Nakamoto consensus*. Observe first what it discards. The parliament is unbounded and anonymous — anyone may join, at any moment, wearing any number of masks; with no membership roll there is no denominator, and with no denominator every quorum argument this course has built evaporates — the crowded club cannot convene when nobody knows how large the room is, and Aunt Agatha, who can print nephews at will, would own any headcount. The substitution is the nine pages' genuine idea: make voting *expensive* rather than *enumerated*. A lottery weighted by computational expenditure; the winner of each round appends the next block to the log — it is always a log — and the honest strategy is to extend the longest chain one has seen; influence is proportional not to identities, which are free, but to electricity, which is not. Agreement is never final, only *probabilistically deepening*: a block buried under six successors is expensive to unseat, under sixty prohibitive, but the mathematics never says never — a sufficiently endowed adversary re-mines history, and safety rests not on quorum intersection but on economics: the attack must cost more than it yields, and the ledger's security budget is, quite literally, a utility bill. Nor is the economics clause hypothetical: on the minor currencies, where the bill is modest, the attack has been executed in the field — computation rented by the hour, history re-mined, settled payments unsettled — the theorem working precisely as stated, on a chain whose security budget a criminal could afford; the premium buys exactly as much finality as it costs, and must be paid in perpetuity merely to keep yesterday settled. It is a genuinely different theorem for a genuinely different model — open membership, monetary adversaries, synchrony assumptions about propagation doing quiet, load-bearing work — and within its own terms it is elegant; the eyebrow is raised at the terms, not the derivation. The course bows and moves on for two reasons, neither snobbery: this syllabus concerns parliaments with a membership roll, where the crowded club is purchasable at five machines rather than a power grid — inside a company's fleet, identity is cheap and trustworthy, and paying Nakamoto's premium there is buying flood insurance for a submarine; and the interesting engineering above Nakamoto — the finality gadgets, the proof-of-stake successions, the committee samplings that quietly reintroduce the very quorums the scheme began by discarding — is a course of its own, taught in a different building, with its own aunts. A raised eyebrow, a genuine nod, and the door closed gently: Decision 11, executed.

5.7 Part Seven — When Not to Convene the Parliament

The senior skill, last, because it is a discipline rather than a doctrine. Consensus is the most expensive ordinary thing a fleet does, and the bill deserves reading once in round figures. One decision: a round trip to a quorum — within a single facility, a millisecond or so; across availability zones, a few; across regions, tens to a hundred and more — plus a synchronous disk write at every voting member before it answers. Chain the arithmetic: a coordination service in one region settles perhaps a few thousand to some tens of thousands of decisions per second, and *every tenant in the estate shares that budget*; a busy service admits hundreds of thousands of requests per second before breakfast. Three orders of magnitude, give or take, between what the parliament can decide and what the front door admits — and that gap is the entire architecture of coordination: it is why leases exist (amortize one decision across ten thousand requests), why leaders batch (amortize one round trip across a hundred log entries), why the kernel stores only small hot metadata, and why every read that must be *current* is a question with a price tag. Here the chapter's last debt is issued, with ceremony. The instinct says a read may simply be answered by the leader from local state — he is the leader, after all; but the resurrection problem is now on the books: a leader can

be deposed and not yet know it, and his confident local answer can be a posthumous letter — stale in the precise, formally defined sense Chapter 7 will name. The disciplined options: route reads through the log — correct, and priced like a write — or serve them under a *lease*, the leader answering locally only while holding a majority-granted time bound, Part Four's instrument now insuring reads, correct exactly insofar as the drift assumption holds; the fine print of that "exactly insofar" is deferred, with the register's blessing, to Chapter 7 (Box 10, line 5).

So the catechism, and it is short. Convene the parliament for what is genuinely constitutional: leadership, membership, locks, the small hot metadata whose loss or fork is catastrophic — and rent even that, from a landlord with a Jepsen dossier. Do not convene it for traffic: for caches, for feeds, for anything whose staleness is survivable — Chapters 6 and 7 build the honest vocabulary for *survivable* — and never on a per-request critical path if a lease can amortize it. The catechism has been watched failing at a design review, and the staging is instructive: a team, burned once by a stale cache, resolved that every read of the product catalogue would henceforth consult the coordination kernel for the current version — a small read, they reasoned, and correctness is correctness. The kernel, shared by the entire estate, thereafter spent the bulk of its decision budget confirming that toasters were still toasters, and the locks guarding actual money queued behind the toasters; the estate learned the constitutional distinction during an incident, which is the most expensive of the available classrooms. The parliaments of this course are constitutional monarchies at best, sir: essential, dignified, and kept well away from the day's actual commerce — which is the next chapter entire.

5.8 The Whiteboard

For the design review you will chair.

1. Rent coordination; do not build it. Keep the tenancy competence in-house — sessions, watches, and above all the session-invalidation callback, which applications ignore at the price of volunteering for resurrection.
2. Every lock in the estate is a lease, whether it admits it or not: find its expiry, then find its fence — one comparison on one rising integer at the resource, the cheapest insurance sold in this industry.
3. Count your f's aloud — $2f+1$ against crashes, $3f+1$ against lies — and say which failures the quorums actually insure. Five seats survive two crashes but only one liar; a Raft cluster is not Byzantine-tolerant because the word consensus appears in its documentation.
4. Know which failures live outside the model entirely, and buy detection there — checksums, sanity bounds, divergence alarms — because the theorem patrols only the terrain named in its premises, and the weather does not read the premises.

The register. PAID: #1, the Post learns to forge — the book's founding debt, receipted in full with theorem and aunt; #12, one value becomes a log; #14, reconfiguration. INSTALMENTS: #2 and #10, armed with leases and fences; final pricing at Chapter 11. ISSUED: #15 — the log stars in its own feature (Chapter 10); #16 — reads under leadership: what *current* means and what a lease buys (Chapter 7). WORD-COUNT DELTA RECORDED: the episode ran 9,858 words at delivery (Session 5), marginally under the 10,000 floor; the assembly session (Session 13) brought it to 10,036 with two staged additions — the rented-computation attack on minor chains, and the toaster catechism from the design review — under a LEDGER §5 drift note.

5.9 Problems

Tier I — Calisthenics

Problem 5.1 (docket bookkeeping)**.** For Exhibit 5.1: tabulate each member's per-slot ledger at every event; state exactly when slot forty-one is chosen, when slot forty-two is chosen, and what Tuppy's prepare must contain. (a) Which slots may the state machines have applied at each moment? (b) Suppose slot forty-two's accept had reached nobody: write Tuppy's succession actions. (c) Why may Tuppy *not* simply skip slot forty-two and renumber?

Problem 5.2 (lease arithmetic)**.** Period ten seconds, drift bound one part in a hundred. (a) Compute the holder's horizon per Box 11 and the worst-case true-time spans of both windows, and exhibit the gap. (b) Recompute with drift one part in twenty (a badly ventilated rack). (c) The holder renews at three-quarters of its horizon: what is the renewal cadence, and what happens to the guarantee if a renewal exchange straddles a pause? (d) At what drift bound does the scheme's usable tenancy fall to half the period?

Problem 5.3 (count your f's)**.** For houses of three, four, five, seven: (a) how many crash failures are survivable with majority quorums; (b) how many Byzantine members are survivable with $3f+1$ sizing and $2f+1$ quorums; (c) the overlap of two quorums in each regime, and the honest residue of the overlap at maximum f. (d) A vendor's five-node cluster claims to "tolerate two failures": write the two follow-up questions the claim requires.

Problem 5.4 (zxid order)**.** Zab stamps: (epoch three, counter nine), (epoch four, counter one), (epoch three, counter ten), (epoch four, counter nought). (a) Sort. (b) A follower in epoch four receives a proposal stamped epoch three: state its action and the Chapter-2 name for the principle. (c) Compare with a Raft (term, index) pair: which comparisons are meaningful in each system?

Problem 5.5 (the determinism audit)**.** Four apply-functions for a replicated state machine; find each one's hidden second input and give the repair per Def. 5.1: (a) *set expiry to now plus one hour*; (b) *assign the request a fresh universally-unique identifier*; (c) *iterate over a hash table of accounts, applying interest in iteration order*; (d) *if the local replica's disk is over ninety percent full, reject the write*.

Tier II — The Real Thing

Problem 5.6 (fencing, designed and proved)**.** Using Box 11's resource rule: (a) prove that once any operation stamped z executes, no operation stamped $z' < z$ ever executes on that object — and conclude that a resurrected holder cannot write after its successor has. (b) State precisely what is *not* guaranteed about two operations of the same token (a holder racing itself), and what discipline restores it. (c) The high-water mark must be durable: exhibit the failure if the store keeps it in memory (Chapter 1's frugal clerk rides again). (d) Extend the design to reads: what must a fenced read check, and against which token? *(Full solution.)*

Problem 5.7 (the pessimistic anchor)**.** Modify Box 11 so the holder starts its horizon at the *receipt of the grant* rather than at the request. (a) Exhibit an execution violating mutual exclusion (let the grant letter dawdle). (b) Identify the exact inequality of Theorem 5.3's proof that fails. (c) Some real systems anchor at grant-receipt anyway and compensate: what quantity must then be bounded, and why is that a stronger model purchase than drift? *(Full solution.)*

Problem 5.8 (sabotage: fencing by wall clock)**.** A designer replaces the rising token with the holder's wall-clock timestamp at grant time: "fresher time means fresher authority, and the clocks are NTP-disciplined

anyway." The store executes an operation iff its timestamp exceeds the stored high-water timestamp. (a) Exhibit an execution in which a paused-and-resurrected holder's write is *accepted* after its successor's write — give concrete clock readings; a modest skew suffices. (b) Exhibit the dual failure: a lawful successor whose writes are *all refused*. (c) State the invariant that rising tokens provide and wall clocks cannot, in one sentence, and name the Chapter-2 principle it instantiates. (d) Would tighter NTP fix it? Cite the relevant part of Chapter 2's Whiteboard. *(Certified fatal per the standing rules: the execution is written out in the solutions and drawn as Exhibit 5.5.)*

Problem 5.9 (Zab and Raft, side by side)**.** Both systems must ensure a new reign carries forward every committed decision. (a) Describe each system's recovery discipline in three sentences: Zab's explicit synchronization phase before broadcast; Raft's up-to-date veto plus commit-own-term rule. (b) For each, identify what a deposed primary's stale proposal encounters (name the stamp that kills it). (c) Exhibit a schedule that Zab's separated phases make easy to reason about and Raft handles with its interleaved rules — and state, with reasons, which discipline you would rather debug at three in the morning. *(Solution sketch.)*

Tier III — For the Ambitious (starred)

Problem 5.10 (⋆ simulate and generalize)**.** Prove the necessity half of Theorem 5.5: with $n \leq 3f$ participants and f Byzantine, interactive consistency is unsolvable. Route: partition the n participants into three non-empty blocs of size at most f each; from a hypothetical n-participant protocol, construct a three-participant protocol in which each general simulates one bloc; verify the simulated protocol would satisfy IC1 and IC2 with one Byzantine general (the bloc containing the traitors), contradicting Theorem 5.4. Mind the bookkeeping: where do the commander's bloc's internal messages live, and why does bloc size at most f let the adversary corrupt a whole bloc? *(Hints only.)*

Problem 5.11 (⋆ sealed letters)**.** Give each participant an unforgeable signature; relays carry the sealed originals. (a) Show Theorem 5.4's construction fails: locate the step where the adversary must compose a letter it cannot seal. (b) Sketch the Lamport–Shostak– Pease signed-messages algorithm for three participants and one traitor: commander signs and sends; lieutenants countersign and relay; decide from the set of validly-sealed values. Verify IC1 and IC2. (c) State what the seals cost the model: what must now be true of key distribution and of the adversary's computational power, and why the desk files this under "re-priced," not "solved." *(Hints only.)*

5.10 Hints

Hint (Problem 5.7)**.** The proof's inequality $t_r \leq t_g$ is free; anchoring at receipt replaces t_r by a time *later* than t_g by an unknown postal delay, and the guard interval must absorb what nothing bounds.

Hint (Problem 5.8)**.** For (a): the resurrected holder's clock need only be *ahead* of the successor's; NTP disciplines toward agreement, not order of authority. Grant times: successor granted later by the landlord, stamped earlier by its own honest, slow clock.

Hint (Problem 5.10)**.** Each general runs its bloc's members faithfully, exchanging intra-bloc messages internally and inter-bloc messages over the three-general channels. IC for the simulation follows because every loyal n-protocol participant lives inside a loyal general.

Hint (Problem 5.11)**.** In World 2, Gussie must show Tuppy a *sealed* retreat order bearing Bingo's signature; Bingo never signed one. For (b), the decision rule "take the set of distinctly-sealed orders received; if singleton, obey it; else default" already works for $n = 3, f = 1$.

5.11 Solutions

Tier I in full (tersely); Tier II in full for Problems 5.6, 5.7, and 5.8 (the last is the sabotage certification), sketch for Problem 5.9; hints only for Tier III.

Solution (to Problem 5.1)**.** Slot forty-one is chosen when a second member persists the accept (quorum of three); slot forty-two is chosen only after Tuppy's re-ratification at ballot eight reaches a quorum — Gussie's lone acceptance at ballot seven chose nothing. Tuppy's prepare must cover all slots and returns confessions: forty-one (ballot seven, the debit) from at least one member; forty-two (ballot seven, the credit) from Gussie only. (a) Members may apply only through the longest chosen prefix they know: forty-one after its choice; never forty-two before forty-one. (b) Confessions for forty-two come back empty; Tuppy proposes NO-OP at forty-two (or fresh business), at ballot eight. (c) Renumbering rewrites history: clients acknowledged against slot numbers, and the apply rule's induction (Thm. 5.1) is over *this* sequence; holes are filled, never collapsed.

Solution (to Problem 5.2)**.** (a) $H = 10 \cdot 0.99/1.01 \approx 9.80$ seconds. Holder's window ends by $t_r + 10/1.01 \approx t_r + 9.90$ true seconds; grantor re-grants no earlier than $t_g + 9.90$; gap $= t_g - t_r > 0$. (b) $H = 10 \cdot 0.95/1.05 \approx 9.05$ s; both windows meet at ≈ 9.52 true seconds from their anchors. (c) Renew near 7.35 s of holder clock; a pause straddling renewal is safe — the old horizon still governs trust, and the renewal merely fails — provided the holder re-checks the horizon *after* any operation resumes; the fence (line 5) covers the residue regardless. (d) Usable tenancy $T(1-\rho)/(1+\rho) = T/2$ at $\rho = 1/3$ — far beyond any sane clock, which is the point: the scheme is robust precisely because ρ is tiny.

Solution (to Problem 5.3)**.** (a) Crashes: one, one, two, three. (b) Byzantine: three seats tolerate none; four, one; five, one; seven, two. (c) Crash regime: overlap at least one, all honest. Byzantine regime ($n = 3f+1$, quorums $2f+1$): overlap at least $f+1$; honest residue at least one — and the protocols demand $f+1$ matching voices, which the residue's honesty anchors. (d) "Failures of which kind?" and "at which quorum size — what does the system do between the second crash and repair?"

Solution (to Problem 5.4)**.** (a) (three, nine) < (three, ten) < (four, nought) < (four, one). (b) Discard: stale epoch; the principle is Chapter 2's "this instruction is stale — I have seen a higher number" — the Lamport clock's entire salary. (c) Zab zxids are totally ordered and comparison is always meaningful; Raft (term, index) pairs order by term for *authority* while index compares *position* — cross-comparing indices across terms is meaningful only under log matching's prefix guarantee.

Solution (to Problem 5.5)**.** (a) Clock input: chair evaluates *now* at proposal, writes the absolute expiry into the entry. (b) Randomness: draw the identifier at proposal, carry it in the entry. (c) Iteration order: sort by account key (or store ordered) — the hash walk differs per replica. (d) Local environment: the disk check is a per-replica fact; either make admission a chair-side check recorded in the entry, or make the rule a function of replicated state (quota accounting in the machine). The common repair: *decide once, upstream, into the log*.

Solution (to Problem 5.6)**.** (a) The mark is durable and monotone: executing z sets $hw \geq z$; any later arrival with $z' < z \leq hw$ fails the guard; induction over executions. The successor's first write carries $z_{\text{succ}} > z_{\text{old}}$ (tokens rise across grants), so after it executes, every posthumous stamp is below the mark. (b) Same-token operations are not ordered by the fence: a holder racing itself (two threads, one lease) may interleave freely; the repair is a per-token sequence number checked monotonically, or the discipline that a holder is single-threaded per resource. (c) In-memory mark: store crashes, mark reborn at zero, the resurrected holder's stale stamp passes — Chapter 1's frugal clerk with a new ledger; persist the mark with the object. (d) A

fenced read checks $z \geq hw$ *without raising* the mark (or raises it, at the price of fencing concurrent same-token writers), and must be answered from state no older than the mark's last write — which is Chapter 7's subject in embryo.

Solution (to Problem 5.7)**.** (a) Grant at true t_g; the granted letter dawdles D true seconds; holder anchors at receipt $t_g + D$ and trusts until $\approx t_g + D + T/(1+\rho)$, while the grantor re-grants at $t_g + T/(1+\rho)$: overlap of nearly D, and D is the Post's to choose. (b) The containment $t_r \leq t_g$ becomes $t_g + D \geq t_g$ with the sign the wrong way — the holder's window is no longer inside the grantor's guard. (c) One must bound the grant letter's delivery delay — a synchrony purchase on the Post itself, strictly stronger than a drift bound on local crystals, and exactly the kind of assumption Chapter 3 taught us to see priced on the label.

Solution (to Problem 5.8)**.** (a) The execution, drawn as Exhibit 5.5. The landlord grants to Bingo; Bingo's clock reads fourteen seconds past the minute — *fast* by ten seconds against true time. Bingo stamps his lease-grant moment fourteen-past and pauses. Session expires; the landlord grants to Tuppy, whose honest clock reads *eight*-past (true time: six-past). Tuppy writes, stamped eight-past; the store's high-water timestamp becomes eight-past. Bingo wakes and writes, stamped fourteen-past: *accepted* — fourteen exceeds eight — and the successor's state is overwritten by a corpse with a fast watch. (b) Swap the skews: successor's clock slow by ten; every lawful write stamped below the corpse's mark is refused until the successor's clock crawls past it — a lawful reign refused service by arithmetic. (c) Rising tokens are issued by *one* authority in grant order, so stamp order equals authority order by construction; wall clocks are many authorities with no order contract — Chapter 2's principle: never order cross-node events by wall clock; a cross-machine timestamp comparison is an opinion wearing digits. (d) No: NTP shrinks typical skew, bounds nothing, and fails silently (Chapter 2, Part One); the Whiteboard's first item demanded the uncertainty bound in writing, and none exists here. Fatality certified.

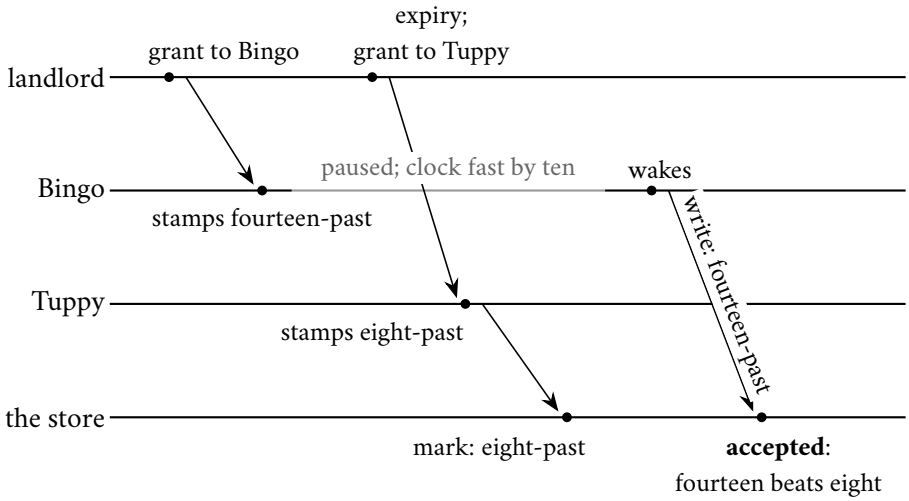


Exhibit 5.5 (fencing by wall clock; the counterfeit millisecond; Solution 5.8). The resurrected holder's fast watch outbids the lawful successor at the store. A timestamp is an opinion wearing digits; a token is an order of authority. Fatality certified.

Solution (to Problem 5.9)**.** (Sketch.) (a) Zab: on election, the new primary runs discovery (learn the highest zxids from a quorum), synchronization (install the chosen history on a quorum), and only then broadcast — three named phases, recovery strictly before new business. Raft: the up-to-date veto keeps ill-read candidates out; log matching repairs followers continuously; commit-own-term closes the inherited tail —

recovery woven into normal operation. (b) The stale primary's proposal dies on its stamp: old epoch (zxid) / old term. (c) Schedules with deep follower divergence are naturally narrated in Zab's phases (the sync phase is the narrative); Raft handles them with backward-stepping consistency checks mid-traffic. Preference is honestly a matter of where one wants the complexity: in a phase diagram (audit it once) or in an invariant set (rely on it continuously). The desk declines to legislate taste, and observes only that both have passed their Jepsen penances.

Solution (to Problems 5.10 and 5.11)**.** By standing policy: hints only (the Hints section above). For Problem 5.10, the arithmetic checkpoint: three non-empty blocs of size at most f exist precisely when $n \leq 3f$ and $n \geq 3$.

5.12 The Reading

Burrows, "The Chubby Lock Service for Loosely-Coupled Distributed Systems," OSDI 2006. Assigned as literature. The candid sections — unexpected uses, abusive clients, the outage table — are the truest published portrait of a coordination kernel's tenancy. Read the caching and session mechanics against Part Three, and the lease discipline against Box 11.

Lamport, Shostak, and Pease, "The Byzantine Generals Problem" (1982); Pease, Shostak, and Lamport, "Reaching Agreement in the Presence of Faults" (1980). The framing paper and the founding mathematics. Read the 1982 three-general argument against Theorem 5.4 and Exhibit 5.3; the signed-messages section against Problem 5.11.

Castro and Liskov, "Practical Byzantine Fault Tolerance," OSDI 1999. The monument. Read §4 against Box 12 and Exhibit 5.4; the view-change section is the adopt-the-highest rite notarized.

Schneider, "Implementing Fault-Tolerant Services Using the State Machine Approach: A Tutorial," ACM Computing Surveys 1990. The log's constitutional theory; Thm. 5.1 in its definitive setting.

The shelf. Chandra, Griesemer, and Redstone, "Paxos Made Live" (PODC 2007) — the industrialization bill, including the disk-corruption and reconfiguration clauses. Gray and Cheriton, "Leases" (SOSP 1989). Kleppmann, "How to Do Distributed Locking" (2016), with the reply from Redlock's author — both sides preserved; the fencing argument in its native habitat. The Cloudflare postmortem, "A Byzantine Failure in the Real World" (November 2020), against the chapter's opening. The Raft single-server membership-change correction (Ongaro, raft-dev, 2015) — the dragons' forwarding address.

Standing companion: Kleppmann, chapters 8–9. The lease-and-fencing treatment in chapter 8 pairs with Part Four; chapter 9's total order broadcast closes the loop on the log.

Chapter 6

Replication, or the Dynamo School

Companion to Episode Six. Replication as a menu with the prices face up, opened by the Decembers of Amazon: carts that must never refuse a write, service judged at the ninety-ninth-point-ninth percentile, and the 2007 paper that founded the leaderless school. The classical column and its recovery-point ledger; the multi-leader bill called conflict, with the GitHub 2018 postmortem walked in full; the school entire — sloppy quorums, hinted parcels, the janitor, the fingerprint library, gossip. Then the set piece, written out with the linearization constructed explicitly: the bare overlap $R + W > N$ buys a regular register, not an atomic one, and one ABD clause repairs it with no consensus anywhere. The kettles' paperwork, the menu as a decision procedure, and a sabotage certified fatal close the file.

6.1 Part One — The Classical Column: One Leader

In the early years of this century the retailer Amazon had a seasonal ailment. Every December the shopping public converged in its gift-buying millions, and the systems under the load were, in their deep tissue, relational databases: magnificent instruments, rigorously transactional, and constitutionally inclined, when overwhelmed or partially failed, to do the safe thing — to stop answering, refusing the write they cannot certify. An honourable instinct, bred from decades of banking, and catastrophically wrong here, because the refused write was a customer placing an item in a shopping cart, and a cart that says no in December is a store turning away money at the door. Two house commandments frame everything that follows. First: the cart is *always writeable* — during partitions, during failures, during the strange weather of December, a customer with an item in hand is never told no. Second: services are judged not at the average but at the far tail — agreements written at the ninety-ninth-point-ninth percentile, the experience of the unluckiest customer in a thousand, on the reasoning that the unluckiest customers are disproportionately the heaviest ones. Beneath both lay measured arithmetic: pages artificially delayed by a hundred milliseconds cost a measurable slice of completed sales — the delayed customer does not complain; he drifts away mid-purchase, and the tills record his absence — and an unavailable cart was priced dearer still, forfeiting a fraction of the shopper's future custom with the abandoned basket. That pricing is the missing premise that makes the chapter's heresies rational: set against Chapter 3's theorem, when refusing a write is denominated in December revenue and accepting one in a little reconciliation labour later, the betrayal writes itself. Dynamo (DeCandia et al., SOSP 2007) is the system those commandments built, and the paper is candid about the price: it would sometimes hold several versions of a cart at once, and something, somewhere, would sort that out later — consistency betrayed, availability kept, on purpose, with receipts. The paper founded a church, and the church had denominations within the decade.

This chapter is the menu: every honest way to keep copies of data on machines that fail, with every

price tag turned face up. The models first — two rooms of them, because the register theorem of Part Four demands sterner hypotheses than the school's engineering posture.

Model of this chapter

For the menu and the school (Defs. 6.1–6.6, Lemma 6.1). Asynchronous; crash-recovery with stable storage; fair-loss links with retransmission beneath (Chapter 1); membership by gossip is advisory — quorum arithmetic is stated against the intended N; sloppy quorums explicitly weaken the fixed-roster hypothesis of Lemma 4.1, exchanging certainty of overlap for likelihood (stated, not proved — an engineering posture, not a theorem).

For ABD (Def. 6.3, Theorem 6.2). Fully asynchronous message passing; n servers of which any minority may crash-stop; reliable delivery to correct processes (retransmission); *one* designated writer client and any number of reader clients, all of which may also crash (a crashed reader mid-write-back harms nothing: Rem. 6.5). No clocks, no leader, no failure detector. The multi-writer extension is Problem 6.10 (starred).

A replication scheme is an answer to exactly two questions — who may accept a write, and who must agree before the client is told yes — and every scheme in the trade is a cell in the resulting small grid, whatever its brochure says. Formally:

Definition 6.1 (the replication grid)**.** A replication scheme is classified by two coordinates. *Acceptance*: which replicas may accept a client write — one (single-leader), one per site (multi-leader), any (leaderless). *Acknowledgment*: whose durable confirmation precedes the client's acknowledgment — the acceptor alone (asynchronous), a fixed set (synchronous; semi-synchronous for "any k of"), or a quorum (N, R, W). Chain replication occupies (single acceptor at the head; agreement = the entire chain, enforced by topology).

The classical column first, the arrangement intuition already owns. *Primary-backup*: one replica — the primary; Bingo in the chair — accepts every write; the others hold copies and follow. What travels down the links decides a whole class of production ghosts: forward the client's *statement* and every follower re-executes it, haunted exactly as Chapter 5's determinism diet warned — a command that consults the clock, draws a random number, or walks an unordered collection executes differently at every desk while all machinery reports health; forward the *effect* — the rows as changed, the bytes as written — and nothing is re-decided. The mature systems ship effects, or statements laundered through a determinism filter: decide once, upstream; replicate the decision, not the deciding.

Then the single most consequential checkbox in replication: does the primary *wait*? Synchronous acknowledgment — Bingo answers Bertie only after followers confirm the write durably theirs — buys a recovery point of zero: nothing confirmed is ever lost to the primary's death. The price arrives on two invoices: every write pays the round trip to the slowest waiter (PACELC's peacetime toll, Chapter 3), and availability is coupled to the *least* available member — Tuppy, synchronously required, takes his afternoon garbage-collection pause, and the primary can acknowledge nothing; a machine bought for durability halts commerce. Hence the semi-synchronous compromise production favours: wait for any k of the followers, so the write provably exists beyond the primary, though nobody undertakes to say where. The disaster-recovery ledger names both columns, and both deserve their number at review: the *recovery point* (RPO — how much confirmed history can vanish; formally, the confirmed-but-unreplicated writes lost on primary death) and the *recovery time* (RTO — how long the estate stands dark while the failover machinery below deliberates).

Asynchronous acknowledgment inverts the trade: the primary answers at once and forwards at leisure. The staged loss, dates left blank for one's own incident reports: Bertie's order lands at half past the hour;

Bingo applies, acknowledges, and queues the letter to Tuppy; at thirty-one minutes past, Bingo's power supply performs its actuarial duty; the letter was still in the queue, and the queue was memory. Tuppy is promoted, healthy, and has never heard of Bertie's order. No component misbehaved: acknowledgment was defined as Bingo's word, and Bingo's word died with Bingo. At any instant an asynchronous estate carries a window of confirmed-but-unreplicated writes, and when the primary dies unrecoverably, that window *is* the data lost. Asynchronous replication is thus a chosen data-loss window — defensible daily, for workloads that can absorb it, and indefensible whenever chosen without the number attached: seconds of exposure, times writes per second, times the worth of a write. The arithmetic fits on an index card, and Problem 6.1 fills one out.

The column's second product is read capacity, shipped with fine print: every follower read is a read of the past, aged by whatever the replication lag happens to be — milliseconds mostly; seconds or worse under load. The small hauntings follow: Bertie updates his address, refreshes the page, and is shown the old one — he has written and cannot read his own writing. The purchasable remedies (pin a user's reads to the primary briefly after his writes; route by session) generalize into the session guarantees, named and priced in Chapter 7. The column's honest summary: follower reads are cheap, plentiful, and stale by an *unstated* amount — and unstated is, as ever, the actionable defect.

Failover is where the column meets Chapter 1: the primary goes silent, and dead-or-slow has no answer from across a network. Every failover system resolves it with a purchased verdict — a timeout — and the wrong verdict has a specific, spectacular failure mode: the *false death*, the doppelgänger's one operational cameo this chapter. Bingo enters a garbage-collection pause of distinction; the monitor's probes go unanswered; the verdict is rendered and Tuppy promoted; forty seconds later Bingo returns — from his own perspective nothing has happened — and resumes accepting writes from every client and every stale routing table that has not heard the news. Two primaries: the wrongly condemned, like Chapter 5's resurrected lock-holder, retains all his old authority and none of the news, and every write he accepts is a fork in the estate's history. The disciplined fencing is Chapters 4 and 5's equipment — promotion through consensus or a coordination kernel, so that at most one promotion can ever be chosen, and an epoch stamped on all replication traffic, so the deposed primary's letters bounce off followers who have seen a higher number; the fencing token's elder siblings. Undisciplined arrangements meet Part Two's postmortem.

One more entry in the column, less famous than it deserves: *chain replication* (van Renesse and Schneider, 2004). Arrange the replicas not in a star but in a chain — Bingo the head, Tuppy the middle, Gussie the tail. Writes enter at the head and ride hop by hop, each desk applying and forwarding; the *tail* acknowledges; reads consult only the tail, answered from local state, no quorum consulted. Feel what the topology bought with no arithmetic at all: whatever the tail has, the whole chain has — a write in Gussie's ledger has by construction passed every desk above him — and whatever the tail lacks is by definition unacknowledged; so tail reads never expose an in-flight write and never miss a confirmed one. Strong reads, served locally, by geometry alone. The costs are equally legible: a write pays the full length of the chain before confirmation, and a broken link demands a reconfiguration dance, spliced by a master typically rented from Chapter 5's landlords. The proof runs on a short skeleton — order operations by their passage through the tail; a write is acknowledged only past the tail, a read is the tail's local state; geometry supplies the total order — and it is yours: Problem 6.6, full solution given.

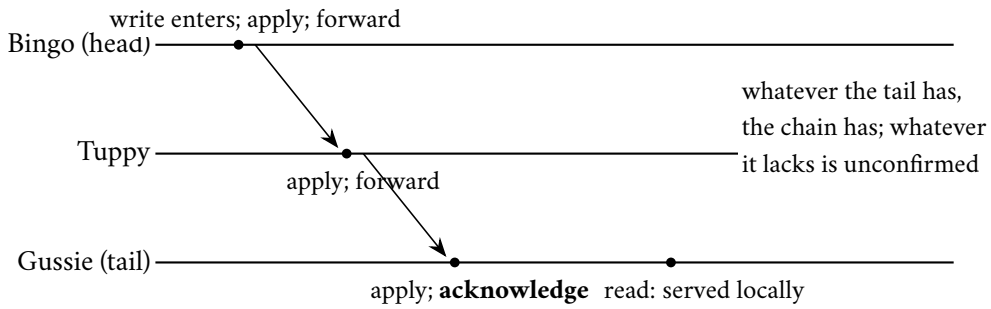


Exhibit 6.1 (chain replication). Writes ride the chain head to tail and are acknowledged at the end; reads consult only the tail. Strong reads by geometry: no quorum, no arithmetic. Problem 6.6 turns the picture into a proof.

6.2 Part Two — Several Leaders, and the Bill

Geography sings its siren song: why should a customer in Tokyo write to a primary in Virginia? *Multi-leader* replication puts a leader in every region, each accepting writes locally, each forwarding to the others — and the bill arrives immediately, because two leaders accepting writes to the same object concurrently (merely elsewhere, in Chapter 2's earned vocabulary) will conflict, and something must adjudicate. Even the postal map among the leaders has failure modes of its own. The tidy geometries — a circle of forwarding, a star about a hub — economize on connections and let one stopped desk dam the relay for everyone downstream; robust all-to-all posting opens races instead: a correction leaves Tokyo a breath behind the update it corrects, travels by a faster intermediary, and arrives first — an effect presenting itself before its cause, a child arriving at the party ahead of its parent — cured by Chapter 2's causal paperwork, the correction held in the anteroom until its ancestor arrives. Note the season's pattern, for it recurs in grander dress below: every time the trade widens a topology in the name of availability, the ordering bill arrives by return post.

The industry's default adjudicator deserves its arraignment by name. *Last-writer-wins*: attach a timestamp to every write; on conflict keep the larger; discard the smaller, silently. The evidence, in four sentences. The Tokyo leader accepts a customer's corrected shipping address at what its clock calls two seconds past the minute; the Virginia leader accepts the same customer's phone-number update at what its clock calls three seconds past; the writes touch the same contact record, cross in replication, and the timestamps adjudicate: Virginia's row — carrying the *old* address alongside the new phone number — wins whole, and the corrected address ceases to exist. Nobody is told; the parcel ships to the old address three weeks later; the investigation finds every system healthy and every log clean, because nothing malfunctioned — a policy executed. Last-writer-wins is not a conflict resolution strategy; it is a conflict *denial* strategy — institutionalized data loss wearing a tie-breaking rule, its verdicts rendered by cross-machine wall clocks that Chapter 2 convicted of perjury; the "last" in the name is an opinion wearing digits. There are workloads where the policy is honestly fine — where any discarded write was genuinely superseded by the survivor: telemetry samples, caches, the last position of a cursor. There is no workload where it is fine by accident, and Problem 6.9 audits a schema for it, column by column. Between the throne and the timestamps sits a third posture much of the industry quietly lives in: *conflict avoidance* — multiple leaders for latency, but every write to a given key routed through one designated home, so that per key the system is single-leader and conflicts cannot form. It works precisely until the routing must change — the customer moves; the home region fails — and in the changeover window the design discovers it was multi-leader all along, the adjudication debt merely deferred and now due at the worst hour.

The postmortem clause of this course licenses only the published record, and here it is: GitHub, 21–22 October 2018, from the company's own account. During routine maintenance replacing network equipment, the connection between the East Coast datacentre and the other facilities failed — for *forty-three seconds*. In those seconds the automated topology manager — a tool whose entire commission is to keep the databases writable by promoting replicas when primaries vanish — observed the East Coast primaries unreachable from where it stood and did its duty: West Coast replicas were promoted; applications, rerouted, kept writing. The connection healed, forty-three seconds old, and the estate now held what multi-leader holds by design and single-leader holds only by accident: two recent histories — East Coast writes the West had never received, West Coast writes the East had never seen — neither discardable, both containing customers' truth, and the timestamps unable to adjudicate because, per Chapter 2, there was no fact of the matter for them to report. What followed, per the postmortem, was more than twenty-four hours of deliberately degraded service — failing over traffic, restoring from backups, replaying the orphaned writes, reconciling colliding records by hand and by script. The exchange rate is the lesson: partitions are cheap to *enter* and ruinous to *leave*, and any design that can produce two primaries owes its reviewers, in writing, the reconciliation plan — before the maintenance window, not during.

Fairness obliges the addendum: multi-leader has honest homes. Clients that go offline and write locally — the laptop on the aeroplane, the telephone in the tunnel — are each, structurally, a leader of one, and no throne can abolish them; collaborative editing is multi-leader by its very product definition. Both homes force the conclusion the shop reached: conflicts must be *merged*, not adjudicated by clock. The honest ancestor is *Bayou* (Terry et al., Xerox PARC, 1995), built for exactly those disconnected laptops decades early: writes accepted anywhere, conflict expected as the normal case, and every write accompanied, by the application at write time, by a *dependency check* — a predicate detecting that the world has changed under this write — and a *merge procedure* — what to do about it when it has. The house example: two colleagues, disconnected, each book the same meeting room for the same hour; on resynchronization the later-ordered booking's dependency check finds the slot taken, and its merge procedure — written by the calendar's own author — books the adjacent hour, rather than double-booking the room or silently deleting a colleague's meeting. Applications resolve; timestamps do not — applications know what the data *means*. Nothing about the mechanism scaled to the modern fleet, and its total-ordering machinery had a primary hiding in it besides; but Dynamo inherited the moral whole: if concurrent writes are accepted, the conflict is application state, and the system's duty is to keep the evidence — both versions, causal papers in order — not to eat one quietly and call the ledger balanced.

6.3 Part Three — The Leaderless School

The Dynamo column proper, staged at the width the choreography needs: five gentlemen — Bingo, Tuppy, Gussie, Barmy, and Oofy — because the school requires neighbours. The first move abolishes the throne entirely: no primary, no elections, no terms; any replica may accept any operation, and correctness is arranged, where it is arranged, by arithmetic. Each key has a *preference list* — the ordered set of replicas responsible for it, read off a hashed ring whose full geometry is Chapter 9's subject; for Bertie's cart it reads Bingo, Tuppy, Gussie, then Barmy and Oofy as understudies. The first N are the home trio, and any list member may coordinate. Three numbers govern everything, and the crowded club works its advertised Sunday shift: $R + W > N$ buys the overlap — any read quorum intersects any write quorum in at least one member (Lemma 4.1) — so at two-and-two-of-three the newest acknowledged version is always somewhere in the room when you ask. The precise strength, and the precise weakness, of that sentence is the whole of Part Four.

Definition 6.2 (quorum configuration)**.** Each key has N replicas (its preference list's home set); a write is acknowledged after W durable confirmations; a read consults R replicas and adopts the causally newest version returned. The configuration is *overlapping* if $R + W > N$. *Sloppy* variants draw the W confirmations from the first healthy members of the extended preference list, tagging displaced copies with hints (Box 13).

But the founding commandment was *always writeable*, and strict quorums are not: with Gussie's rack dark and Tuppy's link having a Cloudflare afternoon, a two-of-three write to the home trio must refuse, and refusal is the one forbidden act. So the school loosens its grip: the *sloppy quorum*. Bingo, coordinating Bertie's kettle in December weather, writes locally — one — and slides down the preference list: the letter meant for Gussie goes to Barmy, marked with a *hint* reading, in effect, *this parcel belongs at Gussie's desk; hold it and deliver when he returns*. Barmy stores the hinted parcel in a separate larder and acknowledges; Bingo has his two, and the client is told yes — in a storm, without a throne. When Gussie's rack revives, Barmy's larder streams its holdings home, each hint discharged on confirmation: hinted handoff, the neighbours holding the parcels. The durability arithmetic was honoured in *count* though not, temporarily, in *address* — and note what was quietly traded, because the paper is honest about it and its cheaper imitators sometimes are not: Lemma 4.1 spoke of quorums drawn from a *fixed* roster, and a sloppy write may land entirely on understudies while a later read consults the recovered home trio. Overlap by arithmetic has become overlap by high likelihood — the model box's stated posture — and the school's wager is that December's revenue is worth a probabilistic corner, with the nets below to sweep it. The paths, assembled:

Protocol — Box 13. The Dynamo-school read and write paths (checked against the 2007 paper, §4)

Per key: preference list $p_1, p_2, \ldots$; home set = first N; any list member may coordinate.

1: **write**$(k, v, context)$: coordinator assigns version = context advanced at its own entry (its dot); sends to the first N *healthy* list members
2: a member stores the version (as sibling if incomparable with holdings); if standing in for an unreachable home member, stores with a **hint** naming the home address
3: acknowledge to client after W durable confirmations (sloppy quorum: the confirmations may include understudies)
4: **hinted handoff:** each member periodically streams hinted holdings to their home addresses; on confirmation, the hinted copy is discharged
5: **read**(k): coordinator queries R members; collects all sibling versions and their vectors
6: answer client with the causally-maximal sibling set and merged context
7: **read repair:** send the returned set to any queried member holding strictly older state
8: **anti-entropy:** background Merkle reconciliation of replica pairs, per key range (Lemma 6.1)
9: **gossip:** membership and ring state exchanged pairwise at random; indirect probes before condemnation (SWIM)

Net the first: *read repair*, the janitor. A read at quorum R collects up to R versions with their causal paperwork; the newest is adopted for the answer, and in the same breath the coordinator posts it back to any queried member holding strictly older state, whose stale copy is quietly retired. Every act of asking tidies the room asked, so the estate's hottest data — the most read — is also its best-repaired, by construction and for free. Attend to the mechanism; Part Four promotes it from janitor to load-bearing wall.

Net the second, and a moment of wonder. Read repair tidies only what is read; cold data drifts — the entries nobody has asked for since the storm are exactly the entries nobody has repaired. The deep net is *anti-entropy*: periodically, pairs of replicas reconcile their entire holdings, which said naively is absurd — two gentlemen comparing millions of entries over the Post to find the dozen that differ. The instrument

that makes it civilized is the *Merkle tree*, the fingerprint library: over each key range, a tree of hashes — leaves fingerprinting small ranges of entries, each parent hashing its children's hashes, up to a single root summarizing the entire library; a million entries, a branching factor of a few dozen, a tree perhaps four levels deep. Bingo and Barmy, reconciling after the storm, exchange one fingerprint each. Roots equal: then — with confidence bounded only by the hash's honesty, which is cryptographic — the libraries are identical, and a million entries have been reconciled at the cost of one comparison. Roots differing: descend a level, compare the children, dismiss every matching branch with its hundreds of thousands of entries unexamined, and recurse into the one disagreeing subtree, until the differing leaf range stands isolated and only the actual divergence crosses the wire. Traffic scales with the disagreement, not the library — verification made exponentially cheaper than transfer, which is why the instrument appears wherever honesty must be checked at scale: in the school's janitorial closet, in the version-control system that names every commit by its fingerprint, and in the lottery-parliaments whose chain of blocks is this tree unrolled in time. One idea, three industries. The cost, as a lemma:

Lemma 6.1 (Merkle reconciliation cost)**.** *Let two replicas hold libraries over the same key space, indexed by a Merkle tree of branching factor b and depth d (leaves cover disjoint ranges; internal nodes hash their children). If the libraries differ in the contents of $\Delta \geq 1$ leaf ranges, the descent protocol (compare roots; recurse into differing children only) exchanges at most $b \cdot \Delta \cdot d + 1$ fingerprints, and transfers entry data only for differing ranges. If $\Delta = 0$, one exchange suffices, up to hash collision.*

Proof of Lemma 6.1. Consider the set D of tree nodes whose subtrees contain at least one differing leaf range; D is a union of Δ root-to-leaf paths, so $|D| \leq \Delta \cdot d$ (the root counted once via the +1 below). The protocol exchanges children fingerprints exactly for nodes in D found unequal — at most b fingerprints per such node — and never recurses into a node outside D, since equal fingerprints terminate the branch (up to hash collision, whose probability is bounded by the hash's collision resistance and union-bounded over comparisons). Total fingerprints: 1 (roots) + $b \cdot |D| \leq 1 + b\Delta d$. Data transfer occurs only at differing leaves by construction. If $\Delta = 0$ the roots match and the exchange stops at one. □

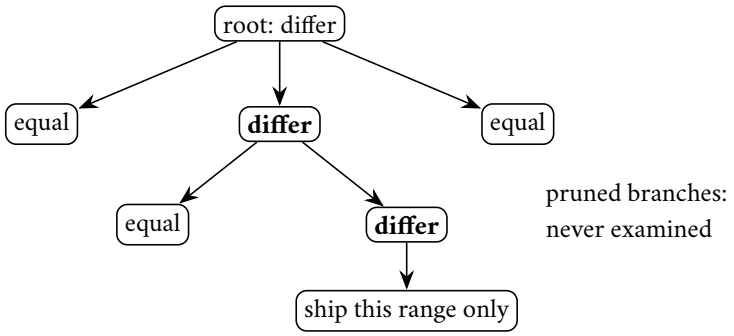


Exhibit 6.2 (the Merkle descent). Fingerprints are compared root-down; equal branches are dismissed with their entire contents unexamined; only the differing leaf range crosses the wire. Traffic scales with the disagreement, not the library (Lemma 6.1).

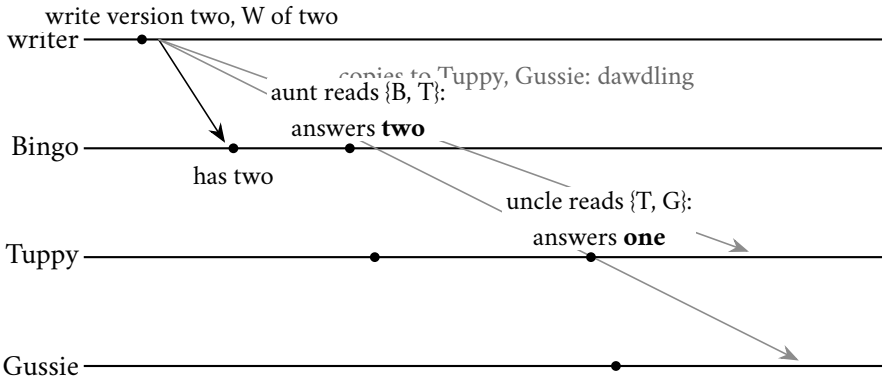
Net the third: membership itself travels by *gossip*. No coordination kernel holds the roster; each gentleman, at a steady modest cadence, rings up one colleague chosen at random, and the two exchange everything they believe about the ring — who is present, who suspected, who newly joined — each keeping the fresher account wherever they differ. The epidemiology is the charm: rumour doubles per round — $O(\log n)$ rounds to saturation, a fleet of a thousand fully informed in about ten — with no coordinator, no broadcast storm, and magnificent indifference to the loss of any particular messenger, the rumour having no spine to cut. The lineage is Demers et al. (1987), epidemic algorithms taken quite literally from the epidemiology

of infection; the modern refinement is SWIM, whose courtesy is the indirect probe — before condemning a silent colleague, ask third parties to probe him on your behalf, publish suspicion as a state distinct from death, and give the accused a gossip round to protest — so that one bad link between two healthy machines can slander neither. This is Chapter 3's failure detector rebuilt without a landlord, its verdicts exactly as purchasable, and as fallible, as every other timeout in this course.

Two mechanical notes complete the anatomy. The coordinator is not appointed: any preference-list member may take the chair, and in several of the school's descendants the client library itself does, posting directly to the replicas and counting acknowledgments at the edge — leaderless all the way out to the caller, the hop saved being real money at the ninety-ninth-point-ninth percentile. And siblings, under heavy concurrent writing to a hot key, do not merely occur but *accumulate*, each unmerged rival breeding further rivals with every write descending from a partial read; the school's estates cap the family size and complain to their operators, because a key with fifty siblings is not a database problem but an application refusing its merge duty (Part Five), and the pager, for once, points at the right team. That is the school entire: no throne to defend, no election to storm, every component symmetric, every corner swept by a net rather than barred by a guarantee.

6.4 Part Four — The Fine Print: What the Overlap Buys

The industry belief, stated as held: *with $R + W > N$, reads always see the latest write — linearizability, near enough*. The set piece dismantles the near-enough, then rebuilds it properly. Honesty about the other dials first, so the belief is met at its strongest rather than at a straw tuning: W=1, R=1 — the school at its fastest — promises only eventual anything, and workloads that choose it know what they bought; the belief concerns the respectable tuning, the overlap real, every read quorum provably touching every acknowledged write's quorum — the tuning routinely sold, and bought, as strong consistency. Now the staged run, in honest weather: no crashes, no partitions, merely the Post being the Post. Bertie's balance sits at version one on Bingo, Tuppy, and Gussie. Bertie writes version two at W=2: the coordinator posts to all three; Bingo's copy is durable at once; Tuppy's and Gussie's dawdle in the Post — the write is, at this instant, in flight. Bertie's aunt reads at R=2, her quorum {Bingo, Tuppy}: two versions return, the causal papers rank version two newer, and she is answered with the new balance — perfectly true, fresher than required, no complaint. A moment later, after her answer has arrived and been acted upon in strict real time, Bertie's uncle reads, quorum {Tuppy, Gussie}: version one, twice — concurring, stale, and served with confidence. The estate's observed history ran backwards; had the aunt telephoned the uncle in between — and Chapter 7 makes exactly that telephone call the formal test — the family watched money flicker. No quorum was violated: the overlap simply delivered the new version to one reader and not the next, because "the write set" was still a moving target. The overlap guarantees the newest acknowledged version is visible in the room; it does not guarantee that, having been seen once, it stays seen.



the aunt saw the new world; the uncle, later, the old:
regular, not atomic — the flicker

Exhibit 6.3 (the crime scene: new-then-old under $R + W > N$). The write of version two is in flight — durable at Bingo only. Both reads hold lawful quorums; the estate's observed history runs backwards. Visibility is not linearizability.

The object the bare overlap builds has a name in Lamport's register taxonomy — *safe*, *regular*, *atomic* — and it is the middle one: both family reads overlapped the dawdling write, so each was individually lawful under regularity; the crime was visible only across them, in the new-then-old order.

Definition 6.3 (the register taxonomy; Lamport)**.** A shared read/write register, accessed by operations with invocation and response times, is: *safe* if a read overlapping no write returns the latest completed write's value (reads during writes unconstrained); *regular* if additionally a read overlapping writes returns the value of some overlapping or latest-preceding write (old or new, never invented); *atomic* if the operations admit a total order, consistent with real time, under which every read returns its latest preceding write — atomicity is linearizability specialized to registers, and Chapter 7 adopts the general word. The flicker of Exhibit 6.3 is the canonical regular-not-atomic behaviour: two sequential reads straddling one in-flight write, returning new then old.

What the overlap does buy is then a theorem, in outline: a read overlapping no write intersects the completed write's quorum and the paperwork ranks that version newest; during an in-flight write both answers are lawful — and successive reads may flicker.

Remark 6.4 (what $R + W > N$ buys)*.* With strict quorums, completed-write visibility is a theorem: a read overlapping no write intersects the last completed write's quorum (Lemma 4.1) and the version paperwork ranks that version newest, so the read returns it — safety and, with the overlap applied to in-flight writes as well, regularity (each read returns a value some overlapping-or-preceding write wrote). Atomicity fails: nothing obliges the new version, once served to one reader, to be served to the next (Exhibit 6.3). The missing clause is exactly ABD's write-back. Sloppy quorums weaken even regularity to high likelihood; the nets (repair, anti-entropy) restore convergence, not ordering.

The repair has a naming ceremony: Hagit Attiya, Amotz Bar-Noy, and Danny Dolev — the field says *ABD* — solved exactly this in work from 1990, published in final form in 1995, years before Dynamo made the question commercial. The repair is one clause, met already in overalls: *the reader who has seen a version must lodge it before repeating it.* A read queries R, adopts the newest version, and — before the client is answered — writes that version back until a quorum holds it: read repair promoted from janitor to load-bearing wall, not an optimization that tidies eventually but a blocking obligation. The protocol, checked against the founders:

Protocol — Box 14. ABD single-writer atomic register (checked against Attiya–Bar-Noy–Dolev)

n replicas each store $(stamp, value)$, initially $(0, v_0)$; majorities = $\lfloor n/2 \rfloor + 1$.

1: **writer, write**(v): $s \leftarrow s + 1$ (own sequence, persistent); send $\langle \text{STORE}, s, v \rangle$ to all
2: complete on acknowledgments from a majority
3: **reader, read**(): send $\langle \text{QUERY} \rangle$ to all; await replies from a majority
4: adopt (s^*, v^*) = the maximal-stamp reply
5: **write-back:** send $\langle \text{STORE}, s^*, v^* \rangle$ to all; await a majority's acknowledgments (skippable iff the query replies were unanimous at s^*)
6: **then** return v^*
7: **replica, on** $\langle \text{STORE}, s, v \rangle$: if $s >$ stored stamp, store (s, v) (durably); acknowledge regardless
8: **replica, on** $\langle \text{QUERY} \rangle$: reply with stored $(stamp, value)$

Line 5 is the load-bearing clause (Theorem 6.2); line 7's monotone store is what makes lodging idempotent and helping safe.

Run the crime scene again under the clause: the aunt's read finds version two at Bingo; before she is answered, version two is lodged at a majority; the uncle's subsequent quorum of two must intersect it. The flicker is dead — once seen, forever seen.

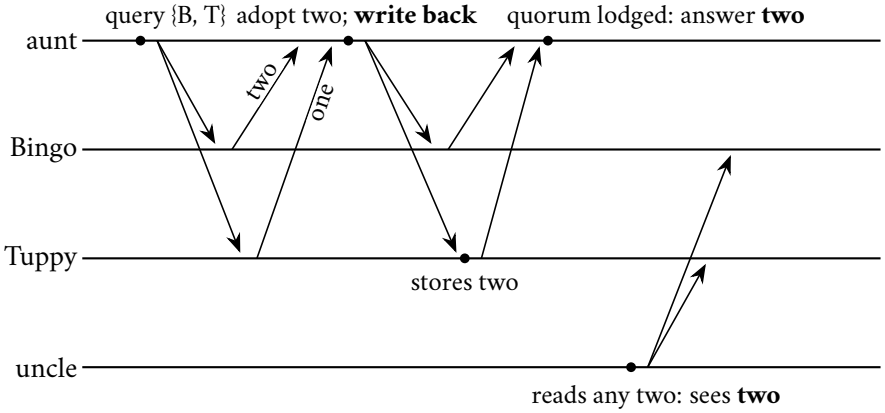


Exhibit 6.4 (ABD's write-back kills the flicker). The aunt lodges version two at a majority *before* being answered; the uncle's quorum must intersect it. Once seen, forever seen. Box 14, Theorem 6.2.

The theorem, skeleton first: write-back closes the flicker; linearize by stamp and completion order, placing each read after the write it adopted; the write-back guarantees any later read's quorum intersects a quorum holding the adopted stamp; and termination needs only majorities.

Theorem 6.2 (ABD; Attiya–Bar-Noy–Dolev)**.** *In the ABD model box, the protocol of Box 14 implements an atomic (linearizable) single-writer multi-reader register: every execution's completed operations admit a real-time-consistent total order in which each read returns the latest preceding write. Moreover every operation invoked by a correct client terminates, regardless of scheduling — no consensus, no leader, no synchrony.*

Proof of Theorem 6.2. Write operations carry stamps $1, 2, 3, \ldots$ from the single writer (its own sequence; persisted across its crashes by assumption of write-once-per-stamp). Notation: for a completed operation o, $\text{inv}(o)$ and $\text{resp}(o)$ are its real-time endpoints. For a completed write w_k (stamp k), its *lodging quorum* Q_k is the majority whose acknowledgments completed it. For a completed read r, its *adopted stamp* $st(r)$ is the

maximal stamp among its query replies, and its *lodging quorum* is the majority acknowledging its write-back of $st(r)$.

Claim 1 (stamp monotonicity along real time). If completed operations o_1, o_2 each lodge stamps s_1, s_2 (a write lodges its own; a read its adopted) and $\text{resp}(o_1) < \text{inv}(o_2)$, then $s_2 \geq s_1$. *Proof:* o_1's lodging quorum holds stamp s_1 (replicas keep the maximum stamp seen: Box 14 line 8) before o_2 begins; o_2's query majority intersects it (Lemma 4.1); the intersection replica replies with stamp $\geq s_1$; a read adopts the maximum, so $s_2 \geq s_1$; a write's stamp exceeds every stamp the writer has used, and the writer's own previous operations include lodging s_1 or earlier — for the single-writer case, $s_2 = s_1 + j$, $j \geq 1$, when o_2 is a write following o_1 in the writer's sequence; when o_2 is a write not following o_1 (i.e., o_1 is a read that adopted a stamp the writer issued earlier), the writer's next stamp still exceeds all issued stamps, hence $\geq s_1$.

Claim 2 (reads return the stamped value). All operations with stamp k concern the single value v_k written at k: replicas store (stamp, value) pairs and Box 14 adopts them jointly; no creation (Chapter 1's FL3) bars other values.

The linearization. Order operations by primary key (lodged stamp), with ties broken: the write of stamp k first, then reads adopting k in the real-time order of their responses (concurrent reads in either order). This is a total order on completed operations. *Real-time consistency:* suppose $\text{resp}(o_1) < \text{inv}(o_2)$ but o_2 precedes o_1 in the constructed order. Then $s_2 < s_1$, or $s_2 = s_1$ with an ordering violation among the tie rules. The first contradicts Claim 1. For the second: equal stamps put the write first — but a write with stamp s cannot be invoked after the response of *any* operation lodging s, since lodging s requires the writer to have issued it (Claim 2 and no creation), i.e., $\text{inv}(w_s)$ precedes every lodging of s (though w_s's own response may follow). The delicate case is a read r with $\text{resp}(r) < \text{inv}(w_s)$ and $st(r) = s$: impossible, again by no creation — r adopted s from some replica, which received it from the writer's phase for w_s, which follows $\text{inv}(w_s)$. Ties among reads follow response order by construction. Hence the order extends real time. *Read correctness:* in the constructed order, the latest write preceding a read r is $w_{st(r)}$ (later writes have larger stamps and are ordered after), and r returns $v_{st(r)}$ by Claim 2.

Termination. Each phase awaits a majority of replies from n replicas of which a minority may crash: the correct majority eventually replies (reliable delivery, no waiting on any specific replica); no phase can block forever and no phase's completion depends on any race being decided. Both operations complete in two round trips (one, for the optimized read of Rem. 6.5(ii)). □

Three consequences follow, filed formally: the reader as *helper*, completing a stranded write; the *toll*, and where it lawfully vanishes; and the *border*, with the race test that draws it.

Remark 6.5 (three notes on the theorem). (i) *Helping*: a reader adopting a stamp lodged at fewer than a quorum completes the stranded write; doubt is resolved toward durability by whoever trips over it. (ii) *The toll*: reads pay a write's postage exactly when the quorum disagreed; the unanimous case may lawfully skip the write-back (Box 14, line 5). (iii) *The border*: the register cannot elect, lock, or compare-and-swap (Herlihy rank one); any operation whose outcome depends on winning a race needs Chapter 4's machinery. CAP is paid by the minority partition blocking.

Pause on what is absent from Theorem 6.2: no leader, no election, no consensus, no FLP shadow. FLP's fence stood on the adversary's power to keep two futures alive, and a register protocol has no fork to hold open — ABD never decides between anything; it lodges copies with majorities and adopts maxima, operations no schedule can leave on a fence. Patience assembles a majority; nothing further need be agreed. The CAP invoice is paid exactly where Chapter 3 said it must be: during a partition the majority side proceeds and the minority side waits — consistency kept, availability surrendered on the small side of the severance, the betrayal chosen and signed. And one clause of fairness, lest the set piece read as indictment: Dynamo claims no linearizability — the paper claims eventual consistency with causal paperwork, and the descendants that bolted the write-back on offer it as a *setting*, under names like "quorum reads with repair," at a

documented latency premium, because the school's founding judgment was precisely that the cart does not need once-seen-forever-seen and should not pay for it. The set piece is a boundary marker, not a scandal, and it issues the one diagnostic question for any vendor page waving quorum arithmetic as a consistency claim: *where does the read-back live, and does it block?*

One border remains, and it surprises nearly everyone. The guarantee just delivered — linearizability — is the very badge Chapter 7 crowns as strongest; how does the summit come without the expensive parliament? Registers rank one on Herlihy's ladder: reading and writing, however strongly consistent, cannot elect, cannot lock, cannot compare-and-swap — two gentlemen with a linearizable register still cannot agree on one bit against a single crash. Linearizable *storage* is cheap — majorities and diligence; it is linearizable *decision* that costs a parliament, and the map that sorts a design review has a single test: does any operation's outcome depend on winning a race? If yes, parliament (Chapter 4's machinery); if no, majorities and diligence suffice, whatever the consistency badge required.

6.5 Part Five — The Two Kettles

Multi-writer reality, and the payment of a debt planted in Chapter 2's rival kettles. In the leaderless school two writes to the same key from different clients, landing at different coordinators, are the normal case, and the honest description is already owned: if neither write's history includes the other, they are not simultaneous but *concurrent* — *merely elsewhere*, and no lawful system may pretend one superseded the other. The school therefore keeps, per key, exactly Chapter 2's instrument, run as a loop: every read returns, alongside the data, its causal *context*; every write carries back the context of the read it descends from, declaring its ancestry in its own hand; and the receiving coordinator compares paperwork — retiring the genuinely superseded, keeping incomparable rivals as *siblings*, and, in the dotted refinement, identifying each version by one precise event coordinate, the *dot*, atop the summarized past.

Definition 6.6 (version vectors; siblings; dots)**.** Per key, each stored version carries a vector of (coordinator, counter) entries; reads return version(s) with a *context* (the merge of returned vectors); writes carry their read-context as declared ancestry. A stored version strictly below an incoming context is superseded; incomparable versions coexist as *siblings*, returned together until an application write merges them under a joint context. Deletion is a tombstone version. The *dotted* refinement identifies each version by its precise coordinator event (the dot) atop a summarized past, eliminating the false-sibling and false-order artifacts of bare per-coordinator counters (Preguiça et al., reading notes).

The kettles, planted four chapters ago, now stand in the same cart. Bertie, in his study, reads the cart — one umbrella, context supplied — and adds a kettle; the write, bearing the study's context, lands at Bingo. At the club, moments later by nobody's clock in particular, he reads the same cart on his telephone — the study's kettle not yet propagated — and adds a kettle of a different and, one fears, redundant model; the write, bearing the club's context, lands at Gussie. The contexts are incomparable: each descends from the umbrella-only cart, neither from the other. Two carts now exist, both real, provably concurrent, and the next read receives both siblings with their paperwork. Resolution belongs to the party that knows what a cart *means* — the application, whose merge for a cart is union: kettle and kettle and the umbrella once, the sin of showing a customer two kettles billed rather lower than the sin of vanishing one. The merged cart is written back bearing both contexts, a descendant of each parent, and the paperwork closes over the reconciliation like a settled estate. The union answered to three laws the shop has been obeying unnamed: indifference to the order in which siblings are presented, or two readers repairing the same rivalry would manufacture new rivals; indifference to repetition, for the nets may deliver the same version twice, or the janitors would breed the very dust they sweep; and no invention — every merged item traceable to some

parent, or the reconciliation is a forgery. Not etiquette but load-bearing algebra: precisely what lets a room of replicas repair in any order, along any routes, meeting any duplicates, and still converge on one cart. Notice also who does the work: the merge runs in the reading client — the school's client libraries famously thick, its servers famously thin — and the three laws receive their government names, and the convergence theorem they underwrite, in Chapter 7.

Three footnotes of due diligence. First, siblings are the honest mode, and last-writer-wins remains on the school's menu as the dishonest default: more than one descendant shipped timestamp adjudication where vector paperwork belonged, or made the honest mode optional and the quiet mode default, and Kingsbury's laboratory has published the autopsies — lost updates reproduced under partition on demand. The first audit question of a school system is not the quorum tuning but *which mode holds your keys*. Second, deletion, the school's quietest trap: an erased entry looks, to every net of Part Three, like an entry the eraser never had, and anti-entropy will helpfully restore it — the deleted kettle rising from the cart, resurrected by machinery doing exactly its job. The lawful instrument is the *tombstone*: deletion recorded as a causally stamped write that propagates, wins its comparisons, and suppresses resurrection — and tombstones must eventually be garbage-collected, which reopens in miniature the windowed-deduplication argument of Chapter 1: discard one while any replica might still reintroduce its ancestor, and the ghost returns — why the school's estates all carry tombstone-lifetime settings, and why the operators who did not know why have met it. Third, the dots: the classical vector in coordinator hands can file versions written through one coordinator for *different* clients as ordered when they were in truth concurrent, or spuriously multiply siblings that never rivalled anyone, because a bare per-coordinator counter says only how many updates a version has seen, never *which* event it is; the dotted refinement of Def. 6.6 restores exact causality at coordinator granularity (Preguiça et al., the reading). The pattern, seen thrice now: causal bookkeeping is an engineering budget — full vectors, dotted refinements, bare timestamps — and every discount below full honesty is a specific, nameable lie one elects to tolerate. What the kettles still lack is a *general* theory of merging — union worked because carts are sets that grow; counters, documents, and calendars want their own laws — and the discovery that a whole algebra of mergeable objects exists, convergence theorem attached, is Chapter 7's second half. The kettles are in the same cart at last; their wedding is Chapter 7.

6.6 Part Six — The Menu as Decision Procedure

Fold the menu into the form it takes at a whiteboard. A replication scheme is an answer to the two questions of Def. 6.1, and the season's systems read back through the grid so it earns its keep at once: primary-backup with synchronous acknowledgment — one acceptor, fixed agreement; the classical bank. Raft and multi-Paxos — one acceptor per term, quorum agreement; the throne with arithmetic underneath. Dynamo at the classic tuning — any acceptor, sloppy quorum; the December machine. Chain replication — one acceptor, agreement defined as the entire chain; geometry standing in for arithmetic. Every system on the market is a cell in that small grid, whatever its brochure says, and most production incidents in replicated estates are one of three category errors: a recovery window nobody priced, a second primary nobody planned the reconciliation for, or a quorum overlap mistaken for an ordering guarantee.

Choose by workload, in the order that actually decides: the consistency need first — will the business absorb siblings, stale reads, lost updates? Chapter 7 arms the question properly — then the latency budget, against which synchronous hops and blocking write-backs are purchases, then topology. Four workloads, worked through. The shopping cart: siblings absorbable, loss forbidden, latency sacred, topology global — leaderless, sloppy, vectors, union-merge; Dynamo is not *a* choice for the cart; the cart is why Dynamo exists. The payments ledger: loss forbidden *and* siblings forbidden — a double negative no quorum tuning satisfies; the ledger wants a throne with synchronous acknowledgment, or the parliament outright; December may

queue. The service-configuration store: small, hot, catastrophic to fork — Chapter 5's rented coordination kernel, fine print included. And the telemetry stream, included because the menu owes last-writer-wins its one honest home: each sample supersedes its predecessors by the data's own meaning — the reading anyone wants is the latest, and a discarded older sample was never going to be consulted again — so the loosest tuning serves and the tie-breaking rule discards only what the workload had already discarded. Mark the reasoning shape rather than the verdict: supersession was in the *schema* before it was ever in the clock, and the same test, run in the opposite direction, convicted the shipping address in Part Two. Estates are plural, and the menu is consulted per dish; the school wars of the trade press are category errors.

Since the procedure is most often exercised not on one's own designs but on somebody's brochure, the chapter folds itself into five questions to be asked aloud at review, answers in writing:

1. Who may accept a write — one desk, one per region, or anyone?
2. Who must agree before the client hears yes — and if a quorum, is it strict, or does it slosh to understudies in weather?
3. If any acknowledgment is asynchronous, what is the recovery point — the number, on the index card: seconds of exposure, times writes per second, times the worth of a write?
4. Where does the read-back live, and does it block — is the consistency on the label bought with lodged evidence, or with arithmetic and optimism?
5. Which mode holds these particular keys — causal paperwork with an owned merge, or timestamps quietly eating the loser — and if tombstones are anywhere in the design, what is their lifetime, and who chose it?

A vendor — or a colleague, or a past version of oneself — who cannot answer all five has not designed a replication scheme; he has purchased a rumour of one.

One temperament note from the operations desk, folklore honestly labelled: leader systems fail *legibly* — a throne to inspect, a failover to narrate, one place where the incident lives — while the school fails *diffusely* — everything degrades a little, and no component owns the incident because no component owns anything. The two incident reports, side by side, make the axis tactile. The leader estate's is a narrative: the primary failed at seventeen minutes past; the detector fired; the failover completed after forty seconds; here is the recovery window, and here is what fell into it — a story with a protagonist, reconstructible from three logs. The school's is a weather system: latency crept for an hour before anyone paged; sibling counts rose on a fraction of keys; hinted larders swelled on machines nobody was watching; no single moment went wrong because no single place could. The first demands an organization that can execute a runbook crisply at night; the second, one that trusts dashboards, tolerates ambiguity, and sweeps its corners on schedule. Neither temperament is superior — but a team with the first temperament operating the second architecture will spend its incidents searching for a throne that does not exist.

6.7 The Whiteboard

For the design review.

1. A replication scheme answers two questions: who may accept a write; who must agree before acknowledgment. File every brochure adjective in the grid.
2. Asynchronous replication is a chosen data-loss window: seconds of lag × writes per second × worth of a write, plus the tail case — the index card, aloud.

3. Quorum overlap buys durability and freshness-on-offer, not linearizability; ask where the read-back lives and whether it blocks. Linearizable storage is majorities and diligence; linearizable *decision* is a parliament — the race test decides which you need.

4. If concurrent writes are possible: prevent them with a throne, or keep the evidence (causal paperwork, owned merges). Last-writer-wins is data loss with a tie-breaking rule; know where it holds your keys.

The register. ISSUED: #17 — the kettles must merge: the algebra of mergeable objects and its convergence theorem, payable Chapter 7; and the word *consistent* owes this course its definition — folded into the standing Chapter 7 ceremony with #11, #13, and #16. WORD-COUNT DELTA RECORDED: the episode ran 8,868 words against the 10,000 floor at delivery (Session 6); the assembly session (Session 13) expanded it to 10,174 with staged, within-spec material — the retail latency arithmetic; the multi-leader topology races; the merge function's three laws; the telemetry workload and the five-question review catechism; the two incident reports — under a LEDGER §5 drift note, per the standing rule that delivered scripts are amended on the record only.

6.8 Problems

Tier I — Calisthenics

Problem 6.1 (the index card)**.** A primary handles two thousand writes per second; asynchronous replication lags one second typically, twelve seconds at the recorded worst; a write is a customer order worth, on average, thirty pounds of eventual revenue. (a) Compute typical and worst-case recovery-point exposure, in writes and in pounds. (b) The vendor proposes semi-synchronous acknowledgment at a cost of two milliseconds median latency: restate the trade in one sentence a merchant would sign. (c) Which number on the card does a partition change, and which does it not?

Problem 6.2 (tunings drill)**.** For $N = 3$: classify each tuning's guarantees (durability on acknowledgment; completed-write visibility; flicker possibility; write availability with one replica down): (a) W=1, R=1; (b) W=3, R=1; (c) W=2, R=2 strict; (d) W=2, R=2 sloppy; (e) W=2, R=2 strict with blocking write-back.

Problem 6.3 (parcels and hints)**.** Preference list Bingo, Tuppy, Gussie, Barmy, Oofy; N=3, W=2. Gussie is down for an hour; twelve writes to the key arrive. (a) Where do the copies land, and what hints exist? (b) Gussie recovers: describe the handoff traffic. (c) During Gussie's absence a strict-quorum read R=2 consults {Tuppy, Gussie}... it cannot; re-answer for the school's actual read path and say what may be missed. (d) State in one sentence what sloppy quorums trade against Lemma 4.1.

Problem 6.4 (sibling paperwork)**.** Stored versions of a key carry vectors A = (B:2, T:1) and B = (B:1, T:2). (a) Ordered or rivals? (b) A write arrives with context (B:2, T:2): what does it supersede, and what vector does its version carry (coordinator Tuppy)? (c) A write arrives with context (B:2) only: enumerate the resulting sibling set. (d) Which of these outcomes would a bare per-key timestamp have silently destroyed?

Problem 6.5 (fingerprint economics)**.** A million-entry range, branching factor thirty-two, leaves of a thousand entries. (a) Tree depth? (b) Fingerprints exchanged when the libraries are identical; when exactly one leaf range differs; when eight scattered ranges differ (bound via Lemma 6.1). (c) Compare with shipping the library, at one fingerprint's size per thousandth of an entry.

Tier II — The Real Thing

Problem 6.6 (chain replication is linearizable)**.** Model the chain of Exhibit 6.1: nodes apply writes in arrival order and forward; the tail acknowledges writes and serves reads from local state; assume reliable FIFO links (purchased per Chapter 1) and no failures (reconfiguration is excluded). (a) Show every node applies the same writes in the same order (induction down the chain). (b) Define the linearization: order each write at its tail-application instant and each read at its tail-service instant; show this order is total, real-time consistent, and read-correct. (c) Mark exactly where "acknowledged only at the tail" is spent — exhibit the violation if the head acknowledged. (d) Why does read-at-head break linearizability but read-at-tail not break write availability? *(Full solution.)*

Problem 6.7 (regularity, proved and bounded)**.** For strict $R + W > N$ with a single writer and versioned values: (a) prove completed-write visibility (Rem. 6.4); (b) prove reads never return invented or superseded-then-revived values (no-creation plus stamp monotonicity at replicas); (c) exhibit the flicker with N=3, R=W=2 and mark which clause of Def. 6.3's atomicity it violates; (d) state the smallest protocol change that restores atomicity, and cite the theorem. *(Full solution: the set piece, formalized.)*

Problem 6.8 (sabotage: acknowledged at the send)**.** A coordinator, "pipelining for the percentiles," counts a write toward W when the STORE letter is *sent*, not when the durable confirmation returns; everything else follows Box 13 with N=3, W=2, R=2 strict. (a) Exhibit an execution in which the client is acknowledged and *no surviving replica holds the write* (two crashes permitted, or one crash plus one lost letter — the Post's ordinary charter). (b) State which line of the recovery-point index card this design quietly rewrites, and by how much. (c) Exhibit the read-side symptom (an acknowledged write invisible to every subsequent R=2 read) and explain why read repair never heals it. (d) Give the repair and its true latency cost, and reconcile with the percentile commandment honestly. *(Certified fatal per the standing rules: the execution is written out in the solutions and drawn as Exhibit 6.5.)*

Problem 6.9 (the LWW audit)**.** A contact-record schema: address (corrected rarely, catastrophic to lose); phone (idem); marketing preferences (latest wins by intent); last-seen timestamp (monotone gauge); cart items (grow-and-merge). The store offers per-column LWW or sibling mode. (a) Assign each column a mode with one-sentence justifications. (b) For each LWW assignment, state the exact anomaly accepted and its worst business case. (c) Rewrite the last-seen column to be LWW-safe by construction. *(Solution sketch.)*

Tier III — For the Ambitious (starred)

Problem 6.10 (★ multi-writer ABD)**.** Extend Box 14 to many writers: a write first queries a majority for the highest stamp, then writes at (that stamp +1, breaking ties by writer identity). (a) Show stamps still totally order writes. (b) Re-prove Claim 1 of Theorem 6.2 — where does the write's query phase substitute for the single writer's private sequence? (c) Construct the linearization and verify real-time consistency. (d) Count round trips per operation and compare with the single-writer costs. *(Hints only.)*

Problem 6.11 (★ repair under fire)**.** Design a read-repair discipline that preserves atomicity when repairs race concurrent writes: specify how a repair's stamp interacts with in-flight writes (never overwrite a higher stamp; monotone stores), show a naive repair that re-lodges a *superseded* version cannot occur under your rule, and identify the exact property of Box 14 line 7 you are relying on. Then weaken the model: with sloppy quorums, exhibit the residual anomaly no repair discipline closes, and say which net (anti-entropy) bounds its lifetime. *(Hints only.)*

6.9 Hints

Hint (Problem 6.8)**.** Let both STORE letters dawdle; acknowledge; then crash the coordinator (its local copy dies with it) and lose one letter under fair loss. One survivor never heard; the other heard and crashed before persisting — or simpler: coordinator counts its own send too.

Hint (Problem 6.10)**.** (b) The query phase makes the writer's stamp exceed every stamp lodged before its invocation — exactly what the private sequence gave for free. Ties: lexicographic (stamp, writer).

Hint (Problem 6.11)**.** Monotone stores make every lodging idempotent and order-insensitive; a repair is just a write-back with an old stamp, and line 7 discards it wherever it has been superseded.

6.10 Solutions

Tier I in full (tersely); Tier II in full for Problems 6.6, 6.7, and 6.8 (the last is the sabotage certification), sketch for Problem 6.9; hints only for Tier III.

Solution (to Problem 6.1)**.** (a) Typical: two thousand writes, sixty thousand pounds at risk; worst: twenty-four thousand writes, seven hundred twenty thousand pounds. (b) "Two milliseconds on every order, to make losing any confirmed order impossible" — a sentence merchants sign quickly. (c) A partition stretches the lag (the exposure), but not the per-write worth; and it converts "typical" to "worst" precisely when failover is most likely — the card's two rows correlate at the wrong moment, which is the honest reading of the GitHub afternoon.

Solution (to Problem 6.2)**.** (a) Durability: one copy; visibility: none promised; flicker: trivially (staleness unbounded); availability: full. (b) Durable everywhere on ack, so any read sees it (visibility by containment); but W=3 blocks writes with one replica down — the unanimity trap of Chapter 4's Part One. (c) Visibility yes (overlap); flicker possible (Exhibit 6.3); one-down: writes and reads both proceed. (d) As (c) in fair weather; under failure the overlap is probabilistic — a strict read may miss a sloppy write entirely until handoff completes. (e) Atomic register behaviour per Theorem 6.2; the toll is the write-back round on disagreeing reads.

Solution (to Problem 6.3)**.** (a) Copies at Bingo, Tuppy, and Barmy-with-hint (first healthy three); twelve hinted versions in Barmy's larder addressed to Gussie. (b) One stream Barmy → Gussie of the latest hinted versions (superseded intermediates may be coalesced), each discharged on confirmation. (c) The school's read consults the first R *healthy* members — {Bingo, Tuppy} or with Barmy substituted; a read landing on {Tuppy, Barmy} sees everything; on {Bingo, Tuppy} likewise (both are home members holding all twelve); the misfortune requires reads consulting a member that received neither the writes nor the hints — possible only under further failures; state it as: sloppiness moved the risk from write-time refusal to read-time likelihood. (d) Sloppy quorums trade the *certainty* of quorum intersection for availability, leaving intersection to probability plus the nets.

Solution (to Problem 6.4)**.** (a) Rivals: each ahead in one coordinate. (b) Context dominates both: supersedes both siblings; new version (B:2, T:3). (c) Context covers only A's Bingo entries... compare: (B:2) versus A: not $\geq$ (A has T:1); versus B: not $\geq$. The write joins as a third sibling: $\{A, B, C\}$ with C = (B:3) (coordinator Bingo advancing its own entry). (d) A timestamp would have kept exactly one of the three — destroying two concurrent rivals rather than presenting them.

Solution (to Problem 6.5)**.** (a) A thousand leaves, branching thirty-two: depth two ($32^2 = 1024$). (b) Identical: one exchange. One differing leaf: roots, then one node's thirty-two children, then the leaf's thirty-two: at most sixty-five plus the range transfer. Eight ranges: bounded by $1 + 32 \cdot 8 \cdot 2 = 513$ fingerprints. (c) Shipping the library costs a thousand leaf-ranges' data; the descent costs hundreds of fingerprints — three orders and a bit, in the lemma's favour, growing with the library while the descent grows only with the disagreement.

Solution (to Problem 6.6)**.** (a) Induction down the chain: node i+1 receives exactly node i's applied sequence over a reliable FIFO link and applies in arrival order; base: the head's own sequence. Hence the tail's applied sequence is a prefix of every node's, and all agree on order. (b) The tail's applications and services are events of one process — totally ordered by its local sequence. Real time: a write's acknowledgment follows its tail-application; a read's response follows its tail-service; so if $\text{resp}(o_1) < \text{inv}(o_2)$, o_1's tail event precedes o_2's. Read correctness: the tail serves its local state, i.e., exactly the writes tail-applied so far, latest last. (c) If the head acknowledged: a write durable nowhere past the head is confirmed; head dies; the tail never applies it — a confirmed write invisible forever to reads (and lost): both atomicity and durability die. The violation trace is Exhibit 6.3's cousin with the chain's geometry. (d) Read-at-head returns state including unacknowledged writes — exposing values that may yet vanish (the converse sin); writes never wait on reads, so tail reads cost writes nothing save the tail's cycles.

Solution (to Problem 6.7)**.** (a) A completed write lodged at W replicas; a later non-overlapping read consults R; $R + W > N$ gives a common replica (Lemma 4.1); versions carry the writer's rising stamps, replicas store monotonically, so the common replica reports a stamp $\geq$ the completed write's, and the read adopts the max $\geq$ it; no higher stamp exists absent later writes. (b) By no-creation every returned pair was stored, every stored pair was sent by the writer; monotone stores bar revival of superseded stamps at any single replica; the adopted max is thus some genuinely-written, never-superseded-at-its-replica value. (c) The Exhibit 6.3 run: aunt's read adopts stamp two; uncle's later read adopts stamp one — in any total order extending real time the uncle's read follows the aunt's, whose latest preceding write must then be the stamp-two write; returning stamp one violates the read-correctness clause. (d) Blocking write-back before answering: Theorem 6.2.

Solution (to Problem 6.8)**.** (a) The execution, drawn as Exhibit 6.5. Coordinator Bingo receives the write; *sends* STORE to Tuppy and Gussie and — counting sends — acknowledges Bertie at once, while persisting locally in the background. Then: Bingo crashes before his local persist completes; the letter to Tuppy is lost (fair loss, clause two); the letter to Gussie dawdles and is delivered — to a Gussie who crashes after receipt, before the durable store, and recovers empty of it. Client acknowledged; no surviving durable copy anywhere. Even the gentler variant — only Bingo's crash — leaves one copy where two were promised: the W=2 durability contract is void either way. (b) The card's exposure line: acknowledged-but-durable-nowhere is recovery-point loss requiring no failover at all — loss without even a primary death; magnitude: every write inside one postal round trip, at all times. (c) All three replicas answer subsequent reads with the old version (nothing durable arrived); read repair propagates only what some replica *has* — it cannot heal what nowhere exists; the acknowledged write is simply gone, and no net notices, because every ledger is internally consistent: Chapter 1's silent divergence, upgraded to silent void. (d) Count confirmations of durable application, as Box 13 line 3 says; the honest cost is one round trip plus the replicas' fsync — and the percentile commandment is served lawfully by batching fsyncs and by sloppy quorums, which trade *address*, never *existence*. Fatality certified.

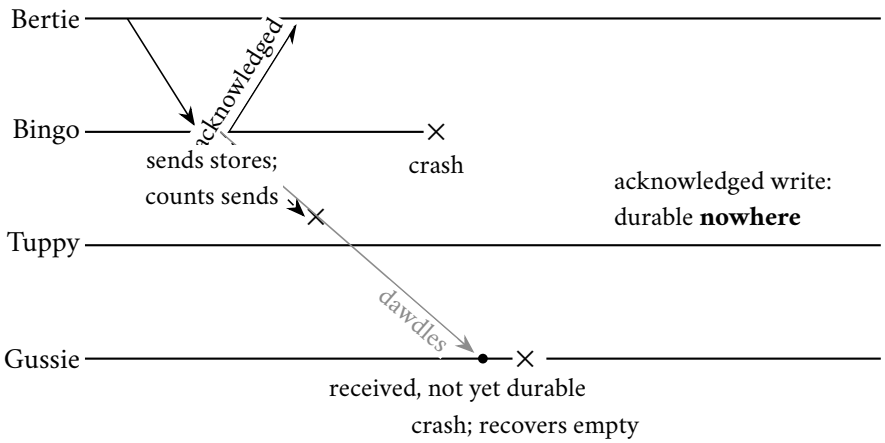


Exhibit 6.5 (acknowledged at the send; Solution 6.8). The coordinator counts letters posted, not durability confirmed; two ordinary misfortunes later, the confirmed write exists nowhere and no net can heal a void. Fatality certified.

Solution (to Problem 6.9)**.** (Sketch.) Address, phone, cart: sibling mode (loss catastrophic; merges: field-wise ask-the-user, ask-the-user, set union). Marketing preferences: LWW acceptable if truly "latest intent wins" — anomaly: a concurrent opt-out and opt-in resolve by clock, worst case a regulatory complaint; many shops therefore prefer siblings here too. Last-seen: LWW-unsafe as a raw timestamp write (clock skew regresses it); rewrite as max-merge — a monotone gauge is a semilattice, and Chapter 7 will name that observation into a theory.

Solution (to Problems 6.10 and 6.11)**.** By standing policy: hints only (the Hints section above). For Problem 6.10(d): two round trips for both operations (query then store), against ABD's one for writes and one-or-two for reads.

6.11 The Reading

DeCandia et al., "Dynamo: Amazon's Highly Available Key-Value Store," SOSP 2007. The chapter's text. Read §4 against Box 13 mechanism by mechanism; §4.4's version-vector discussion against Def. 6.6; the evaluation's percentile tables against Part One's second commandment. Note on every page the watermark: each mechanism is a priced answer to *always writeable*.

Attiya, Bar-Noy, and Dolev, "Sharing Memory Robustly in Message-Passing Systems," JACM 1995. The register theorem in the founders' hand. Read after the proof above, for the pleasure of the original's economy; their multi-writer remarks seed Problem 6.10.

van Renesse and Schneider, "Chain Replication for Supporting High Throughput and Availability," OSDI 2004. Short and geometric; read §2–3 against Problem 6.6, and the reconfiguration section for what Exhibit 6.1 deliberately excluded.

The shelf. The GitHub incident analysis of October 2018 — read with a stopwatch: mark where forty-three seconds becomes twenty-four hours. Terry et al., "Managing Update Conflicts in Bayou" (SOSP 1995) — dependency checks and merge procedures in the ancestors' hand. Preguiça et al. on dotted version

vectors — the footnote's machinery. Demers et al., "Epidemic Algorithms for Replicated Database Maintenance" (PODC 1987) — the gossip mathematics, lovely throughout. Bailis and Kingsbury, reread with landlord eyes.

Standing companion: Kleppmann, chapter 5. The replication chapter proper: his taxonomy of leader, multi-leader, and leaderless designs is this chapter's grid in prose, and his "problems with replication lag" section is Chapter 7's doorstep.

Chapter 7

Consistency: A Field Guide

Companion to Episode Seven, the season's hinge and the volume's widest chapter. The word "consistent," four professions' four terms, pinned and dissected; the client's-eye doctrine — models as sets of permitted histories, contracts silent on machinery; linearizability defined with teeth, three debts retired at a stroke, locality proved; the descent through sequential, causal, session, and eventual landings; the star exhibit staged and varied, and the hierarchy ruled; the transactional axis, where serializability has no clock and snapshot isolation admits the two doctors' write skew; the kettles wed under the strong-eventual-consistency theorem, proved in full; CALM at the horizon; and the half-term audit, briefly ruled.

7.1 Part One — The Client's-Eye Doctrine

The most overloaded word in computing has four professions behind it, each employing a different technical term under the one spelling. To the theorist of Chapter 3, *consistent* is the C in CAP — and it means, precisely, linearizability: the replicated service behaving as one up-to-date copy. To the database professional it is the C in ACID — something else entirely: a transaction carries the database from one state satisfying the application's invariants to another — a statement about *your* rules rather than the system's timing, and famously the one letter of that acronym naming an obligation of the application rather than the engine. To the replication engineer of Chapter 6 it is the degree to which the copies agree — a spectrum with "eventual" at one end, a third meaning. And to the working programmer, "the data looks consistent" means that nothing on the screen embarrasses anyone this afternoon. Contracts have been signed across that ambiguity; postmortems have been written across it. Stage the collision as a single meeting: the architect says the new store is consistent, meaning the replicas converge; the database administrator hears the ACID letter and assumes the invariants are enforced; the product manager hears the programmer's sense and assumes the screens will agree; and the theorist in the corner hears the CAP letter and quietly notes that the system cannot possibly mean *that*, because it stayed available through last month's partition. Four nods, one word, zero shared propositions — and engineering resources committed on the strength of the agreement. This chapter pins the word to the bench: every promise a replicated system can make about what its clients observe, arranged in a hierarchy, priced, and given its government name.

The territory possesses something rare in this field: a consumer-protection bureau. Since 2013, Kyle Kingsbury's Jepsen harness has been taking the databases of the market at their brochures' word. The method is this chapter's doctrine in overalls: the harness raises the system under test as a small estate of nodes; a crowd of client processes performs randomly generated operations — reads, writes, increments, compare-and-swaps — while the harness, playing Chapter 1's documentary villains, cuts the network into halves, pauses processes mid-stride, and skews clocks; every invocation and every response is logged with

its time; and afterwards a checker searches the transcript for a lawful explanation under the model the vendor advertised. No source code is consulted; no benefit of the doubt is extended. Vendors who claimed linearizability were shown histories their systems had produced that no linearizable system could produce; a document store beloved of startups was shown to lose acknowledged writes under partition; a coordination service was caught serving reads it had promised were fresh. Marketing pages were quietly rewritten across an industry. It is the most wholesome sustained act of consumer protection the field has seen, and it made the formal definitions below commercially load-bearing: this vocabulary is the language in which the market's claims are now audited. The chapter is also the season's hinge — the customs house between the machinery-building first half and the industrial second — in which every guarantee the first six chapters built is weighed, named, and given its price tag, so that the chapters to come may shop rather than build.

One doctrine governs everything, and adopting it is half the education: *a consistency model is a contract about observable histories, and it says nothing about implementation.* Nothing about quorums, logs, leaders, vectors; only about what clients can be shown. The vocabulary, installed with care because every definition in this chapter compiles down to it: clients interact through *operations*, and each operation has two visible instants, *posted* and *answered* — formally the invocation $\mathrm{inv}(o)$ and the response $\mathrm{resp}(o)$, with the open interval between them the operation *in flight*; the whole discipline lives in taking that interval seriously. "Did not overlap" becomes real-time precedence: $o \prec o'$ iff $\mathrm{resp}(o) < \mathrm{inv}(o')$. "Merely concurrent" is neither $o \prec o'$ nor $o' \prec o$ — the intervals overlap. The whole transcript is the *history* H: every invocation and every response, from all clients, with their times — and note what a history deliberately forgets: everything inside the machine room. Two systems that can produce exactly the same histories are, to any contract, the same system; this is why the vocabulary transfers wholesale from the shared-memory multiprocessors where it was born to planet-scale storage. Thirty seconds of history, so the notation has a sound: Bingo invokes a write, setting the meeting hour to seven; Tuppy invokes a read; Bingo's write responds, confirmed; Tuppy's read responds, with the *old* hour. The transcript records that the two operations genuinely overlapped — so no contract in this chapter may be scandalized by either answer Tuppy received — and declines to record which replica served him, what the quorums were, or whether the machinery was a chain, a parliament, or a boy on a bicycle.

A model, then, is simply a *set of permitted histories* — a nest of fenced paddocks on the one field of every conceivable transcript. The system promises: every transcript I will ever produce lies in this set. Stronger models are smaller sets — fewer histories permitted, more comfort for the client, more coordination for the implementor; weaker models are larger sets — cheaper machinery, stranger afternoons. Two consequences fall out before a single definition is stated. Strength comparisons become containment checks — one model is stronger than another precisely when its paddock lies inside the other's — which is why the territory can be drawn as a lattice at all. And a system may lawfully be *better* than its contract: a quiet, healthy cluster produces summit-grade transcripts all week under a floodplain contract; the contract is a floor beneath behaviour, not a description of it, and the audit question is never what the system did on a calm Tuesday but what the contract permits on the worst night of the year. Testing, in the Kingsbury manner, is precisely membership checking. One more piece of furniture makes the definitions tractable: the *sequential specification* — what the object would do on one machine, one operation at a time — and every model in the guide is a rule about how a messy concurrent history may be *explained* by some well-behaved one-at-a-time story; the models differ only in what "explained" is allowed to forget. The chapter's models, fixed in the box below before any formal claim is made:

Model of this chapter

For the hierarchy (Defs. 7.1–7.9, Thms. 7.3 and 7.5). Clients interact with named shared objects through operations; the system is a black box producing histories. Asynchrony, crashes, and partitions live *inside* the box and appear only through the transcripts the box can emit. Client clocks play no role; "real time" is the external observer's line on which invocations and responses are placed. Histories are *well-formed*: each client has at most one operation in flight (its own operations are sequential).

For transactions (Defs. 7.11–7.12, Prop. 7.6). Transactions are bracketed programs of reads and writes over named items, submitted by clients; the store is multiversion; commits are atomic events. Transaction programs may branch on what they read (the doctors' guard is a program, not a history).

For CRDTs (Defs. 7.13–7.14, Thm. 7.7). n replicas, each holding a state from a set S; updates execute at any single replica without coordination; replicas exchange *states* over an anti-entropy channel (Chapter 6's nets): eventual, unordered, possibly duplicated delivery to every correct replica; no clocks, no quorums, no leader. Crash-stop replicas simply cease gossiping.

The formal vocabulary follows the doctrine exactly — the history first, then the one-machine story it must be explained by.

Definition 7.1 (histories)**.** An *operation* o on object x by client c comprises an invocation at time $\mathrm{inv}(o)$ and, if completed, a response at time $\mathrm{resp}(o) > \mathrm{inv}(o)$, carrying argument and return values. A *history* H is a finite set of operations, possibly with some *pending* (invoked, unanswered), well-formed per the model box. Real-time precedence: $o \prec o'$ iff $\mathrm{resp}(o) < \mathrm{inv}(o')$; operations unrelated by $\prec$ are *concurrent*. $H|x$ and $H|c$ denote the restrictions to object x and client c. A *completion* of H removes some pending operations and supplies responses to the rest. A *consistency model* is a set of histories; H satisfies the model iff some completion of H lies in the set; model A is *at least as strong as* B iff $A \subseteq B$.

Definition 7.2 (sequential specification)**.** A *sequential history* is one whose operations are totally ordered by $\prec$ (no overlap). Each object type fixes a *sequential specification*: the set of single-object sequential histories it accepts (a register: every read returns the latest preceding write, or the initial value; a set: membership per the adds and removes so far; a queue: first in, first out). A multi-object sequential history is *legal* iff each per-object restriction lies in its object's specification.

One admission before the climb, made freely and with its citation.

Remark 7.3 (checking is hard)*.* Deciding whether a given complete history of reads and writes is linearizable is NP-complete in general (Gibbons–Korach 1997); the search over orderings of concurrent operations is real. The practical bureaus prune: short transcripts, simple objects, per-key locality (Thm. 7.3 audits shard by shard). The definitions are crisp; auditing against them is work.

7.2 Part Two — The Summit: Linearizability, With Teeth

The summit — the strongest of the landings — retires three debts at a stroke: the C of CAP, left as a promissory note in Chapter 3; the phrase *behaves as a single machine*, banked unexamined in Chapter 5; and the fine print on lease-served reads, deferred the same evening. All three were always the same debt, and here is the payment, from Herlihy and Wing's 1990 paper in TOPLAS.

Definition 7.4 (linearizability; Herlihy–Wing)**.** A complete history H is *linearizable* iff there is an assignment of a *linearization point* $p(o)$ to each operation, with $\text{inv}(o) < p(o) < \text{resp}(o)$ and all points distinct, such that the sequential history executing the operations instantaneously at their points is legal. Equivalently: there is a legal sequential history S on the same operations whose total order extends $\prec$. A history with pending operations is linearizable iff some completion is.

The definition in the plain register: every operation *appears* to take effect atomically, at some moment — the *instant*, formally the linearization point — while it was genuinely in flight, and the resulting one-at-a-time story must be a lawful story for the object on a single machine. Two clauses carry all the weight: the story must be *sequentially lawful* — reads return the latest write, sets contain what the story says — and the instants must respect *real time*: non-overlapping operations may never be reordered in the explanation. Overlapping operations may be ordered either way; that freedom is the definition's entire mercy, and it is exactly the freedom Chapter 2 taught us the world actually offers.

The founders' own first exhibit was a queue, and it earns its keep. Bingo enqueues an apple; overlapping him, Tuppy enqueues a pear; then, after both responses, Gussie dequeues — and receives the pear. Lawful: the enqueues overlapped, so the explanation may order the pear first, and Gussie's dequeue then lawfully yields it. But let Gussie dequeue *twice* and receive the pear both times, and no ordering of the enqueues can save it — no sequential queue surrenders the same fruit twice; the fault lies against the sequential specification, not against time. Two ways to die, and the definition polices both; every checking drill at this desk (Problems 7.1 and 7.2) is a hunt for a witness to one or a proof that both hunts fail.

Now test the instrument against Chapter 6's crime scene, because this is what "teeth" means. The aunt read the new balance; the uncle, invoked *after* her response, read the old. The write overlaps both reads, so its instant may be placed variously — but the aunt's read and the uncle's read do not overlap: hers must precede his. For her to read new, her instant follows the write's; for him to read old, his precedes the write's — which puts his instant before hers. Contradiction; no assignment exists; the history is not linearizable — and quorums were never mentioned. The verdict came entirely from the transcript. That is the client's-eye doctrine paying rent, and it is literally how the Jepsen harness convicts a database. The instants, note, are only mystical until they are a line of code: in Chapter 6's chain, a write's point is its application at the tail — the single place where it becomes simultaneously durable and visible; in Chapter 5's replicated state machine, every command's point is its application after commitment — exactly why serving reads through the log preserves the summit and a lagging follower's copy forfeits it; in the ABD register, a write's point falls within the lodging of its stamp at a majority, and a read's may be dragged forward by the write-back it performs on the writer's behalf. The definition demands none of this specificity — it asks only that the instants *exist* — but diagnosis is faster when one knows where implementations customarily keep them.

The three payments, formally. *The C in CAP:* Gilbert and Lynch's proof constructed exactly a history with no lawful assignment — a completed write on one shore of the partition, a later non-overlapping read of the old value on the other; real time demands the read see the write, and the severed cable says it cannot. Weaken the C to any junior tier and the theorem fails to bite — sequential, causal, the whole coordination-free storey below all remain purchasable on both sides of a partition; the theorem forbids exactly the summit, exactly during the severance, exactly for the shore that cannot raise a majority, and no more. Most invocations of CAP in industry meetings are invocations of a theorem about linearizability by parties who have not yet decided whether they need linearizability. *Behaves as a single machine:* that phrase *is* this definition — the single machine is the sequential story, and "behaves as" is the existence of the assignment; Chapter 5's state machine earns the phrase precisely when reads are served in a way that admits the instants. *The lease fine print,* in one sentence: a lease converts a timing assumption into a consistency guarantee, and inherits the assumption's failure modes — the lease is what guarantees no successor's write can have an instant during the old leader's tenancy; let the drift bound fail — one paused leader, one fast

clock — and the transcript can contain a stale read invoked after a new write's response, and audits of the Jepsen grade have caught precisely this in the wild.

The definition's second reading, and the pending-operations clause — the place where the harness spends much of its cleverness, because a crashed client's unacknowledged write is precisely the case that separates honest stores from lucky ones.

Remark 7.5 (the two clauses, and pending operations). The equivalence in Def. 7.4 is Lemma 7.1 read in both directions: points give an extension of $\prec$ trivially; an extension of $\prec$ is realizable by points. Two independent ways to fail follow: the story may be illegal (a queue surrendering the same fruit twice), or every legal story may violate $\prec$ (the aunt and the uncle). The completion clause is where audits earn their keep: a crashed client's unacknowledged write may be declared either absorbed or void, but the checker must find *one* coherent verdict for the whole transcript — a write that visibly took effect cannot later be ruled void.

The equivalence rests on a small lemma with a season of work in it — any total order extending real time is realizable by instants — proved at once, since Theorem 7.3 leans on it too.

Lemma 7.1 (interval realization). *Let T be a finite set of operations and $<_T$ a total order on T extending $\prec$. Then there are distinct points $p(o) \in (\mathrm{inv}(o), \mathrm{resp}(o))$, one per operation, whose time order is exactly $<_T$.*

Proof of Lemma 7.1. Enumerate T in $<_T$-order as $o_1 <_T o_2 <_T \cdots <_T o_n$. Let ε be smaller than one n-th of the least positive gap between any two of the finitely many invocation and response times. Define $p(o_1) = \mathrm{inv}(o_1) + \varepsilon$ and, for $k \geq 2$,

$$p(o_k) \;=\; \max\big(p(o_{k-1}),\, \mathrm{inv}(o_k)\big) + \varepsilon.$$

The points are strictly increasing, hence distinct and ordered as $<_T$; each exceeds its own invocation by construction. It remains to show $p(o_k) < \mathrm{resp}(o_k)$. Unwinding the recursion, $p(o_k) = \mathrm{inv}(o_j) + m\varepsilon$ for some $j \leq k$ and $m \leq n$. Suppose $p(o_k) \geq \mathrm{resp}(o_k)$. By the choice of ε, the slack $m\varepsilon$ cannot bridge a genuine gap, so $\mathrm{inv}(o_j) \geq \mathrm{resp}(o_k)$, that is, $o_k \prec o_j$. Since $<_T$ extends $\prec$, this forces $o_k <_T o_j$, contradicting $j \leq k$ (if $j = k$ the interval would be empty, contradicting $\mathrm{inv}(o_k) < \mathrm{resp}(o_k)$). Hence every point lies strictly inside its interval. □

One clause from the founders' paper corrects a natural slander: linearizability is not a synchronization scheme, and it forbids no concurrency. The definition never requires an operation to wait upon another — a property the paper proves and names, and what separates the *contract* from the coarse machinery (the one big lock, the one grand queue) that most cheaply satisfies it. The contract constrains transcripts; how much parallelism survives beneath is engineering's affair, and the lock-free trades have spent three decades demonstrating that the answer is: a very great deal.

Proposition 7.2 (nonblocking). *Let H be a linearizable history of total operations (the specification answers every invocation in every state) with a pending invocation i. Then some response r makes H extended by r linearizable. Linearizability never forces a pending operation to wait.*

Proof of Proposition 7.2. Let S be a linearization of H without the pending invocation i. The specification is total, so from the state reached by executing S there is a legal response r for i's operation. Append i's operation at the end of S with response r, and give it a linearization point later than every point of S and later than $\mathrm{inv}(i)$, with $\mathrm{resp}(i)$ later still. The extended story is legal (it extends a legal story by a legal step) and extends $\prec$ (nothing follows i's operation in it, and its point lies within its interval). Hence the extended history is linearizable. No completed or concurrent operation had to be awaited: the clause separates the contract from the coarse machinery that most cheaply implements it. □

One more property, quiet and profound — the summit's superpower: **locality**. A system composed of linearizable parts is linearizable as a whole: if every individual object — each key, each queue, each register — separately satisfies the definition, then any history over all of them together does too. No cross-object coordination, no global anything. It sounds like a bookkeeping lemma; it is the licence for the entire modern storage industry — build each shard linearizable, and the estate of ten thousand shards is linearizable, free — and it does silent load-bearing work under everything in Chapter 9's partitioned world. The proof's strategy is one sentence, and the sentence is the point: each object's instants are placed on the *shared* real-time line, and real time is the one resource every object already agrees about.

Theorem 7.3 (locality; Herlihy–Wing)**.** *A complete history H is linearizable if and only if $H|x$ is linearizable for every object x.*

Proof of Theorem 7.3. ($\Rightarrow$) A linearization of H restricts to a linearization of each $H|x$: the restricted order still extends $\prec$, and legality of the whole story is by definition per-object legality of its restrictions.

($\Leftarrow$) Suppose each $H|x$ is linearizable: a legal sequential order $<_x$ on $H|x$ extending $\prec$ restricted to x's operations. By Lemma 7.1 choose points $p_x(\cdot)$ for the operations of $H|x$, inside their intervals, ordered as $<_x$; perturbing by less than the least gap, make all points across all objects distinct. Pool every operation of H with its point and let S be the sequential history executing operations at their points in time order. Three checks. *Same operations:* by construction. *Legality:* S's restriction to object x is ordered exactly as $<_x$ (the points for x realize $<_x$, and pooling with other objects does not reorder them), which is legal; a multi-object sequential history with legal restrictions is legal (Def. 7.2). *Real time:* if $o \prec o'$ then $p(o) < \mathrm{resp}(o) \leq \mathrm{inv}(o') < p(o')$, so S's order extends $\prec$ — and this is the entire trick: the points of different objects are placed on the *same* clock line, and real time is the one order every object already shares. By Def. 7.4, H is linearizable. □

And the negative half of that claim is the standard interview trap: no weaker model in this guide has the property. Take two queues and a history in which each queue, examined alone, is perfectly sequentially consistent — yet any single story explaining both at once must thread both clients' program orders through both objects simultaneously, and none can. The founders' own two-queue history is set as Problem 7.6; work it once and you will never again believe that per-object guarantees of the junior kind add up to an estate-wide guarantee of any kind. Locality is the summit's private privilege: linearizability composes because real time is global; sequential consistency does not, because private timelines negotiated per object owe each other nothing.

7.3 Part Three — Down the Hierarchy

Descend now, model by model, and at each landing: the promise, the price, and the folk name it has been wearing in your systems. The summit's immediate junior is Lamport's — 1979, a two-page note about multiprocessors: keep the sequential story, keep each client's *program order*, but surrender real time *across* clients.

Definition 7.6 (sequential consistency; Lamport 1979)**.** A complete history H is *sequentially consistent* iff there is a legal sequential history S on the same operations whose total order extends the *program order* of every client (the $\prec$-order within each $H|c$) — but not necessarily $\prec$ across clients.

This is the *agreed story*: every client sees a world consistent with his own actions in order, and all clients see the *same* world — but the world may run on a private timeline unmoored from the clock on the wall. What the surrender drops: the coordination that made once-seen-forever-seen hold across clients —

Chapter 6's blocking write-back, the round trips. The definition's birthplace explains its shape: Lamport wrote it for multiprocessor hardware, and modern hardware answers — not even this. Every processor buffers its writes privately, and a core may read its own buffered write before any other core can see it, an optimization that already violates the single-agreed-story clause; hence the relaxed memory models and fence instructions of the hardware trades, the same economics from nanoseconds on a chip to continents on a cable: publishing every write to everyone before proceeding is the price of the agreed story, and both industries concluded the price was negotiable. Folk sighting, with an honest brochure for once: ZooKeeper, Chapter 5's acquaintance, serves reads from any replica by default — one agreed order of writes, program order per session, no freshness clause — and its manual offers a sync operation to purchase the missing real-time guarantee per read. Sequential consistency sold under an accurate label, with linearizability available as an upgrade at the till — the rare shop where the tiers are printed on the menu.

The next landing is where an old acquaintance waits with its papers. Surrender even the single agreed story: clients may now see *different* interleavings — but every client must see them respecting happens-before, the *roads* of Chapter 2: program order and reads-from, chained.

Definition 7.7 (causal consistency)**.** In a complete history H over registers, the *reads-from* relation links each read to the write whose value it returns. *Happens-before* is $\rightarrow = (\text{program order} \cup \text{reads-from})^+$. H is *causally consistent* iff for each client c there is a legal sequential history S_c over c's operations together with all writes in H, whose order extends $\rightarrow$ restricted to those operations. Different clients may use different S_c; concurrent writes may be ordered differently in different views.

If one operation could have influenced another, nobody, anywhere, may observe the effect before its cause; the merely-elsewhere may appear in different orders to different observers, lawfully. This is Chapter 2's corridor purchased as a contract — the causal-delivery debt, paid and priced. The price is exactly the machinery Chapter 2 built: carry the causal paperwork — vector clocks or their engineering rations — and hold mail until its ancestry has arrived. No consensus, no quorum round trips, no waiting on distant continents: causal consistency is the strongest model in this guide that remains available under partition and cheap under geography — and the flattering footnote sharpens it: there are results to the effect that, in its real-time-respecting variant (Rem. 7.10 below), it is essentially the *strongest* contract an always-available, partition-tolerant store can honour. Not a pleasant waypoint but the ceiling of the coordination-free storey, which is why the research shopfronts of the last decade — stores that record each write's dependencies and hold it at a replica until its ancestry has arrived — cluster exactly here. Its weakness is equally exact: no agreed story means no arbitrating races — two concurrent kettle-writes stay two kettles; causality orders the roads, it cannot rank the merely-elsewhere. And the sobering footnote: the paperwork is not free. Dependencies must be captured, carried, and checked; a write's ancestry fans out alarmingly under heavy read traffic — read fifty items before writing, and the write now depends on fifty histories — and the engineering literature on causal stores is, in large part, a literature on pruning, batching, and bounding that fan-out. Causal is cheap compared to consensus; it is not cheap compared to nothing.

The practitioner's landing follows, where four folk names finally meet their government. Terry and colleagues — the Bayou gentlemen, Chapter 6's receipts-keepers — published in 1994 a small taxonomy of promises scoped not to the whole world but to one *session*: one client's continuing conversation.

Definition 7.8 (the session guarantees; Terry et al. 1994)**.** Scope: one session's operations, against the whole history. *Read your writes*: every read sees all session-preceding writes of its own session (returns their value or a later one in the object's version order). *Monotonic reads*: the set of writes visible to a session's reads grows monotonically along the session. *Monotonic writes*: the session's writes take effect everywhere in session order. *Writes follow reads*: any write of the session is ordered, everywhere, after the writes its preceding reads saw. All four together, imposed on every session and closed under transitivity, yield causal consistency; each alone is purchasable with session-local tokens (Box 15).

Each of the four was named after a bruise; each earns its thirty seconds of theatre. *Read your writes:* Bertie posts the notice, refreshes the page, and the notice is gone — routed to a replica his own write had not reached; Chapter 6's stale-follower farce, now outlawed by name. *Monotonic reads:* the aunt sees the new price, hands her own device to the uncle, and the price reverts before his eyes — the session hopped replicas backwards; the flicker, outlawed per session. *Monotonic writes:* Bertie posts a notice and then a correction, and at some replica the correction is applied *before* the notice it corrects has arrived — amendments to documents that do not yet exist. *Writes follow reads:* Bertie reads Tuppy's question and posts the answer, and at a distant replica the answer stands above a question that has not yet appeared — testimony preceding the evidence. The provenance is worth savouring: Bayou was built for machines that expected to be *disconnected* — the roaming laptops of the early nineties, syncing when docked — which is exactly why its promises are scoped to the session and payable in tokens rather than rounds. The *session receipts* are those tokens: the session carries the stamp of its last write and of the freshest state it has read, and every replica answering the session first checks the receipt — am I at least this new? — and otherwise waits or redirects. Four guarantees, four disciplines about when the receipt is updated and consulted: a token in a cookie, not a quorum in a datacentre, and the highest tier purchasable from the machinery for nothing more than honest bookkeeping.

Protocol — Box 15. The session receipts (checked against Terry et al. 1994, §§4–5)

Each session keeps two receipts: a read-set R and a write-set W of write identifiers. Each server s knows the set $\mathrm{DB}(s)$ of writes it has applied.

1: **on read at server s:**
2: *read your writes:* require $W \subseteq \mathrm{DB}(s)$, else wait or redirect
3: *monotonic reads:* require $R \subseteq \mathrm{DB}(s)$, else wait or redirect
4: serve the read; add to R the identifiers of the writes the answer depends on
5: **on write at server s:**
6: *monotonic writes:* require $W \subseteq \mathrm{DB}(s)$
7: *writes follow reads:* require $R \subseteq \mathrm{DB}(s)$
8: apply; add the new write's identifier to W

Each guarantee is exactly one guard; enforce the guards you have sold. The receipts are session-local — a token in a cookie, not a quorum in a datacentre. Practical rations summarize R and W by version vectors rather than identifier sets (Terry et al., §6).

At the bottom lies the floodplain — eventual consistency, the beat this course has owed the estate agents since the founding plan. The promise: if writing ceases, replicas eventually agree. Attend to the grammar.

Definition 7.9 (eventual consistency)**.** An infinite execution is *eventually consistent* iff every update is eventually applied at every correct replica and, if from some point onward no further updates are issued, all correct replicas' states (equivalently, all subsequent reads) eventually agree. This is a *liveness* property: a promise about the limit, not about any prefix.

Every other model in this chapter is a *safety* property — a constraint on every prefix of every history: this you may never show a client. Eventual consistency constrains nothing at any finite moment — any read may return anything, any interleaving, any antiquity, and the contract is unbroken so long as convergence remains scheduled for some unspecified future. "Eventually" is an estate agent's word — "up-and-coming," "full of potential," a promise about a future no one will be present to audit. Stated and proved as a proposition, the whole argument being that any finite history extends to a converging execution.

Proposition 7.4 (the costume)**.** *Every finite history over registers is a prefix of some eventually consistent execution. Eventual consistency, taken alone, therefore forbids no finite transcript whatsoever — it is a liveness property wearing a safety costume.*

Proof of Proposition 7.4. Given any finite history H over registers, extend it as follows: issue no further updates; deliver every write of H to every replica (the model's channels eventually deliver); let each replica adopt, once all deliveries have landed, the same deterministic choice among the delivered writes (say, the one latest in an agreed tie-breaking order on write identifiers — computable locally from the same delivered set). All subsequent reads at all replicas return that value; the execution converges. Nothing in this construction constrained H itself: every read in H may have returned anything at all. Hence the floodplain's paddock contains every finite transcript, which is the formal content of "liveness in a safety costume." □

This is not a dismissal: convergence is a real and valuable property, Chapter 6's nets exist to deliver it, and for carts it is the honest choice deliberately made. The objection is solely to the costume: "our system is eventually consistent" answers no question a client can ask about any read it will ever perform, and the honest follow-up — drilled in Problem 7.5 — is always: *eventual, plus what?* Plus session guarantees? Plus causality? Plus bounded staleness with a number? The floodplain is habitable; one merely insists on seeing the flood insurance. And because it deserves actuaries as well as scorn, note that the trade has learned to measure it: Bailis and colleagues built exactly the instrument the estate agent never volunteers — probabilistic staleness bounds, computed from measured replication delays, answering *how eventual* in the only useful currency: after so many milliseconds, with such-and-such confidence, a read returns the latest write. On a healthy cluster the answer is often startlingly good, which explains the floodplain's popularity better than any brochure — most reads on most days are perfectly fresh, and the contract's weakness is invisible until the day it is not. For the whole territory, Kingsbury's consistency atlas — the models of this chapter and their transactional cousins arranged as a lattice, edges marking strict strength, annotations marking which models survive partitions — is assigned in the reading not as prose but as *equipment*: the one-page answer to "is this model stronger than that one," the course's standing atlas henceforth.

A guide is for carrying, so the field protocol for using the hierarchy in anger — three questions, in order. First: *who compares notes?* If distinct clients share side channels — telephones, dashboards, a common aunt — real time is observable from outside, and the summit or its strict transactional cousin belongs on the requisition; if sessions are solitary, the session guarantees may be the entire bill. Second: *what must survive partition?* If the estate must keep answering on both sides of a severed cable, Chapter 3 has spoken: nothing above the causal landing is available, and the choice is among causal, sessions, and the floodplain with insurance. Third: *what arbitrates races?* If concurrent writes must yield one agreed winner — locks, uniqueness, auctions — no landing in this part suffices at any price, because arbitration is agreement, and that requisition goes to the parliament of Chapters 4 and 5. Most designs, honestly interrogated, decompose room by room: a linearizable sliver where notes are compared, causal or session middleware for the conversation, floodplain for the analytics. The guide's purpose is not to pick one landing for the estate but the cheapest landing for each room.

7.4 Part Four — The Star Exhibit

Now the rite of passage. The distinction between the summit and its immediate junior is the subtlest gap in the guide, the one interviews probe and vendors blur, and the standing ruling is that the separating history be held whole, on one page. The estate: one register — the shop's advertised umbrella price — replicated at Bingo and Gussie; sequential consistency in force via an agreed log, reads served locally; the old price, five shillings. Bertie, at the counter, watches the new price go up: he **writes ten shillings** — the write lands

in the log, applies at Bingo, and *responds*: done, confirmed, finished. Bertie then — the load-bearing act — **telephones his aunt**. The telephone is a *side channel*: information flowing outside the transcript, honoured only by real-time-respecting models. "The price is ten shillings now," he says; the aunt, a diligent woman, rings off and **reads the price** — her read invoked after a call that began after the write's response, the real-time order total and unambiguous. Her read is served at Gussie, where the log entry, lawfully lagging, has not yet applied. She reads: **five shillings**.

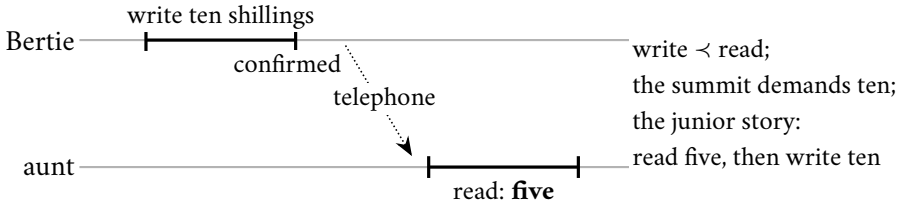


Exhibit 7.1 (the star: sequentially consistent, not linearizable). Bertie's write completes; the telephone (a channel the contract cannot see) precedes the aunt's read; the read returns the past. Not linearizable: write $\prec$ read forces the read to see ten. Sequentially consistent: the story "read five, then write ten" keeps both one-operation program orders and is legal. The gap between the tiers is exactly as wide as the side channels among your clients.

Is the system in breach? Check the summit first. Bertie's write responded before the aunt's read was invoked: non-overlapping; any linearization must order the write's instant first; her read must then see ten. She saw five. *Not linearizable* — the assignment cannot exist. Now check the junior model, honestly. Sequential consistency demands one story, lawful, respecting each client's own order. Take the story: *the aunt's read of five, then Bertie's write of ten*. Her program order — one read — respected; his — one write — respected; the story sequentially lawful. *Sequentially consistent.* The system is in breach of nothing it did not promise — and yet the family has direct experience of the breach, because the telephone call *happened*: information travelled outside the transcript, and the junior tier is precisely the tier that declines to honour channels it cannot see. That is the whole gap, felt in the hand: linearizability respects real time — and therefore respects every side channel reality contains; sequential consistency respects only the orders each client can testify to from inside. Every support ticket of the form "I told her it was updated, and she looked, and it wasn't" was this exhibit, performed by amateurs.

Three variations wear the edge smooth. *Disconnect the telephone:* Bertie writes ten; the aunt, uninstructed, happens to read five a moment later. The transcript is identical — but nobody can now testify to a breach: the aunt did not know a write had occurred, no single client observed both sides of the reordering, and the family's entire experience is explained by the junior story. This is the precise sense in which sequential consistency is the summit *for clients who never compare notes*: the gap between the models is exactly as wide as the side channels among your clients, and for a population of strangers it is observationally nil. *Purchase the freshness check:* let Gussie's replica, on receiving the aunt's read, first confirm against the log that it holds the latest applied entry — one round trip into the machinery — before answering. Her read returns ten; the assignment exists; the summit is restored — and the invoice is itemized: that round trip, on every read, is what linearizability *costs*, and serving reads without it is what sequential consistency *saves*. The entire hierarchy folded into one design decision, examined under a telephone. *Let the read overlap the write* — invoked before his response arrived: now even the summit permits her five shillings, and no telephone can manufacture a breach, because the call itself began before the write was confirmed and so testifies to nothing. Overlap is the definition's mercy, and this is what the mercy looks like across a counter: an answer that would be a scandal a moment later is, a moment earlier, simply the truth of a world still in flight. The same history is set formally, both verdicts to be proved, in Problems 7.1(d) and 7.7(c).

With the exhibit in hand, the whole descent can be ruled formally: each containment is an implication between explanation obligations; each strictness is a two-client exhibit.

Theorem 7.5 (the hierarchy)**.** *As sets of complete histories,*

$$\text{Lin} \subsetneq \text{SC} \subsetneq \text{Causal},$$

and every model in the chain admits every history that eventual consistency (Prop. 7.4's floodplain) admits — the paddocks nest. Strictness witnesses: Exhibit 7.1 (in SC*, not* Lin*); Exhibit 7.2 (in* Causal*, not* SC*).*

Proof of Theorem 7.5. Lin $\subseteq$ SC. A linearization extends $\prec$; program order is $\prec$ within one client (well-formedness: a client's operations never overlap); so the same S witnesses sequential consistency. *Strict:* Exhibit 7.1 is sequentially consistent (the story "aunt reads five, Bertie writes ten" keeps both one-operation program orders and is legal) but not linearizable (write $\prec$ read forces the read to see ten; it saw five) — verdicts proved in the exhibit's caption and Problem 7.1(d).

SC $\subseteq$ Causal. Let S witness sequential consistency of H. In S, every read returns the latest preceding write, so each reads-from edge is ordered by S; program order is ordered by S by definition; hence $\rightarrow \subseteq <_S$ by transitivity. For each client c, let S_c be S restricted to c's operations and all writes: a restriction of a legal register story that keeps every read together with all writes remains legal (each read's latest preceding write is a write, hence retained), and its order still extends $\rightarrow$. The same S serves every client: sequential consistency is causal consistency with unanimity. *Strict:* Exhibit 7.2: the two writes are concurrent (no road connects them), so each reader's view may order them privately — both views respect $\rightarrow$ and are legal, so the history is causal; but any single story must order the two writes once, and then one reader's second read returns a value overwritten in the story before its first read's value — formally, the aunt's pair of reads forces "one then two," the uncle's forces "two then one," and no total order extends both requirements with register legality (worked in full in Problem 7.7(b)).

The floodplain containment is Prop. 7.4: no finite history is excluded, so every paddock lies inside it. □

The second strictness witness — causal but not sequential — follows, in the season's trace convention.

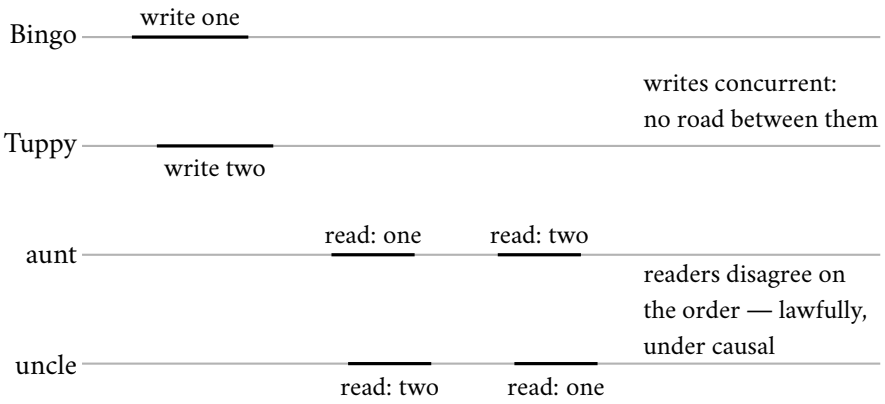


Exhibit 7.2 (causal, not sequential). Two concurrent writes to one register; the aunt reads one then two, the uncle two then one. Each view respects happens-before (the writes are unrelated by it), so the history is causally consistent. No single story serves both readers: the aunt's pair forces one-before-two, the uncle's forces the reverse (Problem 7.7). Causality orders the roads; it cannot rank the merely-elsewhere.

One correction of folklore before the hierarchy is filed, recorded formally because the corrected problem depends on it.

Remark 7.10 (real-time causal, and a folklore correction). Enrich happens-before with external order: $\rightarrow_{\mathrm{rt}} = (\text{program order} \cup \text{reads-from} \cup \prec)^{+}$; *real-time causal* consistency is Def. 7.7 with $\rightarrow_{\mathrm{rt}}$ in place of $\rightarrow$. Folklore asserts that causal and sequential consistency are incomparable; under the standard definitions this is half true — sequential *does* imply causal (Thm. 7.5) — and the genuine incomparability holds between sequential and *real-time* causal. Problem 7.7 settles all four directions.

7.5 Part Five — The Other Axis: Transactions and Isolation

Everything so far concerned single operations on single objects. Half the industry's consistency vocabulary comes from a different tradition — the database hall, where the unit is the *transaction*: several reads and writes, bracketed, demanding to appear as a unit — and the two halls' vocabularies are confused daily. The confusion is not pedantic — it has a body count, and presently it gets two doctors. The database hall's summit is *serializability*: transactions appear to execute one at a time, in *some* order — any order. Note what is present and what is absent: present, the one-at-a-time story, now over multi-step transactions; absent, any clause about real time. Serializability is the transactional cousin of sequential consistency, not of linearizability — a serializable database may lawfully execute your transaction as though it ran yesterday, provided the story stays lawful — and the industry phrase "the gold standard" elides exactly the clause the star exhibit just made physical. The two words are not synonyms, and neither implies the other (Problem 7.9 builds histories exhibiting each without the other). Demand both — one-at-a-time stories *and* real-time respect — and you have *strict serializability*, the strongest promise in the combined guide. Bank that name carefully: in Chapter 8 a company purchases it across continents with atomic clocks, and the invoice makes sense only if the name is exact.

Definition 7.11 (serializability; strict serializability)**.** A history of committed transactions is *serializable* iff some total order of the transactions, executed one at a time from the initial state, gives every read the value it actually returned (and hence the same final state). It is *strictly serializable* iff the order can be chosen to extend transaction real-time precedence ($T \prec T'$ iff T's commit response precedes T''s first invocation). Strict serializability is exactly linearizability with whole transactions as the operations; serializability is its clock-free weakening, the transactional cousin of Def. 7.6.

The missing clause, made physical because it surprises even the seasoned: a read-only transaction served from last night's snapshot is *perfectly serializable* — order it in the story before every transaction that committed today, and the one-at-a-time tale is lawful; the customer staring at yesterday's balance is receiving a legal, gold-standard, serializable view. Strictly serializable it is not: today's deposits responded before the read was invoked, and real time demands they be visible. Vendors have shipped exactly this behaviour under exactly this defence — the backup-restore defence, made formal in Problem 7.9 — and the defence is technically sound, which is the lesson: the gold standard, as the industry recites it, contains no clock. When a brochure says serializable, the trained reader asks the auditor's question: serializable *as of when?*

But the hall's daily reality is not the summit; it is a negotiated tier. Every transaction reads from a consistent *photograph* of the database as of its start — the Chandy–Lamport spirit, industrialized into what the manuals call *multiversion concurrency control*, MVCC on every vendor's feature list: a ledger of versions — and writes succeed unless two transactions wrote the *same* item concurrently, in which case one aborts: first committer wins, formally an abort on write-set intersection with a concurrently committed transaction. Reads never block, writes conflict only on overlap, performance is lovely, and the promise sounds like serializability if recited quickly.

Definition 7.12 (snapshot isolation)**.** The store is multiversion: each committed transaction T stamps its written versions with its commit point $c(T)$. A transaction reads, for each item, the newest version

committed before its start point $s(T)$ (its *photograph*), plus its own writes. Commit rule (*first committer wins*): T may commit iff no transaction T' with $s(T) < c(T') < c(T)$ has write set intersecting T's. Reads are never blocked and never validated.

Notice the asymmetry the doctors are about to exploit: the *writes* are checked for collision; the *reads* — the evidence on which the transaction acted — are checked against nothing at all. A transaction may act on evidence already obsolete the moment it acted, and commit unchallenged, provided it defaced no item that another defaced meanwhile.

Now the doctors. A hospital's rule — the application's invariant, the ACID-C sense of consistent: at least one doctor on call at all times. Two are rostered, Tuppy and Gussie, and each, at the same weary hour, opens a transaction to book off. Tuppy's photograph shows *two on call* — safe to leave, one remains; he writes himself off. Gussie's concurrent photograph shows the same; he writes himself off. Do the write sets collide? They do not: Tuppy wrote Tuppy's row, Gussie wrote Gussie's row — different items, no conflict, first committer wins never invoked, *both commit*. The roster reads: nobody on call. Each transaction, run alone, was impeccable; each read a genuinely consistent photograph; the invariant died *between* them, in the gap the photographs could not see — the rule lived across both rows while the conflict detector watched each row alone. The anomaly is *write skew*, and its formal residence is the 1995 critique that showed the industry's isolation levels were folk taxonomy. The skeleton, before the proof: disjoint write sets pass first committer wins; both photographs lawful; the cross-row invariant dies between them; and no serial order of the same guarded programs reaches the empty roster.

Proposition 7.6 (write skew; Berenson et al. 1995)**.** *Snapshot isolation admits non-serializable executions. In particular, for the doctors' schema (two rows; guard: book off only if the photograph shows at least two on call), snapshot isolation admits a history in which both guarded programs commit and the roster empties, while* no *serial execution of the same two programs can empty it. Serializability, by contrast, preserves every invariant that each transaction program preserves in isolation.*

Proof of Proposition 7.6. The admitted history. Items: rows t (Tuppy) and g (Gussie), both initially `on`. Programs: P_T reads both rows and, iff at least two read `on`, writes $t :=$ `off`; P_G symmetrically writes $g :=$ `off`. History: $s(T_1) = s(T_2) = \sigma$, later $c(T_1) < c(T_2)$, no other transactions. Under Def. 7.12: both photographs at σ show (`on`, `on`); both guards pass; write sets are $\{t\}$ and $\{g\}$ — disjoint — so first committer wins is vacuous and both commit. Final state (`off`, `off`).

No serial equivalent. In either serial order of the two guarded programs, the second reads the first's committed write: its photograph shows exactly one `on`; its guard fails; it writes nothing. Both serial executions end with exactly one row `off`. The admitted history's reads (both seeing two `on`) and final state (none on call) match neither serial execution, so it is not serializable.

Serializability preserves guarded invariants. If each transaction program, run alone from a state satisfying invariant I, ends in a state satisfying I, then induction along any serial order preserves I; a serializable history's reads and final state agree with some serial execution's, hence its final state satisfies I. The doctors' guard preserves "at least one on call"; snapshot isolation nevertheless emptied the roster — which locates the anomaly precisely in the gap between Def. 7.12 and Def. 7.11: reads are never validated, and the invariant lived across both rows while the commit rule watched each row alone. □

The crime scene, drawn once for the whole guide.

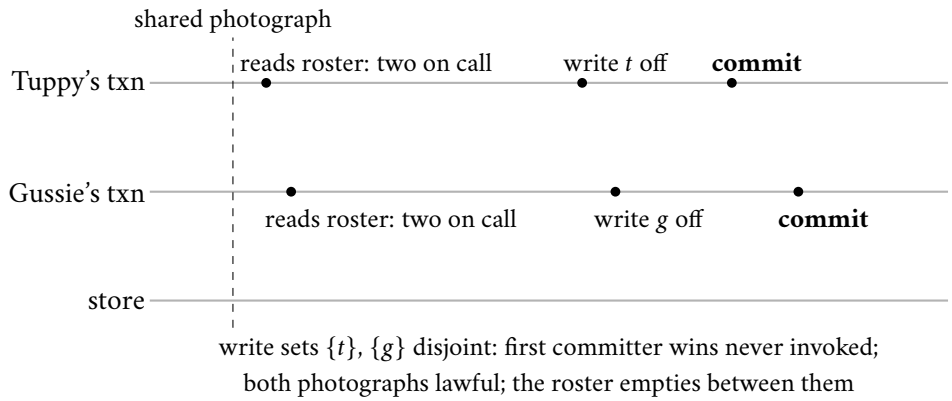


Exhibit 7.3 (write skew: the two doctors). Both transactions read the same photograph, satisfy the guard, and write different rows; snapshot isolation commits both (Prop. 7.6). No serial order of the same guarded programs empties the roster. The invariant lived across both rows; the commit rule watched each row alone.

The shape, abstracted so it can be found without the doctors: *read a neighbourhood, decide on the evidence, write a corner the neighbourhood-rule watches but the detector does not.* Two registrars each check that a username is unclaimed and each insert their claim as a fresh row; two withdrawals each verify that the combined balance of a linked pair of accounts covers the amount and each debit a different account; two officers each confirm the vault will still have a second keyholder tonight and each sign themselves out. Rosters, uniqueness, floors that span accounts, budgets that span line items: any rule that lives across rows is a doctors' rule, and snapshot isolation guards only the rows (Problem 7.4 is the hunt; Problem 7.8 works the vault in full). The remedies, priced. First, true serializability: modern engines offer a serializable refinement of snapshot isolation that keeps the photographs and the optimism but additionally watches the *pattern* of reads and writes among concurrent transactions — who read what another then wrote — and aborts whenever the pattern could close into the cycle write skew requires. Under such an engine the doctors do not both walk free: one commits; the other is aborted with an apology, retries against the new roster, sees one doctor remaining, and stays. The premium is paid in false alarms — the detector is conservative — but the invariant survives without a single application change. Second, materialize the conflict: make the contending transactions touch a common row — an on-call token — so the detector can see the fight; a fine trick, provided every future transaction remembers the etiquette. Third, lock the neighbourhood explicitly and surrender the concurrency. What is *not* a remedy is hoping. The axes now stand assembled, and the opening's confusion resolves into a sentence per profession: the replication hall's models govern single operations against real time; the database hall's levels govern transactions against each other; strict serializability is the corner where both summits meet; and the ACID-C — the doctors' rule — is *your* obligation, which every level below the summit protects only partially, in ways you are now equipped to name.

7.6 Part Six — The Wedding of the Kettles

A change of key: from promises about *ordering* to a discipline that makes ordering *irrelevant*. Chapter 6's honest mode kept concurrent rivals as siblings and handed the merge to the application — union, for carts. Chapter 1, further back, left a glinting observation in a drawer: the replicated *counter*, alone among the exhibits, forgave the Post's reordering entirely, *because addition commutes*. The drawer now opens — the first instalment on the season's oldest debt — and the contents assemble into a theory. The question: for which

objects can replicas accept updates independently — no coordination, no leader, merely-elsewhere writes welcomed — and still be *guaranteed* to converge to one agreed state? The answer arrived in mature form in 2011, from Shapiro, Preguiça, Baquero, and Zawirski: *conflict-free replicated data types*, with a theorem attached. Design the object so that its states form a join-semilattice — the *join* is the least upper bound $\sqcup$, and *upward* the inflationary updates — make every update move a replica upward in that family, and exchange states arbitrarily, merging on receipt; then convergence is not hoped but *proven*. The guarantee's name is *strong eventual consistency*, and mark the upgrade from the floodplain: not "they will eventually reconcile somehow," but "reconciliation is a pure function with algebraic manners — same news, same state, no negotiation, no siblings, no application on call at midnight."

Definition 7.13 (join-semilattice; state-based CRDT)**.** $(S, \sqcup, \bot)$ is a *join-semilattice* with bottom: $\sqcup$ is commutative, associative, idempotent, with $\bot \sqcup s = s$; the induced order is $s \leq t$ iff $s \sqcup t = t$, and $s \sqcup t$ is the least upper bound. A *state-based CRDT with join-decomposable updates* assigns to every update u a *contribution* $c(u) \in S$, fixed at the originating replica when u executes: the origin replaces its state s by $s \sqcup c(u)$; anti-entropy ships states and the receiver replaces s' by $s' \sqcup s$. (All of this chapter's objects are of this form; general inflation-based objects are treated in the reading.) A replica *has absorbed* u if u's contribution has reached it through any chain of updates and merges.

The theorem beneath the ceremony. Its skeleton: every replica's state is the join of the contributions it has absorbed; joins forget order and multiplicity — commutative, associative, idempotent; same news, same state.

Theorem 7.7 (strong eventual consistency; Shapiro–Preguiça–Baquero–Zawirski)**.** *For a state-based CRDT with join-decomposable updates: at every point of every execution, every replica's state equals the join of the contributions of exactly the updates it has absorbed,*

$$s_i \;=\; \bot \sqcup \bigsqcup \{\, c(u) : u \in D_i \,\},$$

where D_i is replica i's absorbed set. Consequently two replicas that have absorbed the same updates — in any order, along any routes, with any duplication — hold identical states; and under the model box's eventual delivery, all correct replicas converge once updates cease. Convergence is a pure function of the news, not of its itinerary.

Proof of Theorem 7.7. By induction over the events of an execution, we prove the displayed identity at every replica, with D_i the absorbed set defined inductively alongside: initially $s_i = \bot$, $D_i = \emptyset$, and the identity holds vacuously.

Local update u at replica i: the new state is $s_i \sqcup c(u)$; the new absorbed set is $D_i \cup \{u\}$. By the induction hypothesis and associativity–commutativity, the new state is the join of $\{c(v) : v \in D_i\} \cup \{c(u)\}$; if u was somehow already present (it is not, being fresh, but duplication costs nothing), idempotence absorbs it.

Merge of an incoming state s_j (sent when j's absorbed set was D_j) into replica i: the new state is $s_i \sqcup s_j$; the new absorbed set is $D_i \cup D_j$. By both induction hypotheses,

$$s_i \sqcup s_j = \Big(\bigsqcup_{v \in D_i} c(v)\Big) \sqcup \Big(\bigsqcup_{v \in D_j} c(v)\Big) = \bigsqcup_{v \in D_i \cup D_j} c(v),$$

the last step by commutativity, associativity, and — for the updates in $D_i \cap D_j$, delivered twice — idempotence. This one line is the theorem's engine: *the join of a set of contributions depends on the set, not on the order or multiplicity of joining.*

Equality of states for equal absorbed sets is now immediate, and it is a safety property holding at every instant. For convergence: after updates cease, the model box's anti-entropy delivers every update's contribution (transitively) to every correct replica, so all absorbed sets become equal to the full update set, and all states equal its join. Strong eventual consistency is the conjunction of the two. □

The wonder budget of the hour was spent here, on the inversion the construction performs: every other model in this chapter fought the Post's disorder with coordination — holding mail, blocking reads, electing sequencers. The semilattice surrenders the battle entirely: it re-designs the *object* until disorder is powerless against it, until merge forgets nothing and therefore order was never the point. Chapter 1's counter knew this in embryo; the shopping cart practised it commercially; the theorem names the species.

The family, built smallest to largest, each member repairing its predecessor's embarrassment. *The grow-only counter:* one tally per replica — Bingo counts Bingo's increments, Tuppy Tuppy's — merge takes the entrywise maximum, the value is the sum. Why maximum and not addition? Because merge must be idempotent — the Post duplicates, and merging the same news twice must change nothing; a maximum absorbs repeats, a sum double-counts them. The shape is the vector clock's merge, repurposed from tracking causality to *being the data*. Run the tallies once, so the counter is a machine and not a metaphor: Bingo records four increments of his own — his row reads four, zero, zero; Tuppy records three; Gussie, offline in the storm, records one. Bingo merges Tuppy's row: four, three, zero — value seven. The Post, in character, redelivers Tuppy's same row: maxima of equals — nothing moves; the duplicate is absorbed without a tremor. Gussie rejoins and merges: four, three, one — eight; Bingo merges Gussie's in return: eight. Any order, any repeats, any routes: eight. And the discipline that made it lawful: each replica increments *only its own* tally. The row is a ledger of testimonies, one column per witness, and the maximum is sound precisely because each witness's own count only ever grows; let Bingo update Tuppy's column even once, and the algebra collapses on the spot into Chapter 6's lost-update crime scene. The grow-only counter cannot decrement; the *positive-negative counter* carries two of them and makes removal itself a growth — the down-tally only ever grows.

Protocol — Box 16. Grow-only and positive-negative counters (checked against the 2011 report)

1: **G-counter** at replica i of n: state P, a vector of n tallies, initially all zero
2: **increment:** $P[i] \leftarrow P[i] + 1$ — own entry only
3: **value:** sum of all entries of P
4: **merge**(P'): entrywise maximum of P and P'
5: **PN-counter:** two G-counters (P, N); increment raises $P[i]$; decrement raises $N[i]$
6: **value:** sum of P minus sum of N; **merge:** componentwise

Maximum, not addition: merge must be idempotent because the Post duplicates. Each replica writes only its own entry — the vector is a ledger of testimonies, and the maximum is lawful because each witness's own count only grows. Contribution form for Thm. 7.7: an increment's $c(u)$ is the vector that is zero save the incremented entry at its new value.

Neither counter yet holds *things*; the wedding gown itself is the *observed-remove set* — adds and removes of elements, with adds winning against concurrent removes: the cart's own law. Every add attaches a fresh unique tag to its element; a remove does not subtract the element — it records as removed *precisely the tags it has observed*. "Tags and testimony" is exact: removal is testimony about what was seen, nothing more, and one may only remove what one has observed.

Definition 7.14 (the observed-remove set)**.** State: a pair (A, R) with A a set of (element, tag) pairs and R a set of tags, both grow-only; $\bot = (\emptyset, \emptyset)$; join is componentwise union (a product of two grow-only semilattices). Operations at a replica with state (A, R): $\mathsf{add}(e)$ draws a globally fresh tag t and contributes $(\{(e,t)\}, \emptyset)$; $\mathsf{remove}(e)$ contributes $(\emptyset, \{\, t : (e,t) \in A \,\})$ — testimony against precisely the observed tags; $\mathsf{contains}(e)$ holds iff some $(e,t) \in A$ has $t \notin R$. Box 17 gives the protocol form.

Protocol — Box 17. The observed-remove set (checked against the 2011 report)

1: **state:** (A, R): added (element, tag) pairs; removed tags. Both grow-only
2: **add**(e): draw fresh tag t (replica id, counter); $A \leftarrow A \cup \{(e, t)\}$
3: **remove**(e): $R \leftarrow R \cup \{\, t : (e, t) \in A \,\}$ — testimony: observed tags only
4: **contains**(e): some $(e, t) \in A$ has $t \notin R$
5: **merge**(A', R'): $A \leftarrow A \cup A'$; $R \leftarrow R \cup R'$
6: **re-add after remove:** a fresh add draws a fresh tag, never covered by old testimony

Add-wins by jurisprudence (Prop. 7.8): a remove defeats exactly the adds it observed. The metadata bill is real: tags and testimony accrue; pruning them requires knowing what every replica has absorbed — a global fact, hence coordination (the season's irony; the settlement at this part's close).

Watch the law operate. Bertie, at the club, removes the kettle — his remove testifies against tag one, the only kettle he has seen; concurrently, in the study, a fresh kettle is added wearing tag three. The states merge, in any order, at any replica: tag one is covered by testimony — that kettle is gone; tag three is covered by nothing — that kettle survives. The concurrent add wins, not by timestamp, not by luck, but because the remove could not testify against news it had never received — causality, baked into the algebra as jurisprudence.

Proposition 7.8 (add-wins jurisprudence)**.** *In the observed-remove set, if an* $\mathsf{add}(e)$ *is not absorbed by the replica executing a* $\mathsf{remove}(e)$ *at the time of the remove (the two are concurrent), then after any replica absorbs both, e is present. A remove defeats exactly the adds it observed: one may only remove what one has seen.*

Proof of Proposition 7.8. Let the remove execute at replica j with state (A_j, R_j), and let the concurrent $\mathsf{add}(e)$ have drawn tag t^*. Concurrency means j had not absorbed the add, so $(e, t^*) \notin A_j$, so $t^* \notin$ the remove's testimony — and testimony sets only ever accumulate tags fixed at their origin. After any replica absorbs both updates, its state contains (e, t^*) in A and $t^* \notin R$ (no other operation testifies against t^*: tags are globally fresh, and removes testify only against observed tags). By Def. 7.14, $\mathsf{contains}(e)$ holds. The freshly added kettle survives because the remove could not testify against news it had never received. □

The two kettles are wed: not by choosing between them, but under a law by which every replica, told the same history in any order, reaches the same cart.

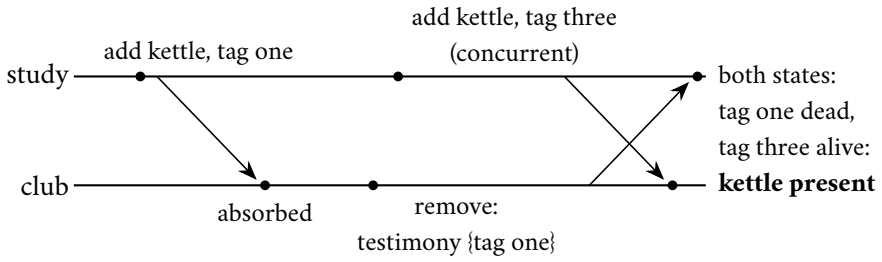


Exhibit 7.4 (the wedding). The club's remove testifies against tag one — the only kettle it has seen; the study's concurrent add wears tag three. Merged in either order, every replica reaches the same cart: tag one covered, tag three alive (Prop. 7.8). Not by timestamp, not by luck: by a law under which removal is testimony about what was observed.

One relation attends the wedding under a cloud, and honesty requires the introduction.

Remark 7.15 (the cautionary relation)*.* The last-writer-wins register — state (value, timestamp), join keeps the larger stamp — satisfies every hypothesis of Thm. 7.7. The theorem guarantees convergence, never justice: the law it converges under is arbitration by the Post's perjured clock, and the discarded rival is discarded silently and forever. Choosing the object is choosing the verdicts; the observed-remove set is celebrated because its law encodes causality, not because it converges — the perjurer converges too.

Two engineering notes complete the settlement, and both are invoices. First, the matter of what travels. Everything above shipped whole *states* — rows of tallies, bags of tags — merged on receipt; that school asks nothing of the Post beyond eventual gossip, and Chapter 6's anti-entropy nets suffice unmodified. The dual school ships *operations* instead — "increment by one," "add tag three" — which travels far lighter but demands far better manners of the Post: operations are not idempotent, so delivery must be exactly-once, and for some objects causally ordered — Chapter 2's machinery, re-hired as a delivery service. The trades' standing compromise ships only the recently changed fragments of state, and takes both advantages at the cost of neither, mostly. Second, the metadata bill, presented without discount: the tags accrue, the testimony accrues, and a set that has lived busily for a year may weigh many times its visible contents. Pruning the archive — discarding testimony that every replica has certifiably absorbed — requires knowing what *every* replica has absorbed, which is a global fact, which requires coordination. There is the season's irony at its purest: the coordination-free object may be *used* forever without agreement, but *slimmed* only with it. Nothing in this field is free; the semilattice merely lets one choose when to pay, and the choosing is the luxury. The costs of the law itself, stated with matching honesty: the law must fit the business — add-wins is a choice, and a banking balance is not a semilattice, however devoutly one wishes it (Problem 7.12); and nothing here provides an agreed *order* — a CRDT cannot run an auction. A large, lawful, coordination-free province with a well-marked border — patrolled, as ever, by the race test.

7.7 Part Seven — The Horizon: CALM

One movement remains, and it is the view from altitude. The season keeps meeting the same fork: some tasks yielded to cleverness without coordination — the photograph, the register, the cart — while others — the bit, the lock, the auction — demanded a parliament every time. Is there a *law* sorting the forks? There is a candidate, and it may be the deepest single sentence the field has produced this century. First the logician's sense of the load-bearing word.

Definition 7.16 (monotone program)**.** A program computing an output set from input sets is *monotone* iff larger inputs never shrink the output: $I \subseteq I'$ implies $P(I) \subseteq P(I')$ — conclusions, once emitted, are never retracted by further news. Union, selection, join, transitive closure: monotone. Negation, universal quantification over a live set, "the final state," "the winner": non-monotone.

Theorem 7.9 (CALM; Hellerstein–Alvaro; proved by Ameloot–Neven–Van den Bussche)**.** *A distributed program has a coordination-free, eventually consistent implementation if and only if it computes a monotone function of its inputs.* (Stated with respect and a pointer: the proof lives in the relational-transducer semantics of the reading, honestly beyond this course's syllabus.)

Non-monotonicity is precisely where coordination becomes not an engineering preference but a mathematical obligation. Anything that must assert a completed totality over inputs still possibly in flight — "the auction is closed, no further bids," "the balance never dipped below zero," "this is the *final* state" — must somewhere *seal the set*; and sealing a set across machines is agreement, which is Chapter 3's terrain with all its invoices. The practical edition fits on the whiteboard and changes design reviews: *find the negations*. Every place a specification says "final," "exactly," "never," "the winner," is a place coordination will be spent;

everything phrased as accumulation can, in principle, flow free. Read your own designs for their quantifiers, and the invoices become legible before the architecture is drawn.

The classification, practised once on the season's own case files. A watchlist assembled by union from many stations: monotone — more reports, more entries, nothing retracted; free to flow. "This parcel's ancestry has arrived": monotone — ancestry only accumulates, and the answer, once yes, stays yes forever; which is exactly why causal consistency needed no parliament. "This replica's copy is the latest": non-monotone at its heart — "latest" is a totality claim over writes still possibly in flight, and there stands the round trip that the star exhibit's freshness-check variation priced. And "the winner of the auction is Bingo, at seven shillings": non-monotone by construction — one further bid retracts the conclusion — and there stands Chapter 3's bit, wearing an auctioneer's coat. Notice, finally, the jurisprudential trick the observed-remove set performed: removal *sounds* like negation — and naive removal is — but recording removals as accumulating testimony against observed tags re-drafts the negation as a growth, and the theorem's licence extends exactly as far as such re-draftings can be found (Problem 7.11 sorts a page of fragments). Sometimes it cannot — finality is finality — and then the invoice is a theorem's invoice, and one pays it with dignity.

7.8 The Whiteboard

For the design review — four items, the course's densest.

1. Name the contract, not the vibe. "Consistent" is four professions; in a review, demand the government name — linearizable, sequential, causal, read-your-writes, eventual-plus-what — and for transactions the isolation level by its admitted anomalies: does this level admit write skew? The Jepsen atlas on the wall settles strength disputes in ten seconds; and when the review stalls on the word itself, hand each profession its sentence from the opening.

2. Linearizability speaks of real time; serializability of transactions; strict serializability of both. Neither of the first two implies the other; the side-channel telephone is the test for the first; and the summit's fee is paid in coordination — buy it where clients compare notes, decline it where they cannot, and remember the field protocol's three questions: who compares notes; what must survive partition; what arbitrates races.

3. Snapshot isolation permits write skew, and your invariants that span rows are unguarded by it. Find the constraint that lives across items; then serialize truly, materialize the conflict, or own the anomaly in writing. The two doctors are on every schema; the drill is finding them before the roster does.

4. If your merge is a join, you may sleep: semilattice objects converge without coordination, by theorem — spend the design effort making the merge lawful rather than the delivery ordered, and audit the *law* as carefully as the lattice, for the perjurer's register is also a lattice. And where the specification negates — final, exactly, never, winner — budget a parliament, by theorem likewise. Between those two theorems runs the entire economics of this field.

The register. Debts paid: #8 (causal delivery, contracted and priced); #11 and #13 (CAP's C and the single-machine phrase, retired jointly by Def. 7.4); #16 (the lease-read fine print); #17 (the kettles wed, Thm. 7.7); and a first instalment on #4 — *commutativity forgives disorder*, first half paid by semilattice, the balance due in the finale, wearing gradients. Issued: #18 — serializability *across shards*: the two doctors were one database; make them two and isolation becomes distribution (Chapter 8). #19 — the Jepsen method industrialized: fault injection as a discipline (Chapter 11). The half-term audit, in summary: ten entries and two instalments paid across the first half; five standing open, all scheduled, crashed-versus-slow chief among them; the books balance, and the second half is financed. Drift note: the plan's Tier II problem "causal neither implies

nor is implied by sequential" is false as stated under the standard definitions (sequential *does* imply causal); the desk sets the corrected problem and recovers the intended incomparability with the real-time-causal variant (Problem 7.7). Episode word count 10,428 — above the floor; the §5 corrective (per-part budgets) was applied for the first time this session.

7.9 Problems

Tier I — Calisthenics

Problem 7.1 (linearizability drills)**.** One register, initial value zero. For each history, give a linearization (as an order with points) or prove none exists. (a) Bingo writes one over $[1, 4]$; Tuppy reads over $[2, 3]$ returning zero; Gussie reads over $[5, 6]$ returning one. (b) Bingo writes one over $[1, 4]$; Tuppy reads over $[2, 6]$ returning one; Gussie reads over $[3, 5]$ returning zero. (c) Bingo writes one over $[1, 3]$; Tuppy writes two over $[2, 4]$; Gussie reads over $[5, 6]$ returning one; the aunt reads over $[7, 8]$ returning two. (d) Bertie writes ten over $[1, 2]$; the aunt reads over $[4, 5]$ returning five (the star, Exhibit 7.1).

Problem 7.2 (the queue drill)**.** A queue, initially empty. (a) Bingo enqueues an apple over $[1, 4]$; Tuppy enqueues a pear over $[2, 5]$; Gussie dequeues over $[6, 7]$ receiving the pear. Linearizable? (b) Same enqueues; Gussie dequeues twice, over $[6, 7]$ and $[8, 9]$, receiving the pear both times. (c) Same enqueues; Gussie dequeues over $[6, 7]$ receiving the pear, and the aunt dequeues over $[8, 9]$ receiving the apple. State, for each verdict, whether the fault (if any) lies against time or against the sequential specification.

Problem 7.3 (name the bruise)**.** Name the violated session guarantee, or declare all four intact. (a) Bertie posts a notice, refreshes, and the notice is absent. (b) The aunt sees the new price on her device, hands the same device (same session) to the uncle, and the price has reverted. (c) Bertie posts a notice and then a correction; a distant replica applies the correction first and the notice second, ending with the notice text. (d) Bertie reads Tuppy's question and posts an answer; at another replica the answer is visible while the question is not, to a fresh client with no history.

Problem 7.4 (the skew hunt)**.** For each schema, exhibit the write-skew shape (the cross-row invariant, the two transactions, the disjoint writes) and propose the smallest materialized conflict that closes it. (a) Usernames: a claim is lawful if no row holds the name; claims insert fresh rows. (b) A linked pair of accounts must keep a combined non-negative balance; withdrawals debit one account each. (c) The vault requires two keyholders signed in overnight; sign-outs update each officer's own row.

Problem 7.5 (name the contract)**.** Name the strongest model each description honestly supports. (a) All writes through one agreed log; reads served by any replica from its local copy, no freshness check. (b) As (a), plus: each client's reads are pinned to one replica, and a session token holds the log index of the client's last write, which the replica must have applied before answering. (c) As (a), plus: every read is relayed through the log itself. (d) A leaderless store, sloppy quorums, last-writer-wins; the brochure says "strongly eventually consistent."

Tier II — The Real Thing

Problem 7.6 (locality's negative half)**.** Two queues p and q, initially empty. Bingo's program: enqueue x on p; enqueue x on q; dequeue from q, receiving y. Tuppy's program: enqueue y on q; enqueue y on p; dequeue from p, receiving x. All six operations complete; overlaps are such that no real-time constraints bind across clients. (a) Show $H|p$ and $H|q$ are each sequentially consistent (exhibit the per-object stories). (b) Show H is not: no single story respects both program orders and both queues' specifications. (c) State in one sentence why Theorem 7.3's proof strategy has no purchase here.

Problem 7.7 (causal versus sequential, corrected)**.** (a) Prove: every sequentially consistent history is causally consistent (fill any gaps in Theorem 7.5's argument in your own hand). (b) Prove Exhibit 7.2 causally consistent and not sequentially consistent, in full detail: construct the two views; then show no single legal total order extends both readers' requirements. (c) Define real-time causal consistency (Rem. 7.10). Prove Exhibit 7.1 sequentially consistent but not real-time causal. (d) Prove Exhibit 7.2 real-time causal (arrange the intervals so the writes overlap) but not sequentially consistent. Conclude: the folklore incomparability is true of the real-time variant, and half true of the plain one.

Problem 7.8 (the vault keyholders)**.** Formalize Problem 7.4(c): two officers' rows, both `in`; guard: sign out only if the photograph shows two `in`. (a) Exhibit the snapshot-isolation history in which both sign out; verify each clause of Def. 7.12. (b) Prove no serial execution of the guarded programs reaches the empty vault. (c) Add the materialized conflict (a duty token row that every sign-out must update) and show first committer wins now aborts one officer. (d) Under the serializable refinement (abort when concurrent transactions form a read-write cycle), trace which of the two aborts and why the retry stays in.

Problem 7.9 (both axes at once)**.** (a) Exhibit a history of two transactions that is serializable but not strictly serializable (the backup-restore defence made formal): a committed deposit, then a later, non-overlapping read-only transaction returning the pre-deposit balance. Prove both halves of the claim. (b) Exhibit a history over two registers in which every single operation is linearizable per object, yet the transactional bracketing is not serializable (each of two transactions reads one register before the other writes it, crosswise). Conclude, in one sentence each way: neither axis implies the other.

Problem 7.10 (sabotage: the vanished kettle)**.** A junior engineer "optimizes" Box 17: removals record the *element value* instead of observed tags, and `contains`(e) holds iff some $(e, t) \in A$ and e is not in the removed-values set; merge unions both components. The states still form a semilattice, so the engineer cites Theorem 7.7 and ships it. Exhibit the fatal execution: a concurrent add whose kettle vanishes, permanently — including the exact states, tags, and merges, and the sentence of the specification it violates. Then state what the engineer's citation actually proves, and why that is not a defence.

Tier III — For the Ambitious (starred)

Problem 7.11 ($\star$ the CALM classification)**.** Classify each as monotone (coordination-free by Thm. 7.9) or non-monotone (name the seal): (a) the union of watchlists from many stations; (b) "this parcel's ancestry has arrived" (causal delivery's guard); (c) "this replica holds the latest value"; (d) the auction's winning bid; (e) the observed-remove set's `remove`; (f) distributed garbage collection ("no live reference anywhere"); (g) a join of two grow-only tables; (h) "the report is final: exactly forty visitors attended." *(Hints only.)*

Problem 7.12 ($\star$ the overdraft)**.** A bank demands a replicated counter that supports coordination-free increments and decrements at any replica *and* never exhibits a negative value at any replica. (a) Show the PN-counter fails the requirement. (b) Argue from Thm. 7.9 that no CRDT can meet it: locate the non-monotone clause. (c) Design the standing compromise — escrow: split the balance into per-replica allowances, spend locally against your allowance, coordinate only to rebalance — and state exactly which operations remain coordination-free. *(Hints only.)*

7.10 Hints

Hint (Problem 7.6)**.** (b) Whoever's dequeue is served first in the story must already have both enqueues of the received item's queue ordered suitably; chase the two program orders and derive that each dequeue must

precede the other. (c) Locality assembles per-object stories using shared real time; sequential consistency has surrendered exactly that shared resource.

Hint (Problem 7.7)**.** (b) From the aunt: one-write before two-write (else her first read is illegal); from the uncle: the reverse. (c) In Exhibit 7.1 the write $\prec$ read edge enters $\rightarrow_{\mathrm{rt}}$, and the aunt's read returns a value whose write is $\rightarrow_{\mathrm{rt}}$-overwritten. (d) With overlapping writes, $\prec$ adds nothing; the plain-causal argument stands.

Hint (Problem 7.9)**.** (b) The crosswise reads force each transaction before the other in any serial order — a cycle; yet every individual read and write linearizes happily per register.

Hint (Problem 7.11)**.** Monotone: (a), (b), (g). The rest each assert a completed totality over inputs possibly in flight — find the phrase "no further" hiding in each. For (e): testimony accumulates (monotone as implemented), yet the *user-facing* membership predicate is non-monotone — the construction's whole cleverness is that re-drafting.

Hint (Problem 7.12)**.** (b) "Never negative" quantifies over all interleavings of decrements still in flight: admitting one more decrement can retract the lawfulness of another — the set of spendable funds must be sealed. (c) Increments and within-allowance decrements stay free; only rebalancing coordinates.

7.11 Solutions

Tier I in full (tersely); Tier II in full — Problem 7.10 is the sabotage certification; hints only for Tier III.

Solution (to Problem 7.1)**.** (a) Points: read at 2.5, write at 3, read at 5.5: zero then one; legal, extends $\prec$. Linearizable. (b) Points: write at 3.4, Tuppy's read at 3.6 (one), Gussie's read at 3.2 (zero): all inside their intervals, story zero–write–one lawful in the order read-zero, write, read-one. Linearizable (order the overlapping reads around the write's point). (c) The writes overlap: order two before one (points: two at 2.5, one at 2.8); then Gussie lawfully reads one at 5.5 — but the aunt, *after* Gussie ($\prec$), reads two: her read's point must follow Gussie's, and the latest write before both is one. Contradiction; and the symmetric ordering fails Gussie. Not linearizable (the flicker, in transaction-free clothing). (d) Write $\prec$ read; any points put the write first; a legal register story then answers ten, not five. Not linearizable — and no machinery was mentioned in the verdict.

Solution (to Problem 7.2)**.** (a) Yes: order pear-enqueue before apple-enqueue (they overlap); dequeue lawfully yields the pear. (b) No: both orderings of the enqueues are available, but no sequential queue yields the same item twice — the fault is against the specification, whatever the times. (c) Yes: pear first, apple second, dequeues in $\prec$ order return pear then apple — first in, first out honoured. Verdicts (a) and (c) exercise the definition's mercy on overlaps; (b) is the story-legality clause alone.

Solution (to Problem 7.3)**.** (a) Read your writes. (b) Monotonic reads (one session, view moved backwards). (c) Monotonic writes (the correction must be ordered after the notice everywhere). (d) All four intact: the fresh client has no session; the anomaly is a *causal* violation (writes-follow-reads binds Bertie's session, not third parties — the cross-client guarantee is Def. 7.7, and this is precisely the gap between the session tier and the causal landing).

Solution (to Problem 7.4)**.** (a) Invariant: at most one row per name — it spans the *absence* of rows; two claimants each read no-row (lawful photographs) and insert distinct fresh rows: disjoint writes, both commit, two rows. Materialize: a per-name reservation row that every claim must update. (b) Invariant spans

two balances; each withdrawal reads both, debits its own. Materialize: a pair-total row updated by every withdrawal. (c) The vault: Problem 7.8. Materialize: the duty token. In each case the materialized row forces write-set intersection, so Def. 7.12's commit rule sees the fight.

Solution (to Problem 7.5)**.** (a) Sequential consistency (one story via the log; program order per client if clients are pinned or the log timestamps their submissions; real time surrendered at the local reads). (b) Sequential consistency plus read-your-writes and monotonic reads for the pinned session — the session tier atop (a). (c) Linearizability: every read's point is its passage through the log. (d) Eventual at best from the brochure — and last-writer-wins makes even convergence a convergence *onto data loss* (Rem. 7.15); "strongly" is doing no work in that sentence unless the merge is a lawful join, which a perjured-clock arbitration technically is — whereupon the honest label is: strong eventual convergence under an unjust law.

Solution (to Problem 7.6)**.** (a) $H|p$: story "Bingo enqueues x; Tuppy enqueues y; Tuppy dequeues x" — respects each client's order on p, and FIFO yields x (enqueued first). $H|q$: symmetrically "Tuppy enqueues y; Bingo enqueues x; Bingo dequeues y." Each object alone: sequentially consistent. (b) Suppose one story S for both. In S, Bingo's dequeue of y from q requires y's enqueue on q before it with x's enqueue on q not before y's (FIFO: the head must be y); since Bingo's program order puts his q-enqueue of x before his dequeue, S must order Tuppy's y-enqueue on q *before* Bingo's x-enqueue on q. By Tuppy's program order, his y-enqueue on q precedes his y-enqueue on p, which precedes his dequeue of x from p; that dequeue requires (FIFO on p) Bingo's x-enqueue on p before Tuppy's y-enqueue on p — and by Bingo's program order, Bingo's p-enqueue precedes his q-enqueue. Chain the requirements: Bingo's q-enqueue after Tuppy's q-enqueue, which is after … Bingo's p-enqueue, which is before Bingo's q-enqueue — consistent so far; the contradiction closes on the dequeues: FIFO on q forces y dequeued while x waits, so Bingo's dequeue precedes any dequeue of x; FIFO on p symmetrically forces Tuppy's dequeue of x to find x at the head, requiring Bingo's dequeue of y *not yet* to have consumed … chase it fully and each dequeue must strictly precede the other in S. No such total order exists. (c) Locality's assembly pools per-object points on the shared real-time line; sequential consistency's per-object stories run on private timelines with no common line to pool them on.

Solution (to Problem 7.7)**.** (a) As Theorem 7.5: the single story orders program order by definition and reads-from by register legality; transitive closure follows; restriction to (client, all writes) preserves legality and order. (b) Views: aunt's S_a: write one; read one; write two; read two — legal, extends $\rightarrow$ (the writes are $\rightarrow$-unrelated, so either order is available to her). Uncle's S_u: write two; read two; write one; read one — likewise. Causally consistent. Not sequentially consistent: a single legal story must order write-one and write-two once. If one is first, the uncle's first read of two requires two after one *before* that read, and his second read of one requires one to be the *latest* write before it — but one precedes two in the story, so after two's appearance, one is never latest again; his second read is illegal. Symmetrically if two is first, the aunt's pair fails. No story serves both. (c) Real-time causal: replace $\rightarrow$ by $\rightarrow_{\mathrm{rt}}$ in Def. 7.7. In Exhibit 7.1, write-ten $\prec$ read, so write-ten $\rightarrow_{\mathrm{rt}}$ read; any legal view of the aunt must place her read after write-ten with no later write of five — five's write (the initial establishment) precedes write-ten in every view extending $\rightarrow_{\mathrm{rt}}$; the read returning five is illegal in all of them. Not real-time causal; yet sequentially consistent (Exhibit 7.1's caption story, which reorders across clients — permitted, since program order alone is preserved). (d) Stage Exhibit 7.2 with the two writes overlapping in real time: $\prec$ then relates neither write to the other nor (arranging the reads after both responses) constrains the reads' sources beyond plain causality; the views of (b) remain lawful, so the history is real-time causal; the proof in (b) that no single story exists is untouched. Hence: sequential and real-time causal are incomparable (7.1 separates one way, 7.2 the other); plain causal is strictly weaker than sequential (a and b). The folklore, corrected, is now equipment.

Solution (to Problem 7.8)**.** (a) Rows $a, b = \mathsf{in}$; $s(T_1) = s(T_2) = \sigma$; both photographs show (in, in); guards pass; T_1 writes $a := \mathsf{out}$, T_2 writes $b := \mathsf{out}$; $c(T_1) < c(T_2)$; write sets $\{a\}$, $\{b\}$ disjoint, so the commit rule passes both. Final: vault unattended. (b) In either serial order the second program's photograph shows one in; its guard fails; it writes nothing; every serial execution ends with at least one officer in — induction as in Prop. 7.6. (c) With the duty token in both write sets, $s(T_2) < c(T_1) < c(T_2)$ and the sets intersect on the token: T_2 aborts by first committer wins; the retry's photograph shows one in and the guard holds it inside. (d) The serializable refinement tracks the crosswise read-write dependencies (T_1 read b which T_2 wrote; T_2 read a which T_1 wrote): a cycle of two; the engine aborts the later committer, T_2; the retry reads $a = \mathsf{out}$, guard fails, stays — invariant preserved with no schema change, the premium paid in the abort.

Solution (to Problem 7.9)**.** (a) History: T_1 deposits (read balance one hundred, write one hundred fifty, commit); later, disjointly, T_2 reads and returns one hundred. Serializable: the order T_2 then T_1 explains every read from the initial state. Not strictly serializable: $T_1 \prec T_2$ in real time, and in the order T_1, T_2 the read must return one hundred fifty. Both halves proved; the defence "it is serializable" is sound and the complaint "it ignored the clock" is also sound — they are different contracts. (b) Registers x, y, both zero. T_1: read y (returns zero), then write $x := 1$. T_2: read x (returns zero), then write $y := 1$. Interleave the four operations without overlap in the order: T_1 reads y; T_2 reads x; T_1 writes x; T_2 writes y. Each individual operation linearizes trivially (single-register histories with no overlaps are their own linearizations — per-object legality throughout). Serializability: T_1 before T_2 requires T_2's read of x to return one; T_2 before T_1 symmetrically; neither serial order matches. Hence: per-object linearizability does not compose into transactional serializability (brackets are a different axis), and serializability does not imply real-time respect — neither axis implies the other.

Solution (to Problem 7.10 — the certification)**.** The fatal execution, tags and all. Two replicas, study and club; empty initial state.

(1) Study executes add(kettle): draws tag one; state $A = \{(\text{kettle}, 1)\}$, removed-values $V = \emptyset$.

(2) Study's state gossips to the club; club absorbs it.

(3) Club executes remove(kettle): under the "optimization" it records the *value*: $V = \{\text{kettle}\}$.

(4) Concurrently — before any further gossip — study executes add(kettle): fresh tag three; study's $A = \{(\text{kettle}, 1), (\text{kettle}, 3)\}$.

(5) The states merge (either direction; both, for completeness): $A = \{(\text{kettle}, 1), (\text{kettle}, 3)\}$, $V = \{\text{kettle}\}$. contains(kettle) tests membership of the *value* in V: **false**. The kettle of tag three — added concurrently with the remove, never observed by it — has vanished. Every replica agrees it has vanished (the semilattice converges faithfully), and no future gossip resurrects it; worse, *no future add ever succeeds*: any later add(kettle) draws a fresh tag, but the value "kettle" remains in the grow-only V forever — the customer can never buy a kettle again.

The violated sentence is Prop. 7.8 — the advertised add-wins specification: an add concurrent with a remove must survive their joint absorption. Step (5) exhibits the breach; the permanence claim follows from V's grow-only nature. What the engineer's citation proves: convergence — Theorem 7.7 holds, all replicas agree, forever, on the *wrong cart*. Convergence is a property of the algebra; correctness is a property of the law; the optimization changed the law (from testimony-against-observed-tags to condemnation-by-name) while keeping the algebra, and the theorem is loyal to the algebra. Certified fatal: acknowledged customer data (the tag-three kettle) is silently and permanently destroyed under an execution requiring no failures at all — one concurrent add and remove, the store's daily weather.

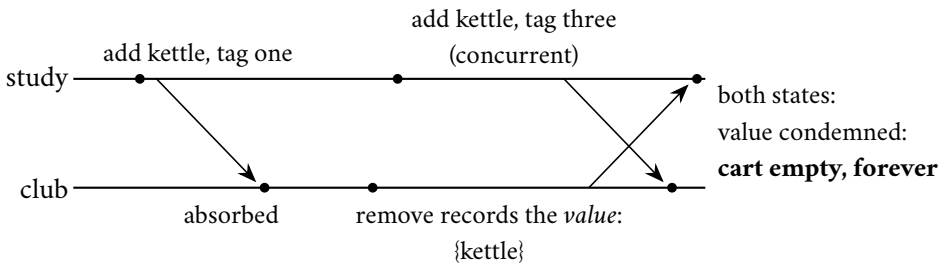


Exhibit 7.5 (the vanished kettle). The same staging as Exhibit 7.4 under the sabotaged rule: the remove condemns the *name*, and testimony it never gave defeats the concurrent add of tag three — permanently, since the condemned-values set only grows. Compare the two exhibits line by line: the jurisprudence is the entire difference.

Solution (to Problem 7.11)**.** Hints stand (Tier III). The classification to argue toward: monotone — (a), (b), (g); non-monotone — (c), (d), (f), (h), each concealing a "no further news" seal; (e) is the instructive straddle: the implementation (testimony accumulates) is monotone, the user-facing predicate (present: some tag not yet condemned) is not — CALM's licence extends exactly as far as such re-draftings.

Solution (to Problem 7.12)**.** Hints stand (Tier III). The shape to reach: (a) two replicas each decrement the last unit against stale photographs; merge; value negative. (b) "never negative" asserts a totality over in-flight decrements — one more input retracts another's lawfulness; the spendable set must be sealed, and sealing is agreement. (c) escrow: allowances are per-replica grow-only ledgers; local spends against local allowance are monotone; only rebalancing (moving allowance between replicas) coordinates — the invoice confined to the rare operation, the season's recurring settlement.

7.12 The Reading

Herlihy and Wing, "Linearizability: A Correctness Condition for Concurrent Objects," TOPLAS 1990. The summit in the founders' prose: the definition, locality, nonblocking, and the queue examples this chapter drills. Read §2–§3; the proof methodology of §4 rewards a second visit.

Lamport, "How to Make a Multiprocessor Computer That Correctly Executes Multiprocess Programs," IEEE Transactions on Computers, 1979. Two pages; sequential consistency named and priced. The hardware trades have spent four decades relaxing it.

Terry, Demers, Petersen, Spreitzer, Theimer, and Welch, "Session Guarantees for Weakly Consistent Replicated Data," PDIS 1994. Short, practical; the four definitions you will use in a meeting within the month, and Box 15's source.

Berenson, Bernstein, Gray, Melton, and the O'Neils, "A Critique of ANSI SQL Isolation Levels," SIGMOD 1995. Write skew's formal residence; the paper that showed the industry's isolation taxonomy was folklore.

Shapiro, Preguiça, Baquero, and Zawirski, "Conflict-Free Replicated Data Types" (the comprehensive 2011 technical report and the SSS conference paper). The catalogue reads like a seed catalogue; linger on the garbage-collection sections, where the honest costs live (Box 17's closing note).

Hellerstein and Alvaro, "Keeping CALM: When Distributed Consistency Is Easy," CACM 2020. The horizon in its authors' voice; the proof lives in Ameloot–Neven–Van den Bussche's transducer papers, cited therein.

Kingsbury: the Jepsen consistency map, with any one full analysis of your acquaintance. Assigned as standing equipment: the atlas beside the whiteboard for the rest of the season, and one autopsy read to watch this chapter's definitions arrest a production database in the act.

The shelf. Ahamad, Neiger, Burns, Kohli, and Hutto, "Causal Memory" (Distributed Computing, 1995) — the causal landing formalized, the hierarchy's middle floor in full. Bailis, Venkataraman, Franklin, Hellerstein, and Stoica, "Probabilistically Bounded Staleness for Practical Partial Quorums" (VLDB 2012) — the floodplain's actuarial tables: eventual, quantified. Gibbons and Korach, "Testing Shared Memories" (SIAM Journal on Computing, 1997) — Rem. 7.3's hardness, for the ambitious.

Chapter 8

Transactions at a Distance

Companion to Episode Eight. Atomic commitment specified beside consensus, the veto clause doing the separating; Gray's two-phase ceremony with its full recovery matrix, wake-state by wake-state; the blocking theorem — proved by two worlds, the doppelgänger's largest conviction of the season; three-phase commit under museum glass; and the modern settlements, one per verb: the fragile roles relocated onto parliaments, Spanner's purchased physics with commit-wait made a theorem one can carry, Percolator's coordinator dissolved into the data, Calvin's abolished conversation, and the sagas' honest abandonment of isolation, its anomaly drawn as a trace. The officiant faints in the opening bar; the couple's paralysis is proved lawful before the close.

8.1 Part One — Atomic Commitment, Specified

The opening image is exact, and the chapter keeps it as equipment. A thorough officiant collects a binding vow from each party before pronouncing anything; by this canon the vow is irrevocable — having said *I do*, a party may not reconsider, whatever follows. Both vows collected, the officiant draws breath to pronounce — and faints on the chancel steps, before a syllable escapes him. Are the couple married? Perhaps the pronouncement was inscribed in the parish register, which lies beneath his unconscious form where nobody may read it; perhaps nothing was inscribed. Canon law offers no third state, and neither party may act: walking away risks bigamy if the register says married; setting up house builds on nothing if it says nothing. They stand at the altar indefinitely, because only the register knows the truth and only the officiant may read it. The canon sounds contrived until one prices the alternative rules. Let the vow be revocable, and the pronouncement becomes worthless: a marriage may be declared whose bride has left. Let the couple act on silence, and two guests who observed different moments of the ceremony will testify to different marriages, which is rather worse than no wedding at all. Every clause is load-bearing; change any one and you have not solved the problem, merely changed which party is ruined. That is two-phase commit, and the image is not a loose analogy but an exact one: the vows are prepare votes, irrevocable by protocol; the register is the coordinator's log; the faint is a crash; and the paralysis is the blocking theorem, proved below by our old friend the doppelgänger.

The stakes are not antiquarian; no corner of the field has seen fashion swing so violently. The classical era built empires on the ceremony — every serious database of the eighties and nineties shipped it, standards bodies blessed it, application servers made the officiant a job title. The web era, staring at the latency and the funerals, swore off distributed transactions altogether — "we simply do not do that here" was, for a decade, a respectable architecture, and Chapter 6's shopping cart was its masterpiece. Then the pendulum, having visited both walls, returned with engineering: the planet-scale ledgers of the last decade concluded that the business world had been right to want the transaction and the web world right about the price, and set

about paying the price properly — with parliaments, with physics, with determinism, or with signed and budgeted apologies. The only question worth asking an architect camped at any station of that pendulum: which theorem are you paying, and in what currency?

The setting, precisely. One transaction has done its work across several *participants* — it has debited a row at Tuppy's branch, credited a row at Gussie's — and the work is staged, provisional, locks held per Chapter 7's machinery. Somebody must now decide: does all of it become permanent, or none of it? This is the atomic commitment problem, and the models under which the chapter's every claim is made are fixed first, per the season's standing habit.

Model of this chapter

For commitment (Defs. 8.1–8.5, Thms. 8.1–8.2). Asynchronous message passing; fair-loss links with retransmission beneath (Chapter 1); crash-recovery processes with stable storage: a process loses volatile state on crash, retains its log, and may recover after unbounded delay — or never. No failure detector; silence and death indistinguishable (Chapter 3). One designated coordinator, n participants; the transaction's provisional work and locks are in place before the protocol begins (isolation is Chapter 7's department).

For commit-wait (Def. 8.6, Thm. 8.4). An absolute time t_{abs} exists (the analysis is external; no process reads it). Every process may call TT.now(), receiving an interval $[e, l]$; the *TrueTime axiom*: at the instant of the call, $e \le t_{\text{abs}} \le l$. Intervals from different calls and processes need share nothing but the axiom.

For sagas (Def. 8.8, Prop. 8.5). Each step and each compensation is a local transaction, atomic and durable at its own service; no lock, snapshot, or isolation mechanism spans steps; concurrent transactions read committed states freely between steps.

Definition 8.1 (atomic commitment)**.** Each of n participants votes YES or NO; each participant and the coordinator may *decide* COMMIT or ABORT. Required: **AC1 (agreement)** no two processes decide differently; **AC2 (validity)** if any participant votes NO, no process decides COMMIT; if all vote YES and no failure occurs, all decide COMMIT; **AC3 (stability)** a decision is never changed; **AC4 (termination)** in every execution in which all failures are eventually repaired, every participant eventually decides (*weak*); a protocol is *non-blocking* if every correct participant decides even while other processes remain crashed (*strong*).

Place the specification beside Chapter 3's consensus, because the family resemblance deceives everybody once. Consensus validity is generous: any proposed value will do. AC2's first half is unanimity in one direction — commit requires every single yes — and hear what that grants each participant: a *veto*. The shyest teller in the branch, the smallest shard in the estate, can refuse the transaction and the refusal is law. Consensus is a parliament: a majority carries the day and a silent minority is simply outvoted. Atomic commitment is a wedding: nobody is outvoted into matrimony, and a silent party who has already said *I do* cannot be left behind, because the vow might already be part of a completed marriage nobody present can see. The veto converts silence from an inconvenience into a paralysis, and the rest of the chapter is the working-out of that sentence.

Remark 8.2 (the veto; commitment versus consensus)*.* Consensus validity permits any proposed value; AC2's first half grants every participant a veto, and its second half forbids gratuitous aborts in failure-free runs. Reductions run both ways in the crash-stop asynchronous model (votes, then consensus on the conjunction, gives commitment with consensus-grade liveness — the route of Boxes 18's parliaments; commitment instances decide consensus-like questions in return), so FLP shadows both. What the veto changes is

failure economics: a consensus majority outvotes the silent; a commit ceremony cannot — the silent voter may hold a NO, or evidence of a sealed COMMIT, and Thm. 8.2 turns that asymmetry into a theorem.

A word on the trade's proper nouns. In the classical architecture the participants are *resource managers* — databases, queues, ledgers, each sovereign over its own durable state — and the coordinator a *transaction manager*, a role the industry standardized in the late eighties so that one vendor's officiant could marry another vendor's parties; the interface survives in every application server to this day, and so do its failure modes, which is one reason the blocking theorem has such a distinguished operational history. On the vote's fine print: a yes is not an opinion but a *certificate of staged durability* — I have written everything to stable storage such that, on command, I can commit through any crash. The certificate is expensive precisely because it must survive the participant's own death and resurrection; a participant that answers yes from memory alone has counterfeited it, and counterfeits are discovered, as counterfeits are, at the worst possible audit. Note too the two grades AC4 encodes: weak termination — if no failures occur, everyone decides — is cheap, and two-phase commit delivers it handsomely; what one wants is the strong, non-blocking grade, every correct participant deciding even while others lie dead. Nothing forbids deciding abort generously — a coordinator may abort on any sneeze, and availability-minded engineers are perennially tempted to buy liveness with a hair-trigger — but the temptation has a limit, and the limit is exactly the vow: aborts are free only before the yes votes are cast; after them, generosity becomes forgery. The protocol's entire character changes at the instant the first vote is surrendered.

The founding text is Gray's "Notes on Data Base Operating Systems" (1978) — plain words, total authority, the tone of a man describing plumbing he has personally soldered. The two-phase ceremony appears there fully formed, along with the observation that would haunt it: the protocol has a window in which a participant's fate lies in another machine's private log. Gray called the trapped state *in-doubt*; the industry came to call it limbo; the couple at the altar call it Tuesday. Housekeeping, before the ceremony: the specification says nothing about *isolation* — nothing about what concurrent transactions may observe while the staging is in progress. This chapter's protocols decide *whether* the work becomes permanent; Chapter 7's levels govern *who may peek* meanwhile. The two concerns compose, and they live in separate pockets; keep them there for Part Eight, when the sagas abandon one and not the other. Finally, the honesty question: is commitment *harder* than consensus? Formally the two trade blows — Rem. 8.2 runs the reductions, and FLP accordingly shadows both, which is why every protocol in this chapter leans on timeouts, majorities, or luck exactly where Chapter 4's did — but the veto changes the failure economics utterly, and no cleverness of protocol design removes a clause of the specification. What cleverness *can* do — and the second half of the chapter is a catalogue of it — is one of three verbs: *relocate* the fragile roles onto sturdier machinery, *shrink* the windows, or *renegotiate* the specification itself. Every modern system below is one of the three.

8.2 Part Two — The Ceremony, Anatomized

The scene: Bertie wishes to move ten pounds from his account, kept at Tuppy's branch, to his aunt's account, kept at Gussie's. The provisional work is done — Tuppy holds the staged debit, Gussie the staged credit, locks on the touched rows — and Bingo officiates: he is the coordinator. The Post carries every word under the ordinary charter, letters delayed, letters drowned. *Phase one, the vows.* Bingo writes to each participant: PREPARE — can you, if commanded, make your portion permanent through any crash, any restart, any inconvenience? Tuppy, receiving it, performs the load-bearing act of the whole protocol: he forces his staged work and his vote to durable storage — writes it to the log, waits for the disk to confirm — and only then replies YES. That log is the *write-ahead log*, so called for the discipline it enforces: the record of an intention is made durable before the intention is acted upon. It is the structure beneath every forced write in this chapter, and beneath the amnesia discipline of Chapters 4 and 5. Durability first, promise

second; a promise made before the ink dries is Chapter 1's inaugural bug wearing a cassock. And from the instant that yes leaves Tuppy's hands, he is bound: Part One's irrevocable vow is, formally, the forced-durable prepared state with the veto surrendered. He may no longer abort his portion unilaterally — not on timeout, not on suspicion, not on the advice of counsel — because his yes may already be part of a commit decision sealed elsewhere. A NO, by contrast, is the one luxurious word in the protocol: a no-voter may abort at once and go home, for no lawful commit can ever include his refusal. *Phase two, the pronouncement.* All votes yes: Bingo writes the commit decision to his own log — *the court of record*: the decision *is* the forced write, not the announcement — and only then tells the participants. Any no, or any silence past his patience: abort. The participants, on hearing the decision, make it so, release their locks, and acknowledge, so Bingo may eventually retire the paperwork. Count the postage in the happy case: prepares out, votes back, decisions out — three messages per participant of consequence, plus acknowledgments — and, more dearly, two forced log writes on the critical path at minimum: each participant's vote, the coordinator's decision. Forced writes are the protocol's heartbeat; every one is a place where a crash is survivable, and every economy on them is a place where it is not.

Two economies are lawful, and worth naming because every production ledger runs them — they descend from Gray's world and survive because they respect the vow. *Presumed abort* — absence means annulment: since the coordinator may abort freely before logging a decision, let the absence of a record mean ABORT, by standing convention. The coordinator then need not force an abort record at all, nor remember aborted transactions, nor answer a recovering participant's query about one with anything but the presumption; forced writes are saved on every failure path, and the awkward recovery question — I find no record; what happened? — acquires a standing answer. Its mirror, presumed commit, exists and is subtler, relocating a forced write rather than removing it; Problem 8.2 sets the bookkeeping as a drill. And the *read-only vote* — leaving before the pronouncement: a participant whose portion wrote nothing has nothing to make permanent and nothing to undo; it answers the prepare with a third word, READ-ONLY, releases its locks at once, and exits the protocol before the pronouncement, since either outcome is the same to it. In estates where most transactions read widely and write narrowly, that one word is worth more latency than any protocol cleverness of its decade.

Now the part Gray insisted upon and folklore forgets: the ceremony is defined as much by its recovery rules as by its happy path — what each party does on waking from a crash, and what each does about silence. The matrix goes by *the widow's rights* — the action on waking, per logged state — and Box 18 sets it out in full; the in-doubt line is the one the whole chapter orbits.

Protocol — Box 18. Two-phase commit with recovery matrix (checked against Gray 1978; presumed abort per the classical treatments)

coordinator, phase one: send PREPARE to all participants; start patience timer
participant, on PREPARE: if unable: reply NO, abort locally (never voted YES)
    if able: *force* staged work + YES to log; reply YES — veto surrendered
    if no writes performed: reply READ-ONLY; release locks; exit protocol
coordinator, phase two: all YES (or READ-ONLY):
    force COMMIT to log (*the decision is this write)*
    any NO, or patience expired: decide ABORT (no forced record under presumed abort)
    send decision to all YES-voters; retransmit until each acknowledges
participant, on decision: apply; release locks; acknowledge (idempotent on repeats)
recovery matrix — on waking, consult own log:
    participant, *working* (no vote): abort locally; a later PREPARE gets NO

12: participant, *prepared*: in doubt — query coordinator (and, cooperatively, fellows); hold locks; wait
13: coordinator, no decision record: decide ABORT; answer queries with ABORT (presumption)
14: coordinator, COMMIT record: re-announce to all; retransmit to the unacknowledged
15: either role, decision applied: answer queries from the log; nothing else to do

Two forced writes on the happy critical path (each vote; the decision). Cooperative termination: a queried fellow with a decision repeats it; a fellow still *working* may vote NO and release the querent. The in-doubt state (line 12) is Thm. 8.2's theorem made flesh: no rule for it decides.

The states, formally, with the couple's paralysis given its dry name.

Definition 8.3 (the two-phase protocol; in-doubt)**.** Box 18's protocol. Participant states, as logged: *working* (no vote recorded); *prepared* (staged work and YES vote forced to the log); *committed*; *aborted*. A participant is *in doubt* while its log says prepared and it holds no decision. The coordinator's states: *collecting*; *committed* (decision forced); *aborted* (forced, or presumed by absence under the presumed-abort convention).

The happy path runs as a thirty-second transcript — prepare, seventeen; force; yes, seventeen; force commit; commit, seventeen; applied, released, acknowledged — and is drawn once, in the season's convention, so the ceremony has a picture as well as a transcript.

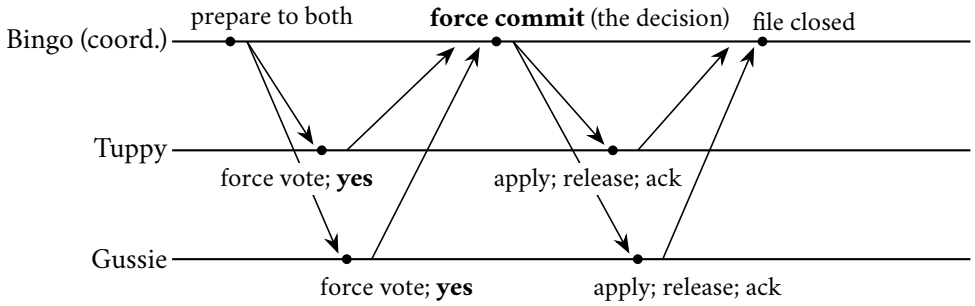


Exhibit 8.1 (the ceremony, happy path). Twelve utterances, two forced writes on the critical path (each vote; the decision). Cut the transcript at any line and consult Box 18's matrix for the widow's rights — Problem 8.1 does exactly that.

The ceremony's safety claim, its skeleton first: commit requires a durable unanimous record, and every recovery rule of Box 18 re-derives its action from forced state, so no crash-and-recovery sequence manufactures disagreement.

Theorem 8.1 (2PC safety)**.** *Under the model box, Box 18 satisfies AC1–AC3 in every execution, through any sequence of crashes and recoveries: no two processes ever decide differently, no* COMMIT *is decided against a* NO *vote or a missing vote, and decisions are stable. It satisfies weak AC4; it is not non-blocking (Thm. 8.2).*

Proof of Theorem 8.1. AC2, first half. A NO voter never enters prepared; the coordinator decides COMMIT only on *n* YES votes (Box 18, line 6), and a recovering coordinator decides only from its forced log, which can contain COMMIT only if the votes were unanimous when forced. Hence no execution with a NO vote contains a COMMIT record anywhere, and participants commit only on receipt of a coordinator COMMIT (or its repetition).

AC1. All participant decisions are copies of coordinator decisions, so it suffices that the coordinator never issues both. The decision is the first forced decision record (line 6/7); the log is sequential and stable;

recovery re-reads and repeats the recorded decision (lines 12–14) and, under presumed abort, treats absence as ABORT — absence is possible only if no COMMIT was ever forced, since COMMIT is forced before any announcement (line 6 precedes line 8). A participant's own unilateral ABORT occurs only from *working* (never voted: lines 15–16), in which case no COMMIT record can ever come to exist (AC2 argument with the missing vote). So all decisions in one execution are equal.

AC3. Decisions are forced log records; every rule in Box 18 consults the log before acting and no rule overwrites a decision record.

Weak AC4. Suppose all failures are eventually repaired and retransmission is stubborn. A working participant eventually receives prepare or the coordinator's abort; a prepared participant eventually reaches a recovered coordinator whose log answers (decision, or absence = abort under presumption — lawful since no commit was forced); committed and aborted states are terminal. The coordinator eventually collects all votes or expires its patience and aborts. Every path terminates in a decision. □

Two engineering footnotes, both scars from real ledgers. First, the court of record must be administered as one: the decision is the forced write itself, not the announcement, and a coordinator that tells participants commit before its own log is durable has created a world where its crash *un-decides* a pronounced marriage — the aunt's credit evaporates on recovery. The pronouncement is the ink, not the breath; the AC1 argument above leans on exactly the line 6-before-line 8 ordering. Second, the in-doubt window is not an abstraction: a participant in doubt holds *locks*, and Chapter 7 taught what held locks do to every other transaction that wants those rows. A fainted officiant does not merely strand one couple; he stalls the whole parish's business calendar, which is why the in-doubt window's length — milliseconds in the happy engineering case, however long recovery takes in the crash case — is the number a practitioner watches above all others in this protocol. And the patience thresholds come from nowhere principled — Chapter 1 settled that — with the two dials pulling opposite ways. The coordinator's patience with voters is cheap generosity: expiring it early merely aborts a transaction nobody had yet promised, and a retry is the Post's ordinary weather. The participant's patience with a silent coordinator, after a yes, buys nothing at any price — that is the theorem ahead — so the practical dial there is not a timeout at all but an escalation: page the operators, surface the transaction on the in-doubt console, and let the recovery drill, not the protocol, decide whether to raise the coordinator from the dead or, in the last resort, adjudicate by hand with the auditors watching. Every mature transaction manager ships that console; its existence is the theorem's signature in the product.

8.3 Part Three — The Blocking Theorem, by Doppelgänger

The claim, stated with care: in any atomic commitment protocol in which participants surrender their veto — and with asynchronous messages, some vote-surrendering step is unavoidable for progress — there is a reachable situation in which a correct, live participant cannot decide anything and must wait, indefinitely, on the recovery of a crashed party. Blocking is not a defect of Gray's ceremony to be engineered away by a cleverer ceremony; it is a property of the specification meeting the world. The instrument is Chapter 3's, tuned one string tighter. Put Tuppy on the stand: he has voted yes, the vow is in his log, and now — silence; Bingo says nothing, Gussie says nothing, the Post shrugs. The *two worlds* are, formally, the indistinguishable executions of the proof below: one in which a commit exists — inscribed in a durable log, acted upon, sealed by parties who then died, the letters to Tuppy crawling through a congested exchange — and one in which nothing was ever decided anywhere in the universe. Tuppy's evidence is identical in both; whatever rule he follows reads only his evidence and must prescribe the same act in both; abort splits world one, commit forges in world two; only waiting survives. The couple stands at the altar because standing is the only correct behaviour — the paralysis is the proof of their good character. And the prosecution requires

no exotic machinery: one crashed coordinator, one crashed participant, one slow letter, every ingredient certified ordinary by Chapter 1's documentary evidence.

Theorem 8.2 (blocking; Gray, Skeen)**.** *No atomic commitment protocol in the model box is non-blocking: every protocol satisfying AC1–AC3 has a reachable configuration in which some correct, live participant has voted* YES *and can neither decide* COMMIT *nor decide* ABORT *until some crashed process recovers. In particular there are executions in which locks are held for the full duration of a coordinator outage.*

Proof of Theorem 8.2. Fix any protocol satisfying AC1–AC3 in the model box, with coordinator B and participants T (Tuppy) and G (Gussie); the argument generalizes verbatim to n participants. Since the protocol satisfies AC2's second half, there is a failure-free execution ρ in which all vote YES and all decide COMMIT. Let ρ_0 be the prefix of ρ ending at the moment T completes its YES vote, and consider the following two extensions, in both of which T receives no further messages for as long as we please (the Post is asynchronous; delay is lawful).

World one. After ρ_0: messages flow between B and G as in ρ; the protocol being live in failure-free runs, B reaches its COMMIT decision and G decides COMMIT (AC2 second half applied to the run where T's silence is merely delay); then B crashes, and G crashes, with every message addressed to T still in flight. A COMMIT decision exists (at G, decided; stability AC3 makes it permanent).

World two. After ρ_0: every message between B and G after ρ_0 is delayed; B and G crash before deciding anything. No decision exists. (If the protocol would have B decide with no further input, run world one's argument from that decision instead; the construction adapts.)

T's local history — its own steps, its log, its received messages — is identical in both worlds: everything after ρ_0 happened elsewhere or not at all, and silence carries no signature. A protocol is a function from local history to action. If it directs T to decide ABORT, then in world one T's ABORT stands against G's COMMIT: AC1 violated. If it directs T to decide COMMIT, consider world two continued: B recovers and — being permitted by AC1–AC3, indeed possibly required, since a vote may never have reached it — resolves the transaction ABORT; alternatively note that in a variant of world two where G voted NO before crashing (indistinguishable to T, whose evidence never mentions G's vote), AC2 *forbids* COMMIT outright. Either way T's COMMIT violates a clause. Hence the protocol may direct neither decision: T must wait, and it waits precisely until some crashed process — a decision-holder — recovers. This is the reachable blocking configuration claimed. □

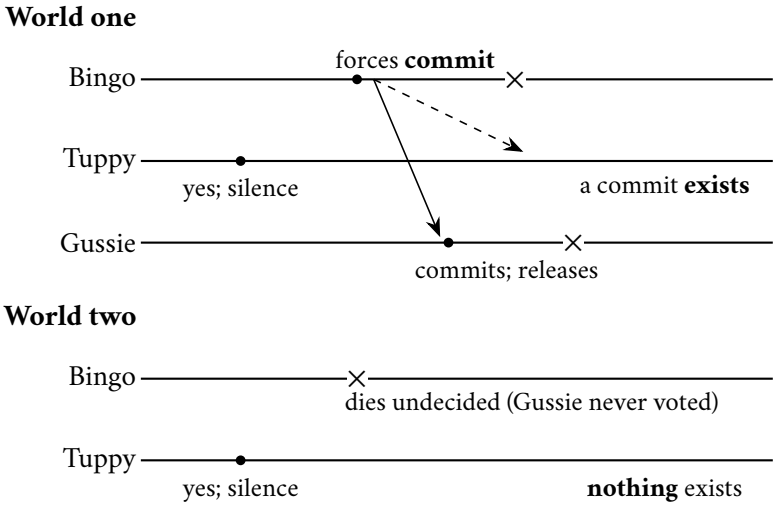


Exhibit 8.2 (the two worlds). Tuppy's evidence — his vote, his log, his silence — is identical in both. Abort splits world one; commit forges in world two; waiting alone is safe in both (Thm. 8.2). The crosses are crashes; the dashed letter to Tuppy is still in flight, indefinitely, lawfully.

Two natural escapes suggest themselves, and both are mirages. First: let Tuppy ask Gussie. By all means — cooperative termination is decent practice, and in many failures it ends the vigil — but the proof has anticipated it: a dead fellow testifies to nothing, and a reachable fellow who is himself yes-and-ignorant merely joins the vigil. The refinement shortens limbo when luck is moderate; it removes nothing when luck is poor, and theorems are priced at poor luck. Second: let the coordinator distribute word of the impending decision before finalizing it. Sir has just reinvented the third phase, and the museum in the next part is its mausoleum.

Remark 8.4 (cooperative termination, and its limit)*.* An in-doubt participant may query its fellows: any fellow holding a decision repeats it; any fellow that has *not* voted may vote NO, decide ABORT, and thereby release the querent (safe by AC2). The refinement fails exactly when every reachable fellow is prepared-and-ignorant and every decision-holder is dark — the total failure of decision holders — which is the configuration Thm. 8.2's proof constructs. Blocking is thereby confined, not removed.

The theorem's exact perimeter, because precision here is what separates it from folklore. Blocking bites when every party that might hold the decision is unreachable — in the staging, coordinator and Gussie both dark: a total failure of the decision-holders — and the companion results draw the line sharply: grant a synchronous network and forbid partitions, and non-blocking protocols exist (the museum's exhibit builds one); allow total failures or partitions, and every protocol has executions that block. The practitioners' summary is blunter: two-phase commit blocks rarely and briefly when engineered well, always possibly, and the distance between those adverbs is the whole craft of Box 18's recovery matrix. Nor does the theorem say the wait is forever in practice — Bingo recovers, reads the record, repeats the pronouncement, and life resumes — or that every failure blocks: a coordinator crash before any commit is sealed resolves cleanly to abort, and a participant crash strands nobody but itself if the coordinator lives. What it says is that there exists a conjunction — yes votes surrendered, decision-holders dark, evidence symmetric — in which no rule can do better than wait, and therefore the worst case of any veto-honouring protocol includes limbo.

Attend, finally, to the sabotage the proof convicts in advance, because it ships in real middleware: the participant configured to "time out and roll back" after voting yes — *heuristic abort*, the vendors call it, with an adjective doing heroic work. The doppelgänger has just shown exactly which world that participant gambles against, and Problem 8.6 — the chapter's sabotage, certified fatal — writes out the split-brain ledger it produces, entry by entry, until the auditors arrive. The operational record is not hypothetical: every transaction-manager vendor's documentation carries the chapter — usually titled something anodyne, *resolving in-doubt transactions* — instructing administrators in committing or rolling back a stranded branch by hand, and the interface standards themselves define the *heuristic outcome* — "we guessed": a unilateral resolution after a yes vote — complete with error codes for the aftermath: heuristically committed, heuristically rolled back, and, the standards body's own name for the split ledger, heuristically mixed. When a protocol's official vocabulary includes a word for *we guessed*, the theorem has been acknowledged in the small print.

8.4 Part Four — The Museum of Three Phases

The first generation's response to the theorem was, naturally, to try to beat it, and the handsomest attempt deserves its glass case: three-phase commit, Skeen, 1981. The insight is genuinely elegant and worth thirty seconds of admiration before the placard. Blocking, Skeen observed, comes from a fatal adjacency: in two-phase commit there is a state — voted yes, awaiting news — from which both commit and abort remain

reachable outcomes, and a participant stranded there can rule out neither. Very well: forbid the adjacency. Insert a buffer state between vowed and married — the *pre-commit* of the definition below: commit reachable, uncertainty excluded — so the states march in single file: uncertain, prepared-to-commit, committed. A participant stranded in uncertainty knows nobody can yet have committed — abort is safe; a participant stranded in pre-commit knows nobody can still be uncertain — and there the protocol's actual cleverness lives, in the termination rule under a new officiant, elected by the survivors with Chapter 4's machinery, smuggled in at the door.

Definition 8.5 (three-phase commit)**.** Skeen's protocol: between prepared and committed insert *pre-commit*. Coordinator, on unanimous YES: disseminate pre-commit; on acknowledgment from all reachable participants, commit. Termination rule under a new coordinator: if any canvassed participant is committed, commit all; if any is still uncertain (prepared, no pre-commit), abort all; if all reachable are in pre-commit, complete the commit.

Proposition 8.3 (3PC, both verdicts)**.** *(i) In a synchronous crash-stop model with reliable bounded delivery and no partitions, 3PC with its termination rule is non-blocking. (ii) With partitions the protocol violates AC1: there is an execution in which one side of a partition commits while the other aborts (Problem 8.7 constructs it). The additional round and its forced writes are paid in every failure-free execution.*

Each termination rule is sound — against crashes; the stated assumptions are the placard: a synchronous network, bounded delays honoured, and no partitions. Seven chapters of this course write the obituary unassisted. Bounded delays honoured are Chapter 1's fiction and Chapter 4's expensive rental; a partition is not an exotic event but a Tuesday; and under a partition the buffer state's logic inverts into a weapon — one shore's survivors, seeing pre-commit, complete the marriage; the other shore's, seeing uncertainty, lawfully annul it; the cable is restored and the estate holds both verdicts at once, the precise catastrophe the extra phase was hired to prevent. Add the bill even in fair weather — an entire additional round of letters and forced writes on every single transaction, paid always, to defend against a crash that comes rarely — and the verdict of history is unanimous: nobody runs it. The scholarly line did continue — quorum-based and enhanced variants repair the split verdict by requiring majorities before any shore may move, at which point one has reinvented, feature by feature, a parliament with extra steps; the literature's eventual settlement was to keep the parliament and drop the steps, which is precisely where the next part begins. The placard, fairly written: *a correct solution to a problem the real network declines to pose, at a price the happy path declines to pay.* The true legacy is diagnostic: it taught the field that the enemy was never the number of phases — it was the assumption of a world in which silence can be told from death. What three-phase commit wanted, without knowing the words, was Chapter 5's failure detector — and the honest version of that bargain, agreement machinery that survives asynchrony by majority rather than by clockwork, already sat in Chapter 4's parliament, waiting for the industry to notice.

8.5 Part Five — Buying Better Physics: Consensus Groups, Spanner, and the Seven-Millisecond Apology

The modern settlement begins with the first verb: *relocate*. The blocking theorem's pressure point was always a singular mortal holding a singular log — the officiant and his register — and Chapter 5 taught the season's standing remedy for singular mortals: make the throne an office, the log a parliament's decree. Run the coordinator as a replicated state machine — its decision is not a forced write on one disk but a committed entry in a consensus group; the officiant who faints is replaced by Chapter 4's election, and the new officiant reads the *replicated* register and completes the ceremony. Run each participant's vote the same

way, and no single machine's death strands anyone. Be precise about what has and has not been purchased: the theorem stands — consensus itself can stall while a partition denies majorities, and during that stall the in-doubt locks stay held; limbo is shortened from "until a particular machine is repaired" to "until a majority can be assembled" — shortened, not abolished. In exchange, every phase now costs a consensus round rather than a disk write. That is the trade the industry made, and it is the correct trade.

Concretely, since everything after this chapter assumes it: each parliament has a leader — Chapter 5's throne — and the transactional roles are played by leaders on behalf of their groups, the coordinator the leader of one designated group, each participant the leader of its shard's group. A participant's yes is no longer a private forced write but a decree — the staged work and the vote committed through its group's log, surviving the leader's death because any successor inherits the log and the promise with it. The coordinator's decision likewise: pronounced the instant the commit entry is chosen by its parliament's majority, whereupon any successor coordinator, elected after any faint, reads the decree and repeats it to whoever still waits. Every mortal in Part Two's ceremony has been replaced by an office; the faint that stranded the couple now merely triggers a by-election, and the vigil lasts one election timeout rather than one hardware repair. The purists' edition of this move exists in the literature under the plainest imaginable title — consensus on transaction commit, written jointly by Gray and Lamport themselves: the votes become Paxos instances outright, the two-phase scaffolding dissolves into one parliament per participant, and the coordinator dwindles to a convenience (Problem 8.11 reconstructs it). The industry mostly builds the pragmatic edition; but when the founder of the ledger and the founder of the parliament finally collaborated, they proved the marriage lawful. The arrangement's most famous storefront is the paper the course opens with some ceremony.

Spanner (Corbett and colleagues, Google, 2012) is the course's single most load-bearing industrial text, and its architecture, in one breath, is everything the season has built, assembled: data partitioned into shards; each shard a Paxos group — Chapter 4's parliament, thousands of instances strong; transactions within a shard decided by its parliament; transactions *across* shards — and here is debt #18 presented for payment — by two-phase commit in which coordinator and participants are all parliaments, the fragile roles relocated wholesale onto replicated stone; isolation by two-phase locking atop snapshot machinery of Chapter 7's kind. So far, a superbly engineered assembly of known parts. What made the paper famous is none of it. What made the paper famous is a confession about *time* — and the audacity of answering a logical problem with a purchase order.

Here is why the clocks must return. Google wanted *external consistency* — the paper's term, and Chapter 7 licenses its government name, strict serializability: if a transaction commits, and afterwards — genuinely afterwards, in the world's own order, telephone calls included — a second transaction begins, the second must see the first. Within one parliament the log's order supplies this for free. Across parliaments, on opposite sides of an ocean, whose order? Chapter 2's verdict was that "afterwards" across machines is precisely what a distributed system does not have: wall clocks order nothing at fine grain. The classical escape — capture causality with vector clocks — dies at planetary scale and, worse, cannot see the telephone: happens-before tracks messages, and reality contains channels the system never carries. To respect *every* side channel, you must respect real time itself. Google's response was not to weaken the promise; it was to change what a clock *is*. TrueTime: in every datacentre, GPS receivers and atomic clocks — caesium-grade hardware, purchased with actual money — and an API of magnificent honesty. Ask the time and it does not return a number — a number would be Chapter 2's lie in a nicer suit. It returns the *confessed interval*: earliest and latest, the true instant staked to lie between, the width the confessed uncertainty — drift since last synchronization, delays in the synchronization itself, all of it measured, bounded, and admitted; single-digit milliseconds in the paper's era. The engineering beneath the confession bears one sentence: time masters in every datacentre — some slaved to the satellites, some to the caesium — cross-examined against one

another so that a liar among them is outvoted; between synchronizations each machine's interval *widens* at an assumed worst-case drift rate and snaps narrow at the next audit. The sawtooth of humility: certainty leaking between confessions, restored by the network's absolution. Upon that one honest confession, the commit-wait rule builds external consistency with arithmetic a child could check.

Definition 8.6 (TrueTime; commit-wait)**.** $TT.\text{now}()$ returns $[e, l]$ with $e \le t_{\text{abs}} \le l$ at the instant of the call. Commit-wait (Box 19): a committing transaction T calls $TT.\text{now}() = [e_1, l_1]$, takes commit timestamp $s(T) = l_1$, and delays announcing (and lock release) until a later call returns $[e_2, l_2]$ with $e_2 > s(T)$. Start rule: a transaction's start event is its first invocation reaching any server.

The set piece, with concrete numbers. A transaction is ready to commit; the coordinating parliament asks TrueTime for the hour and receives earliest ten o'clock and three milliseconds, latest ten o'clock and ten — a confessed uncertainty of seven. Clause one: the timestamp is the *latest* edge, ten-and-ten — not the middle, not the earliest: the pessimistic edge, the one instant no clock in the estate can yet have passed for certain. Clause two — the vigil, the seven-millisecond apology itself: having chosen the stamp, the parliament *waits*, holding the pronouncement until TrueTime's earliest edge climbs past ten-and-ten — about seven milliseconds, one uncertainty's worth of silence — and only then announces and releases the locks. Now the arithmetic pays. Let a second transaction begin genuinely after the first has committed: Bertie sees the confirmation on his screen in London, telephones the aunt in Sydney, and *she* then issues the read — every side channel reality offers is in play. The first stamp was held until it lay in every clock's past, so the true instant of its announcement exceeds its own stamp; the aunt's transaction begins later still, and when Sydney stamps it at *its* latest edge — an edge that cannot precede the true now — her stamp lands strictly above ten-and-ten. Afterwards in the world implies afterwards in the timestamps: no vector, no gossip, no message from London to Sydney carrying causality — the two parliaments never spoke. They merely both deferred to the same purchased, confessed, bounded physics. The argument, structured into a theorem one can carry:

Theorem 8.4 (commit-wait implies external consistency)**.** *Under Def. 8.6, if transaction T_1's commit is announced before transaction T_2 starts (in absolute time), then $s(T_1) < s(T_2)$. Consequently timestamp order extends real-time precedence of transactions, and snapshot execution at these timestamps yields strict serializability (Def. 7.11's clause satisfied with whole transactions as the operations).*

Proof of Theorem 8.4. Write $t_{\text{abs}}(x)$ for the absolute time of event x. *Lemma (the wait pays).* When T_1's announcement occurs, $s(T_1) < t_{\text{abs}}(\text{announce})$. Indeed the protocol announced only after some call returned $[e_2, l_2]$ with $e_2 > s(T_1)$, and the axiom gives $t_{\text{abs}}(\text{that call}) \ge e_2 > s(T_1)$; the announcement follows the call.

Main step. T_2 starts after the announcement: $t_{\text{abs}}(\text{start}_2) > t_{\text{abs}}(\text{announce}) > s(T_1)$ by the lemma. T_2's commit stamp is taken as the *latest* edge of a call made no earlier than its start: $s(T_2) = l' \ge t_{\text{abs}}(\text{call}) \ge t_{\text{abs}}(\text{start}_2) > s(T_1)$, the middle inequality by the axiom's upper half. Hence $s(T_1) < s(T_2)$.

Strictness of the serialization. Order transactions by commit stamp (ties broken arbitrarily but consistently). Multiversion execution at these stamps makes every transaction read the latest stamps below its own — a serial story in stamp order — and the main step shows stamp order extends real-time precedence. By Chapter 7's Def. 7.11, the execution is strictly serializable. Note what was never used: no message between T_1's and T_2's coordinators, no shared machinery — only the axiom, twice, and one deliberate delay of approximately the confessed uncertainty per commit. □

Protocol — Box 19. Spanner commit-wait (checked against Corbett et al. 2012, §§3–4)

1: **roles:** every shard a Paxos group; participants and coordinator are group *leaders*; votes and decisions are group log entries, not private writes
2: **commit, coordinating leader:** collect prepares (each a committed log entry with a prepare stamp)
3: $[e, l] \leftarrow TT.\text{now}()$; then $s \leftarrow \max(l,$ prepare stamps, watermark)
4: commit the decision at stamp s through Paxos
5: **commit-wait:** block until $TT.\text{now}().e > s$; then announce, release locks, apply
6: **read-only transaction at stamp t:** at each shard, read versions below t; any replica whose safe time exceeds t may serve — no locks, no leader required

The wait is approximately one confessed uncertainty; it enforces Thm. 8.4's lemma. Keeping the confession narrow is an operations discipline: every microsecond of clock sloppiness is paid by every writer, in latency.

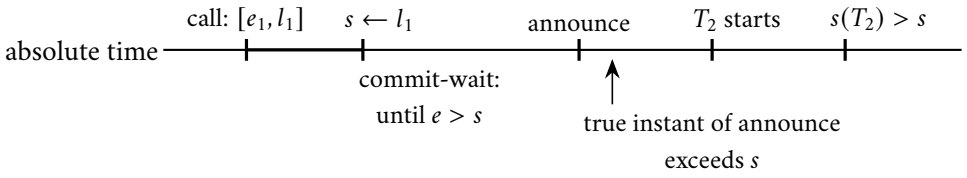


Exhibit 8.3 (commit-wait pays). The stamp is the latest edge; the vigil ends only when every clock's earliest edge has passed it; so the announcement — and everything after it, telephones included — lies strictly above the stamp (Thm. 8.4). One confessed uncertainty of silence per commit buys the ordering no message carried.

One gift falls out of the purchase, paying Chapter 7's photograph forward. Because every commit bears a truthful timestamp, the state of the entire estate as of half past ten is a well-defined object, assembled without locks and without coordination from every shard's versioned history — a consistent photograph, purchased rather than choreographed. A read-only transaction names a timestamp — safely in the past, or the present with a commit-wait-grade guarantee — and reads that snapshot at every parliament it visits, blocking nobody and blocked by nobody; the analysts' enormous queries run against the ledger while the tellers keep writing. The followers serve these reads too: each replica tracks its *safe time* — the follower watermark beneath which its log is known complete — and any read stamped below it needs no leader at all: the throne's postage, saved by honest clocks. Every transaction pays the little vigil, and the estate's engineers are thereby placed on the clock-uncertainty payroll forever — the paper is charmingly frank that keeping the confession narrow is now an operations discipline, since every microsecond of sloppiness in the time infrastructure is paid, by every writer, in latency. Debt #5: presented, and paid in caesium.

One movement more in this hall, for the estates without a hardware budget: hybrid logical clocks (Kulkarni and colleagues, 2014) — the civilian's TrueTime, and CockroachDB's arrangement of record. Weld Chapter 2's two instruments into one stamp: a physical reading, disciplined by ordinary NTP, plus a logical counter that ticks whenever causality demands order the physical part cannot certify — Lamport's clock in a wristwatch case. The stamp tracks wall time closely — you may point at a record and say roughly when — while the counter quietly guarantees that message-connected events never stand in the wrong order. What is *not* purchased, and the honest engineer says so aloud: without a bounded confession there is no commit-wait proof, and the telephone can outrun NTP's error bars — reality's side channels may still glimpse a gap that the stamps deny. The civilian arrangements paper over the residue with retries and uncertainty

windows of their own, and pay for it in tail latency rather than in caesium. The moral is the season's oldest: you may buy the bound, or you may live carefully without it, but you may not have it for free.

Definition 8.7 (hybrid logical clock)**.** Each process keeps a stamp (pt, c): pt a physical part, c a logical counter, ordered lexicographically. On a local or send event with wall reading w: if $w > pt$ then $(pt, c) \leftarrow (w, 0)$ else $c \leftarrow c + 1$. On receiving stamp (pt', c') with wall reading w: set $pt'' = \max(pt, pt', w)$; set c'' to (the matching counter) +1 if pt'' equals pt or pt', else 0. The stamp satisfies the clock condition of Chapter 2, tracks wall time to within the clock error, and its counter is bounded (Problem 8.10, starred).

8.6 Part Six — Percolator: The Coordinator Dissolved

The second verb is *shrink*, and its most instructive specimen is Percolator (Google, 2010) — a system built to solve a homely problem, incremental indexing of the web, atop Bigtable, a store that offered single-row transactions and nothing more. No transaction manager, no officiant anywhere in the building — and the designers' response was a move of real cunning: if no machine may be the coordinator, let the *data* be the coordinator. Dissolve the ceremony into the rows themselves, and let the court of record be a column. The mechanics, set out as the ledger-keeping they are: every row carries, alongside its data, two small bureaucratic columns — a lock column and a write column. A transaction stages its writes at a snapshot timestamp in Chapter 7's multiversion manner and, when the time comes to commit, performs two-phase commit *in the columns*. Phase one: for every written row, plant a lock — a single-row atomic act, Bigtable's one native sacrament — and among all its rows the transaction anoints one as the *primary*: the anointed row's lock cell *is* the transaction's court of record, the single place where its fate will be written, and every other row's lock is a signpost bearing the primary's address. All locks planted — every vow collected. Phase two: at a fresh timestamp, the transaction executes upon the primary a single-row atomic exchange — erase the lock, inscribe the commit record — and that one cell-write *is* the pronouncement: before it, nothing is committed; after it, everything is, and the remaining rows' inscriptions are mere paperwork, completed lazily, whenever convenient. The officiant has become an entry in the parish register that writes itself. Two pieces of supporting furniture: timestamps come from a single oracle — a replicated service dispensing strictly increasing stamps in batches, a standing reminder that not every global agreement needs a parliament, for agreement on the next number is cheap when one office holds the till; and a reader at its snapshot stamp first checks the lock column, since a lock from an earlier stamp means someone's ceremony is mid-air exactly where the photograph would be taken — the reader waits briefly or, past patience, invokes the cleanup rite itself, rather than ever reading around a pronouncement in progress.

Protocol — Box 20. Percolator transactions (checked against the 2010 paper)

1: **columns per row:** data (versioned); lock; write (commit records). Single-row operations are atomic (Bigtable's sacrament)
2: **prewrite (phase one), for each written row:** abort if a newer write record or any lock exists; else stage data at start stamp and plant lock naming the *primary* row
3: **commit (phase two):** at a fresh commit stamp, on the primary, atomically: erase lock, inscribe write record — *this cell-write is the pronouncement*
4: then, lazily: inscribe write records and erase locks on the secondaries
5: **read at snapshot stamp t:** if a lock below t blocks the row: wait briefly, then invoke cleanup; else return newest data below t per the write column
6: **cleanup (any passerby), on a stale lock:** consult its primary: write record present ⇒ roll *forward* (complete secondaries); primary lock still present ⇒ atomically erase it (a binding abort),

roll back debris

The primary's cell is a public court of record; limbo becomes a chore any passerby may finish. Timestamps from a single replicated oracle — agreement on "the next number" needs a till, not a parliament.

Now the part that earns this hall its place: recovery, and the fate of the theorem. A Percolator client is a mortal process; it may faint at any point in the ceremony, leaving locks strewn like dropped vows. Another transaction arrives, finds a stale lock in its path — and here is the difference from Part Three: *the evidence is not private*. The lock names the primary; the primary's cell holds the truth, readable by anyone; and single-row atomicity makes consultation and cleanup safe. Commit record present at the primary: the marriage was pronounced, and the finder helpfully completes the dead transaction's paperwork, rolling the write forward, and proceeds. Primary still locked: no pronouncement ever happened — the finder erases the primary lock atomically (the erasure *is* a binding abort, inscribed in the same court, so no late-waking client can pronounce afterward), rolls the debris back, and proceeds. This is the *passerby's rite* — lazy resolution: any reader completes or annuls a dead client's transaction. The in-doubt window has not vanished — the theorem is not insulted; there is still an instant of fate held in a single cell — but the *waiting* has been transformed into *work that any passerby may finish*. Nobody stands at the altar: whoever next enters the church completes or annuls the ceremony from the public record and gets on with their own errand. Limbo shrunk from a vigil to a chore. The price, paid in the currency this course has taught you to check first: every conflict is discovered by *encounter* rather than by announcement; cleanup rides on the backs of innocent bystanders' latency; and the whole edifice leans on the store's single-row atomicity the way Part Two leaned on forced writes — the sacrament relocated, never removed. The system's homely purpose explains the temperament: Percolator was built to keep a web index fresh, where one transaction's latency mattered far less than the fleet's throughput, and where a stray hour-old lock, cleaned up by a passerby, costs nobody a customer. Transplant it unmodified into a checkout page and the bet inverts; transplant its *pattern* — the public court of record, the anointed primary, the passerby's right of completion — and it improves nearly any ledger it touches. Patterns travel; temperaments do not.

8.7 Part Seven — Calvin: Abolishing the Conversation

The third verb is *renegotiate*, and it has two very different practitioners. The first renegotiates the *machinery*: Calvin (Thomson and Abadi, 2012), a design whose opening move is to ask why transactions quarrel at all. They quarrel — deadlock, block, abort, retry — because each node discovers a transaction's demands *during* execution, in whatever order the Post and the scheduler conspire to produce. Calvin's answer: remove the discovery. *Agree on inputs, not outcomes.* All transactions, before touching any datum, are submitted to a global sequencing layer — Chapter 5's replicated log, in its starring industrial cameo — which batches them and stamps a single, total, replicated order; then every replica executes the same transactions in the same order under a deterministic scheduler: same inputs, same order, same code, therefore same state everywhere, no coordination during execution at all. Two-phase commit evaporates — not defeated but *mooted*: there is nothing to vote on. A distributed transaction in Calvin is not a negotiation among parties who might disagree; it is a scheduled performance of a score already published. The blocking theorem is not overturned — it is *starved*: no vetoes are surrendered mid-flight because there are no mid-flight decisions left.

The execution layer earns a sentence more, because "deterministic" hides the craft. Each replica's scheduler grants locks strictly in the published order — transaction forty acquires before forty-one wherever the two touch, whatever threads the operating system happens to favour — so the interleaving is not merely

repeatable but identical across replicas by construction; and deadlock dies of the same medicine, since a fixed acquisition order admits no cycles. Replication comes free and exact: ship the input log, not the effects — Chapter 5's determinism diet, served as the entire cuisine. And the sequencing layer itself is sharded and batched — parliaments agreeing on batch boundaries rather than on each transaction singly — which is how a design in which every transaction passes through one log avoids making that log the estate's narrowest door. The price is stated at the door, and the honesty is why the hall is in the tour: determinism demands the read and write sets be knowable *before* sequencing — the score must list the instruments. A transaction that must read the database to learn what it will touch runs a reconnaissance pass — once, without commitment, to discover its footprint — and is then resubmitted, declared, with a re-check in case the world moved meanwhile. And the interactive transaction — begin; read; think in application code; write; think again; commit — is surrendered entirely at the door, because the *thinking* is not in the score. Calvin serves estates whose transactions arrive as complete stored procedures, and serves them magnificently: no distributed deadlock, no in-doubt windows, replication for free out of the sequencer; estates that cannot pre-declare their business must shop in the other halls. It is the cleanest deal in the building, and the narrowest. One aside for the season's finale to collect: *agree on inputs, then compute deterministically everywhere* is also, to the letter, how ten thousand accelerators will share one optimizer — Calvin is a database wearing the synchronous training loop's constitution, and neither field, so far as I can determine, has yet sent the other a thank-you note.

8.8 Part Eight — Sagas: Honest Abandonment

The second renegotiator renegotiates the *promise itself*, and its papers are older than most of the chapter: sagas (Garcia-Molina and Salem, 1987), written for long-lived transactions that would hold locks for hours, and rediscovered wholesale by the microservices era, where every business process crosses a dozen services and no two-phase ceremony is on offer because no service will surrender its veto to another's coordinator. The bargain, stated as its authors stated it, without cosmetics: break the long transaction into the *chain and apologies* — a sequence of steps, each a local transaction, atomic in its own house, and for each step, written in advance, a *compensation*: a second local transaction that undoes the step's business effect. Book the flight; the compensation cancels it. Charge the card; the compensation refunds it. Run the chain forward; on failure at any step, run the completed steps' compensations in reverse. Atomicity is thus re-created at the business level — eventually, all of it or none of it, in effect — but isolation is abandoned, and abandoned *honestly*: between a step and any compensation that may later erase it, the intermediate state is visible to the world. There is no photograph, no snapshot, no lock spanning the chain; the aunt may glimpse a booking that will never finally exist.

Definition 8.8 (saga; compensation; pivot)**.** A saga is a sequence $T_1, \ldots, T_n$ of local transactions with compensations $C_1, \ldots, C_{n-1}$, where C_i is a local transaction that reverses the *business effect* of T_i. Lawful executions: $T_1 \cdots T_n$ (success), or $T_1 \cdots T_k\, C_k \cdots C_1$ for some $k < n$ (failure and retreat). A *pivot* decomposition orders the chain so every step before the pivot is compensable, the pivot is the single uncompensable step, and every step after it is retriable to success (forward recovery only).

What survives and what does not, the skeleton first: compensations restore the business state along any failure prefix, but visibility of interim states is not restorable — the anomaly is exhibited, not hidden.

Proposition 8.5 (sagas: what survives, what does not)**.** *(i) Under Def. 8.8, every failure prefix is extended to a terminal state whose business effect equals either the full chain's or the empty chain's — atomicity in effect. (ii) Isolation does not survive: there are executions in which a concurrent transaction reads the state after T_i and*

acts on it, after which C_i runs; the observer's effects rest on retracted business, and no compensation reaches them (Exhibit 8.4). A compensation is a new forward transaction, not an undo of history.

Proof of Proposition 8.5. (i) By induction on the failure point k: each C_i is assumed a total local transaction reversing T_i's business effect, and the reversed composition $C_k \circ \cdots \circ C_1$ applied to the state after $T_1 \cdots T_k$ yields the business effect of the empty chain; retriable steps after a pivot terminate by assumption, yielding the full chain's effect. (ii) Exhibit 8.4's interleaving: observer O runs entirely between T_i's commit and C_i's commit, reading T_i's output — lawful, since steps are ordinary committed local transactions and nothing spans them. O's own writes commit before C_i runs; C_i touches only the saga's state. The final state contains O's effects derived from business the chain has retracted; no lawful execution of $\{\text{saga absent}\} \cup \{O\}$ produces it. The anomaly is therefore real and unremovable without adding cross-step machinery — semantic locks, fences, or pivot placement — which is design, not protocol. □

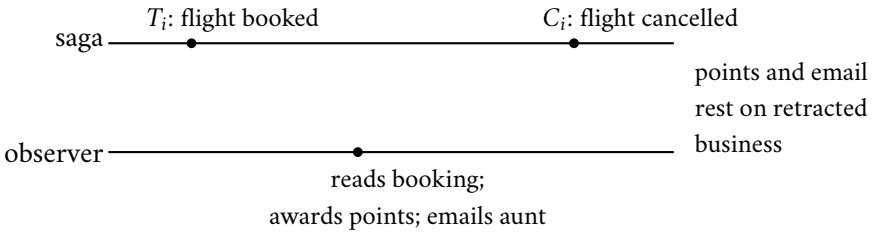


Exhibit 8.4 (the dirty read of half-done business). The observer runs entirely between step and compensation — lawfully, for nothing spans them (Prop. 8.5). A compensation is a new transaction, not an undo: it cannot reach the observer's effects. Fences (states marked provisional) and pivot placement are design remedies, not protocol ones.

Stage the hazard, for it is the two doctors' shabbier cousin and it walks through production systems daily. Bertie's travel saga books the flight — step one, committed, visible — and proceeds toward the hotel; the hotel refuses; the saga turns to compensate. But in the interval, another process *read the booked flight and acted*: awarded Bertie his loyalty points; emailed the aunt the itinerary; sold him, on the strength of his confirmed travel, cancellation insurance. The compensation cancels the flight — it cannot cancel the *reads*. The points ledger, the aunt's expectations, and the insurance book now rest on business that never happened: a dirty read of half-done business, made durable by third parties. Whence the discipline that separates adult saga engineering from abdication with paperwork: compensations designed *with* their steps, not discovered after the outage; interim states marked as provisional where outsiders might read them — a booking in state pending-saga is a fence, and cheap; steps that cannot be compensated — the email sent, the parcel shipped, the cash dispensed — deliberately ordered *last*, as pivots beyond which the saga only rolls forward; and the anomaly budget written down, in prose, where the product manager signs: *these* interim states may be seen, by *these* observers, and *this* is the cost when they are. A team that says "we use sagas" and cannot produce that page is practising not architecture but optimism with a vocabulary.

The original paper is more careful than most of its descendants, and two of its distinctions deserve rescue from the slide decks. Backward versus forward recovery: compensating backward is one lawful response to a failed step; the other is to press forward — retry the failed step until it succeeds, or substitute an equivalent, another hotel in the same city — and a well-built saga declares, per step, which direction it takes, because some business processes must not run backward once begun. And the pivot — the *point of no return*, formalized in Def. 8.8: compensable before, retriable after, the pivot itself the single moment of commitment. In Bertie's itinerary the pivot is the card charge: refundable bookings before it; the non-refundable ticketing and the aunt's email strictly after; so the saga can always retreat before the charge and always

advance beyond it (Problem 8.8 designs the chain and then convicts it under interleaving). The modern trade adds one operational schism worth naming: orchestration — a single conductor process driving the chain, the coordinator reborn at the business layer, with the same funeral to plan — versus choreography, each service reacting to the previous one's events, which distributes the conductor and thereby distributes the confusion. Either can be built well; only one of them can be *read*, and outages are read at three in the morning.

The triage that closes the halls, one breath per verb. The cheapest distributed transaction is the one your schema made unnecessary — co-locate what commits together; where things live is Chapter 9's entire business, and this chapter issues its debt accordingly. Failing that: if your transactions can be declared, schedule them — Calvin's deal. If your invariants span shards and your purse is deep, buy the physics — Spanner's deal. If your steps are loosely coupled business anyway, compensate honestly — the saga's deal, page and all. And if you must hold the classical ceremony, hold it on parliaments, keep the windows short, and plan the officiant's funeral before the wedding.

8.9 The Whiteboard

For the design review.

1. Two-phase commit blocks by theorem, not by bug: plan the coordinator's funeral before the wedding — replicated coordinator state, cooperative termination, alarms on in-doubt age. Heuristic abort after a yes is a split-ledger generator with a reassuring name; the in-doubt console is the theorem's embassy in the product — staff it.

2. Strict serializability across shards is purchased (bounded uncertainty, confessed as an interval, one wait per commit), scheduled (determinism; the interactive transaction surrendered), or renounced (sagas; the apology plan in writing — compensations designed with their steps, pivots ordered last, interim visibility budgeted and signed).

3. The recovery matrix is the protocol; the happy path is the trivial part. Ask of any transactional system: where is the court of record, who may read it, and who may complete a stranger's ceremony?

4. Atomicity and isolation live in separate pockets: sagas abandon only the latter; snapshot stores relax isolation while committing atomically; "distributed transactions" in a brochure names neither. Ask for the anomaly, not the adjective.

The register. Paid, two entries: #5 — physical clocks, issued the moment Chapter 2 convicted wall clocks of ordering nothing: returned armed (TrueTime's confessed interval) and expensive (caesium in the data-centres; one uncertainty of latency per commit) — the invoice denominated exactly as promised. #18 — serializability across shards, issued only a chapter ago: two-phase commit over parliaments, with commit-wait standing guard over the clock. Issued: #20 — everything this chapter assumed the shards were simply *there*; where things live is a load-bearing decision this chapter never takes, and it is Chapter 9's entire business. Episode word count: 10,166 — above the floor; the amended corrective (§5 of the LEDGER: audit immediately after drafting) was applied and caught the shortfall while the material was warm.

8.10 Problems

Tier I — Calisthenics

Problem 8.1 (the widow's rights)**.** For each cut of Exhibit 8.1's transcript, state every party's recovery action per Box 18 on waking, and whether locks are held meanwhile: (a) after Bingo sends prepares, before any vote; (b) after Tuppy forces YES, before Gussie receives prepare; (c) after both votes arrive, before the commit is forced; (d) after the commit is forced, before any announcement; (e) after Tuppy applies, before Gussie hears.

Problem 8.2 (presumed-abort bookkeeping)**.** Under presumed abort: (a) list the forced writes in a committed transaction and in an aborted one, for coordinator and for each participant; (b) explain why the coordinator may forget a transaction the moment all commit acknowledgments arrive, and what it answers, forever after, to a late query about it; (c) show the convention is unsound if a coordinator may answer queries *before* its abort decision is final — construct the two-answers execution.

Problem 8.3 (the third word)**.** A transaction reads at ten branches and writes at one. (a) Count forced writes and message rounds with and without the read-only vote. (b) The lone writer crashes after voting YES; the readers have all exited. Who is in doubt, and who is not, and why is the asymmetry harmless? (c) A reader, after replying READ-ONLY, is asked by an in-doubt fellow for the outcome. What can it truthfully say?

Problem 8.4 (commit-wait arithmetic)**.** TrueTime returns $[e, l]$ with the confessed uncertainty $\varepsilon = l - e$ at four milliseconds. (a) A transaction calls now at true time ten o'clock exactly and receives a centred interval; compute its stamp and the earliest true time its announcement may occur. (b) A second transaction starts one millisecond after the announcement; bound its stamp from below. (c) Uncertainty doubles to eight milliseconds during a time-master outage; restate the commit latency and name who pays it. (d) State in one sentence why the stamp is the *latest* edge and not the midpoint.

Problem 8.5 (name the verb)**.** Classify each design as relocate, shrink, or renegotiate, and name the residual limbo: (a) 2PC with coordinator state in a Raft group; (b) Percolator; (c) Calvin; (d) a saga with orchestrator; (e) Spanner's cross-shard commit.

Tier II — The Real Thing

Problem 8.6 (sabotage: the heuristic fleet)**.** A middleware vendor ships participants that *abort on timeout* after a YES vote ("to protect availability"), defaulting to thirty seconds. Exhibit the fatal execution: a complete message-and-crash schedule in which one participant heuristically aborts while another commits the same transaction, ending with the split ledger — Bertie debited, the aunt uncredited, both banks' books internally consistent and mutually irreconcilable. State the exact line of Box 18 the configuration violates, and which clause of Def. 8.1 dies. Then argue (three sentences) why no timeout value fixes it.

Problem 8.7 (3PC's split verdict)**.** Construct Prop. 8.3(ii)'s execution: five participants, coordinator crashed after disseminating pre-commit to two of them, network partitioned so those two are on one shore and the uncertain three on the other. Apply Def. 8.5's termination rule on each shore; exhibit the two verdicts; then state which assumption of Prop. 8.3(i) each shore violated, and why requiring a majority to proceed (the quorum repair) is tantamount to a parliament.

Problem 8.8 (the travel saga, designed and convicted)**.** Design the booking saga: reserve flight (compensable), reserve hotel (compensable), charge card (pivot), issue ticket (retriable), notify (retriable). (a) Write each compensation and mark forward- versus backward-recovering steps. (b) Exhibit the interleaving in

which a concurrent budget report reads the card charge that a later retreat refunds; name the anomaly in Chapter 7's vocabulary. (c) Add the provisional-state fence and show which observers it protects and which it cannot. (d) Reorder the chain to put the ticket issue before the card charge and exhibit the strictly worse outcome, proving the pivot placement was load-bearing.

Problem 8.9 (the passerby's rite, under fire)**.** A Percolator client crashes (i) after prewriting one secondary, (ii) after prewriting all rows, (iii) after the primary commit cell-write, (iv) midway through secondary roll-forward. For each: what does a reader arriving at a locked secondary conclude from the primary, what does it do, and why can two concurrent passersby not resolve the transaction differently? Cite the single-row atomicity each step leans on.

Tier III — For the Ambitious (starred)

Problem 8.10 (★ the HLC bound)**.** For Def. 8.7: (a) prove the clock condition ($a \to b$ implies $\text{stamp}(a) < \text{stamp}(b)$ lexicographically); (b) prove pt never exceeds the largest wall reading yet issued anywhere, and $pt \geq$ the local wall reading, so $|pt - \text{wall}|$ is bounded by the clock error; (c) prove the counter bound: c at any event is at most the number of events inside one clock-error window of causally connected processes, and exhibit the adversarial schedule approaching it. *(Hints only.)*

Problem 8.11 (★ the founders' wedding)**.** Reconstruct Gray–Lamport Paxos Commit from its idea: one Paxos instance per participant's vote, commit iff every instance decides YES. (a) Specify the acceptors' role and how a new coordinator concludes an instance whose participant is dead. (b) Show AC1–AC3 hold and blocking is confined to minority-loss of acceptors. (c) Count messages and delays against Box 18 with a replicated coordinator, and state when the purists' edition is worth its postage. *(Hints only.)*

8.11 Hints

Hint (Problem 8.2)**.** (c) Let the coordinator answer "abort" from the presumption while its own patience timer still runs, then receive the last YES and force commit.

Hint (Problem 8.6)**.** Cut Exhibit 8.1 at (d): decision forced, announcements in flight. Deliver one, delay the other past thirty seconds. For the last part: the doppelgänger does not read clocks — any finite patience is silence in world one.

Hint (Problem 8.7)**.** The pre-commit shore commits by rule three; the uncertain shore aborts by rule two. Each rule's soundness argument quantifies over *all* participants; the partition breaks the quantifier, not the rule.

Hint (Problem 8.10)**.** (b) Induction on events: every assignment to pt is a max of wall readings and prior pts. (c) c resets whenever honest wall time overtakes; it can climb only while pt stands still, which the error bound confines to a window.

Hint (Problem 8.11)**.** (a) A new coordinator proposes NO in a dead participant's instance; Paxos's own safety arbitrates against a late YES. (b) AC1 reduces to each instance's consensus safety plus the deterministic conjunction.

8.12 Solutions

Tier I in full (tersely); Tier II in full — Problem 8.6 is the sabotage certification; hints only for Tier III.

Solution (to Problem 8.1)**.** (a) Nobody voted: participants abort from *working*; recovered coordinator finds no decision, presumes abort. No locks outlive the crash. (b) Tuppy in doubt (locks held, queries); Gussie aborts from working and answers any cooperative query NO-and-aborted, which releases Tuppy: cooperative termination at its best. (c) As (b) for both participants — votes received but no decision forced means recovery presumes abort; both participants eventually hear it. (d) The decision exists: recovered coordinator re-announces commit; both participants are in doubt until then — this is the blocking window with the shortest fuse. (e) Tuppy is done; Gussie in doubt; coordinator log answers commit on query or re-announcement; cooperative query to Tuppy also answers it.

Solution (to Problem 8.2)**.** (a) Committed: each participant forces vote and commit record; coordinator forces commit (and an end record when acks complete). Aborted: participants that voted force the vote and an abort record; the coordinator forces *nothing*. (b) All acks in hand means no participant will ever ask again; the presumption answers any archaeologist: no record, hence aborted — which is false for this committed transaction, and harmless, because the end record is written before forgetting and no querent remains. (c) With answers permitted before finality: query arrives (answered "abort" by presumption), last YES arrives, commit is forced, second querent hears "commit" — AC1 dead. Presumption is sound only about the past of a *final* log.

Solution (to Problem 8.3)**.** (a) Without: eleven votes forced, eleven decision applications. With: one forced vote (the writer), ten early exits; message rounds unchanged in count but the decision fan-out shrinks to one. (b) Only the writer is in doubt; the readers hold no locks and made no promises — the asymmetry is harmless because AC1 quantifies over deciders, and the readers' outcome is identical under either verdict by construction. (c) Only "I exited read-only; either outcome is consistent with my state" — it holds no evidence, which is exactly why its early exit was lawful.

Solution (to Problem 8.4)**.** (a) Centred: the interval runs from two milliseconds before ten o'clock to two milliseconds after; stamp ten o'clock plus two milliseconds; announcement no earlier than the true instant at which some call's earliest edge exceeds the stamp — at worst-case drift, roughly ten o'clock plus four milliseconds: about one ε after the call. (b) The second transaction's stamp is a latest edge taken after its start: strictly above ten o'clock plus four milliseconds, hence above the first stamp. (c) Commit latency grows to roughly eight milliseconds; every writer pays; readers at old stamps pay nothing. (d) The midpoint may lie in some clock's future; the latest edge, by the axiom, lies in nobody's past at call time and can be waited into everybody's past.

Solution (to Problem 8.5)**.** (a) Relocate; limbo shrinks to a Raft election plus outstanding in-doubt queries. (b) Shrink; limbo becomes the passerby's chore, bounded by encounter time. (c) Renegotiate (the machinery): no mid-flight votes exist; residual stall is the sequencer's own consensus. (d) Renegotiate (the promise): limbo is gone but isolation went with it; the orchestrator's death revives the funeral problem one layer up. (e) Relocate plus purchase: by-election limbo plus one uncertainty of commit-wait per transaction.

Solution (to Problem 8.6 — the certification)**.** The schedule, against Box 18. Participants Tuppy and Gussie, coordinator Bingo; the vendor's thirty-second fuse armed in both participants.

(1) Prepares delivered; both force YES; both fuses start.

(2) Bingo receives both votes, *forces commit* (Box 18 line 6): the decision exists in the court of record.

(3) Bingo emits the two commit letters and crashes. The letter to Gussie arrives at once: Gussie commits, releases locks, credits the aunt — permanently (AC3).

(4) The letter to Tuppy is delayed thirty-one seconds — lawful weather under the Chapter 1 charter. At thirty, Tuppy's fuse fires: he "heuristically aborts," rolls back the staged debit, releases locks, reports failure to Bertie.

(5) Bingo recovers, reads commit, re-announces. Tuppy has already acted; his abort is applied and, per AC3 on his own ledger, he will not now commit. Final state: credit committed at Gussie, debit aborted at Tuppy — ten pounds minted; the banks reconcile never.

The violated text: Box 18 line 12 — a prepared participant's only lawful acts are query and wait; the configuration substituted a decision. The dead clause: AC1 (two processes decided differently); AC2 is also complicit, since Tuppy's ABORT contradicts the unanimous-yes commit in the record. Why no timeout value repairs it: the schedule delays the letter to *fuse plus one second*, whatever the fuse; Thm. 8.2's worlds are indistinguishable at every finite patience, because silence carries no signature — a longer fuse changes the incident's date, not its existence. Certified fatal: money created, AC1 dead, under one crash and one slow letter, both documentary-ordinary.

Solution (to Problem 8.7)**.** The coordinator disseminates pre-commit to P_1, P_2, then crashes; a partition isolates $\{P_1, P_2\}$ from $\{P_3, P_4, P_5\}$ (prepared, no pre-commit). Left shore termination: all reachable in pre-commit, none committed $\Rightarrow$ complete the commit (rule three). Right shore: some reachable uncertain $\Rightarrow$ abort (rule two, whose soundness argued "no absent party can have committed" — true against crashes, false against absence-by-partition, since the absent left shore held pre-commits and is now committing). Heal the cable: AC1 dead. Each shore's rule quantified over "all participants" and silently substituted "all reachable." The quorum repair — neither shore moves without a majority — restores AC1 by making one shore wait, i.e., block: non-blocking was the entire purchase, so the repaired protocol is a parliament with an extra phase, and Part Five's settlement (drop the phase, keep the parliament) follows.

Solution (to Problem 8.8)**.** (a) Cancel-flight, cancel-hotel (backward, before the pivot); charge is the pivot (uncompensable by policy: refunds exist but the business declares the charge the commitment point); issue ticket and notify retriable (forward). (b) Budget report reads the charge after the pivot— no: place the report between charge and a failed ticket issue that forces manual retreat (the pivot's "retriable" promise broken by an operator override): the report now counts revenue the refund retracts — Chapter 7's vocabulary: a dirty read made durable by a third party; strictly, a read of state later compensated, which no isolation level that spans only single transactions even names. (c) The fence (charge marked provisional-until-ticketed) protects observers that honour the flag — the budget report can exclude provisional rows; it cannot protect the aunt's email or any external side effect already emitted. (d) Ticket before charge: a crash between them yields a ticketed, unpaid journey, and the "compensation" is now revoking a delivered external good — strictly worse than refunding money, which is why the uncompensable step is *defined* to be the pivot and everything after it must be retriable, never revocable.

Solution (to Problem 8.9)**.** (i) Primary lock present, no write record: erase primary lock atomically (binding abort), roll back the secondary; a racing passerby attempting the same finds the erase idempotent, and a late-waking client's commit attempt fails at the missing primary lock — the single-row exchange arbitrates. (ii) Same as (i): prewrites complete but no pronouncement; abort is lawful and, once the primary lock is erased, final. (iii) Write record present at primary: roll *forward* — inscribe the secondary's write record; concurrent passersby perform idempotent inscriptions; nobody may abort, since the primary lock no longer exists to erase. (iv) As (iii); the remaining secondaries are completed by whoever arrives. In every case the primary cell's single-row atomicity makes the pronouncement (or its annulment) a unique, linearizable event — Chapter 7's language earning its keep in a column.

8.13 The Reading

Corbett et al., "Spanner: Google's Globally-Distributed Database," OSDI 2012. The course's single most load-bearing industrial text. Read the TrueTime section twice — once for the API's honesty, once for commit-wait's arithmetic against Thm. 8.4 — and notice the season's earlier chapters on every page: Paxos groups, leases, snapshots, locks.

Garcia-Molina and Salem, "Sagas," SIGMOD 1987. Short, clear, decades ahead of the architecture that would need it. Read the compensation discussion with your own outage postmortems open beside it, and note how much of Prop. 8.5(ii) the authors already knew.

Gray, "Notes on Data Base Operating Systems," 1978. The ceremony and the in-doubt state in the founder's plain hand; the recovery discussion is Box 18's ancestor and still its best commentary.

Gray and Lamport, "Consensus on Transaction Commit," TODS 2006. The ledger and the parliament formally wed by their founders; Paxos Commit (Problem 8.11) with the message-count comparison done honestly.

The shelf. Skeen's three-phase paper (1981), as the museum label. Peng and Dabek, "Large-scale Incremental Processing Using Distributed Transactions and Notifications" (OSDI 2010) — Percolator; Box 20 against §2. Thomson, Diamond, Weng, Ren, Shao, Abadi, "Calvin" (SIGMOD 2012) — the determinist bargain. Kulkarni, Demirbas, Madeppa, Avva, Leone, "Logical Physical Clocks" (OPODIS 2014) — HLC and Problem 8.10's bounds. Bernstein, Hadzilacos, and Goodman, *Concurrency Control and Recovery in Database Systems* (1987), chapter seven — the recovery matrix and presumed abort/commit in full dress, free online, and the place to check Box 18's every line.

Chapter 9

Partitioning, or Where Things Live

Companion to Episode Nine. Friendly territory, as the constitution instructs me to flag it: after two chapters of impossibility theorems and fainting officiants, an interlude of good news and elementary arithmetic. The resignation letter that hash modulo N writes to itself; why shard at all, and the marriage of partitioning to replication; hash against range, and the compound-key treaty; the clock face, constructed and proved in full here — balance, movement, the one-dart lottery, and the virtual-node repair; the auction that needs no ring; Chord, and the road the industry actually took; the two churches of the shard map; rebalancing without storms; routing epochs, the fencing token's civilian cousin; and the survival kit for celebrity. The mathematics is coin-flipping throughout, and the sabotage is, as ever, certified fatal in the solutions.

9.1 Part One — Why Shard At All

The chapter opens on the day a cache fleet dies of arithmetic. The estate runs ten cache servers, and every key is assigned by the oldest rule in the book — hash the key, take the remainder modulo the fleet size, and the remainder names the machine. Key to server, in one line of code; it has worked, wordlessly, for two years. Traffic grows, as traffic does, and the operators, prudent souls, add an eleventh server at four in the afternoon, ahead of the evening peak. Consider what the arithmetic does at the moment the divisor changes from ten to eleven: the remainder of nearly every hash changes with it — not some remainders, but roughly ten in every eleven — and the entire address book of the estate is reshuffled in a single stroke. A cache whose address book has been reshuffled is a cache in name only: the evening's first request for nearly every key goes to a machine that has never heard of it, misses, and falls through to the database beneath, which was sized on the assumption that the cache would absorb nine parts in ten of the load. No machine failed; the *arithmetic* failed — at four in the afternoon the operators, meaning only to add capacity, sent the database the entire internet at once. The morning after has a fixed liturgy (the postmortems might share an author): the hit rate off a cliff at four, database latency rising like a startled pheasant at one minute past, queues backing into the application tier by ten past, and the on-call engineer restarting healthy machines one by one, because that is what one does at midnight when arithmetic is the culprit and arithmetic does not appear in the process list. **Hash modulo N is a resignation letter one writes to oneself in advance, postdated to the day the fleet changes size** — Problem 9.3 audits the letter's exact cost, a moved fraction of $N/(N{+}1)$ against the ring's $1/(N{+}1)$ — and every estate that has used it has eventually had the letter delivered. The repair had been published in advance, in 1997, by Karger and colleagues at MIT, under a name that gives away nothing — *consistent hashing* — with two promises that sounded almost too modest to matter: each server holds about its fair share of the keys, and a membership change moves about one key in n rather than nearly all of them. The authors founded Akamai on the idea, which quietly became a load-

bearing wall of the web; the technique escaped the cache trade into storage, into databases, into Chapter 6's Dynamo school; and the fix deployed at dawn is always the same fix: never again shall the address of a key depend on a number that changes.

Before any ring is drawn, though, the disciplined question: must we? The alternative to partitioning is the bigger machine, and it is a genuinely excellent answer for longer than fashion admits — a single well-provisioned server dispatches astonishing volumes, and every theorem of this course is trivially satisfied by an estate of one. But the ceiling is real, and it is triple. The ceiling of *physics and the catalogue*: past some size the machine you want is not sold, and the machine below it is priced as jewellery — cost per unit of capacity bends viciously upward at the top of the catalogue, which is why the estates that define this field were built from warehouses of commodity boxes rather than one cathedral mainframe; the theorems of this course are, among other things, the price of that procurement decision. The ceiling of *blast radius*: one machine is one failure domain, and Chapter 1's documentary evidence applies to it in full — the mainframe's uptime is magnificent until the firmware update, the flooded machine room, the fat-fingered migration, whereupon the business discovers that availability was never about the mean but about the worst Tuesday. And the ceiling of *geography*: a single machine is somewhere, and the speed of light bills every client who is elsewhere — Chapter 2's constant, unrepealed and unappealable. When the business outgrows any of the three, the data must be cut into *partitions* — shards, tablets, ranges, vnodes; the trade has a dozen words and one idea — each piece small enough to live on a machine, move between machines, and fail without taking the estate with it. And once the cut exists, the estate's grammar reorganizes around it: the shard becomes the **unit of everything** — of placement (the map speaks in shards, not keys), of movement (rebalancing ships shards whole, never grains), of recovery (a dead machine's obligations are enumerated as the shards it held, each re-homed independently, which is precisely what makes the fleet's recovery parallel where the single great machine's was serial), of throttling, of quota, of billing, of blame. Too large, and every one of those operations becomes a lumbering indivisible; too small, and the map itself becomes the estate's biggest table. The trade's instinct — shards of modest, movable size, thousands per fleet, many per machine — is the observation that every operational verb above conjugates more gracefully in the plural.

Here too a marriage is formalized, the couple having lived unwed for three chapters. Partitioning and replication are different axes of the same placement decision: replication (Chapter 6's business) makes *copies* for durability and availability — the same shard, several machines; partitioning cuts the data into *different* pieces for capacity — different shards, different machines. Every serious estate does both at once, and the result is the *seating chart*: a matrix of shards down one side and replicas across the top, each shard's replica group very commonly, after Chapters 4 and 5, a little parliament with its own log. Shard seven's parliament is Bingo, Tuppy, and Gussie, with Bingo on the throne; shard eight's is Tuppy, Gussie, and Barmy, with Gussie presiding; each machine hosts replicas of many shards, leads some, follows others — and the placement scheme's real output is this whole seating chart, including the quiet constraint that no shard's members share a rack, a switch, or a storm, because replicas sharing a failure domain are Chapter 1's lesson purchased twice. Keep the axes distinct in the design review: replication answers *what survives*; partitioning answers *what fits and where*; and this chapter's pathologies are all partitioning's own — no amount of replication rescues a scheme that put all the popular keys in one place. Chapter 8's parting debt names this chapter's obligation precisely: every protocol of Chapter 8 assumed somebody had already decided that Bertie's account lives at Tuppy's branch — and observe how much hung on the unexamined decision: co-locate the two accounts and the transfer is one shard's local transaction, no officiant, no vows, no theorem; separate them and the entire fainting-clergy apparatus deploys. The cheapest distributed transaction, that chapter's triage said, is the one your schema made unnecessary. This chapter, somebody decides.

One framing more, the whiteboard's economic heart, stated in advance so the whole chapter can hang evidence on it. **A partitioning scheme is the choice of which fan-out you pay.** Cut the data one way

and reads scatter across every shard while writes stay local; cut it another and writes fan out while reads stay put; rebalance eagerly and the movement itself becomes load. The same trilemma will appear in four costumes — hash against range, local against global indexes, eager against lazy balancing, pull against push routing — and in every costume the resolution is identical: there is no scheme in which reads, writes, and rebalances are all free; there is only the scheme whose expensive operation is the one the business performs least.

9.2 Part Two — Hash Versus Range

Two great schemes divide the world, and each is best introduced by the pathology of the other.

Partition by range. Order the keys — alphabetically, numerically, by time — and cut the ordered line into contiguous stretches. The virtue is *locality*: keys that are neighbours in the order are neighbours in the estate, so a scan — every account opened in March, every surname in one county — touches one shard or a few adjacent ones, not the whole fleet. The image is the encyclopaedia: volume one, A to Bertram; volume two, Bertram to Chatsworth; the spines labelled with their boundaries, so that finding an entry means reading the spines and reading a run of neighbours means pulling one volume, perhaps two. One under-praised virtue while the volumes are before us: the boundaries need not be uniform, and must not be — the letters are not equally populous, so the volumes are cut where the *content* dictates, thick stretches of alphabet in thin volumes and thin stretches in thick ones. Range partitioning inherits exactly that freedom, splitting where the data is dense — and the boundaries are therefore *state*: somebody keeps the list of spines, a fact that matters in Part Five. The characteristic pathology follows directly from the virtue; call it the **monotonic-key dogpile**: all writes to the last range. Let the key be a timestamp or an ever-increasing order number — the parcels ledger keyed by dispatch time is the canonical patient — and every new write lands at the end of the order, in the newest range, on one shard, always. The historical ranges sit in dignified retirement, visited by the occasional auditor; the newest range absorbs the entire living write load of the business, and does so on its busiest day of the year, because monotonic keys and seasonal peaks arrive together — the Christmas post, all of it, addressed by construction to one door. A hundred machines, one at work, ninety-nine spectators; when the range fills and splits, the load moves wholesale to the new last shard, tomorrow. Every time-series estate learns this within a week of launch, and the remedies — salting the key with a prefix, splitting the tail eagerly, writing to the day's shard round-robin — are all confessions that the pure order was unaffordable.

Partition by hash. Feed the key through a good hash and place by the hash value: the key's identity is scrambled into uniform anonymity, and neighbouring keys fly to opposite ends of the estate. The virtue is *uniformity*: a decent hash spreads any workload's keys evenly, monotonic or not; the dogpile dissolves, because consecutive order numbers land on unrelated machines. The price is exactly the virtue inverted: locality is destroyed, and the scan that touched one shard under range now touches every shard in the fleet — scatter the query, gather the results, pay the fan-out on every read that asks for a stretch. Hash is the scheme of point lookups — carts, sessions, profiles, caches, anything fetched by exact name — and Chapter 6's Dynamo school is hash-partitioned to a member, which is no accident: a cart is the point lookup made flesh. The catechism per table is mercifully short. Does the business ever ask for this data *in stretches*? Ledgers, timelines, audit trails, anything with a report: range, and manage the tail. Fetched only by exact name — carts, sessions, tokens? Hash, and never think about it again. Both, honestly both? The compound-key treaty below, or two tables and a pipeline, and no shame in either. And hash has its own celebrity problem, which no uniformity can cure: the hash spreads *keys*, not *popularity* — one key requested a million times an hour hashes to one place, whatever the scheme, and uniform placement of skewed love is still skewed load. The celebrity is Part Seven's business, and the chapter will keep the appointment.

Three footnotes of craft, each a scar. First, the hash itself: the office asks uniformity and stability, not secrecy — a fast non-cryptographic hash is the standard tenant, and the one genuinely non-negotiable property is that the function *never change*: a fleet that upgrades its hash library and discovers the new version disagrees with the old about where everything lives has re-sent itself the resignation letter, through the post of a dependency file. Pin the placement hash, forever; it is the one function in the estate with tenure. Second, the standing armistice: the **compound key** — hash the outer component, range the inner. Partition Bertie's shop by hashing the customer, then ordering by time within the customer: point lookups and per-customer scans both land on one shard, the monotonic dogpile dissolves because a thousand customers write to a thousand arcs, and only the estate-wide scan across all customers pays scatter-gather. Most industrial schemas are precisely this treaty, signed per table. Third, skew generally: uniformity claims are claims about *keys in expectation*, and businesses are not expectations — one tenant is a hundred times the median, one county holds half the population; the schemes bound what arithmetic can bound, the residue is Part Seven's celebrity, and pretending otherwise on the whiteboard merely schedules the discovery for production. The engineering summary: range buys scans and pays with dogpiles; hash buys uniformity and pays with scatter-gather; the compound key is the standing treaty; the mature stores ship all of it and make one choose per table — Part One's fan-out sentence in its first concrete costume. The remaining machinery — rings, directories, rebalancers — operates *beneath* either scheme: given the cut, which machine holds which piece, and how does that answer survive the fleet changing size? Which returns us to the resignation letter, and to the repair.

9.3 Part Three — The Clock Face

Now the set piece, built slowly — the construction is small enough to hold whole — and proved in full. Draw a circle — a clock face, marked not with hours but with every possible hash value, zero at the top, wrapping at the maximum back to zero: the hash space bent into a ring, normalized at the desk to the unit circle $[0, 1)$. Place the *servers* on the ring: hash each server's name and set the machine at the point its hash names — Bingo at one o'clock, say, Tuppy at five, Gussie at nine, scattered artlessly wherever the arithmetic drops them. The placement rule is one sentence: **hash the key onto the same ring, then walk clockwise; the first server you meet is home.** The walk-clockwise-to-the-first-doorstep rule is, formally, the *successor* function — the first server position at or after the key's point, modulo one — and each server owns the *arc* its position closes: its key tenancy. Every key has exactly one home, found with no table, no negotiation, no message; any party who knows the roster can compute every placement alone. The model beneath every probabilistic claim of the chapter, fixed before the first of them:

Model of this chapter

For the probabilistic results (Thms. 9.2–9.5, Prop. 9.4). Hash values are modelled as independent uniform random points on the unit circle $[0, 1)$ — the standard idealization of a good hash; keys and server names hash independently; the workload's keys are not chosen adversarially against the hash. Load is counted in keys (or, equivalently, in key-value mass under the same independence). All results are exact distributional statements under this model; the engineering caveats (skewed *popularity*, adversarial keys) are handled in the text where they arise — the model covers where keys *live*, not how often they are visited.

For routing and rebalancing (Prop. 9.6, Problems 9.7, 9.8). Chapter 1's asynchronous network; crash-stop servers; a shard map with a monotonically increasing version (epoch), administered either by a directory (Chapter 5 kernel, leases and watches) or by gossip. Clients may cache maps of any

staleness.

The definition gathers the construction, the aliases of the variance repair below, and the replication walk in one place.

Definition 9.1 (the ring; consistent hashing)**.** Fix independent uniform positions on the unit circle for m *tokens*; each of n servers owns k tokens (its *virtual nodes*, or *aliases*; $m = nk$), the single-alias scheme being $k = 1$. A key hashes to a uniform point and is placed at its *successor*: the first token clockwise from the key's point, wrapping at one. A server's *share* is the total length of the arcs its tokens close; by uniformity of keys, share equals expected fraction of keys held. Replication walks onward: a key's R replicas are the first R tokens belonging to distinct servers (industrially: distinct failure domains) met clockwise.

Walk three tenants to their lodgings, so the rule is in the hands and not merely the eye. The umbrella ledger hashes — the key's *name*, mind, nothing about its size or worth — to half past two: clockwise, first doorstep is Tuppy's at five. The aunt's flower-committee notice hashes to twenty past nine, just beyond Gussie: clockwise past ten, past eleven, wrap the top of the dial, and Bingo at one o'clock takes her in. Bertie's cart hashes to four: Tuppy again — the hash neither knows nor cares that Tuppy already has the ledger; uniformity is a statistical promise, not a courtesy call. Three keys, three walks, no conversation with anybody: that is the whole runtime of the scheme, and it is the same walk whether the estate holds three keys or three billion.

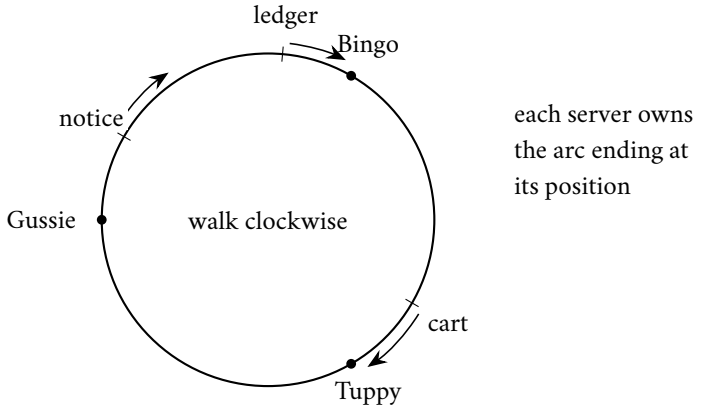


Exhibit 9.1 (the clock face). Servers and keys hash to the circle; a key walks clockwise (arrows) to its successor. Clockwise here runs toward decreasing angle; the ledger and the cart reach Bingo and Tuppy, the notice walks to Gussie's successor. No table, no negotiation: the roster is the map (Def. 9.1).

Now the wonder. Barmy joins the fleet: hash his name, and he lands at three o'clock, in the middle of Tuppy's arc. What changes? Precisely this: the keys between Bingo at one and Barmy at three — keys that formerly walked past three o'clock to reach Tuppy — now meet Barmy first. They, and *only* they, change address; the keys from three to five stay with Tuppy, and every other tenancy on the dial is untouched, unaware. Compare the modulo estate, where the divisor ticking upward reshuffled ten keys in eleven. The argument's spine is two sentences: symmetry places the arcs fairly in expectation — no position is special, so with n servers each arc's expected share is one over n of the circle — and the walk-clockwise rule confines every membership change to a single neighbourhood, for a key moves only if it lies in the one arc the newcomer bisects. **Add a server, move an expected** $1/(n{+}1)$**; remove a server, and only its tenants move:** $1/n$**.** That is the celebrated "one over n" — formally, the expected moved fraction on membership change, Theorem 9.3 below.

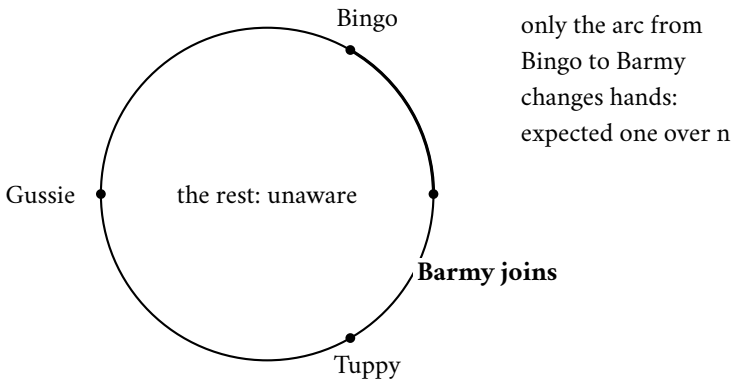


Exhibit 9.2 (one over n). Barmy lands at three o'clock; the keys between Bingo's position and his — the thickened arc, formerly walking on to Tuppy — now stop at Barmy. Nothing else moves (Thm. 9.3); compare modulo N, which reshuffles $n-1$ keys in n on the same event.

Stage the departure too, because operationally it is the more frequent ceremony and the more instructive one. Gussie's machine is retired — planned, a decommissioning rather than a death. His arc ran from Tuppy's five o'clock around to nine; every tenant in it walks clockwise past the vacated position to the next doorstep — Bingo's, once the dial wraps. Bingo inherits, in one motion, everything Gussie held; Tuppy and Barmy are not consulted and not disturbed; and the data movement is a single stream, throttled at leisure, since the retirement was planned. Let the same departure be unplanned — the storm, not the decommissioning — and the ring's marriage to replication pays off: Gussie's arc was never held by Gussie alone but by the walk's next distinct machines as replicas, so his death moves no data at all in the first instant — the successors already hold copies, the ring simply stops routing to a dead position, and the replacement copies are rebuilt in the background at the roadworks budget (Part Six). Placement schemes are judged at four in the afternoon, but they are *designed* for the storm at nine.

The proofs now, in full — friendly territory, as advertised: uniform points, symmetry, one union bound, and the Gamma trick, nothing sharper. The skeleton, in words: arc lengths of uniform points are exchangeable spacings, so a server's share is a Beta variable with mean one over n; a key changes home only if the newcomer lands between the key and its old successor, so exactly the newcomer's arcs move.

Lemma 9.1 (spacings)**.** *Let m independent uniform points partition the circle into m spacings $S_1, \dots, S_m$ (the spacing S_i closed by token i). The vector $(S_1, \dots, S_m)$ is exchangeable, and the sum of any k designated spacings has the Beta distribution with parameters $(k, m-k)$; in particular each S_i has mean $1/m$, and the designated sum has mean k/m and variance*

$$\frac{k\,(m-k)}{m^2\,(m+1)}.$$

Proof of Lemma 9.1. Rotate the circle so token one sits at the origin; rotation leaves the joint law of the other $m-1$ uniform points unchanged, so the spacings are the gaps of $m-1$ iid uniforms on $[0,1)$ plus the wrap gap. For exchangeability and the Beta law, use the standard Gamma representation: let $G_1, \dots, G_m$ be iid exponential variables and $T = G_1 + \cdots + G_m$; the vector $(G_1/T, \dots, G_m/T)$ has exactly the law of the spacings (both are uniform on the simplex — the flat Dirichlet — a classical fact provable by computing the joint density of normalized exponentials). Exchangeability is now manifest. For a designated index set K of size k,

$$\sum_{i\in K} S_i \;\overset{d}{=}\; \frac{\sum_{i\in K} G_i}{\sum_{i\in K} G_i + \sum_{i\notin K} G_i} = \frac{A}{A+B},$$

with A a Gamma variable with shape k, B an independent Gamma with shape $m-k$; and $A/(A+B)$ is, by definition of the Beta family, Beta$(k, m-k)$. The mean k/m and the displayed variance are the standard Beta moments. □

Theorem 9.2 (balance)**.** *Under Def. 9.1, each server's share has mean $1/n$ exactly. With k aliases the share is* Beta$(k,\ (n-1)k)$*-distributed, with standard deviation*

$$\frac{1}{n}\sqrt{\frac{n-1}{nk+1}}\ \approx\ \frac{1}{n\sqrt{k}},$$

so the relative deviation from the fair share falls as $1/\sqrt{k}$: a hundred aliases confine typical imbalance to about ten percent, ten thousand to one percent. Weighted servers take aliases in proportion to capacity and hold, in expectation, exactly their weighted share.

Proof of Theorem 9.2. A server's share is the sum of the k spacings closed by its tokens: by Lemma 9.1 with $m = nk$, it is Beta$(k, (n-1)k)$ with mean $k/m = 1/n$ and variance $k(n-1)k/(m^2(m+1)) = (n-1)/(n^2(nk+1))$; the standard deviation displayed in the statement is its square root, and for large n it is $\approx 1/(n\sqrt{k})$, giving relative deviation $\approx 1/\sqrt{k}$. A server with weight w (taking wk aliases among total m') has share Beta$(wk, m'-wk)$, mean wk/m': exactly its weighted proportion. The numerical clauses: $k = 100$ gives relative deviation about one tenth; $k = 10^4$ about one hundredth. □

Theorem 9.3 (movement; monotonicity)**.** *Under Def. 9.1: (i)* Monotonicity. *When a server joins, a key changes home only if its new successor is a token of the newcomer; no key moves between two incumbent servers. (ii)* Join. *The expected fraction of keys that move when a k-alias newcomer joins n incumbent servers is exactly $1/(n+1)$. (iii)* Leave. *When a server departs, exactly its keys move, each to the next surviving token clockwise; expected fraction $1/n$. With aliases, the departing tenancy is inherited by up to k distinct successors (Problem 9.8 quantifies the single-alias lump).*

Proof of Theorem 9.3. (i) A key's home is its clockwise successor. Adding tokens can only change a key's successor *to one of the new tokens*: if the old successor were replaced by another incumbent token, that token would have been the closer successor already — incumbent positions did not move. Hence all movement is into the newcomer, and no incumbent-to- incumbent movement occurs. (ii) The moved keys are exactly those whose successor is now a newcomer token, i.e. the keys in the newcomer's arcs; the expected total length of those arcs is the newcomer's expected share among $n+1$ servers, which is $1/(n+1)$ by Theorem 9.2. (iii) Removing a server's tokens changes successors only for keys in its arcs (mean share $1/n$); each such key's new successor is the next surviving token clockwise, by definition. With $k = 1$ that entire arc lands on one survivor; with aliases, each sliver lands on its own clockwise neighbour, generically k distinct servers (Problem 9.8). □

I said *expected* share, and an honest narrator leans on the adjective. Throw three darts at a circle and the arcs are rarely fair: someone draws a sliver, someone a third of the dial — with few servers the luck of the hash is the whole story, and the unluckiest machine in a modest fleet may hold several times its fair share. This is the *one-dart lottery* — formally, the variance of the single-alias arc, whose maximum runs to $\Theta(\log n/n)$: the fair share holds *in expectation*, the guarantee that holds *with high probability* carries a logarithmic factor of slack, and the difference between those two phrases is precisely the machine that pages you.

Proposition 9.4 (the one-dart lottery)**.** *With m tokens in all, the largest single arc exceeds $c\ln m/m$ with probability at most m^{1-c} (any $c > 1$); and the expected largest arc is of order $\ln m/m$ (both bounds proved below,*

the lower direction sketched). With one alias per server this is the whole story of ring imbalance: the unluckiest server expects roughly a $\ln n$ *multiple of the fair share — a lottery with logarithmic, not catastrophic, winners — and Thm. 9.2's aliases are the repair.*

Proof of Proposition 9.4. Upper bound. Fix $t \in (0,1)$. Token i's spacing exceeds t exactly when none of the other $m-1$ points lands in the arc of length t clockwise-preceding token i's position, an event of probability $(1-t)^{m-1}$. By the union bound,

$$\Pr\Big[\max_i S_i > t\Big] \le m(1-t)^{m-1} \le m\,e^{-t(m-1)}.$$

With $t = c\ln m/m$ this is at most $m \cdot m^{-c(m-1)/m} \le m^{1-c}\,e^{c\ln m/m}$, which is m^{1-c} up to a factor tending to one — negligible for any $c > 1$ and large m. *Lower direction (sketch, via the Gamma picture).* The spacings are normalized iid exponentials; the maximum of m iid exponentials has mean $H_m = 1 + \frac{1}{2} + \cdots + \frac{1}{m} \sim \ln m$, and the normalizer concentrates at m; hence the expected maximum spacing is $\sim \ln m/m$, and the upper bound is tight to its constant. (The desk records the sketch honestly: making the normalization rigorous is a page of routine concentration, assigned to nobody.) □

Worse than the steady-state lottery: when a single-alias server leaves, its entire arc lands on *one* successor — the neighbour inherits the whole estate of the departed, a lump of exactly the kind a fleet under stress cannot digest (Problem 9.8 quantifies the lump and its repair). The standing repair is the *aliases* — virtual nodes, k positions per server (Def. 9.1): each machine owns k slivers scattered around the whole dial rather than one contiguous arc, the law of large numbers averages the lucky slivers against the unlucky, and the relative deviation falls as $1/\sqrt{k}$ (Theorem 9.2: a hundred aliases confine typical imbalance to about ten percent). The lump dissolves too: a departing machine's hundred slivers fall to a hundred different clockwise successors, the inheritance spread across the entire surviving fleet, each survivor picking up a crumb. And heterogeneity comes free, which is why every industrial ring ships this way: the machine with twice the disk takes twice the aliases and owns, in expectation, exactly twice the estate. Chapter 6's Dynamo ran precisely this arrangement, and its preference lists reread accordingly: the home trio for a key is the first three distinct machines met on the clockwise walk — the ring placing, the walk replicating, both axes of Part One's marriage on one dial. The aliases are not free, in the interest of full disclosure: the roster's ring representation is k times longer, the successor bookkeeping k times busier, and a departing machine's tenancy — beautifully spread — arrives as k small transfers to coordinate rather than one; the alias count is a dial, and the trade turns it to the low hundreds and stops.

Protocol — Box 21. The ring, industrial dress (checked against Karger et al. 1997 and Dynamo 2007 §4)

1: **state:** sorted list of (position, server) for all nk tokens; the placement hash is pinned forever
2: **place**(x): binary-search the first token at or after hash(x), wrapping; that token's server is home
3: **replicas**(x): continue clockwise; take the first R tokens of *distinct servers, distinct domains*
4: **join**(s): insert s's k tokens; stream to s exactly the keys in its new arcs (from each arc's old owner); expected $1/(n{+}1)$ of the estate
5: **leave**(s): delete s's tokens; each sliver's keys stream to its clockwise successor (planned) or are rebuilt from replicas (unplanned)
6: **weighting:** alias count proportional to capacity

Movement is confined by Thm. 9.3; balance by Thm. 9.2; the sorted list is the entire map — computable by any party from the roster, which is the scheme's lightness and, per the two churches, its price.

One refinement of the modern era deserves its sentence, because the pure ring caps nobody — a machine simply receives what its arcs receive.

Remark 9.2 (bounded loads). The pure ring caps nobody. The refinement of Mirrokni, Thorup, and Zadimoghaddam (2017) fixes a capacity factor $c > 1$, caps every server at c times the mean load, and lets a key whose successor is full continue clockwise to the first uncapped server; the paper proves both balance (no server exceeds the cap, by construction) and a movement bound that survives the forwarding. Adopted by the load-balancing trade within the year; the citation is in the reading.

Before leaving the construction, the elegant sibling: the same two promises, no ring at all, and a proof one can do while pouring tea. To place a key, hash the *pair* — key with each server's name — obtaining one score per server, and the key lives with the *highest bidder*. That is the whole algorithm; the auction is, formally, rendezvous (highest-random-weight) hashing, published by Thaler and Ravishankar in the mid-nineties, a whisker before the ring and long overshadowed by it.

Definition 9.3 (rendezvous hashing; Thaler–Ravishankar)**.** For key x and servers $s_1, \dots, s_n$, compute scores $h(x, s_1), \dots, h(x, s_n)$ — modelled as independent uniforms, fresh per (key, server) pair — and place x at the highest-scoring server. Weighted variant: transform each score so that server s wins with probability proportional to its weight (the standard transform is recorded in the reading); no ring, no state beyond the roster.

Theorem 9.5 (rendezvous properties)**.** *Under Def. 9.3: (i)* Exact balance: *each server is the winner for each key with probability exactly* $1/n$*, independently across keys. (ii)* Join: *when a server joins, key x moves iff the newcomer's score beats the incumbents' — probability exactly* $1/(n+1)$ *— and every moved key moves to the newcomer. (iii)* Leave: *exactly the departed server's keys move, each to its runner-up. Balance here is per-key exact with no aliases; the price is n score evaluations per placement.*

Proof of Theorem 9.5. (i) For fixed key x the n scores are iid, hence exchangeable: each is the maximum with probability exactly $1/n$; independence across keys is independence of the score families. (ii) After the join there are $n+1$ iid scores; the newcomer holds the maximum with probability $1/(n+1)$, and precisely in that event does the key move — the incumbents' relative order is untouched, so a key not won by the newcomer keeps its old winner. (iii) Deleting a server deletes one score; keys for which it was not the maximum keep their winner; keys it won pass to the second-highest score, their runner-up. Every clause is per-key and exact; no expectation over arc geometry is involved. □

Protocol — Box 22. Rendezvous placement (checked against Thaler–Ravishankar)

1: **place**(x): for each server s in the roster compute $h(x, s)$; home is the argmax
2: **replicas**(x): the top R scorers (in distinct failure domains)
3: **join/leave:** no state to update — the roster *is* the state; movement per Thm. 9.5
4: **weighted:** transform scores so win probability is proportional to capacity (reading carries the formula)

Exact per-key balance, no aliases, no ring; n evaluations per placement. The scheme of choice where n is modest — balancers, cache clients, sidecar routers.

Set the two constructions side by side once, since the interview question is inevitable. The ring pays memory and setup — aliases, sorted positions, a binary search per lookup — and answers each placement in one hash; the auction pays arithmetic per placement — a hash per server — and holds no state at all beyond the roster. The ring's balance needs the alias repair; the auction's is exact from birth. Both move $1/(n+1)$ on a join — the ring in expectation over arc geometry, the auction per key, by symmetry of the lottery itself

— and both are, at the sizes each is deployed, effectively instantaneous, so the choice is temperament and fleet size, not performance theatre: the ring won the very largest fleets, rendezvous the small and medium ones — the balancer choosing among a dozen backends, the cache client among twenty peers, places where the arithmetic is nothing and the exactness and simplicity are worth everything. Two constructions, one moral, and it is the wonder beat stated plainly: **the resignation letter was never about hashing — it was about coupling every key's address to a number that changes. Decouple the addresses from the fleet size, by geometry or by auction, and growth becomes a local event.** Machines join; almost nothing notices. After eight chapters of theorems about what cannot be done, the course permits itself one of unqualified good news.

9.4 Part Four — The Apotheosis and the Road Not Taken

The ring, having conquered the datacentre's basements, also conquered the academy's imagination. In 2001, Stoica, Morris, Karger — the same Karger — Kaashoek, and Balakrishnan published **Chord**, which asked: suppose the fleet is not a fleet at all but a rabble — thousands of machines across the open internet, joining and leaving at will, no operator, no roster, nobody who knows the whole ring. Can a key still find its home? Chord's answer: yes, and in logarithmic time, by way of *fingers* — a routing table of successors at halving distances.

Remark 9.4 (Chord, for the ambitious). Chord (Stoica–Morris–Karger–Kaashoek–Balakrishnan 2001) keeps per node a successor pointer (correctness) and $O(\log m)$ *fingers* — the first node at distance at least $1/2$, $1/4$, $1/8, \ldots$ around the ring — (acceleration); greedy routing by largest non-overshooting finger reaches any key's home in $O(\log m)$ hops with high probability (Problem 9.10, starred). Inside the datacentre the premises invert — membership is a file, the map is megabytes, accountability is wanted — and the industry took the directory road.

The protocol's deeper cleverness is in how the ladder survives the rabble: nodes join and vanish continuously, so each node runs a standing *stabilization* conversation — verifying its successor, repairing its fingers, handing off keys — and the paper proves the ring heals faster than plausible churn tears it, with correctness resting on the successor pointers alone while the fingers are mere acceleration. A genuinely beautiful result: it launched a thousand papers, its fingerprints are on every distributed hash table the peer-to-peer era produced, and through them on file-sharing networks that carried, at their peak, a respectable fraction of the internet's entire traffic — theory with unusually wide distribution, even if the distribution was not always of theory.

And then the honest observation, delivered with the respect it is owed: **inside the datacentre, the industry mostly took the other road** — not because Chord is wrong, but because Chord answers a question the datacentre does not ask. Weigh the premises one by one, because the exercise is how one audits *any* imported design. Chord assumes membership is unknowable: the datacentre's membership is a file. Chord assumes no node may be special: the datacentre appoints special nodes for breakfast. Chord assumes hops are cheap relative to maintaining global state: in the datacentre the global state is megabytes and a hop is a microsecond spent twenty times over. Every premise inverted, the conclusion inverts with them. There is a deeper reason, which Chapter 5 equipped us to name: *someone must be accountable for the map.* When a shard is misplaced, when two nodes both claim an arc, when a rebalance goes wrong at three in the morning, the operator's first question is *who holds the truth about where things live* — and a protocol whose truth is smeared across every node's private fingers, converging eventually, answers that question poorly at exactly the wrong moment. The peer-to-peer world accepted the price for a reason — it had no operator and wanted none; openness, resistance to any central hand, was the entire point, and twenty hops across strangers' machines was cheap rent for it. The datacentre's premises are the inverse on every axis: machines

that trust one another, microsecond latencies, an operator with a pager, an institutional habit of courts of record — and, after Chapter 5, a consensus kernel sitting right there, already elected, already durable, with room on its docket for one more small item. There is also the plain arithmetic that closes the case: the data may be petabytes, but the map — shard boundaries to machine names — is a list of a few hundred thousand entries at the very worst, megabytes, a rounding error, the one component in the estate that never needed to be distributed for capacity. The two roads compress to a sentence apiece: *Chord distributes the map because it must; the datacentre centralizes the map because it may.* Distribute the data; keep the map close — a slogan worth engraving, since half the over-engineering this course has witnessed consists of distributing things whose entire contents fit in a pamphlet. The map moved in with the parliament, and the next part tours the household.

9.5 Part Five — Two Churches: The Directory and the Gossiped Ring

Where does the shard map live? The industry split into two churches on the question, and both congregations are thriving, which tells you the schism is about temperament as much as truth.

The directory church. Its cathedral is Bigtable (Google, 2006), and its doctrine: placement is *explicit state*, held in a *directory*, administered by a master, guarded by the consensus kernel. Bigtable cuts tables into contiguous ranges called tablets — range partitioning, for the scan-loving workloads it serves — and the map from ranges to servers is itself *data*: stored in a metadata table, itself split into tablets, whose addresses are held in a single root tablet, whose address is held in the little consensus service — Chubby, Chapter 5's lock service in its most famous cameo. The hierarchy, in three words — "root, metadata, landlord" — is precisely this walk: kernel to root tablet to metadata tablet to tablet server. Staged once with the company, since the walk is the church's whole liturgy: Bertie's application wants a row in the middle of the alphabet; it asks the kernel *where is the root*; the root tablet, at Bingo's, names the metadata tablet covering that stretch of the alphabet, at Tuppy's; Tuppy's metadata names the data tablet's landlord, Gussie; and Gussie serves the row. Three lookups on the cold path, all ferociously cacheable, so the steady state is one direct hop with the hierarchy consulted only on a miss. The tenancy itself is fenced with the full Chapter 5 ritual, because two landlords for one tablet is this church's doppelgänger nightmare: a tablet server holds its tenancy under a *lease* in the kernel; lose the lease — a pause, a partition, the classic false death — and the master may reassign the tablet, the lease machinery guaranteeing the old landlord's authority expires before the new one's begins. And the map's changes travel by subscription: watches registered with the kernel — Chapter 5's little notification service in its intended employment — push a tenancy change to whoever cached the old address rather than leaving it to be discovered at leisure. Attend to what the arrangement buys. Placement is now *policy*, not arithmetic: the master may move a tablet because a disk is filling, split a range because it runs hot, park a tenant's shards away from a noisy neighbour — decisions no hash can express, taken by a component whose job is to have opinions. The map is a court of record someone can read; and when the three-in-the-morning question arrives — *who holds the truth?* — the answer is a service with a name, a log, and a page in the runbook. The price is the church's standing anxiety: the directory is machinery on the critical path of every cold lookup, the master is a role that must itself be elected and fenced — Chapter 5's whole curriculum — and the hierarchy is a thing that can be misconfigured at scale. The church pays gladly, and its answer to the anxiety is the caching, the parliament beneath, and the operational fact that the machinery has now run the largest estates on earth for two decades.

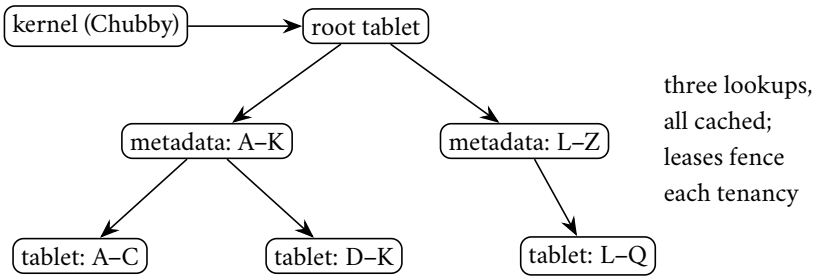


Exhibit 9.3 (the directory church). Bigtable's placement walk: kernel to root to metadata to landlord, every hop cacheable, every tenancy leased through the kernel. The map is data; its changes are pushed by watches; the court of record has a name and a log.

The gossiped-ring church. Its cathedral is Cassandra, heir to Dynamo's furniture: the clock face itself is the map — placement *is* the arithmetic — and what little membership state exists spreads by Chapter 6's gossip, every node learning the token positions of every other, no master anywhere, no directory to fail, any node able to answer any client and forward to the right tenant because every node can do the placement sum itself. The church has kept up with Part Three's craft: the early cathedrals assigned each machine one token and suffered the arc lottery accordingly (Prop. 9.4); the modern service runs virtual nodes as standard issue, each machine holding a bouquet of tokens, with the balance and inheritance benefits Theorem 9.2 computes. The virtues are Dynamo's virtues, inherited intact: no single component whose death stops placement; a symmetry that makes the fleet easy to reason about machine by machine; an estate that keeps accepting writes through weather that would have the directory church paging its bishops. The anxieties are the mirror image: placement-by-arithmetic cannot express policy — the ring puts the arc where the hash says, not where the operator wishes; hot arcs are repaired by moving tokens, which is gossip-coordinated surgery; and the truth about membership is *eventually consistent*, a phrase whose costume Chapter 7 taught you to check — during the disagreement window two nodes can both believe they own an arc, writes accepted under both beliefs, and the nets beneath — read repair, anti-entropy, hinted handoff, the whole Chapter 6 janitorial staff — must reconcile the tenancies afterward. The church's honest answer is that it *planned* for this: the same machinery that reconciles replica disagreement reconciles placement disagreement, one broom for both messes — an economy of mechanism the directory church cannot claim, purchased at the price Chapter 6 itemized.

Two churches, one schism: **explicit map with an accountable owner, versus arithmetic map with no owner to lose.** The course, pressed for its own confession, observes only this: the industry's newest cathedrals — the Spanners, the sharded parliament estates of Chapter 8 — have overwhelmingly built directory-style, on the grounds that the consensus kernel was already paid for, and that at three in the morning, accountability compounds better than symmetry. But the ring church's insight was real and remains banked: wherever the fleet is caches or balancers — where a stale map costs a miss rather than a lost write — the arithmetic map is lighter, and the miss is cheap. Match the church to the price of being briefly wrong, sir; that sentence retires more architecture debates than any benchmark. Carry also the three review questions that fall out of it, applicable to any storefront: *where is the map, and who may write it?* — a service with a log, or every node's gossip state; *what happens when the map is wrong?* — a redirect with a correction, a miss, or a lost write, in descending order of civilization; and *who notices first, the system or the customer?* — the directory church can alarm on its own court of record, where the arithmetic church discovers disagreements when the nets reconcile them. No church answers all three perfectly; a candidate storefront that cannot answer them at all is not a church, merely a hope with a logo. And in fairness to the smallest estates, there is a third pew, unfashionable and frequently correct: the map as a *configuration file* — twelve shards, twelve addresses, checked into version control, changed by pull request. It is a directory

whose parliament is the code review; it scales to precisely the point where placement changes outpace the patience of humans; and until that point it possesses the finest observability property in this entire chapter, which is that one can *read* it. The course mentions it because architecture reviews routinely propose a gossip mesh for what is, on inspection, nine machines and a spreadsheet.

9.6 Part Six — Rebalancing, the Operational Crucible

The map, however held, must sometimes change — a shard outgrows its machine, a machine joins, a hot range needs splitting — and *rebalancing* is where partitioning schemes go to be judged, because moving data is itself load, served by the same disks and cables as the customers. The craft has three commandments, each purchased with an outage somewhere in the trade's collective memory.

First: move on purpose, not by arithmetic. The resignation letter was the extreme case — a scheme in which growth *implies* mass movement — but milder versions lurk everywhere: the fixed rule that splits any range at a size threshold, the eager balancer that chases perfect uniformity. The mature posture treats every movement as a *decision* with a budget: so many megabytes per second of rebalance traffic, throttled beneath customer traffic, preemptible the moment latency climbs. Do the drain arithmetic aloud in the design review, because nobody else will: a machine holding a few terabytes, drained at a throttle polite enough not to disturb the customers, takes *hours* — which means the fleet's true capacity question is never "can we add a machine" but "how many machines can we be mid-draining at once while surviving the next failure," and the answer is the real, honest headroom of the estate. Shard size falls out of the same sum: shards of tens of gigabytes move in minutes and spread a dead machine's inheritance across the fleet; shards of terabytes move in shifts and land like pianos. A rebalance is roadworks: necessary, scheduled, signposted, budgeted in advance, and never conducted across every lane at rush hour.

Second: fear the storm. A hazard the season formalizes in due course, foreshadowed now because rebalancing is its favourite trigger; staged with the company so the shape lodges. Four in the afternoon — the hour has form. Gussie's machine slows: a failing disk retries its reads, not dead, merely laboured — the doppelgänger's oldest trick, crashed-versus-slow, in overalls. The balancer, a dutiful process watching health checks, sees Gussie miss three in a row, declares him unhealthy, and begins conscientiously moving his forty shards to the survivors. The moving is copying; copying is disk and network; Tuppy, receiving eleven shards while serving his own customers, slows — and misses three health checks. The balancer, dutiful, begins moving *Tuppy's* shards; Barmy receives the double burden and follows. Within the half hour the fleet's bandwidth is consumed entirely by furniture removals, every van unloading onto a floor that is being carried out the back, customer traffic starving in the corridors — a **rebalancing storm**, the estate consumed not by load but by its response to load. Mark the signature: remove the original cause — replace Gussie's disk — and the storm *continues*; it is self-sustaining now, each move causing the slowness causing the next move. The medicine became the disease and then became the epidemic; the *storm* is, formally, a rebalancing feedback loop, self-sustaining after its cause is repaired. The remedies are dampers on the loop: *hysteresis*, so a node must be unhealthy for minutes, not moments, before its tenancy moves — the balancer, of all components, must not believe the doppelgänger's first performance; *rate limits* that cap total concurrent movement whatever the apparent need, so the fleet can never spend more than a budgeted fraction of itself on removals; a *global view* before local action — a balancer that can see many nodes degrading at once should conclude "storm" and stand down, where one that sees only its patient concludes "move faster"; and a *panic brake* — the operators' ability to freeze all rebalancing with one switch, which every mature system ships and every postmortem in this genre wishes had been pulled earlier. The full theory of such self-sustaining overloads — metastability, the outage that persists after its cause is gone — is Chapter 11's centrepiece; here, note only that the balancer is a control loop, and control loops without damping oscillate.

Third: split before you must. The monotonic dogpile taught that the newest range burns; the general lesson is that splitting a shard *under* load is surgery on a struggling patient — the split itself costs reads, writes, and a map update, all charged to the busiest machine in the fleet at the hour it can least afford them. The craft pre-splits: a table expected to be large is born as many shards, boundaries guessed from the key distribution and corrected later at leisure; a range observed to be warming is divided while still comfortable, the boundary chosen where the access histogram actually peaks rather than at the arithmetic midpoint — split at the celebrity's doorstep, not halfway down the street. The balance of evidence — how warm, how fast, which boundary — is the directory church's home fixture, since a master with a map and a load report can *decide*, where a ring must wait for its arithmetic to be overruled by hand.

One estate-wide fan-out question belongs in this part, because rebalancing exposes it: **secondary indexes.** The data is partitioned by primary key; the business insists on querying by something else — colour, county, price. Two designs, and the fan-out lands on opposite sides; take Bertie's shop, partitioned by item number, and the customer who insists on browsing by colour. The *local index*: each shard indexes its own tenants — Bingo's shard knows which of Bingo's items are red, Tuppy's knows Tuppy's — so a write updates data and index together, on one machine, in one local transaction, cheaply; and the browse for red umbrellas must ask *every* shard, since red items live everywhere the hash scattered them: scatter-gather on every colour query, the fleet interrogated to answer one shopper. The *global index*: the index is itself a partitioned table, partitioned by colour — red's entries on one shard, blue's on another — so the browse goes to one place and reads the whole answer, cheaply; and every write fans out: the new red umbrella updates the item's data shard *and* the red index shard, two machines, coupled either distributed-transactionally — Chapter 8 entire, invoiced per write — or asynchronously, in which case the index lags the truth and the shopper occasionally clicks a red umbrella that has just sold out: Chapter 7's staleness wearing shop clothes. There is no third design; there is only the choice of victim. Local indexes tax reads; global indexes tax writes — Part One's sentence again, wearing an index's clothes — and the mature stores again ship both and make one declare, per index, which side of the ledger the business can afford.

9.7 Part Seven — Routing, and the Survival Kit for Celebrity

Two practical matters close the tour, and the second is the one every operator marks with dread.

Routing. The map exists; who consults it? Three arrangements, in ascending order of client sophistication, and the tell for each is who gets paged when it misbehaves. The *dumb client* sends any request to any node — or to a load balancer that does — and the receiving node, consulting map or arithmetic, forwards to the right tenant and relays the reply: one extra hop, zero client knowledge, upgrades that never involve the application teams; Cassandra's coordinator-any-node is the type specimen, and when it misbehaves the database operators are paged, which is where the expertise lives. The *proxy tier* centralizes the map in a routing layer between clients and fleet: clients stay dumb, the proxies stay current, the operators get one throat to throttle and one place to observe every query in the estate — at the price of another tier to run, scale, and page, plus a hop nothing removes. The *smart client* holds the map itself — fetched at startup, refreshed on change, often by subscription to the directory so updates are pushed rather than polled — and goes straight to the tenant: zero extra hops, maximum efficiency, the choice of every latency-obsessed estate, and a new failure mode of the first importance, distributed now into every application binary in the company: the client's map can be *stale*. The mature protocols therefore version the map — an epoch, a generation number, stamped on every request — so a tenant receiving a request for an arc it no longer owns answers not with silence or, catastrophically, with obsolete service, but with a correction: wrong address, here is the new epoch, ask again. The shape is Chapter 5's: the routing epoch is the fencing token's civilian cousin. The general law is worth one clean sentence, because it recurs in every system with a map: **a stale**

map is harmless only if the estate refuses to honour it — the staleness must be caught at the *server*, by epoch check, because the client, by definition of staleness, does not know to catch it. Any design in which an old map is silently honoured has decided that placement is advisory; and placement, this chapter has argued at length, is law.

Definition 9.5 (the routing epoch)**.** The shard map carries a version e, increased on every ownership change. A client stamps each request with the epoch of the map it used; a server knows, for each arc it owns, the epoch at which its tenancy began, and the current epoch it has heard. Handover discipline (directory church: leases, Chapter 5): the old owner's tenancy is terminated — lease expired or revoked, no further writes accepted — strictly before the new owner's tenancy begins.

The safety claim, in words: server-side version checks plus fenced handover make every applied write land at the current owner; the sabotage deletes the check and loses a write.

Proposition 9.6 (epoch safety)**.** *Under Def. 9.5, if every server rejects any request for a key it does not currently own (answering with a correction carrying the current epoch), then every applied write is applied by the key's owner at application time, and a client with a stale map suffers at worst an extra round trip. If instead some server honours requests for arcs it has ceded (no check, or tenancy not fenced), there is an execution in which an acknowledged write is never visible to any subsequent read — the lost write of Problem 9.7, exhibited in the solutions.*

Proof of Proposition 9.6. Safety with checks. Consider any applied write. The applying server owned the key at application time (the check is part of application, and tenancy, under the handover discipline, is exclusive: the old owner's authority ends before the new one's begins — Chapter 5's lease argument verbatim). A stale-mapped client's request reaches a non-owner, is rejected with the current epoch, and the client retries against the corrected map: one extra round trip, no lost writes. *The lost write without checks.* Let the arc move from server A (epoch three) to server B (epoch four); client C holds the epoch-three map. C writes key x to A; A, unfenced and unchecking, applies and acknowledges; all subsequent readers consult the current map and read x at B, which never saw the write. The acknowledged update is visible to no future read — the execution of Problem 9.7, drawn as Exhibit 9.4 in the solutions. □

Protocol — Box 23. The routing epoch (distilled from Bigtable §5, Dynamo §4.8, and Chapter 5's fencing)

1: **map:** versioned by epoch e, bumped on every ownership change; held by directory (with watches) or gossip
2: **client:** caches (map, e); stamps every request with e; on correction, refreshes and retries
3: **server, on request for key x:** if it does not currently own x's arc: **reject** with (current e, new owner hint) — never serve, never apply
4: **handover:** old owner's tenancy terminates (lease expiry/revocation) strictly before new owner begins; writes in flight at the boundary are re-driven by clients against the correction

A stale map is harmless only if the estate refuses to honour it; the refusal must live at the server, for the client, by definition of staleness, does not know to refuse (Prop. 9.6).

This chapter's sabotage — smart clients that cache the ring "for performance" and skip the version check — ends, as sabotages here traditionally end, with a write applied at an address that had already been condemned and a customer's update lost in the demolition: Problem 9.7, certified fatal in the solutions, with the trace drawn as Exhibit 9.4.

The celebrity. Now the row every scheme dreads, because it defeats the premise beneath all of this chapter's mathematics: that load, like keys, is divisible. One key — the viral post, the flash-sale item, the national election result — draws a fleet's worth of traffic to a single address, and no placement scheme can divide what is, by construction, indivisible: the ring puts the key on one arc, the directory in one range, and that machine becomes the estate's brightest ember. Staged once, for the record: the aunt's flower-committee notice — some triumph of the parish, photographed becomingly — is taken up by the national press at teatime, and by six o'clock her single row is receiving more traffic than the rest of the estate combined. The hash placed her fairly; fairness was never the issue; the issue is that the world's affection is not uniformly distributed and never will be. The survival kit, in escalating order of surrender. *Cache it*: celebrity is read-heavy in the overwhelming case, and a small cache in front — even a few seconds of staleness — absorbs orders of magnitude; the aunt's admirers do not need the flower count fresher than the kettle boils. But caching under celebrity has its own famous failure, the *stampede*: at seven o'clock the cached copy's lease expires, and ten thousand concurrent admirers miss *together* and descend on Gussie's shard as one — the thundering herd, an outage manufactured entirely by an expiry timestamp. The discipline is standard and cheap: let one request per key refresh while the rest wait a beat or are served the slightly stale copy — request coalescing, stale-while-revalidate; the trade has several names for the one idea, which is that a cache miss should be an event, not a demographic. The full pricing of stampedes is lodged with debt #22. *Split the reads*: replicate the celebrity's shard wider than the standing factor and spread reads across the copies — replication, for once, rescuing partitioning, since reads divide even when keys do not; the follower reads carry Chapter 6's staleness fine print, which for a flower count is no fine print at all, and the widening can be automatic — a shard whose read rate crosses a threshold sprouts extra followers for the afternoon and sheds them at dusk, fame handled as weather. *Salt the writes*: when the celebrity is write-hot — the live counter, the auction of the century — split the key itself: a hundred sub-counters under a hundred salted names, each salted name hashing to its own shard, every write choosing a salt at random, every read summing the hundred rows — a scatter-gather paid only by readers of that one key. The arithmetic is worth doing in full: a hundred thousand increments a second against one row is one shard's funeral; against a hundred salted rows it is a thousand a second apiece, which is a shrug. The construction is Chapter 7's grow-only counter wearing a wig — dividing the indivisible by making the merge lawful, commutativity forgiving what placement cannot fix. And, at the limit, *special-case it*: the greatest estates simply detect their celebrities and hand them to bespoke machinery — a dedicated fleet for the day's viral keys, a broadcast tier that pushes the election result outward rather than letting the world pull it — because past some fame, generality is the wrong tool and the honest architecture says so aloud; the detection itself is a small standing census of the hottest keys, cheap to run and worth its weight in postmortems unwritten. What does not work is denial: the load report always finds the celebrity eventually; the only question is whether the architect finds it first — and the estates that answer "we have no hot keys" divide, on inspection, into those with the census and those with the surprise.

9.8 The Whiteboard

For the design review — economics throughout.

1. Partitioning is the choice of which fan-out you pay — reads (hash, local indexes, scatter-gather), writes (range under monotony, global indexes), or rebalances (modulo N, eager balancers). No scheme escapes all three; match the expensive one to the operation the business performs least, and write the choice down where the next architect can find it — an undocumented fan-out decision is a trap set for one's successor.

2. Hash modulo N is a resignation letter, postdated to the day the fleet changes size. Decouple addresses

from fleet size — the ring with virtual nodes, or the rendezvous auction — and membership change becomes a local event: one over n moves, the rest of the estate unaware. And, as item two and a half for the schema page specifically: co-location is transaction avoidance — keys that commit together should live together, the compound key is the standing treaty between hash and range, and the placement hash has tenure.

3. Every shard map needs an owner, and every rebalance a budget. Directory or ring, someone answers "where does this key live, and who says so" at three in the morning — prefer the answer with a name and a log. Movement is roadworks: throttled, scheduled, preemptible, with hysteresis on the triggers and a panic brake the operators have rehearsed. The balancer is a control loop; undamped control loops are Chapter 11's opening exhibit.

4. The map is state: version every routing table, stamp every request, and let wrong-address answers carry the correction — the routing epoch is the fencing token's cousin, and a client that skips the version check is this chapter's sabotage. For the celebrity: keep the census of hot keys running, cache with stampede discipline, widen the replicas, salt the writes, and past some fame, special-case with dignity — the load report will find the celebrity regardless; arrange to have found it first.

The register. Debt #20 — where things live is a decision, issued only last chapter — is PAID: placement decided, by arithmetic on a dial or a directory under a parliament, the trade-offs priced and the churches named. ISSUED: #21 — everything here partitioned data *at rest*, and the same question stalks computation: where does the *work* live (payable Chapter 10, where the log takes its starring role); #22 — stampedes, storms, and the other self-sustaining weather glimpsed in the balancer's spiral and the cache's herd (payable Chapter 11). For the first time this season the paid column outnumbers the open by better than two to one. Episode word count: 10,023 — above the floor; the corrective's warm-expansion drill required two passes this session (drafts continue to arrive short; see LEDGER §5).

9.9 Problems

Tier I — Calisthenics

Problem 9.1 (placement drills)**.** On Exhibit 9.1's ring (Bingo at one o'clock, Tuppy at five, Gussie at nine, positions read as clock hours): (a) place keys hashing to two, four, six, eight, ten, and twelve o'clock. (b) Barmy joins at three (Exhibit 9.2): which of your six keys move, and to whom? (c) Gussie retires: which keys move, and to whom? (d) With replication factor two (next distinct server clockwise), list each key's replica pair before and after Barmy's arrival, and confirm no pair changed except where Barmy's arc is involved.

Problem 9.2 (the lottery, quantified)**.** Ten servers. Using Theorem 9.2: (a) compute the relative standard deviation of a server's share for one alias, one hundred aliases, and ten thousand aliases. (b) Using Prop. 9.4, bound the probability that some single-alias arc exceeds five times the fair share. (c) A vendor proposes one alias per server "for simplicity" on a twelve-server fleet; write the two-sentence objection, with numbers.

Problem 9.3 (the resignation letter, audited)**.** Under hash modulo N: (a) show the expected fraction of keys whose home changes when N grows to $N{+}1$ is exactly $N/(N{+}1)$ (hint: a key stays iff its hash's residues agree, which for a uniform hash has probability about $1/(N{+}1)$ — make the "about" precise). (b) Compare with Thm. 9.3's $1/(N{+}1)$ and state the ratio. (c) Name one deployment where modulo is nevertheless defensible, and the exact property that excuses it.

Problem 9.4 (name the fan-out)**.** For each design, name which fan-out pays (reads, writes, or rebalances) and the chapter's remedy: (a) local secondary indexes; (b) global secondary indexes, asynchronous; (c) range

partitioning keyed by timestamp; (d) hash partitioning queried by ranges; (e) an eager balancer chasing perfect uniformity.

Problem 9.5 (the auction, run by hand)**.** Three servers; scores for four keys given as a table of your own construction with distinct values. (a) Place the keys. (b) Add a fourth server with fresh scores; verify only keys it wins move. (c) Remove the original winner of one key; verify the runner-up inherits. (d) State the exact join-movement probability and check your table's empirical fraction against it in one sentence.

Tier II — The Real Thing

Problem 9.6 (the split policy)**.** Design a split policy for a range-partitioned parcels ledger keyed by dispatch time. Requirements: bound any shard's write rate to a target; survive the Christmas peak; keep historical shards cold and mergeable. Specify: when to split (trigger and measurement window), where (boundary choice against the access histogram), what pre-splitting the calendar justifies, and the salting fallback if a single hour still exceeds one shard's capacity. Defend each choice in a sentence, and state which church (directory or arithmetic) your policy presumes and why.

Problem 9.7 (sabotage: the stale ring)**.** A smart-client fleet caches the ring at startup "for performance" and omits the epoch stamp; servers, trusting the clients, apply any request addressed to them. Exhibit the lost-write execution of Prop. 9.6: the membership change, the stale client, the acknowledged write, and the proof that no future read at any replica returns it. Then state the two independent repairs (server-side epoch check; fenced handover) and show each alone closes your execution.

Problem 9.8 (the inheritance lump)**.** Single-alias ring, n servers. Server s departs unplanned. (a) Show the successor's expected share doubles: compute the distribution of the merged arc (use Lemma 9.1 with k=2). (b) With k aliases, show the inheritance splits across up to k successors and compute one inheritor's expected increment. (c) Combine with Prop. 9.4 to bound the probability the merged single-alias arc exceeds four times fair share. (d) One sentence: why this, and not steady-state balance, is the strongest argument for aliases.

Problem 9.9 (epoch liveness)**.** Under Box 23: (a) prove that a client whose every request is rejected-with-correction terminates (reaches the true owner) provided ownership changes finitely often during its attempt, and bound its retries by the number of changes. (b) Exhibit the starvation execution when ownership oscillates forever (the storm's routing shadow), and propose the damper.

Tier III — For the Ambitious (starred)

Problem 9.10 (★ Chord's bound)**.** With m nodes at uniform positions and correct finger tables (Rem. 9.4): (a) show each greedy hop at least halves the clockwise distance to the target; (b) conclude $O(\log m)$ hops to reach the predecessor of any key with certainty, and $O(\log m)$ with high probability to finish; (c) bound the routing state per node and the expected stabilization work per membership change. *(Hints only.)*

Problem 9.11 (★ the lottery's tail, both ways)**.** Prove the lower-bound direction of Prop. 9.4 rigorously: with m uniform tokens, the expected maximum spacing is at least $c' \ln m/m$ for some constant $c' > 0$; conclude the single-alias worst server holds $\Theta(\log m)$ times fair share in expectation. *(Hints only.)*

9.10 Hints

Hint (Problem 9.3)**.** (a) Count residue collisions of a uniform integer hash over a large range; the discrepancy from exact uniformity is the "about." (c) A fleet whose size never changes — and say why that property is rarer than its owners believe.

Hint (Problem 9.8)**.** (a) The merged arc is the sum of two designated spacings. (c) Union-bound the two spacings separately, or apply the Beta tail directly.

Hint (Problem 9.10)**.** (a) If the target is more than halfway, some finger jumps at least half; if less, induction on the remaining interval. (b) Distances shrink geometrically; after $\log_2 m$ halvings the interval holds, in expectation, one node.

Hint (Problem 9.11)**.** Exponential spacings: the maximum of m iid exponentials exceeds $\ln m - \ln\ln m$ with probability tending to one; carry the normalizer's concentration (law of large numbers) through the ratio.

9.11 Solutions

Tier I in full (tersely); Tier II in full — Problem 9.7 is the sabotage certification; hints only for Tier III.

Solution (to Problem 9.1)**.** (a) Reading the walk toward the next server position: two o'clock → Tuppy (five); four → Tuppy; six → Gussie (nine); eight → Gussie; ten → Bingo (one, wrapping); twelve → Bingo. (b) Barmy at three captures the arc (one, three]: the key at two moves to Barmy; the key at twelve stays with Bingo (twelve walks past the wrap to Bingo at one — unmoved); all others unmoved. (c) Gussie's tenants (six, eight) walk on to the next survivor clockwise — with Barmy present, that is Bingo at one (wrapping past nine): both move to Bingo; nothing else stirs. (d) Before: two/four → (Tuppy, Gussie); six/ eight → (Gussie, Bingo); ten/twelve → (Bingo, Tuppy). After Barmy: two → (Barmy, Tuppy); four → (Tuppy, Gussie) unchanged; ten/twelve → (Bingo, Barmy) — the second replica changed because Barmy is now Bingo's clockwise neighbour; six/eight unchanged. Only pairs meeting Barmy's position changed, as monotonicity demands.

Solution (to Problem 9.2)**.** (a) Relative deviation $\sqrt{(n-1)/(nk+1)}$: k=1: $\sqrt{9/11} \approx 0.9$ — share swings of order its own size; k=100: $\sqrt{9/1001} \approx 0.095$ — about ten percent; k=10^4: about one percent. (b) Fair share is one tenth; five times is one half: $\Pr[\max > 1/2] \le 10 \cdot (1/2)^9 \approx 0.02$. (c) "With one alias each, the typical spread between your luckiest and unluckiest machine is a factor of several, and the expected worst arc is roughly two and a half times fair share ($\ln 12 \approx 2.5$); a hundred aliases cost kilobytes and confine imbalance to ten percent."

Solution (to Problem 9.3)**.** (a) For a hash uniform on a range vastly exceeding $N(N{+}1)$, residues mod N and mod $N{+}1$ are jointly uniform on the $N(N{+}1)$ pairs up to negligible edge error; the key stays iff the pair maps both residues to the same server, which happens for exactly one new-residue value per old, probability $1/(N{+}1)$ up to that error. Moved fraction: $N/(N{+}1)$. (b) The ratio to the ring's $1/(N{+}1)$ is N: the letter costs N times the neighbourhood. (c) A statically sized fleet — compile-time sharding of a fixed scoreboard, say — where the divisor is a constant of the architecture; the excusing property is not smallness but *provable constancy*, and fleets believed constant have a documented habit of growing on the eve of a peak.

Solution (to Problem 9.4)**.** (a) Reads pay (scatter-gather per query); remedy: accept, or promote hot query axes to global indexes. (b) Writes pay (fan-out, plus staleness of the async lag); remedy: budget the lag, monitor it, and mark provisional reads. (c) Rebalances and the tail shard pay (the dogpile); remedy: compound

key or salting, pre-split the tail. (d) Reads pay (every range scan is estate-wide); remedy: compound keys, or a range-partitioned projection maintained as a pipeline. (e) Rebalances pay (movement as standing load, storm risk); remedy: budgets, hysteresis, and a definition of "balanced enough."

Solution (to Problem 9.5)**.** Any table with distinct scores works; the checks are mechanical. (d) Exactly $1/(n{+}1) = 1/4$ per key; with four keys the empirical fraction is a multiple of one quarter, and agreement with expectation is approximate by design — four coin flips do not an expectation make, which is rather the point of stating exact per-key probabilities.

Solution (to Problem 9.6)**.** Sketch of the defensible design. Trigger: sustained write rate over a window of minutes (hysteresis against bursts) crossing a fraction — half, say — of a shard's measured capacity; splitting at half leaves headroom to perform the split off the peak. Boundary: at the access histogram's current watermark — for a monotonic key, near the present moment, so the cold past stays whole and the hot tail is freshly capacious; never the arithmetic midpoint, which for monotonic keys splits history, not load. Pre-splitting: the calendar knows the peak; create the season's expected tail shards in advance, boundaries at forecast hourly volumes, so Christmas arrives to find the doors already open. Fallback: if one hour still exceeds one shard, the key ceases to be purely temporal — salt the tail (hour, salt) across a small fan of shards and merge on read; this is the counter split, Chapter 7's algebra, applied to placement. Church: directory — every clause above is a *decision* against load reports, which is the directory's home fixture; a pure arithmetic ring can express none of it without a human moving tokens.

Solution (to Problem 9.7 — the certification)**.** The execution, against Box 23 with lines three and four deleted. Epoch three: arc α (containing key x) owned by server A; client C caches the epoch-three ring.

(1) Rebalance: α moves $A \to B$; epoch four published. A, un-fenced, retains its copy and its willingness; C, un-notified (smart client, no watch, no stamp), retains epoch three.

(2) C writes $x{=}v$ addressed to A. A applies and acknowledges. (A owns nothing; nobody checks.)

(3) All current maps say B. Every subsequent reader — including C itself after any refresh — consults epoch four and reads x at B: the value is the old one; v is on a machine no route reaches.

(4) Optional aggravation: anti-entropy between A's stale copy and B runs *backward* (Chapter 6's nets reconcile replicas of one shard, and A is no longer enrolled as one), so no janitor ever carries v home. Acknowledged, then invisible forever: the lost write.

Repairs, each singly sufficient against this execution. *Server-side check:* at step (2), A consults its tenancy, finds α ceded at epoch four, rejects with correction; C refreshes, writes at B; no loss. *Fenced handover:* the transfer at step (1) revokes A's tenancy (lease expiry) before B opens; a revoked A cannot apply at step (2) — the write fails visibly, the client retries, no silent loss. Certified fatal as shipped: one routine rebalance, one cache, zero crashes — acknowledged customer data unreachable by every future read.

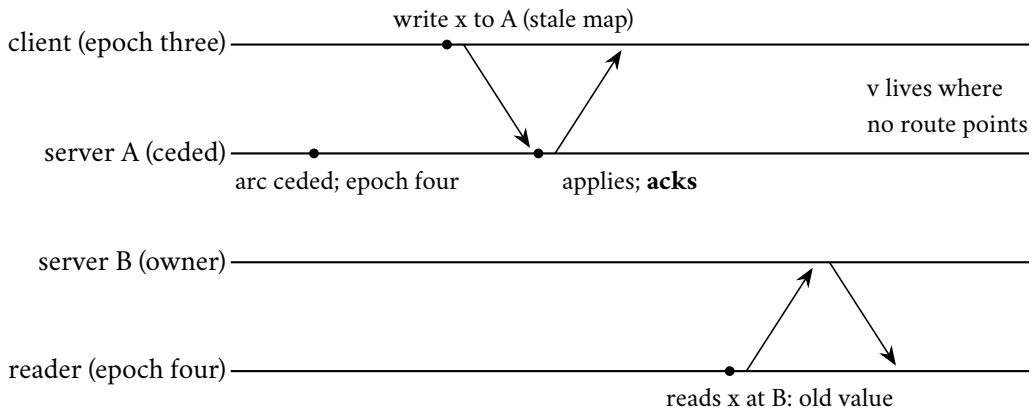


Exhibit 9.4 (the lost write). The stale client writes to the ceded server, which applies unchecked; every current map points elsewhere; the acknowledged value is stranded. Either repair — the server-side epoch check or the fenced handover — breaks step two (Prop. 9.6).

Solution (to Problem 9.8)**.** (a) The merged arc is two designated spacings of $m = n$ tokens: $\text{Beta}(2, n-2)$, mean $2/n$ — the successor's tenancy doubles in expectation, on the worst possible day. (b) Each inheriting successor gains one sliver of the nk-token ring: mean $1/(nk)$, so an inheritor's share rises from $1/n$ by about a $1/k$ fraction of itself — a crumb. (c) $\Pr[\text{Beta}(2, n-2) > 4/n]$: bound each constituent spacing by Prop. 9.4's event or integrate the Beta tail; either route gives a small constant for moderate n — the desk accepts $\Pr \le 2(1-2/n)^{n-1} \approx 2e^{-2}$ by bounding each spacing at $2/n$, roughly a quarter: not rare. (d) "Aliases exist not to smooth the average Tuesday but to ensure that a machine's death is inherited by the fleet rather than by its neighbour: steady-state variance is a nuisance; a doubling under failure is an outage."

Solution (to Problem 9.9)**.** (a) Each rejection carries an epoch strictly larger than the client's; epochs are integers increasing only on ownership change; with c changes during the attempt the client can be corrected at most c times before its map's epoch is current, whereupon its request reaches the owner and is served: retries bounded by c plus one. (b) Oscillation: the arc ping-pongs between servers each round faster than the client's refresh-and-retry; every retry arrives at last round's owner; starvation. Damper: the same as the balancer's — hysteresis on ownership (a minimum tenancy period bounds changes per unit time, restoring (a)'s bound), or have the rejecting server *proxy* the request onward once instead of bouncing the client, converting the last hop into Part Seven's dumb-client mode exactly when the map is too lively to cache.

9.12 The Reading

Karger, Lehman, Leighton, Panigrahy, Levine, and Lewin, "Consistent Hashing and Random Trees," STOC 1997. Short, elementary in the honourable sense, and the rare theory paper whose corporate consequences you have used today. Read the balance and monotonicity claims against Thms. 9.2 and 9.3.

Chang et al., "Bigtable: A Distributed Storage System for Structured Data," OSDI 2006. Read it for the placement machinery — tablets, the metadata hierarchy, the master's duties, the Chubby tenancy — against Exhibit 9.3; the data model was yesterday's news a decade ago, but §5 is the directory church's liturgy.

Thaler and Ravishankar, "Using Name-Based Mappings to Increase Hit Rates," IEEE/ACM Transactions on Networking, 1998. Rendezvous hashing from its authors: the auction, the exact bounds, and the weighted transform Box 22 references.

Stoica, Morris, Karger, Kaashoek, and Balakrishnan, "Chord: A Scalable Peer-to-Peer Lookup Service," SIGCOMM 2001. The fingers, the logarithm, the stabilization protocol — and the premises to audit, one by one, before importing any of it indoors (Rem. 9.4).

Mirrokni, Thorup, and Zadimoghaddam, "Consistent Hashing with Bounded Loads," SODA 2018 (announced 2017). The cap on the unlucky server; balance and movement bounds that survive the forwarding walk (Rem. 9.2).

The shelf. DeCandia et al., Dynamo (SOSP 2007), §4 reread with this chapter's arithmetic — tokens, preference lists, the walk as replicator. Burrows, "The Chubby Lock Service" (OSDI 2006), reread from Chapter 5 now that the tenant list is visible. And Kleppmann, chapter six — partitioning in prose, hot spots and rebalancing included, the standing companion's steadiest chapter.

Chapter 10

The Log and the Dataflow

Companion to Episode Ten. The industry's data machinery, reorganized over two decades around one structure: the log — append-only, totally ordered, durably replayable; agreement, serialized. The lineage, from a file system that made failure boring through two pure functions and a shuffle to recovery by recipe; the log offered alone as a product, and the database turned inside out; batch dissolved into stream, with event time, windows, and watermarks formalized as a failure detector for time; exactly-once cross-examined — delivery dismissed on Two Generals, effect acquitted on three materials, the lemma proved in full; and the photograph of the river, revealed — clause by tabled clause — as Chapter 2's Chandy–Lamport algorithm in industrial dress, the register's oldest debt paid at forty years' maturity. The outbox closes the seam; the sabotage, per the standing rules, is certified fatal with the wire transfer exhibited.

10.1 Part One — The Lineage: Failure Made Boring, Computation Made Pure

In 2013 an engineer at LinkedIn named Jay Kreps published a blog post. Weigh the medium before the message: not a journal paper, not a conference talk with proceedings — a company blog post, "The Log: What every software engineer should know about real-time data's unifying abstraction," some twenty-odd screens of plain prose with hand-drawn rectangles — and it proceeded, over the following decade, to outread the journals: cited in papers, assigned in graduate courses, pressed on new hires by architects with missionary fervour. The field's most influential text of its decade carried no page numbers. Its thesis fits in a sentence, and the sentence is this chapter: the humble log — an append-only sequence of records, numbered in order — is the unifying abstraction of distributed systems, the structure through which data flows between machines, the seam along which every estate can be taken apart and reassembled.

The object's power is exactly its poverty. Two operations: append a record at the tail, receiving the next number; read forward from any number. No update, no delete, no insertion in the middle; the past is immutable, the order is total — within the log — and every reader everywhere, replaying from the top, witnesses the identical sequence and arrives at the identical conclusion. A log is *agreement, serialized* — Chapter 5 built parliaments precisely to decree one — and its three properties — appended only, totally ordered, durably replayable — are, respectively, why writers never conflict, why readers never disagree, and why the past can always be revisited. Everything below is compound interest on those three clauses. The formal record, offsets and all:

Definition 10.1 (log; cursor)**.** A *log* is a sequence of records at consecutive integer offsets, supporting append-at-tail and read-from-offset; records are immutable and retained per policy (time, size, or com-

paction: newest record per key retained, null-body tombstones marking deletions, cleared only after every enrolled reader has passed). A *consumer group* holds one cursor per partition: the offset up to which processing is acknowledged. Rewinding a cursor is the licensed operation that makes input *replayable*.

The post was written from a particular misery, and half the estates on earth still live in it. LinkedIn, like every grown company, had accumulated systems the way a house accumulates drawers — databases, caches, search indexes, warehouses, monitoring, recommendation engines — each needing data from the others, each integrated point to point: with a dozen producers and a dozen consumers, on the order of a dozen dozens of bespoke pipelines, each with its own format, its own failure modes, its own owner who has since left the company. The operational thesis: the hairball dissolves if every system reads from and writes to one central, durable, ordered, replayable structure — producers write once, consumers read at leisure, and the dozen dozens collapse to a dozen plus a dozen. The hub was the business case; the log as *theory* — commit logs, replication, the database's own cellar machinery promoted to civic architecture — is what made the post outlive its trigger. Nor is the structure a stranger here: Chapter 5 decreed one into existence entry by entry, Chapter 8's court of record was one, Chapter 9's directory church kept its map as one. This chapter is the understudy taking the stage it was born for.

The lineage begins at Google in the early 2000s, with a problem of unreasonable size: index the web, on hardware chosen by the accountants. The *Google File System* (Ghemawat, Gobioff, and Leung, 2003) made three decisions that read, after Chapter 9, like a systems-course answer key. Files are huge and append-mostly, so cut them into large chunks — shards of the file — replicated threefold across the fleet. Failure is not an event but a climate — with thousands of commodity machines, something is always dead — so the posture is not to prevent failure but to *metabolize* it: a chunk's death detected, its replicas re-spread, automatically, continuously, as background digestion; *failure made boring* is the paper's real invention, and the temperament survives in every storage system since. And the map — which chunks, which machines — lives with a single master, Chapter 9's directory church founded in one stroke: megabytes of map for petabytes of data, kept current by chunkserver heartbeats, off the data path (clients fetch addresses and then speak to chunkservers directly, so the singular component does plural work), and itself recovered from a checkpoint plus an operations log — Chapter 9's church and this chapter's protagonist, already cohabiting in the founding document. The paper's most quietly radical sentence is about its own arithmetic: at the fleet sizes contemplated, the *normal* state of the system includes failures in progress — the design begins from Chapter 1's documentary premise rather than arriving at it through grief. The honest postscript came with a decade's operation: the single master aged into the design's one regret — metadata growing past one machine's comfort, failover manual in the early years — and the successors sharded the map and put parliaments under it. The church was right; but even directories, Chapter 9 taught, eventually need partitioning, and the lineage's own history obliged by demonstrating it.

Then, 2004, the paper that taught a generation: *MapReduce* (Dean and Ghemawat). Its deep move is easy to state and was revolutionary to commit to: restrict the programmer to two *pure functions* — a map, applied record by record, and a reduce, applied per key to grouped intermediates — with a *shuffle* between them, regrouping every intermediate record by key; nothing else on offer. The canonical job, run once with the company so the shape is in the hands: count every word's occurrences across the library. Bingo, Tuppy, and Gussie each hold chunks of the library; each runs the map over his own chunk — for every word encountered, emit the pair (word, one) — a purely local pass, no conversation. Then the shuffle, the one estate-wide movement: every pair travels to the machine responsible for its word, the destination chosen by *hashing the word* across the reducers — Chapter 9's placement arithmetic, employed to partition not stored data but work in flight; debt #21's payment begins in that clause. Then each reducer, holding every pair for its words, sums the ones and emits the totals. Map, shuffle, reduce; the library counted; and no step of the description mentioned failure, which is the point: pure functions commute with re-execution — run

a map task twice and the world is the same, Chapter 7's idempotence arriving by another staircase — so a failed task is simply *run again*, a lost machine's tasks are re-dealt like cards, and fault tolerance stops being a protocol and becomes a scheduler's habit. Because the functions are pure and the data huge, the scheduler also performs the inversion that pays debt #21's first instalment: *move the computation to the data* — dispatch the map task to the machine that already holds its chunk, shipping a kilobyte of code to a gigabyte of data rather than the reverse; where things live, Chapter 9's entire subject, becomes where things *run*. And for the household accounts: the maps and reduces are cheap and local; the shuffle is the pattern's entire freight bill — every intermediate record crossing the network once, landing on disk at both ends — and two decades of engineering in this lineage are, from the accountant's chair, a sustained campaign against the shuffle's invoice: combine locally before shipping, compress in flight, and, in the successors, keep the regrouped data in memory rather than paying the disk twice.

One mechanism in that paper earns its own paragraph, because the finale has business with it. Jobs were routinely delayed not by failures but by *stragglers* — machines that had not crashed, merely slowed: a failing disk retrying, a noisy neighbour — crashed-versus-slow in overalls yet again. The tail arithmetic of Chapter 6's percentile commandment makes the problem compulsory at scale: if each task independently runs slow one time in a hundred, a job of ten tasks usually escapes, but a job of ten thousand contains in expectation a hundred slow tasks and waits for the slowest of them — at scale the rare event is a certainty, and the job's latency is set by the tail, not the mean. The remedy, *backup tasks*: when a task lingers near the end of a job, launch a duplicate on another machine and take whichever finishes first. Purity makes the race safe — both runners compute the same answer, the loser is discarded, no harm done — and the paper reports price and prize with engineering candour: a few percent of duplicated work, nearly a third of the job's latency returned. Racing duplicates against the slow is filed open as debt #24: in the finale, ten thousand accelerators face the same unluckiest-machine arithmetic every few seconds.

The bureaucratic descendants — query languages compiling to chains of map-reduces, warehouses rebuilt on the pattern — made its cost visible: every stage wrote its entire output to disk, replicated, before the next stage read it back; a ten-stage pipeline paid the storage tax ten times, and iterative computations — the clustering algorithm refining its centres, the ranking computation passing scores around a graph, the model fitted by repeated passes — paid it every lap, revisiting the *same* data each time. The generalization arrived in 2012 from Berkeley: *Spark* (Zaharia and colleagues), the lineage's second inversion. A *resilient distributed dataset* is a collection, partitioned across the fleet, defined not by its bytes but by its *recipe* — the lineage of pure transformations that produced it from stable inputs. Datasets live in memory, uncommitted, unreplicated; and when a machine dies, taking its partitions with it, the system does not restore from a copy — *there is no copy* — it *recomputes*: reads the lost partitions' recipe backward to the last stable ancestor and replays the transformations, for those partitions alone. *Recompute, don't replicate*: fault tolerance as bookkeeping about provenance rather than as redundant bytes; the recipe is kilobytes where the data is terabytes, and purity — always purity — guarantees the second cooking tastes exactly as the first. Two honest footnotes, both in the paper. Recipes are not equally kind: a transformation whose every output partition depends on one input partition — a map, a filter — recomputes a lost partition from one ancestor, cheaply; but a transformation that *shuffles* — a grouping, a join — makes every output partition depend on *all* input partitions, and a single lost partition downstream of a shuffle drags the whole wide ancestry back into the kitchen; whence the engineer's licence to *checkpoint* — materialize a dataset durably, truncating the lineage — precisely at the expensive waists, recompute-versus-replicate a dial rather than a dogma. And memory is a lease, not a promise: datasets are cached as advice, evicted under pressure, recomputed on demand — the recipe always the fallback, which is what lets the promise be cheap. The course files the idea beside Chapter 5's determinism diet and Chapter 8's Calvin: agreement on *inputs and recipe*, with outputs derived rather than negotiated, keeps proving to be the cheapest agreement on the menu.

10.2 Part Two — The Log Ascendant

Kreps's post distilled what LinkedIn had built in *Kafka*: take Chapter 5's log — replicated, append-only, numbered — strip away the parliament's other furniture, and offer the log itself, alone, as a public utility; debt #15, the log's stardom, is paid here. A *topic* is a set of *partitions*; each partition is a log (Def. 10.1): producers append records, the records receive consecutive offsets, and the partition is replicated across brokers by a leader-and-followers arrangement whose ancestry the reader can trace blindfolded — a leader taking appends, followers fetching in order, a committed offset advancing as the in-sync replicas confirm; the parliament's furniture rearranged for throughput, with the fine print Chapter 6 taught still printed on the tin. Before the refusals, the physical insight that makes the product economical, delightfully unfashionable: the log is the *disk's* favourite data structure. Random access is where disks — and, differently, their solid-state successors — go to suffer; sequential appends and sequential reads are where they excel, by orders of magnitude; and a structure that only appends at the tail and reads in runs lives entirely on the hardware's happy path. A pedestrian disk, used sequentially, outruns a fashionable memory grid used badly — whence retention costs shelf space rather than performance: keeping a week of the estate's events is a line item, not an extravagance, and the *replayability* the week buys is the recovery story of the entire modern stack.

The discipline of what the product *refuses* to do is the design. First refusal: global order. Order is guaranteed within one partition and nowhere else — a partition is a shard of the stream, placed and keyed by Chapter 9's arithmetic, so choose the key to make the records whose order matters share a partition: all of one customer's events together, ordered; different customers interleaved arbitrarily, and lawfully. Order is a *per-partition commodity*: within the shard it is free, across shards it costs a parliament, and the product's entire scalability rests on declining to buy what most workloads do not need. The interrogation the review should run: Bertie's order events and the aunt's interleave differently at two consumers — who is injured? Bertie's *own* events out of order would corrupt his story — deposit before account-opening, refund before purchase — so his events share a key and his story is ordered; but whether his Tuesday purchase files before or after the aunt's is a fact no customer, no invariant, and no auditor consults — the only observers who could notice are cross-customer aggregates, and those group by window, not by sequence. Most global-order demands dissolve under the question *ordered for whom*; the residue that survives — a true global sequence, an auction's book — is Chapter 4's business and priced accordingly.

Second refusal: delivery bookkeeping. The queues of the older tradition tracked, per message, per consumer, whether it had been delivered, acknowledged, redelivered — the broker as anxious postmaster, its state growing with its conscience. The log stores nothing of the kind: records sit at their offsets, immutably, for the retention period; each consumer group holds merely a *cursor* — the committed read position, and a cursor, note well, is not a receipt: the broker neither knows nor cares what has been *processed*; consuming is reading, not taking; ten groups read the same log at ten different offsets without disturbing one another or the records. The bookmark is the recovery story entire — a crashed consumer resumes from its last committed cursor and re-reads: *replayable input*, the first leg of a trinity Part Five assembles — and the rewind is the product's finest feature. One Friday from the trade's collective memory stages it: a four-o'clock deploy — the hour has form — ships a consumer with a subtle bug that processes every record and quietly writes rubbish; the rubbish flows all weekend; Monday's dashboards confess. In the anxious-postmaster tradition this is a catastrophe of the first order — the letters were consumed, acknowledged, destroyed; the weekend's truth is gone, and the recovery plan is called *the backup*, may it exist. In the log tradition it is a chore: fix the bug, reset the group's cursors to Friday afternoon, replay the weekend through the corrected code into a fresh view, cut over. The estate's memory was never in the consumer; the consumer was always disposable. Teams that have performed the manoeuvre once do not return to the postmaster.

The reading arrangements hide two Chapter 9 cameos. Consumers organize into *groups* — a group is one logical reader, its members dividing the topic's partitions among themselves: twelve partitions, three

members, four each — a placement problem in miniature, *rebalanced* when a member joins or fails, storms included: a flapping member that triggers rebalance after rebalance can stall a group's reading entirely, and the remedies are Chapter 9's dampers, verbatim. And the parallelism ceiling is set at the keyboard: a topic's partition count bounds its group's useful width — twelve partitions feed at most twelve readers — so the partitioning decision of Chapter 9 is also the concurrency budget of every downstream consumer, decided, as usual, earlier than anyone realized it mattered. Observe, too, what the refusal purchased operationally: the broker's work per record is constant — append, replicate, serve reads by offset — regardless of how many consumers exist, how far behind they lag, or whether they ever return; a group down for maintenance costs nothing but shelf space; a new analytics team subscribing to a year-old topic is merely a new bookmark at offset zero. The log scales its readership the way a library does and a courier service cannot.

Third: compaction, the refinement that makes the log a database's memoir. For topics whose records are keyed updates, retention by time discards history the business may not miss; a *compacted* topic instead retains, per key, at least the latest record, discarding superseded ancestors at leisure — and deletion is handled by an old acquaintance: a record with a key and a null body is a *tombstone*, retained long enough for every reader to witness the death before the stone itself is cleared (Def. 10.1) — Chapter 6's prop, reprised without alteration, because resurrection-by-replay is the same disease as resurrection-by-anti-entropy. The compacted log is then a full snapshot of current state *and* a subscription to its changes, in one structure — replay it from the top and you have rebuilt the table; keep reading and you stay current. Whence the phrase that organized a decade of architecture, Kreps's own: *the database turned inside out*. A classical database is a current-state table with a log hidden in the cellar, feeding the replicas of Chapter 6; invert it — publish the log as the primary artifact, and let every table, cache, index, and search engine in the estate be a *materialized view*: a consumer group with a cursor, rebuilding itself by replay whenever its logic changes. The log becomes the estate's spinal cord: systems communicate not by calling one another — a coupling of availabilities, Chapter 1's oldest tax — but by writing to and reading from the shared, durable, replayable sequence, each at its own pace, each rewindable on its own schedule. The doctrine earns its keep at Bertie's shop on an ordinary Tuesday: the item topic, compacted, feeds the search index, the price cache, the recommendations table, and the warehouse's analytics — four views, four consumer groups, four cursors. The search team ships a better analyzer: they do not migrate a database — they start a fresh index from offset zero, let it chew through the compacted history while the old index serves, and cut over when the new cursor reaches the head; rollback is the same trick backward. The cache is suspected of corruption: discard it entirely and replay — the cache was never the truth, merely a view of the log, and views are disposable by construction. Rebuild anything by rereading; trust nothing but the sequence — an operational lightness its converts describe with the fervour previously reserved for version control, which is, one notes, the same idea applied to code.

And a wry beat to close the part, because Chapter 5's coordination dividend has compounded once more. Kafka's brokers, for their metadata — which brokers live, who leads each partition — leaned for years on ZooKeeper, the consensus kernel in its standard cameo. In its maturity the project retired the external kernel and folded the machinery inside: *KRaft* — the name a pun on Raft, pronounced, the project insists, simply as craft — in which the brokers' metadata is itself a Raft-replicated log managed by the brokers themselves. The shape of the moment: the log-as-a-service, having grown up under the consensus kernel's guardianship, came of age by *becoming* its own parliament — the metadata log and the data logs one species at last. The operational motives were the usual honest ones — one system to run instead of two, faster failover, a metadata ceiling lifted — but the taxonomy is the savoury part: the estate's map, Chapter 9 insisted, wants a court of record, and Kafka's answer, in the end, was that the finest court of record it knew was the product it already sold. The field's plumbing is slowly agreeing with its theory.

10.3 Part Three — The River and the Table, and Time Redux

The old industry org chart put *batch* — nightly jobs over finished files — in one department and *streams* — messages, reactions, dashboards — in another, with different tools, different teams, and different bugs. The log dissolves the department wall, and the dissolution is one sentence: *a stream is an unbounded table being read in motion; a table is a stream compacted to now* — the river and the table. The nightly batch job is merely a consumer whose cursor jumps a day at a time; the dashboard is a consumer whose cursor rides the head of the log; the same log serves both, and the distinction that organized buildings full of engineers reduces to a bookmark's velocity. This is debt #21's second instalment: computation partitioned not only across space — the shuffle's business — but across *time*, batch and stream as two speeds of one dataflow over one structure.

But the dissolution revives this course's oldest ghosts, and they arrive, as Chapter 2 promised, wearing lag. Before the chapter's first formal claim, the models under which every claim below is made:

Model of this chapter

For the dataflow results (Defs. 10.3–10.4, Thm. 10.3, Cor. 10.4). A job is a finite directed *acyclic* graph of operators; channels are FIFO and reliable between correct processes (retransmission beneath, Chapter 1); sources read a durable log supporting rewind to any offset; operators hold state and process one record at a time, deterministically (randomness seeded by record identity; time taken from records and watermarks); processes crash-stop and are replaced. Checkpoint storage is stable.

For delivery and deduplication (Lemma 10.2, Prop. 10.5). At-least-once channels: every sent record is delivered one or more times, finitely often, with a bounded retry horizon T; records carry unique identifiers. Exactly-once *delivery* is unavailable, per Chapter 1's Two Generals theorem — all constructions below are effect-level.

For watermarks (Def. 10.2, Prop. 10.1). Records carry event times; each source emits nondecreasing watermarks and *promises*: a perfect source emits w only after every record with $t_{\text{ev}} < w$ has been emitted; a heuristic source may violate the promise at a measured rate, producing late records.

A batch job over yesterday's file knows the day is *over*; a stream processor computing orders-per-hour must answer a question the file never posed: when is an hour *finished*? The records flowing in carry two different times, and confusing them is the beginner's injury in this trade. *Event time* is when the thing happened — $t_{\text{ev}}(r)$, stamped at the source; *processing time* is when the record reaches the processor — the arrival instant, the clock on the wall. The windows the business asks for come in a small taxonomy — tumbling windows, back to back like paving stones, one per hour; sliding windows, overlapping, the last hour as of every minute; session windows, bounded by gaps in a key's activity, the shopper's visit ending when the shopper does — and every one of them is a *grouping by event time*, which is precisely the grouping the network declines to deliver in order. Between them lies everything Chapters 1 and 2 taught: delays, retries, a partition healing and disgorging an hour of a datacentre's backlog in one minute — and the classic offender, whom every mobile estate meets weekly: Bertie rides the four o'clock train through the long tunnel, tapping purchases into his telephone the while; the application, sensibly, buffers what it cannot send; at six the train emerges, the telephone disgorges two hours of commerce in four seconds, and the processor receives, at six, a burst of records truthfully stamped four-oh-five, four-twelve, four-forty. The four o'clock window closed its books an hour ago — or did it, and on whose authority? Sort the ledger by arrival and the afternoon's history is fiction; hold every window open forever against possible tunnels and

no total is ever final.

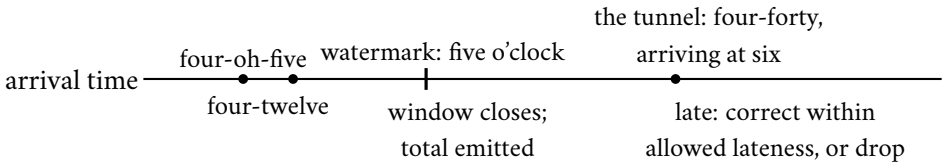


Exhibit 10.1 (the tunnel). Event times on the labels, arrivals on the line. The watermark's passage closes the four o'clock window on stated belief; Bertie's buffered four-forty emerges at six, behind it — late by declaration, handled by policy, counted either way (Def. 10.2).

Every honest computation over "when things happened" must group by event time while living in processing time — which returns us to the question this course has never been allowed to escape: how long must one wait to be sure the past has fully arrived? Chapter 3's answer stands — in an asynchronous world, forever — and so the trade adopted, from the MillWheel and Dataflow lineage at Google (Akidau and colleagues), the instrument this course respects most: the honest confession, flowing through the dataflow as a marker in the stream.

Definition 10.2 (watermarks; lateness)**.** A watermark w at a point in the dataflow asserts: no further record with $t_{\text{ev}} < w$ will arrive at this point. Sources emit watermarks per the model box; an operator with several inputs holds its watermark at the *minimum* of its inputs' and emits accordingly. An event-time window $[a, b)$ may be *closed* when the local watermark passes b. A record arriving behind the watermark is *late*; policy admits it within an allowed lateness (reopening the window and emitting a keyed correction) or drops it with a metric.

An operator closing a window acts not on certainty, which Chapter 3 forbids, but on a stated, tunable, monitorable assumption — Chapter 4's partial synchrony, wearing a timestamp. The soundness claim, its skeleton first: induction on DAG depth — min-propagation preserves the sources' completeness promise, so window closure never precedes its records except where a heuristic source confessed it might.

Proposition 10.1 (watermark soundness)**.** *If every source's promise holds (perfect watermarks) and operators propagate by minimum, then whenever any operator closes window $[a, b)$, it has already received every record of that window destined for it. With heuristic sources, every violation at closure corresponds to a late record at some source, so the failure rate downstream is bounded by the sources' confessed rates. (The watermark is thus a failure detector for time: complete — windows eventually close, if watermarks advance — and accurate exactly as often as the confession is honest; a stalled input freezes closure estate-wide, the price of the minimum.)*

Proof of Proposition 10.1. Induction on topological depth. Depth zero: a perfect source emits w only after all records with $t_{\text{ev}} < w$; the promise is the base case. Depth d: operator P's watermark is min over inlets of watermarks emitted by depth-$< d$ nodes. When P closes $[a, b)$, its local watermark passed b, so every inlet's upstream node had emitted watermark $\geq b$, so (induction) every record with $t_{\text{ev}} < b$ destined for P had been emitted upstream — and FIFO channels deliver emissions before the subsequent watermark's arrival at P. Hence all of $[a, b)$'s records precede closure. With heuristic sources, any record violating closure was late at its source (it arrived behind an emitted watermark there — the violation does not arise in transit, by the same induction), so downstream violations inject no new failures beyond the sources' confessed rate. The failure-detector reading: completeness is watermark progress (add idle-source timeouts, or a stalled inlet freezes the minimum estate-wide); accuracy is the promise's honesty — and as with Chapter 5's detectors, one buys accuracy with waiting, here explicitly, by widening the source's lateness allowance. □

The stall in the proposition's parenthesis is the instrument's known failure mode: one idle source partition, one stuck consumer, and every downstream window freezes in amber — totals never finalizing, state never released, the pipeline healthy by every throughput metric and dead by the only one that matters; whence the standard furniture of idle-source timeouts and watermark alarms, and the reviewer's question that goes with them: what advances your watermark when nothing arrives, and who is paged when nothing does? (Watermark lag, per input, is the first graph a streaming operator watches; Problem 10.4 drills both.) And because assumptions are sometimes wrong, the machinery is honest twice over: the late data are counted, surfaced, and handled by Def. 10.2's declared policy, and the correction has downstream manners — a refined total supersedes its predecessor, so the sink must treat window results as keyed updates, which is to say the *output* wants a compacted log too; honesty about time, once admitted anywhere in the pipeline, propagates its paperwork all the way to the end. The ghosts of Chapter 2 are not exorcised; they are *employed*, with job descriptions: the clock that cannot be trusted now files a confessed estimate; the message that may be late now has a declared budget for its lateness; and the "no now" that haunted the season is answered, in production, by a portfolio of nows — event time for truth, processing time for freshness, the watermark brokering between them — each labelled, each monitored, none pretending to be the one true clock the world declined to provide.

10.4 Part Four — Exactly-Once, Prosecuted

The courtroom, with the injury exhibited before the law, because the law is dry and the injury is not. Bertie's pipeline processes payment instructions from the log: read the record, execute the wire transfer, advance the cursor. At half past four, the process executes a transfer — the money moves — and crashes before the cursor is committed; the supervisor restarts it; it resumes from the last committed cursor; the same record is read again, and the machinery, blameless and obedient, executes the transfer again. The customer has paid twice; the pipeline reports perfect health; and the postmortem contains a sentence one can write from here: "the system provides at-least-once delivery, and the sink was not idempotent." That is *the double payment* — not lost data but repeated effect, a non-idempotent side effect re-executed on replay (Problem 10.7) — and no phrase in this industry's brochures has caused more confusion per syllable than the one that claims to prevent it. The charge sheet distinguishes two defendants that marketing insists on conflating. The first: exactly-once *delivery* — the claim that a message arrives at its recipient once, no more, no fewer, as a property of the channel. The prosecution calls its oldest witness — the Two Generals, subpoenaed from Chapter 1. The final acknowledgment of any conversation may be lost; the sender who has not heard it cannot distinguish a delivered message from a drowned one — and must choose, forever, between resending, which risks twice, and desisting, which risks never. Exactly-once delivery, over a fair-loss network, in the presence of crashes, is not difficult: it is impossible in exactly the sense Chapters 1 and 3 made precise, and every vendor claiming it as a transport property is selling a phrase the mathematics does not underwrite. Case dismissed.

The second defendant walks free, and the distinction is the entire engineering discipline of this part: exactly-once *effect* — however many times the message arrives, the state of the world advances as if it had been processed once. This is achievable, routinely achieved, and built from exactly three materials, all already on the course's shelf. *Idempotence*: make the operation absorb repetition — set the balance to a stated value rather than add to it; upsert the row keyed by the record's identity rather than insert a fresh one; add the tag to a set that ignores duplicates — Chapter 7's algebra hired as plumbing, whereupon at-least-once delivery, the honest and cheap contract, becomes indistinguishable from the impossible one. The craft is in the rewrite: "increment the count" resists idempotence, but "record that event seventeen occurred, and let the count be the size of the recorded set" embraces it — the same re-drafting-toward-accumulation that

CALM licensed in Chapter 7, practised as a plumbing discipline; many an operation is non-idempotent only because it was phrased as a verb when it wanted to be a fact. *Deduplication*: when the operation cannot be made idempotent, make the processing remember — stamp each record with a unique identifier and keep a *dedup ledger*, the set of identifiers already applied, consulted before applying; the duplicate arrives and is recognized, greeted, and discarded. Three clauses of fine print, each a production scar: the ledger must persist across crashes *atomically with the state it protects* — a ledger that dies with the process protects nothing, and one updated in a separate transaction merely relocates the double-payment window; the check-then-apply must itself be atomic against concurrent duplicates — two copies at two workers must not both pass the check, which puts a uniqueness constraint or a single-writer partition under the ledger; and the ledger's window must outlast the retry horizon — remember identifiers for an hour while the upstream retries for a day, and the day's tail replays as fresh business. The sizing is honest arithmetic — rate, times identifier size, times horizon — done with numbers in Problem 10.1; the answers are usually gigabytes, which is to say affordable, and cheaper than the apology. *Transactional sinks*: when the effect and the bookmark can be committed together — the output rows and the new cursor in one transaction, Chapter 8's machinery at last earning its keep in this hall — the classic double-processing window is closed by construction: crash before the commit, neither happened, and replay performs both once; crash after, both happened, and replay resumes beyond the record. The window has no interior, because the two effects were never two — Chapter 8's oldest advice, co-locate what commits together, applied to the bookmark and its consequences. The lemma beneath all three:

Lemma 10.2 (idempotence and deduplication)**.** *Let records with unique identifiers be applied to a state under at-least-once delivery with retry horizon T. The final state equals the exactly-once state if either: (i) the application function is* idempotent *— applying a record twice, in any position among other applications, equals applying it once (sufficient condition: the state is a function of the* set *of applied records, not their multiset or arrival order; or the record's application overwrites a register keyed by its identity); or (ii) a* dedup ledger *of applied identifiers is consulted before and updated atomically with each application, persisted with the state, and retains each identifier for at least T after first application. Both clauses fail if the ledger is separately persisted (a crash between state and ledger re-opens the window) or expires before T (the tail replays as fresh).*

Proof of Lemma 10.2. (i) Set-function form: the final state depends only on the set of applied records; at-least-once delivery applies a superset of multiplicities over the same set; equality follows. Keyed-overwrite form: the last application of record r overwrites the register at key $\mathrm{id}(r)$ with the same value every time (determinism), so the final register bank matches the once-each execution regardless of repetition or interleaving. (ii) Induction on delivery events. Invariant: after any prefix, (state, ledger) equals the exactly-once pair for the set of identifiers in the ledger. A fresh identifier passes the check; the atomic (apply + enrol) step preserves the invariant. A repeated identifier within T of its first application is in the ledger (retention clause) and is discarded, preserving the invariant. No repeat arrives after T (horizon clause). Atomicity is load-bearing twice: a crash between apply and enrol would readmit the identifier (invariant broken — the separately-persisted-ledger failure), and two concurrent copies must serialize on the enrol (the single-writer clause). Both failure modes are exhibited in Problem 10.1(d). □

The materials, boxed for the design review, with the fencing the producers need and the border rule the whole part has been approaching:

Protocol — Box 24. The exactly-once sink materials (distilled from the trade's standard practice and the Kafka design)

1: **idempotent sink:** effects keyed by record identity; apply is overwrite; replays converge (Lemma 10.2(i))
2: **dedup sink:** check ledger; apply and enrol *in one atomic step*, persisted with the state; retain past the retry horizon (Lemma 10.2(ii))
3: **transactional sink:** effects and cursor advance in one transaction; commit on checkpoint completion (two-phase between engine and sink where the sink is external — Chapter 8, redeployed)
4: **producer fencing:** persistent producer identity with epoch; brokers reject stale epochs — the zombie officiant refused (Chapter 5's token, Chapter 8's uniform)
5: **border rule:** any effect outside the wrapped jurisdiction (mail, payments) needs its own material at the end — the end-to-end argument, arriving formally in Chapter 11

Kafka's own transactional machinery assembles all three materials and one old friend. A producer enrolls with a persistent identity and receives an *epoch* — Chapter 5's fencing token in a new uniform: a resurrected or duplicated producer instance carries a stale epoch and is refused by the brokers, the zombie fenced at the door. Idempotent production — sequence numbers per partition, brokers discarding repeats — makes append-at-least-once into append-once. And the transaction wraps a batch of appends across partitions *together with the consumer's cursor advance* into one atomic, all-or-nothing unit, consumers electing to read only committed records — machinery whose internals one can name on sight: a transaction coordinator, prepare-and-commit markers written into the partitions, Chapter 8's two-phase ceremony conducted entirely *inside the log*, with the markers playing the court of record and the epochs fencing the officiants. The season's instruments do not multiply; they redeploy. The result, advertised as exactly-once semantics, is precisely exactly-once *effect* within the log's jurisdiction — the pipeline whose inputs, outputs, and bookmarks all live in the log gets the guarantee end to end, at a price paid in the usual currencies: a coordinator conversation per transaction, and a visibility lag for read-committed consumers, who must wait at the oldest transaction still open rather than drink from the head — one dawdling transaction holds back every record appended behind it, however committed; nothing in Chapter 8 got cheaper by moving indoors. The honest default deserves restating before the machinery's glamour obscures it: for the great majority of pipelines — metrics, views, caches, anything idempotent by nature or by the rewrite — plain at-least-once delivery with idempotent application is simpler, faster, and exactly as correct; the transactional apparatus is for the money paths, and knowing which of your paths are money paths is, as ever, the actual engineering. And the jurisdiction's border is where this chapter's sabotage lives: the moment the pipeline's effect steps *outside* the log — an email sent, a payment initiated, any side door the transaction cannot wrap — the guarantee stops at the threshold, and Bertie's double payment walks through exactly that door; Problem 10.7 exhibits it wire by wire. Under the border glints a general principle, met in fragments all season and named formally in Chapter 11 — the end-to-end argument: a guarantee manufactured in the middle of a system extends only as far as the middle reaches, and correctness at the *ends* — the sink that pays, the mailer that sends — must be arranged at the ends, whatever the middle promises.

10.5 Part Five — The Photograph of the River

And so to the set piece, and the payment of the register's oldest debt. The scene: a streaming job of the modern kind — Flink is the reference house; the mechanism is Carbone and colleagues, 2015 — arranged as a dataflow: Bingo, the *source*, reads the partitioned log; Tuppy, the counting *operator*, keeps running totals

per key — *state*, the accumulated memory of everything processed; Gussie, the *sink*, writes results onward. Records flow continuously; the totals climb continuously; and the operators live on different machines with the Post between them, carrying record streams in order per channel. Weigh the stakes of Tuppy's state before the machinery, because state is what separates this hall from Part One's: a batch job's tasks were pure and stateless — recompute anything from the inputs, the recipes making even that bookkeeping cheap — but Tuppy's totals are the accumulation of everything since the job began — running counts over millions of keys, sessions in progress, joins half-matched — and the job began weeks ago; recompute-from-genesis is a recovery plan measured in days of replay, and the log's retention may not even reach that far. Streaming state is precisely the part of the computation that purity cannot cheaply regenerate, which is why it must be *photographed*. The question the machinery must answer: if Tuppy's machine dies at half past four, taking the totals with it, how does the job resume without either losing the afternoon or counting it twice? The stage, formally:

Definition 10.3 (dataflow job; epochs)**.** Per the model box, a job has sources S, operators, sinks; each source's input is a partitioned log. Checkpoint k is initiated by injecting *barrier* b_k into every source stream; barriers travel in-band. A record belongs to *epoch* k if it was emitted after b_{k-1} and before b_k at its source; FIFO channels preserve the record/barrier order per channel.

The naive answers fail instructively. Pause the world and copy the state: the estate has no pause — Chapter 2 settled that — and the customers would notice the attempt. Snapshot each operator on its own schedule, then, and this failure deserves numbers, because it is the whole motivation: Bingo photographs his bookmark at four o'clock sharp, offset one thousand; Tuppy photographs his totals at five past, by which time he has absorbed records up to offset one thousand two hundred. Now the crash, and the restore: Bingo rewinds to offset one thousand and replays; Tuppy's restored totals *already contain* the two hundred records Bingo is about to send again. Two hundred records counted twice, and no amount of care at either machine alone fixes it — the defect is *between* them, in the photographs' disagreement about which instant they depict.

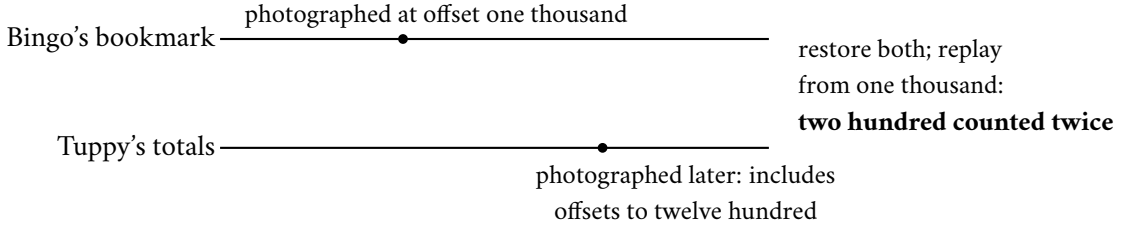


Exhibit 10.2 (the inconsistent snapshot). Photographs on private schedules disagree about which instant they depict; the recovered world double-counts the difference. The defect is *between* the machines — exactly what Def. 10.4 forbids and alignment repairs.

What is wanted is a *consistent* photograph — one logical instant across machines that share no clock. The course has been here before; attend to the mechanism, and to a rising sensation of acquaintance. The photograph, formally — checkpoint k as source offsets plus operator states at the epoch boundary:

Definition 10.4 (consistent snapshot)**.** A *snapshot* of a job is: per source, an offset vector; per operator, a state. It is *consistent at* k if each source offset is exactly the position of b_k, and each operator's state equals the state reached by processing precisely the epoch-$\leq k$ records on every input, in per-channel order — no epoch-$> k$ record absorbed, no epoch-$\leq k$ record missing. (This is a consistent cut in Chapter 2's sense: the frontier of events at the barrier.)

The mechanism. The job manager, on a timer, injects into each source a *barrier* — a special marker record stamped with a checkpoint number; picture a coloured card dropped into each partition's stream. Bingo, on swallowing the card, does two things: records his own position — the offsets up to which he has read — and passes the card *downstream*, into the record stream itself, where it flows in order among the records: everything ahead of the card belongs to the old epoch, everything behind it to the new; the card is a moving frontier in the river. Tuppy has two inlets, and here is the one subtle rule that makes the photograph consistent — *alignment*: when the card arrives on his first inlet, Tuppy does not yet snapshot; he pauses that inlet — buffering its subsequent, new-epoch records — and continues draining the other inlet's old-epoch records into his totals, until the card arrives there too. Cards on all inlets: the old epoch entirely absorbed, the new not yet touched; *now* Tuppy photographs his totals — asynchronously, in the background, while resuming both inlets — and passes the card on to Gussie, who does likewise at the sink's threshold. The alignment has a price, and the honest narrator names it: while Tuppy holds one inlet shut, that inlet's upstream fills its buffers — a deliberate, bounded application of backpressure, the healthy refusal of work that Chapter 11 treats as a first-class discipline; under heavy skew the pause can lengthen, and the modern engines offer an *unaligned* variant that photographs the in-flight records themselves rather than waiting — a fatter photograph for a shorter pause, and, as the reveal will note, a restoration of the original algorithm's channel recording with almost comic fidelity. When every operator's photograph has landed in stable storage, the checkpoint is complete: one coherent picture — sources' bookmarks, operators' totals, sinks' positions — of a single logical instant that no wall clock anywhere witnessed.

Protocol — Box 25. Asynchronous barrier snapshotting (checked against Carbone et al. 2015)

1: **initiate** k**:** job manager injects barrier b_k into every source; each source records its offsets, forwards b_k downstream
2: **operator, on** b_k **from inlet** c**:** mark c aligned; buffer (do not process) subsequent records from c
3: continue processing records from unaligned inlets
4: **when all inlets aligned:** photograph state asynchronously to stable storage; forward b_k on all outlets; release buffered records; resume
5: **sink, on all inlets aligned:** photograph (positions, pending effects per Box 24); acknowledge k
6: **checkpoint** k **complete** when all acknowledgments arrive; retain last complete; discard failed/-partial
7: **recover:** restore all states from last complete k; rewind sources to recorded offsets; replay
8: **unaligned variant:** photograph immediately on first barrier, *including* in-flight records of unaligned inlets — channel recording restored, pause traded for photograph size

Alignment applies bounded backpressure (Chapter 11's discipline, previewed); the interval dial reads: how much river you will re-drink, against how often you photograph.

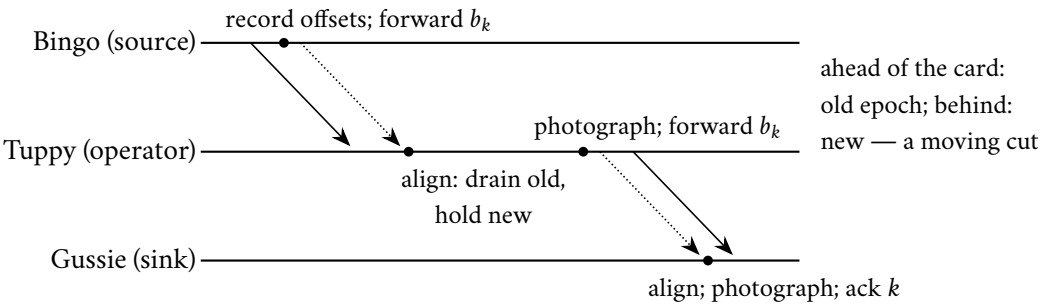


Exhibit 10.3 (the barrier walk). The card flows in-band (dotted); records (solid) ahead of it are old epoch, behind it new; each operator photographs precisely at the frontier (Box 25, Thm. 10.3). Compare Chapter 2's marker figures line for line: the correspondence table is drawn from this picture.

Correctness, the skeleton before the proof: barriers flow with the records, so FIFO channels deliver to every operator a stream cleanly cut into epochs; alignment makes each operator absorb all of epoch k and none of epoch $k{+}1$ before photographing; the union of the photographs is a consistent cut — the marker rule, verbatim.

Theorem 10.3 (barrier snapshot correctness; after Chandy–Lamport)**.** *Under the model box, Box 25's aligned protocol terminates for every checkpoint k, and the recorded snapshot is consistent at k (Def. 10.4).*

Proof of Theorem 10.3. Termination. The graph is acyclic and finite; barriers are broadcast on every outgoing channel upon completion of alignment; induction on topological depth: sources emit b_k immediately on injection; an operator at depth d receives b_k on each inlet from depth $< d$ nodes (which emit it, by induction, after finitely many records — FIFO channels deliver it after the finite epoch-k traffic), completes alignment after draining finitely many records, photographs (asynchronously; completion is bounded by stable-storage availability), and forwards. Sinks acknowledge; the checkpoint completes.

Consistency. Fix operator P and checkpoint k. On each inlet c, FIFO order means every record before b_k on c is epoch $\leq k$ and every record after is epoch $> k$ (induction along the path from the source: sources emit epoch-$\leq k$ records precisely before b_k; an upstream operator, by its own alignment discipline, emits its epoch-$\leq k$ *output* before forwarding b_k — outputs caused by epoch-$\leq k$ inputs are produced during alignment or earlier, and the operator forwards b_k only after those emissions). During P's alignment, inlets on which b_k has arrived are held (their post-barrier records buffered, unprocessed); inlets awaiting b_k are drained — so at the photograph, P has processed exactly the pre-barrier prefix of every inlet: all of epoch $\leq k$, nothing of epoch $> k$. The recorded source offsets are the positions of b_k by construction. Hence every component of the snapshot agrees with Def. 10.4: the checkpoint is the consistent cut at the barrier frontier — Chapter 2's theorem, in the promised overalls. □

Now the reveal, with the full ceremony the constitution allots it. Chapter 2 walked an algorithm for photographing a system that refuses to hold still: markers injected at a node, propagated along every channel; each process recording its state on first marker; channel contents captured as the records arriving between marker and marker; the whole yielding a consistent cut — a photograph in which no effect appears without its cause. Chandy and Lamport published it in 1985; the course spent a chapter on it and has kept it on the shelf, under a promissory note eight chapters old, ever since. Compare, clause by clause: the barrier is the marker; the source injection is the initiation; align-then-snapshot is the marker rule — record on first marker, capture the straggling channel contents until the marker arrives on each remaining channel; the completed checkpoint is the consistent cut. *The asynchronous barrier snapshot is the Chandy–Lamport algorithm in industrial dress.* The tailoring amounts to two alterations, both set as the correspondence's exercise

(Problem 10.10): the dataflow is acyclic, so markers flow one way and terminate at the sinks, sparing the general algorithm's care with arbitrary topologies; and where the original *recorded the channels*, the industrial edition may instead *rewind the source* — the log's replayability standing in for channel capture, since re-reading the river reproduces whatever was in flight (and the unaligned variant, forced to photograph in-flight records after all, restores the channel-recording clause verbatim, closing the circle). Alterations of dress; the theorem inside is the 1985 theorem, unchanged: markers flowing with the traffic partition all events into before and after, and a photograph assembled at the partition line is consistent — no pause, no clock, no now required. The historical note sweetens the payment: Chandy and Lamport wrote the algorithm for the analysts of the eighties — detecting termination, catching deadlocks, debugging — the photograph as an instrument of *inspection*; the notion that it would one day be the beating recovery heart of planetary accounting pipelines, executed every sixty seconds by machinery its authors never imagined, appears nowhere in the paper and everywhere in the industry. Forty years from theorem to overalls, sir, and the overalls fit without alteration. Debt #6 — the register's oldest — presented, and paid in full.

The correspondence, structured. The reveal, clause by clause; Chapter 2's algorithm on the left, the industrial edition on the right.

Chandy–Lamport (1985)	**Barrier snapshots (2015)**
marker, injected by initiator	barrier b_k, injected at every source by the job manager
marker propagates on every outgoing channel	barrier broadcast downstream, in-band, per channel
process records state on first marker	operator photographs after *alignment* (all-inlet barriers)
channel contents recorded between markers	aligned variant: drained into state before the photograph; unaligned variant: recorded verbatim — the original clause restored
consistent cut: no effect without its cause	epoch coherence: all of epoch $\leq k$, none of $> k$ (Def. 10.4)
arbitrary strongly-connected topology	acyclic dataflow; markers terminate at sinks
channel logging for recovery	rewindable log sources replace channel replay

Recovery is then almost anticlimactic: restore the cut, rewind the sources to its offsets, replay — determinism makes the recomputed suffix equal the lost one, and the sink materials convert re-emission into exactly-once effect.

Corollary 10.4 (recovery exactness)**.** *On failure, restore all operators from the last complete snapshot k, rewind every source to its recorded offsets, and replay. The resulting execution's states coincide, record for record, with a failure-free execution's; consequently (i) no input record's effect on state is lost or duplicated, and (ii) with sinks made idempotent, deduplicating, or transactional per Lemma 10.2 and Box 24, emitted effects are exactly-once. Without such sinks, effects between snapshot k and the failure are re-emitted — the double payment of Problem 10.7.*

Proof of Corollary 10.4. Restore and rewind place the job in exactly the Def. 10.4 configuration: states as of the cut, sources at the cut's offsets. The replayed suffix of each source log is byte-identical to the original suffix (immutability, Def. 10.1); channels are FIFO; operators are deterministic per the model box; so by induction on processing steps (per operator, per channel prefix), every operator's state trajectory after restoration coincides with the failure-free trajectory after the cut. (i) follows: each input record's effect is applied exactly once relative to state, since the cut contained precisely the pre-cut effects and the replay applies precisely the post-cut records. (ii) Re-emissions to sinks during replay carry the same identifiers as

the lost originals; Lemma 10.2's materials convert them to exactly-once effect. Without those materials, any sink effect performed between the cut and the crash is performed again — Exhibit 10.4's wire transfer. □

With numbers, for the operator's intuition: checkpoints every minute; the crash at four-forty catches the last complete photograph at four-thirty-nine; recovery restores the totals of four-thirty-nine, rewinds the sources one minute, and re-drinks sixty seconds of river at replay speed — typically far faster than real time, the log serving sequential reads at the disk's happy pace — so the estate is current again within moments, having neither lost the minute nor counted it twice; Box 25's closing line states the interval's dial. The river resumes as if the afternoon's misfortune had been edited out of the footage — and the reader operating a stream processor is operating a running instance of a proof this course walked eight chapters ago; the wonder is not that the machinery is new but that it is *old*, the field's plumbing and its theory, forty years apart, humming the same tune in different registers.

The part closes by assembling the trinity the Whiteboard carries, because the photograph alone recovers nothing. *Replayable input* — the log, retaining and rewindable, so the past can be re-read. *Deterministic-enough compute* — operators whose replay converges to the same totals, purity's household edition, the "enough" earning its adjective: an operator that consults the wall clock, draws randomness, or calls an external service mid-record will replay into a different world, and the disciplines are the model box's — take time from the record and the watermark, never the wall; seed randomness from the record's identity; fence external calls behind Part Four's materials. *Snapshotted state* — the consistent photograph, so replay begins from coherence rather than from the beginning of time. Name all three in a design review or own none of them: a pipeline with snapshots but unreplayable input restores a memory of a world it cannot revisit; replayable input with nondeterministic compute replays into a different world; and both without the photograph replay from genesis, a recovery plan measured in days. Each leg is a chapter of this course wearing overalls: the log, the determinism diet, the photograph. And the lineage's recovery models, laid side by side, turn out to be one doctrine:

Remark 10.5 (lineage recovery, compared)*.* MapReduce persists every stage's output (replicated) and re-runs individual tasks: recovery cost is re-execution of the lost task alone; the price is paid up front, every stage, disk and replication. Spark persists nothing by default and records recipes: recovery recomputes the lost partitions' lineage back to stable input — cheap for narrow dependencies (one ancestor partition per lost partition), expensive past a shuffle (all ancestor partitions), whence checkpointing to truncate lineage at wide waists. The streaming engine of Thm. 10.3 is the third point of the triangle: state that lineage cannot cheaply regenerate (weeks of accumulation) is photographed instead. One doctrine spans all three — replayable input plus determinism makes recomputation a substitute for redundancy — with the systems differing only in what they materialize.

10.6 Part Six — Architectures, Seams, and the Limits of the Church

A paragraph of architectural history first, because its vocabulary persists in design documents. When batch was trusted and streams were new, the *Lambda architecture* proposed running both: a batch layer recomputing truth nightly from the master dataset, a speed layer approximating the last few hours, and a serving layer stitching the two — every metric computed twice, in two codebases, in two frameworks, by two teams, disagreeing in exactly the ways two implementations of anything disagree. The disagreement was not hypothetical; it was a standing agenda item: month-end arrives, the batch number and the stream number differ by a fraction of a percent, and an engineer is dispatched to determine which is wrong — the answer being, reliably, *both*, in different clauses — while the business asks why the dashboard and the report cannot agree on how much money exists. Every estate that ran Lambda owns this meeting's minutes. It was fashion with

a reason, and the reason deserves its due: the early stream processors genuinely could not be trusted with correctness — no consistent photographs, loose delivery discipline — and the batch layer was the apology, recomputed nightly precisely because the daytime arithmetic was known to leak. The *Kappa architecture* — Kreps again, coining wryly — is the observation that once the stream processor acquired this chapter's machinery, the photograph and the trinity, the apology became redundant: keep one pipeline, the streaming one, and when logic changes, *replay the log* through the new code into a new view — reprocessing as a rewind, not a second religion; the Friday-poison manoeuvre of Part Two, promoted from incident response to standing architecture. Fashion, then physics: Lambda was the era when the stream could not be trusted; Kappa is the era when it can; the log's retention is what made the difference, and the audit of any proposed architecture is now one question — is this second codebase an apology for a limitation that still exists?

Second, the everyday dividend, deployed this very week in estates of every size: the *outbox pattern*, and its industrial sibling, *change-data-capture*. The sibling first, in one sentence of mechanism: every serious database already keeps a log in the cellar — the write-ahead log that Chapter 6's replicas drink from — and change-data-capture simply taps it, reading the committed changes in commit order and publishing them outward, with an initial snapshot stitched to the tail of changes so a new consumer can build the whole table and then stay current — the compacted topic's trick, performed at the source; the database's most private structure, promoted to its most public interface. The oldest bug in service architecture motivates the pattern: a service commits a row to its database *and* announces the event to the estate — two effects, two systems, no transaction spanning them. Stage the injury, since most estates contain it somewhere today: Tuppy's order service commits the order row, then calls the message broker to announce order-placed — and crashes between the two. The order exists; the announcement never fired; the warehouse, the email service, and the ledger, all listening, will never hear of an order the customer can see in their account. The mirror-image crash — announce first, commit second — manufactures the opposite ghost: an event for an order that does not exist, and a warehouse packing a phantom. No retry discipline fixes either, because the two effects live in two systems with no transaction spanning them — the dual-write anomaly, Chapter 8's lesson ignored at small scale. The outbox repairs it with the chapter's whole philosophy in miniature: write the announcement *into the database itself* — an outbox row committed atomically with the business row: one transaction, one system, atomicity where it is cheap — and let a relay tail the database's own log, the change-data-capture stream, carrying outbox rows into the public log at its leisure, at-least-once, deduplicated downstream by Part Four's materials. The database's cellar log and the estate's public log, joined by a seam — the log as the boundary object between every system and every other, which is Kreps's thesis operating at the scale of a single feature ticket. The skeleton: one local transaction plus an at-least-once tail with downstream dedup equals a ghost-free, orphan-free announcement stream.

Proposition 10.5 (outbox correctness)**.** *Let a service commit business rows and announcement rows in one local transaction (the outbox), and let a relay read the database's commit log in order, publishing each outbox row at-least-once to a log, with consumers deduplicating by announcement identifier (Lemma 10.2). Then the published announcement stream, after dedup, contains exactly one announcement per committed business event: no ghosts (announcements for uncommitted business), no orphans (committed business without announcement), in commit order. Dual writes — announcing outside the transaction — admit both ghosts and orphans; the crash windows are exhibited in Problem 10.5.*

Proof of Proposition 10.5. No ghosts: an announcement row exists only if its transaction committed (atomicity of the local transaction), and the relay reads only committed log entries; dedup collapses the relay's at-least-once repetitions to one. No orphans: a committed transaction's outbox row is in the database's commit log; the relay reads the log in order and publishes every entry, at-least-once, so the announcement eventually appears (relay crashes resume from its own cursor in the commit log — replayable input, once

more). Order: the commit log's order is the publication order. Dual writes fail both ways: commit- then-announce loses the announcement to a crash between (orphan); announce-then-commit publishes a ghost when the commit fails — the two crash windows of Problem 10.5(a). □

And the borders of the parish, stated plainly lest the church of the log acquire converts beyond its writ, for the chapter has the makings of a theology. The log does not serve *queries*: finding, filtering, joining the current state is the business of tables, indexes, and Chapter 7's stores — the log feeds them; it does not replace them; and the estate that answers "where is the customer's address" with "replay the customer topic" has confused the spinal cord with the filing cabinet — the log's read pattern is sequential and total, a query's is selective and immediate, and no amount of enthusiasm converts the one into the other; the inside-out doctrine says *derive* the filing cabinet from the cord, never abolish it. The log does not *arbitrate*: appending is not deciding, and two producers racing to append conflicting claims have not resolved their conflict, merely recorded it in order — uniqueness, locks, the race test's whole domain remain Chapter 4's business; a log with a parliament beneath it orders entries, it does not adjudicate their meaning. And the log does not span *multi-key invariants*: the doctors of Chapter 7 and the officiant of Chapter 8 are not dismissed by publishing their rows as events — an invariant across two streams needs the machinery of Chapter 8, or a redesign per Chapter 7, exactly as before. The log is the estate's circulatory system — magnificent at moving truth around the body, no substitute for the organs.

10.7 The Whiteboard

For the design review.

1. Order is a per-partition commodity: key partitions to co-locate must-be-ordered records; the partition count is the ordering granularity and the consumer ceiling in one integer — the most consequential integer in the deployment file, set, as usual, on day one. Most global-order demands dissolve under *ordered for whom*.

2. Exactly-once is idempotence engineering, not delivery magic: delivery is at-least-once, by theorem — the Two Generals hold the copyright; effect is exactly-once by materials — idempotent operations, persistent dedup ledgers with sized windows, transactional sinks, fenced producers. Ask which material a brochure means, and where the jurisdiction's border runs: the money paths cross borders by profession, and the double payment lives precisely at the crossing.

3. Recovery is a trinity — replayable input, deterministic-enough compute, snapshotted state; name all three or own none, and audit retention against the outage: a log kept for three days and a bug discovered on the fourth is a trinity with a missing leg, discovered at the worst possible hour.

The register. A ceremony of settlements. PAID: #6 — the photograph in its industrial costume (issued in Chapter 2, the register's oldest resident, retired with the reveal at forty years' maturity). #15 — the log's stardom (issued Chapter 5, paid with a whole chapter's billing). #21 — partitioned computation (issued Chapter 9, paid twice over: the shuffle's move-code-to-data, and batch-and-stream as two speeds of one flow). ISSUED: #23 — the retry's dark side, armed for Chapter 11: every recovery in this chapter leaned on retries and replays, and Chapter 9's stampedes already hinted the retry is also the estate's favourite way of attacking itself. #24 — backup tasks: racing duplicates against the unluckiest machine, filed open for the finale, where the tail arithmetic returns at the scale of ten thousand accelerators and a gradient every few seconds. With the ceremony the season's oldest promissory note has cleared: nothing now stands open that was issued before Chapter 5. EPISODE WORD COUNT: 10,017 — above the floor; the half-draft-by-design drill (LEDGER §5) held.

10.8 Problems

Tier I — Calisthenics

Problem 10.1 (the dedup ledger, sized)**.** A pipeline processes five thousand records per second; identifiers are sixteen bytes; upstream retries persist for at most six hours. (a) Size the ledger's window and memory honestly (with an index overhead factor of your choosing, stated). (b) The operators propose one hour of retention "to save memory"; write the two-sentence objection naming the exact failure. (c) State where the ledger must live relative to the state it protects, and cite the lemma clause. (d) Exhibit the two-step crash that defeats a ledger persisted in a separate transaction.

Problem 10.2 (cursor drills)**.** A topic retains seven days; a consumer group's cursor stands at Friday noon. For each event, state what is possible, impossible, and advisable: (a) a bug shipped Friday noon is discovered Monday; (b) the group is down for maintenance for two days; (c) an analyst wants totals from last month; (d) a second group starts fresh at offset zero on a compacted topic — what exactly does it see, tombstones included?

Problem 10.3 (idempotent or not)**.** Classify under replay, and re-draft the resisters where possible (verb into fact): (a) set customer status to `gold`; (b) increment login count; (c) append line to invoice; (d) send welcome email; (e) upsert row keyed by event identifier; (f) debit account by ten pounds.

Problem 10.4 (watermark propagation)**.** An operator joins three inputs whose watermarks read seven fifty-eight, eight-oh-two, and seven-forty. (a) The operator's watermark, and which window closures are licensed. (b) The third input's partition has been idle an hour; what happens to every downstream window, and name the two standard remedies. (c) One input is heuristic with a confessed one-percent late rate; bound what closure of the eight o'clock window may miss, and say who chose that number.

Tier II — The Real Thing

Problem 10.5 (the outbox, argued end to end)**.** (a) Exhibit both dual-write crash windows (orphan and ghost) with explicit crash points. (b) Prove Prop. 10.5's no-ghost and no-orphan clauses in your own hand, marking where atomicity, log order, and dedup are each load-bearing. (c) The relay crashes mid-publish; trace recovery and name which member of the trinity does the work. (d) A reviewer proposes deleting outbox rows after publishing "to keep the table small"; state what replaces the deletion safely and why deletion-before-confirmed-publish re-opens a window.

Problem 10.6 (alignment's necessity)**.** Modify Box 25: the operator photographs on the *first* barrier and keeps processing all inlets. Exhibit the resulting inconsistency with a two-inlet counting operator and four records (state the totals photographed versus owed); identify which clause of Def. 10.4 dies; then explain in two sentences why the *unaligned* variant (which also snapshots on first barrier) nevertheless recovers correctly, and what it records that the broken variant does not.

Problem 10.7 (sabotage: the double payment)**.** A pipeline reads payment instructions, calls the bank's wire API (non-idempotent, no request identifier), then advances its cursor; checkpoints every minute; the vendor's brochure says "exactly-once semantics end to end." Exhibit the fatal execution: the checkpoint, the transfer, the crash, the replay, and the second wire — timestamps and offsets supplied. State which side of the jurisdiction border the wire lives on, which member of Box 24 was absent, and give the two repairs (an idempotency key at the bank; a transactional pending-transfers outbox with the bank driven from it) with one sentence each on their residual assumptions.

Problem 10.8 (recovery models compared)**.** A ten-stage pipeline loses one machine at stage seven. Compare, with the arithmetic sketched: (a) MapReduce (all intermediates materialized, replicated); (b) Spark, no checkpoints, stages three and eight are shuffles; (c) Spark checkpointed after the stage-three shuffle; (d) a streaming job with minute checkpoints. For each: what is re-read, what is recomputed, what was paid in advance, and which trinity leg is doing the work.

Tier III — For the Ambitious (starred)

Problem 10.9 (⋆ watermark design under a distribution)**.** Lateness is observed distributed as: ninety percent of records within one second, ninety-nine within ten, ninety-nine point nine within two minutes, a heavy tail to hours (the tunnels). Design the watermark and lateness policy for (a) a billing aggregate that must be complete to five significant figures, and (b) a live dashboard refreshed every five seconds. For each: the watermark delay, the allowed lateness, the correction policy, and bounds on error and latency, derived from the distribution. *(Hints only.)*

Problem 10.10 (⋆ unaligned equals channel recording)**.** Formalize the unaligned variant: photograph on first barrier, record in-flight post-barrier records per inlet as part of the checkpoint. Prove its checkpoint is consistent in a suitably extended sense (state + recorded channels reconstruct Def. 10.4's cut), and exhibit the exact correspondence to Chandy–Lamport's channel-recording clause. *(Hints only.)*

10.9 Hints

Hint (Problem 10.6)**.** Let inlet one's barrier arrive, then a post-barrier record from inlet one is processed before inlet two's barrier: the photograph contains an epoch-k+1 effect. On replay that record is replayed too.

Hint (Problem 10.7)**.** Checkpoint at four-thirty-nine, offsets to nine hundred; wire fired for offset nine-fifty at four-forty; crash before the next checkpoint. Replay re-reads nine-fifty.

Hint (Problem 10.9)**.** (a) The five-figure clause fixes the tolerable miss rate; read the delay off the distribution's quantile, and let allowed lateness plus corrections absorb the tail rather than the watermark. (b) Invert the priorities: a short delay, wide lateness irrelevant — the dashboard's next refresh is its own correction.

Hint (Problem 10.10)**.** Define the cut at each operator as (photographed state, recorded in-flight records); replay = restore state, replay recorded records first, then the rewound sources. The recorded records *are* Chandy–Lamport's channel contents.

10.10 Solutions

Tier I in full (tersely); Tier II in full — Problem 10.7 is the sabotage certification; hints only for Tier III.

Solution (to Problem 10.1)**.** (a) Six hours at five thousand per second is one hundred eight million identifiers; sixteen bytes each is roughly one point seven gigabytes raw; with a factor-of-two index overhead, three and a half gigabytes — disk-resident, cheap. (b) "Retries persist for six hours; a ledger remembering one means a duplicate arriving in hours two through six passes the check and is applied as fresh business — the window must outlast the horizon, or it is theatre." (c) Atomically with the protected state, in the same transaction domain (Lemma 10.2(ii)); a ledger elsewhere merely relocates the crash window. (d) Apply effect; crash before enrolling; restart; the identifier is unknown; the effect applies again — the double payment in miniature.

Solution (to Problem 10.2)**.** (a) Possible and advisable: rewind to Friday noon, replay through fixed code (the Friday-poison manoeuvre); the four days of retention headroom made it possible. (b) Nothing is lost; the group resumes at its cursor; the only casualty is lag, monitored. (c) Impossible from this topic — retention is seven days; last month lives in the archived tier or a derived table; the trinity's first leg has a hem. (d) Current state per key in log order plus tombstones for keys deleted-but-not-yet-cleared: replaying yields exactly the live table, deletions honoured; it must treat a tombstone as a delete, not a record.

Solution (to Problem 10.3)**.** (a) Idempotent (overwrite). (b) Resists; re-draft: record login event by identifier, count is the set's size. (c) Resists; re-draft: upsert line keyed by (invoice, event identifier). (d) Resists and cannot be re-drafted inside the pipeline — an external effect; needs Box 24's border rule (idempotency key at the mailer, or an outbox the mailer drains). (e) Idempotent by construction. (f) Resists; re-draft as (e): a posting row keyed by transfer identifier, balance as the sum — the ledger discipline that predates computing, sir, for this exact reason.

Solution (to Problem 10.4)**.** (a) Seven-forty, the minimum; only windows ending at or before seven-forty may close. (b) Every downstream window freezes — no totals finalize, state accumulates; remedies: idle-source timeout (advance the idle partition's watermark by declaration) and watermark-lag alarms on the inlet. (c) At most the confessed one percent of the window's records, in expectation, arriving late and handled by the lateness policy; the number was chosen by whoever accepted the heuristic source's configuration — which is the honest answer's point: the error budget is an *input*, someone's signature is on it.

Solution (to Problem 10.5)**.** (a) Commit-then-announce: crash after commit, before announce — orphan (business without event). Announce-then-commit: crash after announce, before commit — ghost (event without business). (b) No-ghost: the outbox row shares the business transaction; the relay reads only the committed log; dedup absorbs relay repeats — atomicity kills the ghost, the log's committed-only property keeps it dead, dedup tidies the copies. No-orphan: commit implies the row is in the log; the relay's cursor advances only past published entries, so a crash resumes and republishes; at-least-once plus dedup lands exactly one. (c) Relay restarts at its cursor in the commit log and republishes the uncertain suffix; replayable input (the database's own log) does the work; consumers' dedup absorbs the seam. (d) Mark-published (or advance a cursor) instead of delete; deletion before confirmed publication makes the crash window an orphan factory — the row must outlive the uncertainty about its publication, which is the retry horizon again, wearing a table.

Solution (to Problem 10.6)**.** Counting operator, inlets A and B, all records worth one. Barrier b_k arrives on A after two records; the broken operator photographs "two" and keeps processing: takes one more record from A (epoch k+1) and one from B (epoch k), then B's barrier arrives. Owed at the cut: epoch-k records = two on A, one on B = three; photographed: two — and worse, on replay the sources resend A's epoch-k+1 record *and* B's epoch-k record, the former now double-processed relative to any consistent story, the latter missing from the photograph: the state is neither the cut before nor the cut after — no epoch boundary explains it; Def. 10.4's "all of $\leq k$, none of $> k$" dies in both clauses. The lawful unaligned variant also photographs early — but it *records the in-flight post-barrier records as data*, so recovery replays exactly the missing difference: what it records, and the broken variant does not, is the channel contents — Chandy–Lamport's original clause, which alignment merely replaced with draining.

Solution (to Problem 10.7 — the certification)**.** The execution. Checkpoints each minute; the last complete one at four-thirty-nine records cursor offset nine hundred.

(1) Four-forty: the pipeline reads offset nine-fifty — "pay the plumber two hundred pounds" — calls the wire API; the bank executes; money moves. No identifier accompanies the call; the bank knows only that a request arrived.

(2) Four-forty and ten seconds: the worker crashes — before the four-forty checkpoint completes. The cursor of record remains nine hundred.

(3) Recovery restores the four-thirty-nine snapshot and rewinds to offset nine hundred; replay reaches nine-fifty; the pipeline, deterministic and obedient, calls the wire API; the bank, seeing a fresh request, executes. The plumber is paid four hundred pounds; the pipeline reports a clean recovery; every internal guarantee held.

The wire lives *outside* the jurisdiction: the engine's exactly-once wraps state and cursors, and can wrap sinks that accept its materials; the bank's API accepted none. Absent from Box 24: everything — no idempotency key (line 1's material), no dedup at the boundary, no transactional coupling. Repairs: (i) idempotency key — send the payment instruction's identifier; the bank deduplicates; residual assumption: the bank's ledger retention exceeds the replay horizon (Lemma 10.2's window, now on the bank's side of the border). (ii) pending-transfers outbox — the pipeline's transactional sink writes a pending row (idempotent, keyed); a separate driver executes pending transfers and marks them done, itself idempotent against the bank via (i) — which reveals the honest truth that (ii) reduces to (i) at the last hop: at the border, someone must deduplicate, and the end-to-end argument (Chapter 11) says it must be the far end. Certified fatal as shipped: double payment under one crash and one replay, both routine; the brochure's "end to end" extended exactly as far as the vendor's own walls.

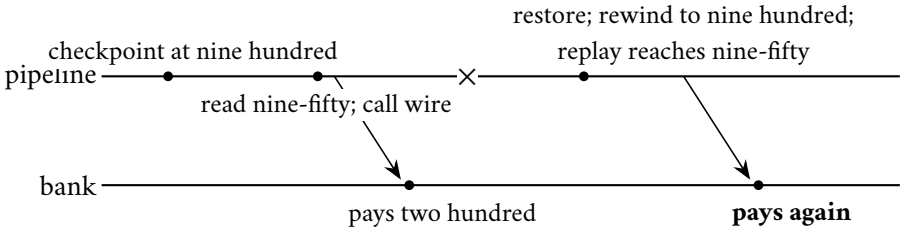


Exhibit 10.4 (the double payment). Every internal guarantee holds; the wire lives past the border, and the border does not dedup; the cross is the crash. The repair is an identifier the far end honours — correctness at the ends, arranged at the ends (Problem 10.7).

Solution (to Problem 10.8)**.** (a) Re-run the lost stage-seven tasks only; inputs read from stage six's replicated output; paid in advance: every stage's materialization and replication, every job, always. (b) The lost partitions at stage seven depend narrowly back to stage four; stage four's inputs cross the stage-three shuffle, a wide dependency, so every stage-three partition — and with it the whole prefix, stages one through three — contributes: recompute back to the last wide boundary and beyond, for want of a checkpoint. (c) The checkpoint truncates lineage at three: recompute stages four through seven for the lost partitions only; paid in advance: one materialization, chosen at the waist. (d) Restore last minute's photograph, rewind, re-drink sixty seconds; paid in advance: a photograph per minute, asynchronous. In every cell the trinity's legs shift weight: (a) leans on materialized input, (b) on determinism alone, (c) and (d) on the chosen mix — which is Rem. 10.5 as a bill of works.

10.11 The Reading

Kreps, "The Log: What every software engineer should know about real-time data's unifying abstraction," 2013. The chapter's text, assigned whole. The clearest statement of the thesis and a model of technical prose without apparatus; read the "logs and consensus" section against Chapter 5 and smile.

Dean and Ghemawat, "MapReduce: Simplified Data Processing on Large Clusters," OSDI 2004. Section three point six — backup tasks — above all: one page the finale will spend a chapter enlarging. The rest is the purity dividend, itemized.

Carbone, Fora, Ewen, Haridi, and Tzoumas, "Lightweight Asynchronous Snapshots for Distributed Dataflows," 2015. Read against your Chapter 2 notes with the correspondence table at your elbow; the pleasure is watching the 1985 constructions acquire buffer pools. Chandy and Lamport (1985) itself, re-read after: the overalls come off and the theorem is still warm.

Zaharia et al., "Resilient Distributed Datasets," NSDI 2012. Lineage as recovery; the narrow/wide distinction and checkpoint truncation of Rem. 10.5 in the authors' hand.

The shelf. Ghemawat, Gobioff, and Leung, "The Google File System" (SOSP 2003) — failure made boring, and the single master's whole story. Akidau et al., "The Dataflow Model" (VLDB 2015) — watermarks, windows, and triggers in full generality. The Kafka design documentation — better literature than most proceedings; the transactions section against Box 24. Kleppmann, chapter eleven, and his "Turning the Database Inside Out" talk — the standing companion on this chapter's whole territory.

Chapter 11

The Engineering of Failure

Companion to Episode Eleven. The paying chapter: five debts due, none issued, and the season's grammar inverted — engineering read from the postmortem backward, opening on the September 2015 DynamoDB incident, the outage that outlived its cause. The mathematics thins and the judgment thickens, and the formal apparatus is set out in full: the phi-accrual dial done honestly; the retry-amplification model with its exponent proved; the deadline-propagation invariant; the metastable tipping point, modelled, with the loop-gain criterion derived; the defused retry, the circuit breaker, and the request's lifecycle as protocol boxes; and, per the season's Decision 8, the one starred appendix — an annotated TLA plus specification, with a checker to run and a protocol to break — contained here and nowhere else.

11.1 Part One — Suspicion With a Dial

First the overture, from the published postmortem — this course, by standing rule, walks only the outages their owners have written up. In the small hours of 20 September 2015, in Amazon's largest region, DynamoDB — the planet-scale descendant of Chapter 6's shopping cart, by then holding up a considerable fraction of the internet — kept its table and partition assignments in an internal metadata service, which the storage fleet consulted on a regular schedule, renewing its picture of who held what. Note how much of the course's furniture stands in the one scene: the metadata service is Chapter 9's directory, the map with an owner; the storage fleet is Chapter 6's school at planetary scale; the renewals are Chapter 5's leases, doing their honest work; and every actor behaves exactly as its chapter taught it to — nothing in the incident is broken in the part-by-part sense. A brief network disruption caused a portion of the fleet to miss their renewals: Chapter 1's weather, routine. The servers did what well-raised servers do — they asked again. But the metadata had been growing — a recently popular feature had made the membership answers substantially heavier — and the renewal requests, arriving now in a synchronized clump rather than a polite trickle, took longer to serve than their timeouts allowed. The requesters, hearing nothing in time, judged their requests failed and asked again; servers that could not complete a renewal within their allowance concluded, prudently, that they should not serve traffic with a stale map, and took themselves out of service — increasing the urgency, and the volume, of the asking. Within minutes the metadata service was fully underwater — not with customer traffic, but with the fleet's own increasingly desperate requests for permission to work. The sentence for which the chapter exists: *the network disruption ended almost immediately, and the outage continued for hours.* The cause was cured; the illness sustained itself. Recovery required the operators to do something that runs against every instinct of the trade: they *turned the fleet away* — throttled the storage servers' own requests, pried open breathing room, added capacity, and only then let the requesters return, gradually, like theatre-goers after a fire alarm.

The outage that outlived its cause names the whole chapter, and the thesis deserves its own line: **mature systems are rarely killed by their failures; they are killed by their reactions to their failures.** The timeout that judged too quickly, the retry that asked too eagerly, the queue that promised too much, the health check that panicked under load — every one a mechanism installed in the name of resilience, and every one, in this chapter, caught moonlighting as the murder weapon. The method inverts accordingly: ten chapters ran guarantee, price, theorem; this one runs death, mechanism, discipline — the mathematics thins, the judgment thickens, and the anatomy of the overture returns as Part Four's set piece, once the vocabulary exists to name what it shows.

Begin where Chapter 1 left the wound open and Chapter 3 salted it: **crashed versus slow**, the confusion no protocol can abolish. Every "is it dead?" in an asynchronous world is answered by a timeout, and a timeout is not a measurement — it is a *wager*, a bet on the tail of the silence distribution, with two ways of losing accepted in advance. Wager too short, and the doppelgänger collects: a slow-but-living machine is declared dead, its work reassigned, its lease revoked, perhaps its throne usurped — Chapter 5's fencing exists precisely to make that loss survivable. Wager too long, and the estate idles behind a corpse: locks held by the departed, elections not called, customers waiting on a machine that will never answer. And where do the timeout numbers actually come from? Interrogate any estate and the answer is archaeological: this one was the framework's default; that one was copied from the service above it; a third was doubled during an outage in a year nobody present remembers. The discipline is to derive rather than inherit: a timeout is a claim about the *distribution* of honest response times — set it at a high percentile of observed latency plus margin, and revisit it when the distribution moves — and it must nest within the caller's own patience: a tier that waits eight seconds on behalf of a caller who abandons at two is conducting a seance, faithfully awaiting answers for the departed. The timeout, the retry allowance, and Part Three's travelling deadline are one budget viewed from three chairs, and the estates that suffer least budget it once, at the top, and subdivide it downward. While the subject is open, sort the estate's timeouts into their three species, set by different logics: the *connection* timeout prices a cheap, local question — an unreachable machine is best discovered in milliseconds — and should be short; the *request* timeout prices the work itself, derives from the latency distribution, and is the one that must nest within the caller's budget; and the *lease*, Chapter 5's tenancy, is a timeout with a title deed — its expiry does not merely abandon a request but transfers authority, so it is set longest of all, with the fencing drill rehearsed for the day the doppelgänger outlives it. An audit that finds all three set to "thirty seconds" has found not a policy but a superstition.

The chapter's three toys are declared together, before the first formal claim — a habit this volume enforces without exception.

Model of this chapter

For phi-accrual (Def. 11.1). Heartbeats from a peer arrive with inter-arrival times drawn, in the healthy regime, from a stationary distribution F estimated over a sliding window; the detector reads the current silence against F's tail. Stationarity is the stated assumption; regime changes (the network's moods) re-estimate F with the window.

For the retry results (Props. 11.2–11.4). A chain of k tiers; a request at tier i issues one call to tier $i{+}1$, retrying on failure or timeout up to r total attempts; during the incident window the bottom dependency fails or times out every attempt (the worst case, stated as such: an honest toy, not a queueing model — its role is to bound the blast, not to predict the weather).

For the metastable toy (Def. 11.3, Prop. 11.6). One service of capacity C requests per second; offered customer load $L < C$; work rejected or expired is retried by clients after a timeout, adding reaction load R; degradation is the unserved fraction. Deliberately crude — one server, memoryless

retries — and flagged so; the starred problem refines it.

The pricing insight, from Hayashibara and colleagues (2004), is disarmingly simple: **stop returning a verdict; return a suspicion level**. The classical detector emits a boolean — alive, dead — manufactured from a threshold it will not show you. The phi-accrual detector instead keeps, per peer, a running record of heartbeat inter-arrival times — the actual, observed rhythm of that peer on this network in this weather — and, when asked, answers with a number: given everything observed, how *implausible* is the current silence? The scale is logarithmic in the improbability — a phi of one means silences this long happen about one time in ten in the normal course; two, one in a hundred; three, one in a thousand — so each added unit multiplies confidence tenfold. This is suspicion with a dial, and the odds are printed on the dial face.

Definition 11.1 (phi-accrual suspicion; Hayashibara et al.)**.** Let t be the time since the last heartbeat and F the estimated inter-arrival distribution. The suspicion level is

$$\varphi(t) \;=\; -\log_{10}\bigl(1 - F(t)\bigr),$$

the log-scaled improbability of a silence at least this long under the healthy regime. A consumer acts at its own threshold Φ; under stationarity, acting at Φ misfires (declares a live peer) on roughly the fraction $10^{-\Phi}$ of silences.

The doctrine is that the consumer chooses its own threshold, per decision, according to the price of being wrong — and the drill deserves its numbers, on the standing cast. Tuppy's heartbeats to Bingo have arrived, these past hours, about every half second, with the occasional straggler at one second and, once in the watch, a two-second gap during a garbage-collection pause. Tuppy falls silent. At one second of silence, phi reads low — silences of this length are Tuesday; nobody moves. At three seconds, phi crosses the routing threshold: Bingo's router quietly stops sending Tuppy new work, a hedge costing nothing if wrong. At eight seconds, phi crosses the failover threshold — silence this long has literally never been observed from Tuppy — and the heavy machinery engages: election, fencing, the works, with Chapter 5's doppelgänger drill standing by in case Tuppy was merely having the pause of his career. Three thresholds, one evidence stream, each consumer paying for exactly the confidence its own blunder would cost. And the detector has quietly *learned the network*: a congested Tuesday widens the observed rhythms, and the same silence that would have alarmed a quiet Sunday is discounted automatically — the wager re-prices itself with the weather, which a fixed timeout, chosen in a meeting three years ago, does not. Problem 11.2 drills the calibration; the honest limits are part of the formal record:

Proposition 11.1 (dial properties)**.** *φ is nondecreasing in the silence t; thresholds order consistently (a higher-Φ consumer never acts before a lower one); and the false-alarm rate at fixed Φ is invariant under re-estimation of F — the dial re-prices itself with the weather, which a fixed timeout does not. What no dial changes: a pause and a death produce the same evidence at every Φ (Chapter 3), so downstream machinery remains fencing-obligated (Chapter 5); and per-peer dials cannot see correlated silence — fleet-level judgment sits above them.*

The suspicion services of Chapter 5 — the kernel's sessions and leases, ZooKeeper's ephemerals, the gossip fleet's accusations — are subscription products built on this same wager, and their price list now reads plainly. You pay in *heartbeat traffic*: a fleet of thousands, all attesting to one another, is a standing chorus the network carries under everything else. You pay in *lag*: no detector may be both instant and honest — Chapter 3's theorem wearing an actuary's suit, for certainty in bounded time is exactly what asynchrony refused to sell. And you pay, if careless, in *correlated panic*, which brings us to the theatre. **Health-check theatre**, the trade calls it, and the sabotage of the week (Problem 11.6) lives here: the check that measures

the wrong thing, splendidly. A process answers its little liveness probe from a thread that does no real work; the disks may be seizing, the queue a mile long — the probe answers instantly: healthy. Or the inverse, the deadlier one: the probe *does* exercise the real path, and under load — precisely when every machine is slow-but-correct — the probes time out fleet-wide, and the orchestrator, obeying its constitution, begins declaring healthy-but-busy machines dead, draining them, and redistributing their load onto the remaining healthy-but-busier machines. Between the two theatres lies the failure mode the trade calls **gray failure**, christened by Huang and colleagues: the machine healthy by every measure the *system* takes and broken by every measure the *customer* takes — the disk that serves probes from cache while failing real reads, the interface dropping one packet in twenty, enough to pass the check and ruin the checkout. The definitional insight is *differential observability*: the checker and the customer are watching different things, and the failure lives precisely in the difference. The remedy is not a cleverer probe; it is humility about probes — measure the paths customers actually travel, weight the verdicts by customer-visible error, and treat any divergence between the health dashboard and the complaint queue as the finding, not the noise. The mercy bias has a concrete implementation the mature orchestrators ship: a *disruption budget* — the fleet may lose at most so many members per minute to health verdicts, whatever the probes claim — so that even a fleet-wide probe panic drains the estate at a governed drip rather than a gulp, and the operators arrive to find a system degraded and complaining rather than one that has conscientiously dismantled itself. The design sentence: **a health check taken under load must be biased toward mercy, because the alternative converts load into membership churn — and membership churn into more load.** That shape is Chapter 9's rebalancing storm, and it is Part Four's subject; above the per-peer dials sits the fleet-level question Part Four makes mandatory — is this a death, or is this weather? The doppelgänger, in his final scheduled appearance, takes a bow: crashed-versus-slow was never solved; it was *priced*, hedged, and dialled — which is what engineering does with the impossible.

11.2 Part Two — The Retry, Weaponized and Defused

Now the sweetest poison in the pharmacy. The retry is the honest child of Chapter 1 — the Post loses letters; asking again is the only remedy the charter permits — and every layer of every estate is therefore taught, in its infancy, to ask again. Nothing in the indictment repeals that: the stubborn link of the first chapter remains the foundation of everything above it, and a fleet that never retried would drop a customer for every flicker of the Post's moods. The crime is never the retry; the crime is the retry *unaccompanied* — without a schedule, without a conscience, without knowledge of its ten thousand siblings doing exactly the same thing at exactly the same moment. The trouble is arithmetical, and the arithmetic deserves small numbers. Picture a modest three-storey establishment: Bingo's web tier calls Tuppy's service tier, which calls Gussie's storage. Each tier, prudently, attempts every request up to three times — a number chosen, in each case, at a different desk, in a different year, by a different team, each reasoning correctly about its own floor. Gussie stumbles — a brief slow spell, every request delayed, none failed. Tuppy's first calls time out; he retries: Gussie now sees three requests where one was meant, each doing real work, none of it wanted twice. Bingo's calls to Tuppy time out meanwhile — for Tuppy is busy retrying — so Bingo retries in turn: three requests to Tuppy, each spawning three to Gussie. **Three at the middle floor, nine at the bottom** — a nine-fold fury delivered to the exact component least able to bear it, manufactured entirely in-house, from one blip and two well-meaning configuration files. Add a fourth storey — an edge proxy with its own three attempts, as most estates have — and the arithmetic runs three, nine, twenty-seven: the bottom floor receives twenty-seven requests where one was meant. Every tier behaved reasonably; the building did not. The skeleton: independent per-tier attempts multiply, so amplification is exponential in the depth of the stack — and the stack was assembled by teams who never met, which is why no local code review can catch

it, and why the reform must be constitutional rather than corrective.

Proposition 11.2 (retry amplification)**.** *Under the model box, a single top-level request generates at most r^i attempts at tier i, and r^k at the bottom; with $r = 3$: one at the top, then three, nine, twenty-seven down the four-storey stack. If instead each attempt independently fails with probability p (partial brownout), the expected attempts at tier i are $\big((1-p^r)/(1-p)\big)^i \le r^i$ — already exponential in depth for any fixed p. Limits of the toy, stated: no queueing, no timeout interaction, no jitter; its office is the exponent, which survives every refinement.*

Proof of Proposition 11.2. Induction on depth. One top-level request is one attempt at tier zero. If tier i sees at most A attempts, each attempt issues at most r attempts to tier $i{+}1$ (the per-call retry cap), so tier $i{+}1$ sees at most rA. Hence r^i at tier i. For the expected form: an attempt sequence at one call site is a truncated geometric — it continues while attempts fail (probability p) up to r — with expected length $(1-p^r)/(1-p)$; independence across sites (the toy's assumption) multiplies expectations down the chain. Both bounds are tight in the stated regimes; neither models queueing — the exponent, not the constant, is the deliverable. □

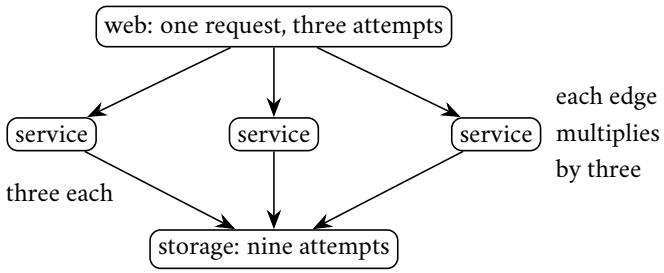


Exhibit 11.1 (the amplification tree). One request, three attempts per tier, two tiers of fan-down: nine arrivals at the floor least able to bear them (Prop. 11.2). Every tier reasonable, the building not; the budget (Prop. 11.4) is the constitutional repair.

The reforms, in the order the trade learned them, each one a scar. **Backoff**: do not ask again at once; wait, and wait longer each time — exponentially, so that a persistent problem sees the fleet's ardour cool geometrically rather than compound. Necessary, and famously insufficient, because of an embarrassment the industry required years to see: synchronized failure produces synchronized retries. The blip strikes ten thousand clients in the same instant; all back off by the same schedule; all return in the same instant — the fleet arrives in waves, like a regiment marching in step across a bridge, and the bridge, which would have borne them strolling, fails at the resonance. Whence **full jitter** — the Brooker doctrine, after the Amazon engineer whose write-up made it canon: randomize the delays across the full range from zero to the backoff ceiling, so the regiment breaks step and the waves smear into a survivable drizzle. Synchronized backoff returns the cohort as an impulse; uniform jitter spreads the same mass over the window — area preserved, peak divided by the window, and it is always the peak, never the area, that breaks the bridge. If one implementation detail survives the chapter, let it be this one: **exponential backoff without jitter is a metronome for stampedes.**

Proposition 11.3 (full jitter)**.** *Let n clients fail simultaneously and retry after backoff W. Without jitter all n arrive in one instant: peak n. With full jitter (delay uniform on $[0, W]$), arrivals form a uniform scatter: expected rate n/W per unit time throughout the window, and the probability that any interval of length δ receives more than $2n\delta/W$ arrivals decays exponentially (a standard Chernoff bound; the proof below records it). Area preserved, peak divided by the window: the regiment breaks step.*

Proof of Proposition 11.3. Without jitter, all n delays equal W: the arrival process is $n\delta$-functions at one instant. With full jitter the arrival times are n independent uniforms on $[0, W]$: the count in any interval

of length δ is binomial with mean $n\delta/W$; Chernoff gives $\Pr[\text{count} > 2n\delta/W] \le e^{-n\delta/(3W)}$. The server's experience changes from one burst of n to a drizzle at rate n/W — with n ten thousand and W two seconds, from a ten-thousand-request instant to five thousand per second, within capacity for any service sized to its steady load and a margin. □

Then the deeper reform, because backoff-with-jitter still shares a flaw with its ancestor: it decides locally, knowing nothing of the estate's condition. The **retry budget** globalizes the conscience: a client — or a whole tier — may spend on retries at most some fraction, say one tenth, of its first-attempt volume; within the allowance, retry freely; beyond it, fail fast and report honestly. In fair weather the budget is invisible — errors rare, the allowance ample. In foul weather — the brownout, the blip — it caps the amplification *by construction*: the nine-fold fury cannot occur, because each tier's asking-again is bounded to a sliver above its asking-once, and the building's total appetite rises gently where it used to explode. The retry converts from an open credit line into a metered indulgence.

Proposition 11.4 (retry budgets)**.** *Let every tier cap retries at fraction b of its first-attempt volume over any window. Then tier i's total volume is at most $(1+b)^i$ times the top-level volume; for $b = 0.1$ and $k = 3$ — the four-storey estate — at most a third again, versus twenty-seven-fold uncapped. The cap binds* by construction*, independent of failure pattern, timeout alignment, or the number of teams who never met — which is why the reform is constitutional where backoff alone is behavioural.*

Proof of Proposition 11.4. Induction on tiers. Tier zero's volume is V. If tier i's total volume (first attempts plus retries) is at most $(1+b)^i V$, its first attempts downstream number at most that, and its retries downstream at most b times its first attempts by the cap, so tier i+1's volume is at most $(1+b)^{i+1}V$. The bound uses only the cap's definition — no assumption about why retries occur or how failures correlate — which is the constitutional character claimed. □

One taxonomy belongs beside the budget, because half the fury in any storm should never have been kindled: not every failure deserves a retry. A timeout, a connection refused, an explicit try-again-later: transient, retriable, within budget. A rejection that says *you are not permitted*, *the item does not exist*, *the request is malformed*: permanent — asking again converts one honest refusal into a drumbeat of identical refusals, load without hope, for the answer will not change because the question has not. And the subtlest class, the overload rejection itself: retrying against a shedding service is pouring petrol on the fire door, and the mature convention is that the rejection carries its own advice — back off this long, or do not return at all — the server, who alone knows its condition, setting the terms of re-engagement. Classify before retrying: the budget governs how often one may ask again; the taxonomy governs whether asking again is even a coherent act. Complementing it, the **hedge**: for the latency tail rather than for errors, send a second copy of a *slow* request after the ninety-ninth-percentile delay, to a different replica, and take the first answer — Chapter 10's backup tasks, miniaturized to a single call. The hedge's own etiquette: it fires on one call in a hundred by construction, so its budget is built in; it belongs on reads and on idempotent writes only; and the polished version cancels the loser, sparing the estate the discarded work. Lawful, all of it, under exactly one condition, stated as the final reform: **no retry, no hedge, without idempotence.** Asking again is only safe when the question absorbs repetition — Chapter 10's lemma cashed here at retail: idempotency keys on every mutating request, dedup at the far end, or the retry storm arrives bearing double payments as well as load. And one structural doctrine above the four adjectives, because the storey-by-storey arithmetic pointed at it: **retry at one layer, not at every layer.** The multiplication came from stacked independent policies; the reform is to designate — per call path, explicitly — which tier owns the retrying, usually the one nearest the user, who alone knows what the request was worth, and to have the tiers beneath fail fast

and report honestly upward. A stack that retries everywhere has no retry policy; it has a retry *ecology*, and ecologies breed storms. The retry, defused, reads so:

Protocol — Box 26. The defused retry (distilled from the Brooker doctrine and the Builders' Library)

1: **classify:** timeout / connection-refused / try-again ⇒ retriable; not-permitted / not-found / malformed ⇒ permanent (never retry); overloaded ⇒ obey the server's advice
2: **budget:** retries ≤ b (a tenth) of first attempts per window; over budget ⇒ fail fast, report honestly
3: **schedule:** delay for attempt a drawn uniform from zero to $\min(\text{cap}, \text{base} \cdot 2^a)$ — full jitter
4: **identify:** every mutating request carries an idempotency key; the far end deduplicates (Ch. 10, Lemma 10.3)
5: **own:** one designated layer retries per path; the tiers beneath fail fast upward
6: **hedge (tail latency only):** after the ninety-ninth-percentile delay, duplicate to another replica; first answer wins, loser cancelled; idempotent calls only

Budgeted, jittered, backed off, idempotent, owned by one layer — each clause an outage's tombstone.

11.3 Part Three — Queues, Deadlines, and Organized Cowardice

The retry was the demand side of self-inflicted overload; now the supply side — what a service does when more work arrives than it can perform. The instinctive answer, the one every framework ships by default, is the queue: buffer the excess, work it off presently. The queue is a fine instrument with one catastrophic misreading: **a queue is a debt instrument.** Work enqueued is a promise to perform work later, issued in the hope of quieter times; a *bounded* queue is a modest overdraft that smooths an afternoon's bumps; an **unbounded queue is an unbounded promise** — and a service running one is a bank issuing unlimited paper against capacity it does not have. The governing arithmetic is one sentence old: the time a request spends waiting is the length of the queue ahead of it divided by the rate of service — the theatre queue's own mathematics, no more — so a queue of ten thousand before a service that performs one thousand a second is, whatever the dashboard's cheerful throughput says, a standing sentence of ten seconds on every arrival; the queue length *is* the latency, wearing units. Watch the default death, the commonest in the genre: load exceeds capacity by a mere tenth; the queue grows — not to some equilibrium, but *without limit*, since arrivals outpace service perpetually; latency climbs through the seconds into the minutes; upstream callers, long past their patience, time out and retry — depositing *fresh copies* of requests whose originals are still queued; and presently the service is spending its whole capacity performing work whose requesters hung up minutes ago — sending answers no one is listening for, in strict arrival order, forever. The service did not refuse a single request, and therefore served none.

The reform is a sentence the trade resisted for a generation because it sounds like defeat — the customer asked, and we said no — and the resistance dissolves only when one sees that the alternative was never "serve everyone": it was "serve no one, slowly, in order of arrival." The sentence, then, engraved: **healthy systems refuse work.** How large should the bound be? Smaller than instinct suggests, and derivable rather than guessable: a queue exists to absorb *bursts*, not deficits, so its proper size is the largest burst the business genuinely produces — a fraction of a second's peak arrivals — and never more than the service rate multiplied by the deadline, since any deeper position in the queue is a request already sentenced to expire in line. A queue sized past that product is not extra safety; it is a morgue with optimistic signage. Bound every queue, then, and when the bound is met, *shed* — reject, immediately, cheaply, at the front door:

an instant "no" costs microseconds and lets the caller act — fail over, degrade, apologize — while a slow "yes" that arrives after the deadline costs everything and helps no one. Shed *early* — at admission, before the request has consumed parsing, authentication, a connection slot; shed *cheap* — the rejection path must be the fastest code in the building, for it will be exercised precisely when nothing else is fast; and shed *by priority*, where the business has one: the checkout survives, the analytics wait — which requires that requests carry their priority as they carry their deadline, declared at the edge where the business context lives, for a storage tier cannot distinguish the checkout's read from the dashboard's without being told, and an estate that cannot tell them apart under pressure sheds by lottery, which is to say it sheds its revenue in proportion to its traffic. Two refinements from the deep end of the practice, both counter-instinctive and both correct. Under genuine overload, consider serving the queue last-in, first-out: the newest arrival is the one whose caller is still waiting; the oldest has likely timed out already, and first-come-first-served under overload is a machine for prioritizing the dead — adaptive establishments switch discipline when the queue ages past the deadline horizon, and switch back with the weather. And shed by *degradation* before shedding by refusal where the product allows: the recommendations rail vanishes, the thumbnails coarsen, the search returns its cached top results — the page arrives, humbler; the trade calls it brownout, and customers forgive a dim room far more readily than a locked door.

Load shedding's portable companion is the **deadline** — the request's parking meter. Every request enters the estate with a patience — the human waits perhaps two seconds; the calling service, five hundred milliseconds — and that patience must *travel with the request*, decremented at each hop: I have three hundred milliseconds remaining; you, downstream, get two hundred of them. A tier that fails to propagate the deadline counterfeits currency — it spends downstream effort against a budget already exhausted, and the deepest tiers of the estate labour earnestly on requests whose customers have left the shop. The drill, with numbers: the shopper's page budget is two seconds; the web tier spends two hundred milliseconds and calls the service tier with one and eight tenths stamped on the request; the service tier spends three hundred, calls storage with one and a half; storage queues the request for one and six tenths — and, consulting the stamp before working, finds the deadline a tenth of a second in arrears: the request expired while it stood in line. Storage declines the corpse and answers "expired" in a microsecond, freeing itself for a request that can still be saved. Without the stamp, storage would have laboured earnestly and shipped its answer into a hung-up telephone — and, under overload, would do so for *every* request, which is the default death of the previous paragraph wearing a tiered costume. The invariant, simple enough for the whiteboard: no work without a live deadline attached, and any request whose deadline has expired is dropped at once, wherever it stands, with a metric to mark the grave.

Definition 11.2 (deadline propagation)**.** Each request carries a remaining budget d. A tier receiving a request with budget d: if $d \leq 0$, reject instantly; otherwise it may work, and may call downstream with the reduced budget $d' = d - \text{elapsed} - \text{margin}$, rejecting the moment its own elapsed time reaches d.

Proposition 11.5 (the parking-meter invariant)**.** *Under Def. 11.2, no tier performs work after the originator's patience has expired, up to accumulated clock-rate error over the request's lifetime (relative drift times duration — microseconds in practice, per Chapter 2's bounds; budgets are durations, not wall times, precisely so that absolute clock skew cancels). Proof by induction on hops: each hop's local spend is bounded by the budget it received, which is bounded by the originator's remaining patience at issue time, monotonically decremented.*

And the pattern that assembles these reflexes into a household appliance: the **circuit breaker** — organized cowardice, endorsed. Walk its three states once, since the pattern appears in every framework's brochure. *Closed* — the healthy state, commerce flowing: Bingo calls Gussie's service and keeps score in a sliding window. Gussie falters; failures cross the threshold — say, half the calls in the last ten seconds — and the breaker *opens*: for a cooling period, every call from Bingo fails instantly, locally, without touching

the wire — the client simply declines to ask, and Gussie receives, from this quarter at least, the gift of silence. The cooling period elapses; the breaker goes *half-open*: one probe call is permitted through. It succeeds — the breaker closes, commerce resumes; it fails — the breaker snaps open for another period, the probe's polite knock replacing what would have been a thousand hammerings. Notice what the breaker is, underneath: a failure detector — Part One's wager, repackaged — driving an admission decision — this part's shedding — with hysteresis against flapping — Chapter 9's damper: the season's parts, snapping together. Its virtue is not sophistication; it is reflex speed and locality: when the storage tier drowns, ten thousand clients each independently stop bailing water into it within seconds, no coordination, no parliament, no committee — cowardice, decentralized, is the fastest form of mercy the estate possesses. The fine print is in the box; the dishonour is not in refusing work — the dishonour is in accepting work one cannot perform, and every mechanism in this part is a way of declining with speed and candour.

Protocol — Box 27. The circuit breaker, with fine print

1: **closed:** calls flow; failures scored on a sliding window
2: **open** (threshold crossed): calls fail instantly, locally, for a cooling period — the dependency receives silence
3: **half-open** (cooling elapsed): admit one probe; success ⇒ closed; failure ⇒ open again
4: **scope** per dependency *and* endpoint — a shared breaker converts one sick route into a general strike
5: **jitter** the cooling periods across the client fleet, or ten thousand breakers probe as one
6: **announce** every transition loudly: a silently open breaker is a silently amputated service

A failure detector driving an admission decision with hysteresis: Parts One, Three, and Chapter 9's damper, snapped together — cowardice, decentralized, as the fastest mercy.

The part's whole discipline, admission to grave, assembles into one lifecycle:

Protocol — Box 28. The request's lifecycle under pressure (admission to grave)

1: **at the edge:** stamp deadline and priority; reject before parsing if the front queue is full (shed early, shed cheap)
2: **per queue:** bound ≤ service rate × deadline horizon; under overload consider newest-first and age-out expiry
3: **per hop:** check remaining budget; decrement; pass it on; drop expired work where it stands, with a metric
4: **under sustained overload:** degrade (brownout) before refusing; refuse by priority before refusing by lottery
5: **on recovery:** readmit gradually, below the ignition threshold (Def. 11.3); the fold is waiting for the simultaneous return

11.4 Part Four — Metastability: The Christening

Now the set piece's theory, and the payment of Chapter 9's debt. The trade had known the shape for decades — congestion collapse brought the early internet to its knees in 1986, and every generation since has rediscovered the pattern in its own plumbing — but it received its proper christening in 2021, when Bronson and colleagues, drawing on years of incidents at the largest estates, named the species: **metastable failure**. Each clause of the definition earns its place. A system has a healthy state, in which it serves its load with

margin. There exists a trigger — a blip, a deploy, a surge — that pushes it into a degraded state. In an ordinary failure, remove the trigger and the system recovers: the weather passes, the water drains. In a metastable failure, the degraded state is *self-sustaining*: some feedback loop, internal to the system, generates enough load — retries, re-elections, cache misses, health-check churn — to keep the system degraded after the trigger is gone. The degraded state is, in the physicist's borrowed vocabulary, stable: the system has found a second equilibrium, on the wrong side of a hill, and it will sit there, burning its whole capacity on its own feedback, until pushed back over the crest by force.

Definition 11.3 (metastable failure; after Bronson et al.)**.** A system state is *vulnerable* when healthy at current load but within reach of a trigger; a failure is *metastable* when, after a transient trigger moves the system into a degraded state, a feedback loop — reaction load generated by the degradation itself — sustains the degraded state with the trigger absent. Diagnostic signature: goodput does not recover when offered load returns to a level previously served with margin (the fold, Exhibit 11.2); recovery requires exogenous loop-breaking (throttles, restarts with admission control) and gradual readmission below the ignition threshold.

The diagnostic picture every operator should carry: plot offered load across the bottom, useful work delivered up the side. In a healthy system the curve climbs with the load, flattens at capacity, and — this is the pathological signature — then bends downward: past a certain point, additional load produces *less* useful work, because a growing share of capacity is consumed by the feedback — retries processed and abandoned, connections churned, elections held, probes failed. And the curve is not retraced on the way back — that is the fold, the hysteresis: reduce the load to a level the system handled easily this morning, and the goodput does not recover, because the loop's own traffic now occupies the margin. The system is not on the morning's curve any more; it is on the other branch, and the only road home passes below the loop's ignition point — which is why every recovery in this genre involves throttles that feel, at the time, like surrender.

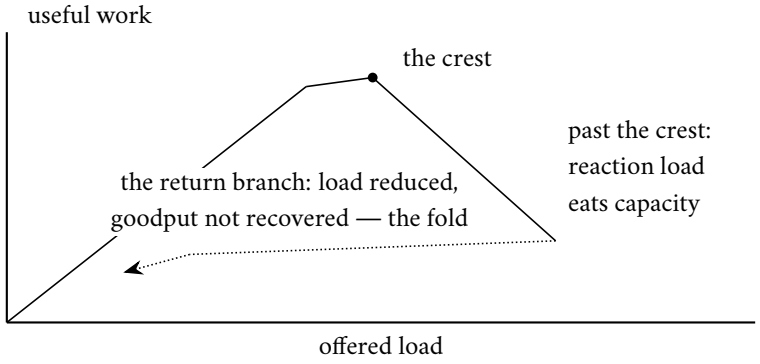


Exhibit 11.2 (the goodput fold). Rising load climbs the healthy branch to the crest, then bends down as feedback consumes capacity; retreating load (dotted) does not retrace the climb — the system is on the degraded branch, and the road home passes below the loop's ignition point (Def. 11.3, Prop. 11.6).

The paper's trigger taxonomy dissolves a common alibi. Triggers include load surges, yes — but also deploys and rollbacks, cache restarts, routine maintenance, a dependency's brief wobble, even a *drop* in capacity while load is steady: anything that momentarily narrows the gap between demand and supply. The alibi it dissolves: "we could not have foreseen the trigger." Quite so — and irrelevant, for triggers are legion, cheap, and perpetual; the vulnerability was never the trigger but the loop and the margin, both of which were in the blueprints. The sustaining loops, collected like pressed flowers across three chapters, named in one breath as a family. The *retry storm*: overload slows answers; slowness breeds timeouts; timeouts breed

retries; retries are load — Part Two's arithmetic, closed into a circle. The *cache-miss avalanche*: the cache fails or cools; misses fall through to an origin sized for one tenth of the traffic; the origin slows; entries expire faster than the struggling origin can refill them; the hit rate falls further — the shield, once dropped, cannot be lifted under fire. The *connection stampede*: timeouts kill connections; re-establishing them costs handshakes, credentials, warm-up — dearer than the steady state; the re-connection load itself keeps the peer too slow to accept connections. The *membership churn* of Chapter 9's storm and Part One's theatre: slowness reads as death; declared deaths trigger data movement and elections; movement and elections are load; load is slowness. One skeleton in every costume: *the system's response to degradation is itself load of the very kind that degrades it* — the feedback closes, and everything turns on the loop's gain. Reaction load below the margin, and the wobble damps out unnoticed, as it does a hundred times a week in every healthy estate; reaction load above it, and each turn of the loop finances a larger next turn — the fire feeds the fire, and the difference between the two fates was a coefficient nobody had measured. The honest toy makes the criterion exact:

Proposition 11.6 (tipping in the honest toy)**.** *In the model box's toy, let unserved work be retried so that total demand is* $T = L + \gamma \cdot \max(0,\, T - C)$*, where* $\gamma \geq 0$ *is the reaction gain (retries per unserved request, net of abandonment). If* $\gamma \leq 1$*, the only equilibrium with* $L < C$ *is healthy (*$T = L$*). If* $\gamma > 1$*, then once any transient pushes demand past capacity, a fixed point* $T^* = (L - \gamma C)/(1 - \gamma) > C$ *exists: demand settles above capacity and stays there even for values of L the system formerly served with margin. Capacity increases move the threshold; only reducing* γ *below one (budgets, jitter, abandonment, admission control) removes the degraded equilibrium. The algebra is Problem 11.9 (starred); the moral is the whiteboard's item four.*

Proof of Proposition 11.6. Demand satisfies the fixed-point equation $T = L + \gamma \max(0, T - C)$. Below capacity the max vanishes: $T = L$, available iff $L \leq C$. Above capacity, $T = L + \gamma(T - C)$ gives $T^* = (L - \gamma C)/(1 - \gamma)$, which for $\gamma > 1$ is positive and exceeds C whenever L is near C — and, for large γ, even for L well below it. Stability: perturb T above T^*; the update map has slope $\gamma > 1$ — and here the honest toy shows its seams, for a slope above one is unstable in the naive iteration — the physical stabilizer is capacity saturation: served work is $\min(T, C)$, so reaction load saturates at $\gamma(T - C)$ only until abandonment and timeouts cap it; the refined model of Problem 11.9 adds the cap and recovers a genuinely stable degraded equilibrium. What the crude algebra already delivers, correctly, is the threshold: *no* degraded fixed point exists at any load when $\gamma \leq 1$ — the loop-gain criterion — and that is the design-review deliverable: measure and reduce γ, do not merely raise C. □

Now the anatomy, walked in full, as the constitution appoints. Return to the overture with the chapter's instruments in hand. *The vulnerable state*: the DynamoDB storage fleet's membership renewals had grown heavy — the metadata answers enlarged by the new feature — so the margin between the metadata service's capacity and the fleet's routine demand had quietly narrowed; the system was healthy, and standing nearer the crest than anyone had measured. *The trigger*: the network disruption — brief, ordinary, cured within minutes — which synchronized a cohort of storage servers into simultaneous renewal. *The loop closes*: the clumped renewals exceeded capacity; requests ran past their timeouts; the requesters retried — Part Two, unjittered by history's misfortune, in step; servers whose renewals failed took themselves out of service — Part One's prudence, correct in isolation, catastrophic in chorus — and out-of-service servers ask *more* urgently, not less. The metadata service was now fully submerged by the fleet's own recovery efforts: the trigger's weather had passed; the loop no longer needed it. *The escape*: not by waiting — the second equilibrium is stable, that is the whole horror — and not by ordinary capacity alone, for the loop's appetite scales with its despair. The operators performed what the metastability paper would later canonize as the only reliable surgery: **break the loop by force** — throttle the renewals themselves, turning the fleet away from its own metadata; let the service breathe, add capacity behind the shelter of the throttle; then

re-admit the requesters *gradually*, below the loop's ignition threshold — gradually being the load-bearing adverb, for a simultaneous readmission of the whole starved fleet is simply the trigger fired again, by one's own hand, at maximum amplitude; the fold in the curve waits patiently for exactly that mistake, and more than one estate in the genre's annals has relit its own fire on the first attempt at dawn. Hours, for an outage whose cause lasted minutes — and the postmortem's remedies are this chapter's syllabus in order: heavier capacity margins (move the crest), tighter retry discipline and slower self-removal (lower the loop gain), and admission control on the metadata service (a permanent, honest throttle — the fire door installed while the ashes cooled).

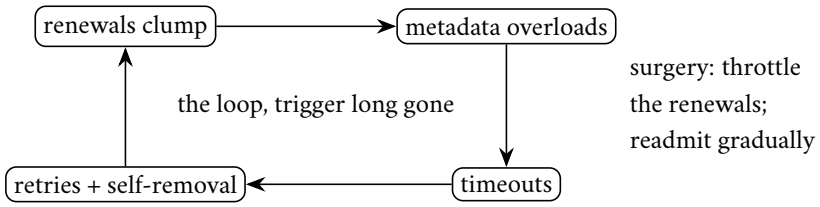


Exhibit 11.3 (the DynamoDB loop). The September 2015 anatomy from the published postmortem: each box feeds the next; the network disruption that started the wheel is nowhere on it. Recovery broke the cycle by force at the renewals edge and readmitted below ignition (the set piece, Part Four).

The doctrine, distilled for the design review. Metastable failures are not prevented by capacity alone — capacity moves the crest; the loop remains, waiting for a larger trigger. They are prevented, or at least disarmed, by **loop surgery**, and the operating table pairs each loop with its governor: the retry storm takes the budget and the jitter — Part Two, applied; the health-check churn takes hysteresis and load-biased mercy — Part One, applied; the cache avalanche takes coalescing, jittered expiry, and serve-stale — Part Five, waiting; the connection stampede takes connection pools with caps, and reconnection backoff of its own; the election flurry takes Chapter 9's minimum-tenancy damper; and beneath them all, admission control on the shared dependency — the permanent, honest throttle that ensures no chorus of governed loops can still, in sum, drown the well. One governor per loop, and one fire door on the building. Because loops are easier to draw than to notice, the review question that finds them: *when this system is at one hundred and ten percent of capacity, list every message it sends that it would not send at ninety* — every entry on that list is a candidate conspirator, and the empty list is always, always wrong. And one organizational clause, on which the paper's authors — engineers of the largest estates, writing from their scars — insist: the postmortem that stops at the trigger has learned nothing. "A network blip caused the outage" is, for a metastable failure, a category error — the blip caused the *transition*; the outage was caused by the loop, which was designed, reviewed, shipped, and waiting. The trigger is weather; the loop is architecture; and a review culture that files these incidents under weather will meet the same loop again, ignited by a different Tuesday.

11.5 Part Five — The Herd, Coalesced

The cache stampede takes its own short part, as Chapter 9 promised, because it is the metastable seed most estates plant on purpose, in their fastest code, with pride. The scene, from that chapter's storm: the aunt's celebrated flower-committee notice — her fame undimmed — cached everywhere, expiring everywhere at the same instant, for the expiry was set by one line of configuration and the herd's watches, unlike its members, agree perfectly. Ten thousand requests miss together; ten thousand identical queries strike the origin as one; the origin, sized for the cache's leavings, staggers; and while it staggers, *more* entries expire. The survival kit: three instruments, in the order of their cost. *Jittered expiries*: never let the herd's watches

agree — randomize each entry's lifetime a little, so expirations smear across minutes rather than detonating on the second; one line of code, again, and the regiment breaks step before the bridge. *Request coalescing* — the trade also calls it single-flight, which says it exactly: at each cache node, let the first miss for a key travel to the origin and make every concurrent miss for that key wait for the same answer — ten thousand misses become one query and one broadcast; a miss becomes an event rather than a demographic. And *serve-stale-while-revalidating*: when the entry expires, keep answering with the slightly stale copy while a single background refresh fetches the new — the origin sees a whisper, the customers see no cliff, and the staleness is bounded and declared, which after Chapter 7 is a contract, not a confession. Two accessories complete the kit. *Negative caching*: when the herd asks for something that does not exist — the deleted notice, the mistyped identifier — cache the absence too, briefly, or the misses-by-definition bypass every layer of protection forever, a stampede that no refresh can ever satisfy. And *prewarming*: a cache restarted cold under full traffic is Part Four's avalanche scheduled by the deploy calendar — warm it from a snapshot, or admit traffic to it gradually, the theatre-goers readmitted after the alarm; a cold cache behind a hot service is not a fresh start, it is a lit fuse. Between them, these five convert the cache from a metastability engine into a shock absorber — and they cost, together, an afternoon of engineering. The trade's failure to deploy them is never expense; it is the belief, refuted at teatime weekly, that the herd will not visit *this* pasture. Problem 11.5 designs the kit with its bounds stated as contracts.

11.6 Part Six — The End-to-End Argument, In Its Own Words

Now the oldest and most quietly load-bearing idea of the chapter, glimpsed at Chapter 10's border and owed its full ceremony. Saltzer, Reed, and Clark, 1984, "End-to-End Arguments in System Design" — a paper about where, in a layered system, a function should live. The worked example is file transfer, and it has not aged a day. You wish to move a file between machines, correctly. The network offers help: checksums on every hop, acknowledgments link by link, careful routers. Suppose you accept, and engineer every hop to perfection. Is the transfer now correct? It is not — and the reasons are the paper's immortal catalogue: the file may be corrupted *before* it reaches the first hop, by the sending host's own disk or memory; or *between* hops, in a router's buffer; or *after* the last hop, landing on the destination disk askew. The hops' diligence covers the hops; correctness is a property of the *ends* — did the bytes that left the source application arrive at the destination application intact? — and only the ends can check it, by an end-to-end verification: a checksum computed by the sender's application, verified by the receiver's, with a retry from the top on mismatch. And here is the argument's cutting edge, the part that licenses simplicity: once the ends must check anyway — and they must — the hops' guarantees are redundant for correctness; whatever the middle promises, the ends cannot dispense with their own audit, so the middle's promises are properly understood as performance optimizations — catching common errors early so end-to-end retries stay rare — never as truth. Build the network fast and simple; put correctness where only it can live: at the ends.

Remark 11.4 (the end-to-end argument; Saltzer–Reed–Clark). A design principle, not a theorem, stated in the founders' frame: a function whose correct implementation requires the knowledge and cooperation of the application endpoints cannot be completely provided by the communication system alone; the endpoints must implement it regardless, whereupon the middle's version is justified only as a performance optimization. The gallery: file-transfer integrity (checksum at the ends; hop checks catch errors early, cheaply, incompletely); delivery acknowledgment (host receipt is not application action); encryption (hop-by-hop leaves plaintext in every middle); duplicate suppression (the transport cannot see the double click). Jurisdictional reading, from Chapter 10's border: every manufactured guarantee has a wrapper boundary, and the double payment lives at the crossing — dedup at the far end (Lemma 10.3) is the argument in overalls.

The gallery generalizes with almost embarrassing reach, and the shape of the error is always the same:

mistaking a property of the pipe for a property of the conversation. The network's acknowledgment says the destination host received the bytes, when what the sender needs to know is that the destination application acted on them — which only the application's own reply can attest; data protected hop by hop lies naked in every intermediate machine's memory, and only end-to-end encryption, keys held at the ends, delivers the property the user actually named; the network may suppress its own retransmissions' duplicates, but the application-level duplicate — the user who clicked twice, the client that retried above the transport — sails through untouched, and only an end-to-end identifier catches it. The argument surfaced twice in Chapter 10 without its name: the exactly-once verdict was the end-to-end argument in streaming clothes — the pipeline's internal machinery may be immaculate, but the double payment happened at the border, and the repair, the idempotency key the *bank* honours, lived at the far end, because only the far end can absorb the duplicate; the middle's transactions were performance, narrowing the window; the end's dedup was correctness, closing it. And the fencing tokens of Chapter 5, the routing epochs of Chapter 9, the producer epochs of Chapter 10 — every one is the receiving end declining to trust the middle's claim that the sender is current. Two honest qualifications, both from the argument's authors. Hop-by-hop still earns its keep where the arithmetic favours it, and the arithmetic deserves one worked breath: send a large file across twenty hops of which one drops a hundredth of its packets, and with end-to-end retries alone the whole transfer re-runs, possibly repeatedly, each failure discarding nineteen hops of honest work; let the lossy hop retransmit locally and the end-to-end check becomes a formality that almost never fires — the middle's diligence multiplies throughput, performance purchased locally, exactly as licensed. And some functions — the network's own routing, congestion control, this chapter's backpressure — genuinely live in the middle, being properties *of* the middle. The argument is not "the middle must be dumb"; it is "the middle cannot be trusted with the ends' correctness, so do not pay it to pretend." As a review instrument it is one question, and it retires half the architecture proposals it meets: *which end checks this, and what happens when the middle lies?* It scales, one notes, past machinery into organization: the platform team's queue, mesh, and retry layer are the middle — genuinely valuable, honestly performance; the product team's invariants — the money adds up, the email sends once — are the ends, and no platform feature will ever discharge them. The teams that flourish are the ones whose contracts say so in writing; the postmortems that recur are the ones where each side believed the other held the checksum.

11.7 Part Seven — Testing the Untestable

The auditors, then — for a course that has spent ten chapters proving what can go wrong owes a part to how the trade now *finds* it going wrong before the customers do. Three disciplines, in ascending order of ambition, and a fourth beneath them all.

Fault injection with verification — the Jepsen method, industrialized. Chapter 7 introduced Kingsbury as consumer protection: partitions and crashes injected, client histories collected, the transcripts checked against the formal definitions. What began as one man's bloody-minded laptop is now a standing discipline with industrial parts: operations generated randomly against a model of what the store claims to be; a *nemesis* process scheduling the faults — partitions drawn from Chapter 1's documentary catalogue, process pauses long enough to expire leases, clocks jerked forward and back; and the checker, afterwards, hunting the transcript for a lawful explanation, with the modern checkers efficient enough to try serious histories rather than toy ones. The same harness logic now runs in the vendors' own pipelines — the audited have internalized the auditor, which is the finest outcome an auditor can have — and its civilian descendant, **chaos engineering**, extends the method to whole estates: inject the Post's malice *on schedule* — kill instances, sever links, inflate latencies — in production or its faithful replica, under a declared hypothesis ("the checkout survives the loss of any one zone") and, non-negotiably, under blast-radius discipline, whose

clauses separate the method from vandalism: begin in the replica, graduate to a sliver of production, and grow the scope only as the hypothesis survives; watch the *customer* metrics, not the system's self-regard, with automatic abort on breach; schedule the experiment when the responders are awake and the business is calm; and treat a surprising result as a finding to fix, not a failure of the exercise — the whole point was to be surprised at a moment of one's own choosing. The lineage deserves its footnote: the practice entered folklore when a streaming company loosed a daemon on its own production fleet — the chaos monkey, terminating instances at random during business hours — on the reasoning that a failure mode rehearsed daily cannot ambush anyone; the name stuck, and the modern form is less monkey than method: hypothesis, injection, measurement, abort handle. The philosophy is Part One's wager matured into policy: the failures *will* occur; the only choice is whether their first rehearsal is attended by engineers with clipboards or by customers with lawyers.

Formal specification, in anger. The trade long filed TLA plus — Lamport's specification language, the same Lamport, his third appearance in this course's founding roles — under academic luxuries; Amazon's engineers, in a Communications of the ACM report by Newcombe and colleagues (2015), ended that. A specification, for the reader who has not met one: a description of the system as states and transitions — what the variables are, how each step may change them — together with the invariants that must hold in every reachable state and the progress properties that must eventually hold; not code, but the *design*, stripped of implementation and made precise enough to be checked by machine. The checker then does what no reviewer can: visits every reachable state the specification admits, under every interleaving, every failure the model permits — exhaustively, joylessly, without the human reviewer's fatal gift for assuming the plausible. They specified their production designs — DynamoDB's replication among them — and the model checker found genuine bugs requiring **thirty-five steps** to manifest: sequences of interleavings and failures no review would imagine, no test would stumble upon, and no amount of production traffic would reliably produce before the one morning it mattered. Thirty-five steps is the depth at which the doppelgänger operates when nobody is watching. The report's summary sentence deserves any whiteboard: formal methods *find bugs that cannot be found any other way*, at design time, when they cost a meeting rather than a postmortem. The honest boundaries, in the authors' own spirit: the checker verifies the design, not the code — the implementation may still betray a correct specification, which is why the discipline pairs with the harnesses on either side of this part; the state space must be kept modest — three servers, not three thousand, on the standard argument that the bugs of these protocols live in the interleavings, not the multitudes; and the investment is real — days to learn, weeks to specify — repaid, in the report's accounting, many times over on exactly the systems whose failures make newspapers. Per the season's Decision 8, this chapter's starred appendix is the on-ramp: an annotated specification, a checker to run, a protocol to break — contained there and nowhere else.

And the wonder beat, held for last: **deterministic simulation testing**, the FoundationDB school. The problem it answers is the tester's oldest grief in this field: the bug appears once, at three in the morning, under an interleaving of messages and timeouts that no one can ever summon again; the failure is real and the reproduction is impossible, and engineering without reproduction is archaeology. The FoundationDB designers' response was of a piece with everything this course admires: they removed the impossibility at the root. The entire database runs, in test, inside a simulator in a single thread — the network is simulated, the disks are simulated, the clocks are simulated, every timeout and delivery and crash driven by one pseudo-random number generator with one seed. Determinism — Chapter 5's diet, Chapter 8's Calvin, Chapter 10's replay, now applied to reality itself: same seed, same universe, same bug, every time. The harness then does what no operations team can: it lives a million catastrophic lifetimes a night — partitions, crashes, disk corruptions, clock swerves, all injected by a malicious scheduler — and the code itself is seeded with cooperating treachery: little markers throughout the source (*buggify*, the practitioners call them), activated

at random under simulation, that make every buffer smaller, every rare branch likelier, every error path a boulevard, so the improbable is rehearsed nightly at a million times its natural frequency. Any lifetime that ends in a violated invariant is preserved — the seed is the incident report, replayable forever, shrinkable to its essence, fixable with certainty. The candour clause, spoken as its practitioners speak it: the discipline is expensive — the simulator must own every source of nondeterminism, which constrains the languages, the libraries, the very style of the code; it is bought at the foundation or not at all, which is why the famous exemplars are storage engines and kernels rather than retrofits. They built the universe twice: once to run, and once — seeded, obedient, replayable — so that failure, the one thing this whole course has treated as the world's caprice, becomes a reproducible laboratory specimen. The field's final answer to the Post's malice was to hire the Post, on a fixed salary, with a logbook.

Remark 11.5 (the testing portfolio)*.* Four instruments, four jurisdictions: the model checker audits the *design* (exhaustive interleavings on small instances; the thirty-five-step bug); the fault-injection harness audits the *implementation* (generated operations, a nemesis schedule, history checking against Chapter 7's definitions); chaos audits the *estate* (hypothesis, bounded blast radius, customer metrics, abort handle, business hours); deterministic simulation audits all three at once for systems built inside it from the foundation (every nondeterminism owned; seeds as permanent incident reports; rare paths promoted by buggify). None substitutes for another; the failure path that has never run does not exist.

The postscript beneath them all: **observability** — because one cannot repair what one cannot narrate. When the estate misbehaves at three in the morning, the first artifact anyone needs is the story: this request entered here, fanned out to these services, stalled in that queue, timed out there, retried, and died at this hop. *Distributed tracing* — a request identifier propagated through every hop, each hop contributing a span (who I am, whom I called, when I started, when I finished), stitched afterward into the request's family tree — is that story, mechanized; sampled in practice, since recording every request's biography would be a workload of its own, with the craft in keeping the interesting biographies — the slow, the failed, the shed — at full fidelity while the happy path is counted and released. Its ancestry needs no prompting: a trace is *happens-before made visible* — Chapter 2's causal roads, instrumented and drawn — the season's oldest dividend, collected at the hour it is worth most. The three instruments hold a division of labour worth stating once cleanly: the metrics say *that* something is wrong, cheaply and continuously, across everything; the traces say *where*, following the one afflicted request through the maze; the logs say *what*, in the afflicted component's own words; and the deadline stamps, retry counts, and breaker states of this chapter's machinery, if faithfully recorded into all three, say *why* — which is the only question the postmortem is actually obliged to answer. An estate that cannot produce that narrative does not have an observability gap; it has scheduled its next metastable outage without an author for the postmortem. And the narrative pays here specifically: every loop of Part Four is visible in traces before it is fatal in dashboards — the retry counts climbing, the deadline expiries clustering at one tier, the breaker states flickering — the storm announces itself in the causal record for whole minutes before the goodput curve folds, to any estate that has arranged to be listening.

11.8 The Whiteboard

For the design review.

1. Every retry carries a budget and jitter, backs off exponentially, and rides an idempotency key — four adjectives or it is a weapon. The amplification arithmetic is exponential in stack depth, and the stack was assembled by teams who never met; the budget is the only clause that binds them all.

2. Every queue carries a bound, and every bound a shedding path — designed, tested, and fast, before the happy path is tuned. A queue is a debt instrument; an unbounded queue is an unbounded promise;

and a slow yes is worth less than an instant no.

3. Every request carries a deadline, and the deadline travels — decremented per hop, honoured at every tier, with expired work dropped where it stands. Effort spent past the deadline is counterfeit currency, and the deepest tier pays it.

4. Hunt the loops: list every message the system sends at a hundred and ten percent of capacity that it would not send at ninety — retries, health checks, elections, refills, reconnections — and put a governor on each. Capacity moves the crest; only loop surgery disarms the second equilibrium. Bias health checks toward mercy under load, or membership churn will finish what the load began — and when the fire comes anyway, recovery re-admits gradually, below the ignition point, or dawn's first act relights the night's work.

5. Inject the Post's malice on schedule, before it volunteers: fault injection with history checking for the protocols, chaos with blast-radius discipline for the estate, a model checker for the designs that must not be wrong, and — where the investment is warranted — a seeded universe in which every failure is a specimen. The four are a portfolio, not a menu: the checker audits the design, the harness the implementation, the chaos the estate, and the simulation all three at once. The failure path that has never run does not exist; it merely has not yet cost anything.

The register. All payments, nothing issued — the first such chapter of the season, itself the signal that the season is landing. Paid: #2, crashed-versus-slow — issued in Chapter 1, priced and dialled here, the doppelgänger retired with full honours; #10, the suspicion services — priced in heartbeats, lag, and correlated panic; #19, the Jepsen method — industrialized into the trade's own pipelines and its chaos-engineering descendants; #22, Chapter 9's storms and stampedes — christened metastable, anatomized in the set piece, disarmed by loop surgery; #23, the retry's dark side — convicted by arithmetic, reformed by four adjectives. The register closes at the finale, where #24 (stragglers and backup tasks, racing the unluckiest machine) and #4's remainder (commutativity's second half, the gradients) fall due. Episode word count: 10,002 — at floor; the half-draft drill required its usual passes.

11.9 Problems

Tier I — Calisthenics

Problem 11.1 (the budget, sized)**.** A tier makes ten thousand first attempts per second; the dependency's steady error rate is half a percent, with incidents to twenty percent. (a) Size the retry budget so steady-state retries are never throttled but incident amplification stays below five percent of first-attempt volume... state the tension and resolve it with a number and a sentence. (b) Compute bottom-tier volume in a four-tier stack at your budget versus uncapped three-attempt retries during a total brownout. (c) State what the budget does to customer experience during the incident, honestly, and why that is the correct trade.

Problem 11.2 (reading the dial)**.** Heartbeats every half second with occasional one-second stragglers and a once-per-hour two-second pause. (a) Choose phi thresholds for: routing away (cheap, reversible), failover (expensive, fenced); justify each with the false-alarm rate. (b) The network enters a congested afternoon and inter-arrivals double. What happens to the silences' phi under a re-estimated F, and what would a fixed eight-hundred-millisecond timeout have done? (c) Fifty peers go silent together. What should fleet-level logic conclude, and why can no per-peer dial conclude it?

Problem 11.3 (the meter runs)**.** A page has a two-second budget; tiers spend two hundred, three hundred, and four hundred milliseconds; queues add what they add. (a) Compute each hop's passed budget with a

fifty-millisecond margin per hop. (b) Storage dequeues a request with ten milliseconds remaining and a two-hundred-millisecond median service time: what should it do, and what metric records it? (c) A reviewer proposes padding every timeout "to be safe"; explain, via the seance clause, what padding the deepest tier's timeout beyond the top's budget purchases.

Problem 11.4 (classify before retrying)**.** For each response, retry or not, and under what schedule: (a) connection refused; (b) malformed request; (c) overloaded, retry after two seconds; (d) deadline exceeded upstream; (e) a timeout on a non-idempotent payment call with no idempotency key (careful, sir).

Tier II — The Real Thing

Problem 11.5 (serve-stale, with bounds)**.** Design the aunt's-notice cache: target hit rate, jittered expiry, single-flight refresh, serve-stale window. Given a one-second origin refresh and a ten-thousand-per-second read rate: (a) bound the origin load in steady state and during a refresh; (b) bound worst-case staleness seen by any reader, and state it as a Chapter 7 contract (eventual-plus-what); (c) add negative caching for a deleted notice and show the resurrection window it must respect (Ch. 6's tombstone clause); (d) state what fails if the refresh lock has no timeout, and the repair.

Problem 11.6 (sabotage: the merciless probe)**.** A fleet's health check calls the real read path with a three-hundred-millisecond timeout, three strikes and out; the orchestrator drains and replaces strikers. A surge pushes median latency to four hundred milliseconds fleet-wide — slow, correct, and profitable. Exhibit the collapse loop turn by turn: strikes, drains, load concentration, more strikes — to the empty fleet; identify the loop gain and the exact clause that makes it exceed one; then repair three ways (mercy bias under load; disruption budget; probe measuring customer error rather than latency) and state which combination survives the surge *and* still catches a genuinely dead machine.

Problem 11.7 (the breaker, tuned)**.** A dependency's contract: five hundred milliseconds at the ninety-ninth percentile, half a percent errors. Tune Box 27: window, threshold, cooling, probe policy, scope — each with a sentence of justification against two adversaries: a real outage (the breaker should open within seconds) and a half-percent bad Tuesday (it should not open at all). Then exhibit what a fleet of un-jittered breakers does to the dependency at cooling expiry.

Problem 11.8 (the amplification audit)**.** A described stack: edge (three attempts), API (three), service (two), storage client (three), each with its own timeouts, no budgets, no jitter, and an edge timeout shorter than the sum beneath it. (a) Compute worst-case storage amplification. (b) Identify the seance: which tiers labour for departed callers, and why the edge's short timeout makes it worse. (c) Prescribe the constitutional reform in four lines (owner layer, budgets, jitter, deadline propagation) and recompute the worst case.

Tier III — For the Ambitious (starred)

Problem 11.9 (⋆ the tipping model, refined)**.** Extend Prop. 11.6's toy: served rate $\min(T, C)$; unserved work retried once after timeout with probability q (abandonment $1-q$); latency rising with utilization so that above capacity, timeouts fire for served work too at a rate you model. (a) Derive the demand fixed points and their stability. (b) Exhibit the hysteresis: the range of L for which both healthy and degraded equilibria exist. (c) Show how a retry budget (q capped) and admission control (effective L capped) each eliminate the degraded branch, and which does so at lower cost to good traffic. *(Hints only.)*

Problem 11.10 (⋆ the appendix's exercises)**.** Using the appendix: (a) run the checker on TCommit and record the state count; (b) add the invariant that no resource manager is committed while another is aborted,

and confirm it holds; (c) break the protocol — allow commit without unanimous prepare — and read the checker's counterexample trace; (d) sketch how TwoPhase (the coordinator edition) exhibits Chapter 8's blocking as a liveness failure under a crashed-coordinator model. *(Hints only.)*

11.10 Hints

Hint (Problem 11.1)**.** Steady retries are half a percent of volume — any budget above that is invisible in fair weather; the incident then sets the cap's value, not the weather.

Hint (Problem 11.6)**.** The gain: each drained node moves its share of a fleet-wide surge onto survivors already past the probe's timeout; strikes per drain exceed one when latency is load-coupled — write the inequality.

Hint (Problem 11.9)**.** (b) Sweep L upward on the healthy branch and downward on the degraded; the fold is where the branches overlap. (c) Budgets lower γ; admission lowers effective L; compare shed volume at equal stability margin.

Hint (Problem 11.10)**.** (c) The checker's trace is a numbered sequence of states; read it bottom-up and name the step where unanimity was needed.

11.11 Solutions

Tier I in full (tersely); Tier II in full — Problem 11.6 is the sabotage certification; hints only for Tier III.

Solution (to Problem 11.1)**.** (a) Ten percent: twenty times the steady half-percent (never throttled in fair weather), yet capping incident retries at a thousand per second against ten thousand first attempts — five percent... the stated target requires half that; either tighten to five percent (still ten times steady) or accept ten — the sentence: "the budget is set between the weather and the storm, closer to the weather." (b) At $b = 0.1$ over four tiers: $(1.1)^4 \approx 1.46$ times top volume; uncapped $3^4 = 81$. (c) During the incident, most failed requests are not retried: customers see errors sooner — honestly the point, since the alternative was errors later plus a downed dependency for everyone; the budget converts a building fire into a room fire.

Solution (to Problem 11.2)**.** (a) Routing at phi around one (one false route in ten — harmless); failover at three or higher (one false election per thousand silences, each survivable by fencing; with the two-second pause on record, place the threshold's implied silence beyond it). (b) Re-estimated F widens; the same absolute silence maps to lower phi; the detector stays quiet through the congestion — the fixed timeout would have declared half the fleet dead at the worst hour. (c) Correlated silence is a network or switch event: freeze membership changes (the storm brake), do not fifty-fold the failover machinery; a per-peer dial models one rhythm and cannot represent common cause.

Solution (to Problem 11.3)**.** (a) Two thousand minus two hundred minus fifty: one thousand seven hundred fifty to the service tier; minus three hundred and fifty: one thousand four hundred to storage; the queues consume from whatever remains at dequeue. (b) Reject instantly: expected completion exceeds remaining budget twenty-fold; the metric is deadline-expired-at-dequeue, the queue-sizing signal of Box 28. (c) Padding the deepest tier past the top's budget buys work performed for the departed: the seance, institutionalized — deeper timeouts must *shrink*, never grow.

Solution (to Problem 11.4)**.** (a) Retriable, jittered backoff within budget — likely a restart or a bad host; hedge to another replica sooner. (b) Permanent: the answer will not change; retrying is a drumbeat. (c) Obey: wait the stated two seconds, jittered, budgeted; the server knows its condition. (d) Do not retry: the caller is gone; count the grave. (e) Neither retry nor not-retry is safe: without a key, a retry risks double payment, no retry risks none — the correct act is to *query* (was instruction seventeen executed?) or fail to a reconciliation path; the deeper repair is the key, after which (e) becomes (a).

Solution (to Problem 11.5)**.** (a) Steady state: one refresh per expiry per key — with single-flight, at most one origin call per second for the hot key, against ten thousand reads; during refresh, concurrent misses wait or take stale: origin load unchanged. (b) Worst staleness: expiry jitter ceiling plus serve-stale window plus one refresh time — state it as "reads may lag writes by at most *s* seconds, always": bounded staleness, Chapter 7's honest tier. (c) The negative entry must outlive anti-entropy's resurrection window for the deletion (the tombstone clause), else the cache re-learns a deleted notice from a stale replica. (d) A crashed refresher holding the single-flight lock starves the key into permanent staleness; the lock carries a lease (Chapter 5, as ever) and a second flight departs on expiry.

Solution (to Problem 11.6 — the certification)**.** The loop, turn by turn, fleet of ten at a surge that puts median latency at four hundred milliseconds.

(1) Probes (three-hundred-millisecond fuse) begin failing fleet-wide — every node slow-but-correct, none dead.

(2) First striker drained: its tenth of the surge lands on nine survivors; their latency rises further above the fuse.

(3) Strikes accrue faster on the more-loaded survivors: each drain raises per-survivor load by a growing increment — the gain exceeds one as soon as one drain produces, on average, more than one new striker, which load-coupled latency guarantees here from the first drain.

(4) The orchestrator, constitution in hand, drains the fleet to zero. Customer availability: zero. Machines dead at any point: zero. The health system executed the healthy.

Loop and gain named: drains concentrate load; concentrated load fails probes; failed probes drain. Repairs: mercy bias (raise the fuse or require sustained breach when fleet-wide latency is elevated) breaks step three's coupling; the disruption budget (at most one drain per interval) caps the gain below one regardless of coupling; the customer-error probe removes the false signal entirely but, alone, would also excuse a machine serving errors slowly — the surviving combination is the budget plus error-weighted probes, which still catches the genuinely dead (they fail *error* checks too, and the budget merely paces their removal). Certified fatal as shipped: total self-inflicted outage from a profitable surge, no failed hardware anywhere in the building.

Solution (to Problem 11.7)**.** Window ten seconds sliding; threshold: error rate above ten percent *and* a minimum call count (twenty), so a quiet period cannot open on three unlucky calls; cooling five seconds with full jitter; probe: single call, real endpoint, budget-stamped; scope per endpoint. Against the outage: ten percent of a busy window crosses within a second or two — open fast. Against the bad Tuesday: half a percent never approaches ten — closed all day. Unjittered fleet at cooling expiry: ten thousand probes in one instant — the herd, Part Five, self-assembled; with full jitter, two thousand per second across five seconds, a drizzle.

Solution (to Problem 11.8)**.** (a) Three times three times two times three: fifty-four attempts at storage per edge request, worst case. (b) The edge abandons before the tiers beneath finish even one honest attempt

each; everything below the first tier labours for the departed on every edge timeout — and the retries below continue after abandonment, so the seance outlives the customer by the sum of the lower timeouts. (c) Edge owns retries (three, budgeted at a tenth, full jitter); all lower tiers: one attempt, fail fast, deadline-checked. Worst case at storage: three — one per owned attempt — and only while the deadline lives.

11.12 Appendix (starred): TLA+ for the Curious

Per the season's Decision 8, the volume's one formal-methods appendix lives here and nowhere else. The specimen is Lamport's own teaching specification of transaction commit — the *abstract* problem of Chapter 8, before any coordinator exists — annotated for a reader who has never opened the toolbox.

The specification, annotated. A specification is a state machine: variables, initial states, permitted steps. TCommit has one variable, the array of resource-manager states.

Protocol — The TCommit specification (after Lamport's *Specifying Systems* lineage)

1: **constant** RM — the set of resource managers, for checking a small concrete set, three names
2: **variable** $rmState$ — a function from RM to {"working", "prepared", "committed", "aborted"}
3: **Init:** $rmState$ = every manager at "working"
4: **Prepare(r):** enabled when $rmState[r]$ = "working"; effect: r moves to "prepared"
5: **canCommit:** true when every manager is "prepared" or "committed"
6: **DecideCommit(r):** enabled when $rmState[r]$ = "prepared" and canCommit; effect: r to "committed"
7: **notCommitted:** true when no manager is "committed"
8: **DecideAbort(r):** enabled when $rmState[r]$ is "working" or "prepared", and notCommitted; effect: r to "aborted"
9: **Next:** some r takes one of the three steps
10: **Invariant (TCConsistent):** no pair (r, s) with $rmState[r]$ = "committed" and $rmState[s]$ = "aborted"

Read line 6 twice: a manager may commit only when *all* are prepared-or-committed — unanimity as an enabling condition, the veto of Def. 8.1 rendered as a guard; and line 8's notCommitted guard is stability: no abort after any commit. The specification says *what* transitions are lawful and nothing about messages, coordinators, or crashes: it is the problem, not a protocol.

Running the checker. Install the TLA+ toolbox or the command-line tools; transcribe the specification (the original text of TCommit ships with the tools' examples); give the checker a model: set RM to three concrete names, name TCConsistent as the invariant, and run. The checker enumerates every reachable state — a few dozen here — and reports the invariant satisfied. Then perform Problem 11.10(c): delete canCommit from DecideCommit's guard and run again; the checker returns a numbered trace — Init, a Prepare, a premature commit, an abort elsewhere — the shortest lawless history, printed like a police report. That trace, sir, is the entire pedagogy: the tool does not argue, it *exhibits*. The two-phase protocol proper (TwoPhase, also shipped with the examples) adds a coordinator and messages and can be checked to *implement* TCommit; add a crashed-coordinator model and the blocking of Thm. 8.2 appears as a liveness failure — the checker, unlike the folklore, will show you the exact stuck state. The thirty-five-step bugs of the Amazon report were found with precisely this workflow, scaled up and made routine.

11.13 The Reading

Bronson, Aghayev, Charapko, and Zhu, "Metastable Failures in Distributed Systems," HotOS 2021. Short, incident-fed, and the rare paper that names a thing sir has already survived; read with your own postmortems open, and Def. 11.3 beside it.

Newcombe, Rath, Zhang, Munteanu, Brooker, and Deardeuff, "How Amazon Web Services Uses Formal Methods," CACM 2015. The thirty-five-step bug, the cultural argument, and the honest costs; the appendix here is its on-ramp.

The Amazon Builders' Library: "Timeouts, Retries, and Backoff with Jitter" (Brooker). The doctrine in the house style, with the graphs Prop. 11.3 only sketches; read alongside the entry on avoiding fallback and the one on load shedding.

Saltzer, Reed, and Clark, "End-to-End Arguments in System Design," TOCS 1984. Eight pages that outrank most shelves; the gallery of Rem. 11.4 in the original prose.

The shelf. Hayashibara et al., "The φ Accrual Failure Detector" (SRDS 2004). The DynamoDB service disruption summary (September 2015) — the set piece's primary source, a model of the postmortem genre. Huang et al., "Gray Failure: The Achilles' Heel of Cloud-Scale Systems" (HotOS 2017) — differential observability named. Dean and Barroso, "The Tail at Scale" (CACM 2013) — hedges and the tail arithmetic. The FoundationDB testing talks and paper — the universe built twice. And Kingsbury's Jepsen analyses, rereadable now as the industrialized method of Rem. 11.5.

Chapter 12

Capstone: The Distributed Systems You Already Run

Companion to Episode Twelve, and the season's last. The costume comes off: the training estate — the largest, most tightly coordinated computation in the species' history — read as the distributed system the reader has operated all along. The dictionary, entry by entry, each alias resolved to a named chapter; the ring all-reduce, parcels round a circle, proven bandwidth-optimal against the two-ledger floor; the season's oldest debt paid in gradients, with the rhyme honestly labelled; the three geometries; failure on a schedule and the Young–Daly interval, derived as a guided problem; the mercurial cores; skew with its profit line; and the grand audit — the register ruled, closed, and tabled here as permanent equipment — before the consolidated reading closes the volume. Two theorems, and no new impossibilities: the finale's work is recognition.

12.1 Part One — The Dictionary

The claim on which the whole finale balances, weighed before it is granted: the largest coordinated computations in history are not the banks, nor the search engines, nor the exchanges whose machinery the preceding eleven chapters dissected. They are training runs. A frontier run coordinates tens of thousands of accelerators across thousands of hosts and multiple halls; it moves gradient traffic measured in terabits per second, sustained, for millions of consecutive rounds; and it must survive — on the arithmetic Part Five does below — hardware failures on a daily schedule without losing the plot of a computation running since spring. The banks are marvels of care over modest traffic; the search engines, marvels of scale over forgiving queries, where a dropped result is a shrug. The training run alone demands both at once: the scale of the largest estates and the lockstep coherence of the tightest, with a single logical object — the model — that every machine must agree about, continuously, or the investment diverges into noise. You have been chairing this seminar your entire career, and nobody ever showed you the minutes. And the disguise held for a reason worth naming: the machine-learning trade hired its infrastructure vocabulary from a different lineage — high-performance computing, with its collectives and ranks and barriers — a tradition raised on reliable, private, exquisite networks, which never needed this course's paranoia until, at modern scale on commodity failure schedules, it suddenly did. Two dialects, one mathematics; the dictionary performs the introduction the two communities were too busy to arrange for themselves.

The entries, then, each with its chapter attached like a pressed flower; the full mapping is tabled in §12.8, permanent equipment.

- **A training step is a round** — a barrier-synchronized iteration, the fleet advancing in lockstep through numbered rounds: Chapter 4's rented synchrony, assumed wholesale, millions of times over,

and paid for in the renter's eternal coin — the round lasts as long as its slowest participant, which is why the straggler is this estate's national enemy and Part Five's whole business. The model of computation Chapter 3 used as a thought experiment is the estate's Tuesday.

- **Gradient accumulation is batching** — the oldest trade in the ledger: fold several rounds of local work into one round of agreement, amortizing the barrier and the parcels, as group commit did in Chapter 5's log and the batched sequencers did in Chapter 8; the exchange rate — latency for throughput — has never once varied.
- **An all-reduce is agreement on a sum** — strictly weaker than consensus, the dictionary's first fine distinction and the subject of a formal card below.
- **A checkpoint is a crash-recovery snapshot** — the Chandy–Lamport spirit's third appearance, after Chapter 2's theory and Chapter 10's streaming costume, with the alignment problem pre-solved: the barrier between steps is a global quiescent instant, so the season's hardest photograph — of a river that would not hold still — is here a formality, this river agreeing to hold still sixty times a minute; the interesting questions move to *how often* (Part Five's arithmetic) and *how cheaply* (Part Five's engineering).
- **A straggler is Chapter 10's unluckiest machine**, reborn on silicon: the accelerator a few percent slow — thermals, a failing memory module, a noisy neighbour on the interconnect — setting the pace for ten thousand peers; the backup-task remedy planted there comes due here, with interest.
- **Parameter staleness is replication lag**: the worker computing gradients against weights three versions old is Chapter 6's stale follower, serving reads behind the primary; every bounded-staleness convergence argument is a session-guarantee negotiation in optimization clothes — Chapter 7's eventual-plus-what, with the *what* denominated in steps of drift.
- **Elastic training is a view change**: members join, members fail, the job continues — Chapter 4's reconfiguration and Chapter 5's dragons at rack scale, with the membership epoch stamped on every collective so that a straggler from the old view cannot corrupt the new. The fencing token in its final uniform: the season's most-travelled instrument, five chapters, five costumes, one idea — authority must expire visibly before it may be reassigned safely.
- **The data pipeline is the log** — Chapter 10 entire: the shuffled, sharded, replayable stream of training data, read by cursors, rewound for reproducibility — the input leg of the recovery trinity, holding up the largest computations on earth.
- **The job scheduler is Chapter 9's placement church**: which accelerators host which shards of which job, against failure domains and interconnect locality — the seating chart drawn at gigawatt scale, with the same commandment about not seating a parliament's replicas on one switch.
- **The feature store is a materialized view** — Chapter 10's inside-out doctrine sold as a product: one definition, one log, every consumer a cursor.
- **The training metrics are the watermark**: the loss curve is the computation's own confession of progress and health, monitored the way Chapter 10 monitored time and Chapter 11 monitored everything — the invariant that reports, before any machine confesses, that something in the estate has begun to lie.

Ten chapters, a dozen entries, and the dictionary could run to fifty. The cumulative effect is the point — somewhere around the fifth entry it becomes clear that *there is nothing else in the building*: the field is not adjacent to distributed systems but made, joist by joist, of this course, and the translation is not metaphor

but identity — the same mathematics under different invoices. The practical privilege follows at once: the dictionary makes forty years of other people's postmortems the reader's own. Every lesson the databases and the queues and the caches purchased with outages now reads directly onto the cluster, name by translated name; an estate that treats its stragglers like Chapter 11's timeouts and its checkpoints like Chapter 10's photographs inherits the corrections without paying for the incidents.

The first fine distinction deserves its formal card at once, because the whole economy of Part Two hangs on it. Chapter 3 forbade deterministic asynchronous consensus because consensus must *arbitrate* — pick one value, exclude the rest. A sum excludes nobody: every contribution is folded in, order irrelevant, nothing to arbitrate — no race, in Chapter 6's language, and the race test was always the border. Weaker demands, cheaper machinery: the all-reduce runs at wire speed precisely because it is not a parliament — and the estate keeps an actual parliament on retainer, the job controller, for the questions that *are* decisions: who is a member, which checkpoint is blessed, who leads. Two agreement products, two prices, one building.

Definition 12.1 (all-reduce)**.** Given vectors $g_1, \ldots, g_n$ at nodes $1, \ldots, n$ and an associative, commutative reduction (element-wise sum throughout), the *all-reduce* terminates with every node holding $g_1 + \cdots + g_n$. It is an agreement on a *value determined by all inputs symmetrically*: no arbitration, no exclusion, no winner — by the race test (Chapter 6), a problem strictly below consensus, and priced accordingly.

12.2 Part Two — Data Parallelism, and the Parcels Passed Round

The workhorse first: *data parallelism*, synchronous stochastic gradient descent across the fleet. Every worker holds a complete copy of the model — replication, Chapter 6, at full factor; each round, every worker computes a gradient on its own shard of data — partitioning, Chapter 9, of the work rather than the parameters; and then the fleet must agree on the *average* gradient before anyone may step. The average is the estate's one point of true coupling, and it is worth pausing on: the mathematics wants the gradient of the whole batch, which is exactly the mean of the workers' shard-gradients — so the fleet's agreement object is a sum, arrived at by pure accumulation, no arbitration, no winner (Def. 12.1); and once agreed, every replica applies the identical update to identical weights, and the copies stay identical *by determinism rather than by reconciliation* — Chapter 5's state machine replication, verbatim. One logical model, ten thousand physical copies, coherent for months without a single anti-entropy repair: the determinism diet's masterpiece, recorded formally with its fine print.

Remark 12.2 (synchronous data parallelism as SMR)*.* With deterministic updates and an agreed summed gradient per round, all replicas of the model evolve identically from an identical start: state machine replication (Chapter 5), with the all-reduce supplying the agreed input and determinism supplying the coherence — replicas equal by construction, not by reconciliation; no anti-entropy, ever, for months. Floating-point non-associativity is the fine print: the reduction *order* must itself be fixed (Box 29 fixes it) or the replicas drift in the last bits and reproducibility dies by topology (Chapter 5's diet, enforced down to the rounding).

The naive agreement is a *parameter server*: ship every gradient to a central estate, sum there, ship the result back — Chapter 9's directory church, and workable; Li and colleagues' 2014 paper made it an architecture, the server tier itself sharded, each server owning a range of the parameters, workers pushing and pulling shards in parallel. Price it with the course's arithmetic: a thousand workers, each shipping a full gradient up and a full model down per round, means the server tier moves two thousand ledgers per round while each worker moves two — the fan-in is the ceiling, the ceiling is bought in server bandwidth, and it does not rise as workers join unless the server estate grows in proportion: a tax collected per round, forever. The trade wanted the sum computed *by the fleet itself*, with no central post office — and the answer came

from an unglamorous corner: high-performance computing had been passing parcels around rings since before the web. The terms for the mathematics ahead, fixed once, in the box.

Model of this chapter

For the ring (Thms. 12.1–12.2). n nodes in a directed ring; each holds a vector of size G (the gradient); links carry β bytes per second each way, full duplex, all links simultaneously usable; per-message latency α acknowledged in remarks, with the bandwidth regime (G large) primary. The goal: every node holds the element-wise sum of all n vectors.

For checkpointing (Thm. 12.3). Failures strike the job as a memoryless process with mean time M between failures (fleet-wide); a checkpoint costs δ of wall time; on failure the job rewinds to the last completed checkpoint, losing on average half an interval; $\delta \ll M$ (the regime of interest; the exact treatment is cited in the reading).

For staleness (Rem. 12.4). Statements only, with pointers, per the season's honesty clause: the convergence analyses live in the optimization literature and are not re-proved at this desk.

The set piece, the company in a circle: Bingo, Tuppy, Gussie, and Barmy, four machines in a ring, each holding its own gradient — a ledger of four hundred pages, say — and each ledger cut into four parcels, one per machine; formally, the gradient of size G cut into n slices, the ring's standing convention (Box 29). The algorithm has two acts. *Act one, reduce-scatter*, three passes, in which the running sums travel: each machine sends one parcel — a hundred pages — to its clockwise neighbour, all four links busy at once, nobody idle, no central post office; each recipient *adds* the arriving parcel to its own corresponding pages and forwards the sum onward on the next pass, so that after three passes each machine holds one parcel *complete* — the full four-machine total of those hundred pages, the sum, scattered. *Act two, all-gather*, three passes more, in which the totals circulate: each machine sends its finished parcel round the ring, copying, not adding, every link still humming, until everyone holds all four completed parcels — the entire summed gradient, everywhere, and every machine steps its copy of the model in the same instant, which is the barrier's release and the round's end. Two tricks are easy to admire past. First: at no moment did any machine hold, or need to hold, anybody's full ledger but its own — the sums travelled as parcels, each link carrying a fraction, so there is no place where everything converges: the convergence itself was partitioned — Chapter 9's doctrine applied not to data at rest but to the agreement in flight. Second: every link worked every pass, and that full employment is precisely where the economy comes from.

Protocol — Box 29. Ring all-reduce (the classical collective, checked against the standard formulations)

1: **setup:** nodes $0 \dots n-1$ in a ring; each vector cut into n parcels; fixed parcel-accumulation order (reproducibility)
2: **reduce-scatter,** $n-1$ **passes:** in each pass, send the designated running-sum parcel to the successor; add the parcel arriving from the predecessor
3: **all-gather,** $n-1$ **passes:** circulate the finished parcels; copy, do not add
4: **result:** every node holds the full sum; traffic $2(n-1)/n$ ledgers per node; every link busy every pass
5: **regimes:** large G: rings; small G: trees (latency); mixed fleets: hierarchical rings, heavy traffic on short wires

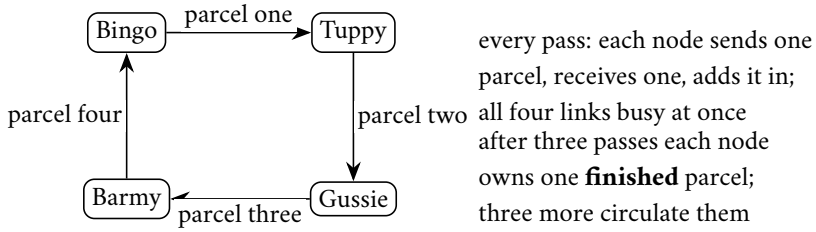


Exhibit 12.1 (the parcels, mid-pass). One reduce-scatter pass of Box 29 on four nodes: running sums travel clockwise, every wire working. Six passes in all; traffic per node six of the four-hundred-page ledger's hundred-page parcels — one and a half ledgers, approaching two as the ring grows (Thm. 12.1).

Now the arithmetic, carried to its punchline: each machine sent one parcel per pass for six passes — six hundred pages against a ledger of four hundred; with ten machines, parcels of a tenth and eighteen passes' worth — one point eight ledgers; with a hundred, one hundred ninety-eight hundredths; with ten thousand, within a whisker of exactly two. *Two ledgers, regardless*: total traffic per machine approaches twice the size of the gradient no matter how many machines join the ring, every link busy every pass — no bottleneck machine, no idle wire, the sum of ten thousand contributions computed for the postage of two.

Theorem 12.1 (ring all-reduce cost)**.** *Cut each vector into n parcels of size G/n. The ring algorithm of Box 29 completes in $2(n-1)$ passes, each node sending and receiving one parcel per pass; total traffic per node $2(n-1)G/n < 2G$, and wall time in the bandwidth regime*

$$T \;=\; \frac{2(n-1)}{n} \cdot \frac{G}{\beta},$$

approaching $2G/\beta$ from below as n grows: the fleet size cancels. Every link carries traffic in every pass — no idle wire, no bottleneck node.

Proof of Theorem 12.1. Index parcels and nodes modulo n. *Reduce-scatter:* in pass p (for $p = 1, \ldots, n-1$), node i sends to node $i+1$ the running sum of parcel $i-p+1$ accumulated so far, and adds the parcel arriving from node $i-1$ to its local copy. Induction: after pass p, node i holds, in parcel slot $i-p$, the sum of that parcel's copies from nodes $i-p, \ldots, i$ — $p+1$ contributions. After pass $n-1$, node i holds the complete n-way sum of parcel $i+1$ (indices mod n): each node owns one finished parcel. *All-gather:* $n-1$ further passes circulate the finished parcels unchanged; after pass $n-1$ every node has all n. Each node sent exactly one parcel of size G/n in each of $2(n-1)$ passes: traffic $2(n-1)G/n$; passes are simultaneous on all links (each node sends to its successor in every pass), so wall time is traffic over β, as displayed. The reduction order within each parcel slot is fixed by the ring's geometry — node $i+1$'s finished parcel always accumulated in the order $i+1, i+2, \ldots$ — giving Rem. 12.2's reproducibility clause for free. □

The punchline is not the ring's cleverness but the problem's own floor, and the spine of the argument fits in one sentence: every machine must *receive* the parts of the sum it did not compute — very nearly one full ledger, its own contribution being one part in the fleet — and every machine's contribution must *leave* it, another ledger outbound, for its pages to reach the total everyone must hold. One ledger in, one ledger out, at minimum, for any algorithm whatsoever that ends with everyone holding everything; the ring's achievement is merely — merely! — to reach the floor while keeping every link busy every pass.

Theorem 12.2 (the two-ledger floor)**.** *Any all-reduce algorithm in the model box has some node whose total traffic is at least $2(n-1)G/n$: at least $(n-1)G/n$ received — the summands' information it does not hold locally, without which the sum at generic inputs is undetermined — and at least $(n-1)G/n$ sent — its own contribution's share of what all other nodes' outputs require. The ring (Thm. 12.1) meets the floor exactly: bandwidth-optimal.*

Proof of Theorem 12.2. Fix any correct algorithm and consider generic inputs (no cancellations; formally, inputs algebraically independent over the reduction). *Receiving floor:* node i's output is the full sum, which at generic inputs is not a function of g_i alone; every other node's contribution must be represented in what i receives. The summands arriving at i may arrive pre-combined — that is the algorithm's freedom — but combination does not compress below the size of one vector's worth of information about the other $n-1$ contributions to each element: whatever i receives must determine, with g_i, all G elements of the sum, and at generic inputs the received data must carry at least the G-element combination of the others' values — of which i can locally reconstruct nothing. A counting argument on element shares (each element's foreign part is $(n-1)$ of n equal contributions) gives at least $(n-1)G/n$ received by *some* node under any balanced scheme, and at least that at the maximum node in general (averaging: total foreign information across nodes is $n \cdot (n-1)G/n$, so the mean per node is $(n-1)G/n$; the maximum is at least the mean). *Sending floor:* symmetrically, g_i appears in every other node's output; the information leaves i only through its sends, pre-combined or not, and the same averaging yields at least $(n-1)G/n$ sent at the maximum node. Summing the two floors at the averaging node gives the bound; the ring achieves it at *every* node, hence optimal. (The desk notes the argument's honest character: an information-counting bound over generic inputs in the standard style of the collectives literature, not a bit-level coding theorem; the reading carries the formal treatments.) □

The courier that runs this ceremony on real racks is NCCL — spoken *nickel*, as the industry insists and the lexicon has recorded since Chapter 1 — a library whose entire profession is this Part conducted at nanosecond stakes: discovering the fleet's topology, choosing among rings and trees, pipelining the parcels so the wires never breathe, and doing it all again when the membership epoch turns. Scale the arithmetic to the frontier once, for awe's sake: a large model's gradient weighs hundreds of gigabytes; the two-ledger bound prices each round at roughly twice that per machine; and the loop demands a round every few seconds, sustained, day and night for a season — which is why the interconnect is the training estate's costliest citizen after the accelerators themselves, and why an algorithm that reaches the bandwidth floor is not an optimization but the difference between the computation existing and not.

Remark 12.3 (latency, trees, and hierarchy). With per-message latency α, the ring pays $2(n-1)\alpha$ in serialized hops — negligible for large G, ruinous for small messages, whence tree-shaped all-reduces ($O(\log n)$ depth, a constant-factor bandwidth premium) for the latency regime, and hierarchical composites — rings within pods, exchanges between — placing the heavy traffic on the short wires (Chapter 9's oldest commandment). Problem 12.6 budgets one.

One honest caveat completes the set piece: the ring is a barrier in miniature — every pass waits for the slowest link and the slowest machine, so its magnificent bandwidth is bought at the price of maximal exposure to Part Five's straggler; the parcel arithmetic assumed everyone keeps step, and keeping ten thousand machines in step is the discipline the rest of the chapter must fund. Was the parameter server, then, merely the wrong church? Not at all — the comparison is Chapter 9's fan-out sentence at silicon speed. The ring buys bandwidth optimality and pays in *synchrony*; the server buys *asynchrony* — workers push and pull at their own pace against the central copy — and pays in fan-in bandwidth and in the staleness Part Three prices. And the server church keeps one virtue the ring cannot counterfeit: *elasticity*. A ring is a topology — a member's death breaks it, a member's arrival re-forms it, each transition a view change with a pause; the server pattern is a shopfront — workers come and go as customers, unannounced, and the estate barely glances up, which is why the elastic and the preemptible provinces still worship there. Topology is destiny: the collective for the tight synchronous core, the server pattern where elasticity and asynchrony matter more than the last drop of wire — and modern estates, predictably, deploy both, per layer.

12.3 Part Three — Asynchrony's Bargain, and the Season's Oldest Debt

The synchronous loop's tax is the barrier; the tempting escape is to stop waiting: let each worker read the current weights, compute its gradient, and apply it *whenever it arrives*, against whatever weights now stand — no barrier, no round, no agreement at all. Every instinct this course has trained rebels: updates applied against versions they were not computed from, interleavings no one ordered — the lost-update crime scene of Chapter 6, re-enacted at every step. Picture it staged, because the image is the argument: a single great ledger open on the counter and every teller in the bank writing corrections into it simultaneously — no queue, no clerk of order, pens colliding. The astonishment — published as *Hogwild* by Recht and colleagues in 2011, the name a confession — is that it works: run the updates with no locks whatsoever, let them collide, and under honest assumptions about sparsity and step size, convergence survives at nearly full speed. The reason is the season's oldest promissory note, presented at last. *Addition commutes.* A gradient update is an increment, not an overwrite: apply mine then yours, or yours then mine, and the sum is the sum; the interleaving that destroys a read-modify-write ledger merely *reorders contributions* to an accumulation, and an accumulation does not care. Chapter 1's replicated counter forgave the Post's disorder because addition commutes; Chapter 7's kettles wed under a join that forgot order because forgetting order was the design; and here, at the summit of the industry, the largest optimization processes ever run survive raw lockless concurrency for exactly the same reason the shopping cart survived it. Debt #4, issued in Chapter 1: half paid at the kettle wedding with a semilattice; the remainder paid here, wearing gradients.

The honesty clause, sounded as deliberately as the rhyme: this is *rhyme, not theorem*. Gradients are not a semilattice — an update is not idempotent (apply mine twice and the model has moved twice); there is no join, no once-absorbed-always-absorbed, and Chapter 7's convergence theorem does not apply. What holds instead is an *analytic* tolerance: the optimization, itself a noisy, error-correcting process, absorbs bounded disorder as one more source of noise — staleness within limits behaves like a slightly clumsier learning rate, and the guarantees are probabilistic convergence results with conditions attached, not algebraic identities. The mature position is the negotiated middle the trade calls *stale-synchronous*: a staleness window, enforced at the barrier — workers may run ahead of the slowest by at most a bounded number of steps s — so the ring's coherence and Hogwild's tolerance meet in a contract the reader can now read at sight: bounded staleness is Chapter 7's honest tier, the window a session guarantee for gradients, and the knob priced in the only trade that exists — wait for coherence, or drift for speed, the drift's cost measured in convergence rather than in customer complaints.

Remark 12.4 (staleness: statements with pointers)*.* Per the honesty clause, stated and not proved here. *Hogwild* (Recht et al. 2011): for sparse problems with suitable step sizes, lock-free concurrent updates achieve convergence rates comparable to serial execution; the mechanism is that updates are *increments* (commutative) and collisions are rare (sparsity). *Stale-synchronous* (bounded staleness s): convergence guarantees degrade gracefully in s, recovering the synchronous rates as $s \to 0$. The rhyme with Chapter 7 is deliberate and bounded: gradients commute but are not idempotent and form no semilattice — the tolerance is analytic (noise absorption), not algebraic (join), and the desk labels it so. Pointers in the reading.

One caveat the practitioner meets before the theorist does: *floating-point addition does not associate*. Sum the same gradients in a different order and the last bits differ; the ring's reduction order, the number of workers, the very topology change the arithmetic, and two runs from the same seed on differently shaped fleets diverge — bit by bit, then visibly — so that the engineer who spends a week hunting a "bug" that is actually the associativity of nothing undergoes a rite of passage the trade could label better. Whence the discipline: bitwise-reproducible training fixes the reduction order (Box 29 does; Rem. 12.2), and Chapter 5's determinism diet turns out to matter in a field whose mathematics pretends addition is addition. The fashion has a familiar arc: the asynchronous school flourished when fleets were small, heterogeneous, and rented; the

frontier estates, on purpose-built pods of engineered uniformity, marched back to full synchrony — the barrier's tax fell as the hardware homogenized, and coherence bought what the loss curve rewards: cleaner convergence, comparable experiments, and determinism, or its close approximation, for debugging runs that cost more than office buildings — Chapter 8's Calvin aside, cashed. And asynchrony keeps honest provinces where its premises hold: fleets too heterogeneous to march — spot instances, mixed generations, the rented and the begged — where a barrier would idle the strong behind the weak all day; and the federated frontier, learning across devices that are Chapter 1's documentary weather personified — phones sleeping, dropping, lying about their clocks — where synchrony is not expensive but *meaningless*, and the parameter-server pattern, staleness budgets and all, remains the only sane church. Per layer, as ever.

12.4 Part Four — The Geometry: Model, Pipeline, and the Optimizer's Estate

When the model itself outgrows one machine's memory, Chapter 9 takes over entirely, and this part is short because the reader can now derive it. The frontier estates run all three geometries at once, composed like Chapter 9's matrix grown a dimension: tensor shards within a pod, pipeline stages across pods, data-parallel replicas across the hall — every accelerator addressed by its coordinates in a three-axis seating chart, the collectives arranged so that each axis's chatter stays on the wires that can afford it. The composition is the design problem; the axes themselves are already owned. *Tensor parallelism*: shard the individual weight matrices across accelerators — range partitioning of the parameter space itself — so that each layer's arithmetic becomes a small collective among the shard-holders, fired within every forward and backward pass: agreement not per round but per layer, thousands of little all-reduces a second, which is why this sharding lives strictly inside a pod, on the interconnect measured in terabits, and dies of latency anywhere else. Locality of reference now means *which accelerators share a matrix*, and the placement rule — the chattiest shards on the fastest interconnect — is Chapter 9's co-location doctrine with the speed of light billing per centimetre. *Pipeline parallelism*: shard the model by layers — stage one the early layers, stage two the middle, and so on — and stream micro-batches through the stages like parcels on a conveyor, the schedules small masterpieces of Chapter 10's discipline: dataflow scheduling with the operators pinned to silicon and the buffers audited to the megabyte. The tax appears immediately and has Chapter 10's shape — *the bubble*: stages idle at the head and tail of every round while the conveyor fills and drains. With eight stages and eight micro-batches in flight, roughly half the conveyor's area is fill and drain — a utilization catastrophe; with sixty-four in flight, the bubbles shrink toward an eighth — the classic remedy of keeping many parcels airborne, bought with activation memory: the same buffer-versus-latency ledger Chapter 10 kept for its rivers, with backpressure enforced by the conveyor itself.

Then *the optimizer's estate*: the revelation, industrialized as ZeRO — spoken *zero*, per the lexicon's founding entries — and its fully-sharded descendants, that the heaviest tenants were never the model's weights but the optimizer's bookkeeping — the momenta, the variances, the master copies, several times the model's own weight — replicated, in the naive data-parallel arrangement, on every one of ten thousand workers: the same pages, ten thousand times, the most redundant library since Chapter 6 taught replication for *durability*, a rationale conspicuously absent here. The repair proceeds in the staged discipline of a good rebalance: shard the optimizer state first — each worker keeps only its slice of the bookkeeping, gathering nothing, since each slice updates from the summed gradient everyone already holds; then the gradients; then the weights themselves, gathered layer by layer just in time for each layer's arithmetic and released behind it — a working set streamed through memory the way Chapter 10 streamed rivers, with the collectives doing the fetching on a schedule. The memory bill falls by the fleet size; the payment is more traffic on wires the ring already taught you to budget. Partition the state, not merely the data: where things live, one final time,

deciding what fits at all.

12.5 Part Five — Failure on a Schedule

Now the arithmetic that turns the course's whole climate into a calendar entry. The estate's components are individually excellent — that must be said first, because what follows is not an indictment of any part but of multiplication itself. Grant a single accelerator-and-attendants a mean time between failures of five years — generous, and the roundness will not matter; put ten thousand of them in one job, and the fleet's mean time between failures is M_{part}/n: ten thousand failures per sixty months, better than a hundred and sixty a month, north of five a day — *one failure every four or five hours, on schedule, forever*. The estimate is generous twice over: the parts age, the new parts arrive with fabrication's infant troubles, and the failures that matter include everything Chapter 1 catalogued beyond the silicon — the optics, the switches, the software of the attendants — so the observed cadence at frontier scale runs faster than this arithmetic, not slower; the direction of the error only sharpens the conclusion. Run the schedule against a job to feel it: a month-long run on that fleet will be killed, on average, every working afternoon of its life, and must treat each death as a lunch break — Chapter 1's climate, quantified; a job that loses everything at each interruption never finishes at all.

Whence the photograph — and the photograph itself, at this weight, is a distributed-systems exercise of its own: petabytes of optimizer state cannot march through one door, so the checkpoint is *sharded* — each worker writes its own slice, in parallel, to nearby storage, the photograph partitioned exactly as the state it depicts, with a *manifest* — a small court of record, Chapter 8's habit — declaring which slices constitute a blessed, complete cut; restore is the gather, and a torn checkpoint, half-written at the moment of death, is excluded by the manifest the way a coordinator's unforced decision never happened. The question the estate actually argues about is *how often*. Checkpoint too rarely and each failure burns hours of lost work; too often and the checkpointing itself becomes the tax that never stops. Two costs pull against each other — the standing tax δ/τ, paid always, falling as the interval lengthens; the expected rework $\tau/(2M)$, about half an interval per failure, rising with it — and the sum bottoms where the terms match, delivering, with the inevitability of all good square roots, the Young–Daly interval.

Theorem 12.3 (Young–Daly interval)**.** *In the checkpoint model, the overhead fraction of wall time for checkpoint interval τ is, to first order,*

$$f(\tau) \;=\; \frac{\delta}{\tau} \;+\; \frac{\tau}{2M},$$

(standing tax plus expected rework rate), minimized at

$$\tau^{*} \;=\; \sqrt{2\,\delta M}, \qquad f(\tau^{*}) \;=\; \sqrt{\tfrac{2\delta}{M}}.$$

Derived as the guided Problem 12.5; with δ five minutes and M four hours, τ^{*} is about fifty minutes and the overhead about twenty percent — the finale's worked numbers. The sensitivity is the practitioner's clause: halving δ halves the overhead but tightens τ^{*} only by root two — invest in cheap photographs, not long gambles.

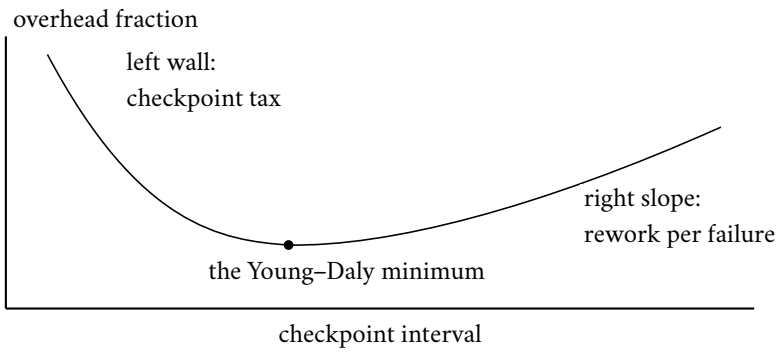


Exhibit 12.2 (the cadence curve). Standing tax δ/τ falls, expected rework $\tau/2M$ rises; their sum bottoms at $\sqrt{2\delta M}$ (Thm. 12.3, Problem 12.5). Argue with the formula, not with opinions — and note how flat the basin is: the win is in cheapening δ, which lowers the whole curve.

With the finale's numbers — a checkpoint costing five minutes, a fleet failing every four hours — the formula prescribes roughly every fifty minutes: call it hourly, a standing tax of about a tenth in checkpoint time against expected rework of under half an hour per incident. The practical gift is the sensitivity: halve the checkpoint cost — asynchronous checkpointing, sharded writes, the photograph taken without stopping the river, Chapter 10's whole craft — and the optimal interval tightens by only a square-root factor while the tax halves outright, which is why the engineering effort goes into cheapening the photograph rather than gambling on the interval. Arithmetic, not folklore; the whiteboard says exactly that.

The stragglers return next, with the interest promised. Chapter 10's backup tasks raced duplicates against the slow at batch scale, where tasks were pure and the loser's work was discarded free of charge. The training loop's version is harder — the barrier makes every straggler everyone's straggler, and a gradient worker is stateful in ways a map task is not — so the estate deploys the whole of Chapter 11 instead: phi-grade detection on step-completion times, because a slowing accelerator announces itself statistically long before it fails, and the round barrier is, conveniently, a fleet-wide heartbeat of perfect regularity — the failure detector's dream client; hedged scheduling of the input pipeline, where the work *is* stateless and the duplicates race free; and hot spares standing by the ring. Walk the swap once, because it is the season's machinery in one motion: Gussie's accelerator degrades mid-run; the detector's dial crosses the acting threshold; at the next round barrier — a natural quiescent point, no Chandy–Lamport gymnastics required — the membership epoch increments, the ring re-forms with Barmy the spare spliced into Gussie's arc, Barmy loads the current weights from a peer or the last photograph, and the fleet resumes: a view change (Chapter 4), fenced by the epoch on every collective (Chapter 5), so that Gussie's late parcels — stamped with the dead epoch — are refused at every doorstep rather than folded into sums they no longer belong to. The input pipeline rides through untroubled: Barmy resumes Gussie's data shard from the committed cursor — Chapter 10's bookmark — and the log neither notices nor cares which machine is drinking from it. Seconds of pause, not a restart from the photograph; the doppelgänger's oldest trick answered by the season's oldest token. And who decrees the epochs? A small controller runs the job — tracking membership, blessing checkpoints, ordering the view changes — and it is, of course, a little parliament: replicated, elected, fenced, Chapter 5's consensus kernel in a lab coat, holding the tiny truths (the member list, the golden checkpoint's manifest, the current epoch) while the herd of accelerators holds the tonnage. The coordination dividend, collected one last time; debt #24, presented at batch scale in Chapter 10, paid at silicon scale here. The whole assembly, boxed:

Protocol — Box 30. The elastic training step (epochs on every collective)

1: **controller** (a small replicated parliament, Ch. 5): holds member list, current epoch e, blessed checkpoint manifest
2: **each round:** workers compute; all-reduce stamped with e; any parcel bearing $e' \neq e$ is refused (fencing, Ch. 5)
3: **straggler detected** (phi on step times, Ch. 11): at the next barrier, controller increments e, splices the hot spare into the ring, publishes the new view
4: spare loads weights from a peer or the manifest's checkpoint; input cursor resumes from the committed bookmark (Ch. 10)
5: **on failure mid-round:** abort the round; view change as above; recompute the round — rounds are idempotent by determinism from an agreed state
6: **checkpoint every τ^*:** each worker writes its shard; manifest committed by the controller when all shards land; recovery = restore manifest's cut, rewind cursors, replay

Chapters four, five, ten, and eleven, snapped together; the biggest computation governed by its smallest component.

And then the deferred horror, kept in a locked drawer since Chapter 5 with a label reading *not yet*. Chapter 5's Byzantine hour concerned machines that lie, and the course said, honestly, that in the datacentre one mostly pays for crash tolerance and reserves the Byzantine premium for the border — advice that stands, for the protocols. The drawer opens because the fleet studies of Meta and Google — published, sober, and quietly hair-raising — documented the exception: *silent data corruption*. Not disks lying about writes — cores lying about *arithmetic*: the *mercurial cores* (the fleet study's own coinage), individual processor cores, one in some thousands, that under particular operands, temperatures, and voltages return the wrong answer to a correct computation and raise no flag whatsoever — defective from fabrication or aged into error, miscomputing a multiply once a day or once a month, silently. In an estate of this scale, that is a standing population of liars — Aunt Agatha, tenured, in the arithmetic unit, her final posting — and their products are not crashes but *poisoned numbers*: a gradient with one wrong element, a checkpoint with a corrupted page, an index computed wrong once and never again, propagating through a computation whose entire posture assumes the arithmetic is honest. The studies' sobering clauses: the rate is far above what the industry's certification models predicted; the errors are data-dependent and intermittent, so the offending core passes every standard test and fails only on the operands of a particular Tuesday; and the damage is characteristically quiet — the training loss goes subtly wrong, or a compressed file unpacks to garbage a month later, and the investigation consumes weeks because every component reports itself healthy: gray failure (Chapter 11), driven down into the transistors.

Remark 12.5 (silent data corruption, economically). The fleet studies ("Cores that don't count," Google; Meta's companion) document cores that miscompute data-dependently, intermittently, and silently, at rates far above certification models — the Byzantine ghost below the protocols. Detection is an economics ladder: full duplication (gold, double cost, reserved for the un-wrongable); random spot-duplication (statistical coverage at a few percent); targeted screening in idle cycles (catches the reproducible); checksums on everything that moves (in-flight and at-rest, blind to birth defects); checkpoint-time validation (poisoned runs caught within one τ^*); and invariant tripwires on the loss (the last, cheapest, slowest alarm). The remedy's shape is the end-to-end argument's final appearance: verify where the meaning lives, by comparison at the ends — here, the ends of the arithmetic itself.

The deepest pattern deserves its sentence: the arithmetic unit was the last *middle* anyone thought to distrust, and the remedy — verify at the level that owns the meaning, by ends-side comparison — is the end-to-end

argument's final word, delivered one layer beneath where anyone expected to need it.

12.6 Part Six — Serving, Briefly: The Consistency Bug With a Profit Line

The estate's other half deserves its translation — and the trade's own postmortems suggest that more machine-learning value dies in the gap this part names than in all the training outages combined; the training failures are loud, and this one whispers. *Training-serving skew* is a consistency violation between two replicas of the same computation, and one staging suffices: the training pipeline computes "customer's average purchase this month" in batch, from the log, with the month complete and the late records settled; the serving path computes "the same feature," reimplemented by a different team in a different language, online, from a cache, mid-month — and the two disagree in a hundred small ways nobody itemized: null handling here, timezone there, the late data included in one and dropped by the other. The model, trained against the batch replica's world and examined daily against the online replica's, degrades for a reason no model metric will name, because the bug is not in the model — it is *between two copies of a computation that no one declared to be replicas*, and therefore no one monitored for divergence. Chapter 7 taught the vocabulary: two replicas, no single source of truth, divergence unmonitored; and the modern remedy is Chapter 10 verbatim — one feature definition, materialized to both consumers from the same log, the inside-out doctrine ending the schism by abolishing the second codebase, exactly as Kappa ended Lambda's — with the residual monitored like any replication pair: sampled online features logged and diffed against their batch recomputation, divergence alarmed before it is a revenue line.

The serving fleet itself now reads at sight: model replicas behind a balancer — Chapter 6 at its most benign, for inference is stateless and reads divide; rollouts as view changes with canaries — a new model version admitted to a sliver of traffic, watched, then promoted, Chapter 11's blast-radius discipline applied to weights instead of code; and rollback as the rewind, kept cheap by keeping the old version warm — the log's oldest lesson wearing a serving mesh. And *feature freshness is replication lag with a profit-and-loss line*: the recommendation model reading a user's behaviour thirty seconds stale is Chapter 6's follower read, except that here the staleness has a measurable revenue gradient, and the trade budgets it — bounded staleness with a number, Chapter 7's honest tier — in currency, per second of lag. The Chapter 7 catechism transplants whole: eventual, plus what? Plus which session guarantees for the user's own actions, plus what staleness bound at what revenue cost, signed by whom. The vocabulary was always going to end up in the pricing meeting; the course merely delivered it there with its proofs attached.

12.7 Part Seven — The Grand Audit, and the Arc Retraced

The company first, because a season closes with a curtain call. The *two generals*, who opened the season on their hill and stand at its close, the field's first and final image. The *Post*, the season's magnificent villain — loser of letters, breeder of twins, perjurer of timestamps, and, in the end, salaried employee of the simulation harness: hired, in the finale's finest irony, by the very trade it tormented. The *doppelgänger*, the master key — Two Generals, FLP, the Byzantine bound, the false death, the blocking proof — who taught the season's deepest lesson, that what cannot be distinguished cannot be acted upon differently, and retired a chapter ago with his pricing finally published. The *crowded club*, whose Sunday-shift lemma held up four chapters of quorums. The *log*, the understudy who became the lead. The *photograph*, forty years from theorem to overalls. And the standing cast: Bingo, coordinator, head of chains, officiant; Tuppy, the literal-minded waiter, in-doubt widow, and on-call doctor; Gussie, who crashed, lagged, was decommissioned, died in a storm, and degraded once more in this finale, the season's most put-upon machine; Barmy, rehabilitated from crash-prone understudy to the finale's hot spare, the arc of the season in one career; Oofy, benched

with distinction; Bertie, the customer whose carts, transfers, tunnels, and telephones supplied every crime scene; the aunt, formal test and flower-committee celebrity, retired undefeated; and Aunt Agatha, the Byzantine terror, whose final posting — tenured, in the arithmetic unit — this chapter disclosed. The company bows.

Now the books. **Twenty-four notes were issued across twelve evenings, and twenty-four were paid** — itemized, issue to payment, in the register table of §12.8, permanent equipment at this desk. The shape of the accounts, surveyed as a connoisseur reads a cellar book: the founding evening alone wrote four — though its own closing ledger declared only three, the fourth slipped into the drawer mid-episode, unnumbered, a line about commutativity forgiving disorder, and caught by the register, which counts what is filed rather than what is announced; that drawer note outlived them all, half paid in Chapter 7 under the wedding canopy and the remainder in Part Three above, wearing gradients. The book's founding entry — the Post learns to forge, issued when the charter was still innocent of malice — was paid in Chapter 5, with a theorem, a bound of three-to-one, and an aunt. The most patient debt was the most profitable: crashed versus slow — managed in Chapter 3, where it powered an impossibility; hedged in Chapter 5, where fencing made its losses survivable; priced and dialled in Chapter 11, where phi printed the odds on the face. The longest-held note was the photograph in industrial costume — eight evenings in the drawer, paid in Chapter 10 with the reveal that a 1985 theorem hums inside every stream processor, its spirit taking a third bow here, at petabyte weight, with a manifest for a blessing. And the last note of all, the racing duplicates — planted in the MapReduce paper's one immortal page — was paid in this last chapter, at silicon scale, with the spares standing by the ring. The register's shape tells the season's story in miniature: the early evenings issued and issued, promises being the natural currency of a course that knows where it is going; the middle evenings traded, paying old paper while writing new; and the last evenings only paid, the eleventh issuing nothing and the twelfth retiring the final two — because a course that ends owing nothing was the constitution's quiet definition of *finished*. The season's numbered decisions stand likewise discharged or recorded — the lexicon locked at its final register, the shakedown waived by ruling, the one starred appendix delivered to its appointed chapter and contained there — and the assembly session's continuity audit will verify by grep what the finale verified by voice.

The arc, retraced in one breath. Two generals stood on a hill outside a besieged city, unable — provably, permanently unable — to agree to attack, because the last messenger might always drown. From that single drowning messenger the course derived a world: the confusion of crashed and slow, and the theory that priced it; the death of the shared clock, and the causal order built in its ruins; the impossibility at the bottom of agreement, and the parliaments that pay their way around it; the log, the lease, the quorum, and the fence; the field guide of promises and the algebra that makes promises unnecessary; the transactions that faint and the clocks that confess; the shards, the maps, the rivers, and the photographs of rivers; the storms systems brew in their own plumbing, and the engineering that governs the weather — until now, at the far end of the same unbroken argument, ten thousand accelerators agree on a gradient, sixty times a minute, for months, and think nothing of it. The two generals could not agree once; the proof is as sound now as the evening it opened the season, and not one system in twelve chapters has violated it. Their descendants agree five million times a day *anyway* — not by refuting the theorem but by wanting something the theorem does not forbid: not certainty about the last message, but convergence in the limit of many; not perfect knowledge, but idempotent action under imperfect knowledge; not the impossible contract, but a priced approximation the business can bank. Nothing was ever solved — not the Post, not the clock, not the doppelgänger; every impossibility of Chapter 3 stands unrepealed still. It was *priced*, and the price list is the season's true syllabus, itemized once at the close: the Two Generals were never satisfied — they were bought off with idempotence and retries, certainty exchanged for convergence; FLP was never overturned — it was rented around with partial synchrony, randomness, and failure detectors, liveness purchased by the timeout and repossessed

in every storm; CAP was never escaped — it was *chosen*, per room, per table, per feature flag, the field guide of Chapter 7 being nothing but the choice made legible; the shared clock was never restored — it was replaced by causal paperwork where messages suffice, and purchased outright, in caesium, where they do not; and the crashed could never be told from the slow — the confusion was dialled, fenced, and budgeted until it cost less than the certainty would have. Engineering, the course has argued from its first hour, is the discipline of paying for what mathematics refuses to give away free, and the register is twelve evenings of exactly those receipts. The three unpleasant facts of the founding arc — machines fail independently, messages are delayed or lost, there is no shared clock — read, at the season's end, not as obstacles but as design instincts: assume the failure and make it boring; carry the causality or buy the clock; and never, ever, trust the middle with the ends' correctness. And the closing wink, laid in the plan from the first day: stochastic gradient descent survives asynchrony for the same reason the shopping cart does. The merge is a join — near enough, in the mean, with the caveats duly sworn; addition forgets nothing but order, and order, for a sum as for a cart, was never the point. The humblest object in this course and the grandest computation on earth, forgiven by the same algebra — the cart and the model, the tuppenny purchase and the gigawatt run, kneeling at the same commutative altar. Chapter 1 placed that counter in a drawer with a glint and a promissory note; twelve chapters later the note retires the season.

12.8 The Grand Tables

The course dictionary (the finale's Part One, tabled).

The estate says	The course says (chapter)
training step	barrier-synchronized round (4)
all-reduce	agreement on a sum; below consensus by the race test (6, 12)
checkpoint	consistent snapshot; the photograph (2, 10, 12)
straggler	the unluckiest machine; backup tasks and hedges (10, 11)
parameter staleness	replication lag; eventual-plus-what (6, 7)
elastic training	view change with epoch fencing (4, 5)
data pipeline	the log: replayable input, cursors (10)
job scheduler	placement against failure domains (9)
feature store	materialized view of one log (10)
loss-curve monitoring	the watermark; invariant tripwires (10, 11)
gradient accumulation	batching: latency for throughput, the oldest trade (5, 8)
training-serving skew	an undeclared replica pair (6, 7)

The debt register, final (issue → payment; the grand audit of the finale's Part Seven, ruled).

Debt	Issued → paid
1. The Post learns to forge	E1 → E5 (theorem and aunt)
2. Crashed versus slow	E1 → E3, E5, E11 (priced, dialled)
3. No shared clock	E1 → E2 (happens-before)
4. Commutativity forgives disorder	E1 → E7 (semilattice), E12 (gradients)
5. Physical clocks return, armed	E2 → E8 (TrueTime, commit-wait)
6. The photograph, industrialized	E2 → E10 (barrier snapshots)
7. Order rented from a leader	E2 → E4
8. Causal delivery as a contract	E2 → E7
9. Partial synchrony cashed	E3 → E4
10. Suspicion services priced	E3 → E5, E11 (phi-accrual)
11. CAP's C defined	E3 → E7 (linearizability)
12. One value becomes a log	E4 → E5
13. "Single machine" named	E4 → E7
14. Reconfiguration's dragons	E4 → E5
15. The log stars	E5 → E10
16. Lease-read fine print	E5 → E7
17. The kettles must merge	E6 → E7 (SEC theorem)
18. Serializability across shards	E7 → E8
19. Jepsen industrialized	E7 → E11
20. Where things live	E8 → E9
21. Partitioned computation	E9 → E10
22. Stampedes and storms	E9 → E11 (metastability)
23. The retry's dark side	E10 → E11
24. Backup tasks return	E10 → E12 (spares by the ring)

12.9 The Whiteboard

For the design review; the season's last.

1. Choose synchrony per layer: synchronous gradients where the mathematics wants coherence, asynchronous data pipelines where the log absorbs the slack, checkpointed everything, always. The barrier is a purchase, the ring is its efficient form, and the straggler is its standing invoice — detect, hedge, and swap by view change, with the epoch on every collective; a fleet without hot spares has decided, implicitly, that every straggler is worth ten thousand machines' idleness, and should at least decide it explicitly.

2. An all-reduce is not consensus. Agreement on a sum arbitrates nothing, excludes nobody, and therefore costs wire speed instead of a parliament; the moment the estate needs a *decision* — membership, leadership, which checkpoint is golden — the race test fires and Chapter 4 presents its bill. Know, at every layer, which of the two you are paying for, and be suspicious in both directions: a parliament asked to move gradients is a bottleneck by design, and a collective asked to arbitrate membership is a split brain on order.

3. Checkpoint cadence is arithmetic, not folklore: $\sqrt{2\delta M}$, then argue with the formula rather than with opinions. The failure rate at fleet scale is a schedule; treat rework as a budget line and the interval falls out — and spend the engineering on cheapening the photograph, where the returns are linear,

rather than on stretching the interval, where the square root blunts them. A blessed checkpoint needs a manifest; a torn one needs to be impossible to mistake for blessed.

4. Skew is a consistency bug wearing a machine-learning costume, and freshness is replication lag with a revenue line. One definition, one log, both consumers — and monitor the divergence between training and serving the way Chapter 11 taught you to monitor anything: before it is a metric the business notices. Two computations nobody declared to be replicas will diverge exactly as thoroughly as two that were declared and neglected; the declaration is the whole of the fix.

The register. Paid: #4's remainder (commutativity forgives disorder, wearing gradients — the rhyme honestly labelled) and #24 (the backup tasks, at silicon scale, with the spares by the ring). Issued: nothing. The register is closed: twenty-four issued, twenty-four paid, nothing outstanding, nothing silently forgotten — the grand audit itemized in §12.8. Episode word count: 10,024 — above the floor. Season status: twelve episodes, twelve chapters; the assembly session (continuity audit, consolidated registers, errata from sir's listening) remains per the constitution's schedule.

12.10 Problems

Tier I — Calisthenics

Problem 12.1 (parcel arithmetic)**.** A gradient of four hundred gigabytes, links of fifty gigabytes per second each way. (a) Compute per-node traffic and bandwidth-regime wall time for rings of four, one hundred, and ten thousand nodes. (b) At what message size does the latency term $(2(n-1)\alpha$, with α ten microseconds and n one thousand) equal the bandwidth term — and which collective shape does the answer recommend for gradient chunks versus for the tiny control messages? (c) State, in one sentence, why the answer to (a) barely changed across three orders of magnitude of fleet.

Problem 12.2 (the schedule)**.** Ten thousand parts at five-year part MTBF. (a) Fleet failure cadence, per day. (b) A month-long run: expected interruptions. (c) The vendor doubles part reliability; how does the answer change, and what does that say about relying on parts over process? (d) With δ four minutes and the cadence from (a), compute τ^* and the overhead fraction.

Problem 12.3 (which agreement)**.** Classify each as sum-grade (collective) or decision-grade (parliament), with the race-test sentence: (a) the averaged gradient; (b) which of two divergent checkpoint manifests is golden; (c) the global batch count; (d) whether node forty is a member this round; (e) the maximum loss across workers.

Problem 12.4 (staleness taxonomy)**.** Translate into Chapter 7's vocabulary and name the price: (a) fully synchronous training; (b) Hogwild; (c) stale- synchronous with window three; (d) a parameter server with unbounded worker lag; (e) federated rounds with stragglers dropped.

Tier II — The Real Thing

Problem 12.5 (Young–Daly, guided)**.** Derive Thm. 12.3. (a) Justify the overhead model $f(\tau) = \delta/\tau + \tau/(2M)$: where does the half come from, and what first-order assumptions are made? (b) Minimize f and obtain τ^* and $f(\tau^*)$. (c) Recompute the finale's numbers (δ five minutes, M four hours) and then the sensitivity: δ halved, M halved, both. (d) State two effects the model ignores (failures during checkpoint and recovery time itself) and argue the direction each pushes τ^*.

Problem 12.6 (the hierarchical budget)**.** Two pods of eight nodes; intra-pod links of two hundred gigabytes per second, inter-pod fibre of twenty. Design the hierarchical all-reduce (reduce-scatter within pods; exchange between; gather within), compute each phase's time for a one-hundred-sixty-gigabyte gradient, identify the bottleneck, and compare against one flat sixteen-node ring forced through the fibre. State the general rule you have rediscovered.

Problem 12.7 (elastic membership, designed)**.** Design Box 30 in full from Chapters 4, 5, 10, and 11: the controller's replication and election; the epoch's issuance and its fencing check in the collective; the spare's state transfer (peer copy versus manifest restore — when each); the input cursor handoff; and the abort-and-recompute rule for mid-round failures, with the determinism assumption that licenses it. Then break your own design twice: remove the fencing and exhibit the corrupted sum; remove the controller's replication and exhibit the stalled fleet.

Problem 12.8 (sabotage: the uncorrected latecomer)**.** An asynchronous scheme applies each arriving gradient to the *current* weights with full step size, however stale the weights it was computed against, no staleness cap, no correction. Characterize the drift: (a) write the update mismatch for a gradient computed at version t and applied at version $t{+}s$; (b) on a one-dimensional quadratic with two workers, one fast and one slow, trace six updates and exhibit the systematic overshoot the slow worker's stale gradients inject near the optimum; (c) state the two standard repairs (staleness cap; staleness-scaled step size) and which Chapter 7 posture each corresponds to. *(Solution in full: the certification of drift, per the standing rules, with the trace written out.)*

Tier III — For the Ambitious (starred)

Problem 12.9 (★ the spare pool)**.** Given the fleet cadence of Problem 12.2, a swap time of ninety seconds, a repair turnaround of six hours, and the cost ratio of an idle spare to a stalled fleet, model the spare-pool size that minimizes expected cost; exhibit the regime where zero spares is defensible and the regime where the answer is "a full rack." *(Hints only.)*

Problem 12.10 (★ the two-ledger bound, tightened)**.** Formalize Thm. 12.2's counting argument: define the algebraic model (linear combinations over generic reals), prove the per-node receive and send floors exactly, and determine whether the $2(n{-}1)/n$ factor is achievable simultaneously at *every* node by any algorithm other than ring-equivalent schedules. *(Hints only.)*

12.11 Hints

Hint (Problem 12.3)**.** (e) is the instructive one: a maximum is associative, commutative, idempotent — sum-grade machinery suffices, and it is even a semilattice, unlike the sum.

Hint (Problem 12.5)**.** (a) Failures land uniformly within an interval in the memoryless model; the mean loss is half. (d) Both push τ^* up slightly; write the second-order term.

Hint (Problem 12.8)**.** (b) Take the quadratic with minimum at zero, learning rate below the stability bound for serial descent; let the slow worker's gradient arrive three steps late, computed at a point the fast worker has since crossed. The stale gradient points *away* from the optimum on arrival.

Hint (Problem 12.9)**.** Poisson arrivals against a repair pipeline: the spare pool is a buffer sized against the arrival rate times the turnaround, plus safety stock at the service level the fleet-stall cost implies.

12.12 Solutions

Tier I in full (tersely); Tier II in full — Problem 12.8 is the sabotage certification; hints only for Tier III.

Solution (to Problem 12.1)**.** (a) Traffic $2(n-1)G/n$: at n=4, six hundred gigabytes; at n=100, seven hundred ninety-two; at n=10^4, essentially eight hundred. Wall time at fifty gigabytes per second: twelve, just under sixteen, and sixteen seconds respectively. (b) Latency term: two thousand hops at ten microseconds is twenty milliseconds; the bandwidth term matches it at $G \approx$ one gigabyte for this n — so: rings for the gradient's giant chunks, trees for the small control traffic, exactly the courier's standing practice. (c) The fleet size cancels in the traffic formula: the ring's whole point, and the reason the training estate can grow without renegotiating its wires per node.

Solution (to Problem 12.2)**.** (a) Part MTBF sixty months; fleet cadence sixty months over ten thousand: about four and a half hours; north of five per day. (b) Roughly one hundred sixty. (c) Doubling part reliability halves the cadence to eleven-ish a week — still a schedule, not an event: process (checkpoints, spares, swaps) dominates parts at fleet scale, which is the paragraph's whole point. (d) $\tau^* = \sqrt{2 \cdot 4 \cdot 270}$ minutes $\approx$ forty-six minutes; overhead at the optimum $f(\tau^*) = \sqrt{2\delta/M} = \sqrt{8/270} \approx$ seventeen percent — painful, and exactly why the engineering (Thm. 12.3's sensitivity clause) attacks δ rather than the interval: at δ one minute the optimum overhead falls below nine percent.

Solution (to Problem 12.3)**.** (a) Sum-grade. (b) Decision-grade: two candidates, one winner — a race; the controller's parliament rules. (c) Sum-grade (a count is a sum). (d) Decision-grade: membership is the archetypal arbitration (Chapters 4, 5). (e) Sum-grade and better: max is idempotent — a true semilattice, the only entry in the list that would have satisfied Chapter 7's wedding requirements outright.

Solution (to Problem 12.4)**.** (a) Strict coherence: every read of the weights sees the latest agreed version — the linearizable tier, bought with barriers. (b) Eventual-plus-sparsity: unbounded staleness in principle, tolerated analytically — the floodplain with an actuary's note. (c) Bounded staleness, window three — Chapter 7's honest middle tier, priced in convergence rate. (d) Eventual with no bound: the costume without insurance; the desk declines to endorse. (e) Synchronous per round with membership churn — a view-change regime where the dropped stragglers are the price of the barrier.

Solution (to Problem 12.5)**.** (a) Per interval of length τ: one checkpoint, cost δ (tax rate δ/τ); failures arrive at rate $1/M$ and destroy, in expectation, half the interval's work (uniform landing under memorylessness): rework rate $\tau/(2M)$. First order: $\delta \ll \tau \ll M$, failures during checkpointing and recovery ignored. (b) Set $f'(\tau) = -\delta/\tau^2 + 1/(2M) = 0$: $\tau^* = \sqrt{2\delta M}$; substitute for $f(\tau^*) = \sqrt{2\delta/M}$. (c) Five minutes and two hundred forty: $\tau^* = \sqrt{2400} \approx$ forty-nine minutes, overhead $\approx \sqrt{1/24} \approx$ twenty percent; halving δ: thirty-five minutes and fourteen percent; halving M: thirty-five minutes and twenty-nine percent; both: twenty-four minutes and twenty percent — the overhead scales as $\sqrt{\delta/M}$, each lever a square root. (d) Failures mid-checkpoint waste the attempt: push τ^* up (checkpoint less often than naive); recovery time adds a constant per failure, indifferent to τ but raising total overhead — both second-order in the stated regime, both worth modelling when δ/M is not small.

Solution (to Problem 12.6)**.** Within pods: reduce-scatter eight ways at two hundred gigabytes per second: $\frac{7}{8} \cdot 160/200 = 0.7$ seconds. Between pods: each of eight partner pairs exchanges its parcel share — twenty gigabytes each way over the twenty-gigabyte fibre split across pairs: with one fibre, $2 \cdot 160/8$ (the pod-reduced shares) over twenty = two seconds — the bottleneck; with a fibre per pair, a quarter second. Gather within: another zero point seven. Flat sixteen-ring through the fibre: nearly all thirty of the $2 \cdot \frac{15}{16} \cdot 160$ gigabytes cross the fibre at twenty: fifteen seconds. The rule: hierarchy confines the unavoidable inter-pod

traffic to the pod-reduced residue — Chapter 9's commandment, heavy traffic on short wires, with the arithmetic showing a factor of five to sixty depending on the fibre count.

Solution (to Problem 12.7)**.** Controller: three or five nodes, Raft-grade (Ch. 4), leader issues epochs; epoch in every collective header, checked by every participant before folding a parcel (Ch. 5 fencing). Spare transfer: peer copy when a healthy replica holds current weights (fast, no storage round trip); manifest restore when the round aborted or peers are suspect. Cursor handoff: the committed offset vector lives in the manifest; the spare resumes from it (Ch. 10). Mid-round failure: abort, view-change, recompute — licensed because a round is a deterministic function of (agreed weights, agreed batch), so recomputation is idempotent in effect (Ch. 10's trinity). Break one: without fencing, the dead view's straggler folds a stale parcel into the new view's sum — a corrupted average applied everywhere; determinism then faithfully replicates the corruption, the diet's dark side. Break two: an unreplicated controller is a single point whose crash stalls every view change — the fleet runs until the first straggler, then waits forever: Chapter 8's blocking, self-inflicted.

Solution (to Problem 12.8 — the certification)**.** (a) The scheme applies $-\eta \nabla f(w_t)$ at w_{t+s}; the serial process would apply $-\eta \nabla f(w_{t+s})$; the mismatch per update is $\eta(\nabla f(w_{t+s}) - \nabla f(w_t))$, which grows with both the staleness s and the curvature crossed in the interim — worst precisely near sharp optima, where the landscape changes fastest. (b) The trace, written out. Quadratic $f(w) = w^2/2$ (gradient $= w$), $\eta = 0.5$, start $w = 4$. Fast worker updates every step; slow worker's gradients arrive three steps late. Steps: (one) fast, grad at 4: $w = 4 - 2 = 2$. (two) fast, grad at 2: $w = 1$. (three) fast, grad at 1: $w = 0.5$ — honest descent, nearly home. (four) *slow worker's parcel arrives*, computed back at $w = 4$: applied now, full step: $w = 0.5 - 0.5 \cdot 4 = -1.5$ — flung past the optimum to the far side, three times farther out than it stood. (five) fast, grad at -1.5: $w = -0.75$. (six) the slow worker's next stale parcel (computed at 2): $w = -0.75 - 1 = -1.75$ — driven *outward* again. The pattern is systematic, not noise: every stale parcel points where the optimum *was*, and near convergence the stale corrections dominate the honest ones — the iterate orbits or diverges instead of settling, and the run's loss curve shows the signature sawtooth every practitioner has met and few have named. Certified as drift, per the standing rules: a deterministic two-worker schedule, six steps, systematic displacement away from the optimum, no randomness required. (c) Repairs: a staleness cap — refuse parcels older than $s_{\max}$ (bounded staleness: Chapter 7's middle tier); or staleness-scaled steps — damp η by the parcel's age (graceful degradation, the actuary's posture). Uncapped, uncorrected lateness is the floodplain costume with weights on: eventual, plus nothing, minus convergence.

12.13 The Reading, Consolidated

The season's shelf, organized for the reader going onward; each chapter's own list stands, and this essay points across them.

The spine, in six papers. Lamport's time-clocks paper (1978; Ch. 2) — reread last of all: the whole course is in it, folded. FLP (1985; Ch. 3), for the shape of impossibility. "Paxos Made Simple" (2001; Ch. 4), for agreement rented honestly. Herlihy–Wing (1990; Ch. 7), for the summit named. The Dynamo paper (2007; Ch. 6), for the other church, priced. Kreps's "The Log" (2013; Ch. 10), for the seam that joins them all.

The industrial texts. Spanner (Ch. 8) — the course's single most load-bearing industrial paper; GFS, MapReduce, and Spark (Ch. 10) — the lineage arc; Bigtable and Chubby (Chs. 5, 9) — the directory church's liturgy; Kafka's design documentation (Ch. 10); the parameter server (Ch. 12).

The corrective literature. Berenson et al. on isolation folklore (Ch. 7); the metastable-failures paper and the DynamoDB postmortem (Ch. 11); gray failure (Ch. 11); "Cores that don't count" with Meta's companion (Ch. 12); Kingsbury's analyses throughout — the consumer-protection bureau whose existence improved every brochure it audited.

The methods. Chandy–Lamport (Ch. 2) beside Carbone et al. (Ch. 10), read as one paper forty years apart; the Amazon formal-methods report with Ch. 11's appendix as on-ramp; the FoundationDB testing lineage; the Brooker Builders' Library essays, for the reflexes.

The standing companion. Kleppmann, whole, at last: the course's chapters map onto his as siblings rather than as summaries, and the reader finishing this volume will find him now readable in an afternoon per chapter, which was, in part, the design.

Onward. For depth in theory: Lynch's *Distributed Algorithms* and Cachin–Guerraoui–Rodrigues for the proofs this desk structured but did not exhaust. For depth in practice: the SRE and Builders' Library corpora, read with Ch. 11's instruments. For the frontier this finale opened: the systems-for-machine-learning literature moves too fast for a reading list to hold; the durable preparation is the season's method itself — find the round, the log, the photograph, and the race test in whatever arrives next, and read its brochure with the register open. *The three unpleasant facts will be waiting there too, sir. They are the terms of trade.*

Chapter 13

The Apparatus

This closing chapter is the only one in the volume with no episode behind it. It is the apparatus the masthead promised: a map of the consolidated registers, an index of the protocol boxes, and the pronunciation register in print. Twelve chapters formalize what the episodes narrate; this one merely makes the volume navigable from the back, which is where a desk volume is most often opened.

13.1 The Consolidated Registers

The season's ledgers were consolidated where the season ended, and this section is chiefly a signpost, so that no reader hunts through twelve chapters for equipment that lives in one. *The course dictionary* — the twelve load-bearing terms, each with the chapter that owns it — and *the final debt register* — all twenty-four notes, issue to payment — stand as the grand tables of Section 12.8, in Chapter 12, where the finale read them aloud. *The reading, consolidated* — the spine in six papers, the industrial texts, the corrective literature, and the methods — closes Chapter 12 as an essay, superseding no chapter shelf but gathering them. *The notation ledgers* remain deliberately chapter-local: each chapter's spoken-phrase-to-symbol table binds the vocabulary of one episode, and a merged table would only launder context out of notation. What the volume lacked until now was an index of its thirty protocol boxes, which the practising reader consults more often than any theorem; it follows, in two sittings.

Protocol box	Chapter
Box 1. The naive replicated counter (the inaugural bug)	1
Box 2. Stubborn point-to-point link, over fair-loss	1
Box 3. Perfect link over fair-loss	1
Box 4. The Lamport clock	2
Box 5. The vector clock	2
Box 6. The Chandy–Lamport snapshot	2
Box 7. Ben-Or's randomized consensus, crash-fault form	3
Box 8. The Synod: single-decree Paxos	4
Box 9. Raft essentials	4
Box 10. Multi-Paxos essentials	5
Box 11. Lease grant and hold, under drift	5
Box 12. PBFT, normal case	5
Box 13. The Dynamo-school read and write paths	6
Box 14. ABD single-writer atomic register	6
Box 15. The session receipts	7

Protocol box	Chapter
Box 16. Grow-only and positive-negative counters	7
Box 17. The observed-remove set	7
Box 18. Two-phase commit with recovery matrix	8
Box 19. Spanner commit-wait	8
Box 20. Percolator transactions	8
Box 21. The ring, industrial dress	9
Box 22. Rendezvous placement	9
Box 23. The routing epoch	9
Box 24. The exactly-once sink materials	10
Box 25. Asynchronous barrier snapshotting	10
Box 26. The defused retry	11
Box 27. The circuit breaker, with fine print	11
Box 28. The request's lifecycle under pressure	11
Box 29. Ring all-reduce	12
Box 30. The elastic training step	12

One specification stands outside the numbering by design: the TCommit specification, in Chapter 11's starred appendix, written in the TLA+ idiom after Lamport — filed as an exhibit of the method rather than as equipment of the course, per the season's numbered decision on formal specification.

13.2 The Pronunciation Register

The audio edition of this course ships with a pronunciation dictionary — two files in the ElevenReader dialect, one carrying alias respellings, one carrying phonemes — generated from a single source of truth in the repository's lexicon directory. This section prints the register's ninety-one entries in the alias form,

for the reader who wants to know how the narrator intends a name before the narrator says it, and for the listener auditing what was said. The phonetic-alphabet forms live in the phoneme file and are omitted here; they say nothing the respelling does not, in a typeface fewer readers enjoy. Entries *flagged for the ear* carry a verification note in the source: the respelling follows the best written evidence, but the course awaits its listener's verdict, and corrections are folded back into the single source, never patched downstream. Possessive forms of every entry are generated automatically and are not listed.

Grapheme	**Spoken as (alias respelling)**
Lamport	LAM-port
Fischer	FISH-er
Paterson	PAT-ur-sun
Deutsch	DOYTCH
Gosling	GOZ-ling
Dwork	DWORK
Stockmeyer	STOCK-my-er
Toueg	too-EG *(flagged for the ear)*
Ghemawat	GEM-uh-waht
Ousterhout	OH-ster-howt
Junqueira	zhoon-KAY-ruh *(flagged for the ear)*
Preguica	preh-GEE-suh — *script spells Preguica plain (flagged for the ear)*
Baquero	buh-KAIR-oh
Zawirski	zuh-VEER-skee *(flagged for the ear)*
Vogels	VOH-gulz
Stoica	STOY-kuh
Renesse	ruh-NESS-uh — *as in van Renesse*
Attiya	ah-TEE-yuh
Dijkstra	DYKE-struh
Dolev	DOH-lev
Herlihy	HER-li-hee
Chandy	CHAHN-dee
Mattern	MAH-tern
Fidge	FIJ

Grapheme	Spoken as (alias respelling)
Zaharia	zuh-HAHR-ee-uh
Akidau	AK-ih-dow *(flagged for the ear)*
Kleppmann	KLEP-mahn
Kingsbury	KINGZ-bair-ee
Bailis	BAY-liss
Skeen	SKEEN
Shostak	SHOH-stak
Pease	PEEZ
Liskov	LIS-kov
Oki	OH-kee
Ongaro	on-GAR-oh
Akkoyunlu	ah-koh-YOON-loo *(flagged for the ear)*
Ekanadham	ek-uh-NAHD-um *(flagged for the ear)*
Halpern	HAL-purn
Guerraoui	gair-ah-WEE *(flagged for the ear)*
Cachin	KAH-shin *(flagged for the ear)*
Rodrigues	rod-REE-gish — *Portuguese (flagged for the ear)*
Tanenbaum	TAN-en-bowm
Demers	duh-MEERZ *(flagged for the ear)*
Saltzer	SALT-zer
Karger	KAR-gur
Hellerstein	HEL-er-styne
Alvaro	AL-vuh-roh *(flagged for the ear)*
Recht	REKT

Grapheme	Spoken as (alias respelling)
Kulkarni	kool-KAR-nee
Hayashibara	hah-yah-shee-BAH-rah
Burrows	BURR-ohz
Corbett	KOR-bit
Thaler	THAY-lur *(flagged for the ear)*
Abadi	uh-BAH-dee
Ben-Or	ben-OR
Hadzilacos	hah-dzee-LAH-kohs *(flagged for the ear)*
Mortier	MOR-tee-ay *(flagged for the ear)*
DeCandia	deh-KAN-dee-uh *(flagged for the ear)*
Bar-Noy	bar-NOY
Garcia-Molina	gar-SEE-ah moh-LEE-nah
Salem	SAY-lem
Kaashoek	KAHS-hook *(flagged for the ear)*
Balakrishnan	bah-luh-KRISH-nun
Ravishankar	rah-vee-SHAHN-ker *(flagged for the ear)*
Akamai	AH-kuh-my — *company's own pronunciation*
Kreps	KREPS
Carbone	kar-BOH-neh *(flagged for the ear)*
Gobioff	GOH-bee-off *(flagged for the ear)*
Leung	lee-UNG *(flagged for the ear)*
Brooker	BRUK-er
Newcombe	NEW-kum
Bronson	BRON-sun

Grapheme	Spoken as (alias respelling)
Daly	DAY-lee
Li	LEE — *Mu Li, parameter server*
Paxos	PAK-soss
Zab	ZAB
etcd	et-see-DEE — *never 'etsed'*
NCCL	NIK-ul — *industry says 'nickel'*
Riak	REE-ak
Ceph	SEF
Jepsen	JEP-sen
ZeRO	ZEE-roh — *the optimizer, Episode Twelve*
CRDT	C R D T — *letterism*
PBFT	P B F T — *letterism*
HLC	H L C — *letterism*
TLA+	T L A plus — *grapheme includes plus sign*
TLA	T L A
Παρλιαμεντ	par-lee-ah-MENT — *Episode Four's Greek aside; reads as faux-Greek 'parliament'*
fsync	EFF-sink
fsynced	EFF-sinkt
Oofy	OO-fee
Gussie	GUSS-ee
Tuppy	TUP-ee
Barmy	BAR-mee

That closes the apparatus, and with it the volume: twelve chapters of argument and one of furniture, which is, I believe, the correct ratio.